snapy Technical Report

Table of contents

Preface **1**
Pins 1

1. Overview and code map **3**
1.1. Exchange before a single-block stage (`MeshBlock.forward`) 3
1.2. Construct a MeshBlock and its modules (`MeshBlockOptions`) 6
1.3. Locate dependencies outside snapy (`find_package`) 10
1.4. Dispatch tensor kernels by device (`CUDA`) 13
1.5. Driver loop and accepted steps (`integration:`) 18
1.6. Follow the hydro increment through seven sections (`dynamics:`) 22
1.7. Initialize primitives, ghosts and conserved states (`restart_file`) 27
1.8. Locate tensor rows, faces and ghost slabs (`NMASS / PartOptions`) 30
1.9. Advance several blocks in one process (`distribute.blocks_per_process`) 34
1.10. Build options from YAML and reject unknown keys (`from_yaml`) 38
1.11. Expose the C++ API to Python (`import snapy`) 41
1.12. Transport passive scalars with dry mass (`scalar:`) 45
1.13. Accumulate and combine one stage (`user_stage_forcings`) 48

2. Governing equations and thermodynamics **57**
2.1. External tabulated EOS interface (`aneos`) 58
2.2. EOS and solver consistency (`none`) 62

2.3. Ideal gas conversion (`ideal-gas`) 66
2.4. Constant-heat-capacity moist mixture (`ideal-moist`) . . . 70
2.5. EOS interface and dispatch (`type`) 75
2.6. Conservation laws and stage ordering (`none`) 79
2.7. Temperature-dependent moist thermodynamics (`moist-mixture`) 83
2.8. Fixed-volume fixed-energy phase equilibrium (`uv-solver`) . 89
2.9. Shallow-water conversion (`shallow-water`) 94

3. Grids and geometry **99**
3.1. One global grid, sliced per block (YAML `geometry`) 101
3.2. Cartesian metric (`cartesian`) 109
3.3. Spherical-polar metric and sources (`spherical-polar`) . . . 113
3.4. Gnomonic equiangular cubed sphere (`gnomonic-equiangle`) 121
3.5. Radial face moments and the face-centroid shift 131
3.6. Layouts and rank maps (`distribute.layout`) 137
3.7. Physical and internal faces (`external`) 143
3.8. Generic ghost exchange (`SyncOptions`) 148
3.9. Cross-panel ghost interpolation (`interpolate`) 153
3.10. Cubed-sphere face-state seam sync 161
3.11. Donor factor across seams: raw copy (`limiter`) 165
3.12. x_1 seams between blocks (`layout: cubed`, `nb1` > 1) 171
3.13. Sync types and the velocity frame at seams (`type`) 178
3.14. x_1-wall corner refresh inside tangential ghost slabs 184
3.15. Registered stub coordinate type (`cylindrical`) 188

4. Spatial discretization **191**
4.1. 4A. Reconstruction, Riemann solvers, divergence and sources 195
4.2. The finite-volume update and the flux divergence 195
4.3. The reconstruction framework (`dynamics/reconstruct`) . . 202
4.4. Donor cell (`type: dc`) 208
4.5. PLM, the van Leer harmonic-mean slope (`type: plm`) . . . 210
4.6. The centred linear polynomials (`type: cp3`, `cp5`) 213
4.7. WENO3 and WENO5 (`type: weno3`, `weno5`) 218

4.8. PPM (`type: ppm`) . . . 222
4.9. The Riemann solver framework and the face-local frame . . 225
4.10. LMARS (`type: lmars`) . . . 229
4.11. HLLC (`type: hllc`) . . . 233
4.12. Roe (`type: roe`) . . . 236
4.13. Shallow-water Roe and plume-roe (`type: shallow-roe`, `plume-roe`) . . . 240
4.14. Single-valued x_1 seam fluxes . . . 243
4.15. Geometric sources on the spherical-polar grid . . . 247
4.16. Geometric sources on the gnomonic cubed sphere . . . 251
4.17. 4B. Face averages against cell averages: the $O(\Delta x_1^2)$ corrections . . . 256
4.18. The x_2 and x_3 face-flux covariance and centroid terms (`SNAP_FLUX_COVARIANCE`) . . . 256
4.19. The x_1 mass-flux covariance (`SNAP_X1_MASS_COVARIANCE`) . 262

5. Hydrostatic and well-balanced treatment 269
5.1. Well-balanced x_1 reconstruction: perturbations about a hydrostatic reference (gravity on) . . . 271
5.2. Hydrostatic reference kernel: face-pressure scan, cell pressure and density reference (with the WB path) . . . 281
5.3. Wall clamp and the wall continuation of the default reference (`dynamics/wb-wall-clamp`) . . . 291
5.4. Reference continuity across x_1 seams: anchor relay and ghost-row exchange (x_1 block split) . . . 300
5.5. Hydrostatic mode: gravity replaced by the cell's own pressure difference (`forcing/const-gravity/non-hydrostatic`) 307
5.6. `balance_column`: projection of a ghost-free column onto the scheme's discrete balance (C++ and Python API) . . . 313
5.7. Fourth-order, cell/face-consistent density reference (`SNAP_WB_REF4`) . . . 323
5.8. r^2-exact x_1 maps on spherical-polar grids (`SNAP_X1_CENTROID_EXACT`) . . . 334

5.9. The $1/R$ remainder on spherical-polar grids: what is left after the corrections (no switch) 344
5.10. Legacy x_1 wall routines: isentropic wall ghosts and the wall-face state copy (no switch) 349

6. Gravity and energy **353**
6.1. The corrected-PE face gravity work (`SNAP_GRAVITY_WORK_RADIAL_EXACT`) 356

7. Positivity, floors and limiters **391**
7.1. The tracer flux positivity limiter (`equation-of-state/limiter`) 395
7.2. The carry of energy and momentum with withheld mass . . 402
7.3. The passive-scalar limiter and the complement bound . . . 406
7.4. The equation-of-state limiter: two passes, floors and marks 409
7.5. The condensate borrow 414
7.6. The vapour column repair 417
7.7. Reconstruction-stage floors 420
7.8. The well-balanced face positivity fallback 423
7.9. Dry-channel positivity inside the implicit solve 426
7.10. Round-off thresholds for positivity events 429
7.11. The floor detector and the redo decision 432
7.12. Dry-carry of passive tracers, and the zero-base guard 434

8. Diffusion, viscosity and forcing **439**
8.1. The forcing framework: parsing, registration and the stage tendency (`forcing:`) 441
8.2. User stage forcings: TorchScript modules called every stage (`set_user_stage_forcings`) 447
8.3. Coriolis force in the coordinate frame or from a Cartesian rotation vector (`forcing/coriolis`) 453
8.4. Isotropic viscosity and heat conduction (`forcing/diffusion`) 460
8.5. Diffusion face coefficient at a wall: one-sided extrapolation instead of the ghost (automatic) 471

8.6. x1 profiles of the diffusion coefficients and the mean-of-products face coefficient (`nu_scale_x1`, `kappa_scale_x1`) . 479
8.7. Body heating, top cooling and bottom heating (`body-heat`, `top-cool`, `bot-heat`) 489
8.8. Bottom relaxation of temperature, velocity and composition (`relax-bot-temp`, `relax-bot-velo`, `relax-bot-comp`) 495
8.9. Sponge layers: Rayleigh drag near the top and bottom boundaries (`top-sponge-lyr`, `bot-sponge-lyr`) 506
8.10. Plume forcing: accepted by the parser, never installed (`forcing/plume-forcing`) 511
8.11. Turbulence closure: not built 512

9. Moist physics coupling **513**
9.1. Saturation adjustment in the step (reactions in the thermo block) 516
9.2. Step rejection for moist causes (`max_redo`) 523
9.3. Condensate repair by parent-vapour borrow (`limiter`) . . . 528
9.4. Column vapour and cloud repair (`limiter`) 534
9.5. Precipitation and evaporation kinetics in the example drivers (driver-level coupling) 541
9.6. Passive tracers per dry air, with an optional upper bound (`scalar`) 547
9.7. Sedimentation of condensates (`sedimentation`) 552

10. Boundary conditions and immersed solids **561**
10.1. Reserve a face or fill a solid mask (`custom; solid`) 563
10.2. Supply ghosts through layout exchange (`SyncOptions`) . . . 568
10.3. Repeat the nearest owned cell (`extrapolation`) 572
10.4. Select outgoing acoustic and advected perturbations (`outflow`) 575
10.5. Wrap values through a callback or the layout (`periodic`) . 583
10.6. Mirror a reflecting wall (`reflecting`) 587
10.7. Register and classify block faces (`boundary-condition/external/*`) 591

10.8. Rectify a mask and close fluid faces against solids (`boundary-condition/internal/*; vars["solid"]`) . . . 595
10.9. Order physical fills, exchange and corner refresh (`apply_boundaries; exchange_ghost_zones`) 602
10.10 Keep wall ghosts consistent with the hydrostatic reference (`dynamics/wb-wall-clamp`) 607

11. Build-time and run-time switches and configurations 613
11.1. Build options and test registration (`cmake -DNAME=value`) . 617
11.2. Spherical cell-to-plain-mean conversion (`SNAP_X1_CENTROID_EXACT`) 622
11.3. Environment parsing and process lifetime (`SNAP_*`) 626
11.4. Lateral face covariance and centroid correction (`SNAP_FLUX_COVARIANCE`) 629
11.5. Layout and communication environment (`BACKEND / rank variables`) 634
11.6. Generated macros and compile definitions (`configure.h`) . 638
11.7. Radial velocity mean correction (`SNAP_X1_MASS_COVARIANCE`) 642
11.8. Switch-combination coverage matrix (`combined configuration`) 646
11.9. Corrected potential-energy face work (`SNAP_GRAVITY_WORK_RADIAL_EXACT`) 654
11.10 Test harness environment (`test environment`) 659
11.11 Cell/face-consistent reference (`SNAP_WB_REF4`) 663
11.12 YAML scheme selection and setup guards (`YAML keys`) . . . 667

12. Verification catalogue 673
12.1. Overview and code-map verification 673
12.2. Thermodynamics evidence 678
12.3. Grids and geometry: tests and checks 680
12.4. Hydrostatic and well-balanced treatment: test evidence . . 685
12.5. Moist physics coupling evidence 692
12.6. Boundary-condition verification (`Chapter 11`) 697
12.7. Configuration and switch evidence 704

Appendices **711**

A. Test index **711**

B. Components outside snapy **729**
B.1. External component interfaces 729

Preface

This report describes snapy as the code implements it: for each scheme, what it does, where it comes from, how it is discretized, where it lives in the code, how it is tested and where it stops working. Every statement is derived on the page, cited to the code at a pinned commit, or cited to a test with its tolerance.

Pins

Code is cited at the snapy main pin, `chengcli/snapy e894700ff7aee30b52882e5202b16461413780b0`. The pins and their meaning are defined in STYLE.md section 3.1 and reproduced here.

1. Overview and code map

1.1. Exchange before a single-block stage (`MeshBlock.forward`)

1.1.1. Summary

> **At a glance**
>
> `MeshBlock::forward` exchanges conserved ghosts before `advance_local` reconstructs them. Always exchanges first; no ordering switch. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

`MeshBlock::forward` exchanges conserved ghosts before `advance_local` reconstructs them. A caller that edits conserved interiors between stages therefore presents those changes to the next exchange.

1.1.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.1.3. Numerical method

The exchange fills hydro conserved ghosts and scalar density ghosts. It then refreshes eligible physical x1 boundaries inside exchanged tangential slabs and recomputes scalar ratios. Outflow is excluded from that corner refresh. This wrapper changes data freshness, not the stencil order. Physical boundary conditions and cubed-sphere interpolation retain their own errors.

Figure 1.1 shows the data or call sequence.

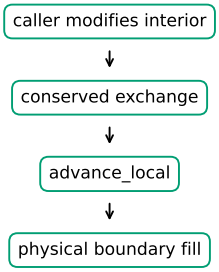


Figure 1.1.: Exchange before a single-block stage. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.1.4. Code

Table 1.1.: Implementation anchors for exchange before a single-block stage.

what	code link	symbol	switch
forward	`meshblock.cpp:545-552`	`forward`	API / see Summary
exchange	`meshblock.cpp:913-929`	`exchange_ghost_zones`	API / see Summary
corners	`meshblock.cpp:951-978`	`refresh`	API / see Summary
meshforward	`mesh.cpp:331-365`	`forward`	API / see Summary

1. Read `forward` for the forward part of this path (`meshblock.cpp:545-552`).
2. Read `exchange_ghost_zones` for the exchange part of this path (`meshblock.cpp:913-929`).
3. Read `refresh` for the corners part of this path (`meshblock.cpp:951-978`).
4. Read `forward` for the meshforward part of this path (`mesh.cpp:331-365`).

1.1.5. Tests

Table 1.2.: Source assertions for block-forward.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
No dedicated test	Missing evidence	Not applicable	Not applicable

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.1.6. Limits

The source comment attributes a removed cubed-sphere seam bias to this placement; this chapter does not promote that comment to measured evidence. A regression isolating split sources between consecutive calls is still missing.

1.2. Construct a MeshBlock and its modules (`MeshBlockOptions`)

1.2.1. Summary

> **At a glance**
>
> Construction resolves the global grid and validates the physical boundary list before `reset` builds layout, outputs, integrator, coordinates, hydro, scalar, solids and stage buffers. Options tree supplied by caller; no construction switch. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

Construction resolves the global grid and validates the physical boundary list before `reset` builds layout, outputs, integrator, coordinates, hydro, scalar, solids and stage buffers. The scalar reconstruction inherits the horizontal hydro reconstruction when omitted in YAML.

1.2.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.2.3. Numerical method

The layout exists before boundary classification. Internal slab or cubed faces lose their physical boundary function; cubed-sphere handling follows its own layout. Output slices must lie in the global half-open bounds and cannot also request sums on the same axis. Stage registers use float64 with the spatial axes in reverse coordinate order. Construction does not approximate a differential operator and has no convergence order.

Figure 1.2 shows the data or call sequence.

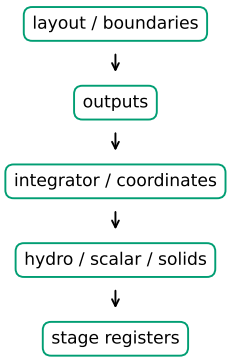


Figure 1.2.: Construct a MeshBlock and its modules. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.2.4. Code

Table 1.3.: Implementation anchors for construct a meshblock and its modules.

what	code link	symbol	switch
construct	`meshblock.cpp:66-88`	`MeshBlockImpl`	API / see Summary
reset	`meshblock.cpp:111-166`	`reset`	API / see Summary
modules	`meshblock.cpp:181-275`	`pintg`	API / see Summary
yaml	`meshblock_options.cpp:13-88`	`from_yaml`	API / see Summary

1. Read `MeshBlockImpl` for the construct part of this path (`meshblock.cpp:66-88`).
2. Read `reset` for the reset part of this path (`meshblock.cpp:111-166`).
3. Read `pintg` for the modules part of this path (`meshblock.cpp:181-275`).
4. Read `from_yaml` for the yaml part of this path (`meshblock_options.cpp:13-88`).

1.2.5. Tests

Table 1.4.: Source assertions for construction.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_yaml_keys.release`	Unknown key path appears in error	exact substring	Configuration contract

The assertion anchor `expect_refused` is in `test_yaml_keys.cpp:22-64`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.2.6. Limits

Layout block count must match partitioning, with the additional panel factor on the cubed sphere. Unsupported output types, unavailable PnetCDF support and an empty hydro state are rejected at setup. These checks are in the cited `reset` ranges; no new setup execution is claimed.

1.3. Locate dependencies outside snapy (`find_package`)

1.3.1. Summary

> **At a glance**
>
> Kintera supplies thermodynamics and restart tensor serialization. Kintera minimum 2.5.13; Harp and Disort required without a version here. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

Kintera supplies thermodynamics and restart tensor serialization. Harp supplies the integrator constructed by the MeshBlock. Python imports pyharp and pydisort as well as kintera; those imports are not a radiation operator inside the hydro pipeline.

1.3.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.3.3. Numerical method

The dependency boundary is visible at creation, stage adjustment and restart IO call sites. The report derives the propagation of supplied stage weights, but their selection and the stop predicate live in the external integrator. Build metadata pins torch 2.10.0 for the build and declares Linux commux and pinc requirements. Runtime version requirements must

be read separately. Appendix E owns external-component equations and full version analysis.

Figure 1.3 shows the data or call sequence.

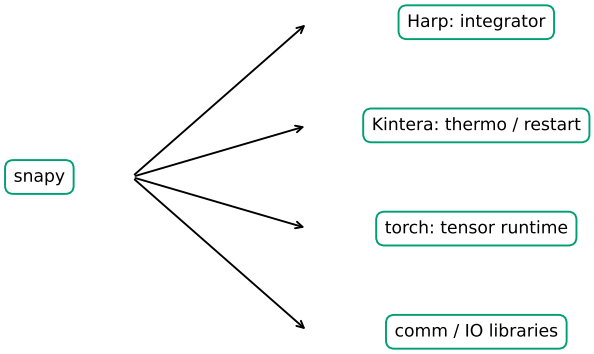


Figure 1.3.: Dependencies used by snapy. Labelled green boxes name independent services and black arrows connect snapy to each service; no arrow implies that one external service calls another.

1.3.4. Code

Table 1.5.: Implementation anchors for locate dependencies outside snapy.

what	code link	symbol	switch
deps	`CMakeLists.txt: 94-107`	`Kintera`	API / see Summary
pyproject	`pyproject.toml: 1-31`	`dependencies`	API / see Summary
pyinit	`__init__.py: 1-16`	`torch`	API / see Summary

what	code link	symbol	switch
modules	`meshblock.cpp:181-275`	`pintg`	API / see Summary
last	`meshblock.cpp:785-842`	`pthermo`	API / see Summary
save	`restart.cpp:84-90`	`save_tensors`	API / see Summary
load	`read_restart_file.cpp:223-230`	`load_tensors`	API / see Summary

1. Read `Kintera` for the deps part of this path (`CMakeLists.txt:94-107`).
2. Read `dependencies` for the pyproject part of this path (`pyproject.toml:1-31`).
3. Read `torch` for the pyinit part of this path (`__init__.py:1-16`).
4. Read `pintg` for the modules part of this path (`meshblock.cpp:181-275`).
5. Read `pthermo` for the last part of this path (`meshblock.cpp:785-842`).
6. Read `save_tensors` for the save part of this path (`restart.cpp:84-90`).
7. Read `load_tensors` for the load part of this path (`read_restart_file.cpp:223-230`).

1.3.5. Tests

Table 1.6.: Source assertions for dependencies.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
No dedicated test	Missing evidence	Not applicable	Not applicable

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.3.6. Limits

No dependency integration run is supplied here. Harp stage coefficients cannot be justified by a snapy line alone. The kintera minimum is tied in the CMake comments to the saturation-failure counter used by redo. Radiation functionality in dependencies is outside this chapter and outside the seven-section hydro call map.

1.4. Dispatch tensor kernels by device (`CUDA`)

1.4.1. Summary

> **i** At a glance
>
> Dispatch stubs connect an operation to CPU, CUDA and, where registered, MPS implementations. CMake default OFF; layout DEVICE default cpu. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data

ownership.

Dispatch stubs connect an operation to CPU, CUDA and, where registered, MPS implementations. The hydrostatic-reference stub illustrates all three registrations. Tensor operations outside custom kernels follow the tensor device.

1.4.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.4.3. Numerical method

The CPU reference kernel dispatches floating-point type and parallelizes over columns. Pointwise CUDA work uses TensorIterator offsets in `gpu_kernel`. Line work uses `stencil_kernel`; reconstruction selects a tiled kernel when the line exceeds the single-block threshold. The tile-width routine checks index range, stencil shape and shared-memory capacity. Dispatch itself adds no truncation order; device agreement requires tests for each operator.

Figure 1.4 shows the data or call sequence.

1.4.4. Code

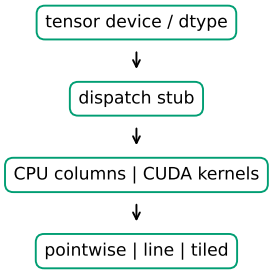


Figure 1.4.: Dispatch tensor kernels by device. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

Table 1.7.: Implementation anchors for dispatch tensor kernels by device.

what	code link	symbol	switch
stub	`hydro_dispatch.hpp:9-16`	`call_hydro_ref_x1`	API / see Summary
cpu	`hydro_dispatch.cpp:14-35`	`hydro_ref_x1_cpu`	API / see Summary
cpureg	`hydro_dispatch.cpp:204-208`	`call_hydro_ref_x1`	API / see Summary
cudareg	`hydro_dispatch.cu:61-63`	`call_hydro_ref_x1`	API / see Summary
gpu	`loops.cuh:52-68`	`gpu_kernel`	API / see Summary
stencil	`loops.cuh:70-108`	`stencil_kernel`	API / see Summary

what	code link	symbol	switch
tiled	recon_dispatch.cu:18-50	recon_tile_width	API / see Summary
build	CMakeLists.txt:7-18	CUDA	CUDA
cubuild	CMakeLists.txt:122-155	cu_src_files	API / see Summary
layoutopts	layout.cpp:221-239	from_yaml	API / see Summary

1. Read call_hydro_ref_x1 for the stub part of this path (hydro_dispatch.hpp:9-16).
2. Read hydro_ref_x1_cpu for the cpu part of this path (hydro_dispatch.cpp:14-35).
3. Read call_hydro_ref_x1 for the cpureg part of this path (hydro_dispatch.cpp:204-208).
4. Read call_hydro_ref_x1 for the cudareg part of this path (hydro_dispatch.cu:61-63).
5. Read gpu_kernel for the gpu part of this path (loops.cuh:52-68).
6. Read stencil_kernel for the stencil part of this path (loops.cuh:70-108).
7. Read recon_tile_width for the tiled part of this path (recon_dispatch.cu:18-50).
8. Read CUDA for the build part of this path (CMakeLists.txt:7-18).
9. Read cu_src_files for the cubuild part of this path (CMakeLists.txt:122-155).
10. Read from_yaml for the layoutopts part of this path (layout.cpp:221-239).

1.4.5. Tests

Table 1.8.: Source assertions for dispatch.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_weno5_cuda_line.release`	CUDA and CPU reconstruction agree	absolute < 1e-12	Test constant; rationale not documented
`test_coordinate.release`	Device/dtype parameter coverage	per assertion	Parameter list is not a tolerance

The assertion anchor `expect_matches_cpu` is in `test_weno5_cuda_line.cpp:83-121`.

The assertion anchor `DeviceTest` is in `device_testing.hpp:33-50`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.4.6. Limits

The shared DeviceTest parameter list has CPU and CUDA float32/float64 cases, but its MPS entry is commented out. Registrations alone are not test evidence. No CUDA or MPS execution was performed for this chapter. The reconstruction tiling test does not establish safe long-line execution for every other stencil user.

1.5. Driver loop and accepted steps (`integration:`)

1.5.1. Summary

> **At a glance**
>
> The driver chooses the time step, executes every stage, checks rejection, and writes accepted states. Caller supplied; integration defaults belong to pyharp. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

The driver chooses the time step, executes every stage, checks rejection, and writes accepted states. There is no main time loop inside a MeshBlock. The C++ example applies kinetics after all dynamics stages and before the rejection decision.

1.5.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.5.3. Numerical method

The driver keeps the clock unchanged when a step is rejected. A positive return from `check_redo` repeats the cycle; a negative return ends it. `max_time_step` first reduces the local hydro limit with MIN, then applies the Courant factor and the redo halving. Outputs are scheduled after the accepted clock increment. The `straka` driver also writes immediately after initialization, including restart. No independent spatial or temporal order belongs to this control flow; it inherits the selected operators.

Figure 1.5 shows the data or call sequence.

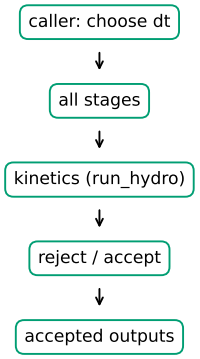


Figure 1.5.: Driver loop and accepted steps. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.5.4. Code

Table 1.9.: Implementation anchors for driver loop and accepted steps.

what	code link	symbol	switch
driver	`run_hydro.cpp: 157-205`	`check_redo`	API / see Summary
straka	`straka.cpp: 136-165`	`forward`	API / see Summary
dt	`meshblock.cpp: 510-528`	`max_time_step`	API / see Summary
redo	`meshblock.cpp: 1278-1304`	`check_redo`	API / see Summary

what	code link	symbol	switch
output	`meshblock.cpp:985-1002`	`make_outputs`	API / see Summary
final	`meshblock.cpp:1121-1144`	`finalize`	API / see Summary

1. Read `check_redo` for the driver part of this path (`run_hydro.cpp:157-205`).
2. Read `forward` for the straka part of this path (`straka.cpp:136-165`).
3. Read `max_time_step` for the dt part of this path (`meshblock.cpp:510-528`).
4. Read `check_redo` for the redo part of this path (`meshblock.cpp:1278-1304`).
5. Read `make_outputs` for the output part of this path (`meshblock.cpp:985-1002`).
6. Read `finalize` for the final part of this path (`meshblock.cpp:1121-1144`).

1.5.5. Tests

Table 1.10.: Source assertions for driver.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_restart_cycle_limit`	Resume cycle and logged values	absolute 1e-12; time 1e-9	Runner constants; rationale not documented

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_check_redo_parallel.release`	Collective redo and rollback	exact equality	Saved tensors are copied back

The assertion anchor `first_restart` is in `run_restart_cycle_limit.py:168-198`.

The assertion anchor `check_redo` is in `test_check_redo_parallel.cpp:43-68`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.5.6. Limits

The restart writer saves output counters before `make_outputs` increments them. Together with the unconditional initial output in `straka`, this is a code-level reason to investigate repeated restart outputs; no overwrite experiment is reported here. Driver-split kinetics is outside the MeshBlock stage.

1.6. Follow the hydro increment through seven sections (`dynamics:`)

1.6.1. Summary

> **At a glance**
>
> Hydro converts conserved state to primitives, constructs directional fluxes, limits species transport, takes the geometric divergence, adds forcing and gravity work, and applies the optional vertical implicit correction. Physics choices are owned by their chapters and Section 11.8. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

Hydro converts conserved state to primitives, constructs directional fluxes, limits species transport, takes the geometric divergence, adds forcing and gravity work, and applies the optional vertical implicit correction. Its result is an increment, not an updated conserved state.

1.6.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.6.3. Numerical method

The seven sections are primitive conversion; x1 reconstruction, Riemann flux, sedimentation and process-seam averaging; horizontal left/right states and panel exchange; horizontal fluxes and positivity; divergence with geometric sources; forcing and gravity work; and implicit correction.

Communication occurs inside this pipeline as well as in the driver wrapper: x1 process-seam fluxes, panel states and donor factors must be synchronized at their own use sites. Disabled directions and unresolved dimensions are gated separately. Orders and wall closures are derived by the owning physics chapters.

Figure 1.6 shows the data or call sequence.

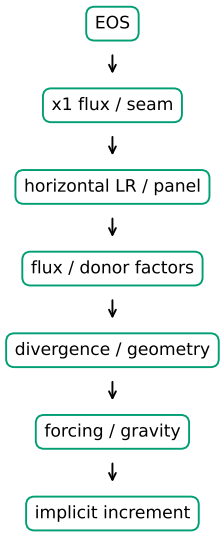


Figure 1.6.: Follow the hydro increment through seven sections. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.6.4. Code

Table 1.11.: Implementation anchors for follow the hydro increment through seven sections.

what	code link	symbol	switch
hydro	`hydro_forward.cpp:198-219`	`forward`	API / see Summary
reference	`hydro_forward.cpp:287-290`	`_hydro_ref_x1`	per physics options
reconstruction	`hydro_forward.cpp:345-358`	`precon1`	per physics options
riemann	`hydro_forward.cpp:399-406`	`priemann`	per physics options
solve	`hydro_forward.cpp:958-969`	`_apply_implicit_correction`	per physics options
sed	`hydro_forward.cpp:417-445`	`psed`	API / see Summary
lr	`hydro_forward.cpp:542-586`	`sync_opts`	API / see Summary
flux	`hydro_forward.cpp:589-627`	`flux_covariance`	`SNAP_FLUX_COVARIANCE`
theta	`hydro_forward.cpp:648-698`	`flux_positivity_theta`	EOS limiter

what	code link	symbol	switch
div	`hydro_forward.cpp:724-765`	`_div`	API / see Summary
implicit	`hydro_forward.cpp:929-944`	`dt_corr`	`implicit-scheme`
weights	`hydro_forward.cpp:985-995`	`cw`	API / see Summary

1. Read `forward` for the hydro part of this path (`hydro_forward.cpp:198-219`).
2. Read `psed` for the sed part of this path (`hydro_forward.cpp:417-445`).
3. Read `sync_opts` for the lr part of this path (`hydro_forward.cpp:542-586`).
4. Read `flux_covariance` for the flux part of this path (`hydro_forward.cpp:589-627`).
5. Read `flux_positivity_theta` for the theta part of this path (`hydro_forward.cpp:648-698`).
6. Read `_div` for the div part of this path (`hydro_forward.cpp:724-765`).
7. Read `dt_corr` for the implicit part of this path (`hydro_forward.cpp:929-944`).
8. Read `cw` for the weights part of this path (`hydro_forward.cpp:985-995`).

1.6.5. Tests

Table 1.12.: Source assertions for hydro.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
No dedicated test	Missing evidence	Not applicable	Not applicable

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.6.6. Limits

The implicit time step is stage-weighted only for an integrator with three stages. A direct hydro caller that has not published a valid stage receives a one-time warning and uses the full time step. Same-process x1 seam flux averaging is explicitly excluded by the cited code. Neither case is newly measured here.

1.7. Initialize primitives, ghosts and conserved states (`restart_file`)

1.7.1. Summary

> **At a glance**
>
> Fresh initialization validates primitive shapes, captures any radiating-boundary reference, fills physical ghosts, exchanges primitive ghosts and then converts to conserved state. Absent: initialize from primitives; present: load restart. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

Fresh initialization validates primitive shapes, captures any radiating-boundary reference, fills physical ghosts, exchanges primitive ghosts and then converts to conserved state. Passive scalar density is seeded using conserved dry-air density.

1.7.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.7.3. Numerical method

Primitive exchange precedes the equation-of-state conversion, so the conserved ghosts reflect the exchanged primitives. Solid data are captured and solid velocities reset before the final conserved boundary fill. Mesh initializes fresh local blocks through concurrent jobs; its restart path instead initializes blocks in a caller-thread loop. Initialization introduces no

independent accuracy order; interpolation and boundary accuracy belong to the geometry and boundary chapters.

Figure 1.7 shows the data or call sequence.

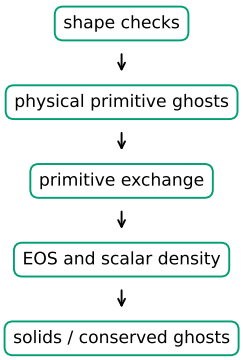


Figure 1.7.: Initialize primitives, ghosts and conserved states. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.7.4. Code

Table 1.13.: Implementation anchors for initialize primitives, ghosts and conserved states.

what	code link	symbol	switch
init	`meshblock.cpp: 360-398`	`initialize`	API / see Summary
initlocal	`meshblock.cpp: 401-451`	`initialize_local`	API / see Summary

what	code link	symbol	switch
initfinal	`meshblock.cpp:454-500`	`finalize_initialization`	API / see Summary
meshinit	`mesh.cpp:280-303`	`initialize`	API / see Summary
restart	`restart.cpp:26-59`	`out_vars`	API / see Summary

1. Read `initialize` for the init part of this path (`meshblock.cpp:360-398`).
2. Read `initialize_local` for the initlocal part of this path (`meshblock.cpp:401-451`).
3. Read `finalize_initialization` for the initfinal part of this path (`meshblock.cpp:454-500`).
4. Read `initialize` for the meshinit part of this path (`mesh.cpp:280-303`).
5. Read `out_vars` for the restart part of this path (`restart.cpp:26-59`).

1.7.5. Tests

Table 1.14.: Source assertions for initialization.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_scalar.release`	Initialization, shape, finite increment	allclose defaults; exact shape	API contract; numeric rationale absent
`test_mesh_multi_block.release`	Local and remote neighbor ghosts	allclose defaults	Library defaults, not exact equality

The assertion anchor `scalar_s` is in `test_scalar.cpp:91-104`.

The assertion anchor `side_is_uniform` is in `test_mesh_multi_block.cpp:19-95`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.7.6. Limits

The restart branch returns before primitive exchange and relies on saved tensor ghosts. The writer saves full defined tensors, except the reconstructible scalar ratio. No dedicated fresh-versus-restart ghost equivalence measurement is supplied.

1.8. Locate tensor rows, faces and ghost slabs (`NMASS / PartOptions`)

1.8.1. Summary

> **At a glance**
>
> Hydro buffers have variable, x3, x2, x1 axes. NMASS default 0; exterior slabs default true. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

Hydro buffers have variable, x3, x2, x1 axes. A resolved direction includes ghosts on both sides; a one-cell direction has no ghost storage. Primitive and conserved row meanings differ even though their row indices agree.

1.8.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.8.3. Numerical method

Cell values occupy the final three axes; x1 is contiguous in the nominal layout. The first species row follows energy or pressure. At a face the left and right reconstruction slots share the face index; `_apply_inplace` offsets their output slices by one. `part` takes offsets in x3, x2, x1 order, returning an interior or boundary slab with optional extensions and depth. In the Code table, `il` and `iu` are the first and last interior math indices, not new physical variables. Layout has no convergence order.

Figure 1.8 shows the data or call sequence.

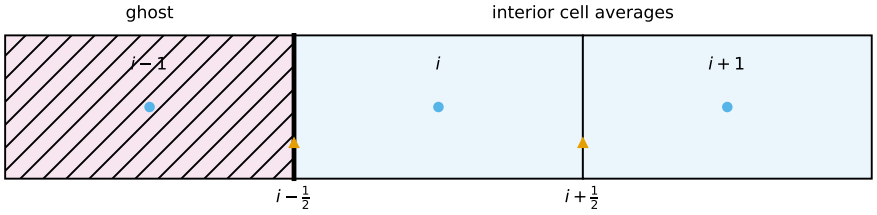


Figure 1.8.: A lower-wall storage stencil. Blue circles are cell averages, orange triangles are face states, the hatched cell is a ghost and the thick black line is the physical wall. The shared face index holds both reconstructed states.

1.8.4. Code

Table 1.15.: Implementation anchors for locate tensor rows, faces and ghost slabs.

what	code link	symbol	switch
indices	`snap.h:34-64`	`ICY`	API / see Summary
sizes	`coordinate.hpp: 59-61`	`nc1`	API / see Summary
ends	`coordinate.hpp: 169-183`	`il`	API / see Summary
partopts	`meshblock.hpp: 105-114`	`PartOptions`	`PartOptions`
part	`meshblock.cpp: 284-325`	`part`	API / see Summary
reconstruct	`reconstruct.cpp: 50-65`	`_apply_ inplace`	API / see Summary
modules	`meshblock.cpp: 181-275`	`pintg`	API / see Summary

1. Read `ICY` for the indices part of this path (`snap.h:34-64`).
2. Read `nc1` for the sizes part of this path (`coordinate.hpp:59-61`).
3. Read `il` for the ends part of this path (`coordinate.hpp:169-183`).
4. Read `PartOptions` for the partopts part of this path (`meshblock.hpp: 105-114`).
5. Read `part` for the part part of this path (`meshblock.cpp:284-325`).
6. Read `_apply_inplace` for the reconstruct part of this path (`reconstruct.cpp:50-65`).
7. Read `pintg` for the modules part of this path (`meshblock.cpp:181-275`).

1.8.5. Tests

Table 1.16.: Source assertions for layout.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_scalar.release`	Initialization, shape, finite increment	allclose defaults; exact shape	API contract; numeric rationale absent
`test_mesh_exchange_python`	Selected neighbor slab	exact equality	Integer rank field copied

The assertion anchor `scalar_s` is in `test_scalar.cpp:91-104`.

The assertion anchor `ghost` is in `test_mesh_exchange.py:37-50`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.8.6. Limits

NMASS greater than zero is rejected by the species-layout static assertion. Degenerate axes cannot supply an ordinary multi-cell stencil. The tests here exercise selected shapes and slabs rather than every depth/extension combination.

1.9. Advance several blocks in one process (`distribute.blocks_per_process`)

1.9.1. Summary

> **At a glance**
>
> Mesh clones and repartitions block options, then runs local blocks through a worker pool. YAML default 1; worker pool for more than one local block. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

Mesh clones and repartitions block options, then runs local blocks through a worker pool. On CUDA the pool assigns a stream to each worker and fences caller and worker work with events. Mesh advances before exchanging, including its one-block fast path.

1.9.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.9.3. Numerical method

For several local blocks the per-block gravity fixer is deferred. On the final stage Mesh sums local budgets, performs one process-group SUM, applies the common correction, refills physical boundaries and exchanges ghosts. Time-step selection takes a local minimum followed by one group

minimum. Redo combines local flags before one reduction. Stream assignment is an execution policy, not evidence of numerical equivalence between decompositions.

Figure 1.9 shows the data or call sequence.

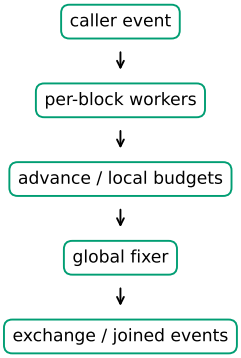


Figure 1.9.: Advance several blocks in one process. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.9.4. Code

Table 1.17.: Implementation anchors for advance several blocks in one process.

what	code link	symbol	switch
pool	`mesh.cpp:47-115`	`BlockWorkerPool`	API / see Summary
meshoptions	`mesh.cpp:210-258`	`blocks_per_process`	API / see Summary

what	code link	symbol	switch
meshdt	`mesh.cpp:306-328`	`max_time_step`	API / see Summary
meshforward	`mesh.cpp:331-365`	`forward`	API / see Summary
meshredo	`mesh.cpp:422-437`	`check_redo`	API / see Summary
tags	`layout.cpp:849-876`	`send_to_block`	API / see Summary
wait	`layout.cpp:879-897`	`take_x1_anchor`	API / see Summary

1. Read `BlockWorkerPool` for the pool part of this path (`mesh.cpp:47-115`).
2. Read `blocks_per_process` for the meshoptions part of this path (`mesh.cpp:210-258`).
3. Read `max_time_step` for the meshdt part of this path (`mesh.cpp:306-328`).
4. Read `forward` for the meshforward part of this path (`mesh.cpp:331-365`).
5. Read `check_redo` for the meshredo part of this path (`mesh.cpp:422-437`).
6. Read `send_to_block` for the tags part of this path (`layout.cpp:849-876`).
7. Read `take_x1_anchor` for the wait part of this path (`layout.cpp:879-897`).

1.9.5. Tests

Table 1.18.: Source assertions for mesh.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_mesh_multi_block.release`	Local and remote neighbor ghosts	allclose defaults	Library defaults, not exact equality
`test_mesh_exchange_python`	Selected neighbor slab	exact equality	Integer rank field copied

The assertion anchor `side_is_uniform` is in `test_mesh_multi_block.cpp:19-95`.

The assertion anchor `ghost` is in `test_mesh_exchange.py:37-50`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.9.6. Limits

A driver-split source after a Mesh call can make previously exchanged ghosts stale before the next advance. This is an inference from call order, not a measured drift. Remote block message tags reject more than 16 blocks per process. The local x1 anchor wait requires concurrent or top-down producers and fails after its five-minute timeout.

1.10. Build options from YAML and reject unknown keys (`from_yaml`)

1.10.1. Summary

> **At a glance**
>
> `ADD_ARG` supplies chainable setters and const or mutable getters. No global default: each reader owns its fallback. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

`ADD_ARG` supplies chainable setters and const or mutable getters. YAML readers build the module tree and selected readers reject keys outside their fixed list. The root reader delegates layout, hydro, scalar, integration, output, coordinates and boundaries.

1.10.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.10.3. Numerical method

Validate at the reader that owns the key. `check_keys` ignores non-map nodes and otherwise reports the unknown key together with valid keys and any delegated reader. The root options reader has no equivalent whole-document whitelist. This is configuration validation, with no numerical stencil, stage or accuracy order.

Figure 1.10 shows the data or call sequence.

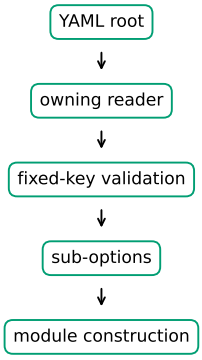


Figure 1.10.: Build options from YAML and reject unknown keys. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.10.4. Code

Table 1.19.: Implementation anchors for build options from yaml and reject unknown keys.

what	code link	symbol	switch
args	`add_arg.h:3-25`	`ADD_ARG`	API / see Summary
keys	`check_keys.cpp:12-30`	`check_keys`	API / see Summary
yaml	`meshblock_options.cpp:13-88`	`from_yaml`	API / see Summary
layoutopts	`layout.cpp:221-239`	`from_yaml`	API / see Summary

what	code link	symbol	switch
scalaropts	scalar_options.cpp:11-54	from_yaml	API / see Summary

1. Read ADD_ARG for the args part of this path (add_arg.h:3-25).
2. Read check_keys for the keys part of this path (check_keys.cpp:12-30).
3. Read from_yaml for the yaml part of this path (meshblock_options.cpp:13-88).
4. Read from_yaml for the layoutopts part of this path (layout.cpp:221-239).
5. Read from_yaml for the scalaropts part of this path (scalar_options.cpp:11-54).

1.10.5. Tests

Table 1.20.: Source assertions for options.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
test_yaml_keys.release	Unknown key path appears in error	exact substring	Configuration contract

The assertion anchor expect_refused is in test_yaml_keys.cpp:22-64.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.10.6. Limits

The accepted `distribute.backend` spelling does not set the backend: the layout reads `BACKEND` instead. Programmatic defaults and YAML defaults must be compared per reader; this chapter does not equate them. Numerical environment-switch coverage is in Section 11.8 and the environment appendix.

1.11. Expose the C++ API to Python (`import snapy`)

1.11.1. Summary

> **i** At a glance
>
> The Python initializer imports torch and the external packages, then the extension. Import sets float64, one intra-op thread and one inter-op thread. Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

The Python initializer imports torch and the external packages, then the extension. The extension exposes row constants, restart loading and component bindings, while the distribution submodule accepts an external process group.

1.11.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.11.3. Numerical method

`MeshBlock.forward` releases the Python GIL around the C++ stage call. The Python API therefore reaches the same stage and exchange ordering described above. Packaging declares build requirements separately from runtime requirements; the command entry points include combine and inspect tools. Import and binding introduce no independent spatial order or ghost treatment.

Figure 1.11 shows the data or call sequence.

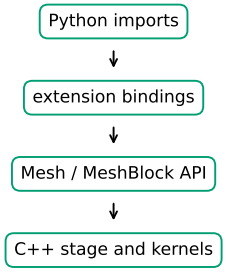


Figure 1.11.: Expose the C++ API to Python. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.11.4. Code

Table 1.21.: Implementation anchors for expose the c++ api to python.

what	code link	symbol	switch
pyinit	`__init__.py: 1-16`	`torch`	API / see Summary

what	code link	symbol	switch
pybind	snapy.cpp:32-60	load_restart	API / see Summary
pymesh	pymesh.cpp:179-186	forward	API / see Summary
pydist	pylayout.cpp:136-141	set_process_group	API / see Summary
pyproject	pyproject.toml:1-31	dependencies	API / see Summary
scripts	pyproject.toml:52-60	pd-combine	API / see Summary
testpath	CMakeLists.txt:350-362	SNAPY_TEST_PYTHONPATH	API / see Summary

1. Read `torch` for the pyinit part of this path (`__init__.py:1-16`).
2. Read `load_restart` for the pybind part of this path (`snapy.cpp:32-60`).
3. Read `forward` for the pymesh part of this path (`pymesh.cpp:179-186`).
4. Read `set_process_group` for the pydist part of this path (`pylayout.cpp:136-141`).
5. Read `dependencies` for the pyproject part of this path (`pyproject.toml:1-31`).
6. Read `pd-combine` for the scripts part of this path (`pyproject.toml:52-60`).
7. Read `SNAPY_TEST_PYTHONPATH` for the testpath part of this path (`CMakeLists.txt:350-362`).

1.11.5. Tests

Table 1.22.: Source assertions for python.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_python_import_path_python`	Import origin and PYTHONPATH ordering	exact path comparisons	Package-under-test contract
`test_jit_user_forcing_python`	Nested scripted module adds increment	assert_close defaults	Library defaults, not specified by test

The assertion anchor `where` is in `test_python_import_path.py:1-23`.

The assertion anchor `expected` is in `test_jit_user_forcing.py:44-71`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.11.6. Limits

Python ctests need `SNAPY_TEST_PYTHONPATH` pointing to the package built from the pin; otherwise CMake warns that they exercise an installed package instead. The import-path check skips with status 125 when no target path is supplied. Legacy convenience scripts are not taken as evidence for the current binding signature.

1.12. Transport passive scalars with dry mass (`scalar:`)

1.12.1. Summary

> **At a glance**
>
> Scalar transport reconstructs the ratio to conserved dry density and upwinds it with the hydro dry-mass flux. YAML nvar default 0; upper-bound default -1 (disabled). Order: code map; no independent measured convergence order. No defining equation: this section maps control flow or data ownership.

Scalar transport reconstructs the ratio to conserved dry density and upwinds it with the hydro dry-mass flux. It therefore follows hydro in each stage. Names supplied in YAML set the scalar count; missing reconstruction inherits the horizontal hydro choice.

1.12.2. Derivation

No separate derivation: this section describes an interface or control-flow path. Numerical operators are derived in their owning chapters.

1.12.3. Numerical method

Horizontal panel exchange uses directional left/right keys with interpolation disabled. The donor factor also uses a raw exchange, followed by physical boundary functions, before flux scaling. The scalar stage result is combined by the parent MeshBlock. Implicit dry transfer and dry-source carry are described in Section 1.13. Reconstruction, boundary and limiter

choices determine accuracy; no scalar convergence rate is measured in this code map.

Figure 1.12 shows the data or call sequence.

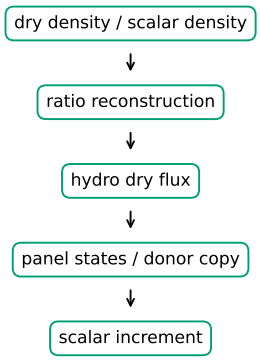


Figure 1.12.: Transport passive scalars with dry mass. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

1.12.4. Code

Table 1.23.: Implementation anchors for transport passive scalars with dry mass.

what	code link	symbol	switch
scalar	`scalar.cpp: 64-98`	`forward`	API / see Summary
scalarlimit	`scalar.cpp: 127-161`	`sync_theta`	API / see Summary

what	code link	symbol	switch
scalaropts	`scalar_options.cpp:11-54`	`from_yaml`	API / see Summary
yaml	`meshblock_options.cpp:13-88`	`from_yaml`	API / see Summary
transfer	`meshblock.cpp:650-678`	`mass_corr`	API / see Summary

1. Read `forward` for the scalar part of this path (`scalar.cpp:64-98`).
2. Read `sync_theta` for the scalarlimit part of this path (`scalar.cpp:127-161`).
3. Read `from_yaml` for the scalaropts part of this path (`scalar_options.cpp:11-54`).
4. Read `from_yaml` for the yaml part of this path (`meshblock_options.cpp:13-88`).
5. Read `mass_corr` for the transfer part of this path (`meshblock.cpp:650-678`).

1.12.5. Tests

Table 1.24.: Source assertions for scalar.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_scalar.release`	Initialization, shape, finite increment	allclose defaults; exact shape	API contract; numeric rationale absent

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_scalar.release`	Positive density, bounded ratio	1e-5 float32; 1e-12 float64	Test constants; rationale not documented

The assertion anchor `scalar_s` is in `test_scalar.cpp:91-104`.

The assertion anchor `tol` is in `test_scalar.cpp:159-163`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.12.6. Limits

The forward routine divides directly by conserved dry density. Its behavior for a zero dry-density field is not established by the positive-density scalar tests cited here. The upper-bound and positivity mechanisms are documented by the moist and positivity chapters; combinations remain in Section 11.8.

1.13. Accumulate and combine one stage (`user_stage_forcings`)

Local symbols: $J^{\mathrm{d}}_{i+1/2}$ is the already time-integrated implicit dry-air transfer through a face [kg], positive toward increasing x_1; $J^{\mathrm{tr}}_{n,i+1/2}$ is its upwind passive-tracer transfer [kg]. Here n labels a passive tracer and r_n is its dimensionless ratio per dry mass. Superscripts base and entry distinguish the source-free stage and source-entry state; carry denotes the selected

carry ratio and source a source increment. These transfer symbols are proposed in the Chapter 1 item in `ISSUES.md`.

1.13.1. Summary

> **At a glance**
>
> `advance_local` publishes the stage, saves the initial state on stage zero, assembles hydro and scalar increments, applies user increments, combines the stage and enforces the conserved limiter. No callbacks unless installed; integrator stages are external. Order: code map; no independent measured convergence order. Defining identities: Equation 1.2, Equation 1.4 and Equation 1.5.

`advance_local` publishes the stage, saves the initial state on stage zero, assembles hydro and scalar increments, applies user increments, combines the stage and enforces the conserved limiter. Last-stage thermodynamic adjustment and the gravity-work fixer precede the final physical boundary fill.

1.13.2. Derivation

The fields in this section are cell averages. The conserved vector $\mathbf{U}$ contains densities, momentum densities and energy density with the component units of the notation table; Δt is in seconds.

Write $\Delta\mathbf{U}$ for the supplied stage increment, with the units of the corresponding conserved component. The source comment states the combine as an in-place affine recurrence:

$$\mathbf{U} \leftarrow w_0^{(s)}\mathbf{U}^n + w_1^{(s)}\mathbf{U} + w_2^{(s)}\Delta\mathbf{U}. \tag{1.1}$$

Equation 1.1 is the recurrence stated beside `cw` in `hydro_forward.cpp:985-995`. The increment here includes whichever explicit and implicit operations were selected; it is not restricted to the notation table's explicit increment $\Delta\mathbf{U}^{(0)}$.

Consider a fixed supplied increment in stage (s), while holding later increments fixed. It enters the combination with coefficient $w_2^{(s)}$. The next combine multiplies this contribution by $w_1^{(s+1)}$; the saved initial state contributes no copy of this new increment. Repeating this substitution through each later stage gives the final coefficient

$$w_2^{(s)} \prod_{s'>s} w_1^{(s')}. \tag{1.2}$$

Equation 1.2 is the coefficient `cw` accumulated in `hydro_forward.cpp:985-995`. The product is over the remaining stages; an empty product is one. This identity is exact for the bookkeeping of supplied increments, not a linearization of their nonlinear state dependence. The combine call `pintg` is at `meshblock.cpp:754-783`; the actual integrator coefficients are external, so this chapter does not assert them from a snapy call site. Check `[C1]` expands a generic three-stage affine recurrence symbolically.

For implicit tracer transfer, start with the conservative passive-tracer law in a fixed cell:

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{V_i} \rho_n \,\mathrm{d}V = -\int_{\partial V_i} \rho_n \mathbf{v}\cdot \mathrm{d}\mathbf{A}.$$

Integrating over the implicit correction and orienting both x1 faces upward turns the surface integral into an incoming transfer minus an outgoing transfer. The code takes the already-clamped dry transfer from the implicit mass-correction buffer. Multiplying by the donor ratio gives

$$J^{\mathrm{tr}}_{n,i+1/2} = J^{\mathrm{d}}_{i+1/2} \begin{cases} r_{n,i}, & J^{\mathrm{d}}_{i+1/2} > 0, \\ r_{n,i+1}, & J^{\mathrm{d}}_{i+1/2} \le 0. \end{cases} \tag{1.3}$$

Equation 1.3 is the upwind `P` constructed in `meshblock.cpp:650-678`. At zero transfer the branch choice has no effect. Dividing the net transfer by cell volume V_i [m^3] yields the increment added to the scalar tendency:

$$\Delta\rho_{n,i} = \frac{J^{\mathrm{tr}}_{n,i-1/2} - J^{\mathrm{tr}}_{n,i+1/2}}{V_i}. \tag{1.4}$$

Equation 1.4 is the `P - P_above` update in `meshblock.cpp:650-678`. Multiplying by V_i and summing cancels every common internal face. The remaining sum is the lower-boundary transfer minus the upper-boundary transfer; closed ends therefore conserve tracer mass exactly in real arithmetic. The code's shifted buffers supply zero beyond stored ends; the physical column closure and seam consistency are additional conditions, not consequences of this local telescoping proof. Upwinding approximates the face ratio; no spatial convergence order of the coupled implicit solve is measured here. Check `[C2]` tests both signs on unequal cell volumes with closed ends and with specified boundary transfers.

Dry-source carry has a different purpose: removing dry mass must not increase a tracer ratio. The code computes a source-free RK result using the hydro increment minus the dry source and the saved scalar increment. If its dry density is nonzero, its ratio is $\rho^{\mathrm{base}}_n/\rho^{\mathrm{base}}_{\mathrm{d}}$. For a negative source it uses this ratio; for a nonnegative source it uses the entry ratio. For zero source-free dry density it falls back to the entry ratio rather than using the quotient. Thus the additional supplied scalar increment uses the selected ratio r^{carry}_n (dimensionless):

$$\Delta\rho^{\mathrm{source}}_n = r^{\mathrm{carry}}_n \Delta\rho^{\mathrm{source}}_{\mathrm{d}}. \tag{1.5}$$

Equation 1.5 is implemented by `carry_dry_source` in `meshblock.cpp: 633-645`. For removal the final dry and tracer densities are their source-free values plus $w_2^{(s)}$ times the respective source increments. Substituting the first branch factors the tracer density into the source-free ratio times the final dry density. Dividing recovers the original ratio whenever that final density is nonzero. This is exact algebra, with floating-point round-off in evaluation; it does not license removing more dry mass than is available. Check `[C3]` verifies the factorization symbolically and the addition and zero-base branches numerically. An independent user scalar source is retained because the callback path saves the scalar base before adding callbacks. These checks live in `src/snapy_report/ch01/stage_check.py`; their committed output is `src/snapy_report/ch01/stage_check.out`. They check formulas without importing snapy.

1.13.3. Numerical method

The hydro call returns a time-integrated increment. Scalar transport then uses the hydro dry-mass flux and the implicit dry-mass transfer. Native dry sources carry tracers before callbacks; user dry sources carry tracers after all callbacks, using the saved pre-callback scalar increment. The callback result accepts only `hydro_du` and `scalar_ds` with matching shapes. Solids are refilled after the RK combine. Saturation adjustment is restricted to interior cells, the last stage and a nonempty reaction list. This map asserts algebraic identities, not an observed convergence order at walls or seams.

Figure 1.13 shows the data or call sequence.

1.13.4. Code

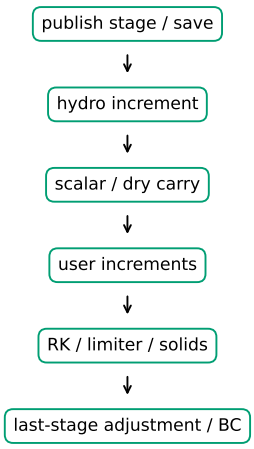


Figure 1.13.: Accumulate and combine one stage. Labelled green boxes name operations or owners; black arrows show the reading sequence, not a timing measurement.

Table 1.25.: Implementation anchors for accumulate and combine one stage.

what	code link	symbol	switch
stage	`meshblock.cpp: 597-626`	`advance_local`	API / see Summary
dry	`meshblock.cpp: 633-645`	`carry_dry_ source`	API / see Summary
transfer	`meshblock.cpp: 650-678`	`mass_corr`	API / see Summary
user	`meshblock.cpp: 690-741`	`user_stage_ forcings`	callbacks installed
combine	`meshblock.cpp: 754-783`	`pintg`	API / see Summary
last	`meshblock.cpp: 785-842`	`pthermo`	API / see Summary
weights	`hydro_ forward.cpp: 985-995`	`cw`	API / see Summary

1. Read `advance_local` for the stage part of this path (`meshblock.cpp: 597-626`).
2. Read `carry_dry_source` for the dry part of this path (`meshblock.cpp: 633-645`).
3. Read `mass_corr` for the transfer part of this path (`meshblock.cpp: 650-678`).
4. Read `user_stage_forcings` for the user part of this path (`meshblock.cpp:690-741`).
5. Read `pintg` for the combine part of this path (`meshblock.cpp:754-783`).
6. Read `pthermo` for the last part of this path (`meshblock.cpp:785-842`).
7. Read `cw` for the weights part of this path (`hydro_forward.cpp:985-`

995).

1.13.5. Tests

Table 1.26.: Source assertions for stage.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_user_output.release`	Ordered hydro/scalar increments	rtol = atol = 1e-12	Test constants; rationale not documented
`test_user_output.release`	Shared callback; bad keys throw	pointer equality; allclose defaults	API contract; numeric rationale absent
`test_jit_user_forcing_python`	Nested scripted module adds increment	assert_close defaults	Library defaults, not specified by test

The assertion anchor `scripted_stage_forcings_add_tendencies_in_list_order` is in `test_user_output.cpp:576-606`.

The assertion anchor `scripted_module_is_shared_across_parallel_blocks` is in `test_user_output.cpp:611-667`.

The assertion anchor `expected` is in `test_jit_user_forcing.py:44-71`.

No snapy runtime result is reported by this draft. Section 12.1 records the source-defined device and rank coverage, the missing runs and the distinction between assertions and observations.

1.13.6. Limits

The stage algebra does not prove positivity after arbitrary user increments, nor accuracy after nonlinear limiting or phase adjustment. Switch combinations remain subject to Section 11.8; no new CPU, CUDA or multi-rank snapy run is claimed.

2. Governing equations and thermodynamics

This chapter follows conserved state through the EOS and the final-stage phase solve. It describes the five EOS types, their energy conventions and the identities required by transport and temperature floors. Floors belong to Chapter 8; rejection and condensate repair belong to Chapter 10.

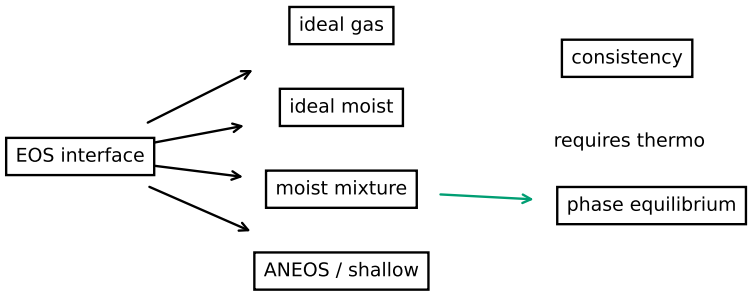


Figure 2.1.: EOS dispatch and the thermo dependency of phase equilibrium. Black arrows denote selection; the green arrow denotes the required thermo data, not a switch implication.

The final switch-matrix dependency map awaits Section 11.8.

Table 2.1.: Chapter scope; no measured convergence order is asserted.

scheme	switch	default	order	main test
conservation laws	none	always	transport-dependent	wall saturation
EOS interface	`type`	moist-mixture	not applicable	hydro options
ideal gas	`ideal-gas`	unselected	local algebra	zero offset
ideal moist	`ideal-moist`	unselected	local algebra	temperature map
moist mixture	`moist-mixture`	selected	iterative tolerance	EOS round trip
ANEOS	`aneos`	unselected	external	none
shallow water	`shallow-water`	unselected	local algebra	shallow examples
consistency	none	required	identity	temperature map
saturation	`uv-solver`	auto	iterative tolerance	saturation redo

2.1. External tabulated EOS interface (`aneos`)

2.1.1. Summary

> **At a glance**
>
> Explicit `type: aneos` and `eos-file`; default EOS remains `moist-mixture`. An external implementation must replace the throwing weak symbols. No tabulated accuracy is available from this tree.

2.1.2. Derivation

No thermodynamic model is shipped to derive. The wrapper defines the effective adiabatic index from the external sound speed, pressure and density:

$$\gamma = c_s^2 \rho / p \tag{2.1}$$

Equation 2.1 is implemented by `cs` in `aneos.cpp:40-45`. This is an interface identity, not a claim that the external material has constant heat capacity. `aneos_check.py` checks the rearrangement only.

2.1.3. Numerical method

`DU->PTL` supplies pressure, temperature and speed from density and internal energy density. `DP->TUL` supplies temperature, internal energy and speed from density and pressure. `DT->PUL` handles the temperature-floor request. All are cell-local external calls. No internal interpolation stencil is shipped.

2.1.4. Code

Table 2.2.: Code map.

what	code link	symbol	switch
keys	`aneos.cpp:12-48`	`compute`	`aneos`
conserved map	`aneos.cpp:71-109`	`_cons2prim`	`aneos`
throwing stubs	`aneos_thermo_dummy.cpp:14-32`	`ANEOSThermoImpl`	`aneos`

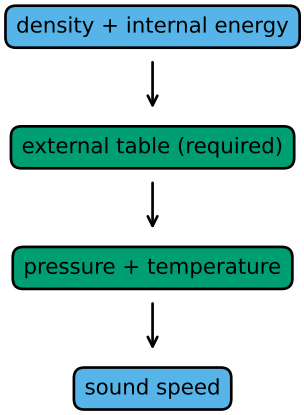


Figure 2.2.: External tabulated EOS interface: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

Read the implementation in this order:

1. Keys: `compute` in `aneos.cpp:12-48`.
2. Conserved map: `_cons2prim` in `aneos.cpp:71-109`.
3. Throwing stubs: `ANEOSThermoImpl` in `aneos_thermo_dummy.cpp:14-32`.

2.1.5. Tests

Table 2.3.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
none	external table behavior	missing evidence	no linked implementation tested

The independent derivation check is `src/snapy_report/ch02/aneos_check.py`; its committed output is `src/snapy_report/ch02/aneos_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.1.6. Limits

The shipped constructor throws. The wrapper lacks moist species and metric raising/lowering. It does not implement `W->A`; callers must use `W->L` then `WL->A`. The base heat-capacity method returns zero. No external ANEOS, wall, seam or forcing compatibility run is claimed. Switch-combination coverage remains pending in Section 11.8.

2.2. EOS and solver consistency (`none`)

2.2.1. Summary

> **At a glance**
>
> No switch; identities apply when the selected maps exist and floors do not change the state. The temperature floor requires consistent energy recovery. The enthalpy identity is Equation 2.3.

2.2.2. Derivation

At fixed conserved partial densities, adding constant-capacity species energies gives

$$\rho e(T) = \rho_{\mathrm{d}} u_{\mathrm{d}}^{\star} + \sum_n \rho_n u_n^{\star} + \left(\rho_{\mathrm{d}} c_{v,\mathrm{d}} + \sum_n \rho_n c_{v,n} \right) T \tag{2.2}$$

Equation 2.2 is implemented by `_temp2intEng` in `ideal_moist.cpp:293-317`.

Thus the zero-temperature intercept is the offset, and the slope includes the dry heat capacity. Substituting the temperature recovered from pressure reproduces `W->I`. Integrating over a uniform cell merely multiplies this identity by V_i. For temperature-dependent capacities, `UT->I` evaluates the same `VT->U` function as `W->I`, rather than assuming affinity.

The code's `species_enthalpy` returns thermodynamic enthalpy plus specific kinetic energy, and excludes the dry row. Adding that dry contribution explicitly, and summing gas pressure shares, gives

$$\rho_{\mathrm{d}} \left(\hat{h}_{\mathrm{d}} + \tfrac{1}{2}|\mathbf{v}|^2 \right) + \sum_n \rho_n \left(\hat{h}_n + \tfrac{1}{2}|\mathbf{v}|^2 \right) = E + p \tag{2.3}$$

Equation 2.3 is implemented by `species_enthalpy` in `ideal_moist.cpp: 260-280`.

The first part of the sum is internal energy plus pressure; the remaining mass-weighted kinetic terms sum to $\frac{1}{2}\rho|\mathbf{v}|^2$. This proves the identity and explains why condensates must have no pressure share. `consistency_check.py` verifies the intercept, temperature round trip and enthalpy sum independently. Exactness is algebraic for the specified energy functions; floors and incomplete Newton convergence break an inverse-map claim.

2.2.3. Numerical method

The conserved limiter asks `UT->I` for a minimum internal energy at its configured temperature. Using a different offset or omitting the dry heat capacity would enforce a different temperature. Species tables must belong to the EOS instance; cloud-parent lookup and sedimentation must not resolve indices against a subsequently replaced global table. These are local data-ownership requirements, not a spatial accuracy improvement.

2.2.4. Code

Table 2.4.: Code map.

what	code link	symbol	switch
floor consumer	`equation_of_state.cpp: 226-233`	`min_ie`	`none`
own species table	`equation_of_state.cpp: 110-134`	`cache_cloud_parents_`	`none`

what	code link	symbol	switch
nonlinear energy evaluation	`moist_mixture.cpp:232-243`	`_temp2intEng`	`none`
enthalpy carry	`moist_mixture.cpp:196-218`	`species_enthalpy`	`none`

Read the implementation in this order:

1. Floor consumer: `min_ie` in `equation_of_state.cpp:226-233`.
2. Own species table: `cache_cloud_parents_` in `equation_of_state.cpp:110-134`.
3. Nonlinear energy evaluation: `_temp2intEng` in `moist_mixture.cpp:232-243`.
4. Enthalpy carry: `species_enthalpy` in `moist_mixture.cpp:196-218`.

2.2.5. Tests

Table 2.5.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_eos_temp2inteng_python`	round trip / affinity / intercept	10^{-12} / 10^{-12} / 10^{-10}	regression thresholds in script

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_eos_species_registry_python`	table rewrite invariance	10^{-14}; step exactly equal	script contract
`test_two_cards_species.release`	second-card independence	precision dependent	test constants; see Chapter 15

The independent derivation check is `src/snapy_report/ch02/consistency_check.py`; its committed output is `src/snapy_report/ch02/consistency_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.2.6. Limits

A dedicated moist-mixture `UT->I` versus `W->I` regression is still needed. The existing temperature-sweep script selects ideal-moist only. Algebraic reuse of `VT->U` is not a replacement for a compiled test. Switch-combination coverage remains pending in Section 11.8.

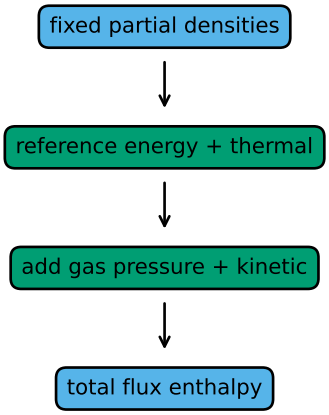


Figure 2.3.: EOS and solver consistency: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

2.3. Ideal gas conversion (`ideal-gas`)

2.3.1. Summary

> **At a glance**
>
> Explicit `type: ideal-gas`; default EOS remains `moist-mixture`. Constant heat capacity and no reference-energy offset. The cell-local algebra is exact before floors, as in Equation 2.4.

2.3.2. Derivation

For a calorically perfect gas, $e = c_{v,\mathrm{d}} T$ and $p = \rho R_{\mathrm{d}} T$, where T is in K and the heat capacities and R_{d} are in $\mathrm{J\,kg^{-1}\,K^{-1}}$. Eliminating T with

$R_\mathrm{d} = (\gamma_\mathrm{d} - 1)c_{v,\mathrm{d}}$ gives

$$\begin{aligned} \rho e &= E - \tfrac{1}{2}\sum_{a=1}^{3} v_a m_a, \\ p &= (\gamma_\mathrm{d} - 1)\rho e, \\ T &= p/(\rho R_\mathrm{d}), \\ c_s^2 &= \gamma_\mathrm{d} p/\rho. \end{aligned} \tag{2.4}$$

Equation 2.4 is implemented by `_prim2cons` in `ideal_gas.cpp:67-118`.

The kinetic contraction pairs contravariant velocity with covariant momentum, so a coordinate raise is required before computing it from conserved data. Integrating a uniform thermodynamic state over a cell multiplies both mass and energy by its volume; division by the volume returns these same relations. Applied to nonuniform cell averages, this is a closure on the average state, not an exact average of a nonlinear pointwise EOS. `ideal-gas_check.py` verifies the inverse maps with a nonorthogonal positive metric.

2.3.3. Numerical method

`_prim2cons` applies primitive floors, multiplies velocities by density, lowers momentum and adds kinetic energy. `_cons2prim` applies conserved floors, dispatches the fused conversion, then applies primitive floors. The operation is local to each supplied cell, with no special wall, seam or RK-stage stencil.

2.3.4. Code

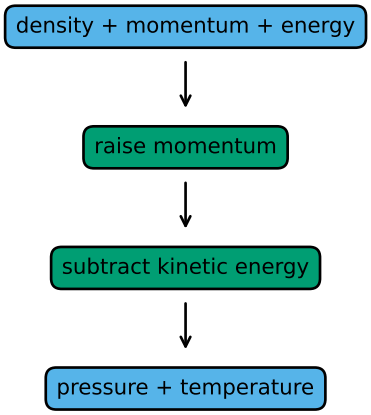


Figure 2.4.: Ideal gas conversion: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

Table 2.6.: Code map.

what	code link	symbol	switch
conversion keys	`ideal_gas.cpp:30-65`	`compute`	`ideal-gas`
metric and energy	`ideal_gas.cpp:67-118`	`_prim2cons`	`ideal-gas`
fused conversion	`ideal_gas_impl.h:16-35`	`ideal_gas_cons2prim`	`ideal-gas`

Read the implementation in this order:

1. Conversion keys: `compute` in `ideal_gas.cpp:30-65`.
2. Metric and energy: `_prim2cons` in `ideal_gas.cpp:67-118`.

3. Fused conversion: `ideal_gas_cons2prim` in `ideal_gas_impl.h:16-35`.

2.3.5. Tests

Table 2.7.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_eos.release`	zero internal-energy offset	10^{-12} absolute and relative	prescribed by `allclose`

The independent derivation check is `src/snapy_report/ch02/ideal_gas_check.py`; its committed output is `src/snapy_report/ch02/ideal_gas_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

The CPU TensorIterator loop and MPS tensor path are `ideal_gas_cons2prim_cpu` and `ideal_gas_cons2prim_mps` in `eos_dispatch.cpp:21-58`. Neither backend is exercised by the independent formula check.

2.3.6. Limits

The outline round-trip example `prim2cons_hydro_ideal_ncloud5` is inside a block comment, not active evidence: `test_eos.cpp:209-273`. Floors make the maps noninvertible. New compiled metric round trips

would strengthen coverage. Switch-combination coverage remains pending in Section 11.8.

2.4. Constant-heat-capacity moist mixture (`ideal-moist`)

2.4.1. Summary

> **At a glance**
>
> Explicit `type: ideal-moist`; requires thermo; default EOS remains `moist-mixture`. Condensates carry mass and heat but no pressure share. The fixed-composition relation is Equation 2.6; no spatial order applies.

2.4.2. Derivation

Let y_n be mass fraction per total mass, not mixing ratio per dry mass. The dry density is $\rho_\mathrm{d} = \rho(1 - \sum_n y_n)$ in $\mathrm{kg\,m^{-3}}$. Add the partial gas pressures and the heat capacities of all species. Dividing the first sum by $\rho R_\mathrm{d} T$ and the second by $\rho c_{v,\mathrm{d}}$ yields

$$\begin{aligned} f_\varepsilon &= 1 + \sum_{n \in \mathcal{V}} y_n(\mu_\mathrm{d}/\mu_n - 1) - \sum_{n \in \mathcal{C}} y_n, \\ f_\sigma &= 1 + \sum_{n} y_n(c_{v,n}/c_{v,\mathrm{d}} - 1). \end{aligned} \tag{2.5}$$

Equation 2.5 is implemented by `f_eps` in `ideal_moist.cpp:319-346`.

The reference energies $u_n^\star$ have units $\mathrm{J\,kg^{-1}}$ and enter additively, including dry air. Using $R_\mathrm{d} = (\gamma_\mathrm{d} - 1)c_{v,\mathrm{d}}$ gives

$$\begin{aligned} \rho e &= \rho_{\mathrm{d}} u_{\mathrm{d}}^{\star} + \sum_n \rho_n u_n^{\star} + \rho c_{v,\mathrm{d}} f_\sigma T, \\ p &= \rho R_{\mathrm{d}} f_\varepsilon T, \\ p &= \frac{(\gamma_{\mathrm{d}} - 1) f_\varepsilon}{f_\sigma} \left(\rho e - \rho_{\mathrm{d}} u_{\mathrm{d}}^{\star} - \sum_n \rho_n u_n^{\star} \right). \end{aligned} \tag{2.6}$$

Equation 2.6 is implemented by `_cons2prim` in `ideal_moist.cpp:164-233`.

For frozen composition, $c_p - c_v = R_{\mathrm{d}} f_\varepsilon$; divide by $c_v = c_{v,\mathrm{d}} f_\sigma$ to obtain

$$\gamma = 1 + (\gamma_{\mathrm{d}} - 1) f_\varepsilon / f_\sigma \tag{2.7}$$

Equation 2.7 is implemented by `gammad` in `ideal_moist.cpp:113-128`.

Species thermal energy is $u_n^{\star} + c_{v,n} T$. A gas adds $R_n T$ to form specific enthalpy; a condensate does not. The carried energy adds the specific kinetic energy to either result, as derived in Section 2.2. These are exact constant-capacity identities for a uniform cell, without quadrature or iteration. `ideal-moist_check.py` verifies the factors, energy recovery and frozen-composition heat ratio.

2.4.3. Numerical method

Both conversion directions use total density for velocity, while the conserved density row stores only dry air. The pressure calculation subtracts all reference energies after kinetic energy. The inverse adds them. There is no neighbour stencil; floors bracket the maps. Thermo reference buffers are populated at reset, so a later process-global species-table change must not alter this EOS.

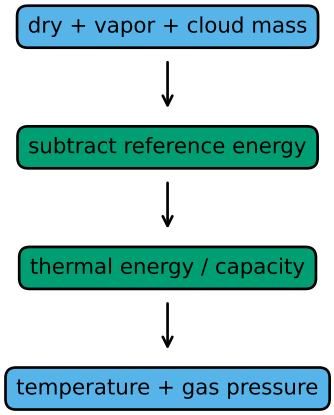


Figure 2.5.: Constant-heat-capacity moist mixture: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

2.4.4. Code

Table 2.8.: Code map.

what	code link	symbol	switch
reset and reference energies	`ideal_moist.cpp:18-51`	`u0`	`ideal-moist`
offset	`ideal_moist.cpp:70-83`	`internal_energy_offset`	`ideal-moist`
maps	`ideal_moist.cpp:164-233`	`_cons2prim`	`ideal-moist`

what	code link	symbol	switch
species energy/enthalpy	`ideal_moist.cpp:235-280`	`species_enthalpy`	`ideal-moist`
temperature map	`ideal_moist.cpp:293-317`	`_temp2intEng`	`ideal-moist`

Read the implementation in this order:

1. Reset and reference energies: `u0` in `ideal_moist.cpp:18-51`.
2. Offset: `internal_energy_offset` in `ideal_moist.cpp:70-83`.
3. Maps: `_cons2prim` in `ideal_moist.cpp:164-233`.
4. Species energy/enthalpy: `species_enthalpy` in `ideal_moist.cpp:235-280`.
5. Temperature map: `_temp2intEng` in `ideal_moist.cpp:293-317`.

2.4.5. Tests

Table 2.9.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_eos.release`	dry offset and linearity	10^{-6}	test-selected `allclose`
`test_eos_temp2inteng_python`	temperature map	10^{-12} relative	algebra regression threshold

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_eos_species_registry_python`	own species table	10^{-14} relative	regression threshold

The independent derivation check is `src/snapy_report/ch02/ideal-moist_check.py`; its committed output is `src/snapy_report/ch02/ideal-moist_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.4.6. Limits

Constant heat capacities are assumed even if a thermo card requests temperature-dependent options. The TODO about iteration in the pressure maps is not a measured defect of this constant-capacity identity. The offset test uses a dry card; it does not by itself test all species offsets. Switch-combination coverage remains pending in Section 11.8.

2.5. EOS interface and dispatch (`type`)

2.5.1. Summary

> **At a glance**
>
> YAML `dynamics/equation-of-state/type` defaults to `moist-mixture`. Five concrete types implement different conversion keys; moist types require thermo options. No discretization order or single defining equation applies to a factory.

2.5.2. Derivation

No physical derivation applies to dispatch. The caller and selected implementation must agree on both key and tensor representation. `W` denotes primitive state, `U` conserved state, `I` internal energy density, `T` temperature, `A` adiabatic index, `L` sound speed, `K` kinetic energy density and `E` species energy density. These are API labels, not additional mathematical symbols.

2.5.3. Numerical method

Dispatch is pointwise on cell tensors, including ghost entries passed by the caller; it performs no halo exchange.

Table 2.10.: Supported conversion branches, as cited in each type's Code layer.

key family	ideal gas	moist types	ANEOS	shallow water
`W->U`, `U->W`	yes	yes	yes	yes
`W->I`, `W->T`, `UT->I`	yes	yes	yes	temperature undefined

key family	ideal gas	moist types	ANEOS	shallow water
`W->A`, `WA->L`	yes	yes	no	index undefined; speed yes
`W->E`, `U->K`	no	yes	no	no
`W->L`, `WL->A`	no	no	yes	no

Unknown keys throw. YAML validates map structure and known keys. A present EOS map supplies `gammad=1.4`, `weight=0.029` in $\mathrm{kg\,mol^{-1}}$, density floor 10^{-6} in $\mathrm{kg\,m^{-3}}$, pressure floor 10^{-3} in Pa, temperature floor 20 in K, `limiter=false`, and empty `eos-file`. If the map is absent, the struct defaults remain: density and pressure floors are both 10^{-10} in their respective units. Moist resets replace `gammad` and `weight` from their thermo.

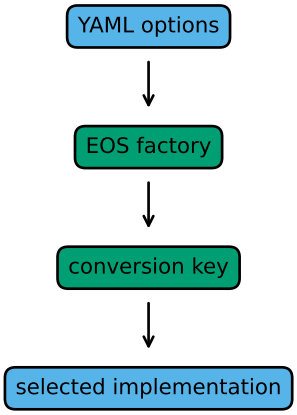


Figure 2.6.: EOS interface and dispatch: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

2.5.4. Code

Table 2.11.: Code map.

what	code link	symbol	switch
YAML validation/defaults	`equation_of_state.cpp:33-99`	`from_yaml`	`type`
struct defaults	`equation_of_state.hpp:45-57`	`density_floor`	`type`
factory	`equation_of_state.cpp:356-374`	`create`	`type`
base fallback	`equation_of_state.cpp:165-173`	`specific_heat_cv`	`type`

Read the implementation in this order:

1. Yaml validation/defaults: `from_yaml` in `equation_of_state.cpp:33-99`.
2. Struct defaults: `density_floor` in `equation_of_state.hpp:45-57`.
3. Factory: `create` in `equation_of_state.cpp:356-374`.
4. Base fallback: `specific_heat_cv` in `equation_of_state.cpp:165-173`.

2.5.5. Tests

Table 2.12.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_hydro_options.release`	unknown dynamics keys rejected	exception	configuration contract
`test_eos.release`	selected EOS behavior	per-type	see the per-type sections

No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.5.6. Limits

Temperature-floor positivity and finiteness are not validated in `from_yaml`. Selecting the default moist EOS without thermo options fails during reset. ANEOS and shallow water do not satisfy the full moist API. Switch-combination coverage remains pending in Section 11.8.

2.6. Conservation laws and stage ordering (`none`)

2.6.1. Summary

> **At a glance**
>
> All stages advance conserved cell averages. There is no independent switch; the integrator sets the stage count. Spatial and temporal order belong to the selected transport and integrator. The flux increment is defined in Equation 2.8.

2.6.2. Derivation

Density ρ has units $\mathrm{kg\,m^{-3}}$, momentum density m_a has units $\mathrm{kg\,m^{-2}\,s^{-1}}$, and total energy density E has units $\mathrm{J\,m^{-3}}$. The primitive velocities are contravariant and the conserved momenta covariant on the nonorthogonal grid. Dry density and species partial densities obey separate continuity equations; summing them gives total continuity. For an inviscid Cartesian cell without phase change,

$$\begin{aligned}
\partial_t \rho_{\mathrm{d}} + \nabla \cdot (\rho_{\mathrm{d}} \mathbf{v}) &= 0, \\
\partial_t \rho_n + \nabla \cdot (\rho_n \mathbf{v}) &= 0, \\
\partial_t \mathbf{m} + \nabla \cdot (\mathbf{m}\mathbf{v} + p\mathbf{I}) &= \rho \mathbf{g}, \\
\partial_t E + \nabla \cdot [(E + p)\mathbf{v}] &= \rho \mathbf{v} \cdot \mathbf{g}.
\end{aligned}$$

Here $\mathbf{g} = (g_1, g_2, g_3)$ is acceleration in $\mathrm{m\,s^{-2}}$. Coriolis, diffusion and configured sources add their tendencies; geometric momentum terms enter through the coordinate operator. Integration over a fixed cell changes the divergence into outward surface flux. Dividing by its volume and replacing surface integrals by numerical area-weighted fluxes gives the flux-only increment (shown in one dimension):

$$\Delta \mathbf{U}_i^{(0)} = -\frac{\Delta t}{V_i} \Delta_i [A \mathbf{F}_1] \tag{2.8}$$

Equation 2.8 is implemented by `du` in `hydro_forward.cpp:754-765`.

Summing $V_i \Delta \mathbf{U}_i^{(0)}$ cancels each shared interior face, leaving boundary fluxes. This cancellation is exact algebra when both neighbours use the same flux; accuracy of that flux is separate. The check `laws_check.py` verifies this telescoping identity and the wall-copy ordering.

Phase adjustment changes interior composition after transport. A wall ghost copied before adjustment retains old composition; reconstruction then sees a jump not present in the adjusted interior. Copying afterward makes the reflected thermodynamic state equal again. This proves consistency of the copied state, not a universal conservation guarantee for an arbitrary reconstruction or boundary condition.

2.6.3. Numerical method

The hydro operator reconstructs cell states at faces, limits fluxes, forms divergence, applies native forcings, and performs VIC if present. User stage callbacks run after the hydro return, before RK averaging. The averaged state is limited, solids are refilled, the last stage adjusts saturation, the applicable gravity fixer runs, and physical boundaries are filled. `meshblock.cpp:754-841` documents `apply_boundaries` at the end. EOS conversions themselves have no spatial stencil. Ghost exchange and seam flux agreement belong to Chapters 3 and 7.

2.6.4. Code

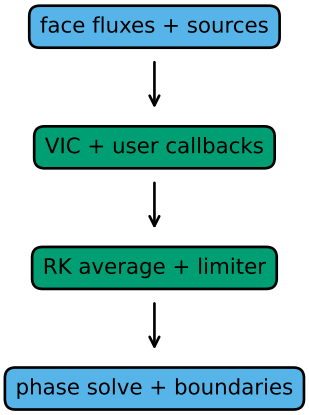


Figure 2.7.: Conservation laws and stage ordering: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

Table 2.13.: Code map.

what	code link	symbol	switch
flux and native sources	`hydro_forward.cpp:754-765`	`du`	`none`
implicit step	`hydro_forward.cpp:915-943`	`dt_corr`	`none`
callbacks	`meshblock.cpp:731-756`	`pintg`	`none`
last stage	`meshblock.cpp:795-841`	`apply_boundaries`	`none`
row layout	`snap.h:34-55`	`ICY`	`none`

Read the implementation in this order:

1. Flux and native sources: `du` in `hydro_forward.cpp:754-765`.
2. Implicit step: `dt_corr` in `hydro_forward.cpp:915-943`.
3. Callbacks: `pintg` in `meshblock.cpp:731-756`.
4. Last stage: `apply_boundaries` in `meshblock.cpp:795-841`.
5. Row layout: `ICY` in `snap.h:34-55`.

2.6.5. Tests

Table 2.14.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wall_saturation.release`	energy and water at both walls	relative 10^{-12}	test threshold; no round-off derivation supplied

The independent derivation check is `src/snapy_report/ch02/laws_check.py`; its committed output is `src/snapy_report/ch02/laws_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.6.6. Limits

`NMASS>0` is refused by the `ICY` static assertion in the Code table. Limiter and source operations can change conserved budgets. A spatial con-

vergence order for the complete stage is not measured here. Switch-combination coverage remains pending in Section 11.8.

2.7. Temperature-dependent moist thermodynamics (`moist-mixture`)

Local symbols: T_{int} is integration temperature in K; $e_n(T)$ is species internal energy; $c_{v,n}^{\mathrm{ref}}$ is its tabulated constant capacity; J is a rotational quantum number; k_{B} is Boltzmann's constant in $\mathrm{J\,K^{-1}}$; Angle brackets in the rotor formula mean a normalized finite ensemble average. These additions are tracked in ISSUES.md. ### Summary

> **At a glance**
>
> Default `type: moist-mixture`; thermo options required. Temperature inversions and heat capacities come from kintera. Sound speed follows Equation 2.9; accuracy is limited by inversion tolerance.

2.7.1. Derivation

kintera's `DY->V` stores partial mass densities ρ_n, including dry air; snapy multiplies by inverse molar masses to obtain $\tilde{c}_n = \rho_n/\mu_n$ in $\mathrm{mol\,m^{-3}}$. The API tensor named `V` is neither cell volume nor molar concentration. This corrects the explanatory sentence in the frozen notation file without changing its mathematical symbols. At fixed composition, differentiate $p = \mathcal{R}T\sum_{n\in\{\mathrm{d}\}\cup\mathcal{V}}\tilde{c}_n Z_n$ with respect to total density. Since every concentration scales with density, the implemented diagonal concentration derivative gives

$$
\begin{aligned}
c_T^2 &= \frac{\mathcal{R}T}{\rho} \sum_{n \in \{\mathrm{d}\} \cup \mathcal{V}} \tilde{c}_n \left(Z_n + \tilde{c}_n \frac{\partial Z_n}{\partial \tilde{c}_n} \right), \\
\gamma &= \frac{\rho_\mathrm{d} c_{p,\mathrm{d}} + \sum_n \rho_n c_{p,n}}{\rho_\mathrm{d} c_{v,\mathrm{d}} + \sum_n \rho_n c_{v,n}}, \\
c_s &= \sqrt{\gamma}\, c_T.
\end{aligned}
\tag{2.9}
$$

Equation 2.9 is implemented by `_adiabatic_index` in `moist_mixture.cpp:245-272`.

At the ideal-gas pin $Z_n = 1$, the speed reduces to $c_s^2 = \gamma p/\rho$. A general concentration-coupled nonideal mixture would require cross derivatives not represented by this formula. For temperature recovery, linearize volumetric internal energy at fixed partial densities. Its derivative is volumetric heat capacity, giving the update

$$
T \leftarrow T + \frac{\rho e - \rho_\mathrm{d} e_\mathrm{d}(T) - \sum_n \rho_n e_n(T)}{\rho_\mathrm{d} c_{v,\mathrm{d}}(T) + \sum_n \rho_n c_{v,n}(T)}
\tag{2.10}
$$

Equation 2.10 is implemented by `_intEng_to_temp` in `kintera thermo_y.cpp:495-517`.

Here $e_n(T)$ is species specific internal energy in $\mathrm{J\,kg^{-1}}$. The pressure inversion uses the residual $T \sum \tilde{c}_n Z_n - p/\mathcal{R}$ and subtracts it divided by $\sum \rho_n (c_{p,n} - c_{v,n})/\mathcal{R}$ over gases. For $Z_n = 1$ that denominator is the exact derivative, and the initial ideal-gas estimate already solves the equation. The damping argument in the source comment is conditional on the stated nonideal Mayer relation; no dissociation implementation is inferred from it.

For NASA-9, integrate each term of $c_{p,n}/R_n$ with respect to temperature to obtain $\hat{h}_n/R_n$, then subtract T to obtain internal energy. Referencing the integral to 300 K retains the existing energy zero:

$$e_n(T) = u_n^\star + c_{v,n}^{\mathrm{ref}}\,300\,\mathrm{K} + \int_{300\,\mathrm{K}}^{T} [c_{p,n}(T_{\mathrm{int}}) - R_n]\,\mathrm{d}T_{\mathrm{int}} \tag{2.11}$$

Equation 2.11 is implemented by `intEng_nasa` in `kintera eval_uhs.cpp:285-300`.

The constant $c_{v,n}^{\mathrm{ref}}$ is the tabulated constant capacity, not its value after enabling NASA-9.

For molecular hydrogen, each rotational level J has energy $k_{\mathrm{B}}T_{\mathrm{rot}}J(J+1)$. The implementation sums levels from 0 through 40 with $T_{\mathrm{rot}} = 87.55\,\mathrm{K}$. Normalize the weights $(2J+1)\exp[-T_{\mathrm{rot}}J(J+1)/T]$ to obtain expectations. Differentiating the mean rotational energy gives the variance formula

$$\frac{c_{v,\mathrm{H_2}}}{R_{\mathrm{H_2}}} = \frac{3}{2} + \left(\frac{T_{\mathrm{rot}}}{T}\right)^2 (\langle [J(J+1)]^2 \rangle - \langle J(J+1) \rangle^2) \tag{2.12}$$

Equation 2.12 is implemented by `h2_rot_reduce` in `kintera eval_uhs.cpp:83-92`.

Equilibrium mode multiplies odd-level weights by three. Normal mode averages separately normalized even and odd ensembles with weights one quarter and three quarters, including their separate variances. Referencing $\frac{3}{2}T + T_{\mathrm{rot}}\langle J(J+1)\rangle$ to 300 K gives the energy correction. The rotational sum is truncated at the coded level limit; no temperature-range error bound is measured here. `moist-mixture_check.py` differentiates this energy numerically in both modes and checks a NASA polynomial antiderivative and the ideal-gas Newton sign. This tests the formulas, not a compiled kintera run.

2.7.2. Numerical method

`U->W` subtracts metric kinetic energy, constructs partial densities, calls `VU->T`, then `VT->P`. `W->I`, `W->T`, `W->A` and `WA->L` refresh the cache if tensor identity or mutation version differs. The operations are cell-local; the cache is not a stencil. Floors invalidate the conserved-to-primitive cache. No spatial wall or seam accuracy order follows from a temperature inversion.

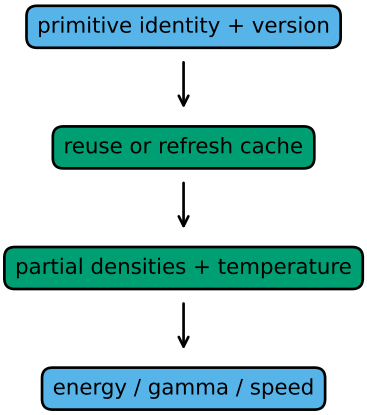


Figure 2.8.: Temperature-dependent moist thermodynamics: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

2.7.3. Code

Table 2.15.: Code map.

what	code link	symbol	switch
reset	`moist_mixture.cpp:18-31`	`reset`	`moist-mixture`
conserved map	`moist_mixture.cpp:124-162`	`_cons2prim`	`moist-mixture`
enthalpy	`moist_mixture.cpp:196-218`	`species_enthalpy`	`moist-mixture`
temperature map	`moist_mixture.cpp:232-243`	`_temp2intEng`	`moist-mixture`
cache	`moist_mixture.cpp:275-294`	`_ensure_cache`	`moist-mixture`
pressure Newton	`kintera thermo_y.cpp:439-472`	`_pres_to_temp`	`moist-mixture`
NASA polynomial	`kintera eval_uhs.cpp:24-58`	`nasa9_cp_R`	`moist-mixture`
rotor modes	`kintera eval_uhs.cpp:95-147`	`eval_h2cp`	`moist-mixture`

Read the implementation in this order:

1. Reset: `reset` in `moist_mixture.cpp:18-31`.
2. Conserved map: `_cons2prim` in `moist_mixture.cpp:124-162`.
3. Enthalpy: `species_enthalpy` in `moist_mixture.cpp:196-218`.
4. Temperature map: `_temp2intEng` in `moist_mixture.cpp:232-243`.

5. Cache: `_ensure_cache` in `moist_mixture.cpp:275-294`.
6. Pressure newton: `_pres_to_temp` in `kintera thermo_y.cpp:439-472`.
7. Nasa polynomial: `nasa9_cp_R` in `kintera eval_uhs.cpp:24-58`.
8. Rotor modes: `eval_h2cp` in `kintera eval_uhs.cpp:95-147`.

2.7.4. Tests

Table 2.16.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_eos.release`	round trip and cache invalidation	10^{-6} absolute and relative	prescribed `allclose`
`test_flux_positivity_carry.release`	NASA/H2 enthalpy sum	10^{-9} relative	source comment distinguishes omitted kinetic energy

The independent derivation check is `src/snapy_report/ch02/moist-mixture_check.py`; its committed output is `src/snapy_report/ch02/moist-mixture_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.7.5. Limits

Newton iteration warns on nonconvergence at its iteration limit and returns the current estimate; this is not the saturation failure counter. `use-`

`nasa9-cp` and `use-h2-cp` do not establish a matching phase-equilibrium closure. H2 dissociation is not claimed. The requested compiled kintera heat-capacity plot remains missing evidence; the formula check supplies no measured accuracy of the installed library. Switch-combination coverage remains pending in Section 11.8.

2.8. Fixed-volume fixed-energy phase equilibrium (`uv-solver`)

Local symbols: ζ_k is reaction extent per volume; $\mathbf{b}$ is the dimensionless linearized residual; J its extent Jacobian; λ are constraint multipliers. $e_n(T)$ is species specific internal energy. These are tracked in ISSUES.md. ### Summary

> **i** At a glance
>
> Runs after the last stage when thermo reactions exist. `uv-solver` defaults to `auto`. `kkt` selects the constrained solve; `partition` requires disjoint vapor-cloud reactions. The fixed-energy constraint is Equation 2.13; this is an operator-split solve.

`uv_solver` defaults to `auto`, `max_iter` to 10 and `ftol` to 10^{-6} in `kintera thermo.hpp:80-87`.

2.8.1. Derivation

A local phase change has no transport across the cell boundary. Integrating the energy equation over that closed cell fixes $V_i \rho e$; with fixed volume this fixes internal energy density. Stoichiometric extents change concentrations without changing conserved elemental totals. Write an extent ζ_k in $\mathrm{mol\,m^{-3}}$ for each reaction:

$$\begin{aligned} \tilde{c}_n^{\mathrm{new}} &= \tilde{c}_n + \sum_k \nu_{nk}\zeta_k \geq 0, \\ \rho e &= \sum_{n\in\{\mathrm{d}\}\cup\mathcal{V}\cup\mathcal{C}} \mu_n \tilde{c}_n^{\mathrm{new}} e_n(T). \end{aligned} \tag{2.13}$$

Equation 2.13 is implemented by `conc` in `kintera equilibrate_uv.h: 507-586`.

For one vapor-cloud pair, a present cloud requires $S_n = 1$; an absent cloud permits $S_n \leq 1$. The general implementation compares the stoichiometrically weighted logarithm of gaseous reactant and product concentrations with the fitted log saturation pressure minus the net gaseous stoichiometry sum times $\ln(\mathcal{R}T)$. At constant energy, differentiating the energy constraint gives

$$\mathrm{d}T = -\frac{\sum_n \mu_n e_n \,\mathrm{d}\tilde{c}_n}{\sum_n \mu_n \tilde{c}_n c_{v,n}},$$

where both sums include dry air. Substitute this into the differential of the log saturation residual. The concentration derivative of each gaseous logarithm is $-\nu_{nk}/\tilde{c}_n$; the temperature derivative adds the log-saturation derivative times the energy coefficient divided by volumetric heat capacity. These are precisely the two contributions to `weight`. Multiplying this row by the active stoichiometry forms `gain`.

The constrained least-squares step minimizes the squared linearized residual subject to $\tilde{\mathbf{c}} + \mathsf{N}\zeta \geq 0$. With local residual vector $\mathbf{b}$, Jacobian J, and multipliers λ, its stationarity and complementarity conditions are

$$\begin{aligned} \mathsf{J}^{\mathsf{T}}(\mathsf{J}\zeta - \mathbf{b}) - \mathsf{N}^{\mathsf{T}}\lambda &= 0, \\ \lambda &\geq 0, \\ \lambda_n(\tilde{c}_n + (\mathsf{N}\zeta)_n) &= 0. \end{aligned}$$

Both gaseous sides enter with their signed stoichiometry. A reaction with required gases absent on both sides is deferred. The zero-extent point is feasible for nonnegative input concentrations. The source additionally

limits each reaction to available reactants, damps large gas-concentration changes, then Newton-solves the energy constraint with a precision-aware tolerance and a bound of 100 temperature iterations. The tolerance is the larger of 10^{-4} K and $8\epsilon_{\mathrm{m}}|T|$. These safeguards mean a raw KKT solution is not the complete update. `saturation_check.py` verifies mass conservation for a balanced reaction, fixed-energy temperature recovery, and an interior constrained least-squares solution. It is not a replica of active-set selection or a proof of global nonlinear convergence.

2.8.2. Numerical method

`MeshBlockImpl` subtracts kinetic energy and passes interior density, internal energy and mass fractions to `ThermoYImpl::forward`, with warm start enabled. Momentum and the total-energy row are not rewritten by the phase solver. It rewrites species fractions; physical ghosts are refreshed afterward. Warm-start reuse requires matching shape and device. Failures are counted from negative diagnostics; CPU warns immediately, while device counters are consumed by the caller. Chapters 7 and 10 own rejection and rollback.

2.8.3. Code

Table 2.17.: Code map.

what	code link	symbol	switch
snapy call	`meshblock.cpp: 795-816`	`pthermo`	`uv-solver`
warm start and solver	`kintera thermo_y.cpp: 275-356`	`uv_solver`	`uv-solver`

what	code link	symbol	switch
failure counter	`kintera thermo_y.cpp: 358-389`	`nfail_`	`uv-solver`
active equations	`kintera equilibrate_ uv.h:383-504`	`weight`	`uv-solver`
diagnostic encoding	`kintera equilibrate_ uv.h:609-612`	`diag`	`uv-solver`

Read the implementation in this order:

1. Snapy call: `pthermo` in `meshblock.cpp:795-816`.
2. Warm start and solver: `uv_solver` in `kintera thermo_y.cpp:275-356`.
3. Failure counter: `nfail_` in `kintera thermo_y.cpp:358-389`.
4. Active equations: `weight` in `kintera equilibrate_uv.h:383-504`.
5. Diagnostic encoding: `diag` in `kintera equilibrate_uv.h:609-612`.

2.8.4. Tests

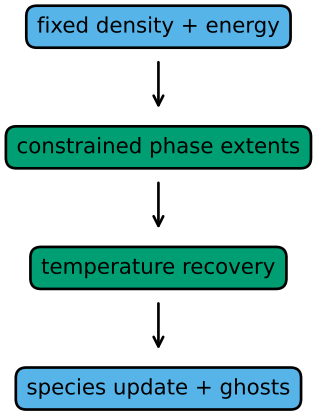


Figure 2.9.: Fixed-volume fixed-energy phase equilibrium: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

Table 2.18.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_check_redo_saturation_python`	failed solve triggers rejection	logical assertions	failure protocol
`test_wall_saturation.release`	phase change and wall budgets	relative 10^{-12}	test threshold

The independent derivation check is `src/snapy_report/ch02/`

`saturation_check.py`; its committed output is `src/snapy_report/ch02/saturation_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.8.5. Limits

The energy kernel receives constant capacities and optional extra-energy function pointers. A matching NASA-9/H2 phase closure is not established by the tensor EOS energy function. Negative input concentrations are clamped with a warning, and nonpositive temperature fails. No full active-set convergence or multi-rank phase-equilibrium run is supplied by this draft. Switch-combination coverage remains pending in Section 11.8.

2.9. Shallow-water conversion (`shallow-water`)

Local notation exception: d_{sw} denotes the depth-like `IDN` slot with gravity absorbed, not the volumetric gas density. The local symbol is tracked in ISSUES.md. ### Summary

> **At a glance**
>
> Explicit `type: shallow-water`; default EOS remains `moist-mixture`. Four rows store depth-like density and three momenta; there is no energy row. The speed is Equation 2.14; thermodynamic temperature is undefined.

2.9.1. Derivation

No three-dimensional thermodynamic closure applies. The API computes speed from its stored depth-like density, with gravity absorbed in that variable:

$$c_s^2 = d_{\mathrm{sw}} \tag{2.14}$$

Equation 2.14 is implemented by `sqrt` in `shallow_water.cpp:26-33`. Here d_{sw} is the shallow-water use of the `IDN` slot is not a volumetric gas density; its dimensions are $\mathrm{m}^2\,\mathrm{s}^{-2}$. This local exception is tracked in ISSUES.md. `shallow-water_check.py` checks the square-root identity on positive states.

2.9.2. Numerical method

Conserved-to-primitive conversion divides momentum by the depth-like slot and raises horizontal components. The inverse multiplies and lowers them. Floors bracket the conversion. Neither temperature nor adiabatic index is returned as a defined tensor. There is no EOS neighbour stencil.

2.9.3. Code

Table 2.19.: Code map.

what	code link	symbol	switch
keys	`shallow_water.cpp:14-35`	`compute`	`shallow-water`
maps	`shallow_water.cpp:38-68`	`_cons2prim`	`shallow-water`

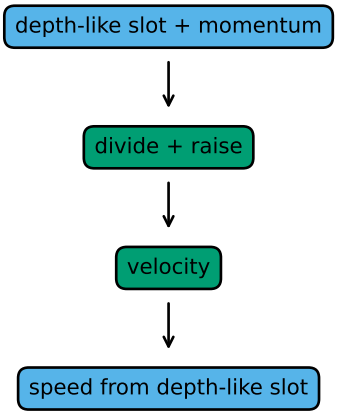


Figure 2.10.: Shallow-water conversion: labelled boxes show the data flow; arrows show evaluation order. Blue boxes hold cell data, green boxes apply the operation.

Read the implementation in this order:

1. Keys: `compute` in `shallow_water.cpp:14-35`.
2. Maps: `_cons2prim` in `shallow_water.cpp:38-68`.

2.9.4. Tests

Table 2.20.: Evidence inventory; assertions are read from the pinned source, not rerun results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_shallow_xy`, `test_shallow_splash`	example regressions	see their comparison scripts	`FULL_TESTS` registrations

The independent derivation check is `src/snapy_report/ch02/shallow-water_check.py`; its committed output is `src/snapy_report/ch02/shallow-water_check.out`, line `[C1]`. No compiled snapy test is run for this draft. CPU/CUDA availability is recorded in Section 12.2; no multi-rank thermodynamic result is claimed.

2.9.5. Limits

The hydro forcing loop requests `W->T`; only forcings that tolerate an undefined temperature are usable. No gas-EOS conservation statement is transferred to shallow water. Switch-combination coverage remains pending in Section 11.8.

3. Grids and geometry

This chapter describes the grids snapy integrates on and what a ghost cell holds. It covers the three working coordinate systems (Cartesian, spherical-polar and the gnomonic equiangular cubed sphere) with the metric quantities the finite-volume update uses: face areas, cell volumes, centroids, widths, geometric source terms and the local frames of the Riemann problems. It also covers the curved-grid helpers that the face-flux corrections of chapter 4 read, how the one global grid is cut into blocks and ranks, and what each kind of ghost contains at a physical face, an internal face, a cubed-sphere panel edge and an x_1 seam between blocks. How ghost values are carried (buffers, tags, backends) is chapter 14; here only what they are is described. All code is cited at `chengcli/snapy` commit `e894700ff7aee30b52882e5202b16461413780b0`. The schemes are listed in Table 3.1; their switch couplings are in the switch coverage matrix of chapter 12.

Table 3.1.: The schemes of this chapter. "order" is the accuracy of the quantity the scheme produces; "exact" means equal to the defining integral or copy up to round-off. Test names omit the `.release` suffix of C++ entries.

scheme	switch	default	order	main test
global grid, sliced per block (Section 3.1)	YAML `geometry`, `distribute`	on	exact	`test_coordinate`

scheme	switch	default	order	main test
Cartesian metric (Section 3.2)	`type: cartesian`	default type	exact	`test_coordinate`
spherical-polar metric (Section 3.3)	`type: spherical-polar`	off	exact metric	`test_coordinate`
gnomonic cubed sphere (Section 3.4)	`type: gnomonic-equiangle`	off	exact metric	`test_cubed_sphere_cell_volume_python`
radial face moments (Section 3.5)	`SNAP_FLUX_COVARIANCE`	off	exact	`test_radial_face_moments`
layouts and rank maps (Section 3.6)	`distribute: layout`	`slab`	n/a	`test_exchange_decomp`
physical and internal faces (Section 3.7)	`boundary-condition`	`reflecting`	n/a	`test_exchange`
generic ghost exchange (Section 3.8)	`SyncOptions`	conserved, no corners	exact copy	`test_mesh_exchange_python`
cross-panel ghost interpolation (Section 3.9)	`interpolate`	on per stage	2	`test_cubed_sphere_exchange`

scheme	switch	default	order	main test
face-state seam sync (Section 3.10)	none	on (cubed sphere)	exact copy	`test_flux_covariance_seams_python`
donor factor raw copy (Section 3.11)	`limiter`	on with limiter	exact copy	`test_flux_positivity_cubedsphere_python`
x_1 seams between blocks (Section 3.12)	`layout: cubed`, `nb1`	`nb1: 1`	round-off	`test_x1_seam_split`
conserved vs primitive sync (Section 3.13)	`type`	conserved	exact	`test_coordinate`
x_1-wall corner refresh (Section 3.14)	none	on	exact	`test_wb_wall_corner`
`cylindrical` stub (Section 3.15)	`type: cylindrical`	off	none	none

3.1. One global grid, sliced per block (YAML `geometry`)

Local symbols (flagged to the editor): $x_a^{\min}, x_a^{\max}$ the declared global bounds of axis a [m or rad]; n_a^{glob} the declared global cell count of axis a (count); Δx_a the global cell width of axis a [m or rad]; o_a the global

index of a block's first interior cell on axis a (count); $X_{a,m}$ entry m of the global face array of axis a.

3.1.1. Summary

> **i** At a glance
>
> **What**: every block's faces are a slice of one global face array per axis, so a face shared by two blocks, or seen under two decompositions, is the same double. **Switch**: none; the YAML `geometry: {type, bounds, cells}` and `distribute: {nb1, nb2, nb3}` define the grid. **Default**: on; a block built without a declared global grid adopts itself as the global grid. **Requires**: each global count divisible by the blocks on its axis. **Order**: exact (bitwise). **Defining equation**: Equation 3.1.

snapy builds the face coordinates of a block by slicing one global array per axis rather than by spacing the block's own interval. Spacing per block produces the same physical face from different floating-point arithmetic in the two blocks that share it and in different decompositions of one problem, and every metric term (areas, volumes, the spherical-polar and gnomonic source coefficients) inherits that difference. With the slice, a face has one value whoever owns it, and the cell width is the global Δx_a in every block.

The scheme is always on. The YAML card declares the global bounds and cell counts; `distribute` declares the number of blocks per axis; each block's own bounds and counts follow from its place in the layout (Section 3.6). A block built programmatically without a global grid adopts its own bounds and count as the global grid. Alias in the history: chengcli/snapy#222.

3.1.2. Derivation

The scheme is a design invariant, not an approximation, so the derivation is the argument that two blocks agree. Axis a is declared as n_a^{glob} cells on $[x_a^{\min}, x_a^{\max}]$, with the global width

$$\Delta x_a = \frac{x_a^{\max} - x_a^{\min}}{n_a^{\mathrm{glob}}}.$$

The global face array X_a has $n_a^{\mathrm{glob}} + 2n_g + 1$ entries spaced over the interval $[x_a^{\min} - n_g \Delta x_a,\ x_a^{\max} + n_g \Delta x_a]$, so entry m is global face $m - n_g$ and the n_g outermost entries on each side are ghost faces. A block whose first interior cell is global cell o_a and which holds n_a interior cells takes the consecutive entries

$$x_{a,\,m} = X_{a,\,o_a+m}, \qquad m = 0, \ldots, n_a + 2n_g, \tag{3.1}$$

where the offset is the rounded quotient

$$o_a = \mathrm{round}\left(\frac{x_{a,\min}^{\mathrm{blk}} - x_a^{\min}}{\Delta x_a} \right). \tag{3.2}$$

The slice (3.1) is computed by `block_faces_` (`coordinate.cpp:307-324`) and the offset (3.2) by `_offset` (`coordinate.hpp:124-128`), called through `ix1` (`coordinate.hpp:92-95`).

Two blocks that share global face g read the same entry $X_{a,g+n_g}$, so their face values are equal bit for bit, and the same holds between any two decompositions that slice the same X_a. The offset (3.2) needs rounding, not truncation: `repartition` places block l at $x_a^{\min} + l\,(x_a^{\max} - x_a^{\min})/n_b$, so the quotient is an integer only up to round-off, and truncating a value just below an integer would shift the block by a whole cell. Nothing is

truncated in this scheme; the statement is exact, and no executable check is needed beyond the bitwise test of the Tests layer.

3.1.3. Numerical method

The face arrays `x1f`, `x2f`, `x3f` hold face positions (points), $n_a + 2n_g + 1$ per axis on a live axis. The ghost faces are the global array continued past the domain by n_g widths, so a ghost cell has the same width as an interior cell. An axis with $n_a \leq 1$ is degenerate: it has two faces and no ghost layers, taken from the global array without the ghost extension, so that $x_a^{\min} + n_a^{\mathrm{glob}} \Delta x_a$, which need not round to $x_a^{\max}$, is never formed. Cell widths on Cartesian and gnomonic grids are the global Δx_a (Section 3.2, Section 3.4); on spherical-polar they are differences of the sliced faces (Section 3.3). The construction runs once, when a block is built; it is not part of any stage. Figure 3.1 shows the slice for two decompositions.

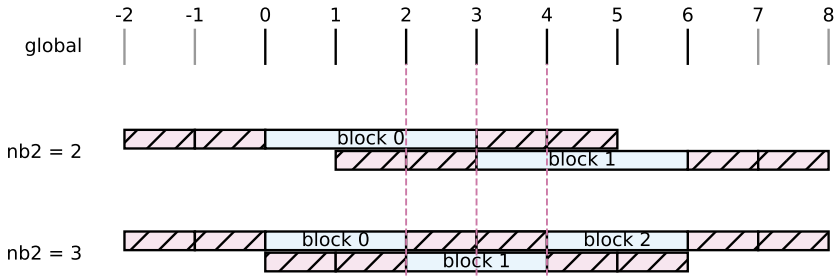


Figure 3.1.: One global face array along x_2 with its ghost extension, sliced for two blocks (top) and three blocks (bottom). Ghost faces of each block are hatched; a face shared by two blocks is one entry of the global array, so both blocks hold the same value.

Setup validates the declared grid. When no global grid is declared on an axis (global count 0), the block adopts itself; otherwise the block must lie inside the global interval within a tolerance of

$16\,\varepsilon\max(|x_a^{\min}|, |x_a^{\max}|, |x_a^{\max} - x_a^{\min}|)$, with ε the double-precision machine epsilon, and must be exactly n_a global cells: both its ends must sit on global faces within that tolerance. The tolerance covers the accumulation in `repartition`, which reaches the last upper bound by adding block widths.

3.1.4. Code

Table 3.2.: Code map of the global grid.

what	code link	symbol	switch
defaults of the grid options	`coordinate.hpp: 97-117`	`global_nx1`	YAML `geometry`
global width	`coordinate.hpp: 76-90`	`dx1`	none
block offset	`coordinate.hpp: 124-128`	`_offset`	none
YAML card	`coordinate.cpp: 62-71`	`from_yaml`	`geometry`
bounds-only card	`coordinate.cpp: 115-123`	`global_nx1`	`geometry.bounds`
divisibility checks	`coordinate.cpp: 143-162`	`nb1`	`distribute`
block bounds	`coordinate.cpp: 235-281`	`repartition`	`distribute`
axis check	`coordinate.cpp: 186-224`	`_resolve_axis`	none
face slice	`coordinate.cpp: 283-305`	`block_faces_`	none
mesh generator	`coordinate.cpp: 326-349`	`reset_ coordinates`	none

In step order (Table 3.2):

1. `from_yaml` reads `geometry.type` (default `cartesian`), the bounds (default $[0, 1]$ on each axis, with `x2min_pi` style keys multiplied by π) and the cell counts into the global fields (`coordinate.cpp:62-113`). Unknown keys are refused by `check_keys`.
2. A card with `bounds` and no `cells` declares one cell per axis spanning the bounds, and sets the block bounds to the same interval: `global_nx1` is set to 1 (`coordinate.cpp:115-123`).
3. Each global count must be divisible by the blocks on its axis; otherwise setup stops with "Number of total x1 grids must be divisible by the number of mesh blocks in x1 direction" (and the same for x2, x3), checked with `nb1` and its siblings (`coordinate.cpp:143-162`).
4. `nghost` is read (default 1) and `interp_order` (default 2); then `repartition` sets the block's bounds from its layout location, with x_1 forced to the first block on the cubed sphere, and its counts to n_a^{glob}/n_b (`coordinate.cpp:235-281`).
5. `resolve_global_grid` runs `_resolve_axis` on each axis: adopt the block when the global count is 0, else check inside-ness and the exact cell count (`coordinate.cpp:186-233`). The two refusals a user can hit read "block x2 = […] lies outside the declared global x2 grid" and "… is not exactly n cells of the declared global x2 grid".
6. The constructor calls `block_faces_` per axis with the global bounds and count and the block's count and offset (`coordinate.cpp:283-305`); `block_faces_` slices the global `linspace` (`coordinate.cpp:307-324`).
7. Each coordinate type's `reset` then builds widths and centres from these faces (the metric schemes of this chapter).

3.1.5. Tests

Table 3.3.: Tests of the global grid. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_coordinate`	faces of each block (nb2 = 2, 3) equal the undecomposed faces	bitwise	a shared face is one double
`test_coordinate`	programmatic blocks span the caller's bounds	$10^{-15}\Delta x_a$	round-off of one spacing
`test_local_horizontal_cells_python`	the horizontal setter leaves every axis resolved	$10^{-12}\Delta x_a$	set in the script
`test_cubed_sphere_exchange`	gnomonic `x2f`, `dx2f` equal across nb2 = 1, 2, 4	bitwise	as above

The bitwise case is `blocks_match_the_undecomposed_grid_bitwise` (`test_coordinate.cpp:892-921`): a six-cell x_2 axis is split into two and three blocks and each block's interior faces are compared with `torch::equal` against the matching slice of the undecomposed grid. The programmatic cases run from `faces_span_the_callers_bounds_on_every_axis` to `a_declared_block_must_span_exactly_nx_global_faces` (`test_coordinate.cpp:755-1030`); they cover the adopted grid, the bounds-only card, a degenerate axis beside live ones, and the two refusals of step 5. The Python setter test checks the resolved counts and the faces of each axis with `NG` ghost layers (`test_local_horizontal_cells.py:28-37`).

The gnomonic case is `subdivided_panel_exchange_matches_one_block` (`test_cubed_sphere_exchange.cpp:159-179`).

All four entries run on CPU; the gnomonic comparison also has a CUDA case in the same binary, which skips without a GPU. None runs on more than one rank. No number is quoted from them here: their pass status at the pin was not run for this chapter (missing evidence). The code comment that motivated the scheme records that at a cubed sphere with 64 cells per panel edge and nb2 = 4, 15 of 71 x_2 faces were inconsistent between the two blocks sharing them and 26 differed from the nb2 = 2 grid, each by one unit in the last place, in the comment on the `x2` faces (`coordinate.cpp:291-298`); the deck and run are not recorded (missing evidence).

3.1.6. Limits

> ⚠ Limits
>
> - A mesh generator passed to `reset_coordinates` rebuilds the faces from the block's own bounds, outside the global-slice invariant (`coordinate.cpp:326-349`).
> - The checks "Number of x1 grids must be greater than the ghost zone size" never fire: they run before `repartition` sets the block count, so they read `nx1` at its default of 1 (`coordinate.cpp:164-179`).

The mesh-generator path is reached only by code that calls `reset_coordinates`; no test at the pin exercises it with a decomposed grid (not measured). The dead `nghost` checks are read from the code; a block narrower than its ghost zone is therefore not refused here, and what happens instead was not measured. Bitwise decomposition invariance of the faces is tested for x_2 splits of 2 and 3 blocks on Cartesian and of 2 and 4 blocks per panel on the cubed sphere; x_1 and x_3 splits of the faces

are not tested on their own. The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.2. Cartesian metric (`cartesian`)

3.2.1. Summary

> **i** At a glance
>
> **What**: uniform widths from the global grid; face areas are products of two widths, the volume the product of three, the centre the midpoint. **Switch**: YAML `geometry.type: cartesian`. **Default**: this is the default type. **Requires**: nothing. **Order**: exact. **Defining equation**: Equation 3.3.

On a Cartesian grid every metric quantity is a product of cell widths, and the widths are the global Δx_a of Section 3.1, so a cell's metric does not depend on which block holds it. The cell centre is the midpoint of its faces, which is also the volume centroid, and the non-orthogonality cosine that the cubed sphere needs (Section 3.4) is identically zero. The finite-volume operator is the plain flux divergence; there are no geometric source terms. The type is the default when `geometry.type` is absent.

3.2.2. Derivation

For a cell $[x_{1,i-1/2}, x_{1,i+1/2}] \times [x_{2,j-1/2}, x_{2,j+1/2}] \times [x_{3,k-1/2}, x_{3,k+1/2}]$ with widths $\Delta x_1, \Delta x_2, \Delta x_3$ the measures are the integrals of $\mathrm{d}x_2\,\mathrm{d}x_3$, $\mathrm{d}x_1\,\mathrm{d}x_3$, $\mathrm{d}x_1\,\mathrm{d}x_2$ and $\mathrm{d}V$:

$$\begin{aligned} A_{i\pm1/2} &= \Delta x_2\,\Delta x_3, \\ A_{j\pm1/2} &= \Delta x_1\,\Delta x_3, \\ A_{k\pm1/2} &= \Delta x_1\,\Delta x_2, \\ V_i &= \Delta x_1\,\Delta x_2\,\Delta x_3. \end{aligned} \tag{3.3}$$

The face areas of Equation 3.3 are `face_area1` to `face_area3` and the volume is `cell_volume` (`coordinate.cpp:425-440`). The centroid of a uniform weight over an interval is its midpoint, $x_{1,i} = \frac{1}{2}(x_{1,i-1/2} + x_{1,i+1/2})$, computed in `reset` for `x1v` (`cartesian.cpp:22-28`). Integrating the conservation law $\partial_t q + \nabla \cdot \mathbf{F} = 0$ over the cell and dividing by V_i gives the update of the cell average by the face-area-weighted flux difference over the volume, with no further term, because the basis vectors are constant. Every quantity here is exact; the executable check is the product test of the Tests layer.

3.2.3. Numerical method

Widths `dx1f`, `dx2f`, `dx3f` are cell quantities, one per cell including ghosts; `x1v` holds the midpoints; `dx1v` (the centre spacing) equals the width. All are built once in `reset` from the global width. The divergence takes the face fluxes $\mathbf{F}_a$ (face averages, from the Riemann solver), multiplies them by the face areas, differences them across each interior cell and divides by V_i; ghost cells receive no divergence. Figure 3.2 draws one cell with its three face areas.

A mesh generator (`reset_coordinates`) may move the faces; the Cartesian override then recomputes widths, midpoints and centre spacings from the moved faces, so the grid is no longer uniform and Equation 3.3 uses the local widths (`cartesian.cpp:50-60` for the x_1 part).

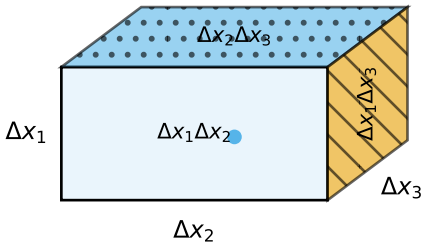


Figure 3.2.: One Cartesian cell with its widths and the three face areas $\Delta x_2 \Delta x_3$, $\Delta x_1 \Delta x_3$ and $\Delta x_1 \Delta x_2$; the centre is the midpoint.

3.2.4. Code

Table 3.4.: Code map of the Cartesian metric.

what	code link	symbol	switch
widths, centres, zero cosine	`cartesian.cpp: 8-48`	`reset`	`type: cartesian`
face areas and volume	`coordinate.cpp: 425-440`	`cell_volume`	none
flux divergence	`coordinate.cpp: 509-553`	`divergence`	none
stage operator (no source)	`coordinate.cpp: 585-591`	`forward`	none
type selection	`coordinate.cpp: 593-610`	`create`	`geometry.type`

In step order (Table 3.4):

1. `create` builds a `Cartesian` module when `type` is `cartesian`; an unknown type stops setup with “Unknown coordinate type” (`coordinate.cpp:593-610`).

2. `reset` sets the three cosine buffers to zero and the widths, midpoints and centre spacings from the global width (`cartesian.cpp:8-48`).
3. In each stage, `forward` returns the `divergence` of the three fluxes and ignores the face pressure (`coordinate.cpp:585-591`); `divergence` forms the area-weighted differences over the interior cells and divides by `cell_volume` (`coordinate.cpp:509-553`).

3.2.5. Tests

Table 3.5.: Tests of the Cartesian metric. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_coordinate`	the volume is the product of the caller's widths	10^{-14} relative	round-off of a three-factor product
`test_coordinate`	a degenerate x_1 beside live axes has width 5	10^{-14} absolute	round-off of one difference

The cases are `cell_volume_is_the_product_of_the_callers_widths` (`test_coordinate.cpp:781-787`) and `a_degenerate_axis_beside_live_ones` (`test_coordinate.cpp:789-806`). Both run on CPU only, one block, one rank. Their pass status at the pin was not run for this chapter (missing evidence).

3.2.6. Limits

No known limits beyond those of the global grid (Section 3.1). The divergence of a mesh-generator grid is not tested at the pin (not measured). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.3. Spherical-polar metric and sources (`spherical-polar`)

Local symbols (flagged to the editor): $r_{\pm} = x_{1,i\pm1/2}$ and $\theta_{\pm} = x_{2,j\pm1/2}$, the faces of one cell; $f^{(a)}_{i\pm1/2}$ the x_1 face flux of the momentum component m_a (`flux1[IVY]`, `flux1[IVZ]`) [kg m^{-1} s^{-2}]; $p_{i\pm1/2}$ the face pressure returned by the Riemann solver [Pa].

3.3.1. Summary

> **i** At a glance
>
> **What**: the exact $r^2 \sin\theta$ measures of a spherical-polar cell, volume centroids for the cell centres, and the curvature source terms of the momentum equations. **Switch**: YAML `geometry.type: spherical-polar`. **Default**: off. **Requires**: x_2 inside $[0, \pi]$. **Order**: the metric is exact; the source terms are second order. **Defining equation**: Equation 3.4.

On a spherical-polar grid $(x_1, x_2, x_3) = (r, \theta, \varphi)$ every face area and volume is the exact integral of the metric over the cell, so the discrete divergence of a uniform field is exact, and the cell centres are volume centroids, so a stored cell value sits where a cell average of a linear field equals the point

value. Because the basis vectors turn from cell to cell, the momentum equations carry geometric source terms. The radial one can be written so that the pressure force reduces to the plain face difference of the Riemann solver's face pressure, which the well-balanced reconstruction of chapter 5 balances against gravity.

The scheme is on when `geometry.type` is `spherical-polar`. Its x_1 pressure source has a second form under `SNAP_X1_CENTROID_EXACT` (chapter 5 owns that form); the curved-face corrections of chapter 4 read the radial face moments of Section 3.5. The coefficient names `coord_src1_i` to `coord_src3_j` follow the code.

3.3.2. Derivation

The volume element is $\mathrm{d}V = r^2 \sin\theta \,\mathrm{d}r \,\mathrm{d}\theta \,\mathrm{d}\varphi$. Integrating it, and the area elements of the three face families, over cell (i, j, k) gives

$$\begin{aligned} A_{i\pm1/2} &= r_\pm^2 \,|\cos\theta_- - \cos\theta_+|\,\Delta\varphi, \\ A_{j\pm1/2} &= \tfrac{1}{2}\,(r_+^2 - r_-^2)\,|\sin\theta_\pm|\,\Delta\varphi, \\ A_{k\pm1/2} &= \tfrac{1}{2}\,(r_+^2 - r_-^2)\,\Delta\theta, \\ V_i &= \tfrac{1}{3}\,(r_+^3 - r_-^3)\,|\cos\theta_- - \cos\theta_+|\,\Delta\varphi. \end{aligned} \tag{3.4}$$

In the notation of NOTATION.md, $V_i = \varpi_{jk} V_i^{(1)}$ with $V_i^{(1)} = \frac{1}{3}(r_+^3 - r_-^3)$ and the solid angle $\varpi_{jk} = |\cos\theta_- - \cos\theta_+|\Delta\varphi$. Equation 3.4 is `face_area1` to `face_area3` and `cell_volume` (`spherical_polar.cpp:178-211`).

The cell centres are the centroids of the measure along each axis:

$$\begin{aligned} x_{1,i} &= \frac{\int r^3 \,\mathrm{d}r}{\int r^2 \,\mathrm{d}r} = \frac{3}{4}\,\frac{r_+^4 - r_-^4}{r_+^3 - r_-^3}, \\ x_{2,j} &= \frac{\int \theta \sin\theta \,\mathrm{d}\theta}{\int \sin\theta \,\mathrm{d}\theta} = \frac{(\sin\theta_+ - \theta_+\cos\theta_+) - (\sin\theta_- - \theta_-\cos\theta_-)}{\cos\theta_- - \cos\theta_+}, \end{aligned} \tag{3.5}$$

with the integrals over the cell's own interval and $x_{3,k}$ the midpoint. Equation 3.5 is `radial_centers` and `polar_centers` (`spherical_polar.cpp:17-28`); with a single x_2 cell the code sets `x2v` to the midpoint instead (`spherical_polar.cpp:64-70`).

The momentum equations in conservation form, written for the physical components $m_a = \rho v_a$, are the divergence of the momentum flux $\rho\mathbf{v}\mathbf{v} + p\mathbf{I}$ plus terms from the turning of the basis. For the radial component the extra terms are $\left[\rho(v_2^2 + v_3^2) + 2p\right]/r$. Averaging over the cell, and using that the measure separates, the factor $1/r$ becomes its volume average and the factor $\cot\theta$ of the θ equation becomes its own:

$$\langle r^{-1}\rangle_{V_i} = \frac{\frac{1}{2}\left(r_+^2 - r_-^2\right)}{\frac{1}{3}\left(r_+^3 - r_-^3\right)},$$
$$\langle \cot\theta\rangle_{V_i} = \frac{\sin\theta_+ - \sin\theta_-}{\cos\theta_- - \cos\theta_+}.$$

These are `coord_src1_i` and `coord_src1_j` (`coord_src3_j` is the same value). Without face pressures the radial source is, to second order,

$$S_i^{(1)} = \langle r^{-1}\rangle_{V_i}\left[\rho_i\left(v_{2,i}^2 + v_{3,i}^2\right) + 2p_i\right], \tag{3.6}$$

computed in `forward` as `src1` (`spherical_polar.cpp:243-279`), which is subtracted from the divergence row of m_1. The approximation is the replacement of the cell average of a product by the product of cell values, an $O(h^2)$ error.

When the Riemann solver returns the face pressures $p_{i\pm1/2}$ and they are inside the m_1 flux, the code replaces $2p_i\langle r^{-1}\rangle_{V_i}$ by

$$S_i^p = \frac{A_{i+1/2}\,p_{i+1/2} - A_{i-1/2}\,p_{i-1/2}}{V_i} - \frac{p_{i+1/2} - p_{i-1/2}}{h_i}. \tag{3.7}$$

Equation 3.7 is the `pressure_gradient` branch (`spherical_polar.cpp:265-275`). Its first term cancels the pressure part of the flux divergence exactly, so the net pressure force on m_1 is $-(p_{i+1/2} - p_{i-1/2})/h_i$, the plain face difference. For a uniform p it equals $p\,(A_{i+1/2} - A_{i-1/2})/V_i = 2p\,\langle r^{-1}\rangle_{V_i}$ by Equation 3.4, so it agrees with Equation 3.6 in that limit. Under `SNAP_X1_CENTROID_EXACT` the face difference is replaced by `x1_pressure_source`, the r^2-weighted moment of a quintic through six faces (`spherical_polar.cpp:246-264`); chapter 5 derives it.

For m_2 the source has a curvature part $\langle r^{-1}\rangle\langle\cot\theta\rangle(\rho v_3^2 + p)$, applied as written, and a part $-\rho v_1 v_2/r$ that the code puts in angular-momentum form. Combined with the x_1 flux divergence of m_2, the target form is

$$\begin{aligned}\frac{A_{i+1/2}f^{(2)}_{i+1/2} - A_{i-1/2}f^{(2)}_{i-1/2}}{V_i} + c_i\left(A_{i-1/2}f^{(2)}_{i-1/2} + A_{i+1/2}f^{(2)}_{i+1/2}\right)\\ = \frac{r_+ A_{i+1/2}f^{(2)}_{i+1/2} - r_- A_{i-1/2}f^{(2)}_{i-1/2}}{x^{\mathrm{m}}_{1,i}\,V_i},\end{aligned} \tag{3.8}$$

with $c_i = h_i/[(r_- + r_+)V_i]$. The identity follows from $1 \pm h_i/(r_- + r_+) = 2r_\pm/(r_- + r_+)$. The right side is a difference of face quantities divided by $x^{\mathrm{m}}_{1,i}V_i$, so $\sum_i x^{\mathrm{m}}_{1,i}V_i m_{2,i}$ changes only through the domain-end faces: the x_1 fluxes conserve this discrete angular momentum. The same form is used for m_3, and for m_3 along x_2 with $\sin\theta$ in place of r. The code coefficient is `coord_src2_i` (`spherical_polar.cpp:92-94`), and the term is added to the m_2 and m_3 rows through `face_area1` (`spherical_polar.cpp:281-307`); the Limits layer records how the code's area factor differs from Equation 3.8.

What is exact: the measures, the centroids, the coefficient averages and the telescoping identity. What is truncated: the product averages in Equation 3.6, at $O(h^2)$. The coefficient averages, the centroid and the two

source identities are checked as lines `[C4]` and `[C5]` of `src/snapy_report/ch03/geometry_check.out`; the tests cited below check the measures, the centroids, the coefficients and the face-pressure form in the code.

3.3.3. Numerical method

`x1v` and `x2v` hold the centroids of Equation 3.5, `x3v` the midpoint; the widths `dx1f`, `dx2f`, `dx3f` are face differences of the sliced faces, ghosts included, and `dx1v`, `dx2v`, `dx3v` centroid spacings. The transverse lengths used by diffusion and the curl, `center_width2` $= r\,\Delta\theta$ and `center_width3` $= r\sin\theta\,\Delta\varphi$, are taken at the centroids. All are built once in `reset`; a mesh generator triggers a rebuild from the moved faces. The sources run in every stage, in `forward`, after the flux divergence and before the stage update, and only on interior cells for the face-flux terms; the point-value terms use the primitives of the whole block. Figure 3.3 shows where the three radial positions of a cell lie.

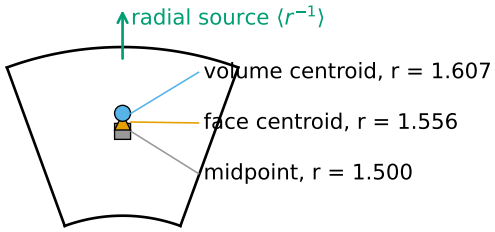


Figure 3.3.: A spherical-polar shell cell between r_- and r_+ with its midpoint $x^{\mathrm{m}}_{1,i}$, its volume centroid $x_{1,i}$ (where the cell value is stored) and the face-measure centroid of its x_2 and x_3 faces. The arrow marks the outward radial source.

With one x_2 cell (`nx2` $= 1$) there is no x_2 flux; the m_3 source then uses the point form $\langle r^{-1}\rangle\langle\cot\theta\rangle\rho v_3 v_2$ instead of the x_2 angular-momentum form.

The shallow-water equation of state drops the pressure terms from the m_1 and m_2 sources. The face pressures `_face_pressure1` exist whenever a block has more than one x_1 cell (`hydro.cpp:160-168`), so in a hydro run the face-pressure form Equation 3.7 is the one that runs and Equation 3.6 keeps its pressure term only on blocks with a single x_1 cell.

3.3.4. Code

Table 3.6.: Code map of the spherical-polar metric.

what	code link	symbol	switch
radial and polar centroids	`spherical_polar.cpp:17-28`	`radial_centers`	none
setup check, widths, coefficients	`spherical_polar.cpp:42-109`	`reset`	`type: spherical-polar`
transverse lengths	`spherical_polar.cpp: 158-176`	`center_width2`	none
face areas and volume	`spherical_polar.cpp: 178-211`	`cell_volume`	none
radial face moments	`spherical_polar.cpp: 213-222`	`face_moment2_x1`	`SNAP_FLUX_COVARIANCE`
geometric sources	`spherical_polar.cpp: 224-310`	`forward`	`SNAP_X1_CENTROID_EXACT`
mesh-generator rebuild	`spherical_polar.cpp: 111-156`	`reset_coordinates`	none

In step order (Table 3.6):

1. `reset` refuses an x_2 range outside $[0, \pi]$ with "Spherical-polar coordinates require x2 in [0, pi]" when there is more than one x_2 cell (`spherical_polar.cpp:42-48`).
2. It zeroes the cosine buffers and builds the widths and, with `radial_centers`, the centroids (`spherical_polar.cpp:52-76`), then the five coefficients `coord_src1_i` to `coord_src3_j` (`spherical_polar.cpp:78-104`).
3. In each stage `forward` calls the base divergence, then forms `src1` with one of three pressure forms: the quintic moment when `SNAP_X1_CENTROID_EXACT` is set and its stencil is usable, the face-pressure form when face pressures are given, else $2p\langle r^{-1}\rangle$ (`spherical_polar.cpp:243-279`).
4. It adds the m_2 curvature term and the x_1 angular-momentum terms of m_2 and m_3, then the x_2 angular-momentum term of m_3 or, with one x_2 cell, its point form, as `use_x2_fluxes` selects (`spherical_polar.cpp:281-307`).

3.3.5. Tests

Table 3.7.: Tests of the spherical-polar metric. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_coordinate`	centroids, areas, volume and the five coefficients equal reference formulas	10^{-12} rel. and abs.	round-off of the closed forms

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_coordinate`	with face pressures the m_1 row ignores the cell pressure	10^{-6} rel. and abs.	set in the test
`test_coordinate`	the m_1 row is the plain face-pressure difference	10^{-6} rel. and abs.	set in the test
`test_x1_centroid_rest_python`	a column at rest stays at rest under `SNAP_X1_CENTROID_EXACT`	`TOL_ON` 10^{-10}	chapter 5

The reference-formula case is `geometry_matches_athena_reference_formulas` (`test_coordinate.cpp:146-221`); it compares `x1v`, `x2v`, the three areas, the volume and `coord_src1_i` to `coord_src3_j` with `torch::allclose`. The face-pressure cases are `spherical_polar_radial_source_preserves_face_pressure_gradient` and `spherical_polar_radial_source_uses_face_pressure_in_x1_momentum` (`test_coordinate.cpp:645-719`); the first sets the face pressure to the face radius and checks that the m_1 row equals $(p_{i+1/2} - p_{i-1/2})/h_i$, the second checks that two cell pressures give the same m_1 row once face pressures are given. The face-pressure cases run on CPU and, through the `DeviceTest` parameter, on CUDA when it is built; the reference-formula case runs on CPU only. All are one block, one rank. No case checks the m_2 and m_3 source terms. Their pass status at the pin was not run for this chapter (missing evidence).

3.3.6. Limits

> **Limits**
>
> - The m_2 and m_3 angular-momentum terms are untested: no test at the pin checks them (read from the tests).
> - `nx2` $= 1$ uses a midpoint for `x2v`, not the polar centroid.

A hypothesis from reading the code, not measured: the angular-momentum terms of `forward` multiply `coord_src2_i`, which divides by the radial factor $V_i^{(1)}$ only, by the full face area `face_area1` $= r_\pm^2 \varpi_{jk}$ (`spherical_polar.cpp:286-296`). Equation 3.8 needs $r_\pm^2$ there, so as written the term carries an extra factor of the cell's solid angle ϖ_{jk}. The x_2 term multiplies `coord_src1_i` and `coord_src2_j` by the full `face_area2` $= \frac{1}{2}(r_+^2 - r_-^2)|\sin\theta_\pm|\Delta\varphi$ where the $\sin\theta$ form needs $|\sin\theta_\pm|$ alone (`spherical_polar.cpp:297-303`). If this reading is right, the $-\rho v_1 v_2/r$ and $-\rho v_1 v_3/r$ terms are scaled by ϖ_{jk} and the x_2 term by a dimensional factor, in every multidimensional spherical-polar run with tangential velocity; a radial column with $v_2 = v_3 = 0$ does not see it. This needs a failing test before any change (the editor tracks it as an open question).

The `x2` refusal is skipped when there is one x_2 cell. The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.4. Gnomonic equiangular cubed sphere (`gnomonic-equiangle`)

Local symbols (flagged to the editor): $X = \tan\xi$, $Y = \tan\eta$ [-]; $C = \sqrt{1+X^2}$, $D = \sqrt{1+Y^2}$, $\delta = \sqrt{1+X^2+Y^2}$ [-]; ψ the angle between

the ξ and η coordinate lines on the sphere [rad] (the code calls its cosine `cosine_cell_kj`); $\hat{\mathbf{u}}$ the unit position vector on the sphere [-].

3.4.1. Summary

> **i** At a glance
>
> **What**: the six-panel gnomonic equiangular grid (r, ξ, η) with exact solid angles, exact volumes and great-circle widths, a non-orthogonal velocity frame, and the curvature sources. **Switch**: YAML `geometry.type: gnomonic-equiangle`, with `distribute.layout: cubed-sphere`. **Default**: off. **Requires**: `nx2 == nx3`, `interp_order: 2`. **Order**: exact measures; second-order sources. **Defining equation**: Equation 3.10.

Each panel of the cubed sphere covers $\xi, \eta \in [-\pi/4, \pi/4]$; a point of the panel is the unit vector $\hat{\mathbf{u}} = (1, X, Y)/\delta$ in the panel's own frame. The coordinate lines meet at an angle ψ that is $\pi/2$ on the panel centre lines and $\pi/3$ or $2\pi/3$ at the corners, so velocities are carried as components along the two unit tangent vectors with metric $g_{23} = \cos\psi$, and each Riemann problem is solved in a local orthonormal frame. Every measure is exact: the x_1 face area is r^2 times the exact solid angle of the cell, the volume is $\frac{1}{3}(r_+^3 - r_-^3)$ times the same solid angle, and the x_2, x_3 face areas are planar sectors. The curvature sources are built so that a uniform pressure at rest is in exact discrete balance.

The type is on when `geometry.type` is `gnomonic-equiangle`. Unlike spherical-polar, the radial cell centre is the midpoint, not the volume centroid. The corrected-PE gravity work of chapter 6 has no form here (Section 6.1). The geometry was set up in the Mathematica notebook `cubed_sphere6.nb`, which is symbolic and is not a prose derivation.

3.4.2. Derivation

Angle between the coordinate lines. With $X = \tan\xi$, $\partial X/\partial\xi = C^2$. Differentiating $\hat{\mathbf{u}} = (1, X, Y)/\delta$ gives

$$\frac{\partial\hat{\mathbf{u}}}{\partial X} = \frac{(-X,\ D^2,\ -XY)}{\delta^3}, \qquad \frac{\partial\hat{\mathbf{u}}}{\partial Y} = \frac{(-Y,\ -XY,\ C^2)}{\delta^3},$$

with lengths D/δ^2 and C/δ^2 (using $C^2D^2 = \delta^2 + X^2Y^2$) and dot product $-XY/\delta^4$. Hence

$$\cos\psi = -\frac{XY}{CD}, \qquad \sin\psi = \frac{\delta}{CD}. \tag{3.9}$$

Equation 3.9 is evaluated at cell centres and at x_2 and x_3 faces as `cosine_cell_kj` and `sine_cell_kj` (`gnomonic_equiangle.cpp:70-94`). $\cos\psi$ vanishes on the centre lines $X = 0$ or $Y = 0$ and is $-\frac{1}{2}$ at the panel corners, where $X^2 = Y^2 = 1$.

Solid angle. The area element of the unit sphere is $|\partial_\xi\hat{\mathbf{u}} \times \partial_\eta\hat{\mathbf{u}}|\,\mathrm{d}\xi\,\mathrm{d}\eta = \sin\psi\, C^2\tfrac{D}{\delta^2}\, D^2\tfrac{C}{\delta^2}\,\mathrm{d}\xi\,\mathrm{d}\eta = \mathrm{d}X\,\mathrm{d}Y/\delta^3$. The function $\arctan(XY/\delta)$ has mixed derivative δ^{-3}, so the solid angle of the cell $[X_{j-1/2}, X_{j+1/2}] \times [Y_{k-1/2}, Y_{k+1/2}]$ is the corner sum

$$\varpi_{jk} = \sum_{a,b\in\{\pm\}} ab\,\arctan\frac{X_{j+a/2}\,Y_{k+b/2}}{\sqrt{1 + X_{j+a/2}^2 + Y_{k+b/2}^2}}. \tag{3.10}$$

Equation 3.10 is `solid_angle_kj` (`gnomonic_equiangle.cpp:124-137`); over a panel the corner sums telescope to $4\arctan(1/\sqrt{3}) = 2\pi/3$, and six panels give 4π.

Measures. With ϖ_{jk} the volume and the x_1 face area are exact:

$$A_{i\pm1/2} = r_{\pm}^2\,\varpi_{jk},\qquad V_i = \tfrac{1}{3}h_i\,(r_+^2 + r_+r_- + r_-^2)\,\varpi_{jk} = \tfrac{1}{3}\,(r_+^3 - r_-^3)\,\varpi_{jk}, \tag{3.11}$$

written in `cell_volume` without the cancelling difference of cubes (`gnomonic_equiangle.cpp:207-227`). An x_2 face lies in the plane through the centre that contains the line $X = X_{j\pm1/2}$, so it is a planar annular sector: its area is $\int r\,\mathrm{d}r = \frac{1}{2}(r_+^2 - r_-^2)$ times the angle it subtends, the great-circle angle between its two η edges. That angle between the unit vectors $\hat{\mathbf{u}}(X, Y_1)$ and $\hat{\mathbf{u}}(X, Y_2)$ is $\arccos[(1 + X^2 + Y_1Y_2)/(\delta_1\delta_2)]$, which is `dx3f_ang_face2_kj` (`gnomonic_equiangle.cpp:110-122`); the code forms $\frac{1}{2}(r_+^2 - r_-^2)$ as `x1v` times `dx1f`, exact because `x1v` is the midpoint. The x_3 faces are the same with X and Y exchanged.

Velocity frame. Velocity components v_2, v_3 are taken along the unit tangent vectors, so $g_{22} = g_{33} = 1$, $g_{23} = \cos\psi$ and $|\mathbf{v}|^2 = v_1^2 + v_2^2 + v_3^2 + 2\cos\psi\, v_2v_3$. Lowering and raising are

$$\underline{v_2} = v_2 + v_3\cos\psi, \qquad \underline{v_3} = v_3 + v_2\cos\psi,$$
$$v_2 = \frac{\underline{v_2} - \underline{v_3}\cos\psi}{\sin^2\psi}, \qquad v_3 = \frac{\underline{v_3} - \underline{v_2}\cos\psi}{\sin^2\psi}, \tag{3.12}$$

with the underline marking a covariant component (local notation). Equation 3.12 is `coord_vec_lower_impl` and `coord_vec_raise_impl` (`coord_utils_impl.h:9-24`). The conserved momenta are covariant; the primitive velocities are contravariant. For an x_1 face the local orthonormal frame is $(v_1,\ v_2 + v_3\cos\psi,\ v_3\sin\psi)$, which keeps $|\mathbf{v}|^2$; the x_2 and x_3 faces use the same idea with the face values of ψ and the normal along the face normal.

Pressure balance. Take a uniform pressure p at rest. In the x_2 face frame the flux is p in the normal slot and zero in the tangential slot; transforming back gives the covariant m_2 flux $p\sin\psi_{j\pm1/2}$ and a zero m_3 flux, and the x_3 faces give a zero m_2 flux. The m_2 divergence is therefore

$\Delta_j[A\,p\sin\psi]/V_i$, and the code subtracts exactly that through the coefficient $\Delta_j[A\sin\psi]/V_i$, `x_ov_rD_kji` (`gnomonic_equiangle.cpp:139-148`). A uniform pressure at rest therefore produces no m_2 or m_3 tendency, to round-off. The velocity terms of the sources, $\rho v_3^2\sin^2\psi$ beside p and $-\rho v_1 v_{\underline{2}}/r$, and the centripetal radial term $\rho(v_2 v_{\underline{2}}+v_3 v_{\underline{3}})/r = \rho|\mathbf{v}_h|^2/r$, follow the notebook's flux-form metric terms; they are stated from the code and are not re-derived in this draft (marked re-derive). In the m_1 equation the face-pressure form of Section 3.3 is used whenever face pressures exist.

What is exact: Equation 3.9 to Equation 3.12 and the uniform-pressure balance. What is truncated: the products of cell values in the velocity source terms, at $O(h^2)$. The identities are checked in `src/snapy_report/ch03/geometry_check.py` (lines `[C6]` to `[C9]` of `geometry_check.out`).

3.4.3. Numerical method

The ξ, η widths are the global uniform widths; `x1v`, `x2v`, `x3v` are midpoints. `cosine_*` and `sine_*` live at cell centres (`_cell_kj`) and on the x_2 and x_3 faces (`_face2_kj`, `_face3_kj`); the great-circle widths `dx2f_ang_kj`, `dx3f_ang_kj` at centres and on faces; `solid_angle_kj` per cell. All are buffers built once in `reset`, constant along x_1. In each stage the Riemann solvers call `prim2local1_` to `prim2local3_` on the face states and `flux2global1_` to `flux2global3_` on the fluxes, and `forward` adds the sources after the divergence. Wall boundary functions rotate ghost velocities into the wall's orthonormal frame with `boundary_velocity_` and back. Figure 3.4 shows $\cos\psi$ over one panel.

3.4.4. Code

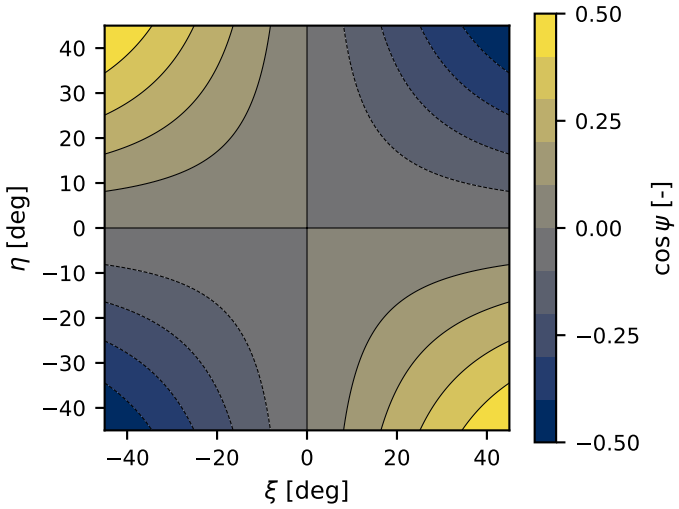


Figure 3.4.: The cosine of the angle between the coordinate lines, $\cos\psi = -XY/(CD)$, over one gnomonic panel. It is zero on the two centre lines and reaches $\pm 1/2$ at the corners, which is where non-orthogonality enters the face states and fluxes.

Table 3.8.: Code map of the gnomonic grid.

what	code link	symbol	switch
setup checks	`gnomonic_equiangle.cpp:18-29`	`reset`	`type: gnomonic-equiangle`
angle and great-circle widths	`gnomonic_equiangle.cpp:69-122`	`cosine_cell_kj`	none
solid angle	`gnomonic_equiangle.cpp:124-137`	`solid_angle_kj`	none
pressure-balance coefficients	`gnomonic_equiangle.cpp:139-148`	`x_ov_rD_kji`	none
areas and volume	`gnomonic_equiangle.cpp:207-227`	`face_area2`	none
face metrics (marked TODO)	`gnomonic_equiangle.cpp:265-293`	`_set_face2_metric`	none
local frames	`gnomonic_equiangle.cpp:295-341`	`prim2local2_`	none
back to covariant	`gnomonic_equiangle.cpp:343-413`	`flux2global2_`	none
sources	`gnomonic_equiangle.cpp:415-473`	`forward`	none
lower and raise	`coord_utils.cpp:12-44`	`coord_vec_lower_`	none

what	code link	symbol	switch
wall frame	`coordinate.cpp: 611-637`	`boundary_ velocity_`	none
panel to spherical	`cubed_sphere_ utils.cpp: 245-257`	`cs_contra_to_ sph_`	none

In step order (Table 3.8):

1. `reset` requires `nx2 == nx3` ("nx2 must equal nx3") and `interp_ order` to be 2 ("Only 2nd order ghost interpolation is supported") (`gnomonic_equiangle.cpp:18-29`).
2. It builds uniform widths and midpoints, the longitude and latitude of the cell centres for output with `cs_ab_to_lonlat` (`gnomonic_equiangle.cpp:59-67`), then the angle, the great-circle widths, the solid angle and `x_ov_rD_kji`, `y_ov_rC_kji` (`gnomonic_equiangle.cpp:69-148`).
3. It builds the ghost interpolation abscissae of Section 3.9 and refuses `interp_order > 2` a second time with `TORCH_CHECK` (`gnomonic_ equiangle.cpp:162-196`).
4. Per stage, `prim2local2_` sets the face metric with `_set_face2_ metric` and projects the face states (`gnomonic_equiangle.cpp: 303-321`); after the solve, `flux2global2_` returns the fluxes to covariant components (`gnomonic_equiangle.cpp:357-384`); likewise for x_1 and x_3.
5. `forward` adds the radial, m_2 and m_3 sources (`gnomonic_ equiangle.cpp:415-473`).

3.4.5. Tests

Table 3.9.: Tests of the gnomonic grid. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_cubed_sphere_cell_volume_python`	six panels sum to $\frac{4}{3}\pi(r_o^3 - r_i^3)$; discrete $\nabla \cdot (r\hat{\mathbf{r}}) = 3$	10^{-12}	`ROUNDOFF` in the script
`test_cubed_sphere_cell_volume_python`	a balanced rest state stays at rest for 20 cycles	$\max\|v\|/c_s < 10^{-6}$	`REST_TOL` in the script
`test_coordinate`	lower then raise returns the input	`allclose` default	float32 and float64
`test_coordinate`	panel, Cartesian and spherical velocity transforms round-trip	10^{-4} rel., 10^{-5} abs.	float32 cases
`test_coordinate`	`prim2local` then `flux2global` then raise is the identity, three faces	`allclose` default	float32 and float64
`test_coordinate`	with face pressures the m_1 row is the face difference	10^{-6}	set in the test

The script checks are in `geometry`, which forms the sum of `cell_volume` and the divergence of $r\hat{\mathbf{r}}$ (`test_cubed_sphere_cell_volume.py:51-65`), and in `main`, which applies `ROUNDOFF` and `REST_TOL` (`test_cubed_sphere_cell_volume.py:126-137`); the deck is `test_cubed_sphere_cell_volume.yaml`, six panels of $8 \times 8 \times 8$ cells in one process. The transform cases are `vec_lower_raise` to `uniform_radial_spherical_velocity_round_trips` (`test_coordinate.cpp:234-412`), the projection cases `flux_projection1` to `flux_projection3` (`test_coordinate.cpp:507-577`) and the face-pressure cases `radial_source_uses_face_pressure_in_x1_momentum` and `radial_source_preserves_face_pressure_gradient` (`test_coordinate.cpp:579-643`). The `DeviceTest` cases run on CPU and CUDA in float32 and float64, as the fixture instantiates them (`device_testing.hpp:44-50`); the Python test also has no CUDA twin at the pin. All are single-rank. Their pass status at the pin was not run for this chapter (missing evidence).

3.4.6. Limits

> **Limits**
>
> - `nx2` must equal `nx3`, and `interp_order` must be 2.
> - The corrected-PE face work is not available on this grid; setup warns with `TORCH_WARN_ONCE` (`hydro.cpp:82-93`).

`DeviceTest.usrc` and `GnomonicEquiangle.area_vol` print values and assert nothing about the geometry, as in `area_vol` (`test_coordinate.cpp:123-145`). `_set_face2_metric` and `_set_face3_metric` carry the comment "TODO(cli):: CHECK" (`gnomonic_equiangle.cpp:265-293`); the round-trip tests above exercise them only with the projection followed by its inverse, which a consistently wrong metric would also pass. The radial centre `x1v` is the midpoint, so a cell value is not stored at the volume centroid as on spherical-polar; the curved-face corrections of chapter 4 rely on `x1v` times `dx1f` being

$\frac{1}{2}(r_+^2 - r_-^2)$. On a block with a single x_1 cell there are no face pressures and the radial pressure source is $2p/r$ at the midpoint, which differs from $\Delta_i[A]p/V_i = 2p\langle r^{-1}\rangle_{V_i}$ by the relative factor $1/(1 + h_i^2/(12(x_{1,i}^{\mathrm{m}})^2))$ (algebra from Equation 3.11; not measured). The velocity source terms are not re-derived here. The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.5. Radial face moments and the face-centroid shift

Local symbols (flagged to the editor): $x_{1,i}^{\mathrm{A}}$ the x_1 centroid of an x_2 or x_3 face of cell i on the face's own measure [m]; $(\sigma_i^{\mathrm{A}})^2$ the second central moment of x_1 on that measure [m^2]; $\delta_i^{\mathrm{A}} = x_{1,i} - x_{1,i}^{\mathrm{A}}$ the face-centroid shift [m]; I_n the integral $\int_{r_-}^{r_+} r^n \,\mathrm{d}r$ [m^{n+1}].

3.5.1. Summary

> **i** At a glance
>
> **What**: the exact x_1 second moment of an x_2/x_3 face and the exact distance between a cell's volume centroid and that face's centroid, in a cancellation-free form. **Switch**: read only when `SNAP_FLUX_COVARIANCE` is set (chapter 12). **Default**: off (the helpers exist on every grid). **Requires**: nothing. **Order**: exact. **Defining equations**: Equation 3.13, Equation 3.14.

The flux through an x_2 or x_3 face is a face average, weighted by the face's own area measure. On a Cartesian grid that weight is uniform in x_1; on the spherical-polar and cubed-sphere grids an x_2/x_3 face spans $r\,\mathrm{d}r$ times an angular factor while the cell spans $r^2\,\mathrm{d}r$ times one. Two numbers follow from the face measure: its x_1 variance, which sets the size of the covariance of two fields over the face, and the offset between where a cell

average lives and where the face average lives. Both enter the $O(\Delta x_1^2)$ face-flux corrections of chapter 4. This scheme supplies them exactly; the flux use is chapter 4's. In the history it is part of chengcli/snapy#289 and its follow-up items.

3.5.2. Derivation

The derivation is in `289-covariance-x3-curved.md` §2.7 (the moment) and §2A.1 (the two centroids); it is rewritten here in the report's notation. The angular factor of an x_2/x_3 face does not vary along x_1 on one face, so it cancels in every average and the face weight is $w(r) = r$. With $h = h_i$ and $x^{\mathrm{m}} = x^{\mathrm{m}}_{1,i} = \frac{1}{2}(r_- + r_+)$,

$$x^{\mathrm{A}}_{1,i} = \frac{I_2}{I_1} = \frac{12(x^{\mathrm{m}})^2 + h^2}{12\,x^{\mathrm{m}}},$$
$$x_{1,i} = \frac{I_3}{I_2} = \frac{3x^{\mathrm{m}}\,(4(x^{\mathrm{m}})^2 + h^2)}{12(x^{\mathrm{m}})^2 + h^2},$$

the second being the spherical-polar volume centroid of Equation 3.5. The second central moment on the face measure is $(I_3 - 2x^{\mathrm{A}}I_2 + (x^{\mathrm{A}})^2 I_1)/I_1$, which simplifies to

$$(\sigma_i^{\mathrm{A}})^2 = \frac{h^2}{12}\left(1 - \frac{h^2}{12(x^{\mathrm{m}})^2}\right). \tag{3.13}$$

Equation 3.13 is `radial_face_moment2_` (`coordinate.cpp:446-459`). Subtracting the two centroids directly would cancel, because both are close to x^{m}; doing the algebra first gives the exact

$$\delta_i^{\mathrm{A}} = \frac{h^2\,(12(x^{\mathrm{m}})^2 - h^2)}{12\,x^{\mathrm{m}}\,(12(x^{\mathrm{m}})^2 + h^2)} = \frac{h^2}{12\,x^{\mathrm{m}}} + O\!\left(\frac{h^4}{(x^{\mathrm{m}})^3}\right). \tag{3.14}$$

Equation 3.14 is `radial_face_centroid_shift_` (`coordinate.cpp:467-488`). On a Cartesian grid the two measures are the same uniform weight, so the moment is $h^2/12$ and the shift is zero; these are the base-class values `face_moment2_x1` and `face_centroid_shift_x1` (`coordinate.cpp:442-465`). Both formulas are exact, not truncated; Equation 3.13 recovers $h^2/12$ as $h/x^{\mathrm{m}} \to 0$. The checks are lines `[C1]` to `[C3]` of `geometry_check.out`.

3.5.3. Numerical method

Both helpers return one value per x_1 cell, ghosts included, shaped like `dx1f`, from the face array alone. A face with $x^{\mathrm{m}} \leq h/2$ (an x_1 ghost that reaches $r \leq 0$) is degenerate: the moment's curvature correction and the shift are set to zero there, and the divisors are clamped to 10^{-300} so the masked branch never divides by zero. Spherical-polar and gnomonic grids override the virtual methods to call the radial forms; on the gnomonic grid the cell volume is the same r^2 integral times the solid angle (Section 3.4), so the same shift applies even though its `x1v` is the midpoint. They are evaluated when the covariance correction asks for them, in the stage, before the x_2/x_3 fluxes. Figure 3.5 plots both against h/x^{m}.

3.5.4. Code

Table 3.10.: Code map of the radial face moments.

what	code link	symbol	switch
interface and its derivation comment	`coordinate.hpp:242-291`	`radial_face_moment2_`	none
Cartesian values	`coordinate.cpp:442-465`	`face_centroid_shift_x1`	none

what	code link	symbol	switch
exact radial moment	`coordinate.cpp: 446-459`	`radial_face_ moment2_`	none
exact radial shift	`coordinate.cpp: 467-488`	`radial_face_ centroid_ shift_`	none
spherical-polar overrides	`spherical_ polar.cpp: 213-222`	`face_moment2_ x1`	`SNAP_FLUX_ COVARIANCE`
gnomonic overrides	`gnomonic_ equiangle.cpp: 229-243`	`face_ centroid_ shift_x1`	`SNAP_FLUX_ COVARIANCE`

In step order (Table 3.10):

1. The caller asks the coordinate for `face_moment2_x1` and `face_ centroid_shift_x1`; virtual dispatch picks the grid's form (`spherical_polar.cpp:213-222`).
2. `radial_face_moment2_` forms h and x^{m} from the faces and masks the degenerate cells with `torch::where` (`coordinate.cpp:446-459`).
3. `radial_face_centroid_shift_` evaluates the cancellation-free quotient and masks the same cells (`coordinate.cpp:467-488`).

3.5.5. Tests

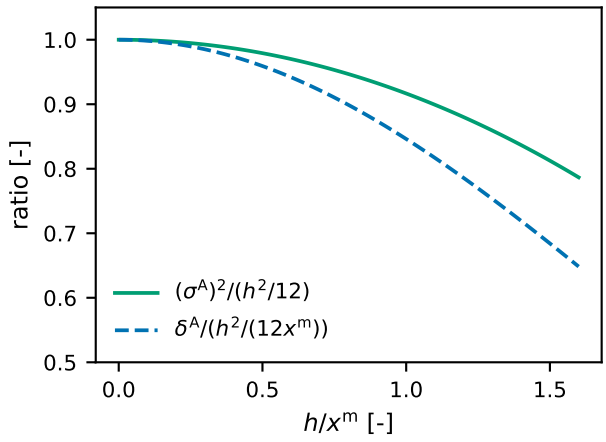


Figure 3.5.: The radial face moment divided by its Cartesian value $h^2/12$ (solid) and the face-centroid shift divided by its leading term $h^2/(12x^{\mathrm{m}})$ (dashed), against the relative cell width h/x^{m}. Both tend to 1 as the cell becomes thin; the moment is the Cartesian one to within $h^2/(12(x^{\mathrm{m}})^2)$.

Table 3.11.: Tests of the radial face moments. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_radial_face_moments`	the hand rationals equal the integrals	10^{-13} relative	double evaluation of the I_n
`test_radial_face_moments`	the closed forms equal the rationals	10^{-14} relative	a few operations of round-off
`test_radial_face_moments`	per-cell indexing on a four-cell column	10^{-14} abs., 10^{-12} rel.	as above, and the integrals' cancellation

The cases are `the_rationals_are_the_integrals`, `closed_forms_match_the_exact_integrals` and `one_value_per_cell_on_a_multi_cell_column` (`test_radial_face_moments.cpp:49-91`); the cells $[1.5, 2.5]$, $[3, 5]$, $[1, 2]$ have shift 47/1176, 47/588, 13/252 and moment 47/576, 47/144, 13/162, as listed in `kCells` (`test_radial_face_moments.cpp:30-34`). These rationals are reproduced exactly by line `[C2]` of `geometry_check.out`. The test runs on CPU, one process; the helpers are pure tensor functions with no device-specific path. Its pass status at the pin was not run for this chapter (missing evidence).

3.5.6. Limits

No known limits beyond the degenerate-face mask, which drops the curvature correction on ghost cells that reach $r \leq 0$. The helpers are used only

by SNAP_FLUX_COVARIANCE; their effect on the solution belongs to chapter 4. The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.6. Layouts and rank maps (distribute.layout)

Local symbols (flagged to the editor): p_x, p_y, p_z the number of blocks along x_2, x_3, x_1 (px, py, pz; count); (r_x, r_y, r_z) a block's logical location in the block grid (count); b the blocks per process (count).

3.6.1. Summary

> **i** At a glance
>
> **What**: cuts the global grid into a block grid and numbers the blocks: slab (a 2-D block grid in x_2, x_3), cubed (a 3-D block grid, the only one that splits x_1) and cubed-sphere (six panels of $p_x \times p_x$ blocks). **Switch**: YAML distribute: {layout, nb1, nb2, nb3, blocks_per_process}. **Default**: slab, one block per axis, one block per process. **Requires**: slab and cubed-sphere need $p_z = 1$; cubed-sphere needs $p_x = p_y$. **Order**: n/a. **Defining equation**: Equation 3.15.

A layout answers three questions for every block: where it sits in the block grid, which block is its neighbour in each of the up to 26 directions, and which process owns it. Blocks are ranked in Morton (Z) order inside the block grid or inside a panel, so that blocks close in space get close ranks; on the cubed sphere the ranks are face-major. Several blocks may share one process; a block's rank then splits into a process rank and a local index. Periodicity is not a layout key: it comes from the boundary names. How a decomposition is configured is also described with the configuration keys

in chapter 12, and how the layout object is built with the block in chapter 1.

3.6.2. Derivation

The rank maps are definitions, not approximations; the derivation states them. Within a block grid of $p_x \times p_y$ (or $p_x \times p_y \times p_z$) blocks, rank ρ is the ρ-th valid point of the Morton curve: the codes $0, 1, 2, \ldots$ are decoded with r_x the even bits and r_y the odd bits (three interleaved bit sets in 3-D), codes that fall outside the block grid are skipped, and the ρ-th surviving code is the location of rank ρ, in `build_zorder_coords2` (`connectivity.cpp:28-61`), with the decoder `morton_decode2` (`connectivity.hpp:67-70`). On the cubed sphere the global rank of the block at Morton rank ρ_f on face f is face-major, and with b blocks per process a block's rank splits as

$$\begin{aligned} \rho &= f\,p_x^2 + \rho_\mathrm{f}, \\ \rho &= b\,\pi + \lambda, \qquad 0 \le \lambda < b, \end{aligned} \tag{3.15}$$

with π the owning process and λ the local index (local notation). The first row is `_global_rank_from_face_local` (`cubed_sphere_layout.hpp:79-90`), the second `owner_process_rank`, `local_block_index` and `global_block_rank` (`layout.hpp:83-93`).

A neighbour inside the block grid is the location plus the offset, wrapped when the axis is periodic and absent (-1) otherwise. On the cubed sphere a step across a panel edge goes through the edge table `CS_FACE_EDGES`: leaving face f by side s arrives on face f' by side s', with the along-edge block index reversed when the table says so (`cubed_sphere_layout.cpp:304-339`). A corner step is two edge steps, ordered so that the first stays on the panel when it can in `neighbor_rank` (`cubed_sphere_layout.cpp:497-547`). Every map here is exact. The Morton order is checked as line `[C10]` of `geometry_check.out`.

3.6.3. Numerical method

The layout is set up once, when the block is built, and is not part of a stage. The YAML maps `nb2` to p_x (the x_2 direction), `nb3` to p_y (x_3) and `nb1` to p_z (x_1); `repartition` then gives each block its share of the global grid (Section 3.1). The number of blocks the processes hold together must equal $p_x p_y p_z$, six times that on the cubed sphere, or setup stops. Periodic flags are set from the boundary names: `periodic` on x_2 or x_3 sets the flag on every layout, `periodic` on x_1 only on `cubed`. Figure 3.6 draws the Morton order of a 4×4 block grid and the face numbering of the cube.

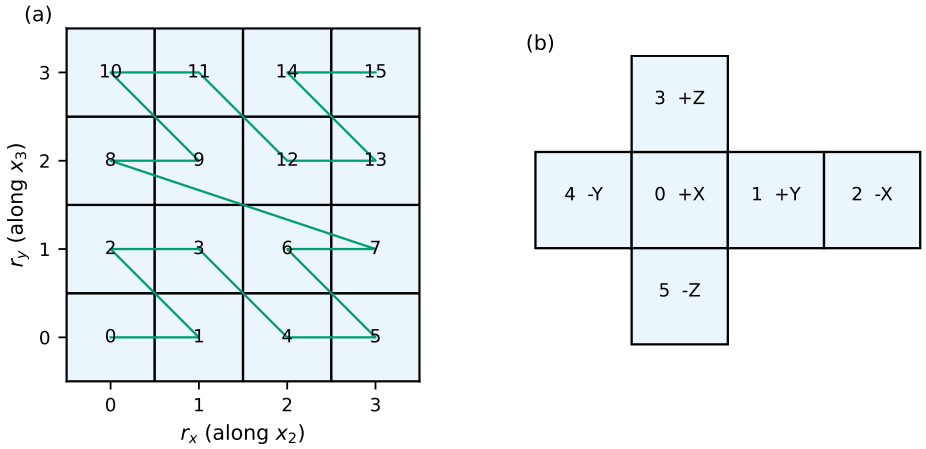


Figure 3.6.: (a) Rank order of a 4×4 block grid: Morton (Z) order, with r_x along x_2 and r_y along x_3; the line joins consecutive ranks. (b) The cubed-sphere face numbering 0 to 5 ($+X$, $+Y$, $-X$, $+Z$, $-Y$, $-Z$) as an unfolded net.

3.6.4. Code

Table 3.12.: Code map of the layouts.

what	code link	symbol	switch
options and block-to-process maps	`layout.hpp:52-131`	`blocks_per_process`	`distribute`
YAML card	`layout.cpp:221-244`	`from_yaml`	`layout`, `nb1`-`nb3`
layout type	`layout.cpp:246-269`	`create`	`layout`
slab	`slab_layout.cpp:10-68`	`neighbor_rank`	`layout: slab`
cubed	`cubed_layout.cpp:10-78`	`neighbor_rank`	`layout: cubed`
Morton order	`connectivity.cpp:28-61`	`build_zorder_coords3`	none
cubed-sphere setup	`cubed_sphere_layout.cpp:413-430`	`_initialize`	`layout: cubed-sphere`
edge stepping	`cubed_sphere_layout.cpp:445-479`	`_step_one`	none
periodic flags	`meshblock_options.cpp:98-157`	`periodic_x`	`boundary-condition`
blocks per process	`mesh.cpp:210-240`	`blocks_per_process`	`distribute`

In step order (Table 3.12):

1. `from_yaml` reads `layout` (default `slab`), `nb3` into `py`, `nb2` into `px`, `nb1` into `pz`; `backend` is an accepted but dead key, read from the `BACKEND` environment variable instead (`layout.cpp:221-244`).
2. `Mesh` reads `blocks_per_process` (default 1) and sets the world size to processes times blocks (`mesh.cpp:210-240`).

3. `slab` refuses a `pz` other than 1 with "pz must be 1 for slab layout" (`slab_layout.cpp:10-24`); `cubed-sphere` refuses a `pz` other than 1 and `px` unequal to `py` (`cubed_sphere_layout.cpp:413-430`); for any other layout name `create` stops with "Unsupported layout type" (`layout.cpp:246-269`).
4. The boundary names set `periodic_x`, `periodic_y` and, on `cubed`, `periodic_z` (`meshblock_options.cpp:98-157`).
5. Each exchange asks `neighbor_rank` for the peer of each direction (`cubed_sphere_layout.cpp:497-547`).

3.6.5. Tests

Table 3.13.: Tests of the layouts. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_exchange_decomp`	ghosts hold the expected neighbour, with local and remote neighbours both seen, for 1 process × 6 blocks, 6 × 1 and 2 × 3	value bounds per ghost	payloads chosen to identify the sender

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_mesh_exchange_python`	on a 2×2 slab in one process each ghost equals its neighbour's rank	exact	integer payloads

`test_exchange_decomp` runs `test_exchange` under `torchrun` on gloo in the three cases of `cases` (`run_exchange_decomp.py:54-71`); the comparison is `ghosts_match_expected` (`test_exchange.cpp:135-180`) on the cubed-sphere deck `test_exchange.yaml`. The slab case, `test_mesh_exchange.py`, sets four blocks with `set_local_horizontal_cells` (`test_mesh_exchange.py:7-46`). Both run on CPU; `test_exchange_decomp` uses up to six processes and runs alone. Their pass status at the pin was not run for this chapter (missing evidence).

3.6.6. Limits

> **Limits**
>
> - Only `cubed` splits x_1; the cubed sphere forces $p_z = 1$.
> - `distribute.backend` is accepted and ignored; the backend comes from `BACKEND`.

The Morton order of `cubed` in 3-D and the corner steps of the cubed sphere with $p_x > 1$ have no dedicated test (read from the tests; not measured). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.7. Physical and internal faces (`external`)

3.7.1. Summary

> **i** At a glance
>
> **What**: decides, per block face, whether its ghosts come from a boundary function (a physical face) or from a neighbour (an internal face): a face is physical exactly when a boundary function is installed on it. **Switch**: YAML `boundary-condition: {external: {x1-inner, ..., x3-outer}}`. **Default**: `reflecting` on every face. **Requires**: nothing. **Order**: n/a. **Defining rule**: stated in the Derivation layer.

Each named boundary condition becomes a boundary function (bfunc) per face. At construction, on the `slab` and `cubed` layouts, every face that has a neighbouring block, and every face of a periodic direction, has its bfunc removed; the exchange then fills its ghosts. The predicate `is_physical_boundary` simply asks whether a bfunc is installed. On the cubed sphere nothing is removed: panel edges carry the no-op `custom` bfuncs, which leave the ghosts to the cubed-sphere exchange, and that exchange does not consult the predicate. The boundary conditions themselves are chapter 11.

3.7.2. Derivation

There is nothing to derive; the scheme is a rule, stated here in full. On the `slab` and `cubed` layouts a face loses its bfunc when the block has a neighbouring block across it or when its axis is periodic; this is the reset of the internal `bfuncs` at construction (`meshblock.cpp:137-179`). Afterwards a face is physical exactly when a bfunc is installed on it, which

is all that `is_physical_boundary` tests (`meshblock_options.cpp:213-234`). For a multi-axis offset the first non-zero component in the order x_3, x_2, x_1 names the face, as `face_of` does in the same lines. On the cubed sphere no bfunc is removed.

3.7.3. Numerical method

The rule runs once, when the block is built. The generic exchange skips every physical face when it packs and unpacks (Section 3.8), so a physical face's ghosts are only ever written by its bfunc. A wall is a narrower notion: `is_wall_boundary` is true only for the whitelisted names `reflecting` and `fixed_temperature`, which diffusion uses, and `is_x1_wall` is a physical x_1 face that is not periodic, which the gravity work and the wall reconstructions use. On an axis with one cell no bfunc is installed, so its faces are never physical. Figure 3.7 shows a 2×2 slab decomposition.

3.7.4. Code

Table 3.14.: Code map of physical and internal faces.

what	code link	symbol	switch
names to bfuncs, default `reflecting`	`meshblock_options.cpp:98-137`	`bfuncs`	`boundary-condition`
internal faces lose their bfunc	`meshblock.cpp:137-179`	`bfuncs`	`distribute`
face of an offset; physical predicate	`meshblock_options.cpp:213-234`	`is_physical_boundary`	none
wall whitelist	`meshblock_options.cpp:243-272`	`is_wall_boundary`	none

what	code link	symbol	switch
the no-op panel-edge bfunc	`bc_func.cpp:7-8`	`custom_inner`	`custom`
x_1 wall	`hydro.cpp:204-209`	`is_x1_wall`	none
exchange skips physical faces	`layout.cpp:496-510`	`is_physical_boundary`	none

In step order (Table 3.14):

1. For each live axis the two names are read (default `reflecting`), suffixed `_inner` or `_outer` and looked up; an unknown name stops setup with "Boundary function … is not defined", from the lookup into `bfuncs` (`meshblock_options.cpp:98-137`).
2. On `slab` and `cubed`, the constructor removes the `bfuncs` of internal and periodic faces (`meshblock.cpp:137-179`); on `cubed-sphere` it removes none.
3. Every consumer then asks `is_physical_boundary`, `is_wall_boundary` or `is_x1_wall`.

3.7.5. Tests

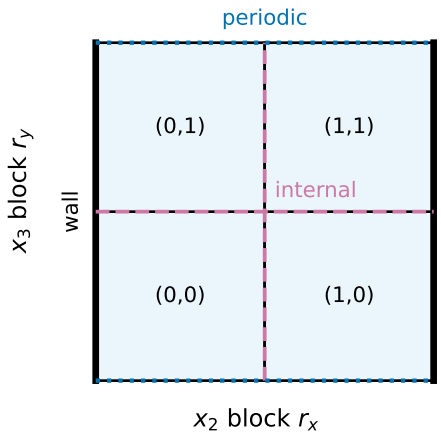


Figure 3.7.: A 2×2 slab decomposition with reflecting x_2 walls and periodic x_3. Thick black lines are physical faces (a boundary function is installed); dashed reddish-purple lines are internal faces between blocks; dotted lines are periodic faces, which are internal too and wrap around.

Table 3.15.: Tests that exercise the face classification. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_exchange_decomp`	internal-face ghosts come from the right neighbour	value bounds	Section 3.6
`test_wb_wall_corner`	a periodic x_2 edge and an x_2 split behave as internal faces	10^{-12}, exact	Section 3.14

No test targets the classification alone. The forcing deck records why it must have more than one x_1 cell: with `nx1: 1` no x_1 bfunc is installed, `is_physical_boundary` is false at the top and bottom, and every top or bottom forcing returns early, so a test of them passes on zeros (`test_forcing_cubed_sphere.yaml:1-11`). The cases above run on CPU; their pass status at the pin was not run for this chapter (missing evidence).

3.7.6. Limits

⚠ Limits

- On the cubed sphere `is_physical_boundary` is true at panel edges, because the `custom` bfunc is installed there.

Code that loops over every installed bfunc applies the no-op `custom` at panel edges, which is harmless; code that asks `is_physical_boundary` on x_2 or x_3 of a cubed sphere would classify a panel edge as physical

(inferred from the code; at the pin the only tangential callers are in the generic layout, which the cubed sphere does not use for its own exchange; not measured). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.8. Generic ghost exchange (`SyncOptions`)

Local symbols (flagged to the editor): $q^{\mathrm{c}}_{k,j}$ a synthesised corner ghost value; q^{L} and q^{B} the ghost strips left (x_2 side) and below (x_3 side) of a corner, with j_0, k_0 their first interior column and row next to the corner.

3.8.1. Summary

> **At a glance**
>
> **What**: for each neighbour across a face, copies the neighbour's interior slab of width n_g into this block's ghost slab, then synthesises each x_2-x_3 ghost corner as the mean of the two adjacent ghost strips. **Switch**: `SyncOptions` (`type`, `skip_corner`, `interpolate`, `dim`, phase flags). **Default**: conserved variables, corners skipped and synthesised, no interpolation. **Requires**: nothing. **Order**: exact copy; the corner is a convention. **Defining equation**: Equation 3.16.

Every ghost that is not at a physical face (Section 3.7) is filled by an exchange. A sync packs, for each direction with a neighbour, the interior slab next to that face into a send buffer, moves the buffers (chapter 14), and unpacks each received buffer into the ghost slab of the same direction. By default only face neighbours are exchanged; afterwards the four x_2-x_3 ghost corners are filled from the edge strips. The per-stage sync exchanges the conserved variables of the hydro and, when there are scalars, of the

scalar; Section 3.13 lists the other syncs. On the cubed sphere the same steps run with interpolation across panel edges (Section 3.9).

3.8.2. Derivation

The face exchange is a copy and needs no derivation. The corner is a convention, stated here. With the left ghost strip q^{L} (offset $\Delta x_2 = -1$) and the bottom strip q^{B} (offset $\Delta x_3 = -1$), the lower-left corner ghost at (k, j) is

$$q^{\mathrm{c}}_{k,j} = \tfrac{1}{2}\left(q^{\mathrm{L}}_{k_0,\,j} + q^{\mathrm{B}}_{k,\,j_0}\right), \tag{3.16}$$

that is, the mean of the left strip's value in the same x_2 column at the row next to the corner and the bottom strip's value in the same x_3 row at the column next to the corner. Equation 3.16 and its three mirror images are `fill_corners` (`layout.cpp:710-751`). It reproduces a constant field; for a field with a gradient it is a first-order fill, because both terms are read up to n_g cells away from the corner cell. It is not checked by an executable check; the tests below cover the face copies.

3.8.3. Numerical method

Slabs are addressed by the offset $(\Delta x_3, \Delta x_2, \Delta x_1) \in \{-1, 0, 1\}^3$; `get_buffer_id` maps an offset to one of 9 buffers (27 on `cubed`). The sent slab is the n_g-wide interior next to the face; the received slab is the ghost zone across it. With `skip_corner` only offsets with one non-zero component are exchanged. The `dim` option restricts a sync to one direction (used for face states, Section 3.10). Physical faces are skipped. `fill_corners` runs at the end of `finalize` when corners are skipped and the sync is not one phase of a split; for a split cubed-sphere sync it runs once after both phases. Only x_2-x_3 corners are synthesised; corners that involve x_1 are not. The per-stage sync runs at the start of `MeshBlock::forward`, before

reconstruction reads the ghosts. Figure 3.8 shows the slabs and buffer ids.

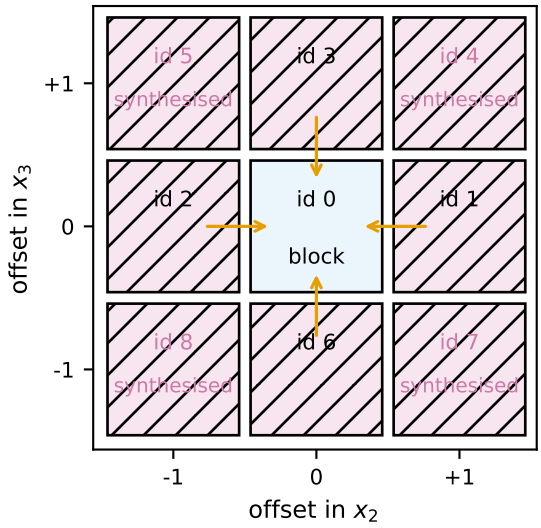


Figure 3.8.: The 3×3 neighbour stencil of one block in x_2-x_3, with the buffer id of each offset. Face neighbours (orange arrows) send their interior slabs into this block's hatched ghost slabs; the four corner ghost blocks (purple) are not exchanged but synthesised from the two adjacent strips.

3.8.4. Code

Table 3.16.: Code map of the generic exchange.

what	code link	symbol	switch
buffer id of an offset	`layout.hpp: 32-35`	`get_buffer_id`	none

what	code link	symbol	switch
sync options and defaults	`layout.hpp: 134-157`	`SyncOptions`	none
pack	`layout.cpp: 477-538`	`serialize`	`skip_corner`
unpack	`layout.cpp: 668-708`	`deserialize`	none
corners	`layout.cpp: 710-751`	`fill_corners`	`skip_corner`
wait, unpack, corners	`layout.cpp: 753-775`	`finalize`	none
one sync, or two phases	`meshblock.cpp: 554-595`	`exchange`	`interpolate`
per-stage sync	`meshblock.cpp: 913-930`	`exchange_ ghost_zones`	none

In step order (Table 3.16):

1. `exchange_ghost_zones` syncs `hydro_u` as conserved with interpolation on, then `scalar_s` as scalar (`meshblock.cpp:913-930`).
2. `exchange` takes the two-phase branch on a subdivided cubed sphere, otherwise calls `_exchange_once`, which is `begin_exchange`, `launch_ exchange`, `finalize_exchange` (`meshblock.cpp:554-595`).
3. `serialize` copies each interior slab into the cached send buffer of its `get_buffer_id` (`layout.cpp:477-538`).
4. `finalize` waits on the messages, calls `deserialize` and then `fill_ corners` (`layout.cpp:753-775`).

3.8.5. Tests

Table 3.17.: Tests of the generic exchange. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_mesh_exchange_python`	each ghost of a 2×2 slab in one process equals its neighbour's payload	exact	integer payloads
`test_cubed_sphere_vertical_velocity_exchange_python`	a uniform radial velocity crosses every panel edge unchanged	10^{-12} abs. $+10^{-12}$ rel.	round-off of the frame rotations
`test_exchange`	ghosts hold the neighbour's payload, two ranks	value bounds	payloads identify the sender

The slab check, in `test_mesh_exchange.py`, is the final `torch.all` comparison (`test_mesh_exchange.py:20-50`). The radial-velocity check sets `ABS_TOL` and `REL_TOL` (`test_cubed_sphere_vertical_velocity_exchange.py:7-9`) and compares the worst ghost error against them with `ideal_match` (`test_cubed_sphere_vertical_velocity_exchange.py:95-110`). The two-rank case uses `ghosts_match_expected` (`test_exchange.cpp:135-180`). The Python tests run on CPU in one process; `test_exchange.release` runs on two ranks with gloo. Their pass status at the pin was not run for this chapter (missing evidence). No test checks the synthesised corners.

3.8.6. Limits

> **Limits**
>
> - Corners that involve x_1 (blocks split in x_1 on `cubed`) are never exchanged or synthesised.

At a physical x_1 wall such corners are rewritten by the corner refresh (Section 3.14); at an internal x_1 seam they keep stale values (read from the code; not measured). No dimension-split stencil reads them; a cross-derivative operator could. The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.9. Cross-panel ghost interpolation (`interpolate`)

Local symbols (flagged to the editor): $\ell = 1..n_g$ the ghost layer counted from the panel edge (count); N_p the cells along a panel edge, $n_2 p_x$ (count); $\Delta\xi = \pi/(2N_\mathrm{p})$ the angular cell width [rad]; u the fractional source abscissa along the edge, in cells of the source panel [-]; η' the along-edge angle of a ghost centre on the source panel [rad].

3.9.1. Summary

> **At a glance**
>
> **What**: a ghost across a panel edge is a linear interpolation along the edge, at an abscissa precomputed once by mapping the ghost centre onto the neighbouring panel; velocities travel in the global spherical frame. **Switch**: `SyncOptions.interpolate(true)`, set by the per-stage conserved sync, the initial primitive sync and the scalar syncs.

Default: on in those syncs; off for face states and the donor factor. **Requires**: `interp_order: 2`; a subdivided panel ($p_x > 1$) runs the sync in two phases. **Order**: second (linear interpolation). **Defining equation**: Equation 3.18.

Across a panel edge the two panels' coordinate lines meet at an angle, so a ghost cannot be a copy of a neighbour's cell. snapy fills each ghost by one-dimensional linear interpolation along the shared edge, from a strip of the neighbour's cells. The source of a ghost in layer ℓ does not sit opposite it: it slides along the edge toward the edge midpoint by up to $n_g - \frac{1}{2}$ cells, independently of resolution. So the strip is widened by n_g cells on each side, and when a panel is split among blocks the widened strip reaches into the sender's own intra-panel halo, which must be current; the sync therefore runs in two phases, intra-panel first. Velocities are rotated from the sender's panel frame to the global spherical (r, θ, φ) frame before sending and back into the receiver's frame after interpolation. In the history: chengcli/snapy#222 (the decomposition fixes).

3.9.2. Derivation

The slide follows from the gnomonic map, as in `sources/canoe__cubedsphere_decomposition_TECH_REPORT.md` §1. Take the right edge of a panel, $\xi = \pi/4$. A ghost of layer ℓ sits at $\xi = \pi/4 + (\ell - \frac{1}{2})\Delta\xi$ and along-edge angle η; its direction is $(1, \tan\xi, \tan\eta)$ in the panel's frame. The neighbouring panel's frame is the same frame turned by $\pi/2$ about the edge, in which the same direction reads $(\tan\xi, -1, \tan\eta)$ up to sign, so its along-edge angle there is

$$\tan\eta' = \frac{\tan\eta}{\tan\xi}. \tag{3.17}$$

On the edge itself $\tan\xi = 1$ and $\eta' = \eta$. Outside it, with $\varepsilon = (\ell - \frac{1}{2})\Delta\xi$ and $\tan\xi = 1 + 2\varepsilon + O(\varepsilon^2)$, expanding gives $\eta' - \eta = -\varepsilon \sin 2\eta + O(\varepsilon^2)$: a slide

of $(\ell-\frac{1}{2})\sin 2\eta$ cells toward $\eta = 0$, at most $n_g-\frac{1}{2}$ cells, at the corners. The code does not use the expansion: `cs_build_ghost_usrc` maps each ghost centre through Cartesian coordinates onto the source panel and converts the angle to a cell abscissa $u = (\eta' + \pi/4)/\Delta\xi - \frac{1}{2}$, reversed when the edge table says so (`cubed_sphere_utils.cpp:78-108`). The ghost value is then

$$q_{\text{ghost}} = (1-w)\, q_{\lfloor u\rfloor} + w\, q_{\lfloor u\rfloor+1}, \qquad w = u - \lfloor u\rfloor, \tag{3.18}$$

with the cell index shifted by an integer per block. Equation 3.18 is the `interp_order == 2` branch of `_interp_ghost_LR` (`gnomonic_equiangle.cpp:475-514`) and of `_interp_ghost_BT` (`gnomonic_equiangle.cpp:516-554`). Linear interpolation is exact for linear data and has an $O(\Delta\xi^2)$ error otherwise. The margin is the largest slide rounded up, $\lceil n_g - \frac{1}{2}\rceil = n_g$, which is `cs_interp_margin` (`cubed_sphere_utils.hpp:95-108`). For $N_{\text{p}} = 64$ and $n_g = 3$ the exact slide from Equation 3.17 is at most 2.4955 cells and departs from the expansion by at most 0.072 cells (check line `[C11]` of `geometry_check.out`).

The ordering constraint is derived in the same source, §5 and §6: on a subdivided panel the widened strip packed for a cross-panel neighbour contains cells this block holds only as intra-panel ghosts, and those are written by the unpack of the same sync. One round cannot satisfy that, so the intra-panel round completes first.

3.9.3. Numerical method

The abscissae are built once in `reset` for the whole edge (face 0, left side, reused for every edge by symmetry) and kept as a double per ghost; the block's offset into the edge is kept as a separate integer and added to the index after `floor`, so the stored weight does not depend on the decomposition. A sync with interpolation on a cubed sphere with $p_x > 1$ or $p_y > 1$ runs as intra-panel (no interpolation), then cross-panel, then

`fill_corners`. In the cross-panel pack the strip is widened by the margin, its velocities are rotated to the spherical frame (conserved momenta are raised to contravariant first), and the strip is relabelled for the neighbour's orientation by the reverse, flip and transpose flags. In the unpack the wide strip lands in the ghost zone and its margin, the ghost is interpolated from it while all of it is still in the spherical frame, and only the narrow ghost zone is rotated into the receiver's frame (and lowered back to covariant for conserved mc

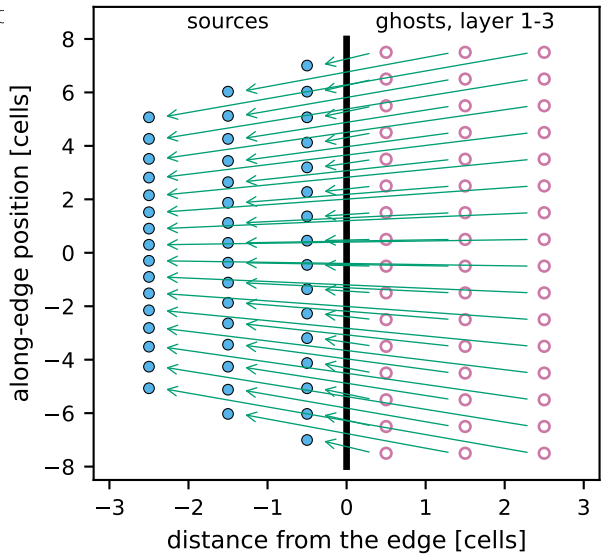


Figure 3.9.: Two panels meeting at an edge (thick line). Ghost centres of layers $\ell = 1, 2, 3$ beyond the edge (open circles) and the points of the neighbouring panel they are interpolated from (filled circles), joined by arrows: the sources slide along the edge toward its midpoint, most at the panel corners. Computed from Equation 3.17 for 16 cells per edge.

3.9.4. Code

Table 3.18.: Code map of the cross-panel interpolation.

what	code link	symbol	switch
ghost centre on the target panel	`cubed_sphere_utils.hpp:43-66`	`cs_equ_ghost_center`	none
margin	`cubed_sphere_utils.hpp:95-108`	`cs_interp_margin`	none
source abscissae	`cubed_sphere_utils.cpp:78-108`	`cs_build_ghost_usrc`	none
global abscissae, integer shift	`gnomonic_equiangle.cpp:162-196`	`usrc_shift_BT`	`interp_order`
interpolation	`gnomonic_equiangle.cpp:245-263`	`interp_ghost`	`interpolate`
two phases	`meshblock.cpp:554-572`	`intra_panel_only`	`interpolate`
pack: phase stop, margin, flags, rotation	`cubed_sphere_layout.cpp:625-793`	`trans_flag`	`type`
unpack: wide landing, interpolate, rotate	`cubed_sphere_layout.cpp:897-958`	`interp_ghost`	`type`
cached rotation matrix	`cubed_sphere_layout.cpp:122-152`	`_velocity_transform`	none

what	code link	symbol	switch
phase filter on messages	`cubed_sphere_layout.cpp:1007-1016`	`same_panel`	none

In step order (Table 3.18):

1. At setup `reset` calls `cs_build_ghost_usrc` for the whole edge and stores the block's slice with the integer shift `usrc_shift_LR` (`gnomonic_equiangle.cpp:162-196`).
2. `exchange` splits an interpolating sync on a subdivided panel into `intra_panel_only` and `cross_panel_only` rounds and calls `fill_corners` after both (`meshblock.cpp:554-572`).
3. `serialize` returns after the intra-panel neighbours in phase 1; in phase 2 it widens the strip with `extend_x2` or `extend_x3` (`cubed_sphere_layout.cpp:625-685`), sets `rev_flag`, `trans_flag` and `flip_flag` (`cubed_sphere_layout.cpp:700-705`), rotates the velocities with `cs_apply_velocity_transform_` (`cubed_sphere_layout.cpp:736-753`) and relabels the strip with `flip_out` (`cubed_sphere_layout.cpp:755-780`).
4. `deserialize` lands the wide strip with `sub_w`, calls `interp_ghost`, then rotates the narrow ghost zone back with `cs_apply_velocity_transform_` (`cubed_sphere_layout.cpp:897-958`).

3.9.5. Tests

Table 3.19.: Tests of the cross-panel interpolation. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_cubed_sphere_exchange`	faces, `dx2f` and the density after one RK stage are equal for nb2 = 1, 2, 4 in one process	bitwise	decomposition must not change a ghost
`test_cubed_sphere_exchange`	a communication tag that would collide is refused	exact	tag arithmetic
`test_exchange`	ghosts across panel edges carry the neighbour's payload, two ranks	value bounds	payloads identify the sender
`test_cubed_sphere_vertical_velocity_exchange_python`	a radial velocity crosses every edge unchanged	10^{-12}	Section 3.8

The bitwise case is `subdivided_panel_exchange_matches_one_block` (`test_cubed_sphere_exchange.cpp:159-179`); the deck is the card written by `write_card`, 8×8 cells per panel, `nb2` = 1, 2 and 4, all $6p_x^2$ blocks in one process (`test_cubed_sphere_exchange.cpp:20-40`). Its CUDA twin

`subdivided_panel_exchange_matches_one_block_cuda` is in the same binary and skips without a GPU (`test_cubed_sphere_exchange.cpp:181-200`). The tag case is `rejects_tags_that_collide_with_the_variable_offset` (`test_cubed_sphere_exchange.cpp:150-157`). Their pass status at the pin was not run for this chapter (missing evidence).

3.9.6. Limits

> **Limits**
>
> - `interp_order` must be 2; the fourth-order branch of `_interp_ghost_LR` and `_interp_ghost_BT` is unreachable.
> - Bitwise decomposition independence is tested for `nb2` = 1, 2, 4 in one process, one stage.

The source report records bit-for-bit agreement of `nb2` = 1, 2 and 4 over one cycle on CPU with gloo and one block per rank (`sources/canoe__cubedsphere_decomposition_TECH_REPORT.md` §7 and §10); that was measured on a development build, not at the pin, and re-running it at the pin needs the report's multi-rank gate, which is not committed. The margin cells of the wide landing zone keep the neighbour's spherical frame after the unpack; the same report shows with a gate that nothing reads them (§8), which was not re-checked at the pin. `_interpolate_to_local` is a stub with a TODO and no effect (`cubed_sphere_layout.cpp:1065-1088`). `DeviceTest.interpolate_LR` sets up linear data and asserts nothing, as written in `interpolate_LR` (`test_coordinate.cpp:424-506`). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.10. Cubed-sphere face-state seam sync

3.10.1. Summary

> **i** At a glance
>
> **What**: before the x_2 and x_3 Riemann solves, the reconstructed left and right states at panel-edge faces are exchanged as raw copies, one face deep, so that both panels solve the same Riemann problem at a shared face. **Switch**: none. **Default**: always on with `layout: cubed-sphere`. **Requires**: the direction-suffixed keys `*_wl:+` and `*_wr:-`. **Order**: exact copy. **Defining rule**: the suffix rule of the Derivation layer.

At a face between two panels, each panel reconstructs the state on its own side from its interior and the state on the other side from its interpolated ghosts. Without a sync the two panels would solve different Riemann problems for the same face and the fluxes would not cancel. snapy sends each panel's interior-side state to the neighbour, which overwrites its ghost-side state with it; then both hold the same pair. Because a panel edge can join two edges of the same orientation, where one panel's "left" is the other's "right" in the other panel's frame, the two states travel under keys whose suffix selects the direction, and the receiving block assigns the role in its own frame. The scalar once used a single key that swapped the states at those edges and made the solver pick the downwind state (chengcli/snapy#220).

3.10.2. Derivation

The argument is in `sources/canoe__tracer_seam_TECH_REPORT.md` §4 to §6; it is stated here against the code. In a block's own frame, at its last face along $+x_2$ the left state q^{L} comes from its interior and the right state

q^{R} from its ghosts; at its first face the reverse. The pack and unpack rules for a key with a suffix are:

- a key ending `:+` is sent only toward a direction with no negative offset and received only from a direction with no positive offset;
- a key ending `:-` is the mirror image.

The rules are the `suffix` tests in the cross-panel pack (`cubed_sphere_layout.cpp:710-719`) and the `suffix` tests in the unpack (`cubed_sphere_layout.cpp:924-932`). With q^{L} sent as `hydro_wl:+` and q^{R} as `hydro_wr:-`, a block sends from its last face only q^{L}, its interior state there, and from its first face only q^{R}, again its interior state. A neighbour that lies on the receiver's negative side delivers into the receiver's q^{L}; one on its positive side into q^{R}. Both rules are evaluated in the receiver's own frame, so at an edge where both panels see the other on their positive side the interior state of the sender lands in the receiver's q^{R}, which in the receiver's frame is the side the sender occupies. In every case the ghost-side state at a shared face is replaced by the neighbour's interior state, and the two blocks hold the same ordered pair. Nothing is approximated.

3.10.3. Numerical method

The sync runs inside each stage, after the x_2 and x_3 reconstruction and before the solves. Both directions are begun, launched and finalised together. The options are `cross_panel_only(true)`, `interpolate(false)` and `type(kPrimitive)`, with `dim` set to the direction, so only the one face layer at panel edges moves; the velocity frame rotation of Section 3.9 is applied to the states, the reverse flag reorders the strip along the edge, and the flip and transpose flags are no-ops on a strip one face deep. The scalar runs the same sync with `type(kScalar)` before its own solves. Figure 3.10 draws the roles at a same-orientation edge.

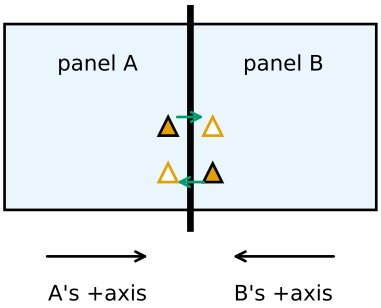


Figure 3.10.: Left and right states at a face shared by two panels whose edges have the same orientation. Each panel's interior-side state (filled triangle) is sent; each ghost-side state (open triangle) is overwritten with the neighbour's interior state, so both panels hold the same pair.

3.10.4. Code

Table 3.20.: Code map of the face-state seam sync.

what	code link	symbol	switch
hydro x_2, x_3 face-state sync	`hydro_forward.cpp: 542-587`	`send_vars2`	`layout: cubed-sphere`
scalar face-state sync	`scalar.cpp: 80-125`	`send_vars2`	`layout: cubed-sphere`
one-face strip	`cubed_sphere_layout.cpp: 668-674`	`part_opts`	`dim`
suffix rule, pack	`cubed_sphere_layout.cpp: 710-719`	`suffix`	none

what	code link	symbol	switch
suffix rule, unpack	`cubed_sphere_layout.cpp:924-932`	`suffix`	none

In step order (Table 3.20):

1. After `precon23` reconstructs the x_2 states, the hydro puts them in `send_vars2` under `hydro_wl:+` and `hydro_wr:-` and begins the exchange; the same for x_3 (`hydro_forward.cpp:542-570`).
2. It launches both and finalises both with `finalize_exchange` (`hydro_forward.cpp:572-587`); the solves follow.
3. The scalar does the same with `scalar_wl:+` and `scalar_wr:-` (`scalar.cpp:80-125`).

3.10.5. Tests

Table 3.21.: Tests that exercise the face-state sync. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_flux_covariance_seams_python`	six panels with the #289 term on conserve mass, vapour and $E + \mathrm{PE}$ over the run	10^{-12} relative drift	`DRIFT_TOL` in the script

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_flux_positivity_cubedsphere_python`	tracer mass is conserved across seams and the hat stays in $[0, 1]$	10^{-13} drift	`DRIFT_TOL`, Section 3.11

The conservation arm is `arm_conservation` and its threshold `DRIFT_TOL` (`test_flux_covariance_seams.py:34-93`); a shared face whose two panels solved different problems would leak and fail it. There is no dedicated test of the suffix rule itself. The tests run on CPU in one process. Their pass status at the pin was not run for this chapter (missing evidence).

3.10.6. Limits

No known limits. The transpose and flip flags are computed for these strips but have no effect on a strip one face deep (read from the code). The suffix rule has no direct unit test (not measured). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.11. Donor factor across seams: raw copy (`limiter`)

Local symbols (flagged to the editor): $\mathcal{F}_f$ the requested species flux through face f before limiting [kg m^{-2} s^{-1}]; $d(f)$ the donor cell of face f, the upwind cell the flux drains.

3.11.1. Summary

> **i** At a glance
>
> **What**: the positivity limiter's donor factor Θ is exchanged into the ghost cells as a raw, index-matched copy, never interpolated, so both blocks that share a face scale its flux by the donor's own factor. **Switch**: `dynamics.equation-of-state.limiter` with at least one species (hydro), and the same setting for scalars. **Default**: on whenever the limiter is on. **Requires**: nothing. **Order**: exact copy. **Defining equation**: Equation 3.19.

The tracer flux positivity limiter of chapter 8 computes, per cell and species, a factor $\Theta \in [0, 1]$ and scales every face flux that drains the cell by it. The limited flux of a face must be one number on both sides, or the mass removed from the donor is not the mass added to the receiver. At a block seam one side's donor is a ghost, so that ghost must hold the donor's own Θ. On a cubed sphere an interpolated ghost would be a blend of several cells' factors, not the donor's; the exchange therefore uses `interpolate(false)`. The species enthalpy that the withheld mass carries is recomputed from the ghost state instead of being exchanged. In the history: chengcli/snapy#220.

3.11.2. Derivation

The argument is `sources/canoe__theta_seam_TECH_REPORT.md` §1 and §6. With the limited flux of face f written as

$$\mathcal{F}_f^{\Theta} = \Theta_{d(f)}\, \mathcal{F}_f, \tag{3.19}$$

the two blocks sharing f each evaluate $\Theta_{d(f)}$, one from its interior and one from its ghost. If the ghost holds a different number $\tilde{\Theta}$, the face

carries $\Theta\mathcal{F}_f$ out of one cell and $\tilde{\Theta}\mathcal{F}_f$ into the other, and $(\Theta-\tilde{\Theta})\mathcal{F}_f A_f \Delta t$ of species mass is created or destroyed per stage. Equality needs the ghost to be the donor's value bit for bit. A raw copy delivers exactly the neighbour's interior cell that occupies the ghost position, after the same reverse, flip and transpose relabelling as every cubed-sphere strip; within a panel and across x_1 seams the ghost is the same cell anyway. The scaling Equation 3.19 is `flux_positivity_scale_`, called after the copy (`hydro_forward.cpp:671-698`). Nothing is approximated.

3.11.3. Numerical method

Θ is a cell quantity per species, computed in each stage after the x_1 seam fluxes are made single-valued (Section 3.12) and before the fluxes are scaled. Its ghosts are filled like conserved-variable ghosts: the exchange at internal faces (with corners synthesised as usual), then every installed bfunc in scalar mode at physical faces. The scalar module does the same for its tracers. Figure 3.11 shows the leak an interpolated factor would cause.

3.11.4. Code

Table 3.22.: Code map of the donor-factor copy.

what	code link	symbol	switch
limiter on, factor and census	`hydro_forward.cpp:648-669`	`flux_positivity_theta`	`limiter`
raw copy and bfunc fill (hydro)	`hydro_forward.cpp:671-688`	`interpolate`	none

what	code link	symbol	switch
raw copy and bfunc fill (scalar)	`scalar.cpp: 137-155`	`sync_theta`	`limiter`

In step order (Table 3.22):

1. With the limiter on and `ny` > 0, `flux_positivity_theta` computes Θ and its meters (`hydro_forward.cpp:648-669`).
2. `species_enthalpy` is recomputed from the primitives, ghosts included; Θ is exchanged with `topts` set to `interpolate(false)` and `type(kScalar)`, and each installed bfunc fills it at physical faces through `bfuncs` (`hydro_forward.cpp:671-688`).
3. The withheld mass keeps its energy and momentum with `flux_positivity_carry_`, and the fluxes are scaled with `flux_positivity_scale_` (`hydro_forward.cpp:690-698`).
4. The scalar does the same in `sync_theta` (`scalar.cpp:137-155`).

3.11.5. Tests

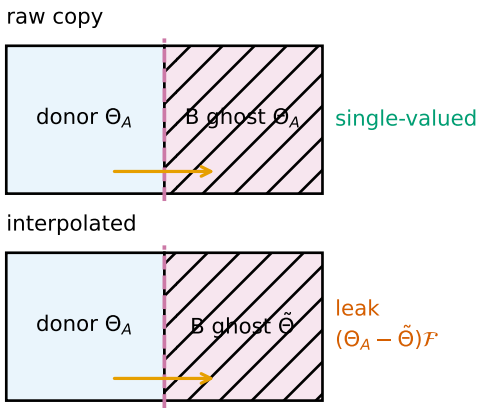


Figure 3.11.: A face on a panel seam with its donor cell on panel A. With a raw copy (top) panel B's ghost holds Θ_A and both sides scale the flux by it. With an interpolated ghost (bottom) panel B scales by a blend $\tilde{\Theta}$, and the difference $(\Theta_A - \tilde{\Theta})\mathcal{F}_f$ leaks at the face.

Table 3.23.: Tests of the donor-factor copy. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_flux_positivity_cubedsphere_python`	both arms conserve every tracer across the +X/+Y seam; the limiter fires; the hat stays in $[0, 1]$	10^{-13} drift; -10^{-15} to $1 + 10^{-5}$	`DRIFT_TOL` and bounds in the script
`test_flux_positivity_cubedsphere_moist_python`	the same with moist species	as above	in the script
`test_sedimentation_cubed_seam`	a two-rank x_1 seam with sedimentation and the limiter keeps mass	in the test	Section 3.12

The seam test places the downwind edge of a tracer hat on the +X/+Y seam so that Θ jumps there, and checks the bounds and `DRIFT_TOL` (`test_flux_positivity_cubedsphere.py:118-135`). Its docstring, on the donor factor `theta`, records a leak of 10^{-3} per 300 cycles with an interpolated fill (`test_flux_positivity_cubedsphere.py:1-11`); that number has no sha, deck or run attached (missing evidence). Both Python tests have CUDA twins (`_cuda_python`) registered when CUDA is built; the sedimentation test runs on two ranks. Their pass status at the pin was not run for this chapter (missing evidence).

3.11.6. Limits

The corner ghosts of Θ are synthesised by `fill_corners` (Section 3.8) and are not any cell's factor; a corner cell is never the donor of a face whose flux the block updates with (read from the code; not measured). The raw copy and the bfunc fill apply the no-op `custom` bfunc at panel edges (Section 3.7). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.12. x_1 seams between blocks (`layout: cubed`, `nb1` > 1)

Local symbols (flagged to the editor): s an x_1 seam face shared by the block below (B) and above (T); $\mathbf{F}_s^{\mathrm{B}}$, $\mathbf{F}_s^{\mathrm{T}}$ the x_1 fluxes the two blocks compute there; p^{top} the reference face pressure a block receives at its upper face.

3.12.1. Summary

> **i** At a glance
>
> **What**: makes a column split into several x_1 blocks behave like one block: the hydrostatic reference scan is relayed down the column, the reference ghost rows are the neighbour's interior rows, the switch-specific ghost rows are exchanged, and across process seams the seam flux is averaged. **Switch**: YAML `distribute: {layout: cubed, nb1}` with `nb1` > 1. **Default**: off (`nb1: 1`). **Requires**: `nghost` ≥ 3 under `SNAP_WB_REF4`; blocks of one process must run concurrently. **Order**: round-off agreement with one block, not bitwise. **Defining equation**: Equation 3.20.

Only the `cubed` layout splits x_1 (Section 3.6). The vertical schemes of chapter 5 are column operations: the hydrostatic reference is a scan from the top of the column, and several switches read windows that reach past a block edge. Four mechanisms make a split column match one block. (a) The reference scan is relayed from the top block down, each block passing its bottom face value to the block below. (b) The reference ghost rows (`pref`, `dref`) are overwritten with the neighbour's interior rows. (c) Under `SNAP_X1_CENTROID_EXACT` the plain-mean ghost cells and the two face pressures past the seam are the neighbour's. (d) Across process seams only, the seam-face flux, face pressure and reference flux are replaced by the mean of the two sides. Wall reconstructions and wall mirroring are gated on a physical face, so they never act at a seam; column-wide EOS repairs gather the whole column.

3.12.2. Derivation

Single-valued seam flux. Block B updates its top cell with $-A_s\mathbf{F}_s^{\mathrm{B}}/V$ and block T its bottom cell with $+A_s\mathbf{F}_s^{\mathrm{T}}/V$. The column total changes at the seam by $A_s(\mathbf{F}_s^{\mathrm{T}} - \mathbf{F}_s^{\mathrm{B}})\Delta t$ per stage, zero only if the two agree. Both blocks replace their value by

$$\bar{\mathbf{F}}_s = \tfrac{1}{2}\left(\mathbf{F}_s^{\mathrm{B}} + \mathbf{F}_s^{\mathrm{T}}\right), \tag{3.20}$$

and floating-point addition is commutative, so both hold the same bits and the sums telescope exactly across the seam. Equation 3.20 is the seam average in `hydro_forward.cpp`, applied to the flux slab and, packed with it, the face pressure and the reference flux `bflux1` (`hydro_forward.cpp: 425-499`). It is applied only across process seams; with mechanism (b) the two sides of an in-process seam already compute the same states, and the code comment states that a same-process seam is not averaged.

Reference relay. The default reference is a scan of face pressures down from an anchor at the top of the column, each cell adding its own hydrostatic drop. Split into blocks, each block needs as anchor the face value the scan has reached at its upper face. The top block anchors itself; each other block takes p^{top} from the block above and, after its own scan, passes its bottom face value down (`take_x1_anchor` and `pass_x1_anchor`, `hydro.cpp:518-579`). The relay is serial from top to bottom, so every block's anchor equals the value the one-block scan has at the same face, up to the order of the additions. The scan kernel itself is chapter 5's.

Ghost rows. A block's ghost rows of `pref`, `dref` computed locally from its ghost quadrature differ from the owner's interior values; the code comment records about 10^2 Pa near a wall. Replacing them by the neighbour's interior rows, sent under `kWbGhostUpTag` and `kWbGhostDnTag`, makes the perturbation field identical on both sides of the seam (`hydro.cpp:581-640`). Nothing in the derivation is truncated; the remaining differences from one block are round-off from the order of operations. The executable check of these identities is not separate from the tests below.

3.12.3. Numerical method

All four mechanisms run in every stage of a split column. The relay and the ghost-row exchange run inside `_hydro_ref_x1`, when the well-balanced reconstruction builds its reference; the plain-mean ghost rows before it; the seam average after the x_1 fluxes are computed and before the positivity limiter; the face-pressure ghost rows after the average. A same-process neighbour is reached through an in-memory board, a remote one through the process group (chapter 14); `_x1_ghost_rows` packs m rows from below the top face (up) and from above the bottom face (down) and copies the neighbour's rows into its ghosts. The message tags are 0x7715 (relay), 0x7717 and 0x7718 (reference ghost rows), 0x7720 and 0x7721 (seam flux), 0x7722 to 0x7725 (face-pressure and plain-mean ghost

rows). EOS column repairs call `gather_x1`, which joins the column's pieces bottom first. Figure 3.12 shows a column of three blocks.

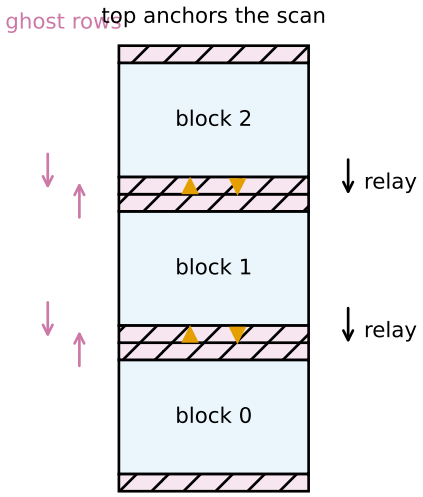


Figure 3.12.: A column split into three x_1 blocks. Black arrows: the reference relay, top block to bottom block, one face value per seam. Purple arrows: each block's interior rows copied into the neighbour's ghost rows. At each seam face the two one-sided fluxes (orange) are replaced by their mean when the seam is between processes.

3.12.4. Code

Table 3.24.: Code map of the x_1 seams.

what	code link	symbol	switch
WB4 needs three ghosts	`hydro.cpp:94-103`	`wb_ref4_enabled`	`SNAP_WB_REF4`

what	code link	symbol	switch
neighbours below and above	hydro.cpp: 261-268	x1_neighbors	nb1
switch ghost rows	hydro.cpp: 270-303	_x1_ghost_ rows	SNAP_X1_ CENTROID_ EXACT
relay and reference ghost rows	hydro.cpp: 502-640	_hydro_ref_x1	nb1
physical-face gating	hydro_ forward.cpp: 240-262	phys_x1inner	none
seam average	hydro_ forward.cpp: 425-499	kSeamFluxUpTag	none
relay and boards	layout.cpp: 879-922	take_x1_ anchor	none
block messages, at most 16 blocks	layout.cpp: 849-877	send_to_block	blocks_per_ process
column gather	layout.cpp: 777-847	gather_x1	none
EOS column repair	equation_of_ state.cpp: 279-285	gather_x1	nb1

In step order (Table 3.24):

1. Setup refuses SNAP_WB_REF4 (or SNAP_X1_CENTROID_EXACT, which implies it) with nghost < 3, through wb_ref4_enabled (hydro.cpp: 94-103).

2. In the stage, `phys_x1inner` and `phys_x1outer` gate the wall treatments (`hydro_forward.cpp:240-262`); under `SNAP_X1_CENTROID_EXACT` the plain-mean ghost rows come from the neighbours with `_x1_ghost_rows` (`hydro_forward.cpp:270-285`).
3. `_hydro_ref_x1` takes the anchor from above, scans, passes its bottom face down, and swaps reference ghost rows with both neighbours using `kWbGhostDnTag` (`hydro.cpp:502-640`).
4. After the x_1 fluxes, a process seam is averaged with `kSeamFluxDnTag` (`hydro_forward.cpp:425-499`), then the face-pressure ghost rows past the seam are exchanged with `_x1_ghost_rows` (`hydro_forward.cpp:501-508`).

3.12.5. Tests

Table 3.25.: Tests of the x_1 seams. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_pref_local_seam`	an in-process split restarts the reference at the seam; split equals one block after 200 steps	10^{-6}; 10^{-12} relative	set in the test

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_x1_seam_split` and arms `_x1_centroid`, `_radial_exact`, `_radial_exact_off`, `_wb_ref4`, `_wb_ref4_gravity_0`, `_x1_mass_covariance`	two blocks in one process equal one block after 20 steps, per switch	10^{-13} relative	round-off of 20 steps
`test_x1_seam_split_mp` and arms	the same split across two ranks	10^{-13} relative	as above
`test_parentless_cloud_nb1_mp`	column repair across two processes keeps mass	in the test	chapter 10

The reference cases are `local_blocks_restart_the_reference_at_the_seam` (`test_pref_local_seam.cpp:99-143`) and `in_process_split_matches_one_block_after_200_steps` (`test_pref_local_seam.cpp:300-331`). The arms run one gtest each with their environment through `seam_arm` (`CMakeLists.txt:36-61`); the comparisons are in `split_gap` and its callers (`test_x1_seam_split.cpp:202-286`). With `SNAP_GRAVITY_WORK_RADIAL_EXACT` the split column is expected to differ and the test asserts only the $E + P$ drift of the split column (10^{-13}) and prints the gap. CUDA twins of four arms are registered by `seam_arm` when CUDA is built (`CMakeLists.txt:62-78`); the `_mp` entries run on

two ranks with `torchrun`. Their pass status at the pin was not run for this chapter (missing evidence).

3.12.6. Limits

> **Limits**
>
> - Blocks of one process in a split column must run concurrently; a block waits at most 5 minutes for its anchor.
> - At most 16 blocks per process can exchange block messages.
> - `SNAP_GRAVITY_WORK_RADIAL_EXACT` keeps a one-sided slope at block edges, so a split column differs from one block.

A split column is not bitwise equal to one block; the tests use 10^{-13} and 10^{-12}. Issue #250 recorded, before the relay and ghost-row exchange existed, that x_1 splits on CPU with gloo differed from one rank by at most 8.0×10^{-15} relative while two GPUs with UCX were bitwise (`sources/gh__ISSUE_THREADS_138-250.md`, issue #250); that measurement predates the pin and was not repeated here. The header comment of `test_pref_local_seam.cpp` still says that `HydroImpl` restarts the scan in a second same-process block and that the test does not fix it, while the assertions expect the fixed behaviour (`test_pref_local_seam.cpp:1-15`). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.13. Sync types and the velocity frame at seams (`type`)

Local symbols (flagged to the editor): M the 3×3 pointwise matrix that takes a panel velocity to the global spherical frame at one ghost position [-].

3.13.1. Summary

> **i** At a glance
>
> **What**: says which variables each sync carries and how their velocities are treated across a cubed-sphere panel edge: conserved momenta are covariant (raised before the rotation, lowered after), primitive velocities are contravariant (rotated only), scalars are not rotated. **Switch**: `SyncOptions.type` (`kConserved`, `kPrimitive`, `kScalar`). **Default**: `kConserved`. **Requires**: nothing. **Order**: exact (a change of basis). **Defining equation**: Equation 3.21.

The stage update exchanges conserved variables, `hydro_u` and `scalar_s`, and rebuilds the scalar primitive from them afterwards; the initial sync exchanges primitives, `hydro_w` and `scalar_r`. The face-state syncs carry primitive face states (Section 3.10) and the donor factor is a scalar (Section 3.11). On a slab or cubed layout the type does not matter, because a copy needs no rotation. On the cubed sphere the velocity of a strip is rotated to the global spherical frame before it leaves the sender and into the receiver's panel frame after it lands; the type decides whether the metric is applied around that rotation.

3.13.2. Derivation

The panel frame's velocity components are contravariant along the unit tangent vectors; the conserved momenta are stored covariant (Equation 3.12). The rotation to the spherical frame is defined on contravariant components, through Cartesian components on the sphere. For a conserved strip the sender therefore raises first and the receiver lowers last, and the code folds both into one matrix per ghost position:

$$\begin{aligned} \mathsf{M}^{\to\mathrm{sph}} &= \mathsf{R}^{\mathrm{sph}\leftarrow\mathrm{panel}}\,\mathsf{G}^{-1}, \\ \mathsf{M}^{\to\mathrm{panel}} &= \mathsf{G}\,\mathsf{R}^{\mathrm{panel}\leftarrow\mathrm{sph}}, \end{aligned} \tag{3.21}$$

with G the lowering of Equation 3.12 acting on (v_2, v_3) and R the panel-to-spherical rotation (local notation). For a primitive strip G is left out. The matrix is built column by column by transforming the three unit vectors, in `cs_velocity_transform_matrix` (`cubed_sphere_utils.cpp: 259-298`). Each factor is exact; the composition is a change of basis and loses nothing but round-off.

3.13.3. Numerical method

The matrix depends only on the angular position, so it is built for one radial plane and broadcast along x_1, and it is cached per sync options, buffer, direction and shape (`_velocity_transform`). In the pack the rotation to the spherical frame acts on the whole packed strip, margin included, before relabelling; in the unpack the rotation back acts only on the narrow ghost zone after interpolation. The scalar primitive of the stage sync is rebuilt by `set_scalar_primitive` after the exchange and the corner refresh. Table 3.26 lists the syncs.

Table 3.26.: The syncs of one block and their types.

sync	type	interpolate	where
stage hydro / scalar	conserved / scalar	yes	`exchange_ ghost_zones`
initial hydro / scalar	primitive / scalar	yes	`initialize`
face states	primitive, scalar	no	Section 3.10
donor factor	scalar	no	Section 3.11

sync	type	interpolate	where
`Mesh` level	caller's choice	yes	`Mesh::` `exchange_` `ghost_zones`

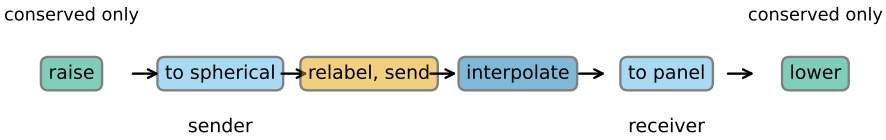


Figure 3.13.: The velocity path of a conserved strip across a panel edge: covariant components on the sender's panel are raised, rotated to the global spherical frame, relabelled and sent; on the receiver they are interpolated in the spherical frame, rotated into its panel frame and lowered. A primitive strip skips the raise and the lower; a scalar strip skips the rotations.

Figure 3.13 draws the path of one strip.

3.13.4. Code

Table 3.27.: Code map of the sync types.

what	code link	symbol	switch
stage sync, scalar rebuild	`meshblock.cpp:` `913-978`	`set_scalar_` `primitive`	none
initial sync	`meshblock.cpp:` `382-394`	`kPrimitive`	none

what	code link	symbol	switch
initial sync under `Mesh`	`meshblock.cpp:434-449`	`initialize_under_mesh`	none
`Mesh`-level sync	`mesh.cpp:389-396`	`exchange_ghost_zones`	`type`
matrix with raise or lower	`cubed_sphere_utils.cpp:259-298`	`cs_velocity_transform_matrix`	`type`
cached matrix, one radial plane	`cubed_sphere_layout.cpp:122-152`	`_velocity_transform`	`type`
pack rotation	`cubed_sphere_layout.cpp:736-753`	`kConserved`	`type`
unpack rotation	`cubed_sphere_layout.cpp:939-956`	`kPrimitive`	`type`

In step order (Table 3.27):

1. The caller picks the type in `SyncOptions`; `_velocity_transform` passes the cosine to the matrix only for `kConserved` (`cubed_sphere_layout.cpp:122-152`).
2. `serialize` applies `cs_apply_velocity_transform_` to the velocity rows of a conserved or primitive strip (`cubed_sphere_layout.cpp:736-753`); `deserialize` applies the inverse matrix with `cs_apply_velocity_transform_` (`cubed_sphere_layout.cpp:939-956`).

3.13.5. Tests

Table 3.28.: Tests of the sync types. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_coordinate`	the cached matrices, with and without the metric, equal the direct transforms	10^{-4} rel., 10^{-5} abs.	float32 cases
`test_cubed_sphere_vertical_velocity_exchange_python`	a radial velocity crosses every edge unchanged	10^{-12}	Section 3.8

The matrix case is `cached_cubed_sphere_velocity_matrices_match_direct` (`test_coordinate.cpp:307-354`); it runs on CPU and CUDA in float32 and float64 through `DeviceTest`. Their pass status at the pin was not run for this chapter (missing evidence).

3.13.6. Limits

No known limits. A tangential velocity crossing an edge is not tested end to end; only the radial component is (not measured). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.14. x_1-wall corner refresh inside tangential ghost slabs

3.14.1. Summary

> **At a glance**
>
> **What**: after the tangential exchange, each installed x_1 boundary function except outflow is applied again inside the x_2 and x_3 ghost slabs, so the ghost corners next to an x_1 wall match the updated column. **Switch**: none. **Default**: always on in the block's per-stage sync. **Requires**: boundary functions that give the same ghosts when called again on an n_g-wide slab. **Order**: exact. **Defining rule**: stated in the Derivation layer.

The tangential exchange fills the x_2 and x_3 ghost slabs with the neighbour's interior, but within those slabs the x_1 ghost rows (the corners next to an x_1 wall) hold whatever the neighbour's wall wrote last, or stale values on a periodic edge. A stencil with cross-derivatives, such as viscosity next to the wall, then reads a wall corner that does not belong to this column. Re-applying the x_1 boundary function on the slabs makes those corners the wall ghosts of the updated slab. In the history: chengcli/snapy#265.

3.14.2. Derivation

There is nothing to derive; it is a consistency rule. For each installed x_1 bfunc that is not outflow, and each tangential direction with more than one cell, the bfunc is called on the first and last $g = \min(n_g, n/2)$ cells of that direction, unless that tangential face is itself physical and not periodic, in which case its own bfunc already wrote the corner. The rule relies on the contract stated with the `BC_FUNCTION` macro: a face function called again on a tangential slab must not depend on the slab's size and must

give the same ghosts (`bc_func.hpp:28-35`). Outflow is excluded because its background must match the whole block and it needs primitives.

3.14.3. Numerical method

The refresh runs at the end of every per-stage sync, on `hydro_u` as conserved and on `scalar_s` as scalar, after both exchanges and before the scalar primitive is rebuilt. It writes only x_1 ghost rows inside the tangential ghost slabs. Figure 3.14 shows the cells it rewrites.

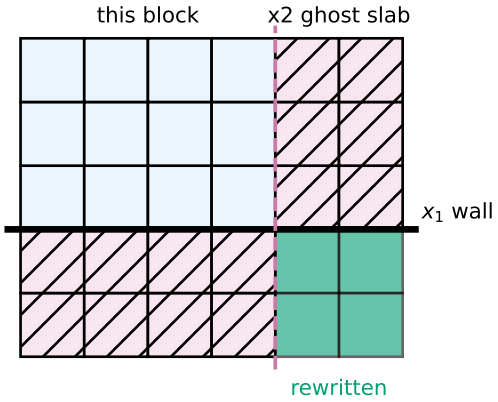


Figure 3.14.: An x_1 wall (thick line) meeting an x_2 block seam (dashed purple). The tangential ghost slab (hatched) is filled by the exchange; its cells below the wall (green) are then rewritten by the x_1 wall function, so they mirror this side of the seam rather than holding the neighbour's earlier values.

3.14.4. Code

Table 3.29.: Code map of the corner refresh.

what	code link	symbol	switch
refresh after the exchange	`meshblock.cpp: 932-974`	`refresh`	none
contract of a boundary function	`bc_func.hpp: 28-35`	`BC_FUNCTION`	none

In step order (Table 3.29):

1. `physical` is true for an installed, non-periodic tangential face (`meshblock.cpp:945-950`).
2. `refresh` loops over the two x_1 faces, skips a missing or `is_outflow` function and an x_1 axis of one cell, and calls the function on each tangential slab whose face is not `physical` (`meshblock.cpp:951-971`).
3. It runs for `hydro_u` and `scalar_s` with their types (`meshblock.cpp: 972-973` for `refresh`).

3.14.5. Tests

Table 3.30.: Tests of the corner refresh. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wb_wall_corner`	a resting polytrope uniform in x_2 with viscosity keeps $\max\|v_2\|/c_s$ and $\max\|\mathbf{v}\|/c_s$ small, for the stock wall and a user-named wall	10^{-12}	rest to round-off
`test_wb_wall_corner`	a two-block x_2 split equals one block	exactly 0	bitwise
`test_wb_wall_corner`	the scalar corner primitive matches its conserved value	exactly 0	bitwise

The cases are `x2_uniform_rest_keeps_u2_zero` and `user_wall_keeps_u2_zero` (`test_wb_wall_corner.cpp:160-190`), `x2_split_matches_one_block_exactly` (`test_wb_wall_corner.cpp:295-318`) and `scalar_corner_primitive_matches_conserved` (`test_wb_wall_corner.cpp:321-336`), each with a `_cuda` twin that skips without a GPU. One process. Their pass status at the pin was not run for this chapter (missing evidence).

3.14.6. Limits

> **Limits**
>
> - `Mesh::exchange_ghost_zones` does not refresh corners; only the block's per-stage sync does.
> - Only x_1 walls are refreshed; x_2-x_3 corners are synthesised (Section 3.8).

The `Mesh`-level sync is a plain exchange (`mesh.cpp:389-396` for `exchange_ghost_zones`). A user boundary function that breaks the contract would write wrong corners without an error (read from the code; not measured). The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

3.15. Registered stub coordinate type (`cylindrical`)

3.15.1. Summary

> **At a glance**
>
> **What**: the name `cylindrical` is accepted and builds a coordinate whose `reset` is empty. **Switch**: YAML `geometry.type: cylindrical`. **Default**: off. **Requires**: nothing. **Order**: none; no metric is built. **Defining equation**: none.

`CoordinateImpl::create` maps `cylindrical` to `CylindricalImpl`, but its `reset` does nothing and the source file that would define it is disabled by its suffix (`cylindrical.cpp_`). No widths, centres, areas or volumes are built. The type is recorded here so that a reader who meets the name knows it does not work.

3.15.2. Derivation

Not applicable: there is no implemented metric. The disabled file holds an unfinished vector transform, starting with `vec_from_cartesian` (`cylindrical.cpp_`), which is not compiled.

3.15.3. Numerical method

None. Figure 3.15 is a one-line status figure kept for the uniform layout of the chapter.

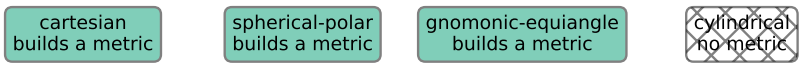


Figure 3.15.: Status of the coordinate types at the pin: Cartesian, spherical-polar and gnomonic equiangle build a metric; cylindrical is registered but builds none.

3.15.4. Code

Table 3.31.: Code map of the cylindrical stub.

what	code link	symbol	switch
empty module	`coordinate.hpp: 363-380`	`CylindricalImpl`	`type: cylindrical`
name registered	`coordinate.cpp: 599-606`	`Cylindrical`	`geometry.type`

In step order (Table 3.31): `create` returns a `Cylindrical` module for the name (`coordinate.cpp:599-606`), whose `reset` is the empty override (`coordinate.hpp:368-374`).

3.15.5. Tests

Table 3.32.: Tests of the cylindrical stub. C++ entries carry the build suffix (`.release`).

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
none	no test selects `cylindrical`	n/a	n/a

Missing evidence: no test at the pin exercises this type.

3.15.6. Limits

> **Limits**
>
> - Selecting `cylindrical` runs on unbuilt metric buffers.

What a run with this type does was not measured; the statement above is read from the code. The switch combinations that are tested are listed in the switch coverage matrix of chapter 12.

4. Spatial discretization

This chapter is the semi-discrete operator of one Runge-Kutta stage: everything that turns a cell-averaged state into a tendency, and nothing that advances it in time. A stage reconstructs the primitive variables to the faces, solves a Riemann problem on each face to get a single flux there, makes that flux single-valued across a process seam, takes its divergence, and adds the part of the curvature that is not a flux difference. Time integration is chapter 7, the well-balanced treatment of the vertical is chapter 5, and the positivity limiter that runs between the Riemann solve and the divergence is chapter 8.

The chapter has two halves because the second has a derivation thread the first does not. Section 4A is the operator and its interchangeable parts: the divergence, the reconstruction framework with its six interpolants, the Riemann framework with its five solvers, seam averaging and the geometric sources. Almost none of that is written down anywhere in the repository, so it is re-derived here from the code. Section 4B is one idea applied twice: a finite-volume flux is an average over a face, a solver computes a flux from averages over cells, and the difference between those is second order in the direction that carries the stratification. That half has a committed derivation behind it and its own test family.

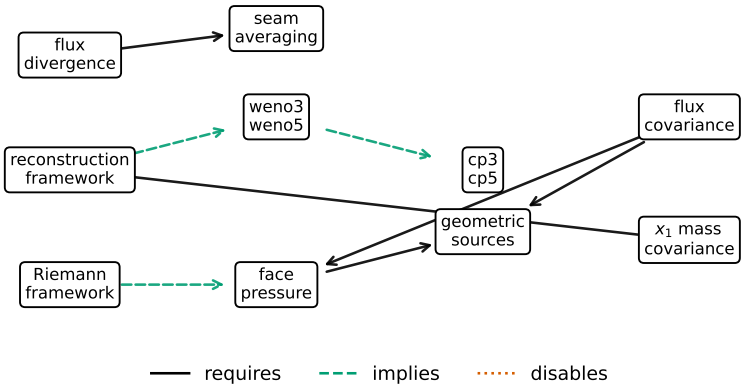


Figure 4.1.: The dependency map of this chapter. A solid black arrow reads requires, a dashed green arrow reads implies. The reconstruction framework implies a pair of interpolants, and selecting either WENO type implies the matching linear polynomial for the velocity and pressure rows. The geometric sources require a solver that returns a face pressure, which only three of the five provide. Both O(dx1^2) corrections require something of the rest of the chapter: the horizontal one needs the face pressure and the curved-grid moments, and the x1 one acts on the reconstructed velocity. Every edge is stated and cited in the section it comes from.

Table 4.1.: The schemes of this chapter. "Order" is the spatial order of the discrete operator where that is meaningful; the Riemann solvers are characterised by their wave structure instead.

scheme	switch	default	order	main test
Finite-volume update (Section 4.2)	`disable-flux-x1|x2|x3`	all on	exact in flux form	`test_cubed_sphere_cell_volume_python`
Reconstruction framework (Section 4.3)	`reconstruct/{type,scale,shock}`	`dc`, false, false	set by the interpolant	`test_plm`
Donor cell (Section 4.4)	`type: dc`	the default	1	`test_plm`
PLM (Section 4.5)	`type: plm`	off	2, 1 at an extremum	`test_plm`
Centred cp3, cp5 (Section 4.6)	`type: cp3`, `cp5`	off	3, 5	this chapter's check
WENO3, WENO5 (Section 4.7)	`type: weno3`, `weno5`	off	3, 5; weno3 is 2 at a critical point	`test_weno`
PPM (Section 4.8)	`type: ppm`	off	1: the interpolant is a stub	none
Riemann framework (Section 4.9)	`riemann-solver/{type,dir}`	`roe`, `omni`	—	`test_riemann`

scheme	switch	default	order	main test
LMARS (Section 4.10)	`type: lmars`	off; the decks select it	consistent, 1 in the upwind branch	`test_riemann`
HLLC (Section 4.11)	`type: hllc`	off	three-wave	`test_riemann`
Roe (Section 4.12)	`type: roe`	the default	linearised, no entropy fix	`test_riemann`
Shallow and plume Roe (Section 4.13)	`type: shallow-roe`, `plume-roe`	off	plume-roe is Lax-Friedrichs	`test_shallow_splash`
Seam averaging (Section 4.14)	none; split x_1 axis	automatic	restores telescoping	`test_x1_seam_split_mp`
Spherical-polar sources (Section 4.15)	none; coordinate type	automatic	exact areas and volume	`test_radial_face_moments`
Gnomonic sources (Section 4.16)	none; coordinate type	automatic	exact solid angle and volume	`test_hydrostatic`
Horizontal flux covariance (Section 4.18)	`SNAP_FLUX_COVARIANCE`	off	removes two $O(\Delta x_1^2)$ terms	`test_flux_covariance_rows`

scheme	switch	default	order	main test
x_1 mass covariance (Section 4.19)	`SNAP_X1_MASS_COVARIANCE`	off	2 becomes 4 in the stored velocity	this chapter's check

Three of the entries above record code that does not do what its name says, and they are flagged where they appear rather than collected here: the PPM interpolant is a stub that returns the cell value, the `plume-roe` solver is a Lax-Friedrichs flux, and four limiter functions beside the PLM slope are never called.

4.1. 4A. Reconstruction, Riemann solvers, divergence and sources

4.2. The finite-volume update and the flux divergence

4.2.1. Summary

> **At a glance**
>
> - One stage forms $\Delta u = -\Delta t \, \nabla \cdot F$ on interior cells, then adds sources.
> - The divergence is a difference of AF over the two faces of each direction, divided by V.
> - Switch: `dynamics/disable-flux-x1|x2|x3`, default false; disabling a direction also zeroes that gravity component.
> - Flux form, so the operator conserves by construction on any

grid.
- Geometric sources are added by the coordinate object, not by the divergence.

Every scheme in this chapter feeds one operator. A Runge-Kutta stage reconstructs the primitive state to the faces, solves a Riemann problem on each face to get a single flux there, and then books the net flux through the cell. Written for one cell i and one direction,

$$\Delta u_i = -\Delta t \, \frac{A_{i+1/2} F_{i+1/2} - A_{i-1/2} F_{i-1/2}}{V_i}, \tag{4.1}$$

summed over x_1, x_2 and x_3. Nothing in Equation 4.1 depends on how F was obtained: the reconstructions of Section 4.3 and the Riemann solvers of Section 4.9 only decide the number that sits on the face. That separation is what makes the operator conservative whatever is plugged into it, and it is why the two correction schemes of Section 4.18 and Section 4.19 can act on the face value alone.

4.2.2. Derivation

The finite-volume balance over a cell is exact, not an approximation: integrating $\partial_t u + \nabla \cdot F = 0$ over the cell and applying the divergence theorem gives

$$\frac{\mathrm{d}}{\mathrm{d}t} \int_{V_i} u \, \mathrm{d}V = - \oint_{\partial V_i} F \cdot n \, \mathrm{d}A. \tag{4.2}$$

The only approximation in Equation 4.1 is that each face integral is replaced by one number times the face area, so $F_{i\pm1/2}$ stands for the *face average* of the normal flux. Section 4B is entirely about the difference between that face average and the value a solver returns from cell data.

Two properties follow directly, and `[C1]` and `[C4]` check them. First, on a Cartesian block the two face areas of a direction are equal, so a flux that does not vary in that direction cancels exactly and a uniform state keeps a bitwise zero tendency. Second, summing V_i times Equation 4.1 over any set of cells leaves only the fluxes through the outer boundary: interior faces appear twice with opposite signs. That telescoping is the whole conservation argument, and it is why a seam between two processes must carry a *single* value (Section 4.14) and why both terms of the 4B corrections are added to the face flux rather than to the cell tendency.

On a curved grid the areas differ between the two faces of a direction and the cancellation is no longer pointwise. What survives is the integral identity: `[C2]` reproduces $\nabla\cdot(r\hat{r}) = 3$ to round-off from the code's own areas and volume, which is the identity the cubed-sphere volume test asserts cell by cell. The remainder of the curvature, the part that is not a difference of face fluxes, is the geometric source of Section 4.15 and Section 4.16.

4.2.3. Numerical method

The stage is `forward` at `hydro_forward.cpp:198-209`, which converts the conserved state to primitive with `peos` and then runs the sweeps in the order of Figure 4.3. The divergence itself is `divergence` at `coordinate.cpp:509-553`: it allocates one tendency tensor, adds the AF difference of each defined direction over the interior index range, and divides by the cell volume. A direction that is switched off simply has no flux tensor, so the same code covers one, two and three dimensions.

The index convention matters when reading the code. Cells run from the first interior index to the last, and the faces of the same direction run one further, so the slices at `face_area1` (`coordinate.cpp:534-538`) read one more face than there are cells. The tendency is written on interior cells only; ghost columns never receive one, which is why a correction that writes a ghost column is harmless.

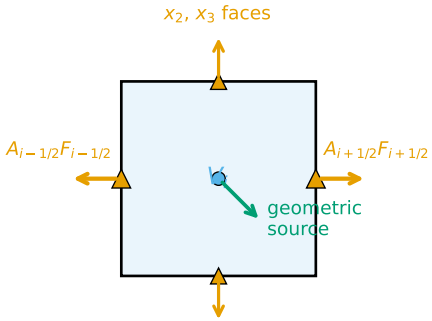


Figure 4.2.: One finite-volume cell and the six face fluxes that update it. The cell value (sky-blue circle) carries the volume average over the cell; each face carries one number (orange triangle) times its area. The two x1 arrows are drawn with their own areas, which differ on a curved grid. The green arrow is the geometric source, which the coordinate object adds and the divergence does not.

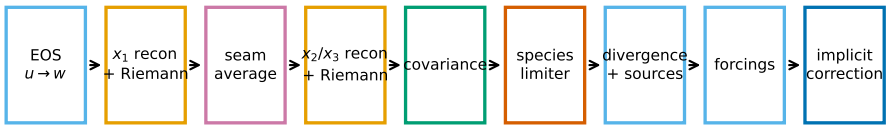


Figure 4.3.: The order of one Runge-Kutta stage. Each box is a step of the stage operator, in the order the code runs them; the colour marks which chapter owns the step, with orange the face work of this chapter, green the fourth-order face corrections of section 4B, vermillion the species limiter of chapter 8 and blue the implicit correction of chapter 7. The divergence is the seventh box: every face value is final before it runs.

The base `forward` at `coordinate.cpp:585-591` is the Cartesian case: it discards the primitive state and the face pressure and returns the divergence alone. Every curved coordinate overrides it and adds its source on top of the same divergence.

Table 4.2.: Where each quantity of the stage operator lives.

quantity	lives at	from
F_1, F_2, F_3	faces of its own direction	the Riemann solver, then the species limiter
$A_{i\pm1/2}$, V_i	faces, cells	`face_area1`, `cell_volume`
Δu	interior cells	minus Δt times the divergence

4.2.4. Code

Every link below is at the pin snapy@e894700ff7aee30b52882e5202b16461413780b0. The symbol column is the citation's anchor: it is the identifier the cited range contains.

Table 4.3.: Code map of the finite-volume update.

what	code link	symbol	switch / condition
the stage operator	`hydro_forward.cpp:198-209`	`forward`	every stage
conserved to primitive	`hydro_forward.cpp:209`	`peos`	

what	code link	symbol	switch / condition
the divergence	`coordinate.cpp: 509-553`	`divergence`	
the x_1 term	`coordinate.cpp: 534-538`	`face_area1`	x_1 flux defined
Cartesian: divergence only	`coordinate.cpp: 585-591`	`forward`	cartesian
per-direction switch	`hydro_options.cpp: 54-56`	`disable_flux_x1`	YAML, default false

4.2.5. Tests

Table 4.4.: Tests of the finite-volume update.

test	asserts	tolerance
this section's check	`[C1]` bitwise zero tendency on a uniform Cartesian block; `[C2]` the radial identity; `[C3]` the volume is the exact radial integral; `[C4]` the sum of $V\,\nabla{\cdot}F$ is the boundary flux	round-off
`tests/test_cubed_sphere_cell_volume.py`	six panel volumes sum to $\frac{4}{3}\pi(r_o^3 - r_i^3)$, and the radial identity per cell	`1e-12`, rest `1e-6`

test	asserts	tolerance
`tests/test_coordinate.cpp`	gnomonic areas and volumes, frame projections	allclose defaults

The check is `src/snapy_report/ch04/operator_check.py`, output `src/snapy_report/ch04/operator_check.out`, 8 of 8 claims passing.

4.2.6. Limits

> **Limits**
>
> - Cylindrical coordinates are not built: the source file carries a trailing underscore and is not compiled.
> - A uniform state is preserved bitwise only on a Cartesian block; on a curved grid the cancellation is an integral identity and holds to round-off, not bitwise.

1. **Ghost tendencies.** No tendency is ever formed on a ghost cell, so a correction that writes one cannot reach the state. That is a property of the current index ranges, not something the code asserts.
2. **One value per face.** The operator assumes each face carries one number. Where two processes own the same face that has to be arranged explicitly (Section 4.14); where the two sides disagree, averaging hides the disagreement rather than removing it.

4.3. The reconstruction framework (`dynamics/reconstruct`)

4.3.1. Summary

> **At a glance**
>
> - Returns a left and a right state for every face of one direction, from the cell primitives.
> - Two interpolants per direction: `interp1` for density and tracers, `interp2` for velocity and pressure.
> - Switches: `dynamics/reconstruct/{vertical,horizontal}/{type,scale,shock}`; YAML defaults `dc`, false, false.
> - With `shock: true` every row takes `interp1` and no floor is applied.
> - Floors come from the equation of state, not from the reconstruct options.

A reconstruction turns cell averages into the two one-sided face states a Riemann solver needs. snapy keeps two of them per block, one for the vertical direction and one shared by the two horizontal directions, because the vertical is the direction with an $O(1)$ background stratification and generally wants a different interpolant from the horizontals.

The framework's own content is small but it decides three things that matter downstream: which rows take which interpolant, where in the index range the reconstruction is allowed to write, and which floors are applied to the result.

4.3.2. Derivation

There is nothing to derive about the splitting itself; it is a design choice and the code is the only record of it. Two parts of it are worth stating plainly because they are easy to misread.

First, the rows are not treated alike. Density and tracers go through `interp1`, which is the interpolant the user named. Velocity and pressure go through `interp2`, which for the two WENO types is *not* the WENO interpolant but the linear centred polynomial of the same width: `weno3` pairs with `cp3` and `weno5` with `cp5`, selected by the last character of the module name at `interpolation.cpp:21-36`. No source in the repository gives a rationale for that pairing, so this section records it as a design choice rather than a derived result; the consequence is that under `weno5` the velocity and pressure face states are fifth-order linear and are not limited, while density and tracers are.

Second, a smoothness switch replaces the split entirely. With `shock: true` every row takes `interp1` and the floors are skipped, at `shock` (`reconstruct.cpp:99-102`), so the whole state is limited by the same nonlinear interpolant.

4.3.3. Numerical method

`forward` at `reconstruct.cpp:79-158` allocates a tensor with a leading axis of two, the left and right face states, and fills it row group by row group. The number of ghost cells the interpolant needs is read from its own stencil width at `nghost` (`reconstruct.cpp:86`), which sets the interior range; a block too thin for the stencil fails the check on the next line rather than silently reading a ghost.

The writing itself is `_apply_inplace` at `reconstruct.cpp:50-65`. Its index map is the part worth reading twice: the right state of face f and

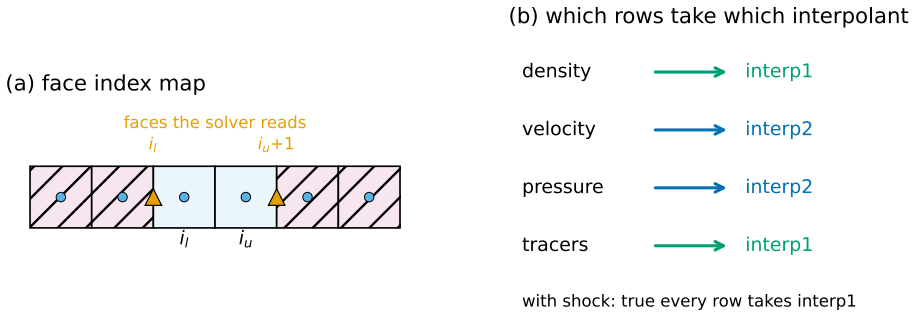


Figure 4.4.: What a reconstruction writes, and which rows take which interpolant. (a) The face index map: interior cells are plain, ghost cells are hatched in light reddish purple, and the orange triangles are the first and last face the solver reads; the dummy faces outside that range are filled by replication, not by the interpolant. (b) The row split with shock false: density and tracers take interp1 (green), velocity and pressure take interp2 (blue). With shock true every row takes interp1.

the left state of face f come from cells on opposite sides, so the two output slices are offset by one, and the faces outside the interior range are filled by replicating the nearest computed face rather than by running the interpolant on ghost data.

The floors are applied after each group and they come from the equation of state options, not from the reconstruct options: density at `density_floor` (`reconstruct.cpp:135-137`), pressure at `reconstruct.cpp:143-145`. The tracer clamp at `clamp_min_` (`reconstruct.cpp:153-155`) is different in kind: it is applied whenever the limiter is on, *even when the caller passed `floor = false`*, so the well-balanced path of chapter 5 that asks for no floors still gets its tracers clamped at zero.

4.3.4. Code

Every link below is at the pin snapy@e894700ff7aee30b52882e5202b16461413780b0. The symbol column is the citation's anchor.

Table 4.5.: Code map of the reconstruction framework.

what	code link	symbol	switch / condition
keys and defaults	`reconstruct.cpp:24-39`	`from_yaml`	type `dc`, scale false, shock false
face index map	`reconstruct.cpp:50-65`	`_apply_inplace`	
the reconstruction	`reconstruct.cpp:79-158`	`forward`	
ghost width from the stencil	`reconstruct.cpp:86`	`nghost`	
shock branch	`reconstruct.cpp:99-102`	`shock`	`shock: true`

what	code link	symbol	switch / condition
density, then its floor	`reconstruct.cpp:133-137`	`pinterp1`	limiter and floor
velocity and pressure	`reconstruct.cpp:141-145`	`pinterp2`	limiter and floor
tracers, clamped regardless	`reconstruct.cpp:151-155`	`clamp_min_`	limiter on
which interpolant is `interp2`	`interpolation.cpp:21-36`	`Center3Interp`	`weno3`, `weno5`

4.3.5. Tests

Table 4.6.: Tests of the reconstruction framework.

test	asserts	tolerance
`tests/test_plm.cpp`	a constant field is reconstructed exactly, for `dc`, `plm` and `weno5` in all three directions	allclose defaults
this chapter's check	`[C1]`-`[C7]` the interpolant weights and their measured orders	round-off and fitted orders

No test covers the row split itself: that density and tracers take `interp1` while velocity and pressure take `interp2` is asserted nowhere, and a swap would be caught only by an accuracy test.

4.3.6. Limits

⚠ Limits

- The interpolant weights are fixed rationals for a uniform mesh. On a stretched x_1 grid they are applied unchanged, so the nominal order is not attained there.
- The programmatic default for `shock` and the YAML default disagree: the options object defaults it true while `from_yaml` writes false. Construction from Python and from a deck give different schemes unless the key is set explicitly.

1. **Dead members.** The options object carries `density_floor`, `pressure_floor`, `limiter` and the two boundary flags, and `forward` reads none of them: the floors come from the equation of state. The report of the options object prints them anyway, which is misleading rather than wrong.
2. **A disabled branch.** `is_boundary_lower` (`reconstruct.cpp: 104-127`) is a commented-out block that would have narrowed the interior range next to a physical boundary before reconstructing velocity and pressure. It is not compiled, and nothing in the repository records why it was switched off. Reading the file without noticing that it is inside a comment is an easy mistake.
3. **`scale: true`.** The key is parsed and passed to the interpolant, and issue #275 reports that setting it changes results in moist bubble decks through the well-balanced reference. Why it does is not explained in any source; this section records it as an open question rather than a mechanism.

4.4. Donor cell (`type: dc`)

4.4.1. Summary

> At a glance
>
> - The face state is the adjacent cell value: no stencil, no limiter.
> - Switch: `type: dc`, and it is the YAML default.
> - First order at the face, so first order in the flux divergence.
> - Needs one ghost cell, which is why it is the only interpolant that runs on a block with `nghost = 1`.

The simplest reconstruction there is, and the default the code falls back to when a deck names no type. The left state of a face is the value of the cell to its left and the right state is the value of the cell to its right, so the face carries a jump wherever the solution is not constant and the Riemann solver does all the work.

4.4.2. Derivation

A cell average equals the point value at the cell centre to $O(h^2)$, but the face is half a cell away from either centre, so using the cell value at the face is wrong at $O(h)$. `[C7]` measures that directly: the face error falls as $9.79 \times 10^{-2} \to 1.23 \times 10^{-2}$ over $n_1 = 32 \to 256$, a fitted order of 1.00.

First order at the face is the whole story of the scheme. It is also why donor cell is the useful control arm in a test: any correction that claims to raise an order can be run against it and the difference is unambiguous.

4.4.3. Numerical method

The implementation is a copy, at `interpolation.hpp:88-105`: `left` and `right` take the neighbouring cell value with no arithmetic at all. Because the stencil is one cell wide, the ghost width the framework computes from it is the smallest of any interpolant, and donor cell is the only type that runs on a block with a single ghost layer.

4.4.4. Code

Table 4.7.: Code map of donor cell.

what	code link	symbol	switch / condition
the interpolant	`interpolation.hpp:88-105`	`DonorCellInterpImpl`	`type: dc`
registration	`interpolation.cpp:11-12`	`DonorCellInterp`	

4.4.5. Tests

Table 4.8.: Tests of donor cell.

test	asserts	tolerance
`tests/test_plm.cpp`	a constant field is reconstructed exactly	allclose defaults
`tests/test_x1_seam_split.cpp`	runs donor cell on a block with one ghost cell	`1e-13`
this chapter's check	`[C7]` first order at the face	fitted order

4.4.6. Limits

1. **First order everywhere.** Nothing in the scheme adapts: smooth regions are reconstructed as badly as discontinuities. A deck that leaves `type` unset gets this scheme, which is rarely what is wanted for a production run and is easy to miss because no warning is printed.

4.5. PLM, the van Leer harmonic-mean slope (`type: plm`)

4.5.1. Summary

> **At a glance**
>
> - A piecewise-linear reconstruction whose slope is the harmonic mean of the two one-sided differences.
> - Switch: `type: plm`.
> - Second order where the field is smooth, first order at an extremum, and total-variation diminishing.
> - The slope is set to zero wherever the two one-sided differences have opposite signs.

PLM takes the three cells around a face and fits a line through the middle one. What makes it a *limited* scheme rather than a plain centred slope is the choice of slope: the harmonic mean of the left and right differences, which collapses to zero whenever they disagree in sign.

4.5.2. Derivation

Write the two one-sided differences as $\Delta_L = \phi_i - \phi_{i-1}$ and $\Delta_R = \phi_{i+1} - \phi_i$. The slope the code uses is

$$\Delta_m = \begin{cases} \dfrac{2\,\Delta_L \Delta_R}{\Delta_L + \Delta_R} & \Delta_L \Delta_R > 0, \\ 0 & \text{otherwise}, \end{cases} \tag{4.3}$$

and the left face value of cell i is $\phi_i - \frac{1}{2}\Delta_m$. Three properties follow, and `[C5]` checks all three on twenty thousand random triples.

The slope vanishes at every extremum, because an extremum is exactly the case $\Delta_L \Delta_R \leq 0$; the check measures the face value there against the cell value and finds them bitwise equal. That is what makes the scheme first order at an extremum and second order elsewhere.

The face value never leaves the interval spanned by the two cells it sits between, which is the total-variation-diminishing property; the check finds no violation.

On a smooth field the two differences agree and Equation 4.3 reduces to their common value, so the face value is the centred one and the scheme is second order. The check confirms the limit exactly on a linear field.

4.5.3. Numerical method

The implementation is `dwm` (`interp_simple.hpp:57-63`) and reads as Equation 4.3 does, with the sign test written as a product. The file also carries `minmod`, `superbee`, `vanleer` and an MC limiter at `interp_simple.hpp:11-33`, none of which the PLM interpolant calls: the harmonic mean is hard-coded. Those four functions are dead code at this pin.

4.5.4. Code

Table 4.9.: Code map of PLM.

what	code link	symbol	switch / condition
the slope and face value	`interp_simple.hpp: 57-63`	`interp_plm`	`type: plm`
registration	`interpolation.cpp: 13-14`	`PLMInterp`	
unused limiters	`interp_simple.hpp: 11-33`	`minmod`	never called

4.5.5. Tests

Table 4.10.: Tests of PLM.

test	asserts	tolerance
`tests/test_plm.cpp`	a constant field is reconstructed exactly in all three directions	allclose defaults
this chapter's check	`[C5]` the harmonic mean: TVD, zero slope at every extremum, centred in the smooth limit	bitwise and round-off

No test measures the order of PLM. Second order away from extrema is an argument from Equation 4.3 and the check's smooth limit, not a measured ladder.

4.5.6. Limits

1. **Dead limiters.** Four limiter functions sit beside the interpolant and none is reachable. A reader choosing a limiter from the file will not find a switch that selects one.
2. **Uniform-mesh slope.** Equation 4.3 assumes equal cell widths; on a stretched grid the harmonic mean of the raw differences is not the limited slope of the underlying linear fit.

4.6. The centred linear polynomials (`type: cp3`, `cp5`)

4.6.1. Summary

> **i** At a glance
>
> - Fixed-weight linear face interpolants of third and fifth order, with no limiter.
> - Switches: `type: cp3`, `type: cp5`; they are also the `interp2` partners of `weno3` and `weno5`.
> - Reach their nominal order on a smooth field and oscillate across a discontinuity.
> - The even-width helpers `cp2`, `cp4`, `cp6` exist for the upwind-biased forms and are not selectable.

These are the plain polynomial reconstructions: fit a polynomial through a fixed number of neighbouring cell averages and evaluate it at the face. Nothing adapts, so they are the fast and accurate choice where the field is smooth and the wrong choice across a shock. They matter beyond their own switch because they are what the velocity and pressure rows actually use whenever a deck selects a WENO type (Section 4.3).

4.6.2. Derivation

The weights are determined by requiring the formula to be exact when the underlying field is a polynomial of degree up to one less than the stencil width, with the cells carrying *averages* rather than point values. `[C1]` checks this symbolically rather than by assertion: for each of `cp2`, `cp3`, `cp4`, `cp5` and `cp6` it raises the degree of a test polynomial until the formula first fails, and recovers exactly the nominal order in every case, that is exact through degree one, two, three, four and five respectively.

Because the cells carry averages, the weights are *not* the Lagrange weights for point values; the distinction is the same one that section 4B is built on, and getting it wrong is the easiest way to lose an order.

`[C4]` then measures the face error on a smooth monotone profile and finds fitted orders of 3.02 for `cp3` and 5.02 for `cp5` over $n_1 = 16 \to 64$. The same measurement through two critical points leaves `cp3` at 2.99: a linear scheme does not care about critical points, which is exactly the contrast Section 4.7 needs.

4.6.3. Numerical method

The weights are written out as rationals at `interp_cp3` (`interp_simple.hpp:73-75`) for `cp3` and `interp_cp5` (`interp_simple.hpp:110-114`) for `cp5`; `cp2`, `cp4` and `cp6` sit beside them at `interp_cp2` (`interp_simple.hpp:43-45`), `interp_cp4` (`interp_simple.hpp:79-82`) and `interp_cp6` (`interp_simple.hpp:118-121`). The even-width three are not registered as types: they exist because the upwind-biased forms `bp2`, `bp3` and `bp5` are built from them as a centred part plus a dissipative part. Those upwind forms are themselves unreachable at this pin, with no type string selecting them.

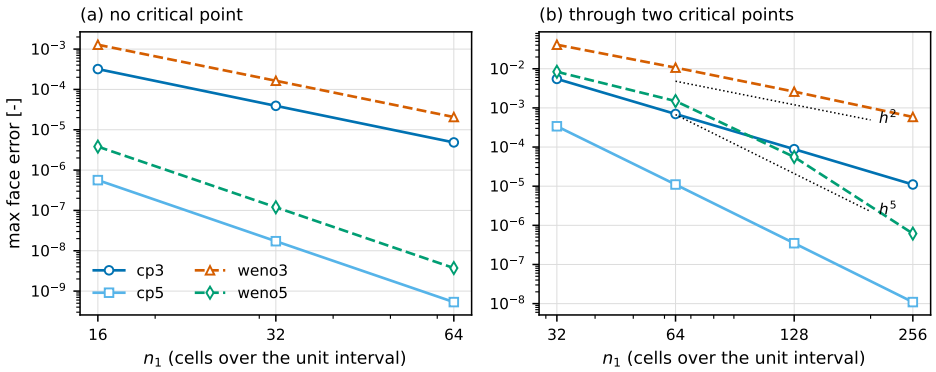


Figure 4.5.: Measured face error of the four selectable high-order interpolants against resolution. (a) On a monotone profile with no critical point in the domain, each reaches its design order: the two linear polynomials (blue circles, sky-blue squares) and the two WENO forms (vermillion triangles, green diamonds) lie on top of one another. (b) Through two critical points the linear cp3 keeps order three while weno3 falls to order two; the dotted lines are reference slopes. Data from the committed output of the chapter's reconstruction check.

4.6.4. Code

Table 4.11.: Code map of the centred polynomials.

what	code link	symbol	switch / condition
third-order weights	`interp_simple.hpp: 73-75`	`interp_cp3`	`type: cp3`, or `interp2` of `weno3`
fifth-order weights	`interp_simple.hpp: 110-114`	`interp_cp5`	`type: cp5`, or `interp2` of `weno5`
even-width helpers	`interp_simple.hpp: 43-45`	`interp_cp2`	not selectable
upwind-biased forms	`interp_simple.hpp: 86-90`	`interp_bp3`	not selectable
registration	`interpolation.cpp: 17-20`	`Center3Interp`	

4.6.5. Tests

Table 4.12.: Tests of the centred polynomials.

test	asserts	tolerance
this chapter's check	`[C1]` exactness by degree; `[C4]` measured orders 3.02 and 5.02 with no critical point	symbolic and fitted orders

test	asserts	tolerance
`tests/test_weno.cpp`	the WENO interpolants, which carry `cp3` and `cp5` as their smooth limit	per test

No ctest exercises `cp3` or `cp5` as a selected type on their own.

4.6.6. Limits

> **Limits**
>
> - No limiter. Across a discontinuity these interpolants oscillate, and nothing in the scheme detects that; the floors of Section 4.3 are the only thing standing between an overshoot and a negative density.

1. **Uniform weights.** The rationals assume equal cell widths, so the measured orders above are orders on a uniform grid only.
2. **Unreachable code.** The even-width polynomials and all three upwind-biased forms are compiled and never called at this pin.

4.7. WENO3 and WENO5 (`type: weno3`, `weno5`)

4.7.1. Summary

> **At a glance**
>
> - A convex combination of candidate polynomials, weighted away from the rough ones by Jiang-Shu indicators.
> - Switches: `type: weno3`, `type: weno5`. Only density and tracers use them; velocity and pressure take the matching linear polynomial (Section 4.3).
> - Where the field is smooth the nonlinear weights approach the linear ones and the scheme becomes `cp3` or `cp5`.
> - At a critical point WENO3 falls to second order. WENO5 does not, on the profile measured here.
> - The smoothness floor is a fixed 10^{-6}, not scaled to the magnitude of the data.

WENO is the scheme that makes a high order survive a discontinuity. Instead of one wide stencil it builds several narrow candidates, measures how rough each one looks, and takes a weighted average that all but discards any candidate straddling a jump. Away from a jump the weights settle to fixed values chosen so that the combination reproduces the wide centred polynomial.

4.7.2. Derivation

Both forms share the same structure: candidates p_k, smoothness indicators β_k, linear weights d_k, and nonlinear weights

$$\alpha_k = \frac{d_k}{(\beta_k + \varepsilon)^2}, \qquad \phi_{\text{face}} = \frac{\sum_k \alpha_k p_k}{\sum_k \alpha_k}, \qquad \varepsilon = 10^{-6}. \tag{4.4}$$

The claim that makes the scheme high order is that the *linear* weights rebuild the wide polynomial exactly. `[C2]` and `[C3]` check that symbolically, not numerically, and both differences are identically zero: with $d = (\frac{2}{3}, \frac{1}{3})$ the two WENO3 candidates rebuild `cp3`, and with $d = (\frac{3}{10}, \frac{6}{10}, \frac{1}{10})$ the three WENO5 candidates rebuild `cp5`. So wherever the indicators are equal, these schemes *are* the centred polynomials of Section 4.6.

What the smoothness indicators then do is break that equality near a jump. They are the Jiang-Shu forms, sums of squared undivided differences, at `beta0` (`interp_simple.hpp:99-100`) and `interp_simple.hpp:141-146`.

The order a WENO scheme attains in practice is not the order of its linear weights, and the chapter measures both regimes rather than quoting the design figure. `[C4]` finds, over $n_1 = 16 \to 64$ on a monotone profile with no critical point, fitted orders of 2.98 for WENO3 and 5.00 for WENO5: the design orders. Through two critical points, over $n_1 = 32 \to 256$, WENO3 falls to a fitted 2.09 while the linear `cp3` on the same profile keeps 2.99. WENO5 keeps 5.62 there.

The WENO3 result is the expected behaviour of the Jiang-Shu weights, not a defect: at a critical point the indicators of the two candidates become equal at leading order and the nonlinear weights fail to approach the linear ones fast enough to preserve third order. It is nonetheless a property a user should know about, because a smooth field full of extrema, which is what a resolved wave field is, is reconstructed at second order by a scheme named for its third.

4.7.3. Numerical method

The two interpolants are `interp_weno3` (`interp_simple.hpp:94-106`) and `interp_weno5` (`interp_simple.hpp:134-153`), written out in full with the candidates, the indicators and the weights in the order of Equation 4.4. The measured orders are drawn in Figure 4.5.

The floor ε is the literal `1e-6` inside both bodies. It is not scaled by the magnitude of the data, so for a field whose squared differences are far below 10^{-6} the floor dominates every indicator, all α_k collapse to the d_k, and the scheme silently becomes its linear partner. For a field far above it the floor is irrelevant. Neither regime is wrong, but the crossover depends on the units the deck is written in, and nothing warns at it.

Only `interp1` is a WENO interpolant. Under `weno5` the density and tracer rows are reconstructed by `interp_weno5` (`interp_simple.hpp:134-153`) while velocity and pressure take `cp5`, selected at `interpolation.cpp:29-36`. A deck that chooses `weno5` to survive a shock therefore gets an unlimited fifth-order linear reconstruction of its velocity and pressure through that same shock.

4.7.4. Code

Table 4.13.: Code map of the WENO interpolants.

what	code link	symbol	switch / condition
WENO3	`interp_simple.hpp:94-106`	`interp_weno3`	`type: weno3`, `interp1` only
WENO5	`interp_simple.hpp:134-153`	`interp_weno5`	`type: weno5`, `interp1` only
WENO3 indicators	`interp_simple.hpp:99-100`	`beta0`	
WENO5 indicators	`interp_simple.hpp:141-146`	`beta0`	

what	code link	symbol	switch / condition
the `interp2` partner	`interpolation.cpp` 29-36	`Center5Interp`	`weno5`

4.7.5. Tests

Table 4.14.: Tests of the WENO interpolants.

test	asserts	tolerance
`tests/test_weno.cpp`	the WENO interpolants on reference data	per test
`tests/test_weno5_cuda_line.cpp`	the CUDA path reproduces the CPU line	per test
`tests/test_plm.cpp`	a constant field is reconstructed exactly, `weno5` arm	allclose defaults
this chapter's check	`[C2]`, `[C3]` the linear weights rebuild `cp3` and `cp5` exactly; `[C4]` measured orders in both regimes	symbolic; fitted orders

4.7.6. Limits

> **Limits**
>
> - WENO3 is second order at a critical point, measured at 2.09 here. If third order through smooth extrema is wanted, this interpolant does not provide it.
> - The smoothness floor is an absolute 10^{-6}. Whether it dominates the indicators depends on the units of the deck, and no warning is printed either way.

1. **Only half the state is limited.** Velocity and pressure bypass WENO entirely. Whether that is the right split is a design question no source in the repository answers.
2. **Uniform-mesh weights.** As for the linear polynomials, the candidates and indicators assume equal cell widths.
3. **WENO5 through critical points.** The fitted 5.62 is above the design order and was measured on one profile over one ladder; it should be read as "did not degrade here", not as a general order claim.

4.8. PPM (`type: ppm`)

4.8.1. Summary

> **At a glance**
>
> - The type is accepted and registered, but the interpolant is a stub: it returns the cell value.
> - Switch: `type: ppm`. Selecting it gives donor cell, at donor cell's accuracy, under another name.

- Nothing warns. A deck naming `ppm` runs first order and reports no error.

PPM is selectable and does nothing. The factory accepts the string and registers an interpolant, but the function behind it returns its centre argument unchanged, which is precisely the donor-cell reconstruction of Section 4.4. This section exists to record that, because the switch is live and silent.

4.8.2. Derivation

There is nothing to derive. `[C6]` evaluates the function on a thousand random five-cell stencils and finds the returned face value equal to the centre cell value in every one, to the last bit.

4.8.3. Numerical method

The body is one line, at `interp_ppm` (`interp_simple.hpp:67-69`): the five arguments are accepted and the middle one is returned. The surrounding machinery is complete, so the stub is reached through the ordinary path: the factory registers it at `PPMInterp` (`interpolation.cpp:15-16`) and the framework calls it exactly as it calls any other interpolant.

Because the declared stencil width is that of a five-point scheme, a block selecting `ppm` is also required to carry the ghost width of a five-point scheme, so the stub costs the ghost cells of a method it does not implement.

4.8.4. Code

Table 4.15.: Code map of PPM.

what	code link	symbol	switch / condition
the stub	`interp_simple.hpp: 67-69`	`interp_ppm`	`type: ppm`
registration	`interpolation.cpp: 15-16`	`PPMInterp`	

4.8.5. Tests

No test selects `type: ppm`. The only check on it is this chapter's `[C6]`, which asserts the stub behaviour rather than any reconstruction property.

4.8.6. Limits

> **Limits**
>
> - Selecting `ppm` silently gives a first-order scheme. There is no warning, and the accuracy loss relative to the name is total.

1. **Open question.** Whether PPM should be implemented or the type removed is a decision for the maintainers. Leaving it selectable is the one option with no defensible reading, since a deck can ask for it and be answered with donor cell.

4.9. The Riemann solver framework and the face-local frame

4.9.1. Summary

> **At a glance**
>
> - Takes the two face states of a direction and returns one flux per face.
> - Switches: `dynamics/riemann-solver/{type,dir}`; YAML defaults `roe` and `omni`.
> - Six types are registered: `roe`, `lmars`, `hllc`, `upwind`, `shallow-roe`, `plume-roe`.
> - The momentum rows are rotated into a face-local frame by an index map, not by a matrix.
> - Only some solvers can return a face pressure; the base class refuses the request outright.

Every face carries one flux, and the solver's job is to produce it from the two one-sided states the reconstruction left there. Which solver is chosen changes the numbers but not the structure of the operator: the divergence of Section 4.2 consumes the result the same way whatever produced it.

4.9.2. Derivation

The physical flux through a face depends only on the component of velocity normal to it and on the two tangential components as passive quantities. Rather than rotate the state with a matrix, the code renames the rows: for direction `dim` the normal momentum row is

$$\mathrm{ivx} = \mathrm{IPR} - \mathrm{dim}, \tag{4.5}$$

and the two tangential rows follow by cycling. [C8] checks that this is a permutation of the three momentum rows for each of the three directions, which is what makes a single solver body correct for all of them. The cost is that the row index of "the normal velocity" is not fixed, and reading any solver body requires holding Equation 4.5 in mind.

The rotation is only the labelling. On a curved grid the face-local frame is also a genuine change of basis, and that projection is done by the coordinate object before the solver is called, not inside it.

4.9.3. Numerical method

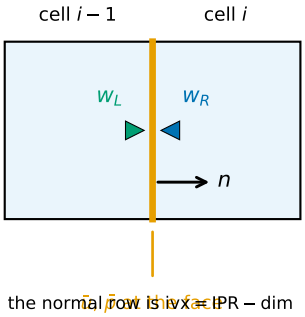


Figure 4.6.: One face and what the solver sees. The reconstruction leaves a left state (green) and a right state (blue) on the two sides of the face (thick orange line); the solver returns a single face velocity and face pressure (orange) from which the flux of every row is built. The arrow labelled n is the face normal, and the normal momentum row is selected by the index map rather than by a rotation matrix.

The factory is `create` (`riemann_solver.cpp:50-71`) and the option parsing `riemann_solver.cpp:23-31`, where the default type is `roe` and the

default direction scope is `omni`. Note that the default is `roe` and not the `lmars` that the example decks select: a deck that omits the key does not get the production solver.

The base class `forward` at `riemann_solver.cpp:39-48` is the `upwind` solver, and it is also the place where the face-pressure contract is enforced: it fails outright if a caller asks it for a face pressure. That matters because the well-balanced x_1 path of chapter 5 and the geometric source of Section 4.15 both need a face pressure, so choosing a solver that cannot produce one silently changes which pressure force the x_1 momentum row sees. Of the six types, `lmars`, `hllc` and `roe` write one.

Each solver is declared in `LmarsSolverImpl` (`riemann_solver.hpp:79-207`) as a thin module wrapping a dispatch call, and the numerics live in a header shared between the CPU and CUDA paths.

4.9.4. Code

Table 4.16.: Code map of the Riemann framework.

what	code link	symbol	switch / condition
option keys and defaults	`riemann_solver.cpp:23-31`	`from_yaml`	type `roe`, dir `omni`
the factory	`riemann_solver.cpp:50-71`	`create`	six registered types
upwind, and the face-pressure contract	`riemann_solver.cpp:39-48`	`forward`	`type: upwind`

what	code link	symbol	switch / condition
the solver modules	`riemann_solver.hpp:79-207`	`LmarsSolverImpl`	
the face-local index map	`lmars_impl.h:20-22`	`ivx`	every direction

4.9.5. Tests

Table 4.17.: Tests of the Riemann framework.

test	asserts	tolerance
`tests/test_riemann.cpp`	the registered solvers against reference states	per test
this chapter's check	`[C8]` the index map is a permutation of the momentum rows for all three directions	exact

4.9.6. Limits

⚠ Limits

- The YAML default solver is `roe`, not the `lmars` the shipped decks use. Omitting the key changes the scheme.
- Three of the six types cannot return a face pressure. Selecting one of those changes the x_1 pressure force on a curved grid and in the well-balanced path, and the only signal is a hard failure

if something asks.

1. **`dir` is parsed and its effect is not documented.** The key is read into the options and no source in the repository explains what scope `omni` selects against the alternatives.

4.10. LMARS (`type: lmars`)

4.10.1. Summary

At a glance

- A low-Mach approximate Riemann solver: one face velocity and one face pressure, then a pure upwind of the state.
- Switch: `type: lmars`. It is what the shipped example decks select, though not the YAML default.
- Consistent: equal left and right states give the exact physical flux.
- Writes a face pressure, so it works with the curved-grid pressure force and the well-balanced path.
- The species rows and a dry row split the mass flux exactly.

LMARS replaces the full wave structure of a Riemann problem with two numbers: an acoustically-corrected face velocity and an acoustically-corrected face pressure. The sign of the face velocity then decides which side the rest of the state is taken from. The result is cheap, robust at low Mach number, and the production choice in every shipped deck.

4.10.2. Derivation

Let $\bar{\rho}$ be the arithmetic mean density and $\bar{c}$ the mean sound speed built from the mean pressure. The face velocity and pressure are

$$\bar{u} = \tfrac{1}{2}(u_L + u_R) + \frac{p_L - p_R}{2\bar{\rho}\bar{c}}, \qquad \bar{p} = \tfrac{1}{2}(p_L + p_R) + \tfrac{1}{2}\bar{\rho}\bar{c}\,(u_L - u_R). \quad (4.6)$$

Each is a centred average plus an acoustic correction proportional to the jump in the *other* variable, with $\bar{\rho}\bar{c}$ the acoustic impedance. `[C6]` checks both halves separately: a pure pressure jump of 0.2 with no velocity jump drives $\bar{u} = -0.080582$, which is exactly the second term of Equation 4.6, and two states converging at 0.3 raise $\bar{p}$ to 1.354965 above the mean of 1.0, which is exactly the impedance times the velocity jump.

Consistency follows immediately and `[C5]` checks it: with $w_L = w_R$ both jump terms vanish, so $\bar{u} = u$ and $\bar{p} = p$, and the returned flux agrees with the exact physical flux to 8.9×10^{-16}. That is the property that makes the scheme usable at all; a solver that fails it would generate flux from a uniform state.

The mass rows are the part most easily misread. The dry row does not carry the total mass flux: it carries $\bar{u}\rho\, r_d$ with $r_d = 1 - \sum_n y_n$, and each species row carries $\bar{u}\rho y_n$. `[C7]` confirms the split is exact, with the dry row and three species rows summing to $\bar{u}\rho$ to 1.1×10^{-16}. A reader who takes the first mass row for the total mass flux will be wrong by the species fraction.

4.10.3. Numerical method

The body is `lmars_impl.h:17-78`. It computes the enthalpies of both sides first, at `wli` (`lmars_impl.h:28-31`), adding the kinetic term and p/ρ to the internal enthalpy the caller passed; then the means and Equation 4.6

at `lmars_impl.h:34-43`; then branches on the sign of $\bar{u}$ and fills every row from the upwind side at `lmars_impl.h:47-77`.

The face pressure is written out at `face_pressure` (`lmars_impl.h:40`) when the caller asked for one, and it is the same $\bar{p}$ that enters the normal momentum row, so the pressure force the geometric source sees and the pressure in the flux are the same number by construction.

The upwind branch is on $\bar{u}$, which already contains the acoustic correction. At low Mach number that correction is what decides the branch when the centred velocity is near zero, which is the mechanism behind the scheme's low-Mach behaviour: the pressure jump, not the velocity, selects the side.

4.10.4. Code

Table 4.18.: Code map of LMARS.

what	code link	symbol	switch / condition
the solver body	`lmars_impl.h: 17-78`	`lmars_impl`	`type: lmars`
face-local row map	`lmars_impl.h: 20-22`	`ivx`	
enthalpies	`lmars_impl.h: 28-31`	`hl`	
the face velocity and pressure	`lmars_impl.h: 34-43`	`ubar`	
face pressure written out	`lmars_impl.h: 40`	`face_pressure`	caller asked
upwind on the sign of the face velocity	`lmars_impl.h: 47-77`	`ubar`	

what	code link	symbol	switch / condition
registration	`riemann_solver.cpp:58-59`	`LmarsSolver`	

4.10.5. Tests

Table 4.19.: Tests of LMARS.

test	asserts	tolerance
`tests/test_riemann.cpp`	the solver against reference states	per test
`tests/run_straka.cmake`	an end-to-end density-current deck against a reference file, WENO5 and LMARS	reference file
this chapter's check	`[C5]` consistency; `[C6]` the acoustic structure; `[C7]` the exact mass-row split	round-off

4.10.6. Limits

⚠ Limits

- The mean sound speed is built from the arithmetic mean of the two pressures and densities. Across a strong jump that mean is not a physical sound speed on either side, and no source in the repository bounds the error that follows.

1. **No entropy fix and no wave structure.** Everything that is not the acoustic correction is pure upwinding, so a strong expansion is resolved by the flux limiter and the floors rather than by the solver.
2. **Hypothesis, untested here.** That the acoustic correction is what gives the scheme its low-Mach behaviour is an argument from Equation 4.6, not a measurement; no deck in the repository measures a Mach-number scaling of the dissipation.

4.11. HLLC (`type: hllc`)

4.11.1. Summary

> **At a glance**
>
> - A three-wave solver: two acoustic waves and the contact between them.
> - Switch: `type: hllc`. Wave speeds come from a PVRS pressure estimate.
> - Writes a face pressure, the contact pressure.
> - Resolves a contact discontinuity exactly, which the two-wave family cannot.

HLLC keeps the structure LMARS discards. It estimates the fastest left- and right-moving signal speeds, places a contact wave between them, and picks the flux according to where the face sits relative to the three. The extra wave is what lets it carry a contact discontinuity without smearing it.

4.11.2. Derivation

The derivation this section can give is the code's own, because no source in the repository derives the wave speed estimates for snapy. The structure is the standard one and the body is written in numbered steps that follow it.

The middle-state pressure comes first, from a PVRS estimate at `rhoa` (`hllc_impl.h:29-38`), and it is that pressure that sets the two outer wave speeds at `bp` (`hllc_impl.h:48-54`): each is the normal velocity on its side, displaced by the sound speed scaled by a factor that is one for a rarefaction and larger for a shock. The speeds are then clipped so that the left speed is never positive and the right never negative, which guarantees the face lies between them and the selection below is well posed.

The contact speed and the contact pressure follow at `cp` (`hllc_impl.h:56-72`), and the contact pressure is what is written out as the face pressure.

4.11.3. Numerical method

The body is `hllc_impl.h:17-136`, in the numbered steps the comments mark: the middle-state estimate, the sound speeds, the outer wave speeds, the contact, the two one-sided fluxes at `vxl` (`hllc_impl.h:74-101`), and the weighted combination at `ivx` (`hllc_impl.h:102-130`) which selects by the sign of the speeds rather than by a branch.

The face pressure is set at `face_pressure` (`hllc_impl.h:72`), so like LMARS this solver can drive the curved-grid pressure force and the well-balanced path.

4.11.4. Code

Table 4.20.: Code map of HLLC.

what	code link	symbol	switch / condition
the solver body	`hllc_impl.h: 17-136`	`hllc_impl`	`type: hllc`
PVRS middle state	`hllc_impl.h: 29-38`	`Step 2`	
outer wave speeds	`hllc_impl.h: 48-54`	`bp`	
contact speed and pressure	`hllc_impl.h: 56-72`	`Step 5`	
face pressure written out	`hllc_impl.h: 72`	`face_pressure`	caller asked
registration	`riemann_solver.cpp: 60-61`	`HLLCSolver`	

4.11.5. Tests

Table 4.21.: Tests of HLLC.

test	asserts	tolerance
`tests/test_riemann.cpp`	the solver against reference states	per test

No check in this chapter ports HLLC: the claims it would assert, consistency and the contact resolution, are not measured anywhere in the repository either, which is recorded below rather than papered over.

4.11.6. Limits

1. **Untested properties.** That this implementation resolves an isolated contact exactly, and that it is consistent in the sense `[C5]` checks for LMARS, are both properties of textbook HLLC. Neither is asserted by a test here and neither is measured in this chapter. They should be read as expected, not as verified.
2. **No moist wave speeds.** The PVRS estimate uses one ratio of specific heats per side. For a moist mixture the relevant speed is not that of either constituent, and no source bounds the resulting error.

4.12. Roe (`type: roe`)

4.12.1. Summary

> **At a glance**
>
> - A linearised solver: the centred flux minus a characteristic decomposition of the jump.
> - Switch: `type: roe`, and it is the YAML default when no type is named.
> - Roe averages use the square-root density weighting; species rows ride on the entropy wave.
> - Writes a face pressure.
> - No entropy fix: the wave speeds are used as absolute values with a floor, not corrected at a sonic point.

Roe's solver linearises the problem about an average state chosen so that the linearised jump reproduces the exact flux difference. The flux is then the average of the two physical fluxes minus the sum over waves of their strength times their speed, which is where the upwinding comes from.

4.12.2. Derivation

The averaging is the square-root weighting, at `sqrtdl` (`roe_impl.h:39-48`): each quantity is weighted by the square root of its density so that the linearised system reproduces the jump exactly. The sound speed follows from the averaged enthalpy and velocity at `cs_sq` (`roe_impl.h:117-118`), with a floor applied through `roe_max` so that a non-positive argument cannot reach the square root.

The three wave strengths at `alpha_m` (`roe_impl.h:137-140`) are the familiar ones: two acoustic strengths carrying the velocity jump against the pressure jump over the squared sound speed, and the two tangential momentum strengths riding on the entropy wave.

The species rows are the part specific to snapy. Each gets its own entropy-wave strength at `alpha_n` (`roe_impl.h:155-161`), built as the jump in that species' partial density minus the pressure jump times the species' share. That construction is what keeps a species from being generated at a contact, and no source in the repository derives it; it is recorded here from the code.

4.12.3. Numerical method

The body is `roe_impl.h:24-217`. It assembles the centred flux, then subtracts half the sum of each wave's absolute speed times its strength times its right eigenvector, with the mass and species rows at `alpha_n` (`roe_impl.h:149-161`).

The absolute speeds at `spd_0` (`roe_impl.h:127-129`) are taken directly, with no entropy fix. At a sonic point one of the three speeds passes through zero and its dissipation vanishes with it, which is the classical condition under which a Roe solver can admit an expansion shock. Nothing in the code detects that case.

The face pressure is written at `face_pressure` (`roe_impl.h:119-124`).

4.12.4. Code

Table 4.22.: Code map of the Roe solver.

what	code link	symbol	switch / condition
the solver body	`roe_impl.h: 24-217`	`roe_impl`	`type: roe`, the YAML default
square-root averaging	`roe_impl.h: 39-48`	`sqrtdl`	
sound speed with a floor	`roe_impl.h: 117-118`	`cs_sq`	
face pressure written out	`roe_impl.h: 119-124`	`face_pressure`	caller asked
wave speeds, no entropy fix	`roe_impl.h: 127-129`	`spd_0`	
acoustic and shear strengths	`roe_impl.h: 137-140`	`alpha_m`	
per-species entropy strengths	`roe_impl.h: 155-161`	`alpha_n`	tracers present
registration	`riemann_ solver.cpp: 56-57`	`RoeSolver`	

4.12.5. Tests

Table 4.23.: Tests of the Roe solver.

test	asserts	tolerance
`tests/test_riemann.cpp`	the solver against reference states	per test

4.12.6. Limits

> **Limits**
>
> - No entropy fix. At a sonic point the dissipation of the vanishing wave vanishes with it, and a Roe solver in that state can admit an expansion shock. No test in the repository exercises a transonic rarefaction.

1. **Default by omission.** This is the solver a deck gets when it names none, while every shipped example names LMARS. The two have different dissipation, so an unset key is a silent change of scheme.
2. **Floors in place of a fix.** The `roe_max` floors keep the square root defined but say nothing about the accuracy of the linearisation where they bind.

4.13. Shallow-water Roe and plume-roe (`type: shallow-roe`, `plume-roe`)

4.13.1. Summary

> **At a glance**
>
> - Two special-purpose solvers for equation sets that are not the compressible one.
> - Switches: `type: shallow-roe` for the shallow-water system, `type: plume-roe` for the plume system.
> - Shallow-roe carries a height and two momenta, with no energy row and no face pressure.
> - Plume-roe is not a Roe solver: it falls back to a Lax-Friedrichs flux.

Both of these exist for equation sets outside the main compressible path, and both are registered through the same factory as the others. They are grouped here because neither can be selected for a compressible deck and neither carries the face pressure the curved-grid path needs.

4.13.2. Derivation

The shallow-water system has no internal energy, so the wave structure reduces to two gravity waves around a height-weighted average. The body at `shallow_roe_impl.h:15-77` uses the same square-root weighting as Section 4.12, with the height taking the place of the density at `sqrtdl` (`shallow_roe_impl.h:28-34`).

That the flux covariance of Section 4.18 excludes the shallow-water equation of state follows directly: the correction acts on an enthalpy flux, and this system has no row to correct.

Plume-roe is named for Roe and is not one. `forward` at `plume_roe.cpp:45-103` builds the two physical fluxes at `calculate_flux` (`plume_roe.cpp:14-25`) and combines them with a Lax-Friedrichs average at `plume_roe.cpp:26-40`: the centred flux minus half the maximum signal speed times the jump. That is the most dissipative of the usual choices and makes no use of the characteristic structure. The name is misleading and is recorded as such.

4.13.3. Numerical method

Neither solver writes a face pressure, so neither inherits the pressure-force path of Section 4.15. On a curved grid that is not a restriction for shallow water, which is solved on the sphere through its own geometry, but it does mean that the `prim` and face-pressure arguments of the coordinate `forward` are unused whenever these solvers run.

4.13.4. Code

Table 4.24.: Code map of the two special-purpose solvers.

what	code link	symbol	switch / condition
shallow-water body	`shallow_roe_impl.h:15-77`	`shallow_roe_impl`	`type: shallow-roe`
height-weighted averaging	`shallow_roe_impl.h:28-34`	`sqrtdl`	
plume forward	`plume_roe.cpp:45-103`	`forward`	`type: plume-roe`
its physical flux	`plume_roe.cpp:14-25`	`calculate_flux`	

what	code link	symbol	switch / condition
its Lax-Friedrichs average registration	`plume_roe.cpp:26-40` `riemann_solver.cpp:64-67`	`lax_friedrichs_flux` `ShallowRoeSolver`	

4.13.5. Tests

Table 4.25.: Tests of the two special-purpose solvers.

test	asserts	tolerance
`tests/run_shallow_xy.cmake`	a shallow-water deck end to end	reference
`tests/test_shallow_splash.py`	a shallow-water splash case	per test

No test selects `plume-roe`.

4.13.6. Limits

> **Limits**
>
> - `plume-roe` is a Lax-Friedrichs flux under a name that promises a characteristic solver. A reader choosing it for its dissipation properties will get the opposite of what the name suggests.

1. **Untested.** Nothing in the repository runs the plume solver, so its behaviour is recorded from reading alone.
2. **No face pressure.** Neither solver can drive the curved-grid pressure force or the well-balanced path of chapter 5.

4.14. Single-valued x_1 seam fluxes

4.14.1. Summary

> At a glance
>
> - Two ranks sharing an x_1 face each compute their own flux there; both are replaced by the average.
> - Switch: none. Active whenever the x_1 axis is split across processes.
> - Restores the telescoping of Section 4.2, which a two-valued face breaks.
> - A same-process seam is not averaged: after the ghost exchange the two sides already agree.
> - The face pressure and the background mass flux travel in the same message.

The conservation argument of Section 4.2 depends on every interior face carrying exactly one number, so that it cancels between the two cells it separates. Across a process boundary that is not automatic: each rank reconstructs from its own ghost cells and solves its own Riemann problem, and the two answers need not agree bit for bit. The difference is a source or sink of mass, momentum and energy sitting on the seam.

4.14.2. Derivation

There is nothing to derive beyond the telescoping identity itself. If the two ranks book $F^{(0)}$ and $F^{(1)}$ on the same face, the pair of cells on either side exchange $A(F^{(0)} - F^{(1)})$ that no other cell balances, so the global sum is no longer the boundary flux. Setting both to any common value restores it, and the average is the choice that is symmetric in the two ranks.

It is worth being precise about what this fixes and what it does not. It makes the scheme conservative; it does not make it *right*. If the two reconstructions disagree, the averaged flux is no better than either, and the disagreement has been hidden rather than removed. The size of that disagreement is not measured anywhere in the repository, and this section does not claim it is small.

A same-process seam is handled differently: the reference ghost exchange has already given both sides the same inputs, so the two face states agree and no averaging is needed. That is an argument about the ghost exchange, recorded in the code comment at `playout` (`hydro_forward.cpp:425-440`), not a measurement.

4.14.3. Numerical method

The exchange is at `_flux1` (`hydro_forward.cpp:425-508`). Two tags distinguish the two travel directions, at `kSeamFluxUpTag` (`hydro_forward.cpp:447-448`), so an upward and a downward payload on the same rank cannot be confused.

Everything that lives on the seam face travels in one message rather than several. The packing at `pack` (`hydro_forward.cpp:460-468`) concatenates the flux rows with the face pressure and the background mass flux when those are present, and the unpacking at `hydro_forward.cpp:468-480` takes them apart in the same order. That matters for correctness

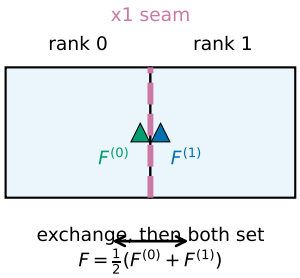


Figure 4.7.: A process seam in x1. Both ranks own the face on the dashed reddish-purple seam line and each reconstructs its own flux there, the green and blue triangles. Each sends its slab to the other and both then set the face to the arithmetic average, so the two cells on either side book the same number and the sum telescopes.

and not only for cost: the face pressure feeds the geometric source of Section 4.15 and the background mass flux feeds the gravity work of chapter 6, so if those were averaged on a different schedule from the flux they would stop being consistent with it.

4.14.4. Code

Table 4.26.: Code map of the seam averaging.

what	code link	symbol	switch / condition
why, and the same-process case	`hydro_forward.cpp:425-440`	`seam`	

what	code link	symbol	switch / condition
the exchange	`hydro_forward.cpp: 425-508`	`_flux1`	more than one x_1 block per axis
the two tags	`hydro_forward.cpp: 447-448`	`kSeamFluxUpTag`	
packing	`hydro_forward.cpp: 460-468`	`pack`	
unpacking	`hydro_forward.cpp: 468-480`	`unpack`	

4.14.5. Tests

Table 4.27.: Tests of the seam averaging.

test	asserts	tolerance
`tests/test_x1_seam_split_mp.cpp`	a column split across two ranks matches the one-block answer, for the switch arms	`1e-13`
`tests/test_x1_seam_split.cpp`	the in-process split, which this scheme does not average	`1e-13`

4.14.6. Limits

⚠ Limits

- Averaging masks a disagreement between the two reconstructions rather than removing it. Conservation is restored; accuracy at the seam is not analysed anywhere.

1. **Coverage.** Multi-process x_1 seams of the well-balanced reference are exercised only through these split tests, with no separate check of the seam flux itself.
2. **Hypothesis, not measured.** That a same-process seam needs no averaging rests on the two sides receiving identical ghost data. That is stated in the code comment and is consistent with the in-process seam test passing bitwise, but no test isolates the claim.

4.15. Geometric sources on the spherical-polar grid

4.15.1. Summary

ℹ At a glance

- The part of the curvature that is not a difference of face fluxes, added after the divergence.
- Switch: none; it is what the spherical-polar coordinate does instead of the Cartesian pass-through.
- The radial source carries the centrifugal terms of the two tangential velocities.
- The radial pressure force uses the face pressures when a solver provides them.
- Areas and volume are exact integrals, not midpoint values.

In curvilinear coordinates the divergence of a tensor is not a difference of face fluxes alone: the basis vectors turn from face to face, and the leftover is a source. The coordinate object adds it on top of the same divergence every grid uses, which is why Section 4.2 can be written once for all geometries.

4.15.2. Derivation

The geometric factors are built once in the coordinate's own buffers and the source is assembled at `forward` (`spherical_polar.cpp:224-250`). The radial source is the centrifugal pair

$$S_1 = \frac{A_{23}}{V}\,\rho\,(v_\theta^2 + v_\phi^2), \tag{4.7}$$

at `coord_src1_i` (`spherical_polar.cpp:243-244`): a parcel moving tangentially at constant radius needs an inward force to stay there, and in a radial momentum equation written in fluxes that force appears as a source. The coefficient is a ratio of the cell's own tangential area to its volume, not a nominal $1/r$, so the identity survives on a stretched radial grid.

The areas and volume themselves are exact integrals of the metric, not midpoint approximations: `face_area1` (`spherical_polar.cpp:178-183`) for the radial face, `cell_volume` (`spherical_polar.cpp:200-211`) for the volume, which carries the exact $(r_+^3 - r_-^3)/3$. `[C3]` shows what that buys: the midpoint form $r^2 h$ would be short by $h^3/12$ per unit solid angle, so the exact form removes a second-order volume error that would otherwise appear in every tendency. `[C2]` then confirms that the areas and volume together reproduce $\nabla\cdot(r\hat{r}) = 3$ to 3.6×10^{-15} over a test block, which is the identity the cubed-sphere volume test asserts cell by cell.

The radial pressure force is the part with a choice in it. With a solver that returns a face pressure the force can be built from those face values; without one it is a plain difference. The code comment at `exact_x1`

(spherical_polar.cpp:246-251) states the consequence carefully: with the face pressures the pressure force is the r^2-weighted average of the gradient, which balances $g\rho$ for a hydrostatic column of r^2 means, while the plain difference balances $g\langle\rho\rangle$ instead. Which of those is wanted is the subject of chapter 5, and the exact-centroid switch that selects it belongs there, not here.

4.15.3. Numerical method

The source is added after CoordinateImpl::forward has produced the divergence, at spherical_polar.cpp:233, so the flux part and the source part are never mixed. The shallow-water equation of state takes a different branch at eos_type (spherical_polar.cpp:245), since it has no pressure row of the kind the force needs.

4.15.4. Code

Table 4.28.: Code map of the spherical-polar geometric source.

what	code link	symbol	switch / condition
the source assembly	spherical_polar.cpp:224-250	forward	spherical-polar
divergence first	spherical_polar.cpp:233	CoordinateImpl	
centrifugal radial source	spherical_polar.cpp:243-244	coord_src1_i	
the exact radial face area	spherical_polar.cpp:178-183	face_area1	

what	code link	symbol	switch / condition
the exact cell volume	`spherical_polar.cpp: 200-211`	`cell_volume`	
face moments for section 4B	`spherical_polar.cpp: 213-222`	`face_moment2_x1`	

4.15.5. Tests

Table 4.29.: Tests of the spherical-polar geometric source.

test	asserts	tolerance
`tests/test_coordinate.cpp`	the geometry against reference formulas, and the frame projections	allclose defaults
`tests/test_radial_face_moments.cpp`	the curved face centroid shift and second moment equal the exact rationals	`1e-14`
this chapter's check	`[C2]` the radial identity to round-off; `[C3]` the exact volume against the midpoint form	round-off, symbolic

4.15.6. Limits

1. **The pressure force depends on the solver.** A solver that returns no face pressure gets the plain difference, so the balance the column achieves depends on a key in a different section of the deck. The consequences are chapter 5's subject.
2. **Only the radial source is covered here.** The tangential sources and their metric coefficients are part of the same `forward` and are described with the grid they belong to in chapter 3.

4.16. Geometric sources on the gnomonic cubed sphere

4.16.1. Summary

> At a glance
>
> - The same structure as the spherical-polar source, on a grid whose two tangential directions are not orthogonal.
> - Switch: none; selected by the `gnomonic-equiangle` coordinate type.
> - The solid angle is exact, from a corner difference, and the radial integral is exact.
> - The non-orthogonal metric adds cosine cross terms the spherical-polar source does not have.
> - The cell centre is the arithmetic mid-radius, which is not the centroid of the cell measure.

The cubed sphere replaces the polar singularity with six panels of an equiangular gnomonic grid. Within a panel the two tangential coordinates

are angles about two different axes, and they are not orthogonal: the metric has an off-diagonal term, and the geometric source carries cross terms because of it.

4.16.2. Derivation

The exactness of the geometry is the part worth stating. The solid angle of a cell is a difference of corner values at `solid_angle_kj` (`gnomonic_equiangle.cpp:128-137`), which is exact for a gnomonic quadrilateral rather than a quadrature of it, and the radial face area and volume are that solid angle times the exact radial factors at `gnomonic_equiangle.cpp:207-208` and `gnomonic_equiangle.cpp:219-226`. Because both use the *same* solid angle, the radial identity `[C2]` checks holds on this grid for the same reason it holds on the spherical-polar one, and the six panels' volumes sum to the shell volume exactly, which is what the cubed-sphere volume test asserts.

The source itself is at `forward` (`gnomonic_equiangle.cpp:415-473`), with the radial part at `src1` (`gnomonic_equiangle.cpp:443-460`) and the two tangential parts at `src2` (`gnomonic_equiangle.cpp:463-470`). The metric coefficients they use are built once at `x_ov_rD_kji` (`gnomonic_equiangle.cpp:142-147`). No source in the repository derives these cross terms; this section records their form from the code and refers the geometry itself to chapter 3.

4.16.3. Numerical method

The radial pressure force is the same plain difference of face pressures as on the spherical-polar grid, so everything said in Section 4.15 about the dependence on the solver applies here unchanged.

One property deserves care. The cell centre on this grid is the arithmetic mid-radius, at `x1v` (`gnomonic_equiangle.cpp:36`), and not the centroid

of the cell's own r^2 measure. A cell average and a value at that centre therefore differ at second order, which matters whenever initial conditions are written as point values at the stored centre and read as cell averages. The derivation note records this as a consistency condition on how a deck's initial state is prepared, not as an error in the grid.

The exact volume is used unconditionally. A consequence recorded in the pull request that made it so is that the usual switch-off property does not hold on this grid: turning the related corrections off does not return the code bitwise to the previous answer, because the volume itself changed. That is consistent with the code and is recorded here for a reader comparing runs across that change.

4.16.4. Code

Table 4.30.: Code map of the gnomonic geometric source.

what	code link	symbol	switch / condition
exact solid angle	`gnomonic_equiangle.cpp: 128-137`	`solid_angle_kj`	
radial face area	`gnomonic_equiangle.cpp: 207-208`	`face_area1`	
exact cell volume	`gnomonic_equiangle.cpp: 219-226`	`cell_volume`	
metric coefficients	`gnomonic_equiangle.cpp: 142-147`	`x_ov_rD_kji`	

what	code link	symbol	switch / condition
the source	`gnomonic_equiangle.cpp: 415-473`	`forward`	gnomonic-equiangle
mid-radius cell centre	`gnomonic_equiangle.cpp: 36`	`x1v`	

4.16.5. Tests

Table 4.31.: Tests of the gnomonic geometric source.

test	asserts	tolerance
`tests/test_cubed_sphere_cell_volume.py`	six panel volumes sum to the shell volume; the radial identity per cell	`1e-12`, rest `1e-6`
`tests/test_hydrostatic.cpp`	a six-panel isentropic shell stays at rest over 100 steps	$\max\|v_1\| < 10^{-8}$
`tests/test_coordinate.cpp`	the radial source uses the face pressure, and preserves a face-pressure gradient	`1e-8`, `1e-6`

4.16.6. Limits

Limits

- The stored cell centre is the mid-radius, not the centroid of the cell measure. Initial conditions written as point values there are consistent with the scheme only if they are intended as cell averages.
- The exact volume is unconditional, so a run before and after that change does not agree bitwise even with the related corrections switched off.

1. **The exact-centroid switch does nothing here.** It is defined for the radial measure of the spherical-polar grid and has no effect on this one.
2. **No derivation on record.** The cross terms of the source are recorded from the code only. A reader wanting to check them has to re-derive the gnomonic metric, which chapter 3 sets out.

4.17. 4B. Face averages against cell averages: the $O(\Delta x_1^2)$ corrections

4.18. The x_2 and x_3 face-flux covariance and centroid terms (`SNAP_FLUX_COVARIANCE`)

4.18.1. Summary

> **At a glance**
>
> - A horizontal face flux built from cell values is short by two terms at $O(\Delta x_1^2)$: a covariance and a centroid offset.
> - Switch: `SNAP_FLUX_COVARIANCE`, a build-time flag.
> - The covariance term is the x_1 second moment times the product of two x_1 derivatives.
> - The centroid term vanishes identically in Cartesian and is of relative size L/r on a curved grid.
> - Both are added to the face flux, so they telescope exactly like the flux they correct.
> - Zero at rest by construction, bitwise.

A finite-volume flux through an x_2 or x_3 face is the *average over that face* of the point flux. The solver does not compute that: it computes a flux *from cell values*, which are averages over the cell. The two differ, and on a stratified atmosphere they differ at an order that matters, because the x_1 direction is the one carrying an $O(1)$ background gradient across the face.

4.18.2. Derivation

The whole scheme rests on one identity, and `[C1]` establishes it symbolically rather than by assertion: for any two smooth fields,

$$\langle fg\rangle = \langle f\rangle\langle g\rangle + \frac{h^2}{12}\, f' g' + O(h^4), \tag{4.8}$$

where the averages are over a cell of width h in x_1. The check expands both sides and finds the h^0 and h^2 parts cancel identically, leaving a remainder whose leading coefficient it prints. A product of averages is not the average of a product, and Equation 4.8 is exactly how much they differ by.

The coefficient $h^2/12$ is the x_1 second central moment of the face's own measure. `[C2]` computes it for both geometries: it is exactly $h^2/12$ in Cartesian, and for an x_2 or x_3 face of a spherical-polar cell, whose measure is r, it is $h^2/12 - h^4/(144r^2)$, which tends to $h^2/12$ as r grows. The code reads it from the coordinate object rather than assuming the Cartesian value.

The second term has no analogue in a flat geometry. A cell stores the average over the *cell* measure, whose x_1 centroid is r_v, while the flux needs the average over the *face* measure, whose centroid is r_c. `[C3]` computes both: in Cartesian the two measures are the same and the offset is identically zero, while on a spherical-polar grid $r_v - r_c = h^2/(12r) + O(h^4)$. So the correction is

$$\mathrm{d}F = \sigma_1^2\, \rho\, h_1 (u_n)_1 \;-\; (r_v - r_c)\, \partial_1 (\rho h u_n)\,, \tag{4.9}$$

the first a covariance and the second a centroid offset. Their ratio is of order L/r with L the local gradient scale, so the centroid term is not a small correction to the covariance term on a curved grid: it is comparable, and dropping it would leave the scheme no better than it started.

Two exact properties follow and `[C5]` checks the first. Every term of Equation 4.9 carries a factor of the face-normal velocity or its x_1 difference, both exactly zero at rest, so a balanced state keeps its exact zero tendency bitwise. The covariance term is also zero for an isothermal state; the centroid term is not, so that second exact zero holds in Cartesian only.

The rows are not all alike, and the code comment at `sigma1` (`hydro_forward.cpp:132-147`) sets out the breakdown that the all-rows treatment rests on. Because cell values are density-weighted, the total mass flux is linear in the stored state and has no covariance at all; a tracer row carries the covariance of its own mass fraction with the velocity; the dry mass row carries minus the sum of the tracer parts; the energy row carries the covariance of the specific enthalpy with the velocity; and the momentum rows carry no covariance, only their centroid part, taken in the face-local frame. That last choice is what keeps rest exact: the pressure part of the momentum centroid term is mirrored in the geometric source of Section 4.15.

What is deliberately left out is recorded in the same comment: the velocity-squared parts of the momentum and energy gaps, which are the scheme's ordinary second-order truncation, and the Hessian of p/ρ along the gradient, estimated there at about 10^{-3} of the energy term. The covariance along the *other* in-face coordinate is dropped too, because only x_1 carries an $O(1)$ background gradient and the horizontal one is quadratic in the perturbation.

4.18.3. Numerical method

The correction is `_flux_covariance` at `hydro_forward.cpp:54-150`. It builds the arithmetic mean of the two face states, forms the enthalpy density from the equation of state's own call so that every species' energy offset is included, and projects the mean state into the face-local frame on its own copy at `prim2local2_` (`hydro_forward.cpp:106-111`) rather than relying on whether the solver projected in place.

The moments come from the coordinate at `face_moment2_x1` (`hydro_forward.cpp:113-128`): the second moment and the centroid shift are both coordinate methods, so the same code is correct on every grid. The per-row assembly uses two small closures, a covariance and a centroid rule, at `cov` (`hydro_forward.cpp:149-158`).

Two guards sit at the top. A column with fewer than three cells returns nothing at `n1` (`hydro_forward.cpp:88-92`), because a centred x_1 difference needs both neighbours; and the shallow-water equation of state returns nothing at `peos` (`hydro_forward.cpp:93-94`), because it has no internal energy row to correct. The correction is applied to the x_2 and x_3 fluxes at `dcov` (`hydro_forward.cpp:599-600`) and `dcov` (`hydro_forward.cpp:622-623`).

4.18.4. Code

Table 4.32.: Code map of the horizontal flux covariance.

what	code link	symbol	switch / condition
the correction	`hydro_forward.cpp:54-150`	`_flux_covariance`	`SNAP_FLUX_COVARIANCE`
the two-term statement	`hydro_forward.cpp:57-86`	`sigma1`	
guards	`hydro_forward.cpp:88-94`	`n1`	$n_1 < 3$, shallow water
face-local projection	`hydro_forward.cpp:106-111`	`prim2local2_`	

what	code link	symbol	switch / condition
moments from the coordinate	`hydro_forward.cpp:113-128`	`face_moment2_x1`	
the per-row breakdown	`hydro_forward.cpp:132-147`	`sigma1`	
the two closures	`hydro_forward.cpp:149-158`	`ctr`	
applied to x_2	`hydro_forward.cpp:599-600`	`dcov`	
applied to x_3	`hydro_forward.cpp:622-623`	`dcov`	

4.18.5. Tests

Table 4.33.: Tests of the horizontal flux covariance.

test	asserts	tolerance
`tests/test_flux_covariance_rows.py`	the per-row correction	per test
`tests/test_flux_covariance_seams.py`	the correction across a seam	per test
`tests/test_horizontal_flux_covariance.py`	the horizontal correction	per test

test	asserts	tolerance
this chapter's check	`[C1]` the identity; `[C2]` the second moment on both grids; `[C3]` the centroid offset; `[C5]` bitwise zero at rest	symbolic, bitwise

The check is `src/snapy_report/ch04/covariance_check.py`, output `src/snapy_report/ch04/covariance_check.out`, 6 of 6 claims passing.

4.18.6. Limits

> **Limits**
>
> - The exact zero for an isothermal state holds in Cartesian only: the centroid term does not vanish there on a curved grid.
> - A column with fewer than three x_1 cells gets no correction at all, silently.

1. **Terms left out on purpose.** The velocity-squared parts and the pressure Hessian are omitted. The comment estimates the Hessian at about 10^{-3} of the energy term; that estimate is recorded in the code and is not re-derived or measured here.
2. **The other in-face coordinate.** Dropping its covariance assumes only x_1 carries an $O(1)$ gradient. That is a statement about the intended application, a stratified atmosphere, not about the discretization, and it would not hold for a problem with a strong horizontal background gradient.

3. **Order not measured here.** This chapter's check establishes the identity, the coefficients and the exact zeros. It does not measure the order of the corrected horizontal flux end to end; the repository's own row and seam tests are the evidence for that, and no convergence ladder for it is committed.

4.19. The x_1 mass-flux covariance (`SNAP_X1_MASS_COVARIANCE`)

4.19.1. Summary

> **At a glance**
>
> - The stored velocity is m_1/ρ, which is not the cell average of the velocity in a stratified column.
> - Switch: `SNAP_X1_MASS_COVARIANCE`, a build-time flag, default off.
> - The correction subtracts $\frac{h^2}{12}\rho_1 w_1/\rho$ from the reconstructed velocity before the solver runs.
> - It needs at least three x_1 cells per block, which the code enforces.
> - Next to a reflecting wall the density slope is one-sided, because the mirrored ghost is not the stratified column.

This is the x_1 member of the same family as Section 4.18, and it comes from the same identity. The difference is where the error enters: not in a face average across the column, but in the primitive variable the code stores and reconstructs.

4.19.2. Derivation

The code stores conserved variables as cell averages and forms the primitive velocity as m_1/ρ. Written out, that is $\langle \rho w \rangle / \langle \rho \rangle$, and by Equation 4.8

$$\frac{\langle \rho w \rangle}{\langle \rho \rangle} = \langle w \rangle + \frac{h^2}{12} \frac{\rho' w'}{\rho} + O(h^4). \tag{4.10}$$

So the quantity the reconstruction is handed is not the cell average of the velocity: in a stratified column, where ρ' is $O(1)$, it is the cell average plus a second-order term. Reconstructing it and multiplying by the face density puts that term straight into the face mass flux.

`[C4]` measures Equation 4.10 directly on an exponential density profile with a sinusoidal velocity, comparing the stored m_1/ρ with the exact cell average of w: over $n_1 = 16 \to 128$ the gap falls as $2.89\times10^{-3} \to 4.56\times10^{-5}$, a fitted order of 2.00, and after subtracting the discrete estimate of the second term it falls as $1.62 \times 10^{-4} \to 4.07 \times 10^{-8}$, a fitted order of 3.99. The correction does what the identity says it should: second order becomes fourth.

That is the whole scheme. It is worth noting what it does *not* claim. It recovers the cell average of the velocity; whether the resulting face mass flux is fourth order then depends on the reconstruction that follows, which for most types is lower order than that. The correction removes one specific second-order error, not every one.

The wall is the one place the stencil changes. A reflecting boundary mirrors the density into the ghost cells evenly, and an even mirror is not the continuation of a stratified column, so a centred difference in the first interior cell would read a density slope that is not there. The code substitutes a one-sided second-order difference over the first three interior cells, and `[C6]` confirms that difference is second order, falling as $2.69\times10^{-3} \to 4.44\times10^{-5}$ with a fitted order of 1.98. The corrected velocity is then mirrored *oddly*

into the ghosts, which is the correct parity for a velocity at a reflecting wall.

4.19.3. Numerical method

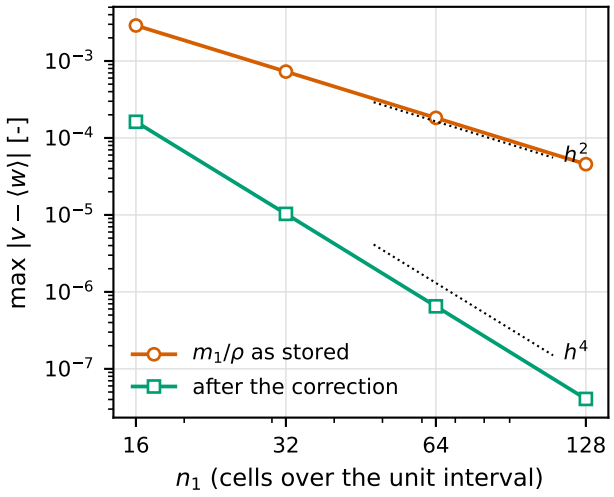


Figure 4.8.: The gap between the stored velocity and the cell average of the velocity, against resolution. Vermillion circles are the stored m1 over rho, which differs from the cell average at second order in a stratified column; green squares are the same quantity after the correction of this section, at fourth order. The dotted lines are reference slopes of two and four. Data from the committed output of the chapter's covariance check.

The block is at `x1_mass_covariance` (`hydro_forward.cpp:314-344`), inside the x_1 sweep and before the Riemann solver runs. The two derivatives use the centred helper at `d1_centred` (`hydro_forward.cpp:40-52`), which copies its end values rather than extrapolating. The correction itself is the single line at `dx1f` (`hydro_forward.cpp:334`), and it is applied to

the reconstructed velocity, so the solver and everything downstream see the corrected value.

The one-sided wall slope is at `one_sided` (`hydro_forward.cpp:320-332`), applied only at a boundary that is both physical and not outflow; an outflow boundary keeps the centred difference, since there is no mirror to distrust. The odd mirror of the corrected velocity follows at `narrow` (`hydro_forward.cpp:335-343`).

The three-cell requirement is enforced where the switch is read, at `TORCH_CHECK` (`hydro.cpp:104-112`): a block with fewer than three x_1 cells fails with a message naming the switch rather than silently reading a ghost. The switch itself is read once per process at `x1_mass_covariance` (`hydro.cpp:231-239`).

4.19.4. Code

Table 4.34.: Code map of the x_1 mass-flux covariance.

what	code link	symbol	switch / condition
the correction block	`hydro_forward.cpp:314-344`	`x1_mass_covariance`	`SNAP_X1_MASS_COVARIANCE`
the centred derivative	`hydro_forward.cpp:40-52`	`d1_centred`	
the correction itself	`hydro_forward.cpp:334`	`dx1f`	
one-sided slope at a wall	`hydro_forward.cpp:320-332`	`one_sided`	physical, not outflow

what	code link	symbol	switch / condition
odd mirror of the result	`hydro_forward.cpp: 335-343`	`flip`	physical, not outflow
the three-cell check	`hydro.cpp: 104-112`	`TORCH_CHECK`	
the switch, read once	`hydro.cpp: 231-239`	`x1_mass_covariance`	

4.19.5. Tests

Table 4.35.: Tests of the x_1 mass-flux covariance.

test	asserts	tolerance
`tests/CMakeLists.txt` seam arm	a block with fewer than three x_1 cells is refused	the check fires
this chapter's check	`[C4]` second order becomes fourth; `[C5]` bitwise zero at rest; `[C6]` the one-sided wall slope is second order	fitted orders, bitwise

4.19.6. Limits

> **Limits**
>
> - Default off. A deck that does not set the flag carries the second-order gap of Equation 4.10 in every stratified column.
> - At least three x_1 cells per block are required, and the run fails rather than degrading if that is not met.

1. **One error, not all of them.** The correction recovers the cell average of the velocity. The order of the face mass flux that follows is set by the reconstruction, and for `dc` or `plm` that is lower than fourth. Figure 4.8 should be read as a measurement of the gap, not of the flux.
2. **Outflow boundaries keep the centred stencil.** The one-sided slope is applied only at reflecting walls. Whether the mirrored ghost at an outflow boundary is a good continuation of the column is not analysed in any source, and this section does not claim it is.
3. **The x_1 centroid conversion is elsewhere.** The radial member of this family, which converts between the r^2 cell measure and the face measure in x_1, is inseparable from the well-balanced reference and is covered in chapter 5.

5. Hydrostatic and well-balanced treatment

A stratified atmosphere at rest is the state snapy's x_1 sweep has to leave alone, and it is the hardest one: its pressure and density change by orders of magnitude along x_1, while the motions of interest are small perturbations of it. This chapter describes how the x_1 reconstruction keeps such a column at rest to round-off and how close the reference it uses comes to the column's own profile. It covers, in the order the code builds them: the well-balanced x_1 reconstruction, which reconstructs perturbations about a discretely hydrostatic reference; the reference kernel, which builds that reference from a top-down pressure scan; its closure at a physical wall; its continuity across x_1 block seams; the hydrostatic mode, which replaces gravity by the cell's own pressure difference; `balance_column`, which projects an initial column onto the scheme's discrete balance; the fourth-order reference `SNAP_WB_REF4`; the r^2-exact x_1 maps `SNAP_X1_CENTROID_EXACT` of spherical-polar grids; what is left of the $1/R$ term after them; and two legacy wall routines. The gravity work these schemes are used with is Chapter 6 (Chapter 6); the switches and their tested combinations are Chapter 12.

All derivations of this chapter are checked by one executable check, `src/snapy_report/ch05/wbref_check.py`, whose output `wbref_check.out` carries the claim tags `[C1]`–`[C21]`; claim `[C11]` is an independent rebuild of the default reference from the equations of this chapter that reproduces the numbers snapy's own wall test prints.

Table 5.1.: The schemes of this chapter. "Order" is the order of the face density reference ρ_{sf} against the exact profile where a number is given; "rest exact" means a discretely balanced column stays at rest to round-off. Full ctest names and tolerances are in each scheme's Tests layer and in the chapter's catalogue in Chapter 15.

scheme	switch	default	order	main test
WB x_1 reconstruction (Section 5.1)	gravity on	on	rest exact	`test_face_floor`
Reference kernel (Section 5.2)	with the WB path	on	2 (ρ_{sf})	`test_hydro_ref_x1`
Wall clamp and continuation (Section 5.3)	`wb-wall-clamp`	on	2 at walls	`test_wb_ref_wall`
Seam continuity (Section 5.4)	x_1 split	on	exact	`test_pref_local_seam`
Hydrostatic mode (Section 5.5)	`non-hydrostatic`	1 (off)	rest exact	`test_hydrostatic`
`balance_column` (Section 5.6)	API call	not called	fixed point	`test_balance_column`
Fourth-order reference (Section 5.7)	`SNAP_WB_REF4`	off	4 (ρ_{sf})	`test_wb_ref4_order`

scheme	switch	default	order	main test
r^2-exact x_1 maps (Section 5.8)	`SNAP_X1_CENTROID_EXACT`	off	rest exact	`test_x1_centroid_rest`
$1/R$ remainder (1)	none	—	$O(h^2/R)$	none
Legacy wall routines (Section 5.10)	none	unreachable	—	none

5.1. Well-balanced x_1 reconstruction: perturbations about a hydrostatic reference (gravity on)

5.1.1. Summary

> **i** At a glance
>
> **What it does.** Reconstructs $p - p_{\mathrm{ref}}$ and $\rho - \rho_{\mathrm{ref}}$ along x_1 instead of p and ρ, then adds the reference face values back, so a discretely hydrostatic column produces no x_1 force. **Switch.** None. On whenever $g_1 \neq 0$, the state has a pressure row and the EOS is not `shallow-water`; also under an x_1 block split. **Requires.** The reference kernel (Section 5.2). **Order.** Exact rest balance; the face states keep the order of the reconstruction. **Defining equation.** Equation 5.1.

A column in hydrostatic balance has pressure and density that fall by orders of magnitude along x_1. A high-order reconstruction of p itself returns

face values whose difference across a cell is not exactly $\rho g_1 \Delta x_1$, so a resting column feels a spurious vertical force of the size of the reconstruction error and starts to move. snapy removes that force by construction: it builds, on every call, a reference $(p_{\mathrm{ref}}, \rho_{\mathrm{ref}})$ whose face pressures satisfy the discrete hydrostatic balance exactly, reconstructs only the perturbations about it, and restores the reference at the faces. The perturbation of a resting column is a constant, which every reconstruction reproduces, so the restored faces carry exactly the balanced pressure difference.

The scheme has no switch and no default to change: it is the x_1 path whenever gravity acts in x_1. The perturbation ghosts at a physical, non-outflow x_1 wall are filled with even parity; the reconstruction runs with its density and pressure floors off; and a face value that comes out non-positive falls back to the reference pressure or to the density of the adjacent cell. The x_2 and x_3 sweeps have no such treatment.

Aliases. "WB" or "well-balanced reconstruction" in comments and commit messages; the "base" reference of `docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §1.

5.1.2. Derivation

Units: p [Pa], ρ [kg m^{-3}], g_1 [m s^{-2}], x_1 and Δx_1 [m]. This layer uses cell averages and face point values; a cell average carries an overbar only where a point value of the same quantity appears.

The x_1 momentum equation with constant gravity is $\partial_t(\rho v_1) + \partial_1(\rho v_1^2 + p) = \rho g_1$. Integrated over cell i of a Cartesian column, with $p^*_{i\pm1/2}$ the face pressures of the Riemann solver,

$$\frac{\mathrm{d}}{\mathrm{d}t}(\overline{\rho v_1})_i = -\frac{\Delta_i[\rho v_1^2 + p^*]}{\Delta x_1} + \overline{\rho}_i g_1.$$

At rest the advective part vanishes and the cell stays at rest exactly when $p^*_{i+1/2} - p^*_{i-1/2} = g_1 \overline{\rho}_i \Delta x_1$. The reference kernel (Section 5.2) builds face pressures p_{sf} that satisfy this difference identically, Equation 5.3, together with a cell pressure $p_{\mathrm{ref},i}$, a cell density $\rho_{\mathrm{ref},i}$ and a face density ρ_{sf}. The reconstruction then works on the perturbations and restores the reference at the faces:

$$\begin{aligned} p'_i &= \overline{p}_i - p_{\mathrm{ref},i}, \\ \rho'_i &= \overline{\rho}_i - \rho_{\mathrm{ref},i}, \\ p^{L,R}_{i-1/2} &= \mathcal{R}^{L,R}[p']_{i-1/2} + p_{\mathrm{sf},i-1/2}, \\ \rho^{L,R}_{i-1/2} &= \mathcal{R}^{L,R}[\rho']_{i-1/2} + \rho_{\mathrm{sf},i-1/2}, \end{aligned} \tag{5.1}$$

with $\mathcal{R}^{L,R}$ the left and right face values of the configured x_1 reconstruction. The subtraction in Equation 5.1 is done by `_hydro_ref_x1` and the two in-place updates that follow it (`hydro_forward.cpp:288-293`), and the restore by `psf_lo` added back to both states (`hydro_forward.cpp:357-363`).

Suppose now that $p'_i = c$ in every cell, for one constant c, and $v_1 = 0$. Every reconstruction of snapy returns a constant exactly, so $p^L = p^R = p_{\mathrm{sf}} + c$ at every face, the two states agree, and the Riemann solver returns $p^* = p_{\mathrm{sf}} + c$ with zero mass and energy flux. The momentum balance above then reads $-(p_{\mathrm{sf},i+1/2} - p_{\mathrm{sf},i-1/2})/\Delta x_1 + \overline{\rho}_i g_1 = 0$ by Equation 5.3. The discrete rest state of the scheme is therefore

$$\overline{p}_i = p_{\mathrm{ref},i}(\overline{p}, \overline{\rho}) + c \quad \text{in every cell,} \tag{5.2}$$

whatever the density reference is: with $v_1 = 0$ the face density enters no flux. Because p_{ref} is rebuilt from the state, Equation 5.2 is a fixed-point condition on the column, which `balance_column` solves (Section 5.6). On spherical-polar grids the face areas and the geometric pressure source enter the same cancellation; the plain-mean form is Section 5.8. The derivation

follows `docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §§1, 6.

Exact and truncated. The rest balance is exact for any column satisfying Equation 5.2. Away from rest the face states carry the reconstruction's truncation error on the perturbation plus the offset $\rho_{\mathrm{sf}} - \mathcal{R}[\rho_{\mathrm{ref}}]$ of the density reference, which is $O(\Delta x_1^2)$ for the default reference (`[C14]`) and $O(\Delta x_1^4)$ with Section 5.7. The identity used in the cancellation, that the hydrostatic correction and the pressure force cancel at rest, is `[C19]`.

5.1.3. Numerical method

Where things live: $\overline{p}$, $\overline{\rho}$ and the perturbations are cell averages; p_{sf} and ρ_{sf} are face point values at the lower face of each cell; the reconstructed states are face values. The scheme runs in every RK stage, in the x_1 block of `HydroImpl::forward`, explicitly.

Ghosts. At a physical x_1 wall whose boundary function is not an outflow, the n_g ghost cells of p' and ρ' are overwritten with the mirror image of the first n_g interior cells, even parity, while the velocity keeps the odd ghosts of the reflecting boundary (Figure 5.1). A resting perturbation ($p' = c$) is then constant across the wall too. At an x_1 seam the ghosts are the neighbour's cells and the reference is made continuous by Section 5.4; at an outflow boundary the ghosts are left as the boundary function wrote them.

Floors. The reconstruction of the perturbations is called with `floor=false`, because a perturbation is legitimately negative and the density and pressure floors would clamp it; the passive-species rows are still clamped at zero when the EOS limiter is on. After the restore, a non-positive face pressure is replaced by p_{sf}; a non-positive left density at face $i - \frac{1}{2}$ by $\overline{\rho}_{i-1}$ and a non-positive right density by $\overline{\rho}_i$, the cell on the same side. The adjacent cell, and not ρ_{sf}, is used because a reference that overestimates the density aloft turns every floor event at a reflecting top

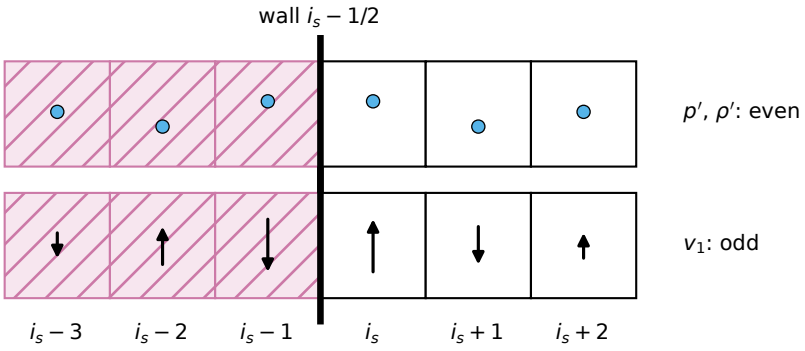


Figure 5.1.: Perturbation ghosts at a reflecting x1 wall (thick black line), x1 increasing to the right. Top row: the perturbations p' and rho' (sky-blue circles) in three interior cells are mirrored with even parity into the three hatched reddish-purple ghost cells. Bottom row: the normal velocity (black arrows) is mirrored with odd parity. A constant perturbation stays constant across the wall, so a resting column sees no wall force.

wall into a large spurious impedance. The shift that builds $\overline{\rho}_{i-1}$ replicates the first cell instead of wrapping around the column. Chapter 7 owns the positivity floors in general.

Order. In the interior the face states have the order of the reconstruction applied to a smooth perturbation, plus the density-reference offset above; at rest the result is exact in interior, wall and seam cells alike. A column that is not at the fixed point Equation 5.2 has a residual force of the size of $p'_i - c$, which is what `balance_column` removes.

5.1.4. Code

Table 5.2.: Code map of the well-balanced x_1 reconstruction.

what	code link	symbol	switch
the gate	`hydro_forward.cpp:262-263`	`wb_x1`	$g_1 \neq 0$, not shallow water
reference and subtraction	`hydro_forward.cpp:288-293`	`_hydro_ref_x1`	
even-parity ghosts	`hydro_forward.cpp:300-312`	`is_outflow`	physical, non-outflow walls
reconstruction without floors	`hydro_forward.cpp:346-348`	`precon1`	
pressure restore and fallback	`hydro_forward.cpp:357-360`	`psf_lo`	

what	code link	symbol	switch
density restore and fallback	`hydro_forward.cpp:362-377`	`rho_below`	
species clamp that stays	`reconstruct.cpp:150-155`	`clamp_min_`	EOS limiter
the non-WB x_1 path	`hydro_forward.cpp:378-384`	`_revise_x1inner_lr`	shallow water, $g_1 = 0$

In the x_1 block of `HydroImpl::forward`, in step order (code indices `is`, `iu` are the first and last interior cell, i_s and i_u):

1. `wb_x1` is set from the gravity, the number of state rows and the EOS type (`hydro_forward.cpp:262-263`).
2. Under `SNAP_X1_CENTROID_EXACT` on spherical-polar grids the input `wx1` becomes the plain-mean view of Section 5.8; otherwise `wx1` is the state itself.
3. `_hydro_ref_x1` returns p_{sf}, p_{ref}, ρ_{sf} and ρ_{ref}, and the two rows of `wx1` are replaced by the perturbations (`hydro_forward.cpp:288-293`).
4. The even-parity ghosts are written where `is_outflow` is false at a physical wall (`hydro_forward.cpp:300-312`).
5. Under `SNAP_X1_MASS_COVARIANCE` the velocity row is corrected before the reconstruction (Chapter 4).
6. `precon1` reconstructs with `floor=false` (`hydro_forward.cpp:346-348`).
7. The rows of `wx1` are restored, `psf_lo` is added to both pressure states with its fallback (`hydro_forward.cpp:357-360`), and the density states get `dsf` with the `rho_below` and own-cell fallbacks (`hydro_forward.cpp:362-377`).

No switch is read here and no warning is issued; the setup checks of the reference are in Section 5.2 and Section 5.7.

5.1.5. Tests

Table 5.3.: Tests of the well-balanced x_1 reconstruction.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_face_floor.cpp` (`test_face_floor.release`), `face_floor_uses_adjacent_density`	dipped column: lower-face mass flux 4.07888; at the dipped face |mass flux| $< 10^{-9}$ and momentum flux > 0.03042	10^{-3} abs; 10^{-9}; bound	the adjacent-density floor gives -6.3×10^{-11}, a ρ_{sf} floor -9.5×10^{-8} (test header)
same, `face_floor_fires_on_an_unresolved_column`	an unresolved column floors to the adjacent density in every arm: flux 2.83191×10^{-8}	10^{-5} relative	pinned value of the wall closure
`tests/test_wb_wall_corner.cpp` (`test_wb_wall_corner.release`)	a uniform-in-x_2 balanced polytrope keeps $\lvert u_2\rvert/c_s$ and $\lvert v\rvert/c_s \leq 10^{-12}$; an x_2 split equals one block	10^{-12}; exact	round-off of a balanced column

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_hydrostatic.cpp` (`test_hydrostatic.release`)	a cubed-sphere hydrostatic atmosphere stays at rest for 100 steps	10^{-8}	round-off margin
this chapter's check, `[C14]`, `[C19]`	density-reference offset; cancellation at rest	per claim	stated in the check

Evidence. All entries above pass at the pin [sha `e894700ff7aee30b52882e5202b16461413780b0`; decks: the test files named; run: `ctest -V` of the chapter's 28 entries on a Release build of the pin, 2026-10-10, CPU, double, 1 rank except the `_mp` entries (2 ranks)]. Printed values from that run: `test_hydrostatic` maximum spurious vertical velocity after 100 steps 7.84×10^{-15}; `test_face_floor` unresolved-column dipped-face mass flux 2.8319×10^{-8} in the default, `_wb_ref4` and `_x1_centroid` arms; `test_wb_wall_corner` CPU corner errors 0. **Coverage.** CPU only in that run (the build had CUDA off, so every `_cuda` case was skipped); the CUDA arms `face_floor_*_cuda` and `*_cuda` of `test_wb_wall_corner` exist and compare CUDA with CPU to 10^{-5} relative and exactly: **pending: a CUDA run at the pin**. Multi-rank: through the seam tests of Section 5.4 only.

5.1.6. Limits

> **Limits**
>
> The x_2 and x_3 reconstructions have no well-balanced treatment. A column that is hydrostatic in the continuum but not at the discrete fixed point Equation 5.2 is not at rest.

1. **No horizontal treatment.** The x_2 and x_3 sweeps reconstruct the full state (`precon23 hydro_forward.cpp:549-551`); on a curved grid a stratified column is therefore balanced in x_1 only.
2. **Floors off for the perturbation, but not for species.** The species clamp at zero still applies under the EOS limiter (table above). Not measured as a source of imbalance.
3. **Moist columns.** No reference meets the floor of a moist column in every case; the choice of reference is a parked study (chengcli/snapy#275) and there is no YAML key to choose it (chengcli/snapy#281).
4. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.2. Hydrostatic reference kernel: face-pressure scan, cell pressure and density reference (with the WB path)

5.2.1. Summary

> **i** At a glance
>
> **What it does.** Builds, per column and per call, the face pressures p_{sf} by a top-down hydrostatic scan, the cell pressure p_{ref} from them, and the density references ρ_{ref} and ρ_{sf} from a smoothed ρ/p. **Switch.** None: always with the well-balanced path (Section 5.1). **Order.** p_{sf} exact up to one column constant; p_{ref} $O(\Delta x_1^6)$ on uniform grids; ρ_{sf} $O(\Delta x_1^2)$. **Defining equation.** Equation 5.3.

The kernel is the reference of Equation 5.1. Its face pressures are the discrete hydrostatic balance itself, summed from an anchor at the top of the column, so the rest balance of the well-balanced reconstruction holds identically. Its cell pressure is the cell average of a quintic through six face pressures, which matches the cell average of a smooth hydrostatic pressure to sixth order up to the same column constant; on a non-uniform grid it is a log-mean instead, exact only for an isothermal cell. Its density reference is the cell pressure times a smoothed ρ/p: smoothed, because an unsmoothed ratio makes the density perturbation a copy of the pressure perturbation and the entropy mode would bypass the reconstruction.

The kernel has a C++ implementation shared by the CPU and CUDA paths and a tensor implementation for MPS. The wall closure of the smoothed ratio and of the six-face rows is Section 5.3; the anchor relay across x_1 seams is Section 5.4; the fourth-order replacement of the density reference is Section 5.7.

Aliases. “smooth5” (the five-point binomial) and “the base reference”

in `docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §1.

5.2.2. Derivation

Units as in Section 5.1; $g = -g_1 > 0$ [m s^{-2}] is the downward magnitude the kernel takes; $r = \rho/p$ [s^2 m^{-2}]. Cell i spans $[x_{1,i-1/2}, x_{1,i+1/2}]$, and $\Delta x_{1,i}$ is its width.

5.2.2.1. Face pressures

Hydrostatic balance integrated over cell i gives $p(x_{1,i-1/2}) - p(x_{1,i+1/2}) = g\int_{\text{cell}} \rho \,\mathrm{d}x_1 = g\,\overline{\rho}_i \Delta x_{1,i}$ exactly, for the cell average $\overline{\rho}_i$. The kernel defines the face pressures by this relation,

$$p_{\text{sf},i-1/2} = p_{\text{sf},i+1/2} + g\,\overline{\rho}_i\,\Delta x_{1,i}, \tag{5.3}$$

downward from the top face of the top interior cell i_u and upward into the upper ghosts, clipped below at the smallest positive normal number. Equation 5.3 is `hydro_ref_x1_scan_impl` (`hydro_ref_x1_impl.h:22-60`). Since each step is exact, p_{sf} equals the exact face pressure of a hydrostatic column plus one constant per column, set by the anchor (`[C13]`: spread of $p_{\text{sf}} - p(x_{1,f})$ 2.2×10^{-16} of p). On a single block the anchor extrapolates the top cell isothermally by half a cell,

$$p_{\text{sf},i_u+1/2} = \overline{p}_{i_u} \exp\left(-\frac{g\,\Delta x_{1,i_u}}{2\,\overline{p}_{i_u}/\overline{\rho}_{i_u}}\right), \tag{5.4}$$

the branch that sets `top_anchor` (`hydro_ref_x1_impl.h:30-38`). It uses the cell averages, so even on an isothermal column it is off by the factor $\sinh a/a = 1 + a^2/6 + O(a^4)$, $a = \Delta x_1/2H$ with $H = p/(\rho g)$ (`[C10]`). That

offset is a column constant and drops out of the rest balance (Equation 5.2 holds for any c).

5.2.2.2. Cell pressure

On a uniform grid the cell pressure is the cell average of the quintic through the six faces $i - \frac{5}{2}, \dots, i + \frac{5}{2}$:

$$p_{\mathrm{ref},i} = \tfrac{1}{1440}(11\,p_{\mathrm{sf},i-5/2} - 93\,p_{\mathrm{sf},i-3/2} + 802\,p_{\mathrm{sf},i-1/2} + 802\,p_{\mathrm{sf},i+1/2} - 93\,p_{\mathrm{sf},i+3/2} + 11\,p_{\mathrm{sf},i+5/2}), \tag{5.5}$$

weights derived exactly in `[C1]` and coded as `w6` (`hydro_ref_x1_impl.h:140-156`). In the two cells next to an array end or a clamped wall the same average is taken over six one-sided faces, rows `w6e` (`[C2]`, Section 5.3). A value outside $[\min, \max]$ of the cell's own two face pressures is rejected and the face mean is kept. Because the faces are exact up to c, $p_{\mathrm{ref},i} = \overline{p}_i + c + O(\Delta x_1^6)$ (`[C13]`: fitted order 5.87 on an isothermal column, $n_1 = 8, 16, 32$).

On a non-uniform grid, where the relative spread of the face widths is 10^{-10} of their mean or more (`[C20]`), the cell pressure is the log-mean

$$p_{\mathrm{ref},i} = \frac{g\,\overline{\rho}_i\,\Delta x_{1,i}}{\ln(p_{\mathrm{sf},i-1/2}/p_{\mathrm{sf},i+1/2})}, \tag{5.6}$$

or the face mean when `ratio` is within 10^{-6} of one (`hydro_ref_x1_impl.h:181-186`). For an isothermal cell p is exponential in x_1, and its cell average is exactly Equation 5.6; for a polytrope it is $O(\Delta x_1^2)$ (`[C9]`: relative error 1.5×10^{-4} at $\Delta x_1 = 0.05$, order 2.02).

5.2.2.3. Density references

With $r_i = \overline{\rho}_i/\overline{p}_i$, the smoothed ratio is the binomial $r_i^s = \frac{1}{16}(r_{i-2} + 4r_{i-1} + 6r_i + 4r_{i+1} + r_{i+2})$, and the references are

$$\begin{aligned} \rho_{\mathrm{ref},i} &= p_{\mathrm{ref},i}\, r_i^s, \\ \rho_{\mathrm{sf},i-1/2} &= p_{\mathrm{sf},i-1/2}\, \tfrac{1}{2}(r_{i-1}^s + r_i^s), \end{aligned} \tag{5.7}$$

coded as `dref` and `dsf` (`hydro_ref_x1_impl.h:193-200`), with the binomial in `hydro_ref_x1_rop_smooth` (`hydro_ref_x1_impl.h:86-108`). At array index 0 the face density uses r_0^s alone.

Why smoothed. Take the unsmoothed ratio instead, $\rho_{\mathrm{ref},i} = p_{\mathrm{ref},i}\overline{\rho}_i/\overline{p}_i$. Then $\rho_i' = \overline{\rho}_i - p_{\mathrm{ref},i}\overline{\rho}_i/\overline{p}_i = \overline{\rho}_i\, p_i'/\overline{p}_i$, so $\rho'/\overline{\rho} = p'/\overline{p}$ in every cell: the density perturbation carries no information the pressure perturbation does not, and an entropy (buoyancy) anomaly is not reconstructed as a density perturbation at all. The binomial keeps the reference on the large scales of the column only. It has unit sum, zero first moment and second moment 1 in cells, and zero response at the grid scale (`[C3]`), so $r_i^s = r_i + \frac{\Delta x_1^2}{2} r'' + O(\Delta x_1^4)$. This re-derives the degeneracy stated in the comment above `hydro_ref_x1_rop_smooth`; the measurements behind it are in `sources/canoe__tall_column_instability_TECH_REPORT.md` §§3d, 4-5.

Face-density offset. What the solver uses is $\rho_{\mathrm{sf}} + \mathcal{R}[\overline{\rho} - \rho_{\mathrm{ref}}] = \mathcal{R}[\overline{\rho}] + \delta\rho_f$ with $\delta\rho_f = \rho_{\mathrm{sf}} - \mathcal{R}[\rho_{\mathrm{ref}}]$. Expanding the ratio of averages, the binomial bias, the two-cell mean and the cell pressure, `docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §2 obtains

$$\delta\rho_f = \frac{\Delta x_1^2}{12}(p'R' + 2pR'') + O(\Delta x_1^4), \tag{5.8}$$

with $R = \rho/p$ the point ratio and primes x_1 derivatives (this paragraph only). `[C14]` measures $\delta\rho_f$ at the middle face of a polytrope against fifth-

order linear reconstruction and finds the ratio to Equation 5.8 1.0022, 1.0005, 1.0001, 1.0000 at $n_1 = 32$ to 256, order 2.00.

Exact and truncated. Exact: the scan step, hence the rest balance. $O(\Delta x_1^6)$: the uniform cell pressure. $O(\Delta x_1^2)$: the log-mean on non-isothermal cells, the smoothed ratio and the face density reference.

5.2.3. Numerical method

Figure 5.2 shows the stencils. The scan is serial along each column; the per-cell part reads six face pressures and five ratios. The kernel runs once per x_1 sweep, in every RK stage, on cell averages (on plain means under Section 5.8), including ghost cells: ghosts get face pressures from the upward scan and, without the wall clamp, ghost ratios enter the binomial. Uniformity is decided once per block from its own `dx1f`.

Paths. CPU and CUDA run the same header functions: one CPU loop over columns, one CUDA block per column whose thread 0 scans and whose threads then fill the cells. The MPS path rebuilds the same quantities with tensor operations (cumulative sums, a padded binomial), and `test_hydro_ref_x1` compares it with the CPU path.

Order. As in the Derivation layer, interior; at walls see Section 5.3; at seams the reference is the neighbour's (Section 5.4).

5.2.4. Code

Table 5.4.: Code map of the reference kernel.

what	code link	symbol	switch
the call and the uniform test	`hydro.cpp:535-556`	`x1_uniform_`	spread $< 10^{-10}$

what	code link	symbol	switch
scan and anchor	`hydro_ref_x1_impl.h:22-60`	`hydro_ref_x1_scan_impl`	
binomial ratio	`hydro_ref_x1_impl.h:86-108`	`hydro_ref_x1_rop_smooth`	
cell pressure, both grids	`hydro_ref_x1_impl.h:136-186`	`use_six_faces`	uniform
thin-block and fit guards	`hydro_ref_x1_impl.h:124-135`	`fits`	
density references	`hydro_ref_x1_impl.h:188-200`	`dsf`	
CPU path	`hydro_dispatch.cpp:14-35`	`hydro_ref_x1_cpu`	
MPS tensor path	`hydro_dispatch.cpp:37-200`	`hydro_ref_x1_mps`	
CUDA kernel	`hydro_dispatch.cu:12-31`	`hydro_ref_x1_cuda_kernel`	CUDA build

1. `HydroImpl::_hydro_ref_x1` classifies the block once as uniform or not and caches it in `x1_uniform_` (`hydro.cpp:535-541`).
2. It reads whether each x_1 end is a physical boundary and calls the dispatch stub `call_hydro_ref_x1` with the wall-clamp option (`hydro.cpp:550-556`).
3. Per column, `hydro_ref_x1_impl` runs the scan and then the cell function for every cell (`hydro_ref_x1_impl.h:203-215`).
4. In the cell function, the guards `thin` and `fits` decide whether one-sided rows may be used (`hydro_ref_x1_impl.h:124-135`), then the cell pressure is formed by `use_six_faces` or the log-mean (`hydro_ref_x1_impl.h:136-186`), and the two density references by Equation 5.7.

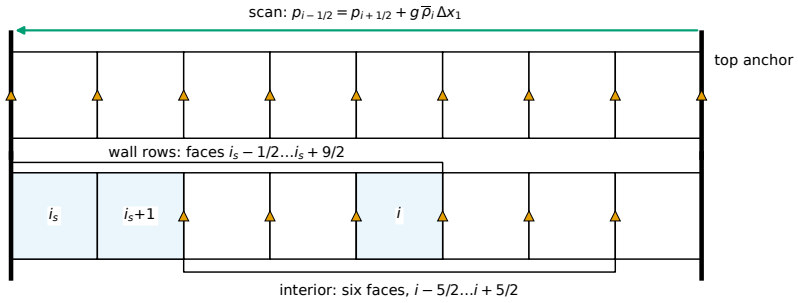


Figure 5.2.: The reference kernel on one column between two walls (thick black lines), x1 to the right. Top: the scan (bluish-green arrow) starts from the top anchor and adds g rho_i dx1 cell by cell; face pressures are orange triangles. Bottom: the six faces whose quintic gives the interior cell pressure of cell i, and the one-sided six-face window used for the two cells next to a wall. Shaded cells are the cells whose pressure the windows define.

5. The fourth-order replacement (Section 5.7) and the seam exchange (Section 5.4) follow in `_hydro_ref_x1`.

No switch is read and no warning is issued by the kernel. A block with fewer interior x_1 cells than ghosts makes the one-sided rows unusable (`fits` false), and the face mean is kept.

5.2.5. Tests

Table 5.5.: Tests of the reference kernel.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_hydro_ref_x1.cpp` (`test_hydro_ref_x1.release`), `cpu_matches_tensor_reference`	the CPU path equals the MPS tensor implementation run on CPU	2×10^{-12} double, 2×10^{-5} float	round-off of two summation orders
same, `wall_clamp_keeps_the_interior_free_of_the_ghosts` and its control	with the clamp, changing ghosts leaves interior references unchanged; without, they change	exact (`torch::equal`)	the property is structural
same, `a_thin_block_keeps_the_interior_free_of_the_ghosts` and control	as above on a block of fewer than five cells	exact	structural

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `the_wall_reference_does_not_depend_on_the_blocks_x1_size`	the wall-cell reference is the same on blocks of different x_1 size	10^{-14} relative	round-off
same, `cuda_matches_cpu`	CUDA equals CPU	per dtype	round-off
`tests/test_hydro_options.cpp` (`test_hydro_options.release`), `wb_wall_clamp_ships_enabled`, `wb_wall_clamp_reaches_the_x1_reference`	the clamp defaults on and changes the reference	differ by more than 10^{-13}	non-vacuity
this chapter's check	`[C1]`–`[C3]`, `[C9]`, `[C10]`, `[C13]`, `[C14]`, `[C20]`	per claim	stated in the check

Evidence. `test_hydro_ref_x1.release` (6 passed, `cuda_matches_cpu` skipped) and `test_hydro_options.release` (11 passed, the `_cuda` case skipped) pass at the pin [sha `e894700ff7aee30b52882e5202b16461413780b0`; decks: the test files; run: `ctest -V`, Release build, 2026-10-10, CPU, double, 1 rank]. Check numbers: [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, rebuilt from this section's equations; deck: the check itself; run: `python3 wbref_check.py`, numpy 2.2.4, sympy 1.14.0, double]. **Coverage.** CPU in the run above; CUDA cases exist but were not run (**pending: a CUDA run at the pin**); the

MPS path itself is compared only through its CPU execution; multi-rank through Section 5.4.

5.2.6. Limits

> **Limits**
>
> On a stretched x_1 grid the cell pressure is the log-mean, exact only for isothermal cells, so a column at rest is at rest to round-off only after `balance_column`.

1. **Non-uniform grids.** The log-mean is a different operator from the six-face rows, not a lower-order version of them, and the reconstruction weights are uniform-spacing on every grid. A stretched-grid rest residual of 7.37×10^{-4}, the same for every density form, is reported in chengcli/snapy#250 (`sources/gh__ISSUE_THREADS_138-250.md`; measured before the pin, not re-run here); chengcli/snapy#280 tracks it. chengcli/snapy#280 also states that the face interpolation uses fixed rationals on stretched grids; at the pin the rationals are used only on a uniform block and the log-mean otherwise.
2. **No positivity guard on ρ/p.** The ratio divides by the cell pressure without a guard (chengcli/snapy#250 §2). Not measured at the pin.
3. **One classification per block.** A grid uniform over part of the column gives different operators to different blocks; `balance_column` must make the same choice (Section 5.6).
4. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.3. Wall clamp and the wall continuation of the default reference (`dynamics/wb-wall-clamp`)

5.3.1. Summary

> **i** At a glance
>
> **What it does.** Keeps every reference stencil inside the owned cells at a physical x_1 wall: one-sided six-face rows for the two wall cells, and ρ/p continued past the wall from the two owned cells next to it. **Switch.** YAML `dynamics/wb-wall-clamp`, default `true`. **Order.** ρ_{sf} second order at faces 0-2 next to each wall (first order with the wall cell repeated). **Defining equation.** Equation 5.10.

The ghost cells of a reflecting wall hold the mirror image of the interior, which is the right input for the Riemann solver but not a hydrostatic continuation of the column: a stratified profile mirrored at the wall has a kink. Any reference stencil that reads a ghost therefore puts that kink into p_{ref} and ρ_{ref} next to the wall. With the clamp on, the cell pressure of the two cells next to a physical wall uses one-sided six-face rows, and the binomial of ρ/p no longer reads ghosts. What it reads instead is the wall continuation: the ratio continued linearly from the two owned cells where it rises towards the wall, and ln-linearly where it falls. A plain repetition of the wall cell, the first version of the clamp, is exact only for constant ρ/p and leaves the face density reference first order at the three faces next to each wall.

The continuation changes only ρ_{sf} at faces 0-2 and ρ_{ref} at cells 0-1 next to each clamped wall (and their ghost rows); p_{sf}, p_{ref} and the rest balance are untouched. It applies with `SNAP_WB_REF4` off; with it on, Section 5.7 replaces the density reference with its own cubic wall values. `balance_column` refuses to run with the clamp off.

Aliases. “wb-wall-clamp”, “the wall closure”, “linear/ln closure”. The continuation arrived as three wall-closure patches (linear, then ln-linear, then the two branches), kept as raw notes in `sources/team-notes/` and squash-merged into the pin by chengcli/snapy#297.

5.3.2. Derivation

Notation as in Section 5.2; cells are counted from the wall, $j = 0$ the wall cell, and $r_j = \overline{\rho}_j/\overline{p}_j$.

5.3.2.1. The repeated wall cell is first order

Near the wall write $r_j = r_0 + j\,a + O(\Delta x_1^2)$ with $a = r'\Delta x_1 = O(\Delta x_1)$. Repeating r_0 for every $j < 0$ gives

$$\begin{aligned} r_0^s - r_0 &= \tfrac{3}{8}\,a, \\ r_1^s - r_1 &= \tfrac{1}{16}\,a, \end{aligned} \tag{5.9}$$

and errors $\frac{23}{32}a$, $\frac{7}{32}a$, $\frac{1}{32}a$ and 0 in the face average $\frac{1}{2}(r^s_{f-1} + r^s_f)$ at faces 0 to 3 (`[C4]`, exact). Relative to r these are fixed multiples of $(r'/r)\Delta x_1$, so the face density reference is first order at faces 0-2. The even-parity perturbation ghosts (Section 5.1) copy the bias of $\rho' = \overline{\rho} - \rho_{\mathrm{ref}}$ into the ghosts, so the solver's face densities there are first order too; the mirror is not the source, the bias in ρ_{ref} is (`docs/derivations/wb-ref-wall.md@e894700ff7aee30b52882e5202b16461413780b0` §3).

5.3.2.2. The continuation

Past a clamped wall the binomial reads, k cells beyond the wall,

$$r_{-k} = \begin{cases} r_0 + k\,(r_0 - r_1), & r_1 \le r_0, \\ r_0\,(r_0/r_1)^k, & r_1 > r_0, \end{cases} \tag{5.10}$$

and r_0 if the value is not positive; the top wall is the mirror image. Equation 5.10 is `hydro_ref_x1_wall_rop` (`hydro_ref_x1_impl.h:68-72`). The binomial reproduces a linear profile exactly, so with the linear branch the $O(a)$ errors of Equation 5.9 vanish at faces 0-2; the ln-linear branch differs from it by $O(a^2)$, errors $\frac{31}{32}a^2/r_0$, $\frac{1}{4}a^2/r_0$, $\frac{1}{32}a^2/r_0$ (`[C5]`, exact series). Either way faces 0-2 keep the interior's $O(\Delta x_1^2)$, and for constant r the linear branch returns r_0 exactly, so an isothermal column is unchanged bit for bit.

Each branch is unbounded on one side: the linear one reaches zero when r rises steeply away from the wall ($r_{-3} = r_0(1-3d)$ with $d = (r_1 - r_0)/r_0$), and the ln-linear one diverges when the neighbour is nearly empty. The code takes the branch that stays closer to r_0, so for $r_1 \ge 0$ the continuation lies in $[r_0(r_0/r_1)^k,\ r_0(1+k)]$ and is positive (`[C6]`, 60000 random samples, none outside).

5.3.2.3. The one-sided cell-pressure rows

The two wall cells take the cell average of the quintic through the six faces starting at the wall, weights `w6e` derived exactly in `[C2]`: $(95/288, 1427/1440, -133/240, 241/720, -173/1440, 3/160)$ for the wall cell and $(-3/160, 637/1440, 511/720, -43/240, 77/1440, -11/1440)$ for the next. They keep the sixth order of Equation 5.5 and read no ghost face.

Exact and truncated. Exact: the continuation for linear r (linear branch), the rows for a quintic. Second order: ρ_{sf} at faces 0-2. The independent rebuild of `[C12]` gives the face-1 error first order with the repeated wall cell (order 1.03) and second order with Equation 5.10 (order 2.01), $n_1 =$ 32 to 256.

5.3.3. Numerical method

Figure 5.3 shows the two branches. The clamp applies on a side when the option is on and that side is a physical boundary; the continuation additionally needs the wall side to own at least two cells, so that only owned cells are read. The one-sided rows need the block to fit them (at least five interior cells, or a seam at the other end), else the face mean is kept. The rule runs inside the kernel, in every call, on the CPU, CUDA and MPS paths alike.

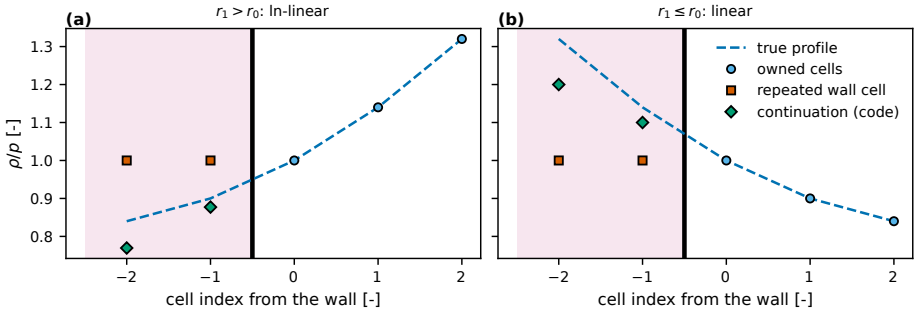


Figure 5.3.: rho/p next to a clamped bottom wall (thick black line; ghost side shaded reddish purple), x1 to the right. Sky-blue circles: the three owned cells; dashed blue curve: the true profile; vermillion squares: the repeated wall cell; bluish-green diamonds: the continuation the code uses at one and two cells past the wall. (a) rho/p rises away from the wall, so the ln-linear branch applies; (b) it falls, so the linear branch applies. The repeated value misses the trend in both.

The measured effect at the pin is Figure 5.4: next to both walls the face density reference falls at second order and stays below the largest interior error.

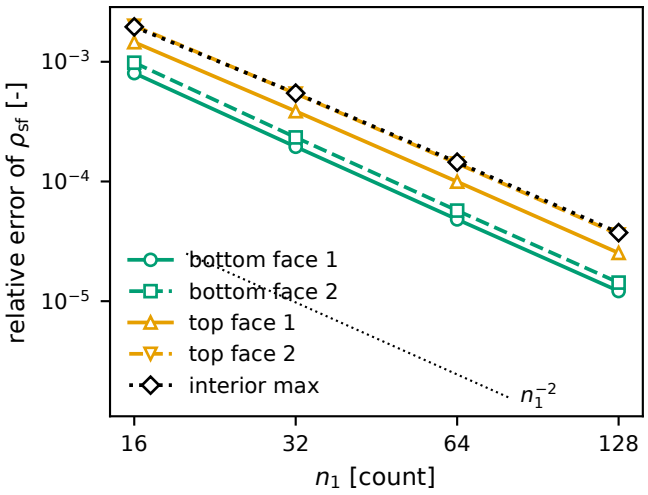


Figure 5.4.: Relative error of the face density reference next to the walls and in the interior, polytrope T = 1 - z/2 one pressure scale height deep, against the number of cells. Bluish-green solid circles and dashed squares: first and second face above the bottom wall; orange solid and dashed triangles: first and second face below the top wall; black dotted diamonds: largest interior error. The dotted black line has slope -2. Every series falls at second order.

5.3.4. Code

Table 5.6.: Code map of the wall clamp and continuation.

what	code link	symbol	switch
the YAML key	`hydro_options.cpp:57`	`wb_wall_clamp`	`dynamics/wb-wall-clamp`
its default	`hydro.hpp:54`	`wb_wall_clamp`	default `true`
Equation 5.10	`hydro_ref_x1_impl.h:68-72`	`hydro_ref_x1_wall_rop`	
continuation inside the binomial	`hydro_ref_x1_impl.h:91-106`	`ext_lo`	
clamp gates of the rows	`hydro_ref_x1_impl.h:132-135`	`clamp_in`	clamp on, physical side
one-sided rows	`hydro_ref_x1_impl.h:158-172`	`w6e`	
ratio range and continuation flags	`hydro_ref_x1_impl.h:189-192`	`ext_hi`	two owned cells
MPS tensor path	`hydro_dispatch.cpp:163-190`	`wall_rop`	

1. The option is read from `dynamics` with default `true` as `wb_wall_clamp` (`hydro_options.cpp:57`) and passed to the kernel by `_hydro_ref_x1`.
2. In the cell function, `clamp_in` and `clamp_out` switch the two wall cells to the rows `w6e` (`hydro_ref_x1_impl.h:158-172`).
3. The owned range `jlo..jhi` and the flags `ext_lo`, `ext_hi` are set

(`hydro_ref_x1_impl.h:189-192`), and the binomial replaces each index past the wall by `hydro_ref_x1_wall_rop` (`hydro_ref_x1_impl.h:91-106`).

4. The MPS path applies the same rule to its padded ratio with `wall_rop` (`hydro_dispatch.cpp:163-190`).

An unknown key in `dynamics` is refused by the key check of the options parser; the clamp itself issues no warning. `balance_column` raises if called with the clamp off (Section 5.6).

5.3.5. Tests

Table 5.7.: Tests of the wall clamp and continuation.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_wb_ref_wall.cpp` (`test_wb_ref_wall.release`), `faces_next_to_the_walls_are_second_order`	observed order n_1 32 $\rightarrow$ 64 of ρ_{sf}, ρ_L, ρ_R at faces 1-2 next to both walls; wall faces within the interior error	order ≥ 1.7; $\leq 1.5\times$ interior max	below the 1.81-2.12 the closure measures; above the 1.05 of the repeated cell

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `order_table`	prints the table of Figure 5.4	none	report aid
same, `faces_next_to_the_walls_are_second_order_cuda`, `order_table_cuda_matches_cpu`	CUDA orders; CUDA equals CPU	as above; 10^{-13}	round-off
`tests/test_face_floor.cpp` (`test_face_floor.release`), `face_floor_fires_on_an_unresolved_column`	dipped-face mass flux with the closure	2.83191×10^{-8}, 10^{-5} relative	pinned value of the closure
`tests/test_hydro_ref_x1.cpp` (`test_hydro_ref_x1.release`) clamp cases	with the clamp, ghosts do not reach the interior	exact	structural
`test_straka_redo`	the straka deck at CFL 1.6 to $t = 60$ s exits 0 and redoes at least one step	exit status; output match	robustness of the redo path, not accuracy
this chapter's check	`[C2]`, `[C4]`–`[C6]`, `[C11]`, `[C12]`	per claim	stated in the check

Evidence. At the pin, `test_wb_ref_wall.release` prints the face errors of Figure 5.4 and the observed orders 2.02, 2.03, 2.03 (bottom face 1: ρ_{sf}, ρ_L, ρ_R), 2.03, 2.06, 2.05 (bottom face 2), 1.96, 1.81, 1.97 (top face 1) and 1.94, 1.94, 2.12 (top face 2), n_1 32 $\to$ 64; on the isothermal column ($\beta = 0$) the wall faces carry no first-order term [sha `e894700ff7aee30b52882e5202b16461413780b0`; deck: `tests/test_wb_ref_wall.cpp order_table`, polytrope $T = 1 - \beta z$, $g = R = 1$, one pressure scale height; run: ctest `test_wb_ref_wall.release`, 2026-10-10, CPU, double, 1 rank; data `src/snapy_report/ch05/data/wb_ref_wall_order.json`]. `test_face_floor` (all three arms), `test_hydro_ref_x1` and `test_straka_redo` pass in the same run. The check's rebuild reproduces all 40 printed numbers of that table to the printed four digits (`[C11]`). It also shows that the test averages its cells over the full-precision column depth but writes `x1max` into the block's YAML with six significant digits, so the block's `dx1f`, which the scan uses, is 4.1×10^{-7} relatively wider at $\beta = 0.5$; with one consistent width the faces next to the bottom wall move by at most 2.6×10^{-7} in relative error, far below the test's tolerances. The repeated-wall-cell numbers are the check's, not a snapy run: face-1 error 2.880×10^{-3}, 1.392×10^{-3}, 6.840×10^{-4}, 3.391×10^{-4} at $n_1 = 32$ to 256 (`[C12]`).
Coverage. CPU; the CUDA cases exist and were not run (**pending: a CUDA run at the pin**); the MPS path was not run (no MPS device; the derivation note says the same); multi-rank only through Section 5.4.

5.3.6. Limits

> ⚠ Limits
>
> The continuation is in cell index, not in physical spacing, so on a stretched grid the wall faces are not covered by the second-order argument. With `wb-wall-clamp: false`, `balance_column` cannot be used.

1. **Stretched grids.** The continuation, like the binomial, assumes uniform spacing. Not measured.
2. **MPS.** The tensor path implements the same rule and is compared with the CPU path only when run on CPU.
3. **Straka at CFL 1.6.** `test_straka_redo` is a robustness test past the stability limit. The derivation note's ensembles (`docs/derivations/wb-ref-wall.md@e894700ff7aee30b5288 2e5202b16461413780b0` §5) were measured on the code of the wall-closure commits, not at the pin, and its $\epsilon = 10^{-4}$ ensemble does not separate the references; the near-zero linear continuation as the cause of the linear form's failure there is a hypothesis of that note.
4. **Stale text in the notes.** `docs/derivations/wb-ref4.md@e8947 00ff7aee30b52882e5202b16461413780b0` §1 still describes the binomial as edge-replicated at a clamped wall; the code at the pin continues the ratio as above.
5. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.4. Reference continuity across x_1 seams: anchor relay and ghost-row exchange (x_1 block split)

5.4.1. Summary

> **i** At a glance
>
> **What it does.** Makes the reference of a column split into several x_1 blocks equal to the reference of one block: the face-pressure scan is relayed from block to block, top down, and the ghost rows of p_{ref} and ρ_{ref} are replaced by the neighbour's interior rows. **Switch.** None. On whenever the layout splits x_1 ($p_z > 1$) and x_1 is not periodic.

Order. Exact: a split column has the one-block reference up to the round-off of the chained scan. **Defining equation.** Equation 5.11.

The reference kernel scans each column from an anchor at the top of the block (Section 5.2). On a column split into n_{b1} blocks along x_1, a block below the top one has no top of the domain to anchor at, and an anchor from its own top cell would start a new scan with a different constant: the reference would jump at every seam, and that jump would enter $p' = \overline{p} - p_{\mathrm{ref}}$ as a spurious perturbation every step. The relay passes the running face pressure down instead, so the scan is one scan across the whole column. The second step exchanges the reference's ghost rows, because the block-local quadrature of the ghost cells (one-sided rows, the $[\min, \max]$ guard) does not reproduce the owner's interior values.

With both steps, the perturbation field the reconstruction sees is identical on both sides of every seam, so the two sides compute the same seam-face states, whether the neighbour runs on another process or on another thread of the same one.

Aliases. "the x1 relay" and "anchor relay" in comments; chengcli/snapy#254 (the seam jump), chengcli/snapy#259 (the in-process relay).

5.4.2. Derivation

Number the blocks of one column $b = 1..n_{b1}$ from the bottom, and write $i_s^{(b)}$, $i_u^{(b)}$ for the first and last interior cell of block b. One block scans Equation 5.3 downward from its top anchor. For blocks $b < n_{b1}$ the anchor is not Equation 5.4 but the bottom-face pressure of the block above:

$$p^{(b)}_{\mathrm{sf},\, i_u^{(b)}+1/2} = p^{(b+1)}_{\mathrm{sf},\, i_s^{(b+1)}-1/2}, \tag{5.11}$$

and block n_{b1}, which owns the domain top, uses Equation 5.4. Since the two faces in Equation 5.11 are the same physical face, and Equation 5.3 adds the same terms $g\overline{\rho}_i\Delta x_{1,i}$ in the same order as a single block would, the face pressures of the split column equal those of one block up to the round-off of the different summation boundaries. Equation 5.11 is `take_x1_anchor` in the block below (`hydro.cpp:525-533`) and `pass_x1_anchor` in the block above (`hydro.cpp:577-579`).

The cell pressure of a cell near the seam reads six faces, two beyond the cell on each side, so its value in the block that owns it differs from the value the neighbour computes for the same cell in its ghost rows: the neighbour has fewer faces there and uses one-sided rows or the face-mean fallback. Overwriting the n_g ghost rows of p_{ref} and ρ_{ref} with the owner's interior rows makes both blocks hold the owner's value for every cell. Then p' and ρ' agree cell by cell across the seam, the reconstruction of each side reads the same data in the stencil across the seam, and the seam-face states agree. The exchange is in `_hydro_ref_x1`, with tags `kWbGhostUpTag` and `kWbGhostDnTag` (`hydro.cpp:593-640`).

Exact and truncated. Nothing is truncated: the split reference equals the one-block reference except for the summation order of the relayed scan. Where the face density of Section 5.7 is used, its cell part runs before this exchange and its face part after it, so the exchanged rows carry the fourth-order cell density and the faces are computed from them on both sides. There is no executable check; the property is tested by the split-equals-one-block tests below.

5.4.3. Numerical method

Figure 5.5 shows the two steps. The relay is serial along the column: each block waits for the block above before it scans. Across processes the anchor is a point-to-point message (tag `0x7715`); between blocks of one process it is posted to a board keyed by the receiving block and taken with a five-minute timeout, so local blocks must run concurrently or from the top

down. The ghost-row exchange uses the same pairing. Both run once per call of `_hydro_ref_x1`, so in every RK stage, before the reconstruction.

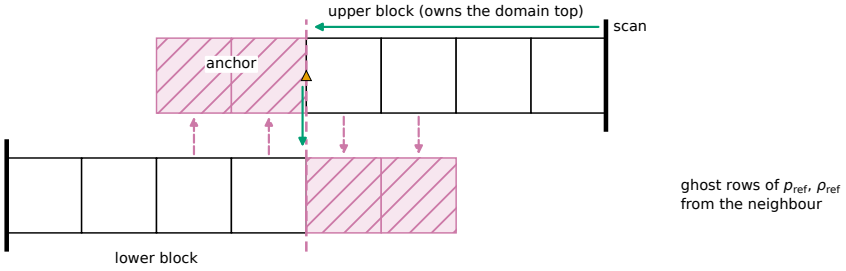


Figure 5.5.: Reference continuity at an x1 seam (dashed reddish-purple line), x1 to the right; the upper block (top row) owns the domain top (thick black line at its right end). The upper block scans from the top (bluish-green arrow along the top) and passes its bottom-face pressure (orange triangle) down as the lower block's anchor (bluish-green vertical arrow). Then each block's hatched ghost cells of p_ref and rho_ref are overwritten with the neighbour's interior rows (dashed reddish-purple arrows).

Two related seam mechanisms belong to other sections: the single-valued seam flux average across process seams (Chapter 4) and, under `SNAP_X1_CENTROID_EXACT`, the exchange of plain-mean and face-pressure ghost rows (Section 5.8).

5.4.4. Code

Table 5.8.: Code map of reference continuity across x_1 seams.

what	code link	symbol	switch
split test and anchor receive	`hydro.cpp:525-533`	`x1_split`	$p_z > 1$, not periodic
anchor pass	`hydro.cpp:577-579`	`pass_x1_anchor`	
ghost-row exchange	`hydro.cpp:593-640`	`kWbGhostUpTag`	
receive, remote or local	`layout.cpp:879-905`	`take_x1_anchor`	
send, remote or local	`layout.cpp:907-922`	`pass_x1_anchor`	
the scan reading the anchor	`hydro_ref_x1_impl.h:30-38`	`anchor`	

1. `_hydro_ref_x1` sets `x1_split` from the layout and, if set, receives the anchor from the block above with `take_x1_anchor`; with no block above the anchor is empty and the kernel builds its own (`hydro.cpp:525-533`).
2. The kernel scans from the received `anchor` (`hydro_ref_x1_impl.h:30-38`); the cell part of Section 5.7 follows.
3. The block's bottom-face pressure goes to the block below with `pass_x1_anchor` (`hydro.cpp:577-579`).
4. The interior rows go up and down with `kWbGhostUpTag` and `kWbGhostDnTag`, and the received rows overwrite the ghost rows (`hydro.cpp:593-640`).
5. The face part of Section 5.7 runs last.

A local anchor that does not arrive within five minutes raises the error of `take_x1_anchor` (`layout.cpp:879-905`), the only error a user can hit here.

5.4.5. Tests

Table 5.9.: Tests of reference continuity across seams.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_pref_local_seam.cpp` (`test_pref_local_seam.release`), `local_blocks_restart_the_reference_at_the_seam`	p_{ref} of the cells either side of the seam equals the one-block value	10^{-6} abs	chained-scan round-off
same, `in_process_split_matches_one_block_after_200_steps`	four in-process blocks equal one block after 200 steps	10^{-12} relative	measured 5.4×10^{-14} in v_1 (test comment)
`tests/test_x1_seam_split.cpp` arms `_x1_centroid`, `_wb_ref4`, `_radial_exact`, `_radial_exact_off`	a split column equals one block after 20 steps, full and hydrostatic-split pressure	10^{-13}	round-off
`tests/test_x1_seam_split_mp.cpp` (2 ranks) and its arms	two ranks equal two in-process blocks and one block	10^{-13}	round-off

Evidence. All pass at the pin [sha `e894700ff7aee30b52882e5202b16461`

`413780b0`; decks: the test files; run: `ctest -V`, Release build, 2026-10-10, CPU, double; 1 rank, the `_mp` entries 2 ranks under `torchrun`]. Printed gaps of that run: `_x1_centroid` split against one block 0 (both `non-hydrostatic` 1 and 0); `_wb_ref4` 6.775×10^{-15} and 1.675×10^{-15}; `_mp` arms, 2 ranks against 2 in-process blocks, 0 in every arm. **Coverage.** CPU, 1 and 2 ranks; CUDA arms exist and were not run (**pending: a CUDA run at the pin**); more than two ranks along x_1 is not tested.

5.4.6. Limits

1. **Serial relay.** The anchor travels down the column block by block, so the latency of the reference grows with n_{b1}. Not measured.
2. **Periodic x_1.** No relay; each block anchors at its own top. A periodic x_1 with gravity is not a hydrostatic column anyway.
3. **Stale test comment.** The header of `tests/test_pref_local_seam.cpp` still says the test is red and does not fix the jump; at the pin both cases pass, after chengcli/snapy#259.
4. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.5. Hydrostatic mode: gravity replaced by the cell's own pressure difference (`forcing/const-gravity/non-hydrostatic`)

5.5.1. Summary

> **i** At a glance
>
> **What it does.** Replaces the fraction $1 - n_h$ of the x_1 gravity force ρg_1 by the difference of the cell's own reconstructed face pressures, and books the same replacement in the energy. **Switch.** YAML `forcing/const-gravity/non-hydrostatic` $= n_h \in [0, 1]$, default 1 (mode off). **Requires.** $g_1 \neq 0$. Under `SNAP_X1_CENTROID_EXACT` it uses the r^2 operator of Section 5.8. **Order.** Exact cancellation at rest; away from rest the force is the difference between Riemann and reconstructed face pressures. **Defining equation.** Equation 5.12.

In the default, non-hydrostatic mode ($n_h = 1$) gravity enters as the body force $\overline{\rho}_i g_1$ of the constant gravity forcing, and vertical motion responds to the imbalance between that force and the pressure fluxes. With $n_h < 1$ part of the body force is replaced by $(p^L_{i+1/2} - p^R_{i-1/2})/\Delta x_1$, the pressure difference between the cell's own reconstructed states at its upper and lower face. At $n_h = 0$ the vertical momentum of a cell then changes only by the jumps $p^* - p^{L,R}$ that the Riemann solver introduces at its faces, which vanish in a column at rest whatever the column is.

The value is checked to lie in $[0, 1]$ when the YAML is read. The mode is independent of the well-balanced reconstruction, but the two are normally used together: the face states it differences are the restored states of Equation 5.1.

Aliases. "hydrostatic-split pressure", "nh" in tests; `rho_grav` in the code.

5.5.2. Derivation

Units as in Section 5.1. The constant gravity forcing adds $n_h \overline{\rho}_i g_1$ to the momentum and $n_h \overline{\rho}_i \overline{v}_{1,i} g_1$ to the energy. The hydrostatic mode adds, in Cartesian geometry,

$$\begin{aligned} G_i &= \frac{p^L_{i+1/2} - p^R_{i-1/2}}{\Delta x_{1,i}}, \\ \Delta(\rho v_1)_i &= \Delta t\,(1 - n_h)\,G_i, \\ \Delta E_i &= \Delta t\,(1 - n_h)\,\overline{v}_{1,i}\,G_i, \end{aligned} \tag{5.12}$$

with $p^L_{i+1/2}$ the left state at the upper face and $p^R_{i-1/2}$ the right state at the lower face, both belonging to cell i. G is `rho_grav` (`hydro_forward.cpp:389-397`), applied by the increments that use `non_hydrostatic` (`hydro_forward.cpp:901-906`). At $n_h = 0$ the momentum balance of cell i becomes

$$\frac{\mathrm{d}}{\mathrm{d}t}(\overline{\rho v_1})_i = -\frac{\Delta_i[\rho v_1^2]}{\Delta x_1} - \frac{(p^* - p^L)_{i+1/2} - (p^* - p^R)_{i-1/2}}{\Delta x_1},$$

so the cell's own pressure difference no longer drives it; only the face jumps do. At rest the two states of every face agree and $p^* = p^L = p^R$, so the force vanishes exactly for any column, balanced or not (`[C19]`). For $0 < n_h < 1$ the force is the blend of the two forms.

Under `SNAP_X1_CENTROID_EXACT` on spherical-polar grids the pressure force is $-(A_{i+1/2}p^*_{i+1/2} - A_{i-1/2}p^*_{i-1/2})/V_i + S_i$ with the source S_i of Equation 5.21, and G takes the same operator on the cell's own states, $G_i = (A_{i+1/2}p^L_{i+1/2} - A_{i-1/2}p^R_{i-1/2})/V_i - S_i$, so the two cancel at rest again (`docs/derivations/x1-centroid-spherical.md@e894700ff7aee30b52882e5202b16461413780b0` §4). This form is the branch that calls `x1_pressure_source` (`hydro_forward.cpp:515-539`).

Exact and truncated. Exact: the cancellation at rest, in both forms. Away from rest the momentum source has the accuracy of $p^* - p^{L,R}$, which depends on the reconstruction and the Riemann solver (Chapter 4).

5.5.3. Numerical method

Figure 5.6 shows the quantities. G is formed right after the x_1 reconstruction (and the immersed-solid face treatment), from the interior faces of each cell, and applied once per stage together with the other source increments of `HydroImpl::forward`. It is explicit. The energy row books $\overline{v}_1 G$, and the gravity-work bookkeeping of Chapter 6 counts the same term (`original_gravity_work`).

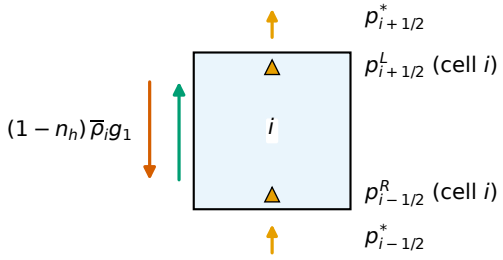


Figure 5.6.: Hydrostatic mode in one cell, x1 upward. Orange triangles: the cell's own reconstructed face pressures, the left state at its upper face and the right state at its lower face; orange arrows: the Riemann face pressures p* at both faces. The vermillion arrow is the fraction (1 - n_h) of the body force that is removed, the bluish-green arrow the difference of the cell's own face pressures that replaces it.

5.5.4. Code

Table 5.10.: Code map of the hydrostatic mode.

what	code link	symbol	switch
the key, default and range	`const_gravity.cpp:25-26`	`non_hydrostatic`	default 1
the remaining body force	`const_gravity.cpp:46-52`	`forward`	
G, Cartesian form	`hydro_forward.cpp:389-397`	`rho_grav`	$n_h < 1$
G, r^2 form	`hydro_forward.cpp:515-539`	`x1_pressure_source`	`SNAP_X1_CENTROID_EXACT`
work counted in the gravity-work budget	`hydro_forward.cpp:850-857`	`original_gravity_work`	
momentum and energy increments	`hydro_forward.cpp:901-906`	`non_hydrostatic`	

1. The YAML value is read into `non_hydrostatic` with default 1 and checked to lie in $[0, 1]$ (`const_gravity.cpp:25-26`).
2. `ConstGravityImpl::forward` adds n_h times the body force and its work (`const_gravity.cpp:46-52`).
3. After the x_1 reconstruction, `rho_grav` is filled for the interior cells (`hydro_forward.cpp:389-397`), and overwritten by the r^2 form with `x1_pressure_source` when that switch acts (`hydro_forward.cpp:515-539`).
4. The increments are added with the weight $1 - n_h$ read from `non_hydrostatic` (`hydro_forward.cpp:901-906`).

A value outside $[0, 1]$ fails the `TORCH_CHECK` of step 1. There is no warning.

5.5.5. Tests

Table 5.11.: Tests of the hydrostatic mode.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_hydrostatic.cpp` (`test_hydrostatic.release`), `cubed_sphere_remains_at_rest`	`non-hydrostatic: 0.`; a cubed-sphere hydrostatic atmosphere keeps its vertical velocity below the bound for 100 steps	10^{-8}	round-off margin
`tests/test_x1_centroid_rest.py` (`test_x1_centroid_rest_python`)	rest force at $n_h = 1$ and 0, with and without the switch	10^{-10} on, $> 10^{-8}$ off	Section 5.8
`tests/test_x1_seam_split.cpp`, `tests/test_x1_seam_split_mp.cpp` arms	split equals one block at $n_h = 1$ and 0	10^{-13}	round-off
this chapter's check, `[C19]`	cancellation at rest, both forms	10^{-12}	round-off

Evidence. At the pin, `test_hydrostatic` prints a maximum spurious vertical velocity after 100 steps of 7.84×10^{-15}, and the $n_h = 0$ rows of

`test_x1_centroid_rest` read, switch off and on, 6.0×10^{-8} and 8.3×10^{-15} (interior, $r_0/H = 5$, explicit) [sha `e894700ff7aee30b52882e5202b16461413780b0`; decks: the test files; run: `ctest -V` and `python tests/test_x1_centroid_rest.py` against the pin's build, 2026-10-10, CPU, double, 1 rank]. **Coverage.** CPU; CUDA twins exist and were not run (**pending: a CUDA run at the pin**); 2 ranks through the seam tests. No test measures the mode away from rest.

5.5.6. Limits

1. **Not a hydrostatic model.** $n_h = 0$ removes the cell's own pressure difference from the vertical momentum, but the vertical velocity is still prognostic; this is not a hydrostatic primitive-equation solver.
2. **Gravity-work interplay.** The energy term enters the gravity-work bookkeeping of Chapter 6 through `original_gravity_work`; its interaction with the face forms is described there.
3. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.6. `balance_column`: projection of a ghost-free column onto the scheme's discrete balance (C++ and Python API)

5.6.1. Summary

> **i** At a glance
>
> **What it does.** Moves a column of cell averages, at fixed p/ρ per cell, to the fixed point $\overline{p} = p_{\mathrm{ref}}(\overline{p}) + C$ of Equation 5.2, so that the solver starts it at rest. **Switch.** None: an API function, `balance_column` in C++ and in the Python package, called by initial-condition builders. Requires `wb-wall-clamp` on, at least five cells, positive p and ρ. **Order.** Fixed point to a residual below `rtol` (default 10^{-10}, at most 120 sweeps). **Defining equation.** Equation 5.13.

A column integrated from the continuum hydrostatic equation is balanced to the truncation error of that integration, not to the scheme's own discrete balance. The well-balanced reconstruction then sees a perturbation p' that is not constant and turns its variation into a standing force on an atmosphere that should be at rest. `balance_column` removes that force before the run starts: it iterates the column onto the discrete rest state, holding the temperature of each cell fixed for ideal mixtures, and returns the balanced column, the residual and the number of updates. The fixed point is defined only up to a free constant C, so the column moves; any pinned quantity (surface pressure, column mass) has to be re-pinned by the caller.

Under `SNAP_WB_REF4` on a non-uniform grid it iterates against the switched cell pressure, which is the fixed point the solver then enforces. Under `SNAP_X1_CENTROID_EXACT` it balances only a column declared Cartesian.

Aliases. “the balance-ic projection” (the bryan example’s `balance-ic`), chengcli/snapy#250 D1.

5.6.2. Derivation

Units as in Section 5.1. Let the column hold $n_1 \geq 5$ cells with no ghosts and both ends physical, and let $\theta_i = \overline{p}_i/\overline{\rho}_i$ be taken once from the input. One sweep evaluates the reference kernel on the whole column, reads the gauge at the top cell and updates

$$\begin{aligned} C &= \overline{p}_{n_1-1} - p_{\mathrm{ref},n_1-1}, \\ \overline{p}_i &\leftarrow p_{\mathrm{ref},i}(\overline{p},\overline{\rho}) + C, \\ \overline{\rho}_i &\leftarrow \overline{p}_i/\theta_i, \end{aligned} \tag{5.13}$$

and stops when

$$\varepsilon = \max_i \frac{|\overline{p}_i - p_{\mathrm{ref},i} - C|}{\overline{\rho}_i\, g\, \Delta x_{1,i}} < \mathrm{rtol}. \tag{5.14}$$

Equation 5.13 and Equation 5.14 are the loop that calls `call_hydro_ref_x1` (`balance_column.cpp:85-104`). A fixed point of Equation 5.13 satisfies Equation 5.2 with $c = C$, so it is at rest in the solver. The residual is checked before the update, so the returned column is the one it describes and an already balanced column comes back after zero updates. ε bounds the residual acceleration in units of g: a perturbation difference $\delta p'$ across a cell drives an acceleration $\delta p'/(\overline{\rho}\Delta x_1)$.

The gauge is read at the top cell, where the pressure is lowest, so the column moves least where it is thinnest; the code comment records that a gauge at the bottom diverges on a deep column. Holding θ_i fixed holds the temperature fixed for `ideal-gas` and for `ideal-moist` at fixed mass

fractions. No convergence proof exists; the sweep is a contraction in practice, and the check shows it on a column that is not balanced to begin with (`[C15]`: a $\beta = 0.3$ polytrope with a 10^{-3} pressure bump at fixed p/ρ, $n_1 = 64$, residual 4.64×10^{-2} to 1.03×10^{-11} in 12 updates, p/ρ unchanged to 1.5×10^{-16}).

Why the result transfers to the blocks. With the wall clamp on, the reference of a block's interior reads only owned cells at a physical wall (Section 5.3), so a ghost-free column and a block holding the same cells get the same p_{ref}, bitwise; that is what the first test below asserts. Without the clamp the wall rows read ghost faces that the ghost-free column does not have, and the two disagree, which is why the function refuses to run. ρ_{sf} differs at the bottom cell only (its fallback keys on the array index), and it plays no part in the fixed point.

Exact and truncated. The returned column satisfies Equation 5.2 to $\varepsilon <$ rtol; nothing is truncated beyond that.

5.6.3. Numerical method

Figure 5.7 shows what the projection does. Each sweep is one call of the reference kernel on the column (CPU, CUDA or MPS, with the column's device), plus the switched cell pressure of Section 5.7 when that switch is on and the column is non-uniform; uniformity is decided with the solver's test on the column's own widths.

A moist initial condition needs one more loop. The projection holds the mass fractions fixed, and the run's first saturation adjustment moves the column off the balance again. The bryan example's `balance-ic` therefore alternates saturation and projection until saturation no longer moves the projected column (relative density change at most $4\epsilon_{\mathrm{mach}}$ and $\max|\Delta p/p| \leq 10^{-13}$), within `balance-ic-passes` (Figure 5.8).

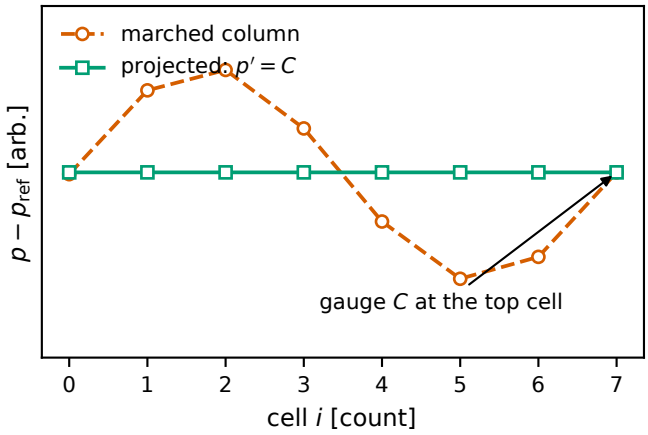


Figure 5.7.: What balance_column does to a column, x1 to the right. Vermillion dashed circles: the perturbation p - p_ref of a column marched from the continuum equation, which varies along the column; bluish-green squares: after the projection it is one constant C, the value at the top cell. The column itself moves by C everywhere. Illustrative values.

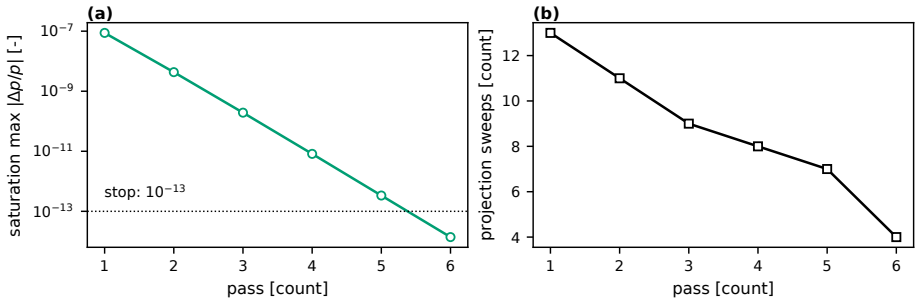


Figure 5.8.: The moist balance-ic loop of the bryan example, per pass. (a) Largest relative pressure change the saturation adjustment makes to the column just projected (bluish-green circles); the dotted line is the stop criterion 1e-13. (b) Projection sweeps balance_column needed in that pass (black squares). The change falls by about a factor 20 per pass and the loop stops after pass 6.

5.6.4. Code

Table 5.12.: Code map of `balance_column`.

what	code link	symbol	switch
refusals: gravity, shape, clamp, geometry	`balance_column.cpp:18-40`	`TORCH_CHECK`	`wall_clamp`
at least five cells, positive state	`balance_column.cpp:42-55`	`nc1`	
uniform test and switched pressure	`balance_column.cpp:65-79`	`ref4`	`SNAP_WB_REF4`

what	code link	symbol	switch
the sweep loop	balance_column.cpp:85-104	sweeps	rtol, max_iter
failure to converge	balance_column.cpp:106-108	err	
declaration and defaults	balance_column.hpp:103-106	balance_column	
Python binding	pyhydro.cpp:49-64	balance_column	
moist caller	bryan.cpp:221-262	balance_ic	balance-ic

1. The arguments are checked: $g > 0$, shape, matching widths, positive `rtol` and `max_iter`, the clamp, and the geometry under `SNAP_X1_CENTROID_EXACT`, each with a `TORCH_CHECK` (`balance_column.cpp:18-40`).
2. The column length `nc1` must be at least 5 and p, ρ positive; θ_i is stored (`balance_column.cpp:42-55`).
3. Uniformity is decided and, for a switched non-uniform column, the stencils of Section 5.7 are built into the flag `ref4` (`balance_column.cpp:65-79`).
4. The loop runs Equation 5.14 and Equation 5.13, counting `sweeps`, and raises if the pressure turns non-positive (`balance_column.cpp:85-104`).
5. A residual `err` still above `rtol` raises (`balance_column.cpp:106-108`).

Errors a user can hit: the clamp off, fewer than five cells, a non-positive input, a spherical-polar geometry under `SNAP_X1_CENTROID_EXACT`, a diverging column, and no convergence within `max_iter`. The caller also

owns four conditions the function cannot check, listed in its header: physical x_1 ends, at least five cells per block of the later run, the same uniform classification in every block, and the saturation loop for moist columns.

5.6.5. Tests

Table 5.13.: Tests of `balance_column`.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_balance_column.cpp` (`test_balance_column.release`), `a_ghost_free_column_reproduces_a_blocks_own_reference`	the column's p_{ref}, ρ_{ref} equal the block's	bitwise (ρ_{sf} except the bottom cell)	the transfer property
same, `without_the_clamp_the_two_references_disagree`	the control	$> 10^{-4}$ before; $<$ rtol after	non-vacuity
same, `a_marched_column_comes_out_at_rest`	a marched column, projected, does not move	residual $< 10^{-10}$	`rtol`
same, `the_last_allowed_update_can_converge`	convergence on the last allowed sweep is accepted (chengcli/snapy#278, chengcli/snapy#279)	10^{-10}	`rtol`

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `the_temperature_and_every_other_channel_stay_put`	p/ρ unchanged, other rows untouched	10^{-14} relative	round-off
same, `a_balanced_column_is_a_fixed_point`, `a_block_thinner_than_its_ghosts_keeps_its_columns_apart`, `it_refuses_what_it_cannot_deliver`	zero updates on a balanced column; column independence; refusals	exact	contract
same, `x1_centroid_switch_implies_the_ref4_predicate`, `x1_centroid_switch_balances_only_a_cartesian_column`	switch predicates and the geometry refusal	exact	contract
same, `a_moist_column_with_one_condensable_comes_out_at_rest`	moist column balanced and at rest after one step	$< 10^{-10}$; marched $> 10^{-6}$	`rtol`; non-vacuity
`test_balance_column_wb_ref4`, `test_balance_column_x1_centroid`	the same tests with each switch set	as above	as above

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_bryan_balance_ic`	one allowed pass fails with the cap error; the default converges; the dry case projects once	exit status, output match	contract

Evidence. All pass at the pin. The moist test prints audit residual 3.357×10^{-1} before and 8.896×10^{-11} after, in 13 sweeps, and $\max|v_1|/(g\Delta t)$ after one step 2.58×10^{-11} balanced against 2.05×10^{-2} marched, identical in all three arms. The bryan default case stops after pass 6 with saturation changes falling from 8.76×10^{-8} to 1.39×10^{-14} (Figure 5.8) [sha `e894700ff7aee30b52882e5202b16461413780b0`; decks: `tests/test_balance_column.cpp`, `tests/run_bryan_balance_ic.cmake` on `examples/bryan.cpp`; run: `ctest -V`, Release build, 2026-10-10, CPU, double, 1 rank; data `src/snapy_report/ch05/data/bryan_balance_ic.json`]. **Coverage.** CPU; the `_cuda` cases exist and were not run (**pending: a CUDA run at the pin**); no multi-rank case (the function works on one column).

5.6.6. Limits

> **Limits**
>
> The balanced column moves by a free constant: the header of `balance_column.hpp` quotes 2.5×10^{-4} of p on a Neptune-like column and 1.2×10^{-2} on a 300 km giant-planet one (no deck given; not re-measured). A spherical-polar column cannot be balanced under `SNAP_X1_CENTROID_EXACT`.

1. **Free gauge.** Nothing ties the result to the input's surface pressure or mass; callers re-pin inside their own loop.
2. **Classification.** The uniform test is made once for the whole column and must match every block's own test.
3. **Moist columns.** The caller must iterate with saturation adjustment, as the bryan example does.
4. **No convergence proof.** Convergence is observed (`[C15]`, the tests), not proven.
5. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.7. Fourth-order, cell/face-consistent density reference (`SNAP_WB_REF4`)

5.7.1. Summary

> **i** At a glance
>
> **What it does.** Replaces the kernel's density reference by $p_{\mathrm{ref}}F(\rho/p)$ with a fourth-order filter F, and the face density by the face value of that same cell field, so the face-density offset is $O(\Delta x_1^4)$; on non-uniform grids it also replaces the cell pressure by a cubic average. **Switch.** Environment variable `SNAP_WB_REF4`, read once per process, default off; implied by `SNAP_X1_CENTROID_EXACT`. Requires $n_g \geq 3$ (checked at setup). **Order.** $\delta\rho_f = O(\Delta x_1^4)$; p_{sf} is never changed, so the rest balance is the kernel's. **Defining equation.** Equation 5.15.

The kernel's density reference is second order: the binomial smoothing biases ρ/p by $\frac{\Delta x_1^2}{2}r''$, and its face density is not the face value of its own cell density, so the solver's face density carries the offset Equation 5.8. That offset enters the x_1 mass flux at linear order and, through it, the gravity work. The switch removes it with two operators: a five-point filter with zero second and third moments, which keeps the reference fourth order while still removing the two-cell mode, and a face value taken from the quartic through the primitive of the cell reference, so that face and cell are consistent by construction. On a non-uniform grid it also replaces the log-mean cell pressure by the average of a cubic through four face pressures.

Guards keep the kernel's values wherever the new ones would leave the local range or the column is under-resolved, and the cell part runs before the seam exchange of Section 5.4, the face part after it. With the corrected-PE face work of Chapter 6 on (its default with `gravity-work: face`) the

one-step spurious stratification of a neutral column converges at about fourth order with the switch and second order without it.

Aliases. "Leg W" in `docs/derivations/x1-centroid-spherical.md`; `SNAPY_WB_REF_EXACT` in the specification `sources/gw__NEXTPR_spec_wbref_exact.md`; #289 in comments.

5.7.2. Derivation

Notation as in Section 5.2; indices are cell indices and $r_i = \overline{\rho}_i/\overline{p}_i$. The derivation is `docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §§2-7, re-written here and checked by `[C3]`, `[C7]`, `[C8]`, `[C14]`.

5.7.2.1. Cell density

$$\begin{aligned} \rho_{\mathrm{ref},i} &= p_{\mathrm{ref},i}\,(Fr)_i, \\ F &= \tfrac{1}{16}(-1,\ 4,\ 10,\ 4,\ -1). \end{aligned} \tag{5.15}$$

F has unit sum, zero first, second and third moments and fourth moment $-\frac{3}{2}$ in cell units, and zero response at $k\Delta x_1 = \pi$ (`[C3]`), so $Fq = q + O(\Delta x_1^4)$ and the two-cell mode is still removed; its reach, ± 2, is the binomial's, so ghost depth and seam exchange are unchanged. With $p_{\mathrm{ref}} = \overline{p} + c + O(\Delta x_1^6)$ (Section 5.2) the cell reference is $\overline{\rho} + cR + O(\Delta x_1^4)$, and the smooth cR is carried consistently by the face value below. Equation 5.15 is built in `wb_ref4_stencils` as the filter row `F` (`wb_ref4.cpp:120-158`) and applied in `wb_ref4_cells` as `fr` (`wb_ref4.cpp:263-272`).

At a clamped wall the two values past the wall are cubic extrapolations of the four owned cells nearest it, $r_0..r_3$ counted from the wall:

$$\begin{aligned} r_{-1} &= 4r_0 - 6r_1 + 4r_2 - r_3, \\ r_{-2} &= 10r_0 - 20r_1 + 15r_2 - 4r_3. \end{aligned} \tag{5.16}$$

All five stencil values of the first two cells then lie on one cubic, which F reproduces, so $(Fr)_0 = r_0$ and $(Fr)_1 = r_1$ for any data ([C7]): filtering with these wall values and leaving the two end cells unfiltered are the same reference. Ghosts are never read.

5.7.2.2. Face density

$$\begin{aligned} \rho_{\mathrm{sf},f} &= P'(x_{1,f}), \\ P(x_{1,s+j}) &= \sum_{k<j} \rho_{\mathrm{ref},s+k}\,\Delta x_{1,s+k}, \end{aligned} \tag{5.17}$$

with P the quartic through the primitive at the five faces $j = 0..4$ (four cells), window $s = f - 2$ kept inside the owned cells at a clamped wall, physical spacing. On a uniform grid the weights on $\rho_{\mathrm{ref},s..s+3}$ are $(-1, 7, 7, -1)/12$ in the interior, $(25, -23, 13, -3)/12$ at the wall face and $(3, 13, -5, 1)/12$ at the next ([C8]). The face is exact for cubic point profiles on any grid, so $\delta\rho_f = \rho_{\mathrm{sf}} - \mathcal{R}[\rho_{\mathrm{ref}}] = O(\Delta x_1^4)$ ([C14]: mid-column offset 3.2×10^{-12} to 2.8×10^{-16} relative from $n_1 = 32$ to 256, fitted order 4.68, against order 2.00 for the kernel). Equation 5.17 is built as the weights `awt` (`wb_ref4.cpp:195-221`) and applied by `wb_ref4_faces` (`wb_ref4.cpp:276-286`).

5.7.2.3. Non-uniform cell pressure

On a non-uniform grid the cell pressure becomes the cell average of the cubic $\tilde{p}$ through the face pressures at faces $i - 1..i + 2$ (window inside the owned faces at a clamped wall), by three-point Gauss-Legendre, exact for a cubic:

$$p_{\mathrm{ref},i} = \frac{1}{\Delta x_{1,i}} \int_{x_{1,i-1/2}}^{x_{1,i+1/2}} \tilde{p}\,\mathrm{d}x_1 = \sum_{q=1}^{3} \frac{w_q}{2}\,\tilde{p}(x_{1,i} + \xi_q \tfrac{\Delta x_{1,i}}{2}), \tag{5.18}$$

with nodes $\xi_q = 0, \pm\sqrt{3/5}$ and weights $w_q = 8/9, 5/9$, the non-uniform branch of `wb_ref4_stencils` with the Gauss weights `gw` (`wb_ref4.cpp: 165-193`). A value outside the cell's two face pressures keeps the log-mean. This moves the discrete fixed point of a non-uniform column by $O(\Delta x_1^2)$, so `balance_column` iterates against it (Section 5.6). On a uniform grid the six-face p_{ref} is kept.

5.7.2.4. Guards and rest balance

A cell whose $(Fr)_i$ lies outside $[\min, \max]$ of r over cells $i-1..i+1$, with relative margin 10^{-10}, keeps the kernel's ratio; a face outside its two neighbouring cell references keeps the kernel's face. A cell with $|\ln(p_{\mathrm{sf},i-1/2}/p_{\mathrm{sf},i+1/2})| > 0.5$, fewer than two cells per pressure scale height, is flagged, the flag is dilated by two cells, and flagged cells and faces next to them keep the kernel's values; ghosts past a clamped wall do not count. The flag reads p_{sf} at most three cells away, which is why $n_g \geq 3$ is required: then both sides of an x_1 seam compute the same flag. The threshold, dilation and margin are design choices, not derived. Since p_{sf} is never changed and on a uniform grid neither is p_{ref}, the rest state Equation 5.2 is the kernel's.

Exact and truncated. Exact: the end-cell identity, the face weights for cubics, Equation 5.18 for cubics. $O(\Delta x_1^4)$: ρ_{ref} and $\delta\rho_f$ in the interior. At the three faces next to a wall the note gives $O(\Delta x_1^3)$ (the anchor constant through the even ghosts); not re-checked here.

5.7.3. Numerical method

Figure 5.9 shows the stencils. The stencils and weights are built once per block, on the CPU in double precision, and cached until device or dtype changes; application is a gather and a weighted sum on the block's device. The cell part runs right after the kernel in `_hydro_ref_x1`, before the seam ghost-row exchange; the face part runs after it, so a column split

along x_1 gets the same faces as one block. A block with a clamped wall and fewer than four owned cells keeps the kernel's reference.

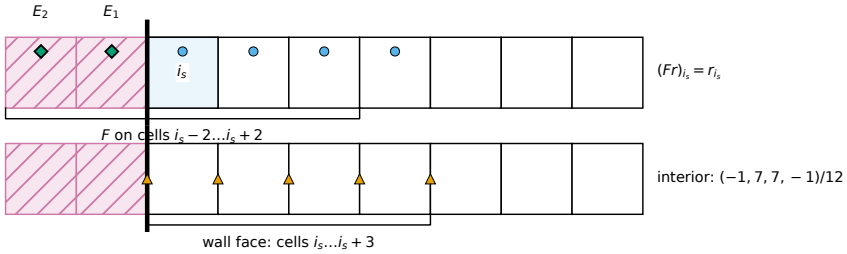


Figure 5.9.: The SNAP_WB_REF4 stencils at a bottom wall (thick black line), x1 to the right. Top: the filter F on the wall cell i_s reads two values past the wall (bluish-green diamonds E_1, E_2), which are cubic extrapolations of the four owned cells nearest it (sky-blue circles), never the hatched ghosts; with them the filtered ratio equals r in the end cell. Bottom: the face density at the wall face from the quartic primitive over the four owned cells next to the wall (orange triangles at the five faces); interior faces use the centred weights (-1, 7, 7, -1)/12.

The oracle and the measured order. The test drives a neutral polytrope ($\gamma = 5/3$, $g = m+1$ with $m = 1/(\gamma-1)$, $T_0 = 1+L_z-z$) seeded with a divergence-free overturning mode of amplitude 10^{-4}, takes one RK3 step, and measures $N^2_{\mathrm{eff}} = \frac{g}{\gamma}\sum_i(-\delta s_i)w_i / \sum_i w_i^2$ from the entropy tendency of the mode minus that of the rest state. For this column the exact cell-average tendencies vanish at linear order, so every N^2_{eff} is scheme error. Figure 5.10 shows the four arms measured at the pin.

With the corrected-PE face work at its default, the observed orders of $|N^2_{\mathrm{eff}}|$ are 4.07, 4.04 (1 e-fold) and 3.88, 3.94 (3 e-folds) with the switch, and 2.13, 2.04 and 2.21, 2.08 without it. With `SNAP_GRAVITY_WORK_RADIAL_EXACT=0` the switched arm returns 2.96, 2.98 and 2.84, 2.92, and its values

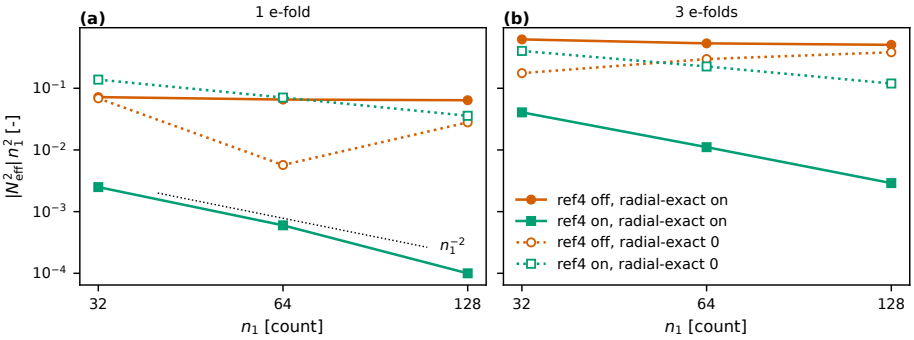


Figure 5.10.: One-step spurious stratification |N2_eff| n1^2 of a neutral column against the number of cells, (a) 1 and (b) 3 pressure e-folds deep. Vermillion circles: SNAP_WB_REF4 off; bluish-green squares: on. Solid lines: the corrected-PE face work at its default (on); dotted lines with open markers: SNAP_GRAVITY_WORK_RADIAL_EXACT=0. A flat line is second-order convergence of N2_eff; the black dotted guide has slope -2, i.e. fourth order. With the default gravity work the switch takes the error from second to fourth order.

-0.0707, -0.0358 (1 e-fold, $n_1 = 64$, 128) and -0.2257, -0.1194 (3 e-folds) equal the table of `docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §9 to the printed digits. The order-3 "wall band" that note leaves unidentified is therefore removed by the corrected-PE face work: tested here by switching that work off and on with everything else fixed, not derived.

5.7.4. Code

Table 5.14.: Code map of `SNAP_WB_REF4`.

what	code link	symbol	switch
the switch, implied by the centroid switch	`wb_ref4.cpp:83-95`	`wb_ref4_enabled`	`SNAP_WB_REF4`
margin and resolution threshold	`wb_ref4.cpp:20-25`	`kMaxLogDrop`	
stencils and weights	`wb_ref4.cpp:97-231`	`wb_ref4_stencils`	
cell part and flag	`wb_ref4.cpp:233-274`	`wb_ref4_cells`	
face part	`wb_ref4.cpp:276-286`	`wb_ref4_faces`	
ghost-depth check	`hydro.cpp:94-103`	`wb_ref4_enabled`	$n_g \geq 3$
calls around the seam exchange	`hydro.cpp:565-575`	`wb_ref4_cells`	
face part after the exchange	`hydro.cpp:642-645`	`wb_ref4_faces`	

what	code link	symbol	switch
`balance_column` follows it	`balance_column.cpp:70-79`	`wb_ref4_enabled`	non-uniform only

1. `wb_ref4_enabled` reads the environment once; values other than empty, `0`, `false`, `off`, `no` turn it on, and `SNAP_X1_CENTROID_EXACT` turns it on too (`wb_ref4.cpp:83-95`).
2. At setup, with gravity on, $n_g < 3$ fails the `TORCH_CHECK` that names `wb_ref4_enabled` (`hydro.cpp:94-103`).
3. In `_hydro_ref_x1` the stencils are built once per block and `wb_ref4_cells` replaces ρ_{ref} (and on a non-uniform grid p_{ref}) and returns the resolution flag (`hydro.cpp:565-575`).
4. After the seam exchange `wb_ref4_faces` replaces ρ_{sf} (`hydro.cpp:642-645`).

The only user-facing error is the ghost-depth check of step 2.

5.7.5. Tests

Table 5.15.: Tests of `SNAP_WB_REF4`.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_wb_ref4_order.py` (`test_wb_ref4_order_python`)	switch on: order of $\lvert N^2_{\text{eff}}\rvert \geq 2.75$ over both doublings at 1 and 3 e-folds, n_1 32/64/128; off: < 2.5 at the last doubling	`ORDER_ON 2.75`, `ORDER_OFF 2.5`	set between the measured off (about 2) and on orders; not derived
`tests/test_x1_seam_split.cpp` (`test_x1_seam_split_wb_ref4.release`), `wb_ref4_flag_at_the_seam_split_matches_one_block`	a split column with the flag switching on next to the seam equals one block	10^{-13}	round-off
same (`test_x1_seam_split_wb_ref4_gravity_0.release`)	with gravity off, $n_g = 1$ still steps	runs	the check applies with gravity only

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_x1_seam_split_mp_wb_ref4` (2 ranks)	two ranks equal two blocks and one block	10^{-13}	round-off
`test_balance_column_wb_ref4`, `test_face_floor_wb_ref4`	`balance_column` finds the switched fixed point; switched dipped-face fluxes -1.19257×10^{-7} and 0.0303502	10^{-3} relative; 10^{-6}	pinned values
this chapter's check	`[C3]`, `[C7]`, `[C8]`, `[C14]`	per claim	stated in the check

Evidence. $N_{\mathrm{eff}}^2 n_1^2$ at $n_1 = 32$, 64, 128, re-measured at the pin (ISSUES.md item 3): switch off +0.0719, +0.0656, +0.0640 (1 e-fold) and +0.6198, +0.5344, +0.5060 (3 e-folds); on +0.0025, +0.0006, +0.0001 and +0.0407, +0.0111, +0.0029 [sha `e894700ff7aee30b52882e5202b16461413780b0`; deck: `tests/test_wb_ref4_order.py`; run: `python tests/test_wb_ref4_order.py` against the pin's build, 2026-10-10, CPU, double, 1 rank; data `src/snapy_report/ch05/data/wb_ref4_order.json`]. The `SNAP_GRAVITY_WORK_RADIAL_EXACT=0` arms of Figure 5.10 come from a second run of the same script with that variable set. The seam tests print gaps of 6.775×10^{-15} and 1.675×10^{-15} ($n_h = 1, 0$) for the split and 0 for two ranks against two blocks. The ctest entry `test_wb_ref4_order_python` itself failed in a first run because the test imported an older installed snapy instead of the pin's build; run against the pin's build it passes, as above. **Coverage.** CPU; the CUDA twin `test_wb_ref4_order_cuda` and the `_cuda` seam arms were not run (**pending: a CUDA run at the pin**);

2 ranks through the `_mp` arm.

5.7.6. Limits

> **Limits**
>
> `SNAP_WB_REF4` needs `nghost` ≥ 3. It is read once per process, so every block of a run makes the same choice.

1. **Thresholds not derived.** The 0.5 resolution threshold, the two-cell dilation and the 10^{-10} margin are design choices (`docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §7).
2. **Untested at the pin.** Stretched-grid dynamics (only `balance_column` exercises the non-uniform branch), multi-species columns, and seams across more than two ranks.
3. **Resolution of the order test.** The test grid is $n_1 = 32/64/128$ against 64/128/256 in the note's table, and the threshold `ORDER_ON` 2.75 no longer distinguishes the fourth order measured at the pin from the third order of the note; the test still separates on from off.
4. **Names.** The specification calls the switch `SNAPY_WB_REF_EXACT`; the code reads `SNAP_WB_REF4`.
5. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.8. r^2-exact x_1 maps on spherical-polar grids (`SNAP_X1_CENTROID_EXACT`)

5.8.1. Summary

> **i** At a glance
>
> **What it does.** On spherical-polar blocks, feeds every x_1 map (reconstruction, scan, reference) the plain means of the cells instead of their r^2 means, and replaces the radial pressure source by the r-moment of a quintic, so a hydrostatic column of r^2 means is balanced to round-off. **Switch.** Environment variable `SNAP_X1_CENTROID_EXACT`, read once per process, default off. Implies `SNAP_WB_REF4`. Requires $n_g \geq 3$. Acts on `spherical-polar` only. **Order.** Conversion exact to degree 4; rest exact for densities of degree ≤ 4. **Defining equation.** Equation 5.21.

A finite-volume cell on a spherical-polar grid holds the r^2 mean of a quantity, whose centroid sits $h^2/(6\overline{r})$ above the mid-radius. Every x_1 cell-to-face map of the solver, from the WENO weights to the scan and the six-face cell pressure, is a formula for plain means. Fed r^2 means they are off by $O(h^2/r)$: the face values, the scan step, and the pressure force, whose plain difference balances gravity only against the wrong scan. The switch converts the inputs of the maps, for the maps only, to plain means with a five-cell stencil exact to degree 4, and makes the radial pressure force the r^2 mean of the gradient of a quintic through six face pressures. The reference on plain means is then the Cartesian one, and the switch implies `SNAP_WB_REF4` so that it is fourth order.

On Cartesian blocks the switch only implies `SNAP_WB_REF4`; on cubed-sphere blocks it does nothing (their x_{1v} is the mid-radius). `balance_column` balances only a column declared Cartesian under it.

Aliases. #289 legs (i)-(iii) in comments; “the centroid switch”.

5.8.2. Derivation

Units: r and h [m]; $A_{i\pm1/2} = r_{i\pm1/2}^2$ and $V_i = (r_{i+1/2}^3 - r_{i-1/2}^3)/3$ are per unit solid angle, as the code forms them. $\langle q\rangle_i = V_i^{-1}\int r^2 q\,\mathrm{d}r$ is the r^2 mean, $\overline{q}_i = h_i^{-1}\int q\,\mathrm{d}r$ the plain mean, and $x = (r-\overline{r}_i)/h_i$ the local coordinate. Re-written from `docs/derivations/x1-centroid-spherical.md@e894700ff7aee30b52882e5202b16461413780b0` §§1-5.

5.8.2.1. The defect

Expanding q about the mid-radius,

$$\begin{aligned}\langle q\rangle_i &= \overline{q}_i + \delta_i\, q'(\overline{r}_i) + O(h^4/\overline{r}^2),\\ \delta_i &= \frac{2h_i^2\,\overline{r}_i}{h_i^2 + 12\,\overline{r}_i^2} = \frac{h_i^2}{6\overline{r}_i} - \frac{h_i^4}{72\,\overline{r}_i^3} + \cdots\end{aligned} \tag{5.19}$$

(`[C18]`, exact). Each uniform-measure map fed $\langle q\rangle$ is off by $\delta q' = O(h^2/r)$; in particular the scan step becomes $gh\langle\rho\rangle$ instead of $g\int\rho\,\mathrm{d}r = gh\overline{\rho}$, and the perturbation of a resting column is $O(h^2/r)$ instead of a constant.

5.8.2.2. Plain means

For the x_1 maps only, each primitive row is replaced by

$$\begin{aligned}\overline{q}_i &= \sum_{k=0}^{4} c_{i,k}\,\langle q\rangle_{s_i+k},\\ \sum_k c_{i,k}\,\langle x^n\rangle_{s_i+k} &= \overline{x^n}_i = \left(1,\ 0,\ \tfrac{1}{12},\ 0,\ \tfrac{1}{80}\right)_n,\end{aligned} \tag{5.20}$$

for the monomials $n = 0..4$, window $s_i = \max(\mathrm{lo}, \min(\mathrm{hi} - 4, i - 2))$, centred, inside the owned cells at a physical wall and inside the array at a seam. The moments are integrated by four-point Gauss-Legendre, exact for degree 6, and the 5×5 system is solved once per block. The conversion is exact for degree 4 and not 5, on uniform and stretched grids and in the wall windows, and its weights sum to 1 (`[C16]`). Equation 5.20 is `x1_plain_mean_stencils` (`x1_centroid.cpp:104-155`). Ghosts past a clamped wall get the mirrored correction of the owned cells, odd for the normal velocity, in `x1_plain_means` (`x1_centroid.cpp:157-182`); at a seam the outer ghosts' windows are off-centre, so their plain means are replaced by the neighbour's.

5.8.2.3. The pressure source

With $\tilde{p}$ the quintic through the six face pressures nearest cell i (window over faces, one-sided at a wall, reaching two faces past a seam), the radial pressure source is

$$S_i = \frac{2}{V_i} \int_{r_{i-1/2}}^{r_{i+1/2}} r\, \tilde{p}\, \mathrm{d}r = \sum_{j=0}^{5} w_{i,j}\, p^*_{s+j}, \tag{5.21}$$

and integration by parts, with $\tilde{p}$ through both faces of the cell, gives $-(A_{i+1/2} p^*_{i+1/2} - A_{i-1/2} p^*_{i-1/2})/V_i + S_i = -V_i^{-1} \int r^2 \tilde{p}'\, \mathrm{d}r$: the net pressure force is the r^2 mean of $-\partial_r \tilde{p}$. A constant pressure gives $S_i V_i = A_{i+1/2} - A_{i-1/2}$ exactly, with V_i in the cube form of the cell volume (`[C17]`, both identities in exact rationals). Equation 5.21 is `x1_pressure_source_stencils` (`x1_centroid.cpp:184-222`), used in place of the plain difference by the branch that tests `exact_x1` (`spherical_polar.cpp:251-264`). The hydrostatic mode takes the same operator (Section 5.5).

5.8.2.4. Rest balance

On a hydrostatic column of r^2 means: the conversion gives $\overline{\rho}$ to $O(h^6/r)$; the scan then gives the exact face pressures plus one constant; the reference on plain means is the Cartesian fourth-order one; and the net force $-V^{-1}\int r^2(\tilde{p}' + g\rho)\,\mathrm{d}r$ has the constant drop out. For a density of degree ≤ 4 every step is exact and the column is balanced exactly, which is the oracle of the test.

Exact and truncated. Exact: the conversion for degree 4, both source identities, rest for degree ≤ 4. For a smooth column the curvature part of the residual is $O(h^6/r)$; what remains is the truncation of the reference itself. Ratios of means (the Favre velocity, $T = p/\rho$, ρ/p inside F) are not made exact (1).

5.8.3. Numerical method

Figure 5.11 shows the windows. In `HydroImpl::forward` the plain-mean view `wx1` is computed before the reference, its seam ghost rows are taken from the neighbour, and the reconstruction and the reference read it; the primitives themselves are unchanged. After the Riemann solve the two ghost face pressures each side of a seam are taken from the neighbour, so the source window is the centred one-block window. Everything runs in every RK stage, explicitly; the stencils are built once per block in double precision.

5.8.4. Code

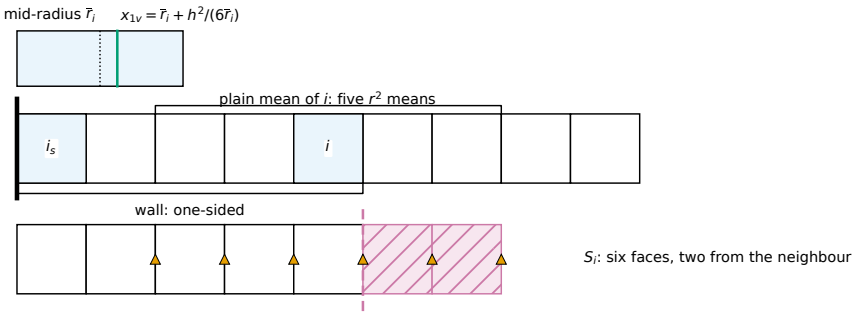


Figure 5.11.: The maps of SNAP_X1_CENTROID_EXACT, x1 = r to the right. Top: a radial cell, its mid-radius (dotted) and its r^2 centroid x1v (bluish-green line) h^2/(6 r) further out. Middle: the five-cell window that converts r^2 means to the plain mean of cell i, centred in the interior and one-sided at the wall (thick black line). Bottom: the six-face window of the pressure source (orange triangles), which reaches two faces past a seam (dashed reddish-purple line) into the neighbour's hatched ghost cells.

Table 5.16.: Code map of `SNAP_X1_CENTROID_EXACT`.

what	code link	symbol	switch
the switch	`x1_centroid.cpp:92-102`	`x1_centroid_exact_enabled`	`SNAP_X1_CENTROID_EXACT`
conversion stencils	`x1_centroid.cpp:104-155`	`x1_plain_mean_stencils`	
conversion and mirrored ghosts	`x1_centroid.cpp:157-182`	`x1_plain_means`	
source stencils	`x1_centroid.cpp:184-222`	`x1_pressure_source_stencils`	
plain-mean view and seam ghosts	`hydro_forward.cpp:270-285`	`x1_plain_means`	spherical-polar
ghost face pressures at a seam	`hydro_forward.cpp:501-508`	`_x1_ghost_rows`	
the source in the momentum equation	`spherical_polar.cpp:251-264`	`x1_pressure_source`	
ghost-row exchange	`hydro.cpp:270-303`	`_x1_ghost_rows`	x_1 split
implies `SNAP_WB_REF4`	`wb_ref4.cpp:92-94`	`x1_centroid_exact_enabled`	

1. `x1_centroid_exact_enabled` reads the environment once (`x1_centroid.cpp:92-102`).
2. On a spherical-polar block `HydroImpl::forward` builds the view with `x1_plain_means` and exchanges its seam ghost rows with `_x1_ghost_rows` (`hydro_forward.cpp:270-285`).
3. The reference and the reconstruction of Section 5.1 read the view;

SNAP_WB_REF4 acts through wb_ref4_enabled (Section 5.7).

4. After the Riemann solve the face-pressure ghost rows are exchanged with _x1_ghost_rows (hydro_forward.cpp:501-508).
5. SphericalPolarImpl::forward adds x1_pressure_source in place of the plain-difference correction (spherical_polar.cpp: 251-264).

The ghost-depth check of Section 5.7 covers this switch, since it implies SNAP_WB_REF4; balance_column refuses a non-Cartesian geometry under it. A block too thin for the stencils (usable false) keeps the plain path silently.

5.8.5. Tests

Table 5.17.: Tests of SNAP_X1_CENTROID_EXACT.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
tests/test_x1_centroid_rest.py (test_x1_centroid_rest_python)	linear-density column of r^2 means, $r_0/H = 5$ and 1000, explicit and vertically implicit, $n_h = 1$ and 0: on, $\max\|\rho v_1\|/(\Delta t\, g\rho) < 10^{-10}$ in interior and wall cells; off, $> 10^{-8}$ at $r_0/H = 5$	TOL_ON 1e-10, TOL_OFF 1e-8	round-off margin; an ignored switch fails

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_x1_seam_split.cpp` (`test_x1_seam_split_x1_centroid.release`), `centroid_exact_split_matches_one_block`	split equals one block after 20 steps, $n_h = 1$ and 0	10^{-13}	round-off
`test_x1_seam_split_mp_x1_centroid` (2 ranks)	two ranks equal two blocks and one block	10^{-13}	round-off
`test_balance_column_x1_centroid`, `test_face_floor_x1_centroid`	the balance and floor tests with the switch	as in their sections	as there
this chapter's check	`[C16]`–`[C18]`	per claim	stated in the check

Table 5.18.: Rest force $\max|\rho v_1|/(\Delta t\, g\rho)$ after one RK3 step, five of the eight cases the test prints. Evidence: sha `e894700ff7aee30b52882e5202b16461413780b0`; deck `tests/test_x1_centroid_rest.py`; run `python tests/test_x1_centroid_rest.py` against the pin's build, 2026-10-10, CPU, double, 1 rank.

r_0/H	implicit	n_h	off, interior / wall	on, interior / wall
5	0	1	2.377×10^{-5} / 2.700×10^{-5}	2.162×10^{-14} / 1.547×10^{-14}
5	0	0	6.001×10^{-8} / 7.447×10^{-7}	8.347×10^{-15} / 1.415×10^{-14}
5	1	1	2.363×10^{-5} / 2.367×10^{-5}	1.432×10^{-14} / 9.680×10^{-15}
1000	0	1	6.510×10^{-10} / 6.984×10^{-10}	2.421×10^{-14} / 3.261×10^{-14}
1000	1	0	1.403×10^{-12} / 1.022×10^{-11}	8.888×10^{-15} / 8.469×10^{-15}

Evidence. Table 5.18: with the switch the rest force is at round-off in every case; without it the $r_0/H = 5$ column feels 2.4×10^{-5}. The seam arm prints a split-against-one-block gap of 0 in both modes, and the 2-rank arm 0 against both two blocks and one block [same sha; decks the test files; run `ctest -V`, 2026-10-10, CPU, double, 1 and 2 ranks]. **Coverage.** CPU; the CUDA twins were not run (**pending: a CUDA run at the pin**).

5.8.6. Limits

> **Limits**
>
> The switch does nothing on cubed-sphere grids. Under it `balance_column` balances only a column declared `geometry='cartesian'`.

1. **Ratios of means.** The Favre velocity, T and ρ/p are still treated as means; the $O(h^2/R)$ remainder is

    1.

2. **Cubed sphere.** Not implemented: the gnomonic grids store the mid-radius as x_{1v} and would need the conversion with their own radial measure.
3. **Note history.** The copy of the derivation in `sources/deriv__x1-centroid-spherical.md` predates the seam exchange and says the switch leaves the reference unchanged; at the pin it implies `SNAP_WB_REF4` and seam ghosts are exchanged.
4. **Untested combinations.** See the switch coverage matrix of Chapter 12.

5.9. The $1/R$ remainder on spherical-polar grids: what is left after the corrections (no switch)

5.9.1. Summary

> **i** At a glance
>
> **What it is.** The part of the $1/R$ error of a spherical column that survives `SNAP_FLUX_COVARIANCE` and `SNAP_X1_CENTROID_EXACT`: an $O(h^2/R)$ term from reconstructing the Favre velocity $\langle\rho w\rangle/\langle\rho\rangle$ as if it were a plain mean. **Switch.** None; a property of the scheme. The related switches are `SNAP_FLUX_COVARIANCE` (Chapter 4), `SNAP_X1_CENTROID_EXACT` (Section 5.8) and `SNAP_GRAVITY_WORK_RADIAL_EXACT` (Chapter 6). **Order.** $O(h^2/R)$. **Defining equation.** Equation 5.22.

On a spherical-polar grid the r^2 cell measure adds terms of relative size h/R to every cell-to-face map, and the divergence adds a $2/r$ factor to every flux error. Chapter 4's covariance corrections and the plain-mean maps of Section 5.8 remove the leading such terms of the face fluxes, the reference and the pressure force. What remains is not a defect of any map but of an input: the x_1 velocity the maps reconstruct is a ratio of two r^2 means, not a mean, and the conversion of Equation 5.20 makes means plain, not ratios exact. This section records that remainder, its signature, and what evidence exists for it at the pin; it has no code of its own.

Aliases. "the 1/R term", "leg (d)" of #289; "hypothesis H1" in `sources/gw__ONEOVERR_split_RESULT.md`.

5.9.2. Derivation

Notation as in Section 5.8; R [m] is the radius of the column, $w = v_1$. The stored velocity of cell i is the Favre mean $\tilde{w}_i = \langle\rho w\rangle_i/\langle\rho\rangle_i$. Expanding ρ

and w about the mid-radius with the moments of Equation 5.19, and the plain mean likewise,

$$\widetilde{w}_i - \overline{w}_i = \frac{h_i^2}{12}\,\frac{\rho'\,w'}{\rho} + \delta_i\,w' + O(h^4), \tag{5.22}$$

where the first term is the Cartesian covariance of a density-weighted mean and the second the centroid offset of the r^2 measure (this paragraph only: primes are r-derivatives; derived for linear ρ and w in `[C21]`, exact in sympy). The conversion of Equation 5.20 treats $\tilde{w}$ as an r^2 mean and so removes the $\delta_i w'$ part; the covariance part stays. Reconstructing the converted value with weights for plain means therefore leaves an $O(h^2)$ face error, which the $2/r$ of the radial divergence turns into an $O(h^2/R)$ tendency. The same holds for $T = p/\rho$ and for ρ/p inside the reference filter. The expansion follows `docs/derivations/x1-centroid-spherical.md@e894700ff7aee30b52882e5202b16461413780b0` §6; the harness and the $+g/(6c_pR)$ signature of the remainder in the effective stratification are in `sources/study__next-1overR_README.md` §§0-3, and the completeness of the face fluxes to $O(h^4)$ including h^2/R is `sources/gw__ONEOVERR_split_RESULT.md` §§1-2.

Exact and truncated. The remainder is $O(h^2/R)$ and vanishes for $R \to \infty$ (Cartesian).

5.9.3. Numerical method

Figure 5.12 shows the effect in one cell. The terms involved live in the x_1 block of `HydroImpl::forward` (the plain-mean view of Section 5.8), in the covariance corrections of Chapter 4 and in the geometric sources of `SphericalPolarImpl::forward`; none of them is specific to this section.

The measured remainder needs a closure run in snapy at the pin (ISSUES.md item 5): $R[\varepsilon n_1^2(R) - \varepsilon n_1^2(\infty)]$ against R/H for the off and on arms, from a committed deck.

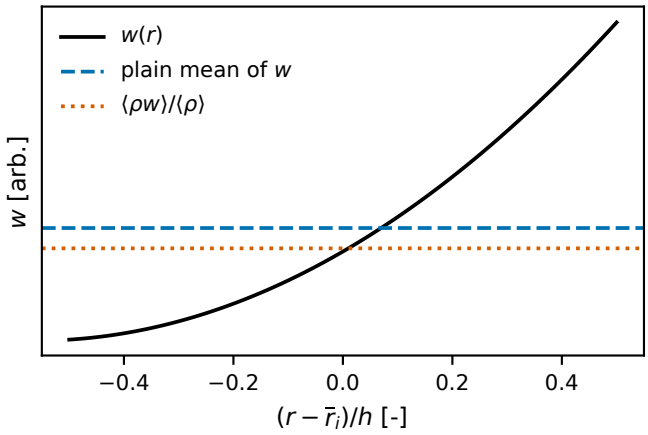


Figure 5.12.: Why a ratio of means is not a mean, in one radial cell. Black: a velocity profile w(r) across the cell; dashed blue: its plain mean; dotted vermillion: the Favre mean / with density falling outward and the r^2 weight. The two means differ by O(h^2), which the x1 maps treat as if it were zero. Illustrative profiles.

PENDING: snapy_report.ch05.fig_oneoverr_remainder.make_fig

Figure 5.13.: The 1/R content of the one-step effective stratification, R times the difference from the Cartesian limit, against R/H, with SNAP_X1_CENTROID_EXACT off and on. Pending: needs the in-snapy closure run at the pin.

5.9.4. Code

No dedicated code. The pieces it depends on are listed in their own sections.

Table 5.19.: Code map of the $1/R$ remainder.

what	code link	symbol	switch
the plain-mean	`hydro_forward.cpp:270-285`	`x1_plain_means`	`SNAP_X1_CENTROID_EXACT`
view the maps read			
the radial source	`spherical_polar.cpp:243-279`	`src1`	

5.9.5. Tests

Table 5.20.: Tests of the $1/R$ remainder.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
none in ctest	no test asserts the remainder; `test_x1_centroid_rest_python` covers rest only	—	—

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
this chapter's check, `[C21]`	the Favre-ratio expansion Equation 5.22 for linear profiles	exact	sympy

Evidence. The derivation note quotes code values of the $1/R$ content, off and on, at $R/H = 5$ to 1000 and $n_1 = 64$ and 128, with no committed script or deck: **missing evidence**, and per ISSUES.md item 5 the one-step harness numbers from commits not in snapy are not used. **Pending: an in-snapy closure run at the pin, with a committed deck, to rebuild the table and Figure 5.13. Coverage.** None.

5.9.6. Limits

1. **Not removed.** The remainder is a known $O(h^2/R)$ term of the scheme; removing it would need the ratios themselves made exact, which no switch does.
2. **Metric of the harness.** The note reports column means with unit weight per cell; weighted by cell volume the metric carries its own $1/R$ across the column's error profile and does not measure the scheme's term (same note, §6).
3. **Gravity-work interplay.** The face-form gravity work and its curvature flux on radial grids contribute their own $1/R$ content; that is Chapter 6 (`sources/study__next-1overR-remainder_README.md` §§0-4).

4. **Stale references.** The study READMEs were written before chengcli/snapy#293 merged; their file and line references do not match the pin.

5.10. Legacy x_1 wall routines: isentropic wall ghosts and the wall-face state copy (no switch)

5.10.1. Summary

> At a glance
>
> **What it does.** Two older wall treatments of the non-well-balanced x_1 path: an isentropic extrapolation of p and ρ into the wall ghosts (defined, never called) and a copy of the right state's p and ρ into the left state at the wall face (called only on the non-well-balanced path with gravity on). **Switch.** None. **Status.** Effectively unreachable: with gravity on and any EOS other than `shallow-water` the well-balanced path of Section 5.1 runs instead, and the shallow-water EOS has no pressure row for these routines to use. **Defining equation.** Equation 5.23.

Before the well-balanced reconstruction, a stratified column at a wall was handled by filling the ghost cells with an isentropic continuation of the wall cell and by making the two states at the wall face agree. Both routines are still in the code. The ghost fill's call in `HydroImpl::forward` is commented out; the face copy runs only on the x_1 path taken when `wb_x1` is false and gravity is on, a combination no configuration reaches with a pressure row.

5.10.2. Derivation

The face copy sets, at the lower wall face $i_s - \frac{1}{2}$ (and symmetrically at the upper one),

$$\begin{aligned} p^L_{i_s-1/2} &\leftarrow p^R_{i_s-1/2}, \\ \rho^L_{i_s-1/2} &\leftarrow \rho^R_{i_s-1/2}, \end{aligned} \tag{5.23}$$

in `_revise_x1inner_lr` (`hydro.cpp:448-453`), so the Riemann solver sees no pressure jump at the wall. The ghost fill holds $K = p/\rho^\gamma$ of the wall cell and integrates $\mathrm{d}(p^a)/\mathrm{d}x_1 = -a\,g/K^{1/\gamma}$, $a = (\gamma-1)/\gamma$, cell by cell into the ghosts, the isentropic hydrostatic profile, in `_revise_x1inner_ghost` (`hydro.cpp:462-480`). Neither is used at the pin, so neither has a check.

5.10.3. Numerical method

Figure 5.14 shows the copy. It would run after the x_1 reconstruction, in every stage, on the faces of physical x_1 walls.

5.10.4. Code

Table 5.21.: Code map of the legacy wall routines.

what	code link	symbol	switch
face copy, lower and upper	`hydro.cpp:448-460`	`_revise_x1outer_lr`	
isentropic ghosts, lower and upper	`hydro.cpp:462-500`	`_revise_x1outer_ghost`	

what	code link	symbol	switch
the commented-out call	`hydro_forward.cpp:247-250`	`_revise_x1inner_ghost`	
the only live call	`hydro_forward.cpp:378-384`	`_revise_x1inner_lr`	`wb_x1` false, gravity on

5.10.5. Tests

No test exercises either routine at the pin; the live call is unreachable in every registered test.

5.10.6. Limits

1. **Dead code.** The ghost fill is never called and the face copy is unreachable with a pressure row; they are kept, not maintained.
2. **If revived.** The isentropic fill assumes one γ for the column and an isentropic wall region; it would conflict with the even-parity perturbation ghosts of Section 5.1.

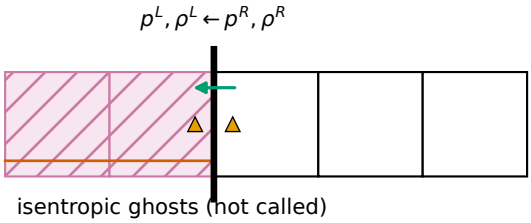


Figure 5.14.: The legacy wall-face copy at a bottom wall (thick black line), x1 to the right: the left state's p and rho at the wall face (orange triangle on the ghost side) are overwritten by the right state's (bluish-green arrow). The vermillion line under the hatched ghost cells marks the isentropic ghost fill, whose call is commented out.

6. Gravity and energy

Gravity does work on the flow, and a finite-volume code has to decide where to book it. This chapter is about that decision and what each answer costs. It covers the cell form, which applies the body force and the work $\rho v_1 g_1$ in the same cell; the face form, which books the work as a flux difference of potential energy across the cell's x_1 faces; the global fixer that repairs the cell form's budget once per step; the corrected-PE face work, which makes the booked work fourth-order consistent with the scheme's own mass fluxes; the gravity work inside the vertical implicit operator; and a closing comparison of what each form conserves and which diagnostic measures it. The thread running through all of them is a single identity, Equation 6.2: choose a discrete potential energy, and the work that conserves total energy plus that potential energy is determined. Every form in this chapter is that identity with a different choice of P, and the differences between the forms are differences in how accurate P is, not in how the work is applied. The momentum equation is untouched throughout; only the energy row changes. The hydrostatic reference state these forms are often used with is Chapter 5, and the implicit solver they couple into is **?@sec-ch07b**.

PENDING: snapy_report.ch06.fig_deps.make_fig

Figure 6.1.: Dependency map of the gravity and energy schemes. Nodes are the schemes of this chapter; a solid arrow reads 'requires', a dashed arrow 'implies', and a vermillion arrow with a bar head 'turns off'. The corrected-PE face work requires the face form, implies the E+P invariant in place of E+PE_d, and turns off the cp3/cp5/weno5 curvature flux; the fixer requires the cell form and is refused with either face form. Read it with the switch coverage matrix, which lists which combinations a test actually exercises.

Table 6.1.: The schemes of this chapter, their switches and defaults, the order of the booked gravity work, and the test that carries each one. Orders are of the booked work against the exact work of the scheme's own x_1 mass fluxes; "global" means the fixer corrects the domain integral, not the local error. Full ctest names and their tolerances are in each scheme's Tests layer.

scheme	switch	default	order	main test
Cell form (**?@sec-ch06-cell**)	`gravity-work: cell`	on	1	`test_forcing`
Face form (**?@sec-ch06-face**)	`gravity-work: face`	off	2	`test_horizontal_flux_covariance`

scheme	switch	default	order	main test
Wall-cell face form (**?@sec-ch06-wallc**)	`gravity-work: face-wallc`	off	1 at walls	`test_gravity_work_fixer`
Gravity-work fixer (**?@sec-ch06-fixer**)	`gravity-work-fixer`	on with `cell`	global	`test_gravity_work_fixer`
Corrected-PE face work (Section 6.1)	`SNAP_GRAVITY_WORK_RADIAL_EXACT`	on with `face`	4	`test_gravity_work_radial_exact`
VIC gravity work (**?@sec-ch06-vic**)	`implicit-scheme` $\neq 0$	off	linearised	`test_implicit_face_work_operator`

6.1. The corrected-PE face gravity work (`SNAP_GRAVITY_WORK_RADIAL_EXACT`)

6.1.1. Summary

> At a glance
>
> **What it does.** Adds one term to the face form of the gravity work so that the booked work is consistent with the scheme's own x_1 mass fluxes to fourth order in every cell, wall cells included. **Switch.** Environment variable `SNAP_GRAVITY_WORK_RADIAL_EXACT`, read once per process. **On** unless set to `0`, `false`, `off` or `no`. **Requires.** `forcing/const-gravity/gravity-work: face`, $g_1 \neq 0$, geometry `cartesian` or `spherical-polar`, the x_1 flux computed, and $n_1 \geq 3$ cells per block. **Turns off.** The cp3/cp5/weno5 curvature flux $\mathcal{K}$. **Order.** $O(h^4)$ for the booked work; the conserved invariant becomes $E + P$, not $E + \mathrm{PE}_d$. **Defining equation.** Equation 6.10.

With `gravity-work: face`, snapy books the energy that gravity exchanges with the flow in the form of a flux difference of potential energy across each cell's x_1 faces (the face form, **?@sec-ch06-face**). This scheme adds one term to that work in every cell, Equation 6.10, where $\dot{\rho}$ is the stage's x_1 density tendency, s_i the centroid slope of Equation 6.8 and σ_i^2 the variance of x_1 over the cell. The added term is the work done by moving the mass of a cell whose density is not uniform inside it.

The face form conserves total energy E plus the discrete potential energy PE_d exactly, but PE_d is itself only a second-order approximation of the true potential energy of the cell averages. The face form therefore books a gravity work with an $O(h^2)$ error in every cell: the trapezoid term $\frac{h^2}{12}\partial_1^2 F$, plus two $O(h^2/r)$ curvature terms on spherical-polar grids. The cp3/cp5/weno5 reconstructions remove part of it with the curvature flux $\mathcal{K}$ (**?@sec-ch06-face**), but that flux is zero at walls, which leaves the

two wall cells of every column first order. This scheme replaces PE_d by a fourth-order potential energy P and books exactly the work that conserves $E + P$. The result is a gravity work that is $O(h^4)$ relative to the exact work of the scheme's own x_1 mass fluxes, in every cell, measured on uniform Cartesian and spherical-polar grids; and $E + P$ conserved to round-off on a closed column, in every RK stage, explicit and implicit. It does not make the energy equation fourth order: the fluxes $F_{i\pm1/2}$ keep the reconstruction's and the Riemann solver's error.

So the default run (`gravity-work: cell`) is unaffected, and every `gravity-work: face` run on a Cartesian or spherical-polar grid uses this scheme unless it is switched off. On a `gnomonic-equiangle` (cubed-sphere) grid with `gravity-work: face` the scheme has no form: setup warns once and the plain face work is kept. With it active the curvature flux $\mathcal{K}$ is turned off, because the scheme already removes the term $\mathcal{K}$ removes and the two together are back to $O(h^2)$; it never meets the gravity-work fixer (**?@sec-ch06-fixer**), which runs only with `gravity-work: cell`; and with an implicit scheme part of it is booked inside the vertical implicit operator (**?@sec-ch06-vic**). Under it the conserved invariant is $E + P$, the cycle diagnostics log P as `pe=`, and every $E + \mathrm{PE}_d$ check measures the wrong quantity.

Aliases. "Leg D" of the gravity-work round, hence "scheme D" in commit messages; "option F" in `docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §7, whose "option D" is a different, rejected two-point weight; "corrected-PE work", and "radial-exact" in identifiers. The implicit part was moved into the VIC operator by snapy@f256ab51fa400641360d16ea3611e78eba85ccc5, and the scheme was made the default with `gravity-work: face` by snapy@84b037f878ac657230b3b1c1d72b23f7714eb880.

6.1.2. Derivation

Every closed form in this layer is checked by `d_face_work_pe_check.py`; the claim tags `[Cn]` point at its output lines. Every `[Cn]` number has the same provenance [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 d_face_work_pe_check.py`, numpy 2.5.3, sympy 1.14.0, double]. Fitted orders use the last three resolutions ($n_1 = 64$, 128, 256) unless stated. Units: g_1 [m s^{-2}], ρ [kg m^{-3}], F [kg m^{-2} s^{-1}], h and x_1 [m], σ^2 [m^2], ϕ [m^2 s^{-2}], W [W m^{-3}]; E, PE$_d$ and P are in J, per unit horizontal area on Cartesian grids and per steradian on spherical-polar. The executable check is nondimensional, with $g_1 = -10$ and scale height $H = 1$.

6.1.2.1. The continuous budget

With constant gravity g_1 along x_1, mass and total energy obey the continuity equation and the total-energy equation in their usual form,

$$\partial_t \rho + \nabla \cdot (\rho \mathbf{v}) = 0,$$
$$\partial_t E + \nabla \cdot \big[(E + p)\mathbf{v}\big] = \rho v_1 g_1.$$

The x_2, x_3 components of gravity book cell work only (**?@sec-ch06-cell**) and play no part here. The potential $\phi(x_1) = -g_1 x_1$ does not depend on time, and $\rho v_1 g_1 = -\rho \mathbf{v} \cdot \nabla \phi$, so $E + \rho\phi$ is conserved:

$$\partial_t (E + \rho\phi) + \nabla \cdot \big[(E + p + \rho\phi)\mathbf{v}\big] = 0.$$

In a closed domain $\int (E + \rho\phi)\, \mathrm{d}V$ is constant.

6.1.2.2. The exact work in one cell

Integrate the gravity source over cell i and write the x_1 mass flux density as $F = \rho v_1$:

$$\int_{V_i} \rho v_1 g_1 \,\mathrm{d}V = g_1 V_i \langle F \rangle_{V_i}.$$

Now multiply the continuity equation by ϕ and integrate over the cell. Use $\phi\,\nabla\cdot(\rho\mathbf{v}) = \nabla\cdot(\phi\rho\mathbf{v}) - \rho\mathbf{v}\cdot\nabla\phi$ and the divergence theorem. Keep only the x_1 faces: on Cartesian and spherical-polar grids the x_2, x_3 faces lie on surfaces of constant ϕ, so their terms are mass fluxes times one value of ϕ, which the horizontal subsection below treats. Then

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{V_i} \rho\phi \,\mathrm{d}V = -\Delta_i[A\phi F] - g_1 V_i \langle F \rangle_{V_i}.$$

Rearranged, the exact gravity work in a cell is

$$\begin{aligned} g_1 V_i \langle F \rangle_{V_i} &= -\frac{\mathrm{d}}{\mathrm{d}t}\mathcal{P}_i - \Delta_i[A\phi F], \\ \mathcal{P}_i &= \int_{V_i} \rho\phi \,\mathrm{d}V. \end{aligned} \tag{6.1}$$

Equation 6.1 is the template for every scheme in this chapter. Pick any discrete potential energy $P = \sum_i P_i$ and *define* the booked work by

$$W_i V_i := -\dot{P}_i - \Delta_i[A\phi F], \tag{6.2}$$

with $\dot{P}_i$ evaluated on the discrete density tendency. Two properties follow at once. **Conservation:** summing Equation 6.2 over a column, the Δ_i terms telescope to the two walls, where $F = 0$, so $\sum_i W_i V_i + \dot{P} = 0$; the

other energy fluxes telescope too, so $E + P$ is conserved exactly, whatever P is. **Accuracy:** subtracting Equation 6.2 from Equation 6.1 gives $W_i V_i - g_1 V_i \langle F \rangle_{V_i} = \frac{\mathrm{d}}{\mathrm{d}t}(\mathcal{P}_i - P_i)$, so the booked work is exactly as accurate as P_i is as an approximation of the cell's potential energy. The whole design problem is to choose a P_i that is accurate and computable from cell averages.

6.1.2.3. The face form is $P = \mathrm{PE}_d$

The discrete continuity equation in x_1 is

$$V_i \dot{\rho}_i = -\Delta_i[AF], \tag{6.3}$$

with $F_{i\pm1/2}$ the total mass flux density the Riemann solver returns: the dry row plus every mass-carrying species row $n = 1..N_y$, after the positivity limiter and sedimentation. Take $P_i = V_i \bar{\rho}_i \phi_i$ with $\phi_i = \phi(x_{1,i})$ at the volume centroid. Then $\dot{P}_i = V_i \dot{\rho}_i \phi_i = -\phi_i \Delta_i[AF]$ and Equation 6.2 gives

$$\begin{aligned} W_i^{\mathrm{face}} &= \frac{1}{V_i}\Big[\phi_i\, \Delta_i[AF] - \Delta_i[A\phi F]\Big] \\ &= \frac{g_1}{V_i}\Big[(x_{1,i+1/2} - x_{1,i}) A_{i+1/2} F_{i+1/2} + (x_{1,i} - x_{1,i-1/2}) A_{i-1/2} F_{i-1/2}\Big], \end{aligned} \tag{6.4}$$

the second form from $\phi_i - \phi_{i\pm1/2} = g_1(x_{1,i\pm1/2} - x_{1,i})$. Equation 6.4 is computed as `face_gravity_work` in `hydro_forward.cpp:794-811` (times Δt). The face form conserves $E + \mathrm{PE}_d$ exactly, by Equation 6.2.

PE_d is the exact $\int \rho\phi \,\mathrm{d}V$ of a piecewise-constant density, so it misses the in-cell density gradient. On a uniform Cartesian grid Equation 6.4 is the trapezoid rule, $W_i^{\mathrm{face}} = g_1(F_{i+1/2} + F_{i-1/2})/2$, with error $g_1 \frac{h^2}{12} \partial_1^2 F$ in every cell. On a spherical-polar grid two more terms appear

(`docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §2, eq. 2):

$$\frac{W_i^{\mathrm{face}} - g_1 \langle F \rangle_{V_i}}{g_1} = \frac{h^2}{12}\partial_1^2 F + \frac{h^2}{6r^{\mathrm{m}}}\partial_1 F - \frac{h^2}{6(r^{\mathrm{m}})^2} F + O(h^4). \tag{6.5}$$

`[C6]` measures the face form at second order in interior and wall cells on both grids.

6.1.2.4. The potential energy of a cell to fourth order

Let $\rho(x_1)$ be smooth inside cell i and expand it about the centroid $x_{1,i}$, with the derivatives taken at $x_{1,i}$: $\rho = \rho(x_{1,i}) + \partial_1\rho\,(x_1 - x_{1,i}) + \frac{1}{2}\partial_1^2\rho\,(x_1 - x_{1,i})^2 + \frac{1}{6}\partial_1^3\rho\,(x_1 - x_{1,i})^3 + \ldots$. With $\phi = -g_1 x_1$,

$$\mathcal{P}_i = -g_1 \int_{V_i} \rho\, x_1 \,\mathrm{d}V = -g_1 \Big[x_{1,i} \int_{V_i} \rho \,\mathrm{d}V + \int_{V_i} \rho\,(x_1 - x_{1,i})\,\mathrm{d}V \Big].$$

The first integral is $V_i \bar{\rho}_i$ exactly, with $\bar{\rho}_i$ the cell average, so the first term is $V_i \bar{\rho}_i \phi_i$. In the second, the centroid makes the first moment vanish, $\int_{V_i} (x_1 - x_{1,i})\,\mathrm{d}V = 0$. Of the remaining moments the second is $V_i \sigma_i^2$ with $\sigma_i^2 = \langle (x_1 - x_{1,i})^2 \rangle_{V_i}$; the third is zero on a Cartesian grid and $O(h^4/r)\,V_i$ on spherical-polar; the fourth is $O(h^4)V_i$. Hence

$$\mathcal{P}_i = V_i \Big[\bar{\rho}_i \phi_i - g_1 \sigma_i^2\, \partial_1 \rho(x_{1,i}) \Big] + O(h^4 V_i). \tag{6.6}$$

On a uniform Cartesian cell the remainder is exactly $-g_1 \frac{h^5}{480}\partial_1^3\rho + O(h^7)$. On a spherical-polar cell the h^0 to h^3 terms of the remainder per unit volume vanish identically `[C3]`. PE_d keeps only the first term, which is why it is $O(h^2)$.

The cell variance σ_i^2 is $h_i^2/12$ on Cartesian grids. On spherical-polar grids, per steradian, with r^{m} the cell midpoint and h the width, it is

$$\begin{aligned} V &= (r^{\mathrm{m}})^2 h + \tfrac{h^3}{12}, \\ \delta_i &= x_{1,i} - r^{\mathrm{m}} = \frac{r^{\mathrm{m}} h^3}{6V}, \\ \sigma^2 &= \frac{(r^{\mathrm{m}})^2 h^3/12 + h^5/80}{V} - \delta_i^2. \end{aligned} \tag{6.7}$$

Equation 6.7 is computed as `x1_variance` in `gravity_work_radial.hpp:13-22`. It is the exact variance on the r^2 measure about the volume centroid (`x1v` on a spherical-polar grid is that centroid). It is written about the midpoint so that nothing cancels at $r/h \gg 1$: the relative error is 1.7×10^{-16} at $r/h = 10^6$ `[C1]`.

$\partial_1 \rho(x_{1,i})$ is not known; only cell averages are. Replace it by the centroid slope

$$s_i[\bar{\rho}] = \frac{\mathrm{d}}{\mathrm{d}x_1}\Big|_{x_{1,i}} \text{ of the quadratic through } (x_{1,l}, \bar{\rho}_l),\ l \in \mathcal{N}_i, \tag{6.8}$$

Equation 6.8 is computed as `centroid_slope` in `gravity_work_radial.hpp:28-51`. The stencil depends on the cell's place in the block: $\mathcal{N}_i = \{i-1, i, i+1\}$ inside the block, and $\mathcal{N}_0 = \{0,1,2\}$, $\mathcal{N}_{n_1-1} = \{n_1-3, n_1-2, n_1-1\}$, one-sided, at the two x_1 ends of every block. With $\Delta_- = x_{1,i} - x_{1,i-1}$ and $\Delta_+ = x_{1,i+1} - x_{1,i}$ the interior weights are

$$s_i[\bar{\rho}] = -\frac{\Delta_+}{\Delta_-(\Delta_- + \Delta_+)}\bar{\rho}_{i-1} + \frac{\Delta_+ - \Delta_-}{\Delta_-\Delta_+}\bar{\rho}_i + \frac{\Delta_-}{\Delta_+(\Delta_- + \Delta_+)}\bar{\rho}_{i+1},$$

and at the first cell, with $a = x_{1,1} - x_{1,0}$ and $b = x_{1,2} - x_{1,1}$,

$$s_0[\bar{\rho}] = -\frac{2a+b}{a(a+b)}\bar{\rho}_0 + \frac{a+b}{ab}\bar{\rho}_1 - \frac{a}{(a+b)b}\bar{\rho}_2,$$

mirrored at the last cell. `[C2]` checks these weights against the derivative of the Lagrange quadratic.

Each stencil annihilates a constant and is exact for a quadratic. A cell average differs from the point value at the centroid by $\frac{1}{2}\partial_1^2\rho\,\sigma^2 + ...$, a smooth $O(h^2)$ amount when h varies smoothly, so $s_i[\bar{\rho}] = \partial_1\rho(x_{1,i}) + O(h^2)$ in interior and end cells alike. `[C4]` measures orders 1.77 to 1.99 for this error, on a stretched Cartesian grid and a spherical column; the spherical fit is still pre-asymptotic at $n_1 = 64$ to 256. Then $\sigma_i^2 s_i = \sigma_i^2\,\partial_1\rho + O(h^4)$, and the corrected discrete potential energy

$$P[\bar{\rho}] = \sum_i V_i\Big[\bar{\rho}_i\phi_i - g_1\sigma_i^2 s_i[\bar{\rho}]\Big] = \mathcal{P} + O(h^4) \tag{6.9}$$

is fourth-order accurate. `[C4]` measures $|P - \mathcal{P}|/|\mathcal{P}|$ at orders 4.08 (stretched Cartesian) and 4.62 (spherical, inner radius $5H$), against 2.00 for PE_d.

P is linear in $\bar{\rho}$ and reads no cell outside the block: no ghost cells. That is the wall closure. From here on this section uses only cell averages, so the bar is dropped: ρ_i is the cell average.

6.1.2.5. The work that conserves $E + P$

Insert $P_i = V_i[\rho_i\phi_i - g_1\sigma_i^2 s_i[\rho]]$ into Equation 6.2. Because s_i is linear, $\dot{P}_i = V_i[\dot{\rho}_i\phi_i - g_1\sigma_i^2 s_i[\dot{\rho}]]$, and comparing with Equation 6.4,

$$\begin{aligned} W_i^{\mathrm{D}} &= W_i^{\mathrm{face}} + g_1\,\sigma_i^2\,s_i[\dot{\rho}], \\ V_i\dot{\rho}_i &= -\Delta_i[AF]. \end{aligned} \tag{6.10}$$

Equation 6.10 is computed by `corrected_pe_work` in `hydro_forward.cpp:817-822`, which adds `corrected_pe_work(-dt * vertical_mass_div, ...)`, i.e. $\Delta t\, g_1 \sigma^2 s[\dot\rho]$, to the face work. This is the whole scheme. It changes only the energy row: mass and momentum are untouched. At rest ($F \equiv 0$, so $\dot\rho = 0$) it adds exactly zero.

6.1.2.6. Conservation, exactly

Sum Equation 6.10 times V_i over a closed column. By construction Equation 6.2, $\sum_i W_i^{\mathrm{D}} V_i + \dot P = -\sum_i \Delta_i[A\phi F] = -(A\phi F)_{\mathrm{top}} + (A\phi F)_{\mathrm{bottom}} = 0$. No property of the slope weights, the grid or the flux enters. The energy fluxes other than gravity telescope as well. So in exact arithmetic $\sum_i V_i E_i + P$ is constant over one evaluation of the right-hand side.

snapy's RK stage is $\mathbf{U} \leftarrow w_0^{(s)}\mathbf{U}^n + w_1^{(s)}\mathbf{U} + w_2^{(s)}\Delta t\, \mathcal{L}(\mathbf{U})$ with $w_0^{(s)} + w_1^{(s)} = 1$. For rk3 the weights $(w_0^{(s)}, w_1^{(s)}, w_2^{(s)})$ are $(0, 1, 1)$, $(\frac{3}{4}, \frac{1}{4}, \frac{1}{4})$ and $(\frac{1}{3}, \frac{2}{3}, \frac{2}{3})$, set in `IntegratorImpl` at `pyharp integrator.cpp:33-61`. Write $\mathcal{E}(\mathbf{U}) = \sum_i V_i E_i + P[\rho]$. $\mathcal{E}$ is linear in $\mathbf{U}$, and $\mathcal{E}(\Delta t\, \mathcal{L}) = 0$ by the identity above. Then $\mathcal{E}(\mathbf{U}^{\mathrm{new}}) = w_0^{(s)}\mathcal{E}(\mathbf{U}^n) + w_1^{(s)}\mathcal{E}(\mathbf{U})$, which equals $\mathcal{E}(\mathbf{U}^n)$ whenever $\mathcal{E}(\mathbf{U}) = \mathcal{E}(\mathbf{U}^n)$. By induction over the stages, $E + P$ is constant over the step to round-off.

`[C7]` steps closed columns 20 times with arbitrary nonlinear fluxes. The max per-step $|\Delta^{\mathrm{stg}}(E+P)|/|E+P|$ is $\le 3.5\times10^{-16}$ on every case: Cartesian stretched; spherical with inner radius over scale height 1, 5 and 1000; two x_1 blocks; a 2-D box. Over the same steps $E + \mathrm{PE}_d$ drifts by 10^{-8} to 10^{-5} per step.

$E + \mathrm{PE}_d$ is not conserved by this scheme. Since $\mathrm{PE}_d = P + g_1 \sum_i V_i \sigma_i^2 s_i[\rho]$, it changes per stage by

$$\Delta^{\mathrm{stg}}(E + \mathrm{PE}_d) = +g_1 \sum_i V_i \sigma_i^2 s_i[\Delta^{\mathrm{stg}}\rho], \tag{6.11}$$

the $O(h^2)$ error of PE_d itself. `[C7]` checks Equation 6.11 on one stage of every case, to $\leq 5.3 \times 10^{-17}$ of the summed magnitudes of its terms. This is the intended behaviour, not a defect: PE_d is the wrong target. At `docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §7 and §8.6 this change carries the same plus sign; check the note's current head before publication.

6.1.2.7. Accuracy

From Equation 6.2, $W_i^{\mathrm{D}} V_i - g_1 V_i \langle F \rangle_{V_i} = \frac{\mathrm{d}}{\mathrm{d}t}(\mathcal{P}_i - P_i)$. By Equation 6.6 and Equation 6.9, $\mathcal{P}_i - P_i = -g_1 V_i \sigma_i^2 (\partial_1 \rho(x_{1,i}) - s_i[\rho]) + O(h^4 V_i)$, which is linear in ρ. Its time derivative is the same expression applied to the density tendency. For a smooth flux field $F(x_1)$ the discrete tendency Equation 6.3 is the exact cell average of $-A^{-1}\partial_1(AF)$, so that expression is $O(h^4 V_i)$ too:

$$W_i^{\mathrm{D}} = g_1 \langle F \rangle_{V_i} + O(h^4) \quad \text{in every cell, wall cells included,} \tag{6.12}$$

where F is the flux field the scheme is handed. To leading order, $\sigma^2 s[\dot{\rho}] = -\frac{h^2}{12}(\partial_1^2 F + 2\partial_1 F/r - 2F/r^2)$, which is minus Equation 6.5: the scheme cancels all three $O(h^2)$ terms of the face form, the trapezoid term and both curvature terms.

`[C6]` uses the profile $F = e^{-z/H}\sin(\pi z/4H)$ on $[0, 4H]$, which vanishes at both walls, on uniform grids. It compares the booked work with the exact cell averages; values are given at $n_1 = 16$ and 256.

Table 6.2.: Max-norm error $\|W - g_1\langle F\rangle_V\|_\infty/|g_1|$ of the booked work against the exact cell average, at $n_1 = 16$ and 256, from check line `[C6]`.

form and grid	interior	order	wall cells	order
face, spherical, inner radius $5H$	4.44e-3 → 2.50e-5	1.95	5.81e-3 → 2.54e-5	1.98
this scheme, spherical, inner radius $5H$	3.47e-5 → 5.35e-10	4.00	5.37e-5 → 1.79e-10	4.49
face, Cartesian	4.99e-3 → 3.11e-5	1.94	7.02e-3 → 3.17e-5	1.98
this scheme, Cartesian	7.44e-5 → 1.16e-9	4.00	1.29e-4 → 1.91e-9	4.02

6.1.2.8. Uniform-grid closed forms

On a uniform Cartesian column, Equation 6.10 can be written in face fluxes. Use $\dot\rho_l = -(F_{l+1/2} - F_{l-1/2})/h$, $\sigma^2 = h^2/12$, the interior slope $s_i = (\dot\rho_{i+1} - \dot\rho_{i-1})/2h$, and the wall slope $s_0 = (-3\dot\rho_0 + 4\dot\rho_1 - \dot\rho_2)/2h$ with $F_{-1/2} = 0$:

$$\begin{aligned} \frac{W_i^{\mathrm{D}}}{g_1} &= \frac{F_{i+1/2} + F_{i-1/2}}{2} - \frac{F_{i+3/2} - F_{i+1/2} - F_{i-1/2} + F_{i-3/2}}{24}, \\ \frac{W_0^{\mathrm{D}}}{g_1} &= \frac{19F_{1/2} - 5F_{3/2} + F_{5/2}}{24}, \end{aligned} \tag{6.13}$$

and $W^{\mathrm{D}}_{n_1-1}$ is the mirror image of W^{D}_0. Cells 1 and n_1-2 use the interior form with the wall flux, which is zero. `[C5]` checks both forms symbolically and their accuracy: the wall form is exact for $F = z, z^2, z^3$ with $F(0) = 0$, and its error for z^4 is $\frac{19}{30}h^4$; the interior form has no error terms below h^4.

Take a face-flux wave $F = \cos(2\pi x_1/\lambda_w)$ and put cell i at $x_1 = 0$, with $\vartheta = \pi h/\lambda_w$. Then the added term over the face term in Equation 6.13 is

$$\frac{1 - \cos 3\vartheta/\cos\vartheta}{12} = \frac{\sin^2\vartheta}{3}.$$

This is $\vartheta^2/3$ for long waves, so $O(h^2)$, but $\frac{1}{4}$ at wavelength $3h$ and up to $\frac{1}{3}$ as the wavelength approaches $2h$ `[C9]`. The added term is a small correction only for resolved flow. At the grid scale it is a fixed fraction of the face work, which matters for the implicit solver.

6.1.2.9. Horizontal directions and several blocks

In 2-D and 3-D, the x_2 and x_3 fluxes move mass between columns at the same x_1 level i and book no gravity work. On spherical-polar grids $V_{ijk} = \varpi_{jk} V_i^{(1)}$; on Cartesian grids the same holds with $\varpi_{jk} = \Delta x_2 \Delta x_3$. And h_i, σ_i^2 and the stencil Equation 6.8 are the same in every column. Because s is linear,

$$\sum_{j,k} P_{jk} = \sum_i V_i^{(1)} \Big[\phi_i \bar{M}_i - g_1 \sigma_i^2 s_i[\bar{M}]\Big],$$
$$\bar{M}_i = \sum_{j,k} \varpi_{jk} \rho_{ijk}.$$

$\bar{M}_i$ is the level's total mass, which horizontal fluxes conserve (periodic or closed lateral boundaries). So $E + P$ stays exact. `[C7]` checks this on a 2-D box with periodic x_2 fluxes: 3.4×10^{-16} per step.

For blocks split in x_1 (explicit runs only; an implicit scheme requires one x_1 block), each block uses its own one-sided stencil at its x_1 ends, so $P = \sum_b P_b$, a sum of per-block functionals. At a shared face both blocks see the same F and ϕ, so the $A\phi F$ terms cancel between them, and global $E + P$ is exact (`[C7]`, two blocks: 2.2×10^{-16}). The split column does not conserve the one-block P, though: the seam cells take a one-sided slope where one block would take the centred one, so the two runs differ. `[C10]` measures the functional difference $|P_{\text{split}} - P_{\text{one}}|/|P|$ falling from 2.6×10^{-8} at $n_1 = 32$ to 5.3×10^{-13} at 256, order 5.2 (fit over all four resolutions). The state gap the code prints is in the Tests layer.

6.1.2.10. How the scheme enters the vertical implicit solve

With `implicit-scheme` $\neq 0$, the vertical implicit correction (VIC, **?@sec-ch07b**) solves each column for a new stage increment δ_i. The explicit increment $\Delta\mathbf{U}^{(0)}$ is its right-hand side, and under rk3 the solve is handed $\Delta t_c = w_2^{(s)} \Delta t$, set as `dt_corr` in `hydro_forward.cpp:929-933`. Row i of the block-tridiagonal system reads

$$\begin{aligned} \mathsf{A}_i \delta_i + \mathsf{B}_i \delta_{i-1} + \mathsf{C}_i \delta_{i+1} &= \Delta\mathbf{U}_i^{(0)}/\Delta t_c, \\ \mathsf{A}_i &= \mathsf{I}/\Delta t_c + \text{Jacobian terms.} \end{aligned} \tag{6.14}$$

Equation 6.14 is assembled from the right-hand side `rhs` in `forward_sweep_impl.h:35-49`, with the block convention of `delta` in `forward_sweep_impl.h:89-126`. Its unknowns are, in order, the total mass, the normal momentum, the two tangential momenta (full VIC only) and the energy. The solved energy is written back from `delta` as the stage's energy increment in `vic_redistribute_impl.h:174-179`. With `gravity-work: face` the energy row already contains the face form for the mass the solve

moves (UCzhangxi/snapy#283; derived in **?@sec-ch06-vic**), assembled under the `face_work` flag in `vic_assemble_full_impl.h:101-120` for full VIC and `vic_assemble_partial_impl.h:122-139` for partial VIC.

Booking the scheme only after the solve would treat $g_1\sigma^2 s[\Delta\rho_{\mathrm{solve}}]$ explicitly, and by Equation 6.13 that term is a quarter to a third of the face work for a grid-scale density change, not an $O(h^2)$ correction. The face work it accompanies is implicit, and stable at vertical acoustic Courant numbers of hundreds. A discretely balanced $11.3H$ rest column (implicit-scheme 9) with the term booked only after the solve grew from round-off to $\max|v_1| = 1.3\times10^{-7}$ m s^{-1}: at step 13 at Courant 197 and step 9 at Courant 250, in both geometries, against $\le 4.9\times10^{-9}$ m s^{-1} with the switch off. Dropping only the post-solve term restored the switch-off level; dropping only the explicit term did not. [sha: the parent of snapy@f256ab51fa400641360d16ea3611e78eba85ccc5, run sha not recorded; deck: `tests/test_implicit_gravity_tall_column.py`; run: CPU; source: `docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §10. A re-run needs `f256ab51fa400641360d16ea3611e78eba85ccc5^` or a mutation of the pinned code that drops the operator coupling.]

The matrix is built by `couple_radial_exact` in `implicit_hydro.cpp:301-333`. Let S be the slope matrix, $(\mathsf{S}q)_i = s_i[q] = \sum_l \mathsf{S}_{il}q_l$; the code builds its transpose as `centroid_slope(eye(n1), x1v)`. Inside a block row i of S has three entries on the tridiagonal. At the two end cells the one-sided slope reads a third cell, which a block-tridiagonal system cannot hold. The matrix therefore holds a lumped $\tilde{\mathsf{S}}$:

$$\begin{aligned}\tilde{\mathsf{S}}_{il} &= \mathsf{S}_{il}\ (|i-l|\le 1),\\ \tilde{\mathsf{S}}_{01} &= \mathsf{S}_{01}+\mathsf{S}_{02},\\ \tilde{\mathsf{S}}_{n_1-1,n_1-2} &= \mathsf{S}_{n_1-1,n_1-2}+\mathsf{S}_{n_1-1,n_1-3}.\end{aligned}\tag{6.15}$$

The third weight is moved onto the neighbour, in the `rx_lo`, `rx_hi` lumping block of `implicit_hydro.cpp:305-318`. Each cell's weights still sum

to zero, so $\tilde{s}$ still annihilates a constant [C8]. At the end cells $\tilde{s}$ is not a consistent slope, though: $\tilde{s}_0 = \frac{2a+b}{a(a+b)}(\rho_1 - \rho_0)$, which is $\frac{3}{2h}(\rho_1 - \rho_0)$ on a uniform grid [C8]. The post-solve remainder below is therefore not small in the two end cells. That part of the scheme stays explicit at the grid scale; the tall-column test is the evidence that this is stable.

The energy row gains a coupling to the total-mass unknown, and its right side gains the matching term in `rx_mass0`, at `implicit_hydro.cpp:326-332`:

$$\begin{aligned}
\mathsf{A}_i^{(E,\rho)} &\leftarrow \mathsf{A}_i^{(E,\rho)} - \frac{g_1 \sigma_i^2 \widetilde{\mathsf{S}}_{ii}}{\Delta t_c}, \\
\mathsf{B}_i^{(E,\rho)} &\leftarrow \mathsf{B}_i^{(E,\rho)} - \frac{g_1 \sigma_i^2 \widetilde{\mathsf{S}}_{i,i-1}}{\Delta t_c}, \\
\mathsf{C}_i^{(E,\rho)} &\leftarrow \mathsf{C}_i^{(E,\rho)} - \frac{g_1 \sigma_i^2 \widetilde{\mathsf{S}}_{i,i+1}}{\Delta t_c}, \\
\frac{\Delta E_i^{(0)}}{\Delta t_c} &\leftarrow \frac{\Delta E_i^{(0)}}{\Delta t_c} - \frac{g_1 \sigma_i^2 \tilde{s}_i [\Delta \rho^{(0)}]}{\Delta t_c},
\end{aligned} \tag{6.16}$$

with $\Delta\rho^{(0)}$ the total-mass part of $\Delta\mathbf{U}^{(0)}$. Only fluid cells get these terms (`rx_fluid`). Reading the energy row of Equation 6.14 with Equation 6.16:

$$\delta E_i = \Delta E_i^{(0)} + g_1 \sigma_i^2 \, \tilde{s}_i \big[\delta\rho - \Delta\rho^{(0)}\big] - \Delta t_c \, (\text{Jacobian terms} \cdot \delta)_{E,i}. \tag{6.17}$$

The solve books the work of the mass it moves, $\delta\rho - \Delta\rho^{(0)}$, implicitly. [C11] assembles a toy column with exactly these entry updates and right-hand side, solves it in the convention of Equation 6.14, and recovers Equation 6.17 to 9×10^{-15}. Flipping the coupling's sign or swapping its lower and upper weights leaves an $O(1)$ residual.

After the solve the raw solution is redistributed among the constituents and clamped where a constituent would go negative (**?@sec-ch07b**). Let $\Delta\rho_i$ be the solved density change, `du` summed over the mass rows after redistribution. The code's `moved` is `du - du0`, i.e. $\Delta\rho_i - \Delta\rho^{(0)}$, and it can differ from $\delta\rho - \Delta\rho^{(0)}$. After the redistribution the code adds, in fluid cells, the remainder

$$g_1\sigma_i^2\Big(s_i[\Delta\rho_i - \Delta\rho^{(0)}] - \widetilde{s}_i[\delta\rho - \Delta\rho^{(0)}]\Big), \tag{6.18}$$

computed from `moved` in `implicit_hydro.cpp:451-470`, so the implicit part books exactly $g_1\sigma^2 s[\Delta\rho_i - \Delta\rho^{(0)}]$ in fluid cells, the work of the mass actually moved. With the explicit part, the stage total is Equation 6.10 for the full x_1 density change, and $E + P$ closes. Lumping only changes how much of the work the solve sees implicitly, never the total.

`[C8]` makes two checks on this. First, a mock solve whose redistribution changes the face masses (a binding clamp at one face, a shifted mass at another): its $E + P$ defect is round-off (2.4×10^{-16} and 5.5×10^{-14} against $\sum|\Delta E\, V|$ of 0.35 and 18). Second, the same solve booked on the raw change instead of the solved one leaves defects of 4.4×10^{-7} and 2.6×10^{-4}. The sum of the matrix part and Equation 6.18 is an identity by construction and is only printed.

In an immersed solid cell the solid rows of the matrix are set by `setIdentity` over Δt_c in `implicit_dispatch.cpp:56-62` and `implicit_dispatch.cpp:112-118`. The coupling and its right-side term are masked there (`rx_fluid`), and the post-solve remainder is gated to zero in the same remainder block, because the slope stencil of a solid cell would read fluid neighbours. A fluid cell next to a solid keeps its centred stencil, which then reads the solid's density change, which is zero.

What is exact and what is truncated. Exact to round-off, on any grid, per stage and per step, explicit and implicit: the conservation of $E + P$ on closed columns and on several x_1 blocks, and the identity

Equation 6.11. Truncated: P approximates the true potential energy to $O(h^4)$ (Equation 6.9); the booked work approximates the exact work of the scheme's own mass fluxes to $O(h^4)$ in every cell (Equation 6.12), for smooth fields on smoothly varying grids; and a split column differs from an unsplit one at $O(h^4)$. This derivation is independently corroborated for the Cartesian case by a different construction in `sources/gw__NEXTPR_indep_cartesian_gravity_work_weight.md` §§2.3-2.6; its §4.4 disputes several claims of the derivation note and must be dispositioned before this section is approved.

The identities above are checked by `d_face_work_pe_check.py`, 11 claims, all passing.

6.1.3. Numerical method

Figure 6.2 shows what the scheme reads: an interior cell, a wall cell, an x_1 block seam and the implicit matrix row. Figure 6.3 shows the order of the work it books.

Nondimensional, with $g_1 = -10$ and $H = 1$; the data of Figure 6.3 are the committed output of the executable check at the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, the same check line `[C6]` as Table 6.2.

In the code the interior cells of a block in x_1 are `is` $\leq$ index $<$ `ie`, with `is = il()` and `ie = iu() + 1`. Math cell $i = 0..n_1 - 1$ is code index `is + i`; face $i - \frac{1}{2}$ is code face index `is + i`.

PENDING: snapy_report.ch06.fig_dwork_stencil.make_fig

Figure 6.2.: The stencil of the corrected-PE face work, on four panels. (a) An interior cell: the face form reads the two face fluxes (orange triangles), and the added term reads the densities of cells i-1, i, i+1 (bluish-green circles), hence four faces. (b) The bottom wall cell: the slope is one-sided on cells 0, 1, 2, the wall face carries no flux, and no ghost cell (hatched, light reddish purple) is read. (c) An x1 seam: each block takes one-sided slopes at its own ends, so P splits into a sum over blocks and the seam face's potential flux cancels between them. (d) The VIC energy row's coupling to the total-mass unknown: row i holds the weights of the cells in the slope of cell i, the one-sided end weight (hatched) is lumped onto the neighbour (arrow), and the post-solve term books the difference.

PENDING: snapy_report.ch06.fig_dwork_order.make_fig

Figure 6.3.: Order of accuracy of the booked work against the exact cell average, interior and wall cells, on log axes with a labelled reference slope, on two panels. (a) Spherical-polar, inner radius 5H. (b) Cartesian. Blue is the reference, vermillion the face form's error, bluish green this scheme's.

Table 6.3.: Where each quantity of the scheme lives.

quantity	lives at	from
$F_{i\pm1/2}$	x_1 faces, owned faces of the block	Riemann solver, then the positivity limiter and sedimentation; summed over the dry row and the species rows `ICY`..`ICY+`N_y`-1`
$A_{i\pm1/2}$, V_i	faces, cells	`face_area1()`, `cell_volume()`
$\phi_{i\pm1/2}$, ϕ_i	faces, cell centroids	$-g_1$`x1f`, $-g_1$`x1v`
$\dot\rho_i\,\Delta t$	interior cells	`-dt *` `vertical_mass_div`
σ_i^2	interior cells	`x1_variance` on the block's interior faces
$s_i[\cdot]$	interior cells	`centroid_slope` on the block's interior centroids
$W_i^{\mathrm{D}}\,\Delta t$	interior cells, energy row	added to `du[IPR]`

Walls. The only data the scheme reads are interior densities, in the form of tendencies, and the grid geometry. At a closed wall the slope is one-sided, conservation uses only $F = 0$ on the wall face, and the wall cells are $O(h^4)$ (`[C6]`: orders 4.49 spherical and 4.02 Cartesian). That is what makes it a wall closure for the gravity work. The cp3/cp5/weno5 curvature flux $\mathcal{K}$ needs a value at the wall face and is set to zero there, which leaves its wall cells first order: Cartesian first cell $1.65 \times 10^{-2} \to 2.05 \times 10^{-3}$, order 1.00, n_1 16 $\to$ 128 (`docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §8.3, §8.7) [deck: `docs/derivations/optionF_replica.py@e894700ff7aee30b52882e5202b16461413780b0`, printed label §6; base sha 8cea3ae5a8297c822

05c1cb9da03681f2343c0dc, which is not a pin of §3.1; run not recorded, so this is re-run work before approval]. The scheme does not use $\mathcal{K}$: the `curv_flux1` branch in `hydro_forward.cpp:828-849` turns it off when the scheme is active.

On open (non-reflecting) x_1 boundaries the same one-sided stencil is used, but there the boundary face's $A\phi F$ no longer vanishes. $E+P$ then changes by exactly the potential-energy flux through the boundary, which is the physical budget. Not separately tested; see the Limits layer.

Block seams in x_1. The scheme is computed per block on the block's interior cells, so every block uses one-sided slopes at both its x_1 ends, physical or not. Seams occur only in explicit runs: an implicit scheme refuses a block whose x_1 faces are not both physical boundaries, in the `TORCH_CHECK` at `hydro.cpp:413-416`. Using the centred slope at a seam would need the neighbour's density tendency in the ghost cells; the reconstruction computes fluxes only on owned faces, so that tendency is not available without a new exchange, and the logged `pe=` would also read ghost densities that need not be current when the diagnostics run. Both sites that run on a split column, the explicit work and the logged P, call the same `corrected_pe_work` with the same per-block stencil, so the booked work and the logged P agree on any decomposition and global $E + P$ is exact. The cost is an $O(h^4)$ difference between a split and an unsplit column.

When it runs in a stage. Within `HydroImpl::forward` the scheme runs in step (6), external forcing, after the x_1, x_2, x_3 fluxes and the tracer positivity limiter, after the flux divergence, and after every forcing module, including the const-gravity cell work $\rho v_1 g_1 \alpha_{\mathrm{nh}}$ in `ConstGravityImpl::forward` at `const_gravity.cpp:46-52`. The energy correction booked is $W_i^{\mathrm{D}}\Delta t - W_i^{\mathrm{cell}}\Delta t$: the cell work is removed and the face form plus this scheme is put in its place, as `gravity_energy_correction` in `hydro_forward.cpp:890-897`. With $\alpha_{\mathrm{nh}} < 1$ the removed cell work includes the hydrostatic-split part of `original_gravity_work`, computed at `hydro_forward.cpp:850-857`; that part is not added by a forcing module, so the `rho_grav` block at `hydro_forward.cpp:902-906` adds it right after

the correction is computed. In an explicit run, or `face` with `implicit-scheme` 0, the correction is added to `du[IPR]` after the implicit step if any, at `hydro_forward.cpp:979-983`. With `face` and `implicit-scheme` $\neq 0$, selected by `face_work_in_operator()`, it is added to `du` *before* the solve, at `hydro_forward.cpp:962-967`; the solve then sees, and inverts, the face-form energy including the explicit term, and adds its own implicit term.

Table 6.4.: Order of accuracy of the booked work, by region and grid.

order of $W^{\mathrm{D}} - g_1\langle F\rangle$	where	evidence
4	interior, Cartesian and spherical-polar, uniform	`[C6]` 4.00, 4.00
4	wall cells, uniform	`[C6]` 4.49 spherical, 4.02 Cartesian
4, not asserted	x_1 block seam, state, split vs one block	printed by `test_x1_seam_split` (`radial_exact_split_gap`): orders 3.7, 4.8 at n_1 32 $\to$ 64 $\to$ 128, re-run work; the functional gap `[C10]` 5.2
4 for P	non-uniform x_1 grid, smooth stretching	`[C4]` P: 4.08 on a stretched grid; the work on non-uniform grids is not measured
not established	non-smooth x_1 grid, jumps in h	see the Limits layer

6.1.4. Code

Every link below is at the pin snapy@e894700ff7aee30b52882e5202b16461413780b0, except the pyharp row, which is at pyharp@4721715855e937c1e8b218e964c0655f46e56e29. The symbol column is the citation's anchor: it is the identifier the cited range contains.

Table 6.5.: Code map of the corrected-PE face gravity work.

what	code link	symbol	switch / condition
read the switch once per process	`hydro.cpp: 241-251`	`HydroImpl:: gravity_work_ radial_ exact()`	`SNAP_GRAVITY_ WORK_RADIAL_ EXACT`, default on
where it acts	`hydro.cpp: 253-259`	`HydroImpl:: radial_exact_ work()`	`face`, $g_1 \neq 0$, cartesian or spherical-polar
declaration and invariant note	`hydro.hpp: 172-180`	`gravity_work_ radial_exact`	
no-form warning, cubed sphere	`hydro.cpp:82-92`	`TORCH_WARN_ ONCE`	`face` on another grid
implicit scheme needs one x_1 block	`hydro.cpp: 413-416`	`TORCH_CHECK`	`implicit-scheme` $\neq 0$
`gravity-work` key, fixer default	`const_ gravity.cpp: 28-41`	`gravity_work`	`cell` (default), `face-wallc`, `face`
fixer forced off for face forms	`hydro.cpp:62-64`	`gravity_work_ fixer`	

what	code link	symbol	switch / condition
const-gravity cell work, removed again for face	`const_gravity.cpp: 46-52`	`forward`	
σ^2, Equation 6.7	`gravity_work_radial.hpp: 13-22`	`x1_variance`	
s, Equation 6.8	`gravity_work_radial.hpp: 28-51`	`centroid_slope`	zero for $n_1 < 3$
$g_1\sigma^2 s[\cdot]$	`gravity_work_radial.hpp: 57-63`	`corrected_pe_work`	
the gravity-work block runs	`hydro_forward.cpp: 773-774`	`disable_flux_x1`	$g_1 \neq 0$, x_1 flux on
total x_1 mass flux	`hydro_forward.cpp: 777-780`	`vertical_mass_flux1`	
face form, Equation 6.4	`hydro_forward.cpp: 794-811`	`face_gravity_work`	
add the scheme, explicit, Equation 6.10	`hydro_forward.cpp: 817-822`	`corrected_pe_work`	`radial_exact_work()` and not `cell`
curvature flux off under the scheme	`hydro_forward.cpp: 828-849`	`curv_flux1`	`!radial_exact`
face minus cell work	`hydro_forward.cpp: 890-897`	`gravity_energy_correction`	

what	code link	symbol	switch / condition
hydrostatic-split part	`hydro_forward.cpp:902-906`	`rho_grav`	$\alpha_{\mathrm{nh}} < 1$
into the operator	`hydro_forward.cpp:962-967`	`face_work_in_operator`	`face` and implicit
after the operator	`hydro_forward.cpp:979-983`	`du`	explicit path
face work in the operator	`hydro.cpp:305-309`	`HydroImpl::face_work_in_operator()`	`face` and `implicit-scheme`$\neq 0$
$\Delta t_c = w_2^{(s)} \Delta t$	`hydro_forward.cpp:929-933`	`dt_corr`	rk3
rk3 weights	`pyharp integrator.cpp:33-61`	`IntegratorImpl`	
VIC: keep the explicit increment	`implicit_hydro.cpp:195`	`_du0`	
VIC: matrix part, Equation 6.15 and Equation 6.16	`implicit_hydro.cpp:301-333`	`couple_radial_exact`	`radial_exact_work()`, $n_1 \geq 3$
VIC: right-hand side	`forward_sweep_impl.h:35-49`	`rhs`	

what	code link	symbol	switch / condition
VIC: block convention	`forward_sweep_impl.h:89-126`	`rhs`	
VIC: solved increment written back	`vic_redistribute_impl.h:174-179`	`delta`	
VIC: face-form rows, full	`vic_assemble_full_impl.h:101-120`	`face_work`	
VIC: face-form rows, partial	`vic_assemble_partial_impl.h:122-139`	`face_work`	
VIC: remainder Equation 6.18, solid gate	`implicit_hydro.cpp:451-470`	`moved`	
VIC: solid rows	`implicit_dispatch.cpp:56-62`	`setIdentity`	
logged P (`pe=`)	`meshblock.cpp:1045-1057`	`radial_exact_work`	`radial_exact_work()`

Walk-through in step order.

1. In the `HydroImpl` constructor, `gravity-work` is validated, `gravity_work_fixer` is forced off for face forms at `hydro.cpp:62-64`, a grid with no form for the scheme warns once in the `TORCH_WARN_ONCE` at `hydro.cpp:82-92`, and an implicit scheme with a split x_1 is refused by the `TORCH_CHECK` at `hydro.cpp:413-416`.

2. Each stage, `HydroImpl::forward` computes the total x_1 mass flux after the positivity limiter and sedimentation, as `vertical_mass_flux1` at `hydro_forward.cpp:777-780`.
3. It forms $\Delta t\, W^{\text{face}}$ as `face_gravity_work` at `hydro_forward.cpp:794-811`.
4. If `radial_exact_work()`, it adds `corrected_pe_work(-dt * vertical_mass_div, ...)` at `hydro_forward.cpp:817-822` and skips the `curv_flux1` term at `hydro_forward.cpp:828-849`.
5. It replaces the cell work by the face form, as `gravity_energy_correction` at `hydro_forward.cpp:890-897`.
6. It adds `gravity_energy_correction` to the energy before the implicit solve at `hydro_forward.cpp:962-967` or, from the same `gravity_energy_correction`, after it at `hydro_forward.cpp:979-983`.
7. In the VIC, `couple_radial_exact` adds the matrix coupling and right side after the assembly at `implicit_hydro.cpp:301-333`, and the remainder, the `corrected_pe_work` of the `moved` mass, after the redistribution at `implicit_hydro.cpp:451-470`.
8. The cycle diagnostics log P under `radial_exact_work` at `meshblock.cpp:1045-1057`, so the logged `ie=` + `pe=` is the conserved $E + P$.

6.1.5. Tests

Tolerances are relative to $|E + P|$ unless stated. Python tests run each switch arm in a child process, because the switch is read once per process. C++ ctest names end in the lower-case build type (`release` in CI).

Table 6.6.: Tests of the corrected-PE face gravity work and the tolerance each uses.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_gravity_work_radial_exact.py` (`test_gravity_work_radial_exact_python`; CUDA `..._cuda_python`), check 1	switch unset and `1` give bit-identical states; `0` differs	exact	the default-on contract
same, check 2	per-step $\|\Delta^{\mathrm{stg}}(E+P)\|_\infty / \|E+P\| \leq 10^{-14}$ over 20 steps, on spherical-polar and Cartesian columns and a 2-D box, each explicit and VIC	`EP_TOL = 1e-14`	about 50 ulp of double on sums of up to 512 cells of one sign, with margin for the VIC solve

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, check 3	one explicit plm stage: $E_{\text{on}} - E_{\text{off}} = g_1 \sigma^2 s[\Delta^{\text{stg}} \rho]$ cell by cell; density unchanged by the switch	$\leq 10^{-13} \max \lvert E \rvert$, term $\geq 10^3\times$ the error	round-off of the energy; the second bound rules out a vacuous pass
same, check 4	a discretely balanced Cartesian rest column keeps $\max \lvert v_1 \rvert / c_s$ no worse than without the scheme, 50 steps	on $\leq 1.05\times$ off $+10^{-14}$	the scheme adds zero at rest; 5 % allows round-off growth
same, check 5	the logged `ie=+pe=` equals $E + P$ with the scheme on and $E + \text{PE}_d$ with it off	`DIAG_TOL = 1e-11`	the printed digits of the log
same, check 6	a gnomonic-equiangle block with `face` sets up and warns with the switch on, not with it off	exact	the no-form contract

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, check 7	one VIC solve with a binding availability clamp: the $E + P$ defect is round-off, and the defect if the raw change had been used is far above it	defect $\leq 10^{-13} \sum \lvert\Delta E\rvert$; mutation $\geq 10^{3}\times$	round-off; the mutation shows the check can fail
same, check 8	VIC column with a 4-cell immersed solid: fluid $E + P$ per step $\leq 10^{-14}$, and the energy correction in solid cells is exactly 0 while the ungated work there is not	`EP_TOL`; exact	the solid is refilled every step; the second part tests the gate

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_x1_seam_split.cpp` (`test_x1_seam_split_radial_exact`, control `..._radial_exact_off`)	spherical column, one block vs two split in x_1, $n_1 =$ 32, 64, 128; prints the state gap (not asserted); asserts the split column's $E + P$ drift	`1e-13`	round-off over up to 80 steps
`tests/test_x1_seam_split_mp.cpp` (`test_x1_seam_split_mp_radial_exact`)	the same split on 2 ranks equals the one-process 2-block split, and the split $E + P$ drift holds	`1e-13`	round-off
`tests/test_implicit_gravity_tall_column.py` (`test_implicit_gravity_tall_column_python`)	$11.3H$ balanced rest column, vertical acoustic Courant 6.6 to 250, both geometries: $\max \lvert v_1 \rvert < 10^{-7}$ m s^{-1}, settled $< 10^{-10}$, and on $\leq 1.1\times$ its switch-off rung	`W_TOL`, `W_SETTLED`, `ON_OFF`	the stability gate; 1.1 allows round-off differences between arms

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_implicit_face_work_operator.py`, `tests/test_implicit_stratified_solid.py`	face-work energy oracles measure $E + P$ with the switch on, $E + \mathrm{PE}_d$ with it off	per test	they read the switch as `hydro.cpp` does
`test_forcing`, `test_gravity_work_fixer_python`, `test_horizontal_flux_covariance_python`, `test_flux_covariance_rows_python`	plain face-form $E + \mathrm{PE}_d$ oracles	per test	the switch set to **0** by `set_tests_properties` in `CMakeLists.txt:269-273`: they test the face form, not this scheme

Evidence. Entries marked re-run were measured on the development commits of the round; their run sha and deck are recorded only in the derivation note, so they must be re-measured at the pin before approval.

- This section's check: every `[Cn]` number, with the provenance given at the head of the Derivation layer.
- $E+P$ in the code: per-step $\Delta^{\mathrm{stg}}(E+P) \leq 3.9\times10^{-16}$ of $E+P$ on the implicit cases of `test_gravity_work_radial_exact.py` and on its

column under partial VIC [`docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §10; deck: that test; run: CPU build, sha not recorded → **re-run**].

- Seam gap, isothermal spherical column with a seam density bump, max relative 1-vs-2-block state gap: 8.0×10^{-7}, 6.3×10^{-8}, 2.3×10^{-9} at $n_1 = 32$, 64, 128 (orders 3.7, 4.8); with the switch off the gap is 0 bit for bit, so the whole gap is the one-sided seam slope; split-column $E + P$ drift $\leq 5 \times 10^{-15}$ [`docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §7 item 3; deck: `tests/test_x1_seam_split.cpp` `radial_exact_split_gap`; run: ctest `test_x1_seam_split_radial_exact`, sha not recorded → **re-run**].
- Tall column with the term in the operator: every rung passes in both geometries, $\max|v_1|$ 4.69 to 4.89×10^{-9} m s^{-1}, at or below the switch-off rung, settled to $\leq 7.4\times10^{-13}$ m s^{-1} by step 40 [same note §10; deck: `tests/test_implicit_gravity_tall_column.py`; run: CPU, sha not recorded → **re-run**].
- $E + \mathrm{PE}_d$ is the wrong oracle here: on the spherical implicit energy check of `test_implicit_stratified_solid`, $E + \mathrm{PE}_d$ changes by -0.166 while $E + P$ changes by 3.5×10^{-14} (scale 2119) [same note §10; deck: that test; run: sha not recorded → **re-run**].
- **Missing evidence.** The onset-rate numbers of §11.2 of the derivation note and the coarse polytrope kinetic-energy numbers carry no deck or run. The onset numbers are $\varepsilon_{\mathrm{eff}} n_1^2$ at $n_1 = 16$, 32, 64: $+0.0306, +0.0158, +0.0080$ without the scheme and $-0.00092, -0.00020, -0.00004$ with it. They are re-run from a committed deck or removed before approval. Note that §11.3 and §11.4 of the note were edited after the pin; check the note's current head before publication.

Coverage. CPU: all of the above. CUDA: `test_gravity_work_radial_exact_cuda_python` and the `_cuda` arms of `test_x1_seam_split`, registered only with `-DCUDA=ON` and skipped without a device. Several ranks: `test_x1_seam_split_mp_radial_exact`, 2 ranks under torchrun.

No test runs this scheme with `SNAP_FLUX_COVARIANCE`, `SNAP_WB_REF4` or `SNAP_X1_CENTROID_EXACT` on; see the switch coverage matrix, `@sec-ch12-matrix`.

6.1.6. Limits

> ⚠ Limits
>
> The limits a user can hit by configuration: there is no form on a cubed-sphere grid, `face-wallc` does not get this scheme, a block with fewer than three x_1 cells books nothing beyond the face form, and under this scheme every $E + \mathrm{PE}_d$ check — including `gravity_work_defect()` — measures the wrong invariant.

1. **Seams change the answer at $O(h^4)$**, explicit runs only. A column split in x_1 conserves its own P, not the one-block P. The split state differs from the unsplit one at about $O(h^4)$; the gap is printed, not asserted. A centred seam slope would need a new ghost exchange of the density tendency. Documented, by design.
2. **Cubed sphere.** No form on `gnomonic-equiangle` grids: the plain face work is kept, with first-order wall cells, and setup warns. There the volume uses the r^2 measure, but `x1v` is the midpoint, not the volume centroid, in `gnomonic_equiangle.cpp:36-37`. So σ^2, which assumes the centroid, the slope point and ϕ_i would all need rework. Not attempted.
3. `face-wallc` does not get this scheme: it keeps the cell work in the wall cells by design.
4. **Short blocks.** For $n_1 < 3$ cells in a block the slope is zero and nothing is booked beyond the face form.
5. **Non-smooth x_1 grids.** The $O(h^2)$ accuracy of s assumes that the cell-average offset $\frac{1}{2}\partial_1^2 \rho\, \sigma^2$ varies smoothly, i.e. that h varies smoothly. Across a jump in h the slope error is $O(h)$ and P drops

to $O(h^3)$ locally. Conservation is unaffected. Not measured; nor is the work on smoothly stretched grids.

6. **Order of the energy equation.** The scheme makes the gravity work fourth-order consistent with the mass fluxes. The fluxes themselves keep the order of the reconstruction and the Riemann solver (Chapter 4).
7. **The end cells of the implicit solve.** The lumped slope $\tilde{s}$ is not a consistent slope in the two end cells, so part of the scheme stays explicit there at the grid scale. The tall-column test passes; no stability bound is derived.
8. **Open x_1 boundaries.** $E + P$ changes by the boundary potential-energy flux, as it should. No test covers the scheme with an outflow x_1 boundary.
9. **Diagnostics and oracles.** Under this scheme `gravity_work_defect()` and any $E + \mathrm{PE}_d$ check measure the wrong invariant. Tests that check the plain face form set the switch to 0 explicitly.
10. **Explicit solids.** The explicit path books the scheme in every interior cell, solid or not; immersed solid cells are refilled every step. The VIC gates them. The explicit path with solids is not separately tested.
11. **Remaining error of the onset rate.** With the wall closure and this scheme, the linear onset rate of convection keeps a second-order truncation error that the face and cell forms share. The convergence table in `docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §11.3, a placeholder at the previous pin, is filled at this one, but from runs at shas that are no pin of §3.1, so the numbers are **re-run** work before approval; check the note's current head before publication.
12. **Untested switch combinations.** This scheme with `SNAP_FLUX_COVARIANCE`, `SNAP_WB_REF4`, `SNAP_X1_CENTROID_EXACT` or `SNAP_X1_MASS_COVARIANCE`, and on moist columns with several mass rows, is covered by no ctest entry. The onset numbers above used `SNAP_FLUX_COVARIANCE` and `SNAP_WB_REF4`; the note names their deck, but no ctest covers the combination. See the switch

coverage matrix, `@sec-ch12-matrix`.

13. **Float32.** σ^2 is written without cancellation (Equation 6.7), but the $E + P$ identity has been checked only in double precision.
14. **Open disagreement in the sources.** `sources/gw__NEXTPR_indep_cartesian_gravity_work_weight.md` §4.4 lists claims it holds to be wrong or under-qualified in the derivation note this section rests on, in particular §4.4(a) on the Cartesian "conflict" sentence. Each item is dispositioned before this section is approved.

7. Positivity, floors and limiters

This chapter is every mechanism that keeps the state physically admissible, or detects that it is not.

The most important thing to say comes first, because the rest of the chapter reads differently once it is said: **snapy guarantees no thermodynamic positivity of density or pressure.** There is exactly one guarantee in this chapter, and it covers tracer channels under explicit transport: the limiter of Section 7.1, which cannot drain a cell below zero in a stage. Everything protecting density, pressure and temperature is a floor, which is to say a clamp that invents whatever the state was missing, and the only backstop is a detector that asks for the step to be taken again. The chapter's own check measures what the floors invent rather than describing them as safeguards.

The second thing worth knowing early is that one switch controls most of it. `equation-of-state/limiter`, default false, turns on the tracer limiter, the equation-of-state floors, the reconstruction face floors and the species clamps together. They cannot be selected independently. That coupling is a documented design decision, not an accident of the implementation.

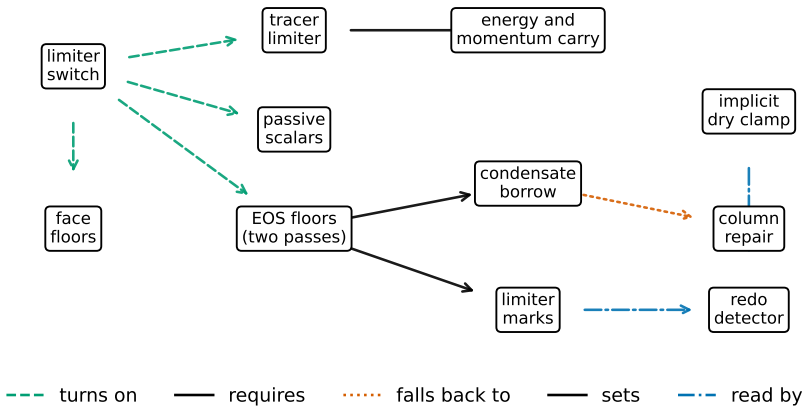


Figure 7.1.: The dependency map of this chapter. Dashed green arrows read turns on: one switch controls the tracer limiter, the passive scalars, the equation-of-state floors and the face floors together. Solid black arrows read requires or sets, dotted vermillion reads falls back to, and dash-dotted blue reads read by. The two paths on the right end at the redo detector, which is the only consumer of the limiter marks and of the implicit dry clamp. Every edge is stated and cited in the section it comes from.

Table 7.1.: The mechanisms of this chapter. "Guarantee" is what the mechanism actually promises; most promise nothing and repair instead.

scheme	switch	default	guarantee	main test
Tracer flux limiter (Section 7.1)	`limiter`	off	non-negative tracers under transport, per stage	`test_flux_positivity`
Energy and momentum carry (Section 7.2)	with the limiter	off	the withheld mass keeps its energy	`test_flux_positivity_carry`
Passive-scalar limiter (Section 7.3)	with the limiter	off	the same, for scalars	none
EOS limiter and floors (2)	`limiter`	off	none; a clamp that invents mass	none
Condensate borrow (Section 7.5)	with the limiter	off	conservative in mass and elements	none
Vapour column repair (Section 7.6)	with the limiter	off	conservative in mass, or it throws	none
Face floors (Section 7.7)	`limiter`	off	none; a clamp at the face	none

scheme	switch	default	guarantee	main test
Well-balanced face fallback (Section 7.8)	with the reference	—	no-op at rest	chapter 5 tests
Implicit dry clamp (Section 7.9)	with the implicit scheme	—	none; recorded, not prevented	none
Round-off thresholds (Section 7.10)	—	always	separates a repair from arithmetic	this chapter's check
Floor detector and redo (Section 7.11)	—	always	detection only	none
Dry-carry zero-base guard (Section 7.12)	with scalars and dry sources	—	finiteness, *not* positivity	`test_dry_carry_zero_base`

Two notes on how this chapter is organised against the outline, both of which the outline asks for. The carry appears there twice, as two inventories' accounts of one mechanism, and is one section here. The equation-of-state limiter appears three times, as a conserved pass, a primitive pass and a combined account with the floors and the marks; all three are one section here, because the single switch and the shared marks are one story. The vapour column repair and the condensate borrow are lifted out of the conserved pass into their own sections, which is what the outline recommends and what their difference in kind deserves: they are the only repairs in the chapter that conserve anything.

Where this chapter touches others: the limiter runs inside the stage operator of Section 4.2, between the Riemann solve and the flux divergence, and after the seam averaging of Section 4.14. The face floors belong to the reconstruction framework of Section 4.3. The implicit clamp is inside the solve of chapter 7, and the redo budget that ends a redo loop is chapter 7's too.

7.1. The tracer flux positivity limiter (`equation-of-state/limiter`)

7.1.1. Summary

> **At a glance**
>
> - Scales each face's tracer flux by a factor of the cell it drains, so no cell is emptied past zero.
> - Switch: `equation-of-state/limiter`, default false, and only where there are tracer rows.
> - Acts on tracer channels only. Density and pressure get floors, not a limiter.
> - Exactly conservative, because both cells of a face use the same factor.
> - Inactive where a cell can afford its outflow, so the high-order flux is untouched almost everywhere.

This is the only mechanism in snapy that *guarantees* anything about positivity, and it guarantees it for tracers alone. For each cell and species it computes the largest uniform scaling of that cell's outgoing fluxes that cannot drain it below zero in one forward-Euler step, then applies the factor of the donor cell to every face. It runs inside the stage operator of Section 4.2, after the seam averaging of Section 4.14 and before the flux divergence.

7.1.2. Derivation

Charge each cell for what leaves it. Summing the outgoing part of AF over the cell's faces,

$$\mathrm{out}_i = \sum_f \max(\pm A_f F_f,\, 0), \qquad \theta_i = \min\!\left(1,\ \frac{u_i V_i}{\Delta t\, \mathrm{out}_i}\right), \tag{7.1}$$

with the sign chosen so that only genuinely outgoing contributions count: inflow is not charged. Applying θ of the donor cell to every face then gives $u_i + \Delta t\, \Delta u_i \geq 0$ cell by cell.

`[C1]` checks the guarantee on the state rather than on the formula. Over four thousand random flux fields the unlimited update drives some cell negative in every one, worst -5.879; the limited update never does, worst exactly 0. The check computes the divergence independently of the limiter, so it measures the result and not the construction.

Two properties are worth separating, because they are easy to conflate and the chapter is more useful if they are kept apart.

Where the limiter acts. `[C2]` finds that every one of the 25 974 cells that can afford their outflow keeps $\theta = 1$ bitwise, and that no cell which cannot afford it was left unlimited. The scheme is therefore not a smoothing: away from depletion the high-order flux passes through untouched.

Why it conserves. The conservation does **not** come from the donor rule. It comes from both cells of a face seeing one number, so that the face cancels when the divergence is summed. `[C3]` separates these: on a closed block the limited tendency sums to 1.7×10^{-16} of the flux scale, while a per-cell limiter, in which each cell scales only its own outflow and its neighbour does not agree, leaks 8.5×10^{-1}. Any single factor per face telescopes; the donor rule is what buys positivity, and `[C7]` checks that rule separately on forty-six thousand faces.

The step is limited at the full Δt even inside a multi-stage integrator, and that is sound because of how the shipped integrators are built. [C5] reads their weights rather than assuming them: each stage is $w_0 U^n + w_1(U + \alpha\,\Delta U)$ with $w_0 + w_1 = 1$ and $\alpha = w_2/w_1 \leq 1$, a convex combination of a state already known non-negative and a damped full-step Euler update.

7.1.3. Numerical method

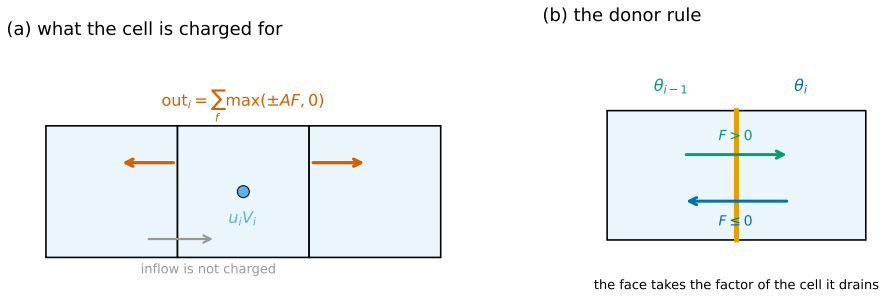


Figure 7.2.: What a cell is charged for, and which factor scales a face. (a) Only the outgoing parts of the face fluxes enter the sum (vermillion); an inflow (grey) is not charged, so a cell fed by its neighbours is not limited on their account. (b) The donor rule: the face takes the factor of the cell it drains, the lower cell where the flux is positive (green) and the upper cell otherwise (blue). Both cells of the face see that one number, which is what makes the scaling conservative.

The factors are built by `flux_positivity_theta` at `flux_positivity.cpp:22-72`. The outgoing sum is accumulated over exactly the faces the divergence will consume, with a pair of rectified terms per direction at `slice` (`flux_positivity.cpp:39-47`), so the limiter and the divergence cannot disagree about which face drains which cell.

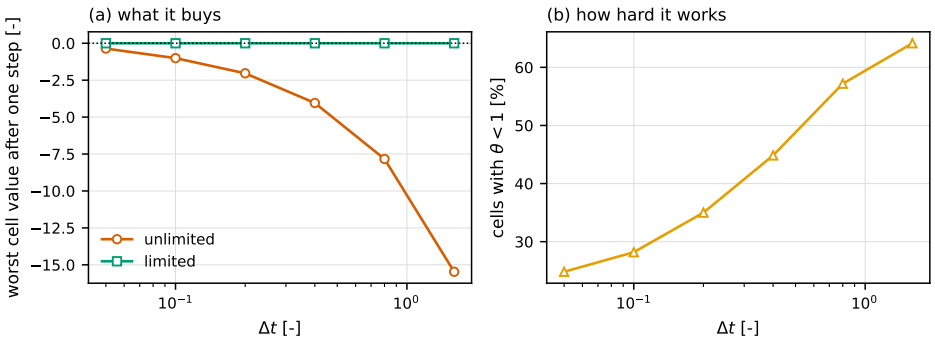


Figure 7.3.: How hard the limiter works and what it buys, against step size. (a) The worst cell value after one step: vermillion is the unlimited update, which goes increasingly negative as the step grows, and green is the limited update, which stays at zero. (b) The percentage of cells whose factor is below one, which rises with the step because more cells can no longer afford their outflow. Data from the committed output of the chapter's positivity check.

Two details of that function matter to a reader. The available mass stops a little short of zero, at `margin` (`flux_positivity.cpp:63-66`), because an exact-zero target rounds negative; that margin is Section 7.10. And ghost cells are given $\theta = 1$ here, because their outflow is never accumulated: the caller must fill the ghost layer the way conserved variables are filled before scaling, or the two ranks sharing a face would apply different factors and conservation would fail.

The scaling is `flux_positivity_scale_` at `flux_positivity.cpp:74-104`, which selects the donor per face with a `where` on the sign of the flux. The call site is `narrow` (`hydro_forward.cpp:648-669`), which also keeps the census that Section 7.10 describes, and the ghost fill it depends on follows at `pmb` (`hydro_forward.cpp:671-686`), a raw copy that is never interpolated because an interpolated ghost is not the neighbour's factor.

7.1.4. Code

Every link below is at the pin snapy@e894700ff7aee30b52882e5202b16461413780b0. The symbol column is the citation's anchor.

Table 7.2.: Code map of the tracer flux positivity limiter.

what	code link	symbol	switch / condition
the factors	`flux_positivity.cpp:22-72`	`flux_positivity_theta`	limiter on, tracers present
the outgoing sum	`flux_positivity.cpp:39-47`	`relu`	
the round-off margin	`flux_positivity.cpp:63-66`	`margin`	

what	code link	symbol	switch / condition
the scaling	`flux_positivity.cpp: 74-104`	`flux_positivity_scale_`	
the call in the stage	`hydro_forward.cpp: 648-669`	`flux_positivity_theta`	
the ghost fill it needs	`hydro_forward.cpp: 671-686`	`interpolate`	
the contract, written out	`flux_positivity.hpp: 32-67`	`positivity_theta`	

7.1.5. Tests

Table 7.3.: Tests of the tracer flux positivity limiter.

test	asserts	tolerance
`tests/test_flux_positivity.py`	the limiter on a tracer deck	per test
`tests/test_flux_positivity_cubedsphere.py`	the same across panel seams	per test
`tests/test_flux_positivity_cubedsphere_moist.py`	the moist arm across seams	per test

test	asserts	tolerance
this section's check	`[C1]` the guarantee measured on the state; `[C2]` inactive away from depletion; `[C3]` conservation and its counter-example; `[C5]` the integrator argument; `[C7]` the donor rule	bitwise and round-off

The check is `src/snapy_report/ch08/positivity_check.py`, output `src/snapy_report/ch08/positivity_check.out`, 7 of 7 claims passing.

7.1.6. Limits

⚠ Limits

- Tracers only. Nothing here keeps dry density or pressure positive; those have floors

  (2) and a redo detector (Section 7.11), and neither is a guarantee.

- The guarantee is per stage, for transport only. A source term, a sedimentation flux added after the limiter, or the implicit solve can still drive a tracer negative.

1. **The ghost fill is a precondition, not a check.** If a caller scales before filling θ's ghost layer, the two ranks of a seam face apply

different factors and conservation breaks silently. The code says so in its header; nothing asserts it at runtime.

2. **Hypothesis, not measured here.** That limiting at the full Δt is enough for the multi-stage update rests on the convex-combination argument `[C5]` verifies for the shipped weights. It is an argument about the weights, not a measurement of a multi-stage run; no check in this chapter integrates a full Runge-Kutta step.

7.2. The carry of energy and momentum with withheld mass

7.2.1. Summary

> **At a glance**
>
> - Mass the limiter withholds must not leave its energy and momentum behind.
> - Switch: the same as the limiter, and only where the equation of state defines a species enthalpy.
> - The withheld mass of a face takes the enthalpy and velocity of that face's donor.
> - The x_1 flux is split into an advected and a settling part, each with its own donor.

The limiter of Section 7.1 scales a species mass flux down. That mass does not vanish: it stays in the donor cell. If the energy and momentum fluxes are left unscaled, the mass stays behind but its enthalpy and momentum leave, and the cell loses energy it still holds. This section is the correction that keeps the books straight.

The outline carries this scheme twice, as 8.2 and 8.3, from two inventories. They describe the same mechanism and are merged here into one section.

7.2.2. Derivation

Write the unscaled face flux of species n as F, and let θ_d be the factor of that face's donor. The limiter replaces F by $\theta_d F$, so the mass withheld at that face is

$$\mathrm{d}m = (1 - \theta_d)\, F. \tag{7.2}$$

The energy flux of the face must lose $\mathrm{d}m\, h_{\mathrm{spec}}$ and the momentum flux $\mathrm{d}m\, v$, both evaluated **at the donor**, because that is the cell the mass did not leave.

`[C6]` checks precisely that correspondence rather than restating Equation 7.2: it computes the energy the withheld mass would have carried as the difference between the unscaled and scaled mass flux times the donor's enthalpy, independently of the expression the code uses, and compares. The two agree to 1.8×10^{-15} in energy and 8.9×10^{-16} in momentum over two thousand face sets.

The donor is chosen per face and per part of the flux, not per cell. In x_1 the flux carries two physically distinct pieces, an advected part and a settling part from sedimentation, and they need not move in the same direction: a species may be advected upward while settling downward through the same face. Each part therefore takes its own donor, which is why the code applies the withholding twice in x_1 and once elsewhere.

7.2.3. Numerical method

The carry is `flux_positivity_carry_` at `flux_positivity.cpp:106-149`. The withheld share is taken from the *net* flux's donor at `theta` (`flux_positivity.cpp:124-125`), matching the choice `flux_positivity_scale_` made, so the mass that was actually withheld is the mass that is accounted for. The energy and momentum subtraction is

`part` (`flux_positivity.cpp:127-135`), with the enthalpy and velocity taken from the lower or upper cell according to the sign of the part being withheld.

The x_1 split is `sed` (`flux_positivity.cpp:136-143`): when a sedimentation flux is given, the advected part is the net flux minus it, and `withhold` is called once on each. The other two directions have no settling flux and take the single branch.

The species enthalpy comes from the equation of state at `species_enthalpy` (`hydro_forward.cpp:675-677`), computed on the primitive state with ghosts included. The code comment there records that exchanging it alongside the factors changed no bit, so it is not exchanged; that is a measurement someone made, not a derivation, and it is recorded as such.

7.2.4. Code

Table 7.4.: Code map of the carry.

what	code link	symbol	switch / condition
the carry	`flux_positivity.cpp:106-149`	`flux_positivity_carry_`	limiter on, species enthalpy defined
the withheld share, from the net donor	`flux_positivity.cpp:124-125`	`share`	
energy and momentum subtraction	`flux_positivity.cpp:127-135`	`withhold`	
the x_1 advected and settling split	`flux_positivity.cpp:136-143`	`fsed`	x_1 only

what	code link	symbol	switch / condition
the species enthalpy	`hydro_forward.cpp:675-677`	`species_enthalpy`	

7.2.5. Tests

Table 7.5.: Tests of the carry.

test	asserts	tolerance
`tests/test_flux_positivity_carry.cpp`	the carry of energy and momentum with withheld mass	per test
this chapter's check	`[C6]` the withheld energy and momentum match the withheld mass at the same face and donor	round-off

7.2.6. Limits

> **Limits**
>
> - The carry runs only where the equation of state defines a species enthalpy. With an equation of state that does not, the limiter still withholds mass and the energy is not corrected, so the two mechanisms are not switched together.

1. **Kinetic energy.** The withheld energy is the species enthalpy times the mass. Whether the kinetic energy of the withheld mass is accounted for consistently with the momentum correction is not something this chapter measures, and no test in the repository isolates it.
2. **Hypothesis.** That the settling and advected parts need separate donors because they can move in opposite directions is an argument from the physics of sedimentation and from the shape of the code. No test in the repository constructs a face where the two parts disagree in sign.

7.3. The passive-scalar limiter and the complement bound

7.3.1. Summary

> **At a glance**
>
> - Passive scalars get the same limiter as the hydro species, from the same function.
> - Switch: the same limiter switch; the scalar module runs its own call.
> - The factors are exchanged as a raw copy, never interpolated, for the same reason as the hydro ones.
> - A scalar bounded above by its complement needs a second constraint, which is noted here, not derived.

Passive scalars are transported by the same divergence as the hydro species and are limited by the same scheme. This section exists because the call is a separate one, in a separate module, with its own counters, and a reader looking for it in the hydro path will not find it.

7.3.2. Derivation

Nothing new: the derivation is Section 7.1, and the scalar module calls the same two functions. The argument for the ghost exchange being a raw copy rather than an interpolation is also the same, and the code comment points back to the hydro call site rather than repeating it.

The one thing specific to scalars is the upper bound. A scalar that represents a fraction is bounded above by the positivity of its complement, so enforcing non-negativity on the scalar alone is not enough to keep it below one. The code comment at `the` (`scalar.cpp:28-32`) records this; the chapter records it as a stated design intent and does not claim a derivation, because none is committed.

7.3.3. Numerical method

The call is `_flux1` (`scalar.cpp:158-162`), which builds the factors with the same `flux_positivity_theta`, counts the cells it limited, and scales through the synchronised factors. The synchronisation is the lambda at `theta_opts` (`scalar.cpp:140-155`): it puts the factors into a variable map, exchanges them with interpolation switched off, and applies the physical boundary functions, which is exactly the fill the limiter's contract requires.

7.3.4. Code

Table 7.6.: Code map of the passive-scalar limiter.

what	code link	symbol	switch / condition
the scalar limiter call	`scalar.cpp: 158-162`	`flux_positivity_theta`	limiter on
the factor exchange	`scalar.cpp: 140-155`	`sync_theta`	
the complement bound, as intent	`scalar.cpp: 28-32`	`positivity`	

7.3.5. Tests

No test in the repository selects the passive-scalar limiter on its own. Its behaviour follows from Section 7.1, whose checks cover the shared function, but the scalar call path itself is covered by nothing.

7.3.6. Limits

⚠ Limits

- The upper bound is not enforced. The limiter keeps a scalar non-negative; nothing in this path keeps it below its complement, and the code comment describes that as the intent rather than the behaviour.

1. **Untested path.** The scalar call, its counters and its exchange are exercised by no test, so this section is written from reading alone.

7.4. The equation-of-state limiter: two passes, floors and marks

7.4.1. Summary

> At a glance
>
> - Two passes: one on the conserved state, one on the primitive state. They are not the same operation.
> - Switch: `equation-of-state/limiter`, default false. Floors default to 10^{-6}, 10^{-3} and 20 K.
> - Every floor is a clamp, so every floor invents mass or energy. Nothing removes it again.
> - The order of the two conserved floors is load-bearing: the energy floor reads the floored density.
> - A repair in the interior sets a mark that the redo detector reads.

The outline carries this material three times: as the conserved pass, as the primitive pass, and as a second inventory's account of both together with the floors and the marks. All three are merged here, because the single switch, the shared floors and the shared marks are one story and splitting them puts the reader in two places at once.

7.4.2. Derivation

There is no derivation for a floor. What there is instead are three facts about the operator that a reader needs and that are easy to get wrong, and the check establishes each separately.

The passes do not commute. The conserved pass floors the dry density first and then floors the total energy at the kinetic energy plus the internal energy of a temperature floor, and that internal energy is computed from

the density the previous statement has already clamped. [C1] reorders the two statements and measures the result: on three thousand random states straddling the floors, the energy differs in 2862 of them, by up to 1.33×10^4. The order is not incidental.

The floors are repairs, not conservative operations. A clamp can only add. [C2] measures how much: on four thousand cells straddling the floors the passes add 4.736×10^{-4} of dry mass and 1.294×10^2 of total energy, both strictly positive. Nothing anywhere in the code removes it again. This is the sense in which snapy guarantees no positivity of density or pressure: it repairs them, at a cost it does not track.

The two passes floor different things. [C5] separates them. The primitive pass floors density and pressure and clamps species mass fractions at zero; it has no energy row and applies no temperature floor. The conserved pass has no pressure row and floors energy through the temperature floor instead. Reading either as the other applied to different variables is wrong.

The marks are the link to the redo machinery. [C4] checks the three cases that matter: an untouched state marks nothing; a density below the floor sets the patch mark; a NaN sets the NaN mark; and the same repair outside the interior marks nothing at all, because a repair in a ghost cell must not force a block to redo its step.

7.4.3. Numerical method

The conserved pass is `cons` (`equation_of_state.cpp:192-235`) and runs in a fixed order: NaNs are zeroed and marked at `equation_of_state.cpp:201-203`, the dry density is clamped at `equation_of_state.cpp:209`, and the energy is clamped at `equation_of_state.cpp:226-234` to the kinetic energy plus the internal energy of the temperature floor. The kinetic energy there is built from the momentum raised with

the cell's own metric, so the floor is correct on a curved grid and not only in Cartesian.

The primitive pass is `prim` (`equation_of_state.cpp:325-350`): NaNs, the density floor at `equation_of_state.cpp:333`, the species clamp at `equation_of_state.cpp:346` and the pressure floor at `equation_of_state.cpp:349`.

The floor values are YAML keys with defaults at `density_floor` (`equation_of_state.cpp:65-72`): a density floor of 10^{-6}, a pressure floor of 10^{-3} and a temperature floor of 20 K. They are absolute numbers in the deck's own units, not fractions of anything, so whether they are conservative or catastrophic depends on the scaling of the problem.

The marks are a two-element device-side boolean buffer, reset at `reset_limiter_marks` (`equation_of_state.cpp:352-354`) and read by the redo detector of Section 7.11. Entry one is the NaN mark and entry zero the patch mark, set at `limiter_marks_` (`equation_of_state.cpp:235`) by comparing the density and energy rows against a copy taken before the floors.

7.4.4. Code

Table 7.7.: Code map of the equation-of-state limiter.

what	code link	symbol	switch / condition
the conserved pass	`equation_of_state.cpp:192-235`	`apply_conserved_limiter_`	`limiter: true`
NaN to zero, marked	`equation_of_state.cpp:201-203`	`masked_fill_`	

what	code link	symbol	switch / condition
the dry density floor	`equation_of_state.cpp:209`	`density_floor`	
the energy floor	`equation_of_state.cpp:226-234`	`min_ie`	an energy row exists
the patch mark	`equation_of_state.cpp:235`	`limiter_marks_`	interior only
the primitive pass	`equation_of_state.cpp:325-350`	`apply_primitive_limiter_`	`limiter: true`
its density floor	`equation_of_state.cpp:333`	`density_floor`	
its species clamp	`equation_of_state.cpp:346`	`clamp_min_`	species present
its pressure floor	`equation_of_state.cpp:349`	`pressure_floor`	
the floor defaults	`equation_of_state.cpp:65-72`	`density_floor`	YAML
the mark buffer	`equation_of_state.cpp:352-354`	`reset_limiter_marks`	

7.4.5. Tests

Table 7.8.: Tests of the equation-of-state limiter.

test	asserts	tolerance
this section's check	`[C1]` the two floors do not commute; `[C2]` the mass and energy a floor invents; `[C4]` the marks and the interior rule; `[C5]` the two passes floor different rows	measured, bitwise

The check is `src/snapy_report/ch08/floors_check.py`, output `src/snapy_report/ch08/floors_check.out`, 5 of 5 claims passing. No ctest in the repository isolates the floors themselves; they are exercised incidentally by every deck that switches the limiter on.

7.4.6. Limits

> **Limits**
>
> - The floors are absolute values in the deck's units. A deck whose densities are legitimately below 10^{-6} will have every cell floored, silently and on every call.
> - One switch turns on this limiter, the tracer limiter, the reconstruction floors and the species clamps together. They cannot be selected independently, which is a design decision recorded in the sources, not an accident.

1. **The invented mass is not tracked.** `[C2]` measures it in a synthetic state; the code keeps a boolean mark but no budget. A run cannot report how much mass its floors created.

2. **Open question.** Whether the energy floor should use the floored or the unfloored density is not discussed in any source. [C1] shows the choice is observable; the code makes it one way, and this chapter records which, not which is right.

7.5. The condensate borrow

7.5.1. Summary

> At a glance
>
> - A negative condensate is repaired by borrowing the deficit from the parent vapour in the same cell.
> - Switch: part of the conserved limiter; applies to clouds that carry parent-vapour metadata.
> - Exactly conservative in mass, energy and elements, unlike clamping the condensate to zero.
> - Per-cell, so a ghost copy receives the identical repair and ranks stay in step.
> - A cell whose vapour cannot cover the deficit goes vapour-negative and is left to the column repair.

This is the one repair in the chapter that is conservative, and it is worth separating from the floors it sits among for exactly that reason. The outline asks for it as its own scheme and the distinction earns the space.

7.5.2. Derivation

A negative condensate does not mean mass is missing. It means the tracer flux over-drained the cell and the mass is in a neighbour. Clamping it to zero invents that mass, one way, every time.

The alternative the code takes is to move mass between rows of the same cell: take the deficit out of the parent vapour and set the condensate to exactly zero. Total mass in the cell is unchanged, and so are the elements, because vapour and its condensate are the same substance in different phases.

[C3] measures the difference rather than asserting it. Over two thousand cells with a negative condensate, clamping to zero adds 3.770×10^{-2} of mass, while the borrow changes the total by exactly 0. The code comment at `parentless` (`equation_of_state.cpp:242-252`) records the motivation from a measurement: the clamp was responsible for 102% of the total-mass drift in silicate cloud runs, which is a number from a run that this chapter cites rather than reproduces.

7.5.3. Numerical method

The repair is `parent_slot` (`equation_of_state.cpp:252-269`). For each cloud row with parent metadata, the deficit is the negative part at `clamp_max` (`equation_of_state.cpp:264`), it is added to each parent vapour in proportion to its mass fraction at `equation_of_state.cpp:265-267`, and the condensate is then set to exactly zero.

Two conditions are worth noting. A cloud not produced by nucleation has no parent metadata and is skipped, flagged for the column repair of Section 7.6 instead. And the repair is purely local, so a ghost cell that holds a copy of the same state receives the identical repair without any communication, which is what keeps ranks consistent.

7.5.4. Code

Table 7.9.: Code map of the condensate borrow.

what	code link	symbol	switch / condition
the motivation, measured	`equation_of_state.cpp:242-251`	`clamp_min_`	
the repair	`equation_of_state.cpp:252-269`	`parentless`	clouds with parents
the deficit	`equation_of_state.cpp:264`	`deficit`	
borrowing from the parents	`equation_of_state.cpp:265-267`	`parent_slot`	

7.5.5. Tests

Table 7.10.: Tests of the condensate borrow.

test	asserts	tolerance
this chapter's check	`[C3]` the borrow conserves mass where a clamp invents it	exact

No ctest isolates the borrow.

7.5.6. Limits

⚠ Limits

- A cell whose vapour cannot cover the deficit goes vapour-negative. The borrow does not fail there; it hands the problem to the column repair, which may itself fail.

1. **The phase shift is not free.** The repair moves mass between phases, which the code comment puts at about one per cent of the local value and expects the saturation adjustment to re-equilibrate within the same cycle. That expectation is a hypothesis recorded in the code; no test in the repository measures the residual phase error after the adjustment.
2. **Parentless clouds.** Clouds without nucleation metadata get no borrow at all, and the chapter cannot say how common that case is in practice.

7.6. The vapour column repair

7.6.1. Summary

ℹ At a glance

- A repair along a whole x_1 column, not a floor: it moves mass within the column.
- Switch: part of the conserved limiter, where a vapour row has gone negative.
- Conserves mass by weighting with cell volumes, so it repairs mass and not density.
- Gathers the column across ranks when the x_1 axis is split.

- Clouds tolerate an unrepairable column and clamp; vapour must throw instead.

When the borrow of Section 7.5 cannot cover a deficit from the parent vapour in the same cell, the vapour itself goes negative and a local repair has no mass to work with. The column repair is the fallback: it looks along the whole x_1 column for mass to move.

7.6.2. Derivation

The scheme is a redistribution, so the thing to establish is what it conserves. It weights by cell volume at `cell_volume` (`equation_of_state.cpp:271-272`), which is the point recorded in the code comment there: a column repair that moved *densities* would not conserve mass on a non-uniform grid, because equal densities in unequal cells are unequal masses. Weighting by volume makes the redistribution conservative in mass.

This chapter does not port the repair, and the reason is worth stating rather than hiding: the search itself is a device kernel reached through a dispatch call, so a numpy port would be a reimplementation rather than a port, and a check built on it would test my reimplementation and not the code. The conservation property above is read from the code and its comment; it is not measured here.

7.6.3. Numerical method

The repair is the lambda at `split` (`equation_of_state.cpp:274-300`). When the x_1 axis is split across ranks it gathers the column first, so the repair sees the whole column rather than one block's piece of it.

The two callers are deliberately different, and the comment at `repair_column` (`equation_of_state.cpp:274-276`) says why: clouds tolerate a

column that cannot be repaired and fall back to a clamp at `clamp_min_` (`equation_of_state.cpp:310`), while vapour must throw rather than copy a bad gathered column back, which is the check at `err` (`equation_of_state.cpp:296-299`). A failure in the vapour path is therefore a hard stop, not a silent repair.

7.6.4. Code

Table 7.11.: Code map of the vapour column repair.

what	code link	symbol	switch / condition
volume weighting, so mass is conserved	`equation_of_state.cpp:271-272`	`vol`	
the two callers differ by design	`equation_of_state.cpp:274-276`	`repair_column`	
the repair	`equation_of_state.cpp:274-300`	`repair_column`	vapour negative
vapour must succeed	`equation_of_state.cpp:296-299`	`TORCH_CHECK`	`require_success`
clouds clamp instead	`equation_of_state.cpp:310`	`clamp_min_`	parentless or unrepairable

7.6.5. Tests

No test in the repository isolates the column repair, and this chapter does not check it either, for the reason given in the Derivation. It is the least

verified mechanism in the chapter.

7.6.6. Limits

⚠ Limits

- The vapour path throws when the column cannot be repaired. That is a crash, not a degraded run, and it is the intended behaviour rather than a defect.
- The repair needs the whole column. On a split x_1 axis it gathers across ranks, so it is a collective operation inside what is otherwise a local limiter.

1. **Not ported, not measured.** The conservation claim is read from the code and its comment. No check in this chapter and no ctest in the repository demonstrates it.
2. **Cost.** Gathering a column is communication in the middle of the equation-of-state call. Nothing in the repository measures how often it fires or what it costs.

7.7. Reconstruction-stage floors

7.7.1. Summary

ℹ At a glance

- Floors applied to the face states, not to the cell state, right after each reconstruction.
- Switch: the same limiter switch; the floor values come from the equation of state.
- Density and pressure take the floors; tracers are clamped at

zero.
- The tracer clamp fires even when the caller asked for no floors, which the well-balanced path does.

A high-order reconstruction can overshoot into a negative face density or pressure even from a perfectly admissible cell state. These floors catch that at the face, before the Riemann solver sees it. They belong to the reconstruction framework of Section 4.3 and are described here because they are part of the positivity story rather than part of the interpolation.

7.7.2. Derivation

None: a floor is a clamp, and everything said about non-conservation in 2 applies here too. What is worth recording is the asymmetry between the three row groups, because it is not what a reader would guess.

Density and pressure are floored only when the caller asked for floors. Tracers are clamped at zero whenever the limiter is on, *regardless* of what the caller asked, at `clamp_min_` (`reconstruct.cpp:153-155`). The well-balanced path of chapter 5 calls the reconstruction with floors switched off precisely so that it can handle its own perturbation variables, and it still gets its tracers clamped.

7.7.3. Numerical method

The three groups are floored in place as each is reconstructed: density at `eos` (`reconstruct.cpp:135-137`), pressure at `eos` (`reconstruct.cpp:143-145`) and the tracers at `reconstruct.cpp:153-155`. The values are read from the equation-of-state options and not from the reconstruction options, which carry floor members that `forward` never reads.

7.7.4. Code

Table 7.12.: Code map of the reconstruction-stage floors.

what	code link	symbol	switch / condition
the face density floor	`reconstruct.cpp:135-137`	`density_floor`	limiter and floor
the face pressure floor	`reconstruct.cpp:143-145`	`pressure_floor`	limiter and floor
the tracer clamp	`reconstruct.cpp:153-155`	`clamp_min_`	limiter on, floor ignored

7.7.5. Tests

No test isolates the face floors. They fire inside every limited deck.

7.7.6. Limits

1. **The tracer clamp ignores its caller.** A path that switches floors off to protect its own variables still has its tracer face states clamped. Whether that is intended is not recorded anywhere; the chapter records the behaviour.
2. **Same absolute values.** The face floors use the same absolute numbers as the cell floors, so the scaling caveat of 2 applies to them unchanged.

7.8. The well-balanced face positivity fallback

7.8.1. Summary

> **At a glance**
>
> - In the well-balanced x_1 path the face state is a reference plus a perturbation, and the sum can go negative.
> - Switch: active with the well-balanced reference; the ordinary face floors are switched off there.
> - A non-positive face pressure falls back to the reference pressure.
> - A non-positive face density falls back to the *adjacent cell's* density, not to the reference.
> - At rest the perturbation vanishes, so the fallback never fires and well-balancing is preserved.

The well-balanced path reconstructs a perturbation about a hydrostatic reference and adds the reference back at the face. Near the top of a column the reference is tiny and a nonlinear overshoot in the perturbation can take the sum below zero. This fallback catches that, and the choice of what to fall back to is the interesting part.

7.8.2. Derivation

The pressure case is the simple one: a face pressure that comes out non-positive is replaced by the reference pressure at that face, at `psf_lo` (`hydro_forward.cpp:357-360`). The reference is positive by construction, and at rest the perturbation is zero, so the condition is never met and the well-balanced property survives untouched.

The density case is where a reasonable-looking choice is wrong, and the code comment at `dsf` (`hydro_forward.cpp:362-367`) records why from experience rather than from theory. Falling back to the reference density

fails when the reference overestimates the density aloft: a bottom-anchored isentrope did so by orders of magnitude, and every floor event at a reflecting top wall then became a large spurious wall impedance. The fallback is therefore the adjacent cell's density, which is a real local value rather than an extrapolated reference.

A second detail in the same block is a latent-bug note worth carrying into the report. The shift along x_1 uses edge replication rather than a circular roll, and the comment at `density` (`hydro_forward.cpp:368-373`) says plainly that the circular word is not consumed today, but would become live the moment a caller changed the face range. That is a hazard recorded in the code and not a current defect.

7.8.3. Numerical method

The block is `copy_` (`hydro_forward.cpp:350-372`). The reconstruction that feeds it is called with floors switched off at `wtmp` (`hydro_forward.cpp:346-348`), precisely because the ordinary reconstruction floors would clamp a legitimate negative perturbation; this fallback replaces them for this path.

7.8.4. Code

Table 7.13.: Code map of the well-balanced face fallback.

what	code link	symbol	switch / condition
reconstruction with floors off	`hydro_forward.cpp:346-348`	`precon1`	well-balanced x_1

what	code link	symbol	switch / condition
the fallback block	`hydro_forward.cpp:350-372`	`wtmp`	
the pressure fallback	`hydro_forward.cpp:357-360`	`psf_lo`	face pressure non-positive
why not the reference density	(`hydro_forward.cpp:362-367`)	`dsf`	
edge replication, not a roll	`hydro_forward.cpp:368-372`	`roll`	

7.8.5. Tests

No test isolates the fallback. Its no-op-at-rest property is covered indirectly by the well-balanced tests of chapter 5, which would fail if it fired on a resting column.

7.8.6. Limits

⚠ Limits

- The density fallback substitutes a neighbouring cell's density into a face state. That is a local value, but it is not the reconstruction's answer, and nothing measures the error it introduces when it fires.

1. **Hypothesis, from a run not reproduced here.** That the reference-density fallback caused a large spurious wall impedance is recorded in the code comment from a bottom-anchored isentrope case. This chapter cites that reasoning; it does not reproduce the run.
2. **A latent hazard.** The edge-replication note describes a wraparound that is unreachable with today's face range. It is recorded because the range is the only thing keeping it unreachable.

7.9. Dry-channel positivity inside the implicit solve

7.9.1. Summary

> At a glance
>
> - The vertical implicit solve can return a mass change that would take a cell dry-negative.
> - Switch: none of its own; active with the implicit scheme of chapter 7.
> - The clamp is recorded in a per-step buffer, which the redo detector reads.
> - The limiter of Section 7.1 does not cover this: it acts on explicit transport only.

Everything else in this chapter acts on explicit fluxes. The implicit correction of chapter 7 solves a column and hands back a mass change that no face limiter has seen, and that change can exceed what a cell holds. This section is the guard on that path.

7.9.2. Derivation

There is nothing to derive; the mechanism is a clamp with a record. What matters for the reader is the boundary of responsibility. The tracer limiter guarantees that *transport* cannot empty a cell, and that guarantee is per stage, for the explicit flux divergence. The implicit solve runs afterwards, on the same state, and its correction is outside that guarantee entirely. A cell that survived the explicit stage can still be driven dry by the implicit one, and that is the gap this clamp covers.

7.9.3. Numerical method

The buffer is registered at `_dry_clamp_step` (`implicit_hydro.cpp:120-121`) and updated at `implicit_hydro.cpp:379-380`, where it takes the running maximum of the clamp applied in this step rather than a count. Storing a magnitude rather than a flag is what lets the redo detector distinguish a round-off clamp from a real one.

It is read through the accessor at `dry_clamp_step` (`implicit_hydro.hpp:92`) and cleared between steps at `reset_dry_clamp_step` (`implicit_hydro.hpp:93`), so the value a redo decision sees belongs to the step being judged.

7.9.4. Code

Table 7.14.: Code map of the implicit dry-channel clamp.

what	code link	symbol	switch / condition
the buffer	`implicit_hydro.cpp:120-121`	`_dry_clamp_step`	implicit scheme on

what	code link	symbol	switch / condition
the running maximum	`implicit_ hydro.cpp: 379-380`	`_mass_corr`	
the accessor	`implicit_ hydro.hpp:92`	`dry_clamp_ step`	
cleared between steps	`implicit_ hydro.hpp:93`	`reset_dry_ clamp_step`	

7.9.5. Tests

No test in the repository isolates this clamp. It is written here from reading.

7.9.6. Limits

> ⚠ Limits
>
> - The implicit path has no positivity guarantee, only this clamp and the redo detector it feeds.

1. **Not ported.** This chapter's checks cover the explicit limiter and the floors. The implicit clamp is inside a solve this chapter does not reproduce, so its behaviour is recorded from the code.
2. **A magnitude, not a budget.** The buffer records the largest clamp of a step. How much mass the clamps created over a run is not tracked anywhere.

7.10. Round-off thresholds for positivity events

7.10.1. Summary

> **At a glance**
>
> - Two constants decide what counts as a real positivity event rather than arithmetic noise.
> - The limiter stops short of zero by 4096 ulp in float64 and 64 ulp in float32.
> - The thresholds are fractions of the cell's *total gas mass*, not of the species being limited.
> - The clip itself is always applied; the threshold only decides whether it is reported and marked.

A limiter that aims at exactly zero will overshoot, because the target rounds negative. A census that counts every such overshoot will report a flood of events that mean nothing. These two constants are how the code separates a real repair from arithmetic, and the float32 value is not the float64 one.

7.10.2. Derivation

The margin leaves a fixed fraction of the cell's own species mass behind on purpose. In float64, 4096 ulp is 9.09×10^{-13} of the gas mass, which `[C4]` computes: negligible against anything physical.

The same 4096 ulp in float32 would be 4.88×10^{-4}, which the header calls the size of a real repair. Using it would make the margin itself larger than the events it is meant to filter out. The float32 constant is therefore 64 ulp, giving 7.63×10^{-6}, and `[C4]` checks all three numbers together so the reason for the second constant is visible rather than asserted.

The threshold is measured against the cell's total gas mass over all species, not against the species being limited. That is what makes it robust for a trace species: an almost-empty cell drained by a round-off face flux has a factor of zero at any step size, and its withheld mass is around 10^{-20} of the cell's. Scaled against the trace species itself, that would look like total depletion; scaled against the cell's gas mass, it is correctly reported as nothing.

7.10.3. Numerical method

The constants are `constexpr` (`flux_positivity.hpp:11-23`), where the comment states both the intent and the float32 reasoning. The margin enters the factors at `margin` (`flux_positivity.cpp:63-66`).

The census is at `cells` (`hydro_forward.cpp:656-669`). An event is counted as *severe* only when the factor is below 0.9 **and** the withheld mass is above the threshold times the cell's gas mass, the two conditions at `hydro_forward.cpp:663-668`. The smallest factor seen is recorded separately, so a run can report how hard the limiter worked without that number being distorted by round-off events.

7.10.4. Code

Table 7.15.: Code map of the round-off thresholds.

what	code link	symbol	switch / condition
the two constants and why	`flux_positivity.hpp:11-23`	`kPositivityRoundoffUlp`	

what	code link	symbol	switch / condition
the margin in the factors	`flux_positivity.cpp: 63-66`	`margin`	
the census	`hydro_forward.cpp: 656-669`	`_positivity_hits`	limiter on
the severity test	`hydro_forward.cpp: 663-668`	`_positivity_severe`	

7.10.5. Tests

Table 7.16.: Tests of the round-off thresholds.

test	asserts	tolerance
this chapter's check	`[C4]` the two margins as fractions of the gas mass, and why float32 needs its own	exact arithmetic

7.10.6. Limits

1. **The constants are not derived.** They are chosen so that the margin sits below a real repair and above arithmetic noise. The header gives the reasoning for the float32 value; neither constant comes from an error analysis.
2. **A fixed fraction is left behind.** The limiter deliberately does not drain a cell to exactly zero. That residue is bounded by the margin and is not accounted anywhere.

7.11. The floor detector and the redo decision

7.11.1. Summary

> **At a glance**
>
> - Detection, not repair: it decides whether the step just taken should be redone.
> - A primitive at or below 1.001 times a floor counts as a hit.
> - The marks set by the limiter passes are read alongside it.
> - The detector's own restore must not mark a repair, so the marks are reset around it.

Everything else in this chapter keeps the state admissible or records that it could not. This is the mechanism that decides the step was bad enough to take again, and it is the closest thing snapy has to a positivity failure path.

7.11.2. Derivation

The test is a margin above the floor rather than equality with it, at `hit` (`meshblock.cpp:1199-1202`): a minimum density at or below 1.001 times the density floor counts as a hit, and the same for pressure where there is a pressure row. Testing equality with the floor would miss a cell that was floored and then nudged by a later operation; the margin catches the neighbourhood of the floor instead.

The ordering around it carries a subtlety the code calls out at `reset_limiter_marks` (`meshblock.cpp:1259-1261`): the detector restores primitives in order to look at them, and that restore itself runs a conversion which can trip the limiter and set a mark. The marks are therefore reset so that the detector's own work is not mistaken for a repair of the step being judged.

7.11.3. Numerical method

The detector is `hydro_u` (`meshblock.cpp:1190-1204`). The marks are read through `limiter_hits` (`meshblock.cpp:1211-1218`), which returns the pair set by the limiter passes of 2, and the implicit clamp of Section 7.9 is read through its own accessor at `vic_dry_clamp_hit` (`meshblock.cpp:1206-1208`). The redo decision combines them at `meshblock.cpp:1276-1281`, where the floor test is run first and the marks are read afterwards, in that order, for the reason above.

7.11.4. Code

Table 7.17.: Code map of the floor detector.

what	code link	symbol	switch / condition
the detector	`meshblock.cpp:1190-1204`	`floor_hit`	
the margin above the floor	`meshblock.cpp:1199-1202`	`density_floor`	
the implicit clamp accessor	`meshblock.cpp:1206-1208`	`dry_clamp_step`	implicit on
reading the limiter marks	`meshblock.cpp:1211-1218`	`limiter_marks`	
resetting around the restore	`meshblock.cpp:1259-1261`	`reset_limiter_marks`	
the decision	`meshblock.cpp:1276-1281`	`floor_hit`	

7.11.5. Tests

No test in the repository isolates the detector. It is written here from reading.

7.11.6. Limits

> ⚠ Limits
>
> - A redo repeats the step; it does not make the step succeed. A column that floors at every step size will keep flooring, and the redo budget of chapter 7 is what ends the loop.

1. **The margin is a constant.** The factor 1.001 is not derived and not configurable. A cell legitimately sitting a tenth of a per cent above the floor is read as a hit.
2. **Detection is per block.** One floored cell marks the whole block for a redo, and nothing records which cell it was.

7.12. Dry-carry of passive tracers, and the zero-base guard

7.12.1. Summary

> ℹ At a glance
>
> - When a forcing changes dry mass, the passive tracers must be carried with it.
> - An addition carries the ratio at source entry; a removal carries the source-free stage ratio.

- A zero source-free density would make that ratio 0/0, so it falls back to the entry ratio.
- This is a finiteness guard, not a positivity one: the result may legitimately be negative.

This is the one entry in the chapter that is not about positivity at all, and the distinction is worth making loudly because its name and its neighbours both suggest otherwise. It keeps an arithmetic expression finite. The tracer it protects can still go negative, and the repository's own test for it accepts a negative value.

7.12.2. Derivation

A forcing that writes a dry-mass tendency has to say what happens to the tracers riding on that mass. Addition and removal are not symmetric, and the code comment at `carry_dry_source` (`meshblock.cpp:631-634`) gives the reason: an addition carries the ratio as it stands at source entry, while a removal carries the ratio the stage would have had with no source at all, so that the removal cannot amplify a ratio that transport or the implicit solve has just changed.

Computing that source-free ratio means dividing the source-free scalar by the source-free density, and nothing prevents that density from being zero in a cell that is empty with the source removed. The guard at `rho_base` (`meshblock.cpp:640-642`) substitutes the entry ratio there, which keeps the expression finite.

What the guard does *not* do is bound the result. The repository's own test asserts a finite answer at a zero base and accepts a bottom value of -5×10^{-4}: a negative tracer is the expected outcome, not a failure. Passive scalars under a dry sink have no positivity guarantee from this path or from any other.

7.12.3. Numerical method

The carry is the lambda at `scalar_r` (`meshblock.cpp:634-646`). The source-free density and scalar are rebuilt by running the integrator's own stage combination on the tendency with the dry source subtracted at `rho_base` (`meshblock.cpp:637-639`), so the "source-free" state is the one the integrator would have produced and not a cell-wise approximation of it. The choice between the two ratios is made per cell on the sign of the dry source at `carry_r` (`meshblock.cpp:643-644`).

7.12.4. Code

Table 7.18.: Code map of the dry-carry guard.

what	code link	symbol	switch / condition
why addition and removal differ	`meshblock.cpp: 631-634`	`carry_dry_ source`	
the carry	`meshblock.cpp: 634-646`	`carry_dry_ source`	scalars present, a forcing writes dry mass
the source-free stage state	`meshblock.cpp: 637-639`	`rho_base`	
the zero-base guard	`meshblock.cpp: 640-642`	`remove_r`	source-free density zero
addition against removal	`meshblock.cpp: 643-644`	`carry_r`	

7.12.5. Tests

Table 7.19.: Tests of the dry-carry guard.

test	asserts	tolerance
`tests/test_dry_carry_zero_base.py`	a finite result at a zero base; the bottom scalar equals -5×10^{-4}	`1e-12`
`tests/test_stage_forcing_dry_tracer.py`	a user stage forcing that writes dry mass carries its tracers	per test

The first test is the clearest statement in the repository that this is not a positivity mechanism: it asserts a negative value to twelve digits.

7.12.6. Limits

⚠ Limits

- No positivity guarantee for passive scalars under a dry sink. The guard keeps the arithmetic finite and nothing more.

1. **A fallback, not a correction.** When the source-free density is zero the entry ratio is used instead. Whether that ratio is the physically right one in an emptied cell is not argued anywhere; it is the finite choice.

8. Diffusion, viscosity and forcing

This chapter covers every source term snapy adds to the stage increment besides the flux divergence and gravity: the forcing framework that parses, installs and sums them; user stage forcings written in TorchScript; the Coriolis force; the isotropic viscosity and heat conduction with their face coefficients at walls and their optional x_1 profiles; prescribed heating and cooling; the bottom relaxations of temperature, velocity and composition; and the sponge layers. All of them are explicit and act through the same increment, in a fixed order that does not change the result. Constant gravity and its work are Chapter 6; sedimentation is Chapter 10. Two entries record code that is present but never runs: the plume forcing and the unbuilt turbulence closure.

Table 8.1.: The schemes of this chapter. "Order" is the spatial order of the discrete operator, or the order in time for the relaxations; all are explicit.

scheme	switch	default	order	main test
Forcing framework (Section 8.1)	`forcing:` keys	none	—	`test_forcing`
User stage forcings (Section 8.2)	`set_user_stage_forcings`	none	user	`test_stage_forcing_dry_tracer_python`

scheme	switch	default	order	main test
Coriolis (Section 8.3)	`coriolis`	off	exact pointwise	`test_forcing`
Viscosity and conduction (Section 8.4)	`diffusion`	off	2 (1 at walls)	`test_diffusion`
Wall face coefficient (Section 8.5)	automatic	on at named walls	2	`test_diffusion`
x_1 coefficient profiles (Section 8.6)	`nu_scale_x1`, `kappa_scale_x1`	none	2	`test_diffusion_x1_scale`
Heating and cooling (Section 8.7)	`body-heat`, `top-cool`, `bot-heat`	off	exact pointwise	`test_forcing`
Bottom relaxation (Section 8.8)	`relax-bot-temp`, `-velo`, `-comp`	off	1 in time	`test_forcing`
Sponge layers (Section 8.9)	`top-sponge-lyr`, `bot-sponge-lyr`	off	1 in time	`test_forcing`
Plume forcing (Section 8.10)	`plume-forcing`	never installed	—	none
Turbulence (Section 8.11)	none	not built	—	none

8.1. The forcing framework: parsing, registration and the stage tendency (`forcing:`)

8.1.1. Summary

> **i** At a glance
>
> **What it does.** Turns each key of the YAML block `forcing:` into one module that adds $\Delta t\,\mathbf{S}$ to the stage increment $\Delta\mathbf{U}$, after the flux divergence. **Switch.** The presence of each key; no key, no module. Default: no forcing. **Requires.** Nothing; the modules have their own requirements (the sections below). **Order.** Each module is first order in time within a stage; the Runge–Kutta average makes the sum of the stages as accurate as the integrator. **Defining equation.** Equation 8.1.

snapy treats every source term other than the flux divergence as a *forcing*: a module that reads the stage-entry primitives and adds its contribution, multiplied by the full step Δt, to the same increment the flux divergence started. The modules run in a fixed order, they see the same primitives, and none of them sees the others' contributions, so the order does not change the result. The framework also records how much dry-air density the forcings added, so that passive tracers can be carried with it.

The YAML parser refuses unknown keys under `forcing:` and refuses the removed key `fric-heat` with a message that explains why. Twelve keys are accepted: `const-gravity` (Chapter 6), `coriolis`, `diffusion`, `body-heat`, `top-cool`, `bot-heat`, `relax-bot-comp`, `relax-bot-temp`, `relax-bot-velo`, `top-sponge-lyr`, `bot-sponge-lyr` and `plume-forcing`. The last is accepted but never installed (Section 8.10).

Aliases. chengcli/snapy#217 introduced the key list as "twelve forcing keys".

8.1.2. Derivation

The semi-discrete update of cell i is $\mathrm{d}\mathbf{U}_i/\mathrm{d}t = \mathcal{L}_i(\mathbf{U})$, with the right-hand side the flux divergence plus the sum of the source terms $\mathbf{S}^m$ of the registered modules $m = 1..M$. One stage forms the increment

$$\Delta\mathbf{U}_i = -\Delta t\,(\nabla\cdot\mathbf{F})_i + \Delta t\sum_{m=1}^{M}\mathbf{S}_i^m(\mathbf{W},T) \tag{8.1}$$

and hands it to the integrator, which forms $\mathbf{U}\leftarrow w_0^{(s)}\mathbf{U}^n + w_1^{(s)}\mathbf{U} + w_2^{(s)}\Delta\mathbf{U}$ (Chapter 7). Equation 8.1 is built in `HydroImpl::forward` (`hydro_forward.cpp:754-765`): the divergence term on interior cells, then the module loop. Each $\mathbf{S}^m$ is evaluated on the same $\mathbf{W}$ and on T computed once from it, so the sum does not depend on the order of the loop; only round-off does. Nothing here needs a derivation beyond this sentence: the framework is a sum. No executable check is attached, because there is no closed form to check; the tests below check the two properties the sum must have, linearity in Δt and registration.

The dry-air part of the forcings' increment, $\Delta\rho_{\mathrm{d}}^{\mathrm{forc}} = \Delta\mathbf{U}[\rho_{\mathrm{d}}]$ after the loop minus before it, is stored as `_forcing_dry` only when the block carries passive scalars (`hydro_forward.cpp:760-765`). The tracer carry that consumes it is Chapter 10.

8.1.3. Numerical method

Figure 8.1 shows the registration order and where the increment goes.

The loop runs in every Runge–Kutta stage, inside `HydroImpl::forward`, after the flux divergence and before the gravity-work bookkeeping and the vertical implicit correction (Chapter 6 and Chapter 7). All modules are explicit. Each module decides itself which cells it touches: `body-heat` touches interior cells only, the diffusion module adds a divergence

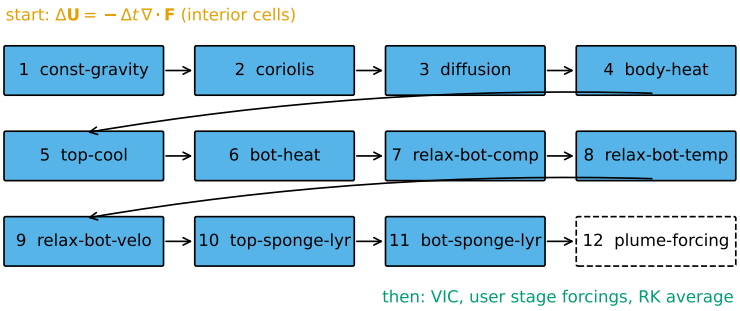


Figure 8.1.: Registration order of the native forcing modules. Each sky-blue box is a module, numbered in the order it is installed; the dashed box is accepted by the parser but never installed at the pin. The orange line is the flux-divergence increment the loop starts from, and the green line what happens to the increment after the loop.

of face fluxes, and the boundary modules (`top-cool`, `bot-heat`, the three relaxations, the two sponges) act only on a block that owns the physical x_1 boundary they name. Increments a module writes into ghost cells do not survive: the ghosts are refilled from the interior after the stage (Chapter 11).

8.1.4. Code

Table 8.2.: Code map of the forcing framework.

what	code link	symbol	switch
refuse `fric-heat`	`hydro_ options.cpp: 68-72`	`fric-heat`	removed key
the twelve keys	`hydro_ options.cpp: 73-76`	`check_keys`	`forcing:`
per-module parsing	`hydro_ options.cpp: 78-120`	`from_yaml`	each key
install in fixed order	`register_ forcing_ modules.cpp: 6-84`	`_register_ forcings_ module`	each key
register as submodules	`hydro/ hydro.cpp: 143-152`	`_register_ forcings_ module`	
increment and loop	`hydro_ forward.cpp: 754-765`	`forcings`	

what	code link	symbol	switch
dry increment for tracers	`hydro_forward.cpp:760-765`	`_forcing_dry`	scalars present
tracer carry	`meshblock.cpp:676-678`	`carry_dry_source`	scalars present

1. `HydroOptionsImpl::from_yaml` refuses `fric-heat` (`hydro_options.cpp:68-72`), checks the keys with `check_keys` (`hydro_options.cpp:73-76`) and parses each module's options with its `from_yaml` (`hydro_options.cpp:78-120`). The `plume-forcing` options are parsed only with `plume-eos`.
2. The `HydroImpl` constructor calls `_register_forcings_module` (`register_forcing_modules.cpp:6-84`) and registers each module under its key (`hydro/hydro.cpp:143-152`).
3. Each stage, `HydroImpl::forward` starts the increment from the divergence, computes T once and runs the loop (`hydro_forward.cpp:754-765`).
4. `MeshBlockImpl::advance_local` hands the dry increment to the tracers with `carry_dry_source` (`meshblock.cpp:676-678`).

8.1.5. Tests

Table 8.3.: Tests of the forcing framework.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_hydro_options.cpp` (`test_hydro_options.release`), `reject_removed_and_unknown_forcing_keys`	`fric-heat` and unknown keys are refused	exact (throws)	a parser contract
`tests/test_forcing.cpp` (`test_forcing.release`), `meshblock_registers_parent_dependent_modules`	a MeshBlock built with body heat and diffusion installs both modules	exact	a registration contract
same file, `boundary_fluxes_scale_with_timestep`	the boundary-flux modules give $\Delta\mathbf{U}(2\Delta t) = 2\Delta\mathbf{U}(\Delta t)$	`allclose` defaults	linearity in Δt; default tolerances of `torch::allclose` (rtol 10^{-5}, atol 10^{-8})

Evidence. No number is quoted in this section. The three tests above have not been run at the pin: **pending: one pinned build and ctest run at the pin** (listed in the chapter's pending list). **Coverage.** CPU

only; no CUDA arm and no multi-rank arm.

8.1.6. Limits

1. **Ghost-cell increments.** Modules that do not restrict themselves to interior cells (the sponges, `top-cool`, `bot-heat` across x_2, x_3) also write into ghost cells. The refill after the stage overwrites them. Not a defect; stated so that a reader of `du` is not surprised.
2. **Immersed solids.** The loop sees the solid placeholder state written into **W** before it by `mark_prim_solid_` (`hydro_forward.cpp:217-218`). The diffusion module is the measured case (Section 8.5); the relaxations and `body-heat` read the same state and are not measured.
3. `plume-forcing` is accepted by the parser and never installed (Section 8.10).
4. **Untested combinations.** No ctest runs two boundary modules together. See the switch coverage matrix (Chapter 12).

8.2. User stage forcings: TorchScript modules called every stage (`set_user_stage_forcings`)

8.2.1. Summary

> **i** At a glance
>
> **What it does.** Calls user TorchScript modules once per Runge–Kutta stage and adds what they return to the hydro and scalar increments. **Switch.** The C++ or Python call `MeshBlockImpl::set_user_stage_forcings(filenames)`; there is no YAML key. Default: none. **Requires.** Each module returns `Dict[str, Tensor]` with

keys `hydro_du` and/or `scalar_ds` of the increments' shapes; `scalar_ds` needs passive scalars. **Order.** As the user writes it; the returned value is an increment over the full step. **Defining equation.** Equation 8.2.

A user stage forcing is a source term written in Python, compiled with `torch.jit.script`, saved, and loaded by file name. In every stage the block hands each module one dictionary with all its buffers and the stage variables, the step Δt and the stage index, and adds what the module returns to the native increments, after the native forcings, the vertical implicit correction and the passive-scalar update, and before the Runge–Kutta average. A user dry-air density change carries the passive tracers like a native one.

The returned `hydro_du` is added as it is. The framework does not multiply it by Δt: a module that models a rate $\mathbf{S}$ returns $\Delta t\,\mathbf{S}$ itself.

8.2.2. Derivation

Let $\Delta\mathbf{U}^{\mathrm{nat}}$ be the native increment of the stage (Equation 8.1 plus the implicit correction) and $\mathbf{G}^q$ the `hydro_du` of user module $q = 1..Q$. The stage hands the integrator

$$\Delta\mathbf{U} = \Delta\mathbf{U}^{\mathrm{nat}} + \sum_{q=1}^{Q} \mathbf{G}^q(\mathbf{V}, \Delta t, s), \tag{8.2}$$

with $\mathbf{V}$ the dictionary of block buffers and stage variables. Equation 8.2 is applied to the `user_stage_forcings` in `meshblock.cpp:689-742`. Each module sees $\mathbf{V}$ as it was before any user module ran, because the dictionary is built once by `stage_forcing_variables` (`meshblock.cpp:731-738`). Through the Runge–Kutta average a user increment enters the step with the stage weight $w_2^{(s)}$, exactly as a native one; the test below asserts that

for rk1, rk2, rk3 and rk3s4. No closed form is involved, so no executable check is attached.

8.2.3. Numerical method

Figure 8.2 shows the interface. The modules are moved to the block's device when they are loaded by `set_user_stage_forcings` (`meshblock.cpp:96-109`) and are called on that device. They run once per stage of every step, and again in a redone step (Chapter 7).

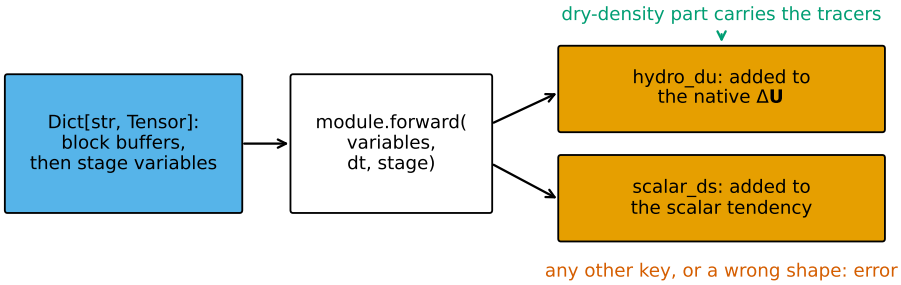


Figure 8.2.: Interface of a user stage forcing. The sky-blue box is the dictionary the block builds once per stage; the white box is the user module; the orange boxes are the two keys it may return and where each is added. The green note marks the dry-air part that carries passive tracers; the vermillion note the inputs the block refuses.

8.2.4. Code

Table 8.4.: Code map of the user stage forcings.

what	code link	symbol	switch
load, move to device	`meshblock.cpp:96-109`	`set_user_stage_forcings`	API call
input dictionary	`meshblock.cpp:38-48`	`stage_forcing_variables`	
output checks	`meshblock.cpp:50-65`	`stage_forcing_result`	
add `hydro_du`, `scalar_ds`	`meshblock.cpp:690-723`	`apply_extra_forcing`	
call each module	`meshblock.cpp:731-738`	`user_stage_forcings`	
dry carry	`meshblock.cpp:740-742`	`carry_dry_source`	scalars present

1. `set_user_stage_forcings` loads every file with `torch::jit::load`, moves it to the block's device and puts it in evaluation mode, in `set_user_stage_forcings` (`meshblock.cpp:96-109`).
2. In `advance_local`, after the native and scalar tendencies, the block builds the dictionary with `stage_forcing_variables` (`meshblock.cpp:38-48`): every named buffer first, then the stage variables, which replace a buffer of the same name.
3. It calls each of the `user_stage_forcings` with the dictionary, Δt and the stage index (`meshblock.cpp:731-738`), checks the result with `stage_forcing_result` (`meshblock.cpp:50-65`) and adds it in `apply_extra_forcing` (`meshblock.cpp:690-723`), which refuses a wrong shape, an unknown key, or `scalar_ds` without scalars.
4. The user dry-density change carries the tracers with `carry_dry_source` (`meshblock.cpp:740-742`), against the scalar increment

from before the user modules, so an explicit `scalar_ds` is not debited by a simultaneous dry removal.

8.2.5. Tests

Table 8.5.: Tests of the user stage forcings.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_jit_user_forcing.py` (`test_jit_user_forcing_python`)	a scripted module with a nested submodule and a buffer adds $0.25 \cdot 1 + 0.5$ to the dry density in one stage at $\Delta t = 0$	`torch.testing.assert_close` defaults (float64: rtol 10^{-7}, atol 10^{-7})	the returned value is added as is, not times Δt

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_stage_forcing_dry_tracer.py` (`test_stage_forcing_dry_tracer_python`)	for rk1, rk2, rk3, rk3s4 a user dry increment in stage s changes the density by $w_2^{(s)}$ times it; an explicit `scalar_ds` is not coupled to a dry removal; tracer ratios stay in $[-10^{-12}, 1 + 10^{-12}]$	10^{-12} absolute	round-off of a few operations on values of order 1

Evidence. No number is quoted. **Pending: pinned ctest run** of both tests at the pin. **Coverage.** CPU in CI. `test_stage_forcing_dry_tracer.py` takes `--device cuda`; whether a CUDA arm is registered at the pin is not checked here. No multi-rank arm.

8.2.6. Limits

1. **No time scaling.** A module that returns a rate instead of Δt times the rate is wrong by a factor Δt, with no warning.
2. **Through the limited EOS.** A module whose increment drives the state through a floor makes the step redo (chengcli/snapy#226); the redo machinery is Chapter 7 and Chapter 8.

3. **Ordering.** User modules see the stage variables, not the native increment; a module cannot react to what the native forcings did in the same stage.

8.3. Coriolis force in the coordinate frame or from a Cartesian rotation vector (`forcing/coriolis`)

8.3.1. Summary

> **i** At a glance
>
> **What it does.** Adds the Coriolis force $-2\Omega \times \rho\mathbf{v}$ to the momentum. **Switch.** `forcing/coriolis: {type, omega1, omega2, omega3, traditional}`; defaults `xyz`, 0, 0, 0, `false`. Off unless the key is present. **Forms.** `123`: Ω given in the cell's coordinate frame. `xyz`: a Cartesian rotation vector, projected onto the cell's frame (Cartesian, spherical-polar, gnomonic cubed sphere). **Order.** Exact pointwise; explicit, first order in time per stage. **Defining equation.** Equation 8.3.

The module adds the Coriolis force evaluated at the cell centre from the stage-entry density and velocity. With `type: 123` the three numbers are the components of Ω along x_1, x_2, x_3 and are used as they are. With `type: xyz` (the default) the numbers are a rotation vector in a fixed Cartesian frame, read as `omega1` $= \Omega_z$, `omega2` $= \Omega_x$, `omega3` $= \Omega_y$, and projected once, at construction, onto the local basis of every cell. On the gnomonic cubed sphere the force is formed in the local spherical basis and lowered to the covariant momentum the solver stores. The option `traditional` keeps only the radial component of Ω there and is forced on for the shallow-water equation of state.

Aliases. "Fix Coriolis (#168)" is the cubed-sphere covariant form.

8.3.2. Derivation

In an inertial frame rotating at Ω the momentum equation gains $-2\Omega\times\rho\mathbf{v}$. With $\mathbf{m}=\rho\mathbf{v}$ in an orthonormal frame, one stage adds

$$\Delta\mathbf{m} = -2\Delta t\,\Omega\times\mathbf{m}, \tag{8.3}$$

computed component by component in `CoriolisXYZImpl::forward` (`coriolis.cpp:152-192`) and, for `type: 123`, in `Coriolis123Impl::forward` (`coriolis.cpp:50-66`). Because $\mathbf{m}\cdot(\Omega\times\mathbf{m})=0$ the force does no work: the energy row is not touched and needs no term [check `src/snapy_report/ch09/coriolis_check.py`, output `coriolis_check.out`, line `[C2]`].

Projection onto spherical-polar cells. A Cartesian vector $(\Omega_x,\Omega_y,\Omega_z)$ has, at colatitude θ and azimuth φ, the components

$$\begin{aligned}
\Omega_1 &= \sin\theta\cos\varphi\,\Omega_x + \sin\theta\sin\varphi\,\Omega_y + \cos\theta\,\Omega_z,\\
\Omega_2 &= \cos\theta\cos\varphi\,\Omega_x + \cos\theta\sin\varphi\,\Omega_y - \sin\theta\,\Omega_z,\\
\Omega_3 &= -\sin\varphi\,\Omega_x + \cos\varphi\,\Omega_y,
\end{aligned} \tag{8.4}$$

the rows of the rotation from the Cartesian to the local $(\hat r,\hat\theta,\hat\varphi)$ basis, implemented as `omega1`, `omega2`, `omega3` (`coriolis.cpp:100-108`). The rotation preserves $|\Omega|$, and a rotation about z gives $(\Omega\cos\theta,-\Omega\sin\theta,0)$ [check `coriolis_check.py`, `[C3]`]. On a Cartesian grid x_1 is the vertical, so `omega1` $=\Omega_z$ is the x_1 component unchanged (`coriolis.cpp:86-93`; `[C4]`).

Cubed sphere. The panel velocity is contravariant, v^a, and the stored momentum covariant, m_a. The module maps ρv^a to the local spherical basis with the matrix of Chapter 3, forms Equation 8.3 there with Ω projected the same way, and maps the force to covariant components with `cs_apply_velocity_transform_` (`coriolis.cpp:154-180`). With

`traditional` the two horizontal components of Ω are set to zero, so the force is $-2\Omega_1 \hat{r} \times \mathbf{m}$, purely horizontal, and only the two horizontal momentum rows are updated.

What is exact. Equation 8.3 is exact at the cell centre for the cell-centre state; the only discretisation is that the cell average of $\Omega \times \mathbf{m}$ is replaced by its centre value, which is $O(h^2)$ where Ω varies across the cell (spherical-polar, cubed sphere) and exact on a Cartesian grid.

8.3.3. Numerical method

Figure 8.3 shows the projection of Equation 8.4 for a rotation about z. The force is a pointwise source at the cell centre: no stencil, no ghosts. It runs in every stage, explicit, after the diffusion module in the forcing loop (Section 8.1). The projected components are computed once at construction and stored with `register_buffer` (`coriolis.cpp:147-149`), so they move with the block between devices.

8.3.4. Code

Table 8.6.: Code map of the Coriolis force.

what	code link	symbol	switch
options, defaults	`coriolis.cpp: 19-37`	`CoriolisOptions::type, from_yaml`	`type: omega1..3, traditional`
choose the module	`register_forcing_modules.cpp: 14-23`	`CoriolisXYZ`	`type`
`123` force	`coriolis.cpp: 50-66`	`Coriolis123Impl::forward`	`type: 123`

what	code link	symbol	switch
projection, all grids	`coriolis.cpp:75-150`	`CoriolisXYZImpl::reset`	`type: xyz`
spherical-polar projection	`coriolis.cpp:100-108`	`omega1`	spherical-polar
cubed-sphere transforms	`coriolis.cpp:109-142`	`cs_velocity_transform_matrix`	gnomonic-equiangle
`xyz` force	`coriolis.cpp:152-192`	`CoriolisXYZImpl::forward`	`type: xyz`

1. `CoriolisOptionsImpl::from_yaml` checks the keys and refuses a type other than `xyz` or `123` (`coriolis.cpp:19-37`).
2. `_register_forcings_module` installs `CoriolisXYZ` for `xyz` and `Coriolis123` otherwise (`register_forcing_modules.cpp:14-23`).
3. `CoriolisXYZImpl::reset` projects Ω for the grid (`coriolis.cpp:75-150`); an unsupported grid throws. On the cubed sphere it also builds the two transform matrices and sets `traditional` (`coriolis.cpp:109-142`).
4. Each stage, `forward` adds Equation 8.3 (`coriolis.cpp:152-192`).

8.3.5. Tests

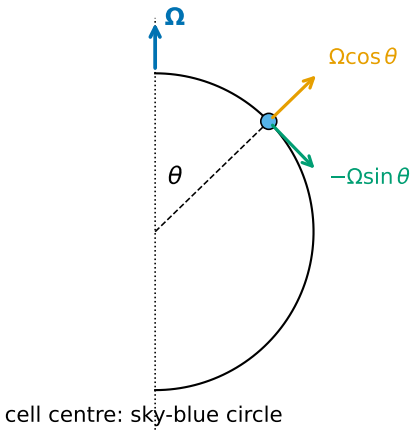


Figure 8.3.: Projection of a rotation about the z axis (blue arrow) onto the local basis of a spherical-polar cell at colatitude theta (sky-blue circle). The orange arrow is the radial component Omega cos(theta) and the bluish-green arrow the colatitude component -Omega sin(theta); the azimuthal component is zero.

Table 8.7.: Tests of the Coriolis force.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_forcing.cpp` (`test_forcing.release`), `parse_coriolis_traditional`	`traditional` parses, default `false`	exact	parser contract
same, `cubed_sphere_coriolis_uses_cartesian_rotation_vector`	on all six panels the force equals the force of the Cartesian Ω on the Cartesian velocity, mapped to the panel	rtol 10^{-12}, atol 10^{-12}	round-off of two 3×3 transforms on values of order 1
same, `cubed_sphere_coriolis_supports_traditional_approximation`	the same with only the radial component; the x_1 row is exactly unchanged	10^{-12}; exact	as above
same, `cubed_sphere_shallow_water_uses_traditional_coriolis`	shallow water forces `traditional`	10^{-12}; exact	as above

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
this section's check, `coriolis_check.py`	`[C1]`–`[C5]`	per claim	stated in the check

Evidence. The check numbers have the provenance [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 coriolis_check.py`, numpy 2.4.6, double]. The ctest cases: **pending: pinned ctest run**. **Coverage.** CPU only. No test covers the Cartesian or spherical-polar `xyz` branches or the `123` form through the solver; the check covers their formulas.

8.3.6. Limits

1. **`traditional` acts only on the cubed sphere.** It is read in the gnomonic branch only (`coriolis.cpp:115-116`); on Cartesian and spherical-polar grids, and with `type: 123`, it is parsed and ignored. Not tested.
2. **`123` in two dimensions** drops the x_3 momentum row when $n_3 = 1$ (the test `w.size(1) > 1`, `coriolis.cpp:60-62`), so the force then does work: $\mathbf{m} \cdot \Delta\mathbf{m} \neq 0$ in general [`[C5]`]. The `xyz` form keeps all three rows.
3. **Axis naming.** In `xyz`, `omega1` is Ω_z, `omega2` Ω_x, `omega3` Ω_y (`coriolis.cpp:86-88`). On a Cartesian grid this is the (x_1, x_2, x_3) order; on a sphere it is the Cartesian frame whose z is the polar axis.
4. **Cylindrical** has a projection branch, but the coordinate type is not functional (Chapter 3).

5. **No energy check through the solver** and no convergence test of the cell-centre approximation; not measured.

8.4. Isotropic viscosity and heat conduction (`forcing/diffusion`)

8.4.1. Summary

> At a glance
>
> **What it does.** Adds the Newtonian viscous stress, its work, and a Fourier heat flux as face fluxes whose divergence enters the stage increment. **Switch.** `forcing/diffusion: {nu_iso, kappa_iso, dynamic, nu_scale_x1, kappa_scale_x1}`; defaults 0, 0, `false`, none, none. Off unless the key is present and a coefficient is positive. **Requires.** A `cartesian` grid; $n_{\text{ghost}} \geq 2$ for viscosity; an EOS with a positive reference c_v for conduction. **Order.** Second order in space in the interior; first order in the kernel of the wall normal stress; explicit, with a parabolic time-step bound. **Defining equation.** Equation 8.5.

The diffusion module is snapy's only closure for sub-grid momentum and heat transport: constant isotropic coefficients, optionally shaped in x_1 (Section 8.6). With the default `dynamic: false`, `nu_iso` is a kinematic viscosity ν [m^2 s^{-1}] multiplied by the face density, and `kappa_iso` a thermal diffusivity κ [m^2 s^{-1}] multiplied by the face volumetric heat capacity ρc_v. With `dynamic: true` the two numbers are a dynamic viscosity μ [Pa s] and a conductivity, still written κ but in W m^{-1} K^{-1} (NOTATION.md §8), and no density enters. The heat flux acts on the temperature, not on the potential temperature; so a resting adiabatic column, which is neutrally stable, is heated in the interior and at the walls (Limits).

The module adds its own time-step bound, $\Delta t \le h_{\min}^2/(2n_{\dim}K)$, which the solver takes together with the hyperbolic bound before the Courant factor is applied (Chapter 7).

Aliases. The removed option `on_theta` (chengcli/snapy#253, #259; removed in #268, which closed #261) is not part of the code at the pin. Legacy keys `K` and `type` are refused.

8.4.2. Derivation

All quantities in this layer are per unit volume; h is the cell width, uniform in each direction (the module refuses every grid but Cartesian). Units: $\mathbf{v}$ [m s^{-1}], T [K], fluxes of momentum [Pa], of energy [W m^{-2}].

8.4.2.1. Continuous fluxes

The Newtonian stress for a fluid with no bulk viscosity is $\tau = \mu[\nabla\mathbf{v} + (\nabla\mathbf{v})^{\mathsf{T}} - \frac{2}{3}(\nabla\cdot\mathbf{v})\mathsf{I}]$, and the momentum and energy equations gain $\nabla\cdot\tau$ and $\nabla\cdot(\tau\cdot\mathbf{v})$ plus the divergence of the Fourier heat flux, a conductivity times ∇T. Written as fluxes through a face normal to x_a, with $\mu = \nu\rho$ and the conductivity $\kappa\rho c_v$ in the default form,

$$\begin{aligned} F_a^{(m_b)} &= -\nu\rho\,[\partial_a v_b + \partial_b v_a - \tfrac{2}{3}\delta_{ab}\nabla\cdot\mathbf{v}], \\ F_a^{(E)} &= \sum_b v_b\,F_a^{(m_b)} - \kappa\rho c_v\,\partial_a T, \end{aligned} \tag{8.5}$$

and the stage adds $-\Delta t\,\nabla\cdot\mathbf{F}$ to momentum and energy. With `dynamic: true`, $\nu\rho$ is replaced by μ and $\kappa\rho c_v$ by the conductivity κ.

8.4.2.2. Discrete fluxes

On the face $i-\frac{1}{2}$ normal to x_1, between cells $i-1$ and i, the code uses

$$\begin{aligned} \partial_1 q\big|_{i-1/2} &= \frac{q_i - q_{i-1}}{h_1}, \\ \partial_b q\big|_{i-1/2} &= \tfrac{1}{2}\Big[\frac{q_{i,j+1} - q_{i,j-1}}{2h_b} + \frac{q_{i-1,j+1} - q_{i-1,j-1}}{2h_b}\Big], \\ \overline{q}_{i-1/2} &= \tfrac{1}{2}(q_{i-1} + q_i), \end{aligned} \tag{8.6}$$

the face-normal difference, the average of the two cell-centred tangential differences, and the two-cell face average, in `face_normal_derivative`, `centered_derivative` and `face_average` (`diffusion.cpp:70-108`). The divergence $\nabla \cdot \mathbf{v}$ is formed with centred differences in every cell, `div_vel` (`diffusion.cpp:467-476`), and then face-averaged, `div_face` (`diffusion.cpp:517`). The stress row $b = a$ is $2\partial_a v_a - \frac{2}{3}\overline{\nabla \cdot \mathbf{v}}$, a row $b \neq a$ is $\partial_a v_b + \partial_b v_a$ with the second term dropped when direction b is inactive (`diffusion.cpp:516-541`). The viscous work uses the `face_average` of the velocity, and the conductive term the face-normal temperature difference times the face coefficient of Section 8.5 (`diffusion.cpp:538-561`). The tendency subtracts Δt times the `divergence` of the three flux arrays (`diffusion.cpp:566-568`).

Three properties follow and are checked against a line-by-line port of the module [check `src/snapy_report/ch09/viscosity_check.py`, output `viscosity_check.out`]:

1. **Conservation.** Momentum and total energy change only by flux differences, so their column sums telescope; on a periodic column they are zero to round-off (`[C4]`).
2. **Dissipation.** The kinetic-energy part of the update is removed and the same amount is added to the internal energy through the work term; for a random shear the net internal-energy change is positive (`[C5]`).

3. **Accuracy.** For a transverse velocity the tendency is $\nu\rho$ times the three-point Laplacian to round-off, which decays a mode of wavelength λ_w at $4\nu \sin^2 \vartheta/h^2 = \nu(2\pi/\lambda_w)^2[1 + O(\vartheta^2)]$, with $\vartheta = \pi h/\lambda_w$, second order (`[C2]`). The normal-stress row acts with the effective coefficient $\frac{4}{3}\nu$, again second order; its stencil is wider because the divergence is a centred difference averaged to the face (`[C3]`).

8.4.2.3. Time-step bound

Forward Euler on the n_{dim}-dimensional three-point Laplacian with coefficient K amplifies the checkerboard mode by $G = 1 - 4n_{\mathrm{dim}}K\Delta t/h^2$, so $|G| \le 1$ requires

$$\Delta t \le \frac{h_{\mathrm{min}}^2}{2n_{\mathrm{dim}}K}, \tag{8.7}$$

with $K = \max(\nu\varsigma_{\mathrm{max}}^{\nu}, \kappa\varsigma_{\mathrm{max}}^{\kappa})$ in the default form (ς_{max} the largest value of an x_1 profile of Section 8.6, 1 without one) and $K = \max(\mu/\rho_{\mathrm{min}}, \kappa/(\rho c_v)_{\mathrm{min}})$ with `dynamic: true`, in `max_time_step` (`diffusion.cpp:572-619`). At the bound $|G| = 1$ exactly (`[C6]`). The solver takes the minimum of Equation 8.7 and the hyperbolic bound through `pdiffusion` (`hydro/hydro.cpp:396-398`) and then multiplies by the Courant factor `cfl` (default 0.9; `meshblock.cpp:528`), so the diffusive step runs at `cfl` times its limit.

8.4.2.4. What `kappa_iso` diffuses

The energy row receives $\nabla\cdot(\kappa\rho c_v\nabla T)$. At fixed density the internal energy is $\rho c_v T$, so $\partial_t T = \kappa\nabla^2 T$: `kappa_iso` is the thermal diffusivity of T. At fixed pressure the heat goes into enthalpy, $\rho c_p \partial_t T = \kappa\rho c_v \nabla^2 T$, and T diffuses at κ/γ (`[C7]`, 0.714286 for $\gamma = 1.4$). A run sits between the two, depending on how fast the pressure adjusts.

On a resting adiabat, $\partial_z T = -g/c_p$ and $\rho \propto T^{c_v/R}$; the flux is uniform in T-gradient but not in ρc_v, and the interior heats at

$$\partial_t T = \frac{\kappa g^2 c_v}{R\, c_p^2\, T}, \tag{8.8}$$

7.93×10^{-7} K s^{-1} per unit κ at $T = 300$ K for dry air ($R = 287$, $c_p = 1004.5$ J kg^{-1} K^{-1}, $g = 9.8$ m s^{-2}). The discrete operator reproduces Equation 8.8 at second order (`[C8]`: relative error 1.85×10^{-6}, 4.63×10^{-7}, 1.16×10^{-7} at $n_1 = 64$, 128, 256).

Exact and truncated. Conservation of momentum and total energy is exact. The interior fluxes are second order; at a reflecting x_1 wall the coefficient is second order (Section 8.5) but the face-averaged divergence in the normal stress is first order.

8.4.3. Numerical method

Figure 8.4 shows the stencil of one x_1 face in two dimensions.

Each face flux of Equation 8.6 reads the two cells across the face and, for the cross terms and the divergence, their neighbours in the tangential directions: one ghost layer for conduction, two for viscosity (the face average of a centred difference reaches two cells), hence the setup check $n_{\mathrm{ghost}} \geq 2$. Ghosts are filled by the boundary functions and the block exchange before the stage (Chapter 11, Chapter 14). Faces at a reflecting x_1 wall take the one-sided coefficient of Section 8.5; every other face, including seams between blocks, takes the two-cell average. The module runs in every Runge–Kutta stage, explicit, third in the forcing loop (Section 8.1). Evidence for the orders: interior second order from `[C2]`, `[C3]` and `[C8]` (fitted slope 2.00 over three resolutions); the wall coefficient second order from Section 8.5; the order of the full operator at a wall through the solver is **not measured**.

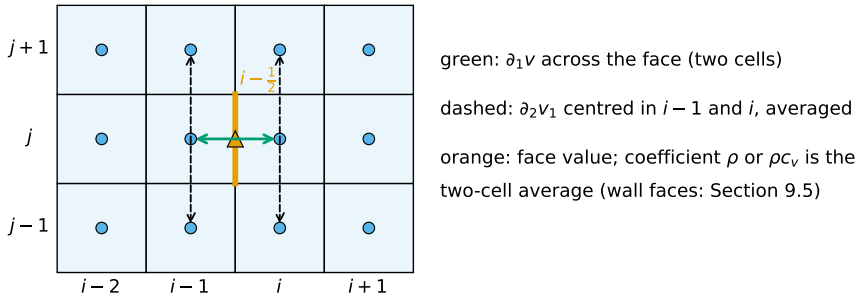


Figure 8.4.: Stencil of the diffusive flux through the x1 face i-1/2 (orange triangle on the orange face) on a 2-D Cartesian grid. Sky-blue circles are cell-centre primitives. The bluish-green arrow is the face-normal difference across the face; the dashed black arrows are the centred x2 differences in cells i-1 and i whose average gives the cross term of the shear stress. The face coefficient is the two-cell average except at a wall.

8.4.4. Code

Table 8.8.: Code map of the isotropic viscosity and heat conduction.

what	code link	symbol	switch
options, refusals	`diffusion.cpp:246-325`	`DiffusionOptionsImpl::from_yaml`	unknown keys
setup checks	`diffusion.cpp:334-345`	`DiffusionImpl::reset`	grid, `nghost`, c_v
stencil helpers	`diffusion.cpp:70-108`	`face_normal_derivative`	
cell divergence	`diffusion.cpp:467-476`	`div_vel`	`nu_iso` > 0
ρc_v	`diffusion.cpp:477-481`	`rho_cv`	`kappa_iso` > 0, not dynamic
stress and work	`diffusion.cpp:516-542`	`momentum_flux`	`nu_iso` > 0
heat flux	`diffusion.cpp:544-562`	`dtdn`	`kappa_iso` > 0
tendency	`diffusion.cpp:566-568`	`divergence`	
time-step bound	`diffusion.cpp:572-619`	`DiffusionImpl::max_time_step`	
bound used by the solver	`hydro/hydro.cpp:396-398`	`max_time_step`	

1. `DiffusionOptionsImpl::from_yaml` refuses `K` and `type`, checks the keys, and requires finite non-negative coefficients and a literal `true`/`false` for `dynamic` (`diffusion.cpp:246-325`).
2. `DiffusionImpl::reset` refuses a non-Cartesian grid when a coefficient is positive, viscosity with fewer than two ghosts, and conduction without a positive reference c_v (`diffusion.cpp:334-345`); it then

builds the x_1 profiles (Section 8.6).

3. `forward` forms the cell divergence `div_vel` (`diffusion.cpp:467-476`) and `rho_cv` (`diffusion.cpp:477-481`), then, per active direction, the face coefficient, the stress rows and their work, `momentum_flux` (`diffusion.cpp:516-542`), and the heat flux from `dtdn` (`diffusion.cpp:544-562`).
4. It subtracts Δt times the `divergence` of the three flux arrays (`diffusion.cpp:566-568`).
5. `max_time_step` returns Equation 8.7 (`diffusion.cpp:572-619`); it throws on a non-finite coefficient or a non-positive bound.

8.4.5. Tests

Table 8.9.: Tests of the isotropic viscosity and heat conduction.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_diffusion.cpp` (`test_diffusion.release`)	legacy keys, non-bool `dynamic`, negative or non-numeric coefficients, curved grids and conduction without c_v are refused	exact (throws)	parser and setup contracts
five `diffusion_options.*` cases			
same, `uniform_state_has_zero_tendency`	a uniform state gives zero tendency	rtol, atol 10^{-6}	float32 arm

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `transverse_velocity_uses_viscous_laplacian`	$v_2 = x_1^2$ gives the uniform $\Delta m_2 = 0.1$ of $\nu\rho\,\partial_1^2 v_2\,\Delta t$ in every interior cell	10^{-5}	float32 arm
same, `temperature_uses_conductive_laplacian`	$T = x_1^2$ gives $\Delta E = 0.05\,c_v$ in an interior cell; mass and momentum untouched	10^{-3} absolute	float32 on a value of order 10^2
same, `viscous_sine_mode_matches_analytic_decay`	100 forward-Euler steps at $0.1h^2/\nu$ decay $\sin x$ as $e^{-\nu t}$	3×10^{-4}	the $O(\vartheta^2)$ modified-wavenumber error at $n_1 = 64$ plus float32
same, `timestep_uses_largest_diffusivity`, `timestep_rejects_*`	Equation 8.7 with the larger coefficient; non-finite or non-positive bounds throw	10^{-12}	closed form in double
same, `dynamic_coefficients_carry_no_density`	with `dynamic: true` the fluxes are μ and the conductivity κ times the gradients; the bound uses $\rho_{\min}$	10^{-12} relative	closed form in double

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_diffusion_moist.cpp` (`test_diffusion_moist.release`), `moist_conduction_uses_local_mixture_specific_heat`	the conductive coefficient uses the local mixture c_v	10^{-5}; 10^{-3}	float32 arm
same, `dynamic_conduction_timestep_uses_local_volumetric_heat_capacity`	the dynamic bound uses $(\rho c_v)_{\min}$	10^{-12} relative	closed form
this section's check, `viscosity_check.py`	`[C1]`–`[C8]`	per claim	stated in the check

Evidence. Check numbers: [sha: the formulas of snapy@e894700ff7aee3 0b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 viscosity_check.py`, numpy 2.4.6, double]. ctest cases: **pending: pinned ctest run**. A data figure of the sine-mode decay against $e^{-\nu t}$ from the ctest is **pending** the same run. **Coverage.** The `DeviceTest` cases run on CPU in float32 and float64 and on CUDA in both when a device is present (`tests/device_testing.hpp`). No multi-

rank test of the module.

8.4.6. Limits

> **Limits**
>
> Cartesian grids only. `kappa_iso` conducts on T, so a resting adiabat drifts. Immersed solids give a wrong face coefficient by a factor of about 500 (Section 8.5). No species or tracer diffusion.

1. **Conduction on T.** A resting adiabatic column heats in the interior at Equation 8.8, and its wall cells cool at the bottom and warm at the top. chengcli/snapy#252 measured the bottom and top cells at -8.60×10^{-1} and $+8.83 \times 10^{-1}$ K after 1000 steps at $\kappa = 75$ m^2 s^{-1} and the interior within 9 % of Equation 8.8 [sha: snapy@69e008452b373b9697c1668eebbd1ea51e7e41e4 on chengcli/snapy main, which is not a pin of this report; deck: `tests/test_kappa_adiabatic_rest.py` from a red-test commit recorded in the issue and not in any listed repository; run: CPU, recorded in the issue]. The remedy `on_theta` was removed in #268; the drift is not addressed at the pin. Re-measuring needs that deck committed at the pin: **pending**.
2. **Immersed solids** are read as fluid; see Section 8.5.
3. **Wall normal stress.** At a reflecting x_1 wall the face-averaged divergence takes the cell value, an $O(h)$ error in the kernel of the normal stress that the wall coefficient does not remove (`sources/canoe__diffusion_ghost_TECH_REPORT.md` §5, "What the fix deliberately does not touch").
4. **Block edges.** Viscosity created v_2 beside x_2 block edges at an x_1 wall row (chengcli/snapy#264), fixed by the corner refresh of #265 (Chapter 11). Not re-measured at the pin.
5. **Python setters** on `DiffusionOptions` bypass the YAML checks of step 1 of the walk-through (chengcli/snapy#216).

6. **Time step.** The bound uses the largest coefficient everywhere and the smallest cell width in any direction; the one-sided wall coefficient can exceed the cell value by about 17 % at $h/H = 0.35$ (`sources/canoe__diffusion_ghost_TECH_REPORT.md` §7, off-pin) and is not included. The default `cfl` of 0.9 covers it; not measured at the pin.
7. **Untested combinations.** Diffusion with an implicit scheme, with sedimentation, or on several ranks is not run by any ctest listed here. See the switch coverage matrix (Chapter 12).

8.5. Diffusion face coefficient at a wall: one-sided extrapolation instead of the ghost (automatic)

8.5.1. Summary

> At a glance
>
> **What it does.** On an x_1 face that is a reflecting wall, takes the diffusion coefficient (ρ, ρc_v, or either times an x_1 profile) by linear extrapolation from the two nearest interior cells instead of averaging the wall cell with its ghost. **Switch.** None: automatic when the x_1 boundary function is named `reflecting_inner`/`_outer` or `fixed_temperature_inner`/`_outer`. Every other face keeps the two-cell average. **Order.** Second order at the wall face, against first order for the mirror average. **Defining equation.** Equation 8.9.

A reflecting wall fills its ghost cells with the mirror image of the interior. That is the right state for the Riemann solver, but the average of a cell and its mirror is the cell's own value, half a cell away from the wall, so the face coefficient of a stratified column is off by a factor $e^{\mp h/2H}$ at the walls, an $O(h)$ error. The diffusion module therefore extrapolates the coefficient from the two interior cells next to the wall, which reads no ghost at all and is exact for a linear profile.

The fix is restricted to x_1 and to a whitelist of boundary names. An x_2 or x_3 reflecting face is as likely to be a symmetry plane, where the mirror is exact and the average is the better estimate; and `reflecting` is the default for every face a configuration does not name, so widening the rule would change runs that did not ask for it. Periodic faces, outflow, custom and solid boundary functions keep the average.

Aliases. "Defect B" of `sources/canoe__diffusion_ghost_TECH_REPORT.md`; its "Defect A", the immersed-solid placeholder state, is not fixed (Limits).

8.5.2. Derivation

Let cell 0 touch the wall and cell 1 be its interior neighbour, with widths h_0, h_1. The wall face is $h_0/2$ from the centre of cell 0 and the two centres are $(h_0 + h_1)/2$ apart, so the linear interpolant through the two centre values, evaluated at the face, is

$$q_f = (1 + t)\, q_0 - t\, q_1, \qquad t = \frac{h_0}{h_0 + h_1}, \tag{8.9}$$

the same expression at both walls, because the offset to the face and the direction to the second cell change sign together. Equation 8.9 is `extrapolate_to_wall` (`diffusion.cpp:114-136`). It is exact for a linear q on any widths [check `src/snapy_report/ch09/wallcoef_check.py`, output `wallcoef_check.out`, `[C1]`, max error 8.9×10^{-16} over 100 random stretched columns] and so second order for a smooth one. If $q_f \le 0$ the code returns q_0, which also reads no ghost (`[C3]`).

The mirror average, by contrast, returns q_0 itself. For $\rho = \rho_0 e^{-x_1/H}$ with the lower wall at $x_1 = 0$ the ratio of the mirror average to the wall value is exactly $e^{-h/2H}$ at the lower wall and $e^{+h/2H}$ at the upper one, an $O(h)$ error; the extrapolation error falls at second order (`[C2]`: fitted orders 0.994 and 1.985 for h/H from 1/32 to 1/64).

Which fluxes the coefficient matters for follows from the mirror parities. Density and temperature are mirrored even, so the face-normal temperature difference at a reflecting wall is exactly zero and the conductive wall flux vanishes whatever the coefficient. The normal velocity is mirrored odd, so $\partial_1 v_1 = 2v_{1,0}/h_0$ at the wall (`[C4]`), and the viscous normal stress at the wall carries the coefficient. The fix changes the viscous wall flux and, with an x_1 profile on conduction, nothing else at a reflecting wall.

Exact and truncated. The coefficient is exact for linear profiles and $O(h^2)$ otherwise; the normal-stress kernel at the wall stays $O(h)$ (Section 8.4, Limits).

8.5.3. Numerical method

Figure 8.5 compares the two estimates at the lower wall.

The extrapolation is applied to the first and last x_1 face of a block when that face is a whitelisted wall and the block has at least three x_1 faces (two interior cells); with fewer the average is kept, in `face_coefficient` (`diffusion.cpp:142-159`). A seam between two blocks has no boundary function and so is never a wall. The same rule applies to the product of a coefficient and an x_1 profile, which is extrapolated as one quantity (Section 8.6). It runs wherever the module computes a coefficient, in every stage.

8.5.4. Code

Table 8.10.: Code map of the wall face coefficient.

what	code link	symbol	switch
Equation 8.9 and the fallback	`diffusion.cpp: 114-136`	`extrapolate_ to_wall`	

what	code link	symbol	switch
average, or wall faces	`diffusion.cpp: 142-159`	`face_ coefficient`	
x_1 only	`diffusion.cpp: 496-502`	`is_wall_ boundary`	x_1 faces
the whitelist	`meshblock_ options.cpp: 243-272`	`is_wall_ boundary`	boundary names
unnamed function warns	`meshblock_ options.cpp: 250-258`	`TORCH_WARN_ ONCE`	
installed-function test	`meshblock_ options.cpp: 229-234`	`is_physical_ boundary`	

1. For each active direction `DiffusionImpl::forward` asks `is_wall_ boundary` for the lower and upper x_1 face only (`diffusion.cpp: 496-502`).
2. `is_wall_boundary` returns true for an installed function whose recorded name is on the whitelist (`meshblock_options.cpp: 243-272`); a function with no recorded name warns once with `TORCH_WARN_ONCE` and returns false (`meshblock_options.cpp: 250-258`).
3. `face_coefficient` averages every face and overwrites the wall faces with `extrapolate_to_wall` (`diffusion.cpp:142-159`, `diffusion.cpp:114-136`).

8.5.5. Tests

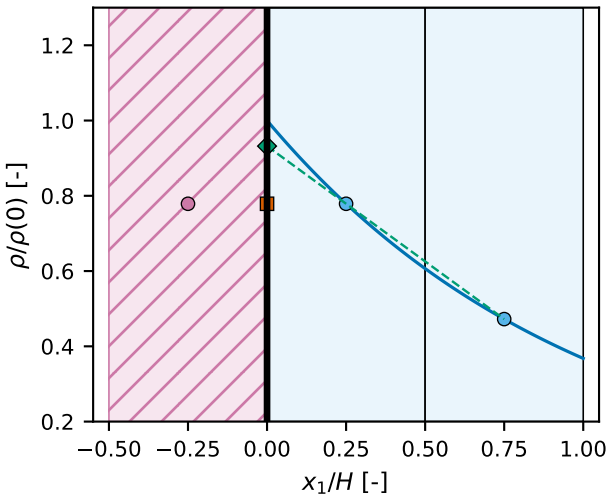


Figure 8.5.: The diffusion coefficient at a reflecting x1 wall (thick black line) for rho = exp(-x1/H) with h = H/2. The blue curve is the exact profile, the sky-blue circles the two interior cell values and the hatched reddish-purple cell the mirror ghost (reddish-purple circle). Their average is the vermillion square on the wall, which equals the wall cell's value; the bluish-green dashed line is the extrapolation from the two interior cells, which lands on the bluish-green diamond near the exact wall value.

Table 8.11.: Tests of the wall face coefficient.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_diffusion.cpp` (`test_diffusion.release`), `wall_face_coefficient_reads_no_ghost`	on a linear density a uniform $\mathrm{d}T/\mathrm{d}n$ gives the same tendency in every interior cell, wall cells included	10^{-5}	float32 arm
same, `wall_flux_does_not_depend_on_the_ghost_density`	changing the ghost density leaves the wall flux unchanged; the wall flux is not zero	10^{-10}; $> 10^{-6}$	float64 round-off; non-vacuity
same, `x2_wall_is_not_one_sided`	an x_2 reflecting wall keeps the average	10^{-4} relative	the half-slope signature of the average
same, `periodic_x1_face_is_not_extrapolated`	a periodic x_1 face keeps the average	10^{-4} relative	as above
same, `wall_names_are_an_exact_whitelist`	only the four names count as walls	exact	contract

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
this section's check, `wallcoef_check.py`	`[C1]`–`[C5]`	per claim	stated in the check

Evidence. Check numbers: [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 wallcoef_check.py`, numpy 2.4.6, double]. The source report measured the wall coefficient off by −14.72 % and +15.52 % at $h/H = 0.35$ before the fix [`sources/canoe__diffusion_ghost_TECH_REPORT.md` §4; sha snapy@43e04fda904d7401f1ee204c73b34460f09ff5fe, not a pin of this report; deck and run as described there]; re-measuring at the pin needs a committed deck: **pending**. ctest cases: **pending: pinned ctest run**. **Coverage.** CPU float32/float64 and CUDA when present for the `DeviceTest` cases; no multi-rank test; no stretched grid, because snapy has none, so the general weight $t \neq \frac{1}{2}$ is covered only by `[C1]`.

8.5.6. Limits

> **Limits**
>
> Without a `boundary-condition.external` block no boundary functions exist and the rule never fires. Immersed solids are read as fluid by the diffusion module, with a face heat capacity about 500 times too large.

1. **Immersed solids (not fixed).** The solid placeholder state, $\rho = 10^3$ and $p = 10^9$ by default, is written into **W** by `mark_prim_solid_`

before the forcings run (`hydro_forward.cpp:217-218`). A fluid face next to a solid then has ρc_v 500.5 times the fluid value and a temperature jump of a factor 10 (`[C5]`). The source measured $+1.126125 \times 10^3$ of energy per stage in the two flanking cells of a uniform column [`sources/canoe__diffusion_ghost_TECH_REPORT.md` §3; sha snapy@43e04fda904d7401f1ee204c73b34460f09ff5fe, not a pin of this report]. No configuration in the tree combines a solid mask with diffusion; the path is deliberately left as it was (same report §3a). Not tested at the pin.

2. **Dormant without named boundaries.** No boundary block, or a function installed from Python with no `name`, means no wall: the average is used, with a one-time warning in the second case.
3. `fixed_temperature_*` is on the whitelist but no such boundary function is registered in the tree (chengcli/snapy#216), so that entry changes nothing at the pin.
4. **Positivity fallback** is element-wise and silent; a wall face can be second order in some columns and first order in others. Not observed to fire; not measured.
5. **Block too short.** With fewer than two interior x_1 cells, `nface` < 3, the average is kept (`diffusion.cpp:147-148`).

8.6. x1 profiles of the diffusion coefficients and the mean-of-products face coefficient (`nu_scale_x1`, `kappa_scale_x1`)

8.6.1. Summary

> **i** At a glance
>
> **What it does.** Multiplies the kinematic `nu_iso` and/or `kappa_iso` by a positive profile in x_1, and takes the x_1 face coefficient as the mean of the two cells' products. **Switch.** YAML `forcing/diffusion/nu_scale_x1: {x1: [...], scale: [...]}` and `kappa_scale_x1`; Python `nu_scale_x1` / `kappa_scale_x1` (per cell) or `..._table`. Default: no profile. Refused with `dynamic: true`. **Order.** Second order in the face flux; exact for a coefficient that is constant in x_1. **Defining equation.** Equation 8.10.

Convection decks often want a dynamic viscosity or conductivity that is constant in height over a strongly stratified column. In the kinematic form the module multiplies by the face density, so such a deck sets ν and an x_1 profile $\varsigma(x_1) \propto 1/\rho_0(x_1)$ whose product with ρ is constant. (The report writes the profile ς, because s is the centroid slope of NOTATION.md; see the local-symbol note.) That balance holds on the faces only if the face coefficient is the mean of the cells' products, $\frac{1}{2}(\varsigma_{i-1}q_{i-1} + \varsigma_i q_i)$, and not the product of the means: the latter adds a second-order, always positive excess that makes a conductive or viscous equilibrium generate a tendency, largest at the top of the column.

A profile is given either as a table of knots in x_1, linear between knots and held beyond them, or, from Python, as one value per x_1 cell centre of a block, ghosts included. It is frozen when the block is built.

Aliases. "K" in the gravity-work round's review notes (`sources/team-notes/`); the commit snapy@cbef588c1264dce86fea537f19479a4646e6b575, "Take the x1 diffusion face coefficient as the mean of the cells' products".

8.6.2. Derivation

The derivation is `docs/derivations/diffusion-face-coefficient.md@e894700ff7aee30b52882e5202b16461413780b0` §1, rewritten here in the report's notation and checked against the code.

On the x_1 face $i-\frac{1}{2}$ between cells $i-1$ and i the coefficient is a product $c=\varsigma q$ of the profile and a cell field, $q=\rho$ for viscosity or $q=\rho c_v$ for conduction. The two candidate face values are

$$\begin{aligned} c^{\mathrm{MP}}_{i-1/2} &= \tfrac{1}{2}(\varsigma_{i-1}q_{i-1}+\varsigma_i q_i), \\ c^{\mathrm{PM}}_{i-1/2} &= \tfrac{1}{2}(\varsigma_{i-1}+\varsigma_i)\cdot\tfrac{1}{2}(q_{i-1}+q_i) \\ &= c^{\mathrm{MP}}_{i-1/2}-\tfrac{1}{4}(\varsigma_i-\varsigma_{i-1})(q_i-q_{i-1}). \end{aligned} \tag{8.10}$$

The second line is an identity [check `src/snapy_report/ch09/x1profile_check.py`, output `x1profile_check.out`, `[C1]`]. The code uses c^{MP} on x_1 faces: `face_scaled_coefficient` multiplies the cell field by the profile and hands the product to `face_coefficient` (`diffusion.cpp:168-175`), which also extrapolates the product at a wall (Section 8.5).

A constant product. Let $\varsigma q=C$ in every cell. Then $c^{\mathrm{MP}}=C$ on every face, while

$$c^{\mathrm{PM}}_{i-1/2}=C\Big[1+\frac{(q_i-q_{i-1})^2}{4q_{i-1}q_i}\Big]=C\big[1+\tfrac{1}{4}(h\,\partial_1 q/q)^2+O(h^3)\big].$$

For a linear temperature, whose exact flux F_0 is uniform, the tendency of cell i is $-(F_0/h)(e_{i+1/2}-e_{i-1/2})$ with e the bracketed excess, $O(h^2)$ in the

interior. In the cell next to a wall one face is the extrapolated product, which is exact, so the tendency there is $O(h)$. The port reproduces both: c^{MP} gives $|\dot{E}| < 10^{-12} F_0/h$ at $n_1 = 16$ to 128, and c^{PM} converges at orders 1.40, 1.66, 1.81 in the interior and 0.71, 0.84, 0.92 over all cells (the maximum sits in a wall cell) (`[C2]`).

A smooth product. c^{MP} is the trapezoidal average of c across the face and differs from $c(x_{1,i-1/2})$ by $O(h^2)$; c^{PM} differs from it by $\frac{1}{4}\varsigma' q' h^2$. Both give a second-order face flux (`[C3]`: orders 1.98 and 1.98 from $n_1 = 128$ to 256). The two forms differ only where a balance should be exact.

x_2 and x_3 faces. Both cells of such a face share one x_1, so the two cells' ς are equal and the two forms are equal in exact arithmetic, but not bit for bit (`[C5]`: relative difference 2.1×10^{-16}). The code keeps the product of means there, `face_average` times `scale` (`diffusion.cpp:176-193`), so a run without an x_1 split is unchanged bit for bit by the change.

The table. `profile_from_table` evaluates a table of knots $(x^{\mathrm{k}}_\ell, \varsigma_\ell)$ at the cell centres: linear between knots, the end value beyond the ends, the knot value exactly at a knot (`diffusion.cpp:216-225`; `[C4]`). Because it is a function of x_1, every block of an x_1 split computes the same values for the same cells.

Exact and truncated. Exact: zero tendency for a constant product, and decomposition invariance of the table. Truncated: the face flux at $O(h^2)$.

8.6.3. Numerical method

Figure 8.6 shows the tendency both forms give on the constant-product column.

The profile is stored per x_1 cell centre, ghosts included, in float64 on the CPU at construction, by `profile` (`diffusion.cpp:359-402`), and copied once to the state's device and dtype on first use, by `on_state` (`diffusion.cpp:442-450`). A profile of ones gives the coefficient without

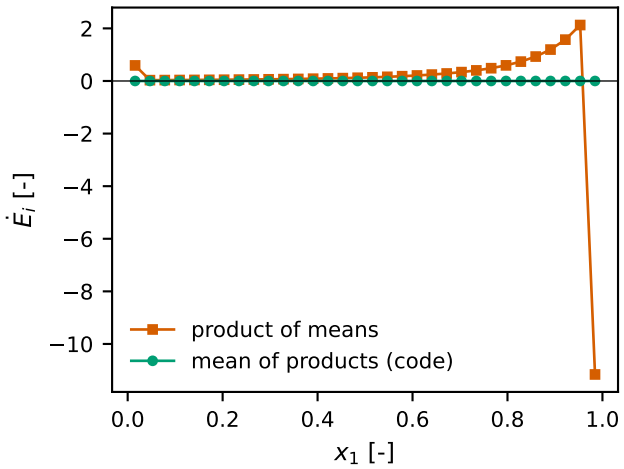


Figure 8.6.: Conductive energy tendency on a column with a constant coefficient s q = 1 over rho = (1.25 - x1)^1.5 and a linear temperature, 32 cells, from the port of the module. Vermillion squares: the product of the face means; bluish-green circles: the mean of the cells' products, which the code uses and which is zero in every cell. The product of means heats the column, most strongly near the top, and cools the top wall cell.

a profile bit for bit, because $q \times 1 = q$. The time-step bound uses the largest profile value (Equation 8.7). The stencil is that of Section 8.4; only the face coefficient changes.

8.6.4. Code

Table 8.12.: Code map of the x_1 coefficient profiles.

what	code link	symbol	switch
YAML table parsing	`diffusion.cpp: 289-323`	`take_table`	`nu_scale_x1`, `kappa_scale_x1`
table checks	`diffusion.cpp: 198-212`	`checked_table`	
table at the centres	`diffusion.cpp: 216-225`	`profile_from_table`	
build, refusals	`diffusion.cpp: 359-413`	`profile`	per cell or table
frozen after build	`diffusion.cpp: 415-434`	`check_profiles`	
Equation 8.10 on x_1 faces	`diffusion.cpp: 168-175`	`face_scaled_coefficient`	
x_2/x_3 faces	`diffusion.cpp: 176-193`	`face_average`	
use in the fluxes	`diffusion.cpp: 504-561`	`face_scaled_coefficient`	profile given

1. `from_yaml` reads a table `{x1, scale}` and checks it with `checked_table`: at least two knots, finite, strictly increasing x_1, positive values (`diffusion.cpp:289-323`, `diffusion.cpp:198-212`).
2. `reset` refuses a profile given both ways, any profile with `dynamic: true`, a per-cell profile when x_1 is split into several blocks, of the

wrong length, not float64, or not positive, and an inference tensor (`diffusion.cpp:359-413`).

3. At every `forward` and `max_time_step`, a profile set, replaced or changed after construction is refused, by identity, version counter and value, in `check_profiles` (`diffusion.cpp:415-434`).
4. `forward` uses `face_scaled_coefficient` in place of `face_coefficient` when a profile is set (`diffusion.cpp:504-561`).

8.6.5. Tests

Table 8.13.: Tests of the x_1 coefficient profiles.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_diffusion_x1_scale.cpp` (`test_diffusion_x1_scale.release`), `unity_profile_is_bitwise_no_profile`	a profile of ones changes no bit	exact	$q \times 1 = q$

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `viscous_sine_mode_decays_at_the_scaled_rate`, `conductive_sine_mode_decays_at_the_scaled_rate`	a constant profile ς scales the decay rate by ς	`allclose` as in the unscaled test	modified-wavenumber error plus float32
same, `linear_profile_gives_the_analytic_tendency`	a linear profile gives the closed-form tendency	10^{-12} relative	closed form in double
same, `constant_dynamic_coefficient_column_has_no_tendency` (+`_cuda`)	constant product: $\lvert\dot{u}\rvert < 10^{-12}$ of the flux scale at $n_1 = 16$, 64, conduction and shear	10^{-12}	round-off; fails with the product of means
same, `smooth_profile_flux_converges_at_second_order` (+`_cuda`)	face fluxes recovered from the tendency converge at order > 1.85 for $n_1 = 32$, 64, 128	1.85	second order less a margin

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `table_profile_is_independent_of_the_x1_split`	the same table on one and on two x_1 blocks gives the same step	per test	decomposition invariance

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `yaml_table_*`, `reset_refuses_bad_profiles`, `cells_profile_is_refused_when_x1_is_split`, `profile_change_after_build_is_refused`, `inference_tensor_profile_is_refused`, `profile_change_is_refused_by_max_time_step`, `profile_write_through_outside_storage_is_refused`, `timestep_uses_the_largest_scaled_coefficient`	parsing, refusals, freezing, time step	exact or 10^{-12}	contracts

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_diffusion_x1_scale.py` (`test_diffusion_x1_scale_python`)	a profile of ones from Python and from YAML gives the step without a profile bit for bit; a YAML table on the centres equals the per-cell tensor bit for bit; a non-uniform profile changes the step	exact	contracts
this section's check, `x1profile_check.py`	`[C1]`–`[C5]`	per claim	stated in the check

Evidence. Check numbers: [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 x1profile_check.py`, numpy 2.4.6, double]; they reproduce the numbers of `docs/derivations/diffusion-face-coefficient.md@e894700ff7aee30b52882e5202b16461413780b0` §3. The fitted orders of the solver's own face fluxes are printed by no test: a data figure of the solver's convergence at $n_1 = 32$, 64, 128 needs the ctest to print them and a pinned run, **pending**. ctest cases: **pending: pinned ctest run**. **Coverage.** CPU; the two `_cuda` cases on a device

(a CUDA-arms note in `sources/team-notes/` reports them passing on an RTX 5090 at a development commit that is not a pin of this report). No multi-rank arm.

8.6.6. Limits

1. **Per-cell profiles on one x_1 block only.** A per-cell tensor covers one block's cells, so it is refused when x_1 is split, `nb1` > 1 (`diffusion.cpp:376-383`); use a table.
2. **Kinematic form only.** A profile is refused with `dynamic: true`, which has no density to balance.
3. **Frozen.** A profile changed after construction is refused, not used.
4. **The table is linear between knots**; a profile with curvature needs enough knots. Not measured.
5. **The wall cell keeps the $O(h)$ error** of the normal-stress kernel (Section 8.4).

8.7. Body heating, top cooling and bottom heating (`body-heat`, `top-cool`, `bot-heat`)

8.7.1. Summary

> **i** At a glance
>
> **What it does.** `body-heat` heats every interior cell whose pressure lies in a band at a fixed rate of temperature change at constant volume; `top-cool` and `bot-heat` remove or add a fixed energy flux, spread evenly over the top or bottom `depth` cells. **Switch.** `forcing/ body-heat: {dTdt, pmin, pmax}`, defaults 0, 0, 1 [K s^{-1}, Pa, Pa]; `top-cool: {flux, depth}`, `flux` ≤ 0, default 0 and 1; `bot-heat:`

`{flux, depth}`, `flux` ≥ 0, default 0 and 1 [W m^{-2}, cells]. Each off unless its key is present. **Order.** Exact sources; explicit, first order in time per stage. **Defining equation.** Equation 8.11 and Equation 8.12.

These three modules are prescribed heat sources for idealised convection. They touch only the total energy, so they change the temperature and, through it, the pressure; mass and momentum are untouched. `body-heat` is a volumetric rate; `top-cool` and `bot-heat` stand for a radiative or surface flux at the top or bottom of the domain, applied only by the block that owns that physical x_1 boundary.

The default `pmax` is 1 Pa: a `body-heat` block that does not set `pmin` and `pmax` heats only cells with $0 \leq p \leq 1$ Pa, which in most decks is none.

8.7.2. Derivation

Body heating. At constant volume the internal energy changes by $\rho c_v \, \mathrm{d}T$, so a heating rate $\dot{T}$ adds, in one stage and in every interior cell with $p_{\min} \leq p_i \leq p_{\max}$,

$$\Delta E_i = \Delta t \, \dot{T} \, \rho_i c_{v,i}, \tag{8.11}$$

with c_v the mixture value of the equation of state at the cell's state, in `BodyHeatImpl::forward` (`body_heat.cpp:43-55`). For a constant c_v at fixed density this raises T by exactly $\dot{T}\Delta t$ [check `src/snapy_report/ch09/heating_check.py`, output `heating_check.out`, `[C3]`]; a c_v that depends on T makes it first order in Δt.

Boundary fluxes. A flux F [W m^{-2}] spread over the d cells next to the boundary is applied to each of those cells as a uniform volumetric rate

$$\Delta E_i = \Delta t \, \frac{F}{h_\mathrm{b} \, d}, \tag{8.12}$$

with h_b the width of the boundary cell, `dz`, in `TopCoolImpl::forward` (`top_cool.cpp:42-56`) and `BotHeatImpl::forward` (`bot_heat.cpp:42-56`). The energy delivered to a column of unit horizontal area is $\sum_i \Delta E_i h_i = \Delta t \, F \, \bar{h}/h_\mathrm{b}$, with $\bar{h}$ the mean width of the d cells: exactly F on uniform cells (`[C1]`), and F times $\bar{h}/h_\mathrm{b}$ otherwise, 0.872 for cells growing upward by 10 % with $d = 4$ (`[C2]`). On a spherical-polar grid the cell volume is not h times a fixed area, so the delivered power is not F either; not measured.

Exact and truncated. Both sources are exact pointwise for the stage-entry state; nothing is truncated in space.

8.7.3. Numerical method

Figure 8.7 shows where each module acts.

All three are cell sources: no stencil and no ghosts are read. `body-heat` acts on interior cells only. `top-cool` and `bot-heat` slice only in x_1, so they also write the x_2, x_3 ghost cells of their rows, which the boundary fill overwrites. They run in every stage, explicit, in the forcing loop (Section 8.1).

8.7.4. Code

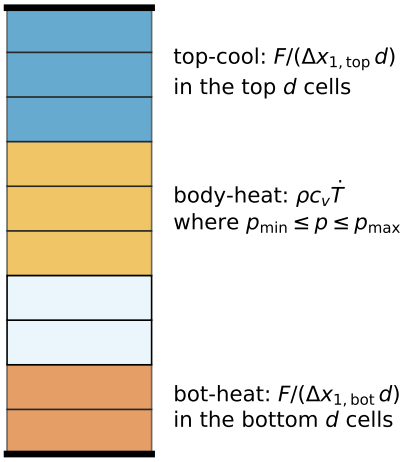


Figure 8.7.: Where the three heating modules act in an x1 column bounded by walls (thick black lines). Blue: the top d cells cooled by top-cool; vermillion: the bottom d cells heated by bot-heat; orange: the cells inside the pressure band of body-heat; light blue: untouched cells.

Table 8.14.: Code map of the heating modules.

what	code link	symbol	switch
options, `pmin` ≤ `pmax`	`body_heat.cpp:15-30`	`BodyHeatOptionsImpl::from_yaml`	`body_heat`
Equation 8.11	`body_heat.cpp:43-55`	`BodyHeatImpl::forward`	
options, `flux` ≤ 0	`top_cool.cpp:16-30`	`TopCoolOptionsImpl::from_yaml`	`top_cool`
Equation 8.12, top	`top_cool.cpp:42-56`	`TopCoolImpl::forward`	
options, `flux` ≥ 0	`bot_heat.cpp:16-30`	`BotHeatOptionsImpl::from_yaml`	`bot_heat`
Equation 8.12, bottom	`bot_heat.cpp:42-56`	`BotHeatImpl::forward`	

1. Each `from_yaml` checks its keys and signs (`body_heat.cpp:15-30`, `top_cool.cpp:16-30`, `bot_heat.cpp:16-30`).
2. `BodyHeatImpl::forward` masks the interior by the pressure band and adds Equation 8.11 (`body_heat.cpp:43-55`).
3. `TopCoolImpl::forward` and `BotHeatImpl::forward` return at once unless the block's upper or lower x_1 face is physical, then add Equation 8.12 (`top_cool.cpp:42-56`, `bot_heat.cpp:42-56`).

8.7.5. Tests

Table 8.15.: Tests of the heating modules.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_forcing.cpp` (`test_forcing.release`); `body_heat_uses_pressure_mask_and_mixture_cv`	heating only inside the band, with the mixture c_v	`allclose` defaults	closed form
same, `boundary_fluxes_scale_with_timestep`	`top-cool` and `bot-heat` are linear in Δt	`allclose` defaults	linearity
same, `meshblock_registers_parent_dependent_modules`	`body-heat` is installed by a MeshBlock	exact	registration
same, `reject_invalid_values`	`body-heat` option refusals	exact (throws)	contracts
this section's check, `heating_check.py`	`[C1]`–`[C3]`	per claim	stated in the check

Evidence. Check numbers: [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check it-

self; run: `python3 heating_check.py`, numpy 2.4.6, double]. ctest cases: **pending: pinned ctest run**. **Coverage.** CPU only. No test of the delivered column power, of `depth` > 1, or under an x_1 split.

8.7.6. Limits

1. **Stretched cells.** Equation 8.12 uses the boundary cell's width for all d cells, so the delivered power is F only on uniform cells (`[C2]`).
2. **Curved grids.** No correction for the cell volume of spherical-polar or cubed-sphere grids; not measured.
3. **`depth` larger than the block** is not checked against the block's x_1 cells; not tested.
4. **`body-heat` default band** `pmax` $= 1$ Pa (above).
5. **Under an x_1 split** only the block that owns the boundary applies `top-cool` or `bot-heat`; a `depth` that reaches into the next block is truncated there. Not tested.

8.8. Bottom relaxation of temperature, velocity and composition (`relax-bot-temp`, `relax-bot-velo`, `relax-bot-comp`)

8.8.1. Summary

> **i** At a glance
>
> **What it does.** Nudges the first interior cell row above the physical lower x_1 boundary toward a target temperature, velocity, or composition with a time scale τ. **Switch.** `relax-bot-temp: {tau, btemp, at-face}` (`btemp` required, `at-face` default `false`); `relax-bot-velo: {tau, bvx, bvy, bvz}` (targets default 0); `relax-bot-`

`comp: {tau, species, xfrac}` ($\sum$`xfrac` ≤ 1). $\tau > 0$ [s] for all three. Each off unless its key is present. **Order.** Explicit relaxation, first order in time per stage; the at-face target is exact for a linear T. **Defining equation.** Equation 8.13, Equation 8.15 and Equation 8.16.

The three modules stand in for a surface layer that the grid does not resolve. Each acts only in the bottom row of interior cells and only on the block that owns the physical lower x_1 boundary. `relax-bot-temp` adds heat at constant volume to bring the temperature toward `btemp`; with `at-face: true` it relaxes the temperature extrapolated to the lower face instead of the cell value, because a prescribed surface temperature is a value at the face. `relax-bot-velo` adds a momentum drag toward the target velocity, lowered to covariant components on the cubed sphere. `relax-bot-comp` exchanges mass between dry air and the named species at fixed density and temperature, with the internal-energy change computed by kintera.

None of the three changes the total energy for the momentum part: `relax-bot-velo` leaves E untouched, so the kinetic energy it removes becomes internal energy, as for the sponges (Section 8.9).

Aliases. chengcli/snapy#219 introduced `at-face` with the uniform-grid weights 1.5/−0.5 and the gain 1/1.5; #279 generalised them to the coordinate spacing.

8.8.2. Derivation

8.8.2.1. Temperature

Let cell 0 be the bottom interior cell and cell 1 the one above it, with centroids $x_{1,0}$, $x_{1,1}$ and the lower face at $x_{1,-1/2}$. The cell form adds, in one stage,

$$\Delta E_0 = \frac{\Delta t}{\tau}\,\rho_0 c_{v,0}\,(T_\mathrm{b} - T^\mathrm{tgt}), \tag{8.13}$$

with T_b = `btemp` and $T^\mathrm{tgt} = T_0$ (`relax_bot_temp.cpp:80-104`). With `at-face` the target is the linear extrapolation of the two cell values to the face,

$$\begin{aligned} T^\mathrm{tgt} &= (1+\alpha_\mathrm{f})\,T_0 - \alpha_\mathrm{f}\,T_1, \\ \alpha_\mathrm{f} &= \frac{x_{1,0} - x_{1,-1/2}}{x_{1,1} - x_{1,0}}, \end{aligned} \tag{8.14}$$

and the increment is multiplied by the gain $1/(1+\alpha_\mathrm{f})$, `gain` (`relax_bot_temp.cpp:82-103`). The target is exact for a linear T on any spacing [check `src/snapy_report/ch09/relax_check.py`, output `relax_check.out`, `[C1]`]; on uniform cells $\alpha_\mathrm{f} = \frac{1}{2}$, the target is $1.5T_0 - 0.5T_1$ and the gain $\frac{2}{3}$ (`[C2]`). The gain follows from the chain rule: only cell 0 is heated, and $\partial T^\mathrm{tgt}/\partial T_0 = 1+\alpha_\mathrm{f}$, so heating cell 0 at $(T_\mathrm{b} - T^\mathrm{tgt})/[\tau(1+\alpha_\mathrm{f})]$ moves the face value at exactly $(T_\mathrm{b} - T^\mathrm{tgt})/\tau$ (`[C3]`).

8.8.2.2. Velocity

With $\mathbf{v}_\mathrm{b}$ = (`bvx`, `bvy`, `bvz`) the target for the contravariant components (v^1, v^2, v^3) of the bottom cell,

$$\Delta m_a = \frac{\Delta t}{\tau}\,\rho_0\, g_{ab}\,(v^b_\mathrm{b} - v^b_0), \tag{8.15}$$

where g_{ab} lowers the index; it is the identity on orthogonal grids and on the cubed sphere has $g_{23} = \cos\theta_{jk}$; the force is lowered with `coord_vec_lower_` (`relax_bot_velo.cpp:65-75`), whose kernel is `coord_vec_lower_impl` (`coord_utils_impl.h:19-24`). The argument for lowering is the one of Section 8.9: the stored momenta are covariant, and an unlowered force rotates the wind instead of braking it.

8.8.2.3. Composition

Let y_n be the mass fractions of the species $n = 1..N_y$ in the bottom cell, ρ the total density, and y_n^{tgt} the mass fractions of the target state: the current mole fractions with the named species set to `xfrac` and dry air taking the rest, converted back to mass fractions. One stage adds

$$\begin{aligned} \Delta\rho_{\mathrm{d}} &= -\frac{\Delta t}{\tau}\,\rho \sum_n (y_n^{\mathrm{tgt}} - y_n), \\ \Delta(\rho y_n) &= \frac{\Delta t}{\tau}\,\rho\,(y_n^{\mathrm{tgt}} - y_n), \\ \Delta E &= \frac{\Delta t}{\tau}\,\left[U(\rho y^{\mathrm{tgt}}, T) - U(\rho y, T)\right], \end{aligned} \tag{8.16}$$

as `delta` (`relax_bot_comp.cpp:95-117`). The total density is unchanged (`[C4]`) and the momentum is untouched. $U(\cdot, T)$ is kintera's volumetric internal energy at the cell temperature, `VT->U` (`kintera thermo_y.cpp:240-245`), with the partial densities from `DY->V` (`kintera thermo_y.cpp:212-226`) and the mole/mass conversions `Y->X` (`kintera thermo_y.cpp:196-211`) and `X->Y` (`kintera thermo_x.cpp:140-150`). So the swap is isothermal provided kintera's U is the same function as snapy's equation of state uses; the test below checks that for the ideal-moist fixture.

Exact and truncated. All three are explicit relaxations: for $\Delta t \ll \tau$ the bottom row approaches its target as $e^{-t/\tau}$ to first order in $\Delta t/\tau$; with $\Delta t = \tau$ the composition reaches its target in one step, which the test uses.

8.8.3. Numerical method

Figure 8.8 shows the at-face target.

The modules read the bottom row of interior cells and, for `at-face`, the row above it; they read no ghost. `at-face` needs two interior cells in the

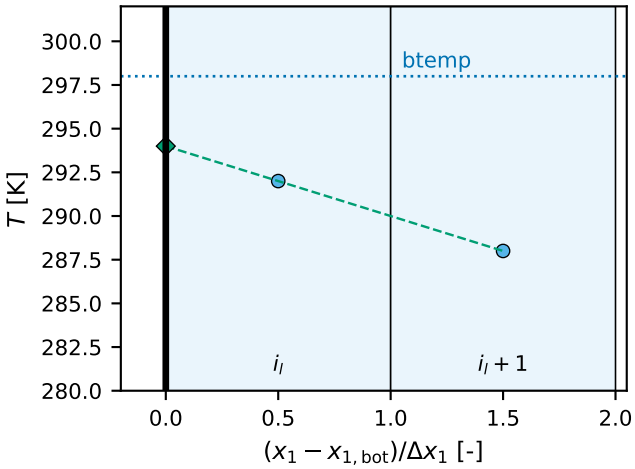


Figure 8.8.: The at-face target of relax-bot-temp on uniform cells. Sky-blue circles: the temperatures of the bottom cell and the one above it; bluish-green dashed line and diamond: their linear extrapolation to the lower face (thick black line), the value that is relaxed toward btemp (dotted blue line).

row selection, which `PartOptions::depth` caps at n_g, so it requires $n_g \geq 2$ (`relax_bot_temp.cpp:86-92`). They run in every stage, explicit, in the forcing loop (Section 8.1).

8.8.4. Code

Table 8.16.: Code map of the bottom relaxations.

what	code link	symbol	switch
temperature options	`relax_bot_temp.cpp:15-45`	`RelaxBotTempOptionsImpl::from_yaml`	`btemp`, required
Equation 8.13, Equation 8.14	`relax_bot_temp.cpp:58-106`	`RelaxBotTempImpl::forward`	`at-face`
velocity options	`relax_bot_velo.cpp:17-34`	`RelaxBotVeloOptionsImpl::from_yaml`	
Equation 8.15	`relax_bot_velo.cpp:47-77`	`RelaxBotVeloImpl::forward`	
composition options	`relax_bot_comp.cpp:20-52`	`RelaxBotCompOptionsImpl::from_yaml`	`species`, `xfrac`
species lookup, kintera modules	`relax_bot_comp.cpp:61-80`	`RelaxBotCompImpl::reset`	
Equation 8.16	`relax_bot_comp.cpp:82-119`	`RelaxBotCompImpl::forward`	
struct default `btemp`	`forcing.hpp:450-451`	`btemp`	not used from YAML

1. Each `from_yaml` checks keys and values: $\tau > 0$; `btemp` present and positive; `at-face` a literal `true`/`false`; `species` and `xfrac` of equal length, no duplicates, each `xfrac` in $[0, 1]$, sum ≤ 1 (`RelaxBotTempOptionsImpl::from_yaml`, `relax_bot_temp.cpp:15-45`; `RelaxBotVeloOptionsImpl::from_yaml`, `relax_bot_velo.cpp:17-34`; `RelaxBotCompOptionsImpl::from_yaml`, `relax_bot_comp.cpp:20-52`).
2. `RelaxBotCompImpl::reset` builds its own kintera `ThermoY` and `ThermoX`, and refuses an unknown species or the dry species (`relax_bot_comp.cpp:61-80`).
3. Each `forward` returns at once unless the lower x_1 face is physical, then adds its increment to the bottom row (`relax_bot_temp.cpp:58-106`, `relax_bot_velo.cpp:47-77`, `relax_bot_comp.cpp:82-119`).

8.8.5. Tests

Table 8.17.: Tests of the bottom relaxations.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_forcing.cpp` (`test_forcing.release`), `relax_bottom_temperature`	Equation 8.13 in the bottom row, zero elsewhere	`allclose` defaults	closed form

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `relax_bottom_temperature_at_face_rejects_non_bool`	`at-face` must be `true` or `false`	exact (throws)	parser contract
same, `relax_bottom_temperature_at_face_under_an_inversion`	the at-face tendency with $T_0 < T_1$	rtol 10^{-13}	round-off
same, `relax_bottom_temperature_at_face_uses_coordinate_spacing`	a linear T on stretched centres (0.5 and 2.5 above the face) gives zero tendency when $T^{\mathrm{tgt}} = T_{\mathrm{b}}$	exact	linear exactness
same, `relax_bottom_temperature_at_face`	default off is bit for bit the cell form; on uniform cells the at-face tendency uses $1.5/-0.5$ and gain $\frac{2}{3}$ and reads no ghost	exact; rtol 10^{-13}	contracts; round-off

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `relax_bottom_velocity`	Equation 8.15 on an orthogonal grid	`allclose` defaults	closed form
same, `cubed_sphere_sponge_drag_is_covariant`	covers `relax-bot-velo` on six panels: drag antiparallel to the covariant momentum	10^{-12} of the larger product	see Section 8.9
same, `relax_bottom_composition_preserves_state`	with $\Delta t = \tau$ the bottom mole fraction becomes `xfrac`; density, velocity unchanged; the temperature after snapy's ideal-moist U→W is unchanged	`allclose` defaults; 10^{-8} for T	single step, float64
same, `relax_bottom_composition_handles_multidimensional_ghost_zones`	the same on a 3-D block	per test	

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
same, `reject_invalid_values`, `reject_unknown_composition_species`	option refusals	exact (throws)	contracts
`tests/test_tracer_dry_convention.py` (`test_tracer_dry_convention_python`), relax arm	the tracer rides the dry air `relax-bot-comp` converts, both ways	`TOL` $= 10^{-12}$	round-off
`tests/test_dry_carry_zero_base.py` (`test_dry_carry_zero_base_python`)	with `relax-bot-comp` and a user source driving the dry base of the tracer carry to zero, the result stays finite and equals the closed form	10^{-12} absolute	round-off
this section's check, `relax_check.py`	`[C1]`–`[C4]`	per claim	stated in the check

Evidence. Check numbers: [sha: the formulas of snapy@e894700ff7a ee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 relax_check.py`, numpy 2.4.6, double]. ctest cases: **pending: pinned ctest run**. **Coverage.** CPU only; no multi-rank arm.

8.8.6. Limits

1. **`at-face` cost stability** on the one initial condition tried (chengcli/snapy#219 Limits); it is off by default. Not re-measured.
2. **kintera's U against snapy's.** `relax_bottom_composition_preserves_state` shows the swap is isothermal for the ideal-moist fixture `test_diffusion_moist.yaml` (constant heat capacities). For the moist-mixture equation of state, or species with temperature-dependent heat capacities, it is not tested.
3. **`btemp` default.** The options struct defaults `btemp` to 300 K (`forcing.hpp:450-451`), but YAML requires it; a Python user who builds the options directly gets 300 K silently.
4. **Velocity target naming.** `bvx`, `bvy`, `bvz` are the targets of v^1, v^2, v^3; with x_1 vertical, `bvx` is the vertical target, not an x component.
5. **Energy of the velocity drag.** E is untouched, so the removed kinetic energy heats the bottom row; intended for a drag, but not stated in the code.
6. **Solids.** The modules read the solid placeholder state if a solid cell sits in the bottom row (`sources/canoe__diffusion_ghost_TECH_REPORT.md` §7). Not tested.

8.9. Sponge layers: Rayleigh drag near the top and bottom boundaries (`top-sponge-lyr`, `bot-sponge-lyr`)

8.9.1. Summary

> **At a glance**
>
> **What it does.** Damps all three velocity components toward zero in a layer of given width at the top or bottom of the domain, with a $\sin^2$ ramp. **Switch.** `top-sponge-lyr: {tau, width}`, `bot-sponge-lyr: {tau, width}`, both > 0 [s, m]. Off unless the key is present. **Order.** Explicit relaxation, first order in time per stage. **Defining equation.** Equation 8.17.

A sponge layer absorbs waves that would otherwise reflect from the top or bottom boundary. snapy's sponges are a Rayleigh drag $-\rho\mathbf{v}/\tau$ multiplied by a profile that is 1 at the boundary and falls to 0 at `width` away from it as $\sin^2$. The drag acts on the momentum only; the total energy is untouched, so the kinetic energy it removes becomes internal energy in the same cell. On the cubed sphere the drag is lowered to covariant components so that it brakes the wind instead of rotating it.

Aliases. "Defect 1 — the drag was not antiparallel to the wind" in `sources/canoe__forcing_io_TECH_REPORT.md`; the same lowering as "Fix Coriolis (#168)".

8.9.2. Derivation

For a cell whose lower face is $x_{1,i-1/2}$, the depth fraction inside a top layer of width w is

$$\zeta_i = \min\Big(1, \max\Big(0, \frac{w - (x_{1,\mathrm{top}} - x_{1,i-1/2})}{w}\Big)\Big),$$

with $x_{1,\mathrm{top}}$ the upper face of the block, and for a bottom layer the distance is $x_{1,i-1/2} - x_{1,\mathrm{bot}}$ (`eta` in the code; `top_sponge_lyr.cpp:58-61`, `bot_sponge_lyr.cpp:58-61`). One stage adds

$$\Delta m_a = -\frac{\Delta t}{\tau}\,\sin^2\!\big(\tfrac{\pi}{2}\zeta_i\big)\,\rho_i\,g_{ab}v_i^b \tag{8.17}$$

to the three momentum rows and nothing to the energy (`force`, `top_sponge_lyr.cpp:62-73`). Because the distance is measured from each cell's *lower* face, the bottom cell of a bottom sponge has $\zeta = 1$, while the top cell of a top sponge has $\zeta = 1 - h/w$ [check `src/snapy_report/ch09/sponge_check.py`, output `sponge_check.out`, `[C1]`: scale 0.8536 for $h/w = 0.25$].

Heating. With $a = \Delta t \sin^2(\pi\zeta/2)/\tau$ and E unchanged, the internal energy of the cell rises by the kinetic energy removed, $\frac{1}{2}\rho|\mathbf{v}|^2[1-(1-a)^2]$, which is positive for $0 < a < 2$ (`[C2]`). The sponge is therefore an energy-conserving drag with frictional heating, not a sink of total energy.

Covariant lowering. The primitive velocities v^b are contravariant and the stored momenta m_a covariant. On the cubed sphere the horizontal metric has $g_{22} = g_{33} = 1$ and $g_{23} = \cos\theta_{jk} \neq 0$, and `coord_vec_lower_impl` forms $f_2 = f^2 + \cos\theta\, f^3$, $f_3 = f^3 + \cos\theta\, f^2$ (`coord_utils_impl.h:19-24`). The lowered drag is antiparallel to the covariant momentum and its power $v^a f_a = -\rho a\, g_{ab}v^a v^b$ is negative for $|\cos\theta| < 1$ (`[C3]`). Without the lowering the cross product of force and covariant momentum is $\cos\theta\,(v_3^2 - v_2^2)$ per unit ρa, non-zero away from the panel centres (`[C4]`: 0.455 at a corner with $\cos\theta = -\frac{1}{2}$, $\mathbf{v} = (1, 0.3)$). On an orthogonal grid $\cos\theta = 0$ and the lowering is the identity bit for bit.

Exact and truncated. The profile is exact as defined; the drag is explicit, first order in $\Delta t/\tau$.

8.9.3. Numerical method

Figure 8.9 shows the two profiles per cell.

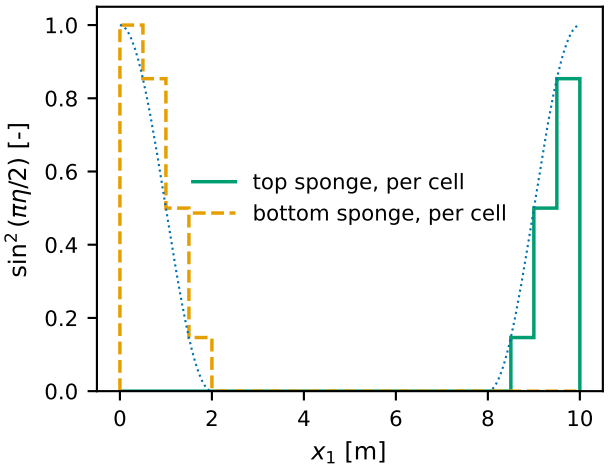


Figure 8.9.: Scale factor of the top (bluish green, solid) and bottom (orange, dashed) sponge layers per cell, for 20 cells of 0.5 m and a width of 2 m, from the port of the module; the dotted blue curves are the continuous sin^2 profiles. Each cell takes the value at its lower face, so the bottom profile is full in the bottom cell and the top profile is below 1 in the top cell.

The drag is a cell source at every cell of the block, ghosts included; ghost increments are overwritten by the boundary fill. Only the block that owns the physical boundary applies it (`is_physical_boundary`, `top_sponge_lyr.cpp:52`). It runs in every stage, explicit, in the forcing loop (Section 8.1).

8.9.4. Code

Table 8.18.: Code map of the sponge layers.

what	code link	symbol	switch
options	`top_sponge_lyr.cpp:17-34`	`TopSpongeLyrOptions::from_yaml`	`top_sponge_lyr`
top drag	`top_sponge_lyr.cpp:47-76`	`TopSpongeLyrImpl::forward`	
options	`bot_sponge_lyr.cpp:17-34`	`BotSpongeLyrOptions::from_yaml`	`bot_sponge_lyr`
bottom drag	`bot_sponge_lyr.cpp:47-76`	`BotSpongeLyrImpl::forward`	
lowering	`coord_utils_impl.h:19-24`	`coord_vec_lower_impl`	cubed sphere

1. `from_yaml` requires $\tau > 0$ and $w > 0$ (`top_sponge_lyr.cpp:17-34`, `bot_sponge_lyr.cpp:17-34`).
2. `forward` returns unless the block owns the physical boundary, forms ζ and the $\sin^2$ scale, builds the force from ρv^b, lowers it and subtracts it (`top_sponge_lyr.cpp:47-76`, `bot_sponge_lyr.cpp:47-76`).

8.9.5. Tests

Table 8.19.: Tests of the sponge layers.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`tests/test_forcing.cpp` (`test_forcing.release`); `cubed_sphere_sponge_drag_is_covariant`	on six panels the top sponge, the bottom sponge and `relax-bot-velo` give a drag antiparallel to the covariant momentum; the unlowered direction differs; the drag is not zero	$\lvert f \times m \rvert \leq 10^{-12}$ of the larger product; $> 10^{-10}$; > 0	double round-off; non-vacuity
same, `reject_invalid_values`	option refusals	exact (throws)	contracts
this section's check, `sponge_check.py`	`[C1]`–`[C4]`	per claim	stated in the check

Evidence. Check numbers: [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 sponge_check.py`, numpy 2.4.6, double]. The covariance test was found vacuous in its first form and rewritten (`sources/canoe__forcing_io_TECH_REPORT.md`, Defect 1, "Verification"; its PASS/FAIL arms were run at commits that are not pins of this report). ctest cases: **pending: pinned ctest run**. **Coverage.** CPU only. The test checks the direction of the drag, not its size: **no test of the drag magnitude**

or of the profile (a gap; `[C1]` and `[C2]` cover the formulas only).

8.9.6. Limits

1. **Truncated under an x_1 split.** Only the block owning the boundary applies the sponge; a `width` taller than that block is cut off at its other face. Not tested.
2. **Lower-face convention.** ζ uses each cell's lower face, so the top and bottom sponges are not mirror images (`[C1]`).
3. **Drag on v^1.** The vertical velocity is damped as well as the horizontal ones.
4. **Not run at scale on the cubed sphere** (chengcli/snapy#219 Limits).
5. **Leftover file.** `src/forcing/sponge_lyr.cpp_` is not compiled (wrong extension).

8.10. Plume forcing: accepted by the parser, never installed (`forcing/plume-forcing`)

`PlumeForcingImpl::forward` (`plume_forcing.cpp:29-41`) adds entrainment and buoyancy sources for a one-dimensional plume model whose primitive vector is radius, vertical velocity, buoyancy and a fourth variable, with options `entrainment` (default 0.1) and `N2` (default 0). The key `plume-forcing` is on the list `check_keys` accepts (`hydro_options.cpp:73-76`), but its options are parsed and the module is installed only when the equation of state is `plume-eos` (`hydro_options.cpp:116-119`, `register_forcing_modules.cpp:77-81`), and `EquationOfStateImpl::create` refuses that type as unknown (`equation_of_state.cpp:356-375`; the plume EOS was removed in chengcli/snapy#282). So at the pin a deck with `plume-forcing:` is accepted and the key has no effect, and no

test exercises the module; it is dead code and this report does not describe it further. chengcli/snapy#217 still lists it among the valid keys.

8.11. Turbulence closure: not built

The directory `src/turbulence/` holds a k-epsilon model and its base class carried over from Athena++ (it includes `athena/athena.hpp`, `k_epsilon_turbulence.cpp:1-25`, and reads its parameters from an Athena++ input file). The library is built from the list of source directories `src_files` (`CMakeLists.txt:35-53`), which does not include `turbulence/`, no YAML key refers to it, and no test builds it. snapy therefore has no turbulence closure: sub-grid transport is the constant isotropic viscosity and conduction of Section 8.4, optionally shaped in x_1 (Section 8.6). The files are not described further.

9. Moist physics coupling

This chapter is about how phase change and condensates enter snapy's step. The equations of state and the phase equilibrium itself are Chapter 2 (3); this chapter starts where they are called. It covers the saturation adjustment's place in the step; the two causes for rejecting a step that moist physics raises; the two species repairs of the conserved limiter, which keep condensate and vapour non-negative without creating mass; the reaction kinetics that the example drivers, not the library, run between steps; passive tracers carried per unit dry air; and the settling of condensates. One rule runs through all of them: latent heat is never a separate term. It is the difference of the species reference energies, so every operator here except sedimentation writes the species rows only and leaves the total energy, the momentum and the dry air alone. Sedimentation moves condensate with its own energy and momentum. The redo machinery in general is Chapter 7 and the positivity limiter's theory Chapter 8.

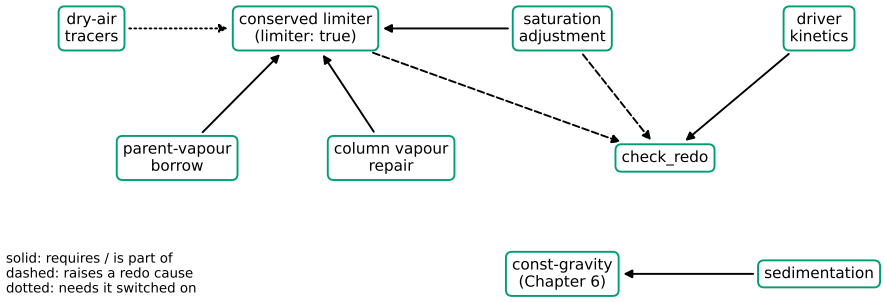


Figure 9.1.: Dependency map of the moist-coupling schemes. Solid arrows read ‘requires’ or ‘is part of’, dashed arrows ‘raises a redo cause in’, the dotted arrow ‘needs it switched on’. The borrow and the column repair are parts of the conserved limiter, which the saturation adjustment calls first; the limiter (cause 4) and the adjustment (cause 16) feed check_redo, which the driver kinetics calls; sedimentation requires constant gravity.

Table 9.1.: The schemes of this chapter, their switches and defaults, and the test that carries each one. "Driver" means the scheme is in the example drivers, not in the library, and has no library switch. `limiter` defaults to false both in the C++ options struct and in YAML. Full ctest names and tolerances are in each scheme's Tests layer and in Section 12.5.

scheme	switch	default	order	main test
Saturation adjustment (Section 9.1)	reactions in the thermo	on with reactions	n/a	`test_wall_saturation`
Redo causes (Section 9.2)	`max_redo`	5	n/a	`test_check_redo_saturation`
Parent-vapour borrow (Section 9.3)	`limiter`	off	n/a	`test_condensate_conservation`
Column vapour repair (Section 9.4)	`limiter`	off	n/a	`test_fix_vapor_volume`
Driver kinetics (Section 9.5)	driver	driver	1 in time	`test_uranus_cycle1_abort`
Dry-air tracers (Section 9.6)	`scalar`	off	as `reconstruct`	`test_tracer_dry_convention`

scheme	switch	default	order	main test
Sedimentation (Section 9.7)	`sedimentation`	off	1 in space	`test_sedimentation_guards`

9.1. Saturation adjustment in the step (reactions in the thermo block)

9.1.1. Summary

> **i** At a glance
>
> **What it does.** Brings every interior cell to phase equilibrium once per step, at fixed density and fixed internal energy, by moving mass between each vapour and its condensates. **Switch.** None of its own: on whenever the thermo block declares reactions; kintera's `uv-solver`, `max-iter` and `ftol` tune the solve. **Requires.** The whole-column conserved limiter, which runs just before it. **Runs.** On the last RK stage, interior cells only, before the boundary fill. **Defining equation.** Equation 9.1.

Phase change is fast compared with the dynamics, so snapy does not resolve it inside the RK stages. It lets the stages advect vapour and condensate as tracers that carry their own reference energies, and once per step, after the last stage has been averaged, it hands each interior cell to kintera's fixed-volume fixed-energy equilibrium solve (3) and writes the new mass fractions back. Latent heat is never a separate source: it is the difference of the species reference energies $u_n^\star$ (NOTATION.md section 2), so moving mass between phases at fixed ρe changes the temperature and

leaves E alone. This section is the snapy side of that solve: when it runs, what it is handed, what it writes, and what happens when it fails.

The adjustment is on whenever the thermo block of the YAML card lists reactions; there is no flag to turn it off while keeping the reactions. Its failures do not stop the step: they are counted and turned into redo cause 16 (Section 9.2). The ghost cells are not adjusted; they are refilled from the adjusted interior afterwards (the #206 ordering, chengcli/snapy#206).

9.1.2. Derivation

The derivation of the equilibrium itself, its energy constraint and its active-set solve, is 3; it is not repeated here. What this section adds is the state handed to it and the proof that the write-back changes only the species rows.

Step (6) of the stage update forms, in every interior cell, the total density, the internal energy density and the mass fractions from the conserved state:

$$\begin{aligned} \rho &= \rho_{\mathrm{d}} + \sum_n \rho_n, \\ \rho e &= E - \tfrac{1}{2}\rho|\mathbf{v}|^2, \\ y_n &= \rho_n/\rho. \end{aligned} \tag{9.1}$$

Equation 9.1 is formed as `rho`, `ie` and `yfrac` in `MeshBlockImpl::forward` (`meshblock.cpp:800-806`, `yfrac`). kintera's `ThermoYImpl::forward` receives ρ and ρe by value and y_n by reference, solves the equilibrium at fixed ρ and fixed ρe, and overwrites y_n (`kintera thermo_y.cpp:275-383`). snapy then writes $\rho_n \leftarrow \rho\, y_n$ for every species row, with the same ρ it handed in (`meshblock.cpp:816`, `yfrac`).

Three consequences follow from these lines alone. First, the energy row is not written, so E is unchanged to the last bit; since momentum is not

written either, $\rho e = E - \frac{1}{2}\rho|\mathbf{v}|^2$ is unchanged as long as ρ is. Second, the dry row is not written, so ρ_d is unchanged. Third, ρ is unchanged if and only if $\sum_n y_n$ is: the phase reactions move mass between species rows and never create it, so kintera returns fractions whose sum equals the input sum up to its solver tolerance, and ρ after the write-back equals ρ before to that tolerance. The energy is therefore conserved exactly and the total water to the tolerance of the phase solve; `test_wall_saturation` measures both below 10^{-12} relative over a whole run (Tests layer). No executable check is added in this chapter: the identities above are read off the code, and the solve's own algebra is checked by `saturation_check.py` of chapter 2.

9.1.3. Numerical method

Figure 9.2 places the adjustment in the step. Every stage ends with the RK average and a whole-column call of the conserved limiter (`meshblock.cpp:755-756`, `apply_conserved_limiter_`). On the last stage, and only if the thermo has reactions, the limiter runs once more and the adjustment follows (`meshblock.cpp:796-798`, `reactions`), so kintera never sees a negative species density. The solve is restricted to interior cells through the `exterior(false)` part of the block (`meshblock.cpp:811-814`, `exterior`), and the physical ghosts are refilled from the adjusted interior by `apply_boundaries`, after the gravity-work fixer (`meshblock.cpp:841`). The order of that fill relative to the exchange and the corner refresh is Section 10.9. Every quantity here is a cell average; the adjustment is pointwise in each cell, with no stencil.

The call passes `warm_start = true`: kintera reuses the active set of its last call when the shape and the device match, and starts cold otherwise (`kintera thermo_y.cpp:293-310`, `reuse`). A cell the solve could not adjust is marked by a negative diagnostic and counted; on CPU kintera also warns at once, on CUDA the count stays on the device until snapy reads it after the step (Section 9.2).

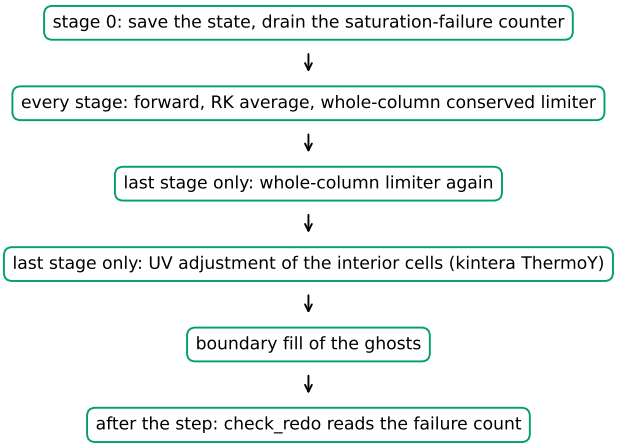


Figure 9.2.: Where the saturation adjustment sits in one RK step. Each box is one step of the stage update, read top to bottom; the last three boxes run on the last stage only, and the final box is the driver's call after the step. Green outlines mark operations, black arrows the order.

9.1.4. Code

Table 9.2.: Code map of the saturation adjustment in the step.

what	code link	symbol	switch
step (6) gate and call	`meshblock.cpp:795-816`	`pthermo`	reactions
per-stage limiter	`meshblock.cpp:755-756`, `apply_conserved_limiter_`	`apply_conserved_limiter_`	`limiter`
boundary fill after it	`meshblock.cpp:841`	`apply_boundaries`	
equilibrium solve	`kintera thermo_y.cpp:275-383`	`ThermoYImpl::forward`	`uv-solver`
warm-start reuse	`kintera thermo_y.cpp:293-310`, `reuse`	`reuse`	

1. The gate: last stage and a non-empty reaction list (`meshblock.cpp:796-797`, `reactions`).
2. The whole-column limiter, which removes negative species first (`meshblock.cpp:798`, `apply_conserved_limiter_`).
3. The state of Equation 9.1 (`meshblock.cpp:800-806`, `yfrac`).
4. The module found by name, so that every EOS type with a ThermoY is served (`meshblock.cpp:808-809`, `pthermo`).
5. The interior-only call with the warm start (`meshblock.cpp:811-814`, `warm_start`).
6. The write-back of the species rows (`meshblock.cpp:816`, `yfrac`).
7. The boundary fill (`meshblock.cpp:841`, `apply_boundaries`).

The step has no setup check of its own and prints no warning; kintera's

CPU warning on a failed cell is the only message a user sees, and the redo log line of Section 9.2 names the cause.

9.1.5. Tests

Table 9.3.: Tests of the saturation adjustment in the step. Assertions are read from the test sources at the pin; none was run for this draft.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wall_saturation.release`	energy and water over a run	10^{-12} rel.	test threshold
`test_balance_column.release`	moist column stays at rest	10^{-10}	dry case's bound
`test_vic_moist_device_python`	CPU equals CUDA, one step	10^{-9}	17 times the recorded gap
`test_check_redo_saturation_python`	a failed cell redoes the step	logical	failure protocol

`test_wall_saturation` runs a 64 m column next to a lower and an upper wall, once condensing and once evaporating, and asserts relative drifts of total energy and total water below 10^{-12} and an actual phase change (`test_wall_saturation.cpp:63-89`, `EXPECT_LT`). The moist `test_balance_column` case holds a subsaturated column, so it shows the adjustment leaves a balanced state at rest, with the condensate row exactly zero and the vapour within 10^{-12} (`test_balance_column.cpp:505-513`, `f_balanced`). `test_vic_moist_device_python` compares CPU

and CUDA for one moist step; its bound 10^{-9} is about 17 times the gap recorded in its source comment (`test_vic_moist_device.py:19-20`, `DEVICE_TOL`); that comment's number has no sha, deck or run of its own and is not quoted as evidence here. Coverage: the wall test is registered on CPU and, in CUDA builds, on CUDA; the device test exists only in CUDA builds; no test runs the adjustment on more than one rank. Measured numbers at the pin are **missing evidence** until a pinned build runs these entries (Section 12.5).

9.1.6. Limits

> **Limits**
>
> Only interior cells are adjusted; ghosts are copies, so a scheme that reads ghost species before the next boundary fill sees unadjusted values. A failed cell is left unadjusted for the rest of the step and costs a redo.

- No decomposed moist test with reflecting walls exists (chengcli/snapy#208).
- The warm start is discarded whenever kintera is called with another shape or device in between, for example by a diagnostic call from Python; correctness is unaffected, cost is not measured.
- Water is conserved to the phase solve's tolerance, not to round-off; the solve's tolerances are those of

  3.

- The switch coverage matrix (Section 11.8) has no row for moist runs; the combinations with `SNAP_WB_REF4`, `SNAP_X1_CENTROID_EXACT` and the face gravity work are covered only by the moist `test_balance_column` arms.

9.2. Step rejection for moist causes (`max_redo`)

9.2.1. Summary

> At a glance
>
> **What it does.** After each step, combines six failure flags over every block and rank into one cause word, then accepts the step or restores it and redoes it with a smaller time step. **Switch.** Integrator `max_redo`, default 5. **Moist causes.** Species repair by the limiter (4) and a cell the saturation adjustment left unadjusted (16). **Defining equation.** None: a decision rule, listed in Table 9.4 and checked by `redo_check.py`.

The redo machinery itself (how Δt is halved, what the driver does with the return value) belongs to Chapter 7 and is outlined in Section 1.5. This section covers the two causes moist physics raises and the parts of the rule that matter to them. A species repair by the conserved limiter that is larger than round-off (Section 9.3, Section 9.4) sets the limiter mark, cause 4. A cell that the saturation adjustment (Section 9.1) failed to bring to equilibrium is counted by kintera, and a non-zero count after the step is cause 16. Either one, on any rank, rejects the step everywhere.

The saturation cause is always on when the thermo has a ThermoY; the limiter cause is on with `limiter: true`. The redo restores `hydro_u`, recomputes `hydro_w`, restores the scalars, drops the gravity-work fixer's count of the rejected step and decrements the cycle. Aliases: the saturation cause was added by PR #248, the round-off bound on the limiter cause by #256 (chengcli/snapy#256).

9.2.2. Derivation

No derivation: the scheme is a decision rule, not a discretization. Its three properties that the rest of the chapter relies on are checked by `src/snapy_report/ch10/redo_check.py` against a line-for-line port. The six flags map to the bits 1, 2, 4, 8, 16, 32 in the order floor, clamp, limiter, nan, saturation, vic-solve (check line `[C1]`). The maximum over ranks of 0/1 flags is the bitwise OR of every rank's cause word, so one rank's failure is every rank's (`[C2]`, all 4096 two-rank combinations). Past `max_redo`, a step with a moist cause terminates without restoring; only the VIC solve cause (32) restores the state first (`[C3]`). The provenance of every `[Cn]` line is [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 redo_check.py`, numpy, double].

9.2.3. Numerical method

Figure 9.3 shows the decision. The saturation counter is drained once at the start of every step, so a failure counted by a ThermoY call between steps (a diagnostic from Python, say) is not charged to the next step (`meshblock.cpp:621`, `saturation_failures`). After the step, `local_redo_flags` reads the floor test first, then the limiter marks it may have set, then drains the saturation counter exactly once (`meshblock.cpp:1278-1286`). The counter read is one device-to-host copy on CUDA. A `Mesh` with several local blocks ORs their flags before the one collective reduction (`mesh.cpp:422-450`).

9.2.4. Code

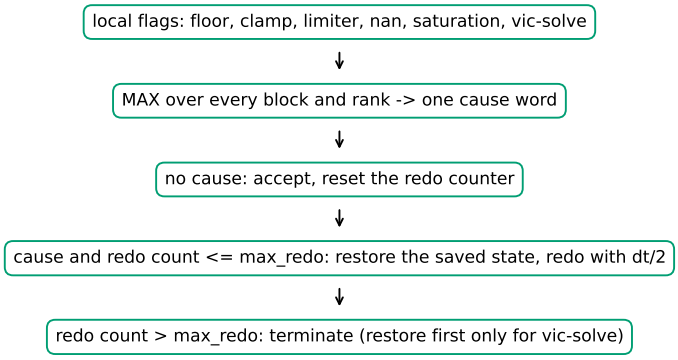


Figure 9.3.: Step acceptance for the moist causes. Each box is one step of the decision, read top to bottom; the last three boxes are the three outcomes. Green outlines mark operations, black arrows the order.

Table 9.4.: Code map of the moist redo causes.

what	code link	symbol	switch
entry point	`meshblock.cpp: 1302-1304`	`check_redo`	
restore or accept	`meshblock.cpp: 1229-1276`	`apply_redo`	`max_redo`
saturation counter	`meshblock.cpp: 1220-1227`	`saturation_ failures`	
kintera's counter	`kintera thermo_y.cpp: 385-390`	`take_ saturation_ adjustment_ failures`	

default of max_redo	pyharp integrator.hpp: 58	max_redo	max_redo

1. Drain the counter at stage 0 (`meshblock.cpp:621`, `saturation_failures`).
2. Form the six local flags (`meshblock.cpp:1278-1286`, `local_redo_flags`).
3. Reduce with MAX over ranks and pack the bits (`meshblock.cpp:1288-1300`, `reduce_redo_flags`).
4. Name the causes in the log line (`meshblock.cpp:1237-1240`, `causes`).
5. Terminate past `max_redo` unless cause 32 (`meshblock.cpp:1241-1247`, `max_redo`).
6. Restore the saved state, recompute the primitives and the scalar ratios (`meshblock.cpp:1249-1256`, `drop_gravity_work_fix`).

The user-visible messages are the log line “Redoing the step with smaller dt (causes: …)” and “Maximum number of redo attempts exceeded. Terminating.”, both in `apply_redo`.

9.2.5. Tests

Table 9.5.: Tests of the moist redo causes; assertions read from the test sources at the pin, not run.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_check_redo_saturation_python`	five arms, saturation cause alone	logical	failure protocol
`test_check_redo_parallel.release`	one decision on 2 ranks	exact	saved tensors copied back
`test_forcing.release`	species repair redoes; round-off does not	ulp bound	`kPositivityRoundoffUlp`
`test_uranus_cycle1_abort.release`	column survives; abort exits nonzero	exit status	log strings

`test_check_redo_saturation` starts from one NH4SH cell 150 times supersaturated. With `max-iter: 1` at least one cell is left unadjusted, and the step must be redone with the saturation cause alone and then accepted; a control step with the default `max-iter` must be clean; a failure counted between steps must not be charged; and a two-block `Mesh` with the cell in block 1 must read block 1's count (`test_check_redo_saturation.py: 1-11`, `check_redo`). The `test_forcing` cases put -10^{-4} in a species at stage 1 and require cause 4; a repair below `kPositivityRoundoffUlp` ulp of the gas density is applied but not redone (`test_forcing.cpp:805-848`, `limiter_species_repair_redoes_the_step`). The CUDA arm `test_check_redo_saturation_cuda_python` exists only in CUDA builds; the 2-rank test is the only multi-rank evidence. Measured results at the pin

are **missing evidence** (Section 12.5).

9.2.6. Limits

> ⚠ Limits
>
> A failure that a smaller time step cannot cure exhausts `max_redo` and ends the run (PR #248). A species repair above round-off redoes the step even when it would be harmless.

- A stale driver state can make every attempt fail (chengcli/snapy#263, closed as a driver defect).
- `examples/uranus.yaml` aborted at cycle 342 on an H2S repair in the issue history (chengcli/snapy#260); not re-measured at the pin.
- Past `max_redo` a moist failure terminates with the rejected state in memory; only the VIC cause restores first (`redo_check.py [C3]`).
- Moist redo combinations are absent from the switch coverage matrix (Section 11.8).

9.3. Condensate repair by parent-vapour borrow (`limiter`)

Local symbols: $\mathcal{P}_c$ is the set of parent vapours of condensate c. Tracked in ISSUES.md.

9.3.1. Summary

> **i** At a glance
>
> **What it does.** Fills a negative condensate density from its parent vapours in the same cell, split by stoichiometric mass, then sets the condensate to exactly zero. **Switch.** EOS `limiter: true`; default false. **Conserves.** Mass and every element, cell by cell, ghosts included. **Defining equation.** Equation 9.2.

A negative condensate density means the positivity-limited transport over-drained a cell: the mass is in a neighbour, not missing. Clamping it to zero would create mass one way; the borrow instead takes the deficit from the vapours the condensate is made of, in the same cell, in the proportions of the reaction that makes it. A cell whose vapour cannot cover the deficit goes vapour-negative and is handled by the column repair that follows (Section 9.4). The borrow runs inside every call of the conserved limiter, so on every stage and once more before the saturation adjustment (Section 9.1).

It is on with `limiter: true` and acts only on condensates that some nucleation reaction produces; a condensate with no such reaction (rain, for example) is repaired by the column repair instead. Aliases: the parent cache was introduced in PR #223 and moved to the thermo's own species table in #235 (chengcli/snapy#235).

9.3.2. Derivation

Let $c \in \mathcal{C}$ be a condensate and k the first nucleation reaction listed with c among its products. Its parents $\mathcal{P}_c$ are the reactant vapours of k, dry air excluded. With $|\nu_{mk}|$ the reactant coefficient of m in k and μ_m its molar mass, every cell applies

$$\begin{aligned} \rho_m &\leftarrow \rho_m + \frac{|\nu_{mk}|\,\mu_m}{\sum_{m' \in \mathcal{P}_c} |\nu_{m'k}|\,\mu_{m'}}\ \min(\rho_c, 0), \\ \rho_c &\leftarrow \max(\rho_c, 0) \end{aligned} \tag{9.2}$$

for every $m \in \mathcal{P}_c$. Equation 9.2 is applied to the `deficit` in `apply_conserved_limiter_` (`equation_of_state.cpp:262-268`, `deficit`), with the shares normalised once, at construction, in `cache_cloud_parents_` (`equation_of_state.cpp:155-161`, `parent_mass`).

The shares sum to one, so the parents lose exactly $-\min(\rho_c, 0)$ and the condensate gains it: the cell's total density is unchanged (check line `[C1]`). Elements are conserved too. For $A + B \to AB$ the molar mass of the condensate is $\mu_A + \mu_B$; filling a deficit d fills $d/(\mu_A + \mu_B)$ moles of condensate and debits $d\,\mu_A/(\mu_A + \mu_B)/\mu_A$ moles of A, the same number, and likewise for B (`[C2]`, symbolic). Energy is not corrected: E is untouched, so the shift between phases moves the temperature through the reference energies by the latent heat of the borrowed mass. The comment at the call site estimates the shift at about one per cent of the local value and notes that the saturation adjustment re-equilibrates it within the same step; that estimate has no measurement behind it here. Checks: `src/snapy_report/ch10/borrow_check.py`, output `src/snapy_report/ch10/borrow_check.out` [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 borrow_check.py`, numpy and sympy, double].

9.3.3. Numerical method

The repair is pointwise in each cell and acts on the whole array, ghosts included, so a ghost copy receives the identical repair and neighbouring ranks stay in step. It runs before the column repair, inside the same limiter call. Two properties of the cache matter: only the first reaction that produces c is read, and dry air is skipped even if a card lists it as a

reactant (`[C3]`, `[C4]`). The cache is built from the thermo's own copy of the species names and molar masses, so a later card that refills kintera's global tables does not change it. Figure 9.4 shows one cell.

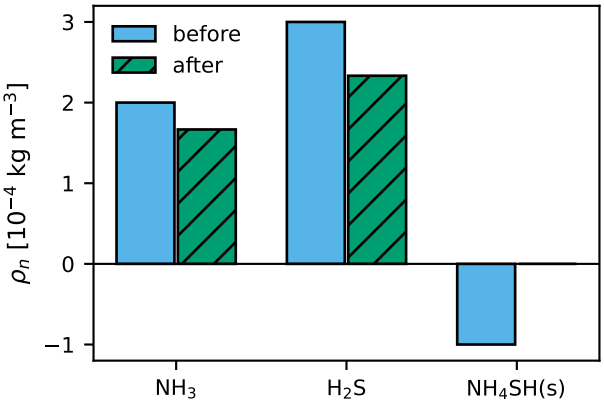


Figure 9.4.: An NH4SH(s) deficit of 1e-4 kg m^-3 filled from NH3 and H2S in one cell. Sky-blue bars are the partial densities before the borrow, green hatched bars after it; the condensate ends at exactly zero and the two vapours lose one third and two thirds of the deficit, their molar-mass shares.

9.3.4. Code

Table 9.6.: Code map of the parent-vapour borrow.

what	code link	symbol	switch
parent cache	`equation_of_state.cpp: 110-163`	`cache_cloud_parents_`	

what	code link	symbol	switch
dry slot never a parent	`equation_of_state.cpp:142-145`	`vapor_it`	
first producing reaction	`equation_of_state.cpp:138-153`	`break`	
borrow and clamp	`equation_of_state.cpp:262-268`, `deficit`	`deficit`	`limiter`

1. At construction, take the thermo's own names and molar masses (`equation_of_state.cpp:128-130`, `populate_thermo`).
2. Refuse a thermo whose tables disagree in length (`equation_of_state.cpp:131-135`, `TORCH_CHECK`).
3. Record the parents and their mass shares (`equation_of_state.cpp:137-161`, `cloud_parent_cache_`).
4. In each limiter call, skip parentless clouds and borrow for the rest (`equation_of_state.cpp:253-269`, `parentless`).

The only setup check is the table-length refusal of step 2, "[EquationOfState] thermo carries … species names".

9.3.5. Tests

Table 9.7.: Tests of the parent-vapour borrow; assertions read from the test sources at the pin, not run.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_condensate_conservation.release`	total mass; molar-mass split	10^{-12}; 10^{-6}	test thresholds
`test_cloud_parent_slots.release`	parents with unused species	10^{-12}; 10^{-6}	test thresholds
`test_diffusion_moist.release`	limiter reads nucleation parents	10^{-12}; 10^{-6}	test thresholds
`test_two_cards_species.release`	split survives a second card	10^{-10} (f64)	test threshold

`test_condensate_conservation` keeps the total mass to 10^{-12} and checks the NH3 and H2S debits against their molar-mass shares to 10^{-6}, with the condensate exactly zero after (`test_condensate_conservation.cpp:47-53`, `allclose`). The 10^{-6} of the share comparison is looser than the arithmetic; the source gives no reason for it, and `borrow_check.py [C1]` shows the shares themselves are exact to round-off. The two-card test uses 10^{-10} in float64 and 10^{-4} in float32 (`test_two_cards_species.cpp:78-90`, `tol`). All four run on CPU, and on CUDA where the device fixture is compiled; none is multi-rank. Measured results at the pin are **missing evidence** (Section 12.5).

9.3.6. Limits

- Only the first nucleation reaction that produces a condensate sets its parents; a second producing reaction is ignored (`borrow_check.py [C3]`).
- No energy correction: the borrowed mass changes phase at fixed E, so T moves by the latent heat; the size of the move is not measured.
- A deficit larger than the parents' vapour leaves the vapour negative for the column repair; if the column cannot cover it, the vapour repair throws (Section 9.4).
- Not in the switch coverage matrix (Section 11.8).

9.4. Column vapour and cloud repair (`limiter`)

Local symbols: $\mathcal{M}$ is the window of x_1 cells one scan step rewrites. Tracked in ISSUES.md.

9.4.1. Summary

> **i** At a glance
>
> **What it does.** Removes negative vapour, and negative parentless cloud, along each x_1 column by mixing it with the cells below at one common ratio to the dry density, conserving the column's mass. **Switch.** EOS `limiter: true`; default false. The gather of a column split over x_1 blocks is on only for the call from the stage update. **Fails.** A vapour column that cannot be repaired throws; a cloud column is clamped. **Defining equation.** Equation 9.3.

After the parent-vapour borrow (Section 9.3) a cell can still hold negative vapour, and a condensate with no nucleation parent cannot borrow at

all. Both are repaired along the column: a negative cell is mixed with the cells below it until the mixture is non-negative, and the mixture is spread at one ratio to the dry density. The repair conserves the column's mass $\sum_i V_i \rho_{n,i}$, not its density sum, which matters on grids whose cell volume changes with height. If nothing below can cover a deficit, the cells above supply it, bounded by the deficit. A vapour column that cannot be repaired at all is an error; a cloud column in net deficit is clamped to zero, which creates exactly the missing mass.

The repair is on with `limiter: true`. Aliases: the volume weights are #241, the upward branch #243, the whole-column gather #244 and #268 (chengcli/snapy#268); PR #230's "the repair is per meshblock" is superseded by the gather.

9.4.2. Derivation

Index the column from the bottom, $i = 0..N-1$, with cell volumes V_i. The scan starts at the top. A cell with $\rho_{n,i} \geq 0$ is passed over. A negative cell starts a window $\mathcal{M}$ that grows downward one cell at a time until $\sum_{l\in\mathcal{M}} V_l \rho_{n,l} \geq 0$ or the bottom is passed. If the sum is non-negative, every cell of the window is rewritten as

$$\rho_{n,i} \leftarrow \rho_{\mathrm{d},i} \frac{\sum_{l\in\mathcal{M}} V_l \, \rho_{n,l}}{\sum_{l\in\mathcal{M}} V_l \, \rho_{\mathrm{d},l}} \tag{9.3}$$

for $i \in \mathcal{M}$, and the scan resumes below the window. Equation 9.3 is computed in `fix_vapor_impl`: the window sums are `sum_vapor` and `sum_major` (`fix_vapor_impl.h:32-39`), and the common ratio `yfrac` is written to every cell of the window (`fix_vapor_impl.h:66-70`). The code weights by V_l/V_0, the volume relative to the column's bottom cell; the factor cancels in the ratio.

Multiplying Equation 9.3 by V_i and summing over the window gives back $\sum_{l\in\mathcal{M}} V_l \rho_{n,l}$: the window's mass is unchanged, and every rewritten value

has the sign of the dry density, so it is non-negative. Windows are disjoint and the scan index only decreases, so the scan visits each cell at most twice and terminates. If the bottom is passed with the sum still negative, the deficit $D = -\sum_{l \le i_0} V_l \rho_{n,l} / V_0$ of the cells from the starting cell i_0 down is taken from the cells above, in order, each giving at most what it holds; the cells from i_0 down are set to zero. Before writing anything this branch checks that the cells above hold at least D, so it writes nothing when it fails (`fix_vapor_impl.h:41-64`, `deficit`). A window repaired earlier in the same scan stays rewritten, though: a column that fails below an already repaired window is returned with that window flattened, mass kept. The kernel fails on a non-positive dry density; otherwise, in every case `fixvapor_check.py` tried, only on a column whose total vapour mass is negative.

All of this is checked by `src/snapy_report/ch10/fixvapor_check.py`, a line-for-line port of the kernel. It replays the inputs of the `eos_limiter` gtests (`[C1]` to `[C3]`), keeps $\sum_i V_i \rho_{n,i}$ to 2.2×10^{-15} relative on 1995 repaired random columns with unequal volumes (`[C4]`), finds no failure on a column with non-negative total (`[C5]`), repairs an interior and a bottom deficit in one scan (`[C6]`) and confirms the partial rewrite before a failure (`[C7]`) [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 fixvapor_check.py`, numpy, double]. The partial rewrite is a property of the port; it has not been reproduced in a snapy run.

9.4.3. Numerical method

The kernel runs once per x_1 column of interior cells, one CPU thread or one CUDA thread per column, inside each call of the conserved limiter. Parentless clouds are repaired first without requiring success and then clamped; vapour is repaired second and must succeed (`equation_of_state.cpp:307-312`, `parentless`). When the call comes from the stage update and x_1 is split over several blocks (`pz` > 1), every block gathers the

full column with the dry density and the volumes, repairs it and keeps its own slice (`equation_of_state.cpp:276-305`). Failures are counted with `std::atomic` on CPU and `atomicAdd` on CUDA, and read back once per call, which costs one device synchronisation. Figure 9.5 shows a column with an interior and a bottom deficit.

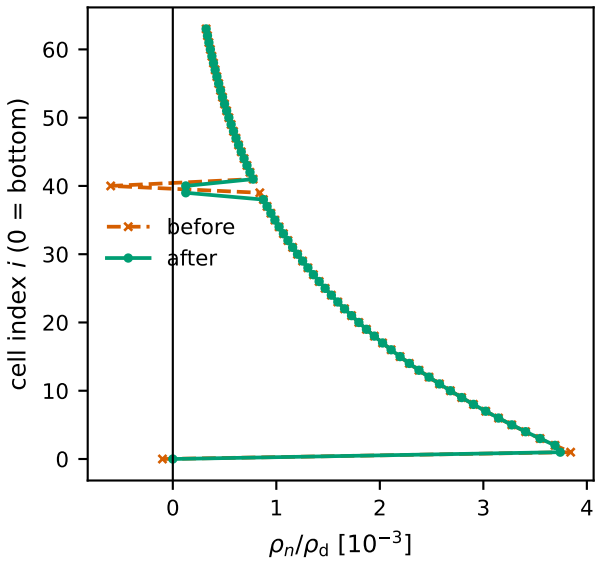


Figure 9.5.: One 64-cell column before and after the repair, as the ratio of vapour to dry density. The vermillion dashed line with crosses is the column before, with a negative cell at index 40 and one at the bottom; the green line with dots is after: cells 40 and 39 share one ratio, and the bottom deficit is taken from cell 1 above it. The column's mass is unchanged.

9.4.4. Code

Table 9.8.: Code map of the column vapour and cloud repair.

what	code link	symbol	switch
column kernel	`fix_vapor_impl.h:9-79`	`fix_vapor_impl`	
CPU dispatch	`eos_dispatch.cpp:79-101`	`call_fix_vapor_cpu`	
CUDA dispatch	`eos_dispatch.cu:52-74`	`call_fix_vapor_cuda`	
column repair, gather	`equation_of_state.cpp:276-305`	`repair_column`	`whole_column`
parentless, then vapour	`equation_of_state.cpp:307-312`, `parentless`	`parentless`	

1. Take the cell volumes so the repair conserves mass (`equation_of_state.cpp:272-273`, `vol`).
2. Gather the column when split over x_1 blocks (`equation_of_state.cpp:280-288`, `gather_x1`).
3. Run the kernel per column (`equation_of_state.cpp:289-296`, `call_fix_vapor`).
4. Throw for vapour (`equation_of_state.cpp:297-299`, `require_success`).
5. Copy back this block's slice (`equation_of_state.cpp:300-304`, `copy_`).
6. Mark the limiter only above round-off (`equation_of_state.cpp:313-321`, `positivity_roundoff`).

A user can hit one message: "[EquationOfState] apply_conserved_lim-

iter_: Failed to fix vapor mass fractions."

9.4.5. Tests

Table 9.9.: Tests of the column repair; assertions read from the test sources at the pin, not run.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_eos.release`	five `eos_limiter` columns	10^{-14} rel.	test threshold
`test_fix_vapor_volume.release`	column mass, unequal volumes	10^{-12}; 10^{-6} f32	test thresholds
`test_parentless_cloud.release`	cloud kept, clamped, zero column	10^{-12}; 10^{-6} f32	test thresholds
`test_vapor_column_nb1.release`	split column equals one block	bitwise	test threshold
`test_fix_vapor_reports_failure_python`	unrepairable column throws	n/a	message match

The `eos_limiter` cases are the inputs replayed by `fixvapor_check.py`; the bottom-cell case asserts the transfer itself to 10^{-25} absolute, because a tolerance on the supplier's value would also pass a supplier that gave nothing (`test_eos.cpp:68-89`, `repairs_bottom_cell_from_above`).

`test_fix_vapor_volume` keeps the column mass of a spherical-polar column to 10^{-12} in float64 and 10^{-6} in float32 (`test_fix_vapor_volume.cpp:82-84`, `tol`). The split-column test requires every vapour cell of a two-block column to equal the one-block result exactly (`test_vapor_column_nb1.cpp:133-143`, `vapor_repair_keeps_the_mass_with_nb1_2`). Also registered: `test_parentless_cloud_nb1.release`, the 2-rank `test_parentless_cloud_nb1_mp` (rain and total mass to 10^{-12}), `test_fix_vapor_counts_every_column_python` (one broken column in 16384 is reported) and the CUDA arm `test_fix_vapor_reports_failure_cuda_python`. Measured results at the pin are **missing evidence** (Section 12.5).

9.4.6. Limits

> **Limits**
>
> A vapour column whose total is negative stops the run. A cloud column whose total is negative is clamped, creating the missing mass.

- The downward branch mixes: it moves vapour beyond the deficit, over the whole window.
- A failure below an already repaired window leaves that window rewritten (`fixvapor_check.py [C7]`, port only). For vapour the run stops anyway; for parentless clouds the rewrite stays, mass kept.
- Serial callers with several x_1 blocks wait at the gather (PR #244); the cross-process gather has not run on CUDA.
- The weight is relative to the bottom cell, so last-bit differences in spacing can move results at round-off (PR #243).
- Not in the switch coverage matrix (Section 11.8).

9.5. Precipitation and evaporation kinetics in the example drivers (driver-level coupling)

Local symbols: $\dot{\omega}$ is the vector of reaction rates [$\mathrm{mol\,m^{-3}\,s^{-1}}$] and $\mathsf{J}_{\dot{\omega}}$ its Jacobian with respect to the molar concentrations; x is the extent of a two-product evaporation to equilibrium [$\mathrm{mol\,m^{-3}}$]. Tracked in ISSUES.md. The subscript keeps $\mathsf{J}_{\dot{\omega}}$ apart from chapter 2's J of the phase solve.

9.5.1. Summary

> **i** At a glance
>
> **What it does.** Between two steps, advances the species by one linearised backward-Euler step of kintera's reaction rates (evaporation, precipitation), then asks `check_redo` whether to keep the step. **Switch.** None in the library: it runs only in drivers that build a kintera `Kinetics` from the card. **Changes.** Species rows only; E is untouched. **Order.** First order in time. **Defining equation.** Equation 9.4.

The snapy library does not run reaction kinetics. Two example drivers do: `examples/run_hydro.cpp` and `examples/jupiter_evap_precip_1d.cpp`. After all RK stages of a step, they refresh the primitive state, evaluate kintera's rates and their Jacobian at the post-step state, take one linearised implicit step for the concentrations, add the change to the species rows of the conserved state and call `check_redo`. The coupling is therefore operator-split, first order, and a property of those drivers: a run built on the library alone has no kinetics unless its own driver adds the same block. As with the saturation adjustment, latent heat is carried by the reference energies, so the energy row is not touched.

Aliases: the refresh before kinetics is #257 (chengcli/snapy#257); the two-product extent law is kintera #114.

9.5.2. Derivation

The rates are evaluated once, at the concentrations $\tilde{\mathbf{c}}$ of the step's end. Linearising $\dot{\omega}$ about them and taking one backward-Euler step of $\mathrm{d}\tilde{\mathbf{c}}/\mathrm{d}t = \mathsf{N}\dot{\omega}(\tilde{\mathbf{c}})$ gives

$$\Big(\frac{\mathsf{I}}{\Delta t} - \mathsf{N}\mathsf{J}_{\dot{\omega}}\Big)\Delta\tilde{\mathbf{c}} = \mathsf{N}\dot{\omega} \tag{9.4}$$

for the change $\Delta\tilde{\mathbf{c}}$ over the step Δt. Equation 9.4 is assembled as `A` and `sr` in `evolve_implicit_cell` (`kintera evolve_implicit_impl.h:66-71`, `inv_dt`) and solved by LU with partial pivoting, falling back to a minimum-norm least-squares solve for a singular cell (`kintera evolve_implicit_impl.h:77`, `dsolve_lu`). For a linear rate law the step is exactly backward Euler (check line `[C2]`). For the reversible linear reaction $A \rightleftharpoons B$ with rate constants k_f, k_b it moves the distance to equilibrium by the factor $1/(1 + \Delta t(k_\mathrm{f} + k_\mathrm{b}))$, which lies in $(0, 1]$ for every Δt: one step never overshoots equilibrium, however large the step (`[C3]`). The driver converts the change to partial densities, $\Delta\rho_n = \mu_n\Delta\tilde{c}_n$; for a mass-balanced reaction $\sum_n \mu_n\nu_{nk} = 0$, so the update has zero net mass (`[C4]`).

For a condensate evaporating into two gases, $AB(s) \to A{+}B$, kintera's rate is proportional to the extent x that would bring the gases to equilibrium, the positive root of $(\tilde{c}_A + x)(\tilde{c}_B + x) = K$. The code evaluates it in a form without cancellation,

$$x = \frac{2\,(K - \tilde{c}_A\tilde{c}_B)}{\tilde{c}_A + \tilde{c}_B + \sqrt{(\tilde{c}_A - \tilde{c}_B)^2 + 4K}}, \tag{9.5}$$

and sets it to zero for supersaturated cells. Equation 9.5 is computed as `x` in `EvaporationImpl::forward` (`kintera evaporation.cpp:180-191`, `root`). `[C1]` shows symbolically that it is the positive root. The full

derivation of the steady-diffusion rate law behind it is in `sources/canoe__EVAPORATION_MULTIPRODUCT_TECH_REPORT.md` section 4, written in symbols this report reserves for other quantities; its translation into the report's notation is still to be written. Checks: `src/snapy_report/ch10/kinetics_check.py`, output `kinetics_check.out` [sha: the formulas of kintera@c55b13b2204997d2d09e04498558ab9495d8ee77 and snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 kinetics_check.py`, numpy and sympy, double].

9.5.3. Numerical method

The block runs once per accepted or attempted step, after the stages and before `check_redo` (`run_hydro.cpp:171-195`). It first recomputes `hydro_w` from `hydro_u`, so the rates see the saturation-adjusted state (`run_hydro.cpp:175-176`). It acts on the whole array, ghosts included, with no exchange; the next stage's boundary fill and exchange restore consistency. No conserved limiter runs between the update and `check_redo`, so a negative species produced by the kinetics shows up as a limiter repair, and possibly a redo, at the next step. Figure 9.6 compares the extent law with the earlier law $K - \tilde{c}_A \tilde{c}_B$.

9.5.4. Code

Table 9.10.: Code map of the driver-level kinetics.

what	code link	symbol	switch
kinetics block	`run_hydro.cpp:171-191`	`evolve_implicit`	driver

what	code link	symbol	switch
redo after kinetics	`run_hydro.cpp:193-195`	`check_redo`	
same, 1-D driver	`jupiter_evap_precip_1d.cpp:244-261`	`evolve_implicit`	driver
implicit step	`kintera evolve_implicit_impl.h:55-77`	`evolve_implicit_cell`	
evaporation rate	`kintera evaporation.cpp:128-195`	`EvaporationImpl::forward`	

1. Refresh the primitives from the adjusted conserved state (`run_hydro.cpp:175-176`, `hydro_w`).
2. Form temperature, pressure and concentrations (`run_hydro.cpp:177-181`, `conc`).
3. Evaluate rates and Jacobian (`run_hydro.cpp:184-186`, `jacobian`).
4. Take the implicit step and convert to densities (`run_hydro.cpp:187-190`, `del_rho`).
5. Add to the species rows (`run_hydro.cpp:191`, `hydro_u`).
6. Accept, redo or terminate (`run_hydro.cpp:193-195`, `check_redo`).

9.5.5. Tests

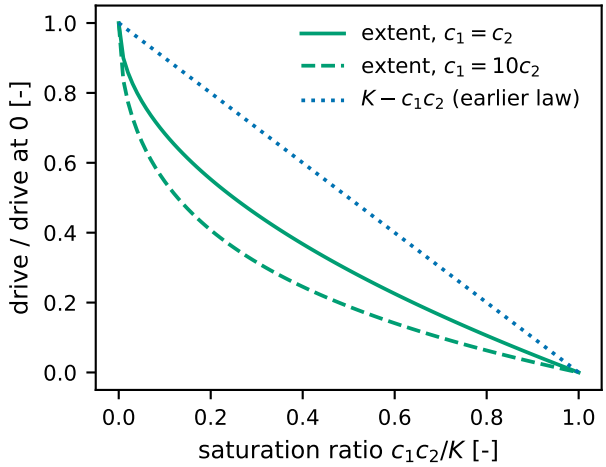


Figure 9.6.: Evaporation drive of a two-product condensate against the saturation ratio, each curve divided by its value at zero saturation. Green: the extent law, solid for equal gas concentrations, dashed for one gas ten times the other; blue dotted: the earlier law K - c_A c_B. The extent law falls faster near zero saturation, and depends on how the product is split between the gases.

Table 9.11.: Tests of the driver-level kinetics; assertions read from the test sources at the pin, not run.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_uranus_cycle1_abort.release`	column reaches cycle 40, no redo	exit status	log strings

`UranusLate.column_reaches_cycle_40` runs `run_hydro` on a Uranus column and requires exit code 0, no "Redoing the step" and no "Terminating abnormally" in the log (`test_uranus_cycle1_abort.cpp:115-127`, `column_reaches_cycle_40`). That is the only snapy test that runs the kinetics block. There is no snapy test of one kinetics step's mass and energy budget, of the evaporation rate or of `jupiter_evap_precip_1d`: **missing evidence** on the snapy side; the rate law is tested in kintera. CPU only; no CUDA or multi-rank arm.

9.5.6. Limits

> ⚠ Limits
>
> Kinetics exists only in drivers that implement it. A user driver that calls only `forward` and `check_redo` has none.

- First order in time, operator-split from the dynamics; the split error is not measured.
- No conserved limiter between the update and `check_redo` (`run_hydro.cpp:191-193`), so negative species from kinetics surface as a limiter repair at the next step (chengcli/snapy#256, #263).
- The update covers ghosts without an exchange.

- The evaporation note's status line ("not landed") is superseded: the extent law is present at the kintera pin.
- Not in the switch coverage matrix (Section 11.8).

9.6. Passive tracers per dry air, with an optional upper bound (`scalar`)

Local symbols: b is the upper bound of a tracer's mixing ratio (YAML `scalar/upper-bound`); F_n and F_d are the x_1 face fluxes of tracer n and of dry air. Tracked in ISSUES.md.

9.6.1. Summary

> **i** At a glance
>
> **What it does.** Keeps passive tracers as mixing ratios r_n per unit dry mass, so they move with the dry air under every operator that moves dry air; keeps them non-negative and, optionally, at or below a bound b. **Switch.** YAML `scalar` (`nvar` or `names`); `upper-bound` default -1 (off), 0 refused, needs the EOS limiter. **Defining equation.** Equation 9.6.

Chapter 1 maps the scalar transport path and derives how the ratio rides with the implicit dry transfer and with dry-density sources (Section 1.12, Section 1.13, Equation 1.3, Equation 1.5). This section adds what makes those tracers part of the moist coupling: why the reference mass is dry air and not total mass, the second limiter that bounds the ratio from above, and the conditions under which that bound holds.

A tracer measured per unit total mass would change its ratio whenever water condenses, evaporates or settles out of a cell, although no tracer

moved. Per unit dry mass the ratio changes only when dry air moves, which every moist operator of this chapter leaves alone: the saturation adjustment, the borrow, the column repair and the kinetics write species rows only, and sedimentation moves condensate, not dry air. Aliases: the dry-air convention is PR #220.

9.6.2. Derivation

Write $\rho_{\mathrm{d}} r_n$ for the tracer density, so the code's `r = u / rho` with `rho` the dry density is r_n (`scalar.cpp:71-72`, `r`). The transport flux is $F_n = F_{\mathrm{d}}\, r_n^{\mathrm{up}}$ with the upwind ratio, so a uniform ratio stays uniform under transport; under the implicit dry transfer the tracer increment is r_n times the dry increment (check line `[C3]`), so it stays uniform there too.

The upper bound uses the complement. If $r_n \le b$ then $b\rho_{\mathrm{d}} - \rho_{\mathrm{d}} r_n \ge 0$, and that quantity is transported by the flux $G = bF_{\mathrm{d}} - F_n$. Its non-negativity is exactly what the positivity limiter already enforces for a density, so the code applies the same limiter to G, which returns a factor $\Theta \in [0, 1]$ per face, and puts back only the change:

$$F_n \leftarrow F_n + (1 - \Theta)\,(bF_{\mathrm{d}} - F_n) = \Theta F_n + (1 - \Theta)\, bF_{\mathrm{d}}. \tag{9.6}$$

Equation 9.6 is applied as `h - g` in `ScalarImpl::forward` (`scalar.cpp:175-198`, `upper_bound`). The second form shows the limited flux is a blend between the tracer's own flux and the flux of a tracer sitting at the bound, so a plateau at b keeps advecting (`[C1]`, symbolic). Adding the change rather than recomputing $bF_{\mathrm{d}} - G$ leaves F_n bitwise unchanged on every face where $\Theta = 1$; recomputing perturbs nearly every face by round-off, 9994 of 10000 in a float32 test (`[C2]`). The bound inherits the positivity limiter's premise: it holds while the mass Courant number stays below one and ρ_{d} changes only by the transport divergence. The implicit vertical correction and any forcing that writes the dry row

break that premise, as the code comment states. Checks: `src/snapy_report/ch10/tracer_check.py`, output `tracer_check.out` [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0, ported line for line; deck: the check itself; run: `python3 tracer_check.py`, numpy and sympy, double and float32].

9.6.3. Numerical method

The ratio is reconstructed with the scalar block's own `reconstruct` and upwinded with the hydro dry-mass flux `flux1[IDN]` (and `flux2`, `flux3`); the Riemann solver must be `upwind`. With the EOS limiter on, the positivity limiter scales the tracer fluxes first; the complement limiter, if `upperbound` > 0, scales them second. The divergence then gives the stage tendency on interior cells only (`scalar.cpp:202-205`, `divergence`). The implicit dry transfer and the dry-source carry follow, as in Section 1.13. Figure 9.7 shows the donor choice of that transfer.

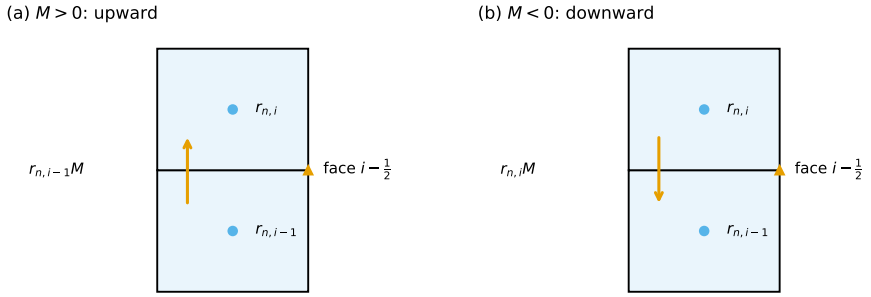


Figure 9.7.: Donor choice of the implicit tracer transfer M at one x1 face, between cells i-1 and i. Sky-blue dots are the cell ratios, the orange triangle the face, the orange arrow the direction of the dry transfer. (a) An upward transfer carries the ratio of the cell below; (b) a downward transfer carries the ratio of the cell above.

9.6.4. Code

Table 9.12.: Code map of the dry-air tracers; chapter 1 holds the rest of the scalar path (Table 1.23).

what	code link	symbol	switch
YAML keys	`scalar_options.cpp:11-50`	`from_yaml`	`scalar`
setup checks	`scalar.cpp:16-31`	`ScalarImpl::reset`	`upper-bound`
positivity limiter	`scalar.cpp:158-162`	`flux_positivity_theta`	`limiter`
complement bound	`scalar.cpp:175-198`	`upper_bound`	`upper-bound`
implicit transfer	`meshblock.cpp:661-676`	`mass_corr`	

1. Parse the keys; `upper-bound` defaults to -1 (`scalar_options.cpp:27`, `upper_bound`).
2. Refuse a solver other than `upwind`, a bound of 0, and a bound without the limiter (`scalar.cpp:20-31`, `TORCH_CHECK`).
3. Form the ratio per dry air (`scalar.cpp:71-72`, `rho`).
4. Limit for positivity (`scalar.cpp:158-162`, `theta`).
5. Limit the complement and add the change (`scalar.cpp:175-198`, `flux_positivity_scale_`).
6. Take the divergence on interior cells (`scalar.cpp:202-205`, `divergence`).

Setup messages a user can hit: "[Scalar] Passive scalars currently require the upwind solver", the refusal of `upper-bound: 0`, and "[Scalar] upper-bound needs the eos limiter".

9.6.5. Tests

Table 9.13.: Tests of the dry-air tracers; assertions read from the test sources at the pin, not run.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_scalar.release`	bound holds on both sides	10^{-12}; 10^{-5} f32	test thresholds
`test_tracer_dry_convention_python`	init, implicit and relax arms	10^{-12}	1000-fold non-vacuity
`test_forcing.release`	dry source keeps scalar bounds	10^{-12}	test threshold
`test_dry_carry_zero_base_python`	the 0/0 guard of the carry	10^{-12} abs.	test threshold

`scalar_upper_bound_holds_both_sides` advects 12 steps with the bound at 1 and requires $r_n \leq 1 + 10^{-12}$ and $\rho_{\mathrm{d}} r_n \geq -10^{-12}$ in float64 (10^{-5} in float32) (`test_scalar.cpp:159-163`, `tol`). `test_tracer_dry_convention` uses `TOL` $= 10^{-12}$ and also requires each arm's signal to exceed 1000 times `TOL`, so a pass is not vacuous (`test_tracer_dry_convention.py:32`, `TOL`). `test_stage_forcing_dry_tracer_python` and `test_flux_positivity_cubedsphere_python` (seams) are also registered. Measured results at the pin are **missing evidence** (Section 12.5).

9.6.6. Limits

> **Limits**
>
> In practice `reconstruct` is mandatory when `nvar` > 0. The upper bound holds only while the mass Courant number is below one and the dry density moves by transport alone.

- `ScalarOptions` has `thermo` and `kinetics` members that are never parsed or used.
- A uniform-tracer test cannot detect a wrong donor choice (docstring of `test_tracer_dry_convention.py`).
- Not in the switch coverage matrix (Section 11.8).

9.7. Sedimentation of condensates (`sedimentation`)

Local symbols: a_n is the particle radius of condensate n [m], $\rho_{\mathrm{p},n}$ its material density [$\mathrm{kg\,m^{-3}}$], β_n the Cunningham slip factor [-], λ the mean free path of the background gas [m], $\mathrm{Kn} = \lambda / a_n$ the Knudsen number [-], and m, d, ϵ the molecular mass [kg], diameter [m] and Lennard-Jones well depth [J] of the background gas; k_B is Boltzmann's constant, as in chapter 2. Tracked in ISSUES.md. μ is the dynamic viscosity of NOTATION.md section 8.

9.7.1. Summary

> **i** At a glance
>
> **What it does.** Lets each listed condensate settle at a terminal velocity, as a donor-cell x_1 flux of mass, momentum and energy added after the Riemann solver. **Switch.** Top-level YAML `sedimentation`; off when absent. **Requires.** `forcing/const-gravity`; inactive when $g_1 = 0$ or the x_1 flux is off. **Order.** First order in space. **Defining equation.** Equation 9.7.

Condensate particles fall through the gas at their terminal velocity. snapy adds that motion as an extra x_1 flux of the condensate's mass, the momentum it carries and its energy, on top of the advective flux the Riemann solver returns. The velocity is the slip-corrected Stokes velocity of a sphere in the background gas, or a prescribed constant. The flux is donor-cell, sealed at physical x_1 walls and open at internal block seams. Dry air does not move, so the dry-air tracers of Section 9.6 are unaffected.

It is on when the YAML card has a top-level `sedimentation` block listing cloud species, and the block refuses to build without `forcing/const-gravity`. Aliases: PR #216 (fused kernels, mean-free-path fix), PR #217 (removal of `fric-heat`), PR #258.

9.7.2. Derivation

Stokes drag on a sphere of radius a_n moving slowly through a gas of viscosity μ balances its buoyant weight at the velocity $2a_n^2 g_1(\rho_{\mathrm{p},n} - \rho)/(9\mu)$. When the mean free path of the gas is not small compared with the particle, the drag is reduced by the Cunningham factor $\beta_n = 1 + \mathrm{Kn}\,(1.256 + 0.4\,e^{-1.1/\mathrm{Kn}})$. Unless a non-zero `const-vsed` replaces it, the terminal velocity is

$$w_{\mathrm{s},n} = \frac{2\beta_n\, a_n^2\, g_1\,(\rho_{\mathrm{p},n} - \rho)}{9\,\mu}, \tag{9.7}$$

clamped to ±`upper-limit`. Equation 9.7 is computed in `SedVelImpl::forward` as `stokes` (`sed_vel.cpp:60-70`). The viscosity is the Chapman-Enskog value for a Lennard-Jones gas, $\mu = \frac{5}{16}\sqrt{m k_\mathrm{B} T/\pi}\,(k_\mathrm{B}T/\epsilon)^{0.16}/(1.22\,d^2)$, with molecular mass m, diameter d and well depth ϵ from the YAML defaults of an H_2 background, and the mean free path is $\lambda = (\mu/p)\sqrt{\pi k_\mathrm{B} T/(2m)}$ (`sed_vel.cpp:45-58`, `eta`). `sed_check.py` confirms the sign convention ($g_1 < 0$ and a dense particle settle, `[C1]`), the two limits of β_n (1 in the continuum, 1.656 Kn in the free-molecular limit, `[C2]`), that μ depends on T only (`[C3]`), and the clamp and the prescribed velocity (`[C4]`).

The flux through the face below cell i takes its donor from cell i when $w_{\mathrm{s},n} < 0$ and from cell $i-1$ when $w_{\mathrm{s},n} > 0$; faces at or beyond a physical x_1 wall carry nothing (`sed_hydro_dispatch.hpp:12-24`, `sedimentation_upwind`). The donor carries $\rho_n w_{\mathrm{s},n}$ of mass, $\mathbf{v}\,\rho_n w_{\mathrm{s},n}$ of momentum and $\rho_n w_{\mathrm{s},n}(u_n^\star + c_{v,n}T + \frac{1}{2}|\mathbf{v}|^2)$ of energy: the condensate's own internal energy plus kinetic energy, with no $R_n T$ share, because a condensate does no pressure work (`sed_hydro_impl.h:84-92`, `species_energy`). With both walls sealed the column's condensate mass changes only by what the advective flux does (`[C5]`). Checks: `src/snapy_report/ch10/sed_check.py`, output `sed_check.out` [sha: the formulas of snapy@e894700ff7aee30b52882e5202b16461413780b0 with kintera@c55b13b2204997d2d09e04498558ab9495d8ee77's Boltzmann constant, ported line for line; deck: the check itself; run: `python3 sed_check.py`, numpy, double].

9.7.3. Numerical method

The flux is added to `flux1` after the x_1 Riemann solve, and its condensate part is saved as `fsed1` so the positivity limiter can carry the right energy and momentum with it (`hydro_forward.cpp:418-422`, `fsed1`). With the

ideal-moist EOS and contiguous tensors a fused CPU or CUDA kernel computes velocity and flux in one pass; otherwise a tensor path does the same in separate operations; MPS has its own path (`sed_hydro.cpp:68-88`, `SedHydroImpl::forward`). The wall bounds are set only where the block face is a physical boundary, so seams between blocks stay open. Figure 9.8 shows a column; Figure 9.9 the velocity.

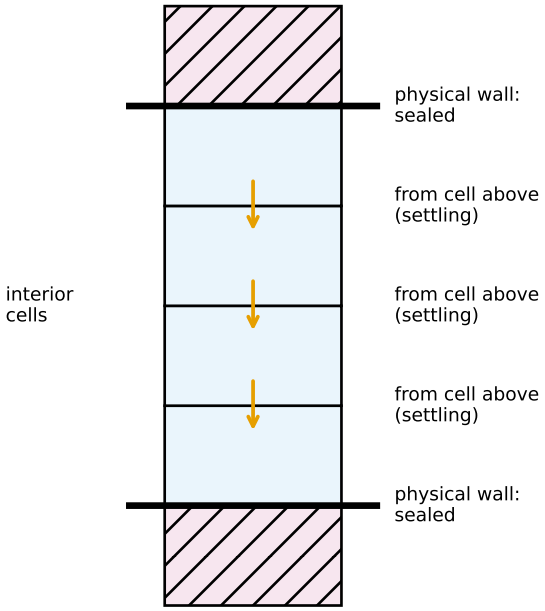


Figure 9.8.: Donor cells of the settling flux in one x1 column. Light-blue boxes are interior cells, hatched pink boxes ghosts, thick black lines physical walls. Orange arrows show settling fluxes, each taken from the cell above its face; the two wall faces carry no flux.

9.7.4. Code

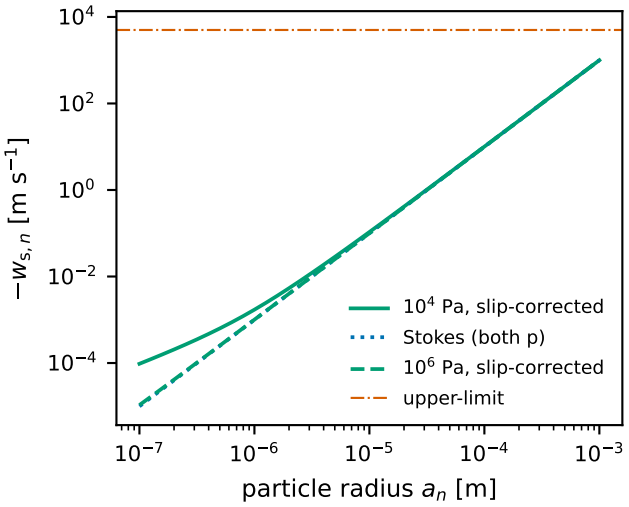


Figure 9.9.: Settling speed against particle radius for the H2 defaults at T = 150 K and g_1 = -24.79 m s^-2. Green solid: slip-corrected at 1e4 Pa; green dashed: at 1e6 Pa; blue dotted: plain Stokes, the same at both pressures; vermillion dash-dot: the default upper-limit. Small particles fall faster than Stokes at low pressure, where the mean free path is long.

Table 9.14.: Code map of sedimentation.

what	code link	symbol	switch
YAML keys	`sed_options.cpp:47-50`	`check_keys`	`sedimentation`
terminal velocity	`sed_vel.cpp:34-71`	`SedVelImpl::forward`	`const-vsed`
gravity required	`sed_hydro.cpp:54-58`	`SedHydroImpl::reset`	`const-gravity`
face kernel	`sed_hydro_impl.h:100-127`	`sedimentation_flux_impl`	
call site	`hydro_forward.cpp:418-422`	`fsed1`	

1. Parse the block and require cloud species (`sed_options.cpp:17-35`, `from_yaml`).
2. Read the gas constants and the clamp, with their defaults (`sed_options.cpp:120-123`, `upper_limit`).
3. Refuse a card without constant gravity (`sed_hydro.cpp:54-58`, `grav`).
4. Skip when $g_1 = 0$ or nothing settles, seal physical walls (`sed_hydro.cpp:73-83`, `is_physical_boundary`).
5. Choose the fused or the tensor path (`sed_hydro.cpp:85-88`, `sedimentation_flux_tensor`).
6. Add to `flux1` and save the condensate part (`hydro_forward.cpp:418-422`, `psed`).

Setup messages: “[SedHydro] sedimentation reads grav1; set forcing/const-gravity”, “Missing sedimentation particles in the clouds.”,

"Sedimentation radius must be positive." and "Sedimentation density must be positive."

9.7.5. Tests

Table 9.15.: Tests of sedimentation; assertions read from the test sources at the pin, not run.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_forcing.release`	fused equals tensor path; sealing	10^{-10} rel.; exact	test thresholds
`test_sedimentation_guards.release`	gravity required; rising from below	10^{-9} rel.	test threshold
`test_sedimentation_cubed_seam.release`	2-rank seam, mass conserved	10^{-12} rel.	test threshold
`test_two_cards_species.release`	velocity after a second card	10^{-10} (f64)	test threshold

The fused kernel must equal the tensor path to 10^{-10} relative and settle (negative flux) inside the column (`test_forcing.cpp:1407-1425`, `fused_sedimentation_matches_tensor_path`). A rising cloud must be taken from the cell below, mass and energy fluxes to 10^{-9} relative, on CPU and CUDA (`test_sedimentation_guards.cpp:80-83`, `EXPECT_NEAR`). On two ranks of a cubed sphere, the condensate mass is conserved to 10^{-12} relative and two seam sums are pinned to 10^{-7} and 5×10^{-3}

absolute (`test_sedimentation_cubed_seam.cpp:113-117`, `conserved`); the Gloo arm `test_sedimentation_cubed_seam_gloo` must reproduce the UCX result. Measured results at the pin are **missing evidence** (Section 12.5).

9.7.6. Limits

> **Limits**
>
> The time step has no sedimentation term: `HydroImpl::max_time_step` does not read $w_{s,n}$, so a fast settling speed can over-drain a cell, and only the positivity limiter catches it.

- The MPS path is untested (PR #216).
- Casting a block to float32 turns the index buffer into floats (PR #258).
- `fric-heat` was removed because it double-counted the face gravity work (PR #217; Chapter 6, Chapter 6).
- The default gas constants describe H_2; other backgrounds must set them.
- Not in the switch coverage matrix (Section 11.8).

10. Boundary conditions and immersed solids

This chapter distinguishes physical face callbacks, exchange-owned ghosts, and immersed-solid interfaces. It follows the existing stage map and configuration matrix, deriving boundary-specific parity, acoustic selection, admissibility, mask rectification and implicit closure. Source tests are catalogued separately from the report's executed formula checks; no solver runtime measurement is supplied here.

Figure 10.1 shows the verified local requirements. The complete switch matrix is Section 11.8; the stage ownership map is Section 1.13.

Table 10.1.: Boundary scheme map.

scheme	switch	default	order	main test
Section 10.7	`boundary-condition/external/*`	reflecting	Local identity; solver order unmeasured	Section 12.6
Section 10.6	`reflecting`	reflecting on external faces	Local identity; solver order unmeasured	Section 12.6

scheme	switch	default	order	main test
Section 10.5	`periodic`	off; external faces default reflecting	Local identity; solver order unmeasured	Section 12.6
Section 10.3	`extrapolation`	off; fallback for outflow auxiliary scalars and shallow water	Local identity; solver order unmeasured	Section 12.6
Section 10.4	`outflow`	off; external faces default reflecting	Local identity; solver order unmeasured	Section 12.6
Section 10.1	`custom; solid`	off	Local identity; solver order unmeasured	Section 12.6
Section 10.9	`apply_ boundaries; exchange_ ghost_ zones`	called by initialization and stage paths	Local identity; solver order unmeasured	Section 12.6
Section 10.2	`SyncOptions`	interpolate false; skip_corner true	Local identity; solver order unmeasured	Section 12.6
Section 10.8	`boundary- condition/ internal/ *; vars["solid"]`	max-iter 5; density 1e3; pressure 1e9	Local identity; solver order unmeasured	Section 12.6

scheme	switch	default	order	main test
Section 10.10	`dynamics/wb-wall-clamp`	true	Local identity; solver order unmeasured	Section 12.6

10.1. Reserve a face or fill a solid mask (`custom`; `solid`)

10.1.1. Summary

> **At a glance**
>
> The custom callbacks do nothing; solid callbacks write literal ones into a ghost slab. Their meanings depend on the field supplied. Default: off. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.1.2. Derivation

No PDE discretization is introduced. The custom map is the identity on the supplied tensor. The solid map replaces every ghost entry by one. Reapplying either map returns the same tensor, so both are idempotent. For a Boolean mask, one denotes solid; for a hydro state vector it is merely the number one in every row and is not a physical state construction. `custom_inner` (`bc_func.cpp:7-8`) and `solid_inner` (`bc_func.cpp:68-87`) define these operations directly. C1 verifies identity and constant-one ghost fills without assuming a thermodynamic meaning.

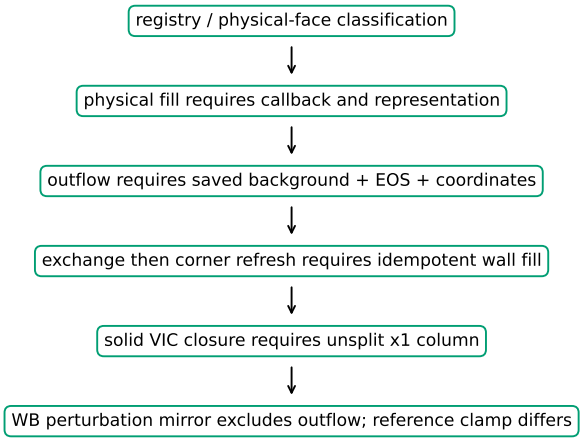


Figure 10.1.: Boundary dependencies and exclusions. Each labelled row states a local requirement; the sequence is a reading map, not a timing measurement.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.1.3. Numerical method

Custom keeps a non-null callback slot for a user or layout operation. Solid acts on all variables supplied and skips collapsed axes. Rectification applies solid callbacks to its mask before the directional sweeps, then optionally applies caller-supplied callbacks. No independent spatial accuracy order applies.

Figure 10.2 illustrates the operations, with labelled roles independent of colour.

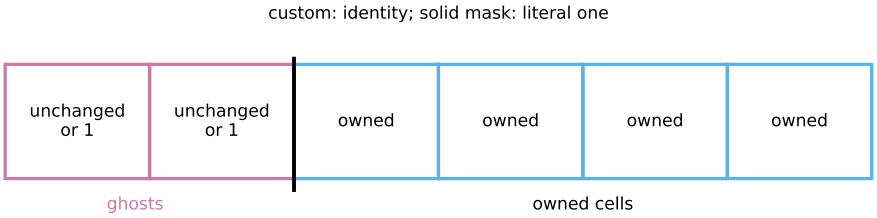


Figure 10.2.: Reserve a face or fill a solid mask. Schematic only; no simulation measurements.

10.1.4. Code

Table 10.2.: Implementation sites for custom.

what	code link	symbol	switch
custom	`bc_func.cpp:7-8`	`custom_inner`	See Summary
solidbc	`bc_func.cpp:68-87`	`solid_inner`	See Summary
rectify	`rectify_solid.cpp:109-168`	`rectify_solid`	See Summary
classify	`meshblock_options.cpp:218-271`	`is_wall_boundary`	See Summary

1. Leave the tensor unchanged for custom. The anchor `custom_inner` is in `bc_func.cpp:7-8`.
2. For solid, assign one to the requested ghost slab. The anchor `solid_inner` is in `bc_func.cpp:68-87`.
3. Use solid on a mask; do not infer a hydro wall from the function name. The anchor `rectify_solid` is in `rectify_solid.cpp:109-168`.

10.1.5. Tests

Table 10.3.: Source-defined assertions for custom; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_rectify.release`	Every inner and outer ghost slab remains solid	Exact equality to one	Test constant / assertion

The assertion anchor `test2` is in `test_rectify.cpp:91-124`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.1.6. Limits

A custom no-op does not supply missing physical ghost data. A non-null custom face still passes the physical-face predicate. Using solid on hydro writes literal ones; it does not construct a no-slip or thermodynamic wall. Caller-supplied rectification callbacks must be callable; the loop does not guard null entries.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.2. Supply ghosts through layout exchange (`SyncOptions`)

10.2.1. Summary

> **At a glance**
>
> Use layout exchange for seams and periodic neighbours. Cubed-sphere field exchange can interpolate and rotate vector components; raw limiter exchange avoids interpolation. Default: interpolate false; skip_corner true. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.2.2. Derivation

This is a pointer section: interpolation weights and coordinate transforms belong to the geometry chapter, and communication scheduling belongs to the parallel chapter. Here the directly implemented corner identity is the arithmetic average of two edge strips. For generic values q already in the destination frame,

$$q^{\mathrm{corner}} = \tfrac{1}{2}(q^{\mathrm{edge1}} + q^{\mathrm{edge2}}). \tag{10.1}$$

`fill_corners` (`layout.cpp:710-740`) implements Equation 10.1 by broadcasting adjacent strips. The weights sum to one, so constants are preserved exactly in real arithmetic; this does not prove a two-dimensional interpolation order. C4 checks this constant-preservation identity and records why nonconstant edge values need not reconstruct the corner value.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.2.3. Numerical method

For subdivided cubed-sphere panels, interpolated exchange first completes raw intra-panel exchange, then cross-panel exchange, and finally synthesizes skipped corners. This ensures the widened panel donor strip contains its tangential halo. `interp_ghost` widens the strip with `cs_interp_margin`; deserialization interpolates when requested and rotates conserved or primitive vector components. The hydro donor factor uses `interpolate(false)` with scalar type. This is a data-ownership map; no measured seam order is claimed.

Figure 10.3 illustrates the operations, with labelled roles independent of colour.

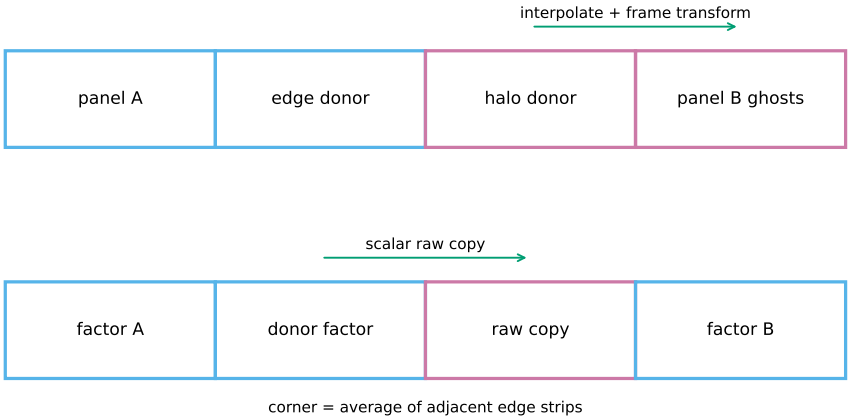


Figure 10.3.: Supply ghosts through layout exchange. Schematic only; no simulation measurements.

10.2.4. Code

Table 10.4.: Implementation sites for exchange.

what	code link	symbol	switch
sync	`layout.hpp: 149-154`	`skip_corner`	See Summary
advance	`meshblock.cpp: 545-572`	`exchange_ ghost_zones`	See Summary
panel	`cubed_sphere_ layout.cpp: 922-956`	`interp_ghost`	See Summary
interp	`gnomonic_ equiangle.cpp: 245-264`	`interp_ghost`	See Summary
average	`layout.cpp: 710-740`	`fill_corners`	See Summary
raw	`hydro_ forward.cpp: 673-680`	`interpolate`	See Summary
wrap	`slab_ layout.cpp: 41-70`	`periodic_x`	See Summary

1. Read sync flags; defaults do not interpolate and do skip corners. The anchor `skip_corner` is in `layout.hpp:149-154`.
2. For the subdivided interpolated panel path, finish intra-panel exchange before packing cross-panel donors. The anchor `exchange_ ghost_zones` is in `meshblock.cpp:545-572`.
3. Deserialize, interpolate when selected, transform vector components, and fill synthetic corners when requested. The anchor `interp_ghost` is in `cubed_sphere_layout.cpp:922-956`.
4. Keep raw scalar donor-factor exchange separate from interpo-

lated hydro exchange. The anchor `interpolate` is in `hydro_forward.cpp:673-680`.

10.2.5. Tests

Table 10.5.: Source-defined assertions for exchange; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_cubed_sphere_exchange.release`	Subdivided fields match one-block panel	Exact equality	Test constant / assertion
`test_mesh_exchange_python`	Received scalar identifies neighbour	Exact equality	Test constant / assertion

The assertion anchor `torch::equal` is in `test_cubed_sphere_exchange.cpp:161-196`.

The assertion anchor `assert` is in `test_mesh_exchange.py:24-49`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.2.6. Limits

The corner average is not the panel-edge interpolation formula. CPU/CUDA equality assertions in the panel suite do not establish multi-process coverage. The vertical-velocity exchange suite checks its

own `ideal_match` criterion (`test_cubed_sphere_vertical_velocity_exchange.py:81-109`); no result is claimed without a run. Broader seam flux and positivity coverage belongs to the geometry and switch-matrix chapters.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.3. Repeat the nearest owned cell (`extrapolation`)

10.3.1. Summary

> **At a glance**
>
> Fill every ghost layer with the same boundary-adjacent owned cell. This is the zero-gradient copy condition. Default: off; fallback for outflow auxiliary scalars and shallow water. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.3.2. Derivation

For a smooth field with a prescribed zero normal derivative, a constant extension supplies ghost values. The discrete rule is the nearest owned-cell average repeated at every ghost depth:

$$q_{i-l} = q_i. \tag{10.2}$$

At the lower wall i is the first owned index and $l = 1, \ldots, n_g$; the upper rule uses the last owned cell. `extrapolation_inner` (`bc_func.cpp:52-66`) broadcasts a width-one slab, implementing Equation 10.2. For a general smooth field, Taylor expansion between the donor and ghost centroids gives a leading difference proportional to their separation times the normal derivative, hence $O(h)$ at fixed depth on a uniform grid. This is a consistency estimate for ghost values, not a measured solver convergence rate. C1 checks exact constant preservation and idempotence.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.3.3. Numerical method

Collapsed axes are skipped. Primitive, conserved and scalar calls all use the same copy. `radiating` selects this fallback before validating EOS and reference context for auxiliary `kScalar` fields or shallow water. Thus an auxiliary limiter field does not undergo the acoustic split. This callback runs with physical fills, including the supported corner refresh.

Figure 10.4 illustrates the operations, with labelled roles independent of colour.

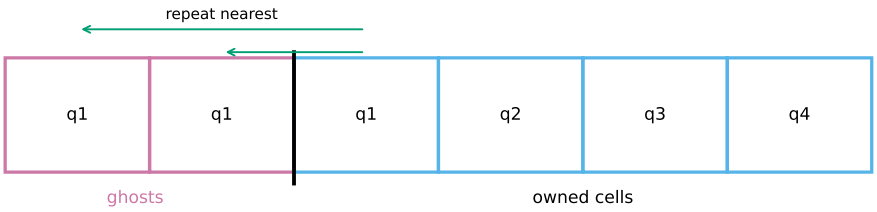


Figure 10.4.: Repeat the nearest owned cell. Schematic only; no simulation measurements.

10.3.4. Code

Table 10.6.: Implementation sites for extrapolation.

what	code link	symbol	switch
extrap	`bc_func.cpp:52-66`	`extrapolation_inner`	See Summary
radiating	`bc_func.cpp:108-133`	`radiating`	See Summary

1. Select the first or last owned width-one slab. The anchor `extrapolation_inner` is in `bc_func.cpp:52-66`.
2. Broadcast it to the whole ghost slab without changing owned cells. The anchor `extrapolation_inner` is in `bc_func.cpp:52-66`.
3. Use the same operation for the early scalar/shallow-water outflow fallback. The anchor `radiating` is in `bc_func.cpp:108-133`.

10.3.5. Tests

Table 10.7.: Source-defined assertions for extrapolation; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_radiating_boundary.release`	Acoustic pulse comparison arm	No assertion on extrapolation arm	Test constant / assertion

The assertion anchor `EXPECT_LT` is in `test_radiating_boundary.cpp:395-441`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.3.6. Limits

Zero gradient is not an acoustic nonreflection condition. The pulse test prints an extrapolation comparison but only asserts the outflow arm, so no measured reflection fraction for extrapolation is claimed.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.4. Select outgoing acoustic and advected perturbations (`outflow`)

Local symbols: $\mathbf{W}^{\mathrm{bg}}$ denotes the saved initial boundary background, which is distinct from the well-balanced reference. v_{out} is outward velocity $[\mathrm{m\,s^{-1}}]$; Z_{ac} is acoustic impedance $[\mathrm{kg\,m^{-2}\,s^{-1}}]$; a_+ and a_- are acoustic perturbations $[\mathrm{m\,s^{-1}}]$; d_{ent} is the advected density combination $[\mathrm{kg\,m^{-3}}]$; α_{bc} is a dimensionless admissibility factor. All are proposed in the chapter's notation issue. A prefix δ below means subtraction of this background, not the centroid offset δ_i.

10.4.1. Summary

> **At a glance**
>
> Build primitive ghosts from the saved initial background plus outgoing perturbations. A shared factor keeps hydro composition and tracers admissible. Default: off; external faces default reflecting. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.4.2. Derivation

Freeze an admissible background and an outward Cartesian face frame. Linearized mass, normal momentum and pressure equations have material derivatives proportional respectively to minus background density, minus inverse background density and minus background density times squared sound speed, all multiplying the appropriate normal derivative. For the outer x1 face, write the frozen material derivative as $\mathrm{D}/\mathrm{D}t = \partial_t + v_{\mathrm{out}}^{\mathrm{bg}}\partial_1$. The three equations are

$$\begin{aligned}
\frac{\mathrm{D}\delta\rho}{\mathrm{D}t} &= -\rho^{\mathrm{bg}}\partial_1\delta v_{\mathrm{out}},\\
\frac{\mathrm{D}\delta v_{\mathrm{out}}}{\mathrm{D}t} &= -\frac{1}{\rho^{\mathrm{bg}}}\partial_1\delta p,\\
\frac{\mathrm{D}\delta p}{\mathrm{D}t} &= -\rho^{\mathrm{bg}}(c_s^{\mathrm{bg}})^2\partial_1\delta v_{\mathrm{out}}.
\end{aligned}$$

Eliminating the pressure and velocity coupling gives the combinations

$$\begin{aligned}
Z_{\mathrm{ac}} &= \rho^{\mathrm{bg}}c_s^{\mathrm{bg}},\\
a_{\pm} &= \delta v_{\mathrm{out}} \pm \delta p/Z_{\mathrm{ac}},\\
d_{\mathrm{ent}} &= \delta\rho - \delta p/(c_s^{\mathrm{bg}})^2.
\end{aligned} \tag{10.3}$$

Adding/subtracting the linearized momentum equation and the pressure equation divided by impedance yields transport of $a_{\pm}$ at the two acoustic speeds; subtracting the pressure equation divided by squared sound speed from continuity cancels its divergence term and leaves advection of d_{ent}. Integrating each constant-coefficient transport equation over a boundary-adjacent cell gives its incoming minus outgoing face transport. A characteristic entering from outside has no interior-determined perturbation: the boundary chooses zero perturbation for that incoming mode. The code uses the *current* outward velocity and sound speed for this selector, while Equation 10.3 uses the background impedance and sound speed. Thus it is a locally linear characteristic prescription, not the exact nonlinear exterior solution.

Keep a_+ when $v_{\mathrm{out}} + c_s \geq 0$, keep a_- when $v_{\mathrm{out}} - c_s \geq 0$, and otherwise set each to zero. Keep d_{ent}, tangential velocity, composition and tracer-ratio perturbations when $v_{\mathrm{out}} \geq 0$; otherwise set them to zero. Equality selects the outgoing branch. Invert the split:

$$\begin{aligned} \delta v_{\mathrm{out}} &= (a_+ + a_-)/2, \\ \delta p &= Z_{\mathrm{ac}}(a_+ - a_-)/2, \\ \delta \rho &= d_{\mathrm{ent}} + \delta p/(c_s^{\mathrm{bg}})^2. \end{aligned} \tag{10.4}$$

`plus`, `minus` and `entropy` (`bc_func.cpp:135-165`) implement both Equation 10.3 and Equation 10.4. For the lower face, reversing the coordinate normal before splitting and reversing it back after inversion accounts for the sign in the velocity update. The same selected interior perturbation is supplied to every ghost, but each ghost retains its own saved background.

For any lower-bounded component q with floor $q_{\min}$, admissibility requires $q^{\mathrm{bg}} + \alpha_{\mathrm{bc}}\delta q \geq q_{\min}$. If $\delta q < 0$, division gives the bound $(q^{\mathrm{bg}} - q_{\min})/(-\delta q)$; otherwise that component imposes no reduction. Start with one, take the minimum of all bounds, multiply each binding bound below one by 0.99, and clamp the result to $[0, 1]$. Apply this to density, pressure, every y_n, the

complement $1 - \sum_n y_n$, and every passive tracer ratio r_n. Nonnegative complement enforces the composition sum. The same factor multiplies every perturbation, including tracers:

$$\mathbf{W}^{\text{ghost}} = \mathbf{W}^{\text{bg,ghost}} + \alpha_{\text{bc}} \delta \mathbf{W}. \tag{10.5}$$

`alpha` (`bc_func.cpp:167-207`) implements Equation 10.5. Convexity of these linear inequalities proves admissibility for an admissible background in exact arithmetic. It proves neither a positive temperature for every EOS nor nonlinear nonreflection. C2 checks the invertible acoustic algebra; C3 verifies the shared limiter against independent inequality bounds, including a species-complement and tracer-limited case.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.4.3. Numerical method

Capture the background before the first physical fill, including initialized ghost values. Transform both current and saved velocities to the face frame, compute a perturbation from the nearest owned cell, select outgoing modes, limit each ghost, then transform back. The operator writes only ghosts. Conserved calls go through `apply_boundaries`: consecutive outflow faces share a primitive conversion; before another callback, pending faces are converted and flushed. Tracer densities use dry density, not primitive total density. The cartoon shows mode directions and the convex contraction separately. No coupled wall/seam convergence or reflection measurement is reported here.

Figure 10.5 illustrates the operations, with labelled roles independent of colour.

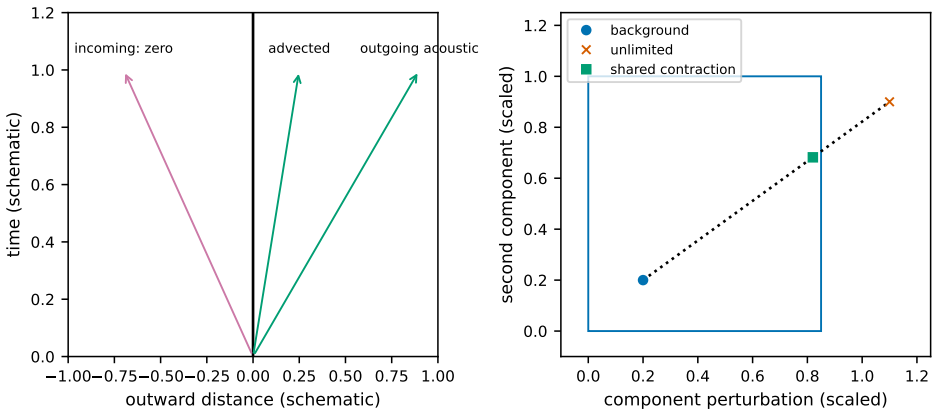


Figure 10.5.: Select outgoing acoustic and advected perturbations. Schematic only; no simulation measurements.

10.4.4. Code

Table 10.8.: Implementation sites for outflow.

what	code link	symbol	switch
init	`meshblock.cpp: 401-431`	`boundary_ reference_w`	See Summary
background	`bc_func.cpp: 92-106`	`check_ background`	See Summary
radiating	`bc_func.cpp: 108-133`	`radiating`	See Summary
split	`bc_func.cpp: 135-165`	`plus`	See Summary
alpha	`bc_func.cpp: 167-207`	`alpha`	See Summary
outflow	`bc_func.cpp: 210-218`	`is_outflow`	See Summary

what	code link	symbol	switch
apply	`meshblock.cpp:1441-1519`	`apply_boundaries`	See Summary
fixer	`meshblock.cpp:850-907`	`solid`	See Summary

1. Use the scalar/shallow-water fallback first. Otherwise require EOS, coordinates and a saved same-shape reference. The anchor `radiating` is in `bc_func.cpp:108-133`.
2. Require a supported EOS, primitive type, finite input, positive finite sound speeds, and admissible background density, pressure and composition. The anchor `check_background` is in `bc_func.cpp:92-106`.
3. Split in the face frame, select modes using current speeds, and reconstruct the selected perturbation. The anchor `plus` is in `bc_func.cpp:135-165`.
4. Validate tracer background shape, finiteness and nonnegativity; reduce the shared factor and verify reconstructed ghosts are finite. The anchor `alpha` is in `bc_func.cpp:167-207`.
5. Flush conserved pending faces in face order using dry-density tracer conversion. The anchor `apply_boundaries` is in `meshblock.cpp:1441-1519`.

10.4.5. Tests

Table 10.9.: Source-defined assertions for outflow; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_radiating_boundary.release`	Modes on all faces and three EOS types	Float32 0.03; float64 1e-8	Test constant / assertion
`test_radiating_boundary.release`	Mixed-face representation and order	Hydro rtol 1e-12, atol 1e-9	Test constant / assertion
`test_radiating_boundary.release`	Dry-air tracer ratios	Below 1e-12	Test constant / assertion
`test_radiating_boundary.release`	PLM/WENO5 pulses, both signs	Reflected fraction below 0.05	Test constant / assertion
`test_radiating_boundary.release`	Joint limiter and restart	Inequalities; exact restart equality	Test constant / assertion
`test_radiating_boundary.release`	CPU/CUDA agreement	rtol = atol = 1e-10	Test constant / assertion
`test_radiating_boundary_python`	Manual fill preserves reference and interior	Exact equality	Test constant / assertion

The assertion anchor `EXPECT` is in `test_radiating_boundary.cpp:46-98`.

The assertion anchor `EXPECT` is in `test_radiating_boundary.cpp:177-259`.

The assertion anchor `EXPECT` is in `test_radiating_boundary.cpp:288-360`.

The assertion anchor `EXPECT_LT` is in `test_radiating_boundary.cpp:395-441`.

The assertion anchor `alpha` is in `test_radiating_boundary.cpp:536-561`.

The assertion anchor `allclose` is in `test_radiating_boundary.cpp:563-578`.

The assertion anchor `assert` is in `test_radiating_boundary_python.py:16-36`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.4.6. Limits

Outflow is skipped by the tangential corner refresh (`meshblock.cpp:951-978`). The gravity-work fixer is not an open-boundary energy budget: its x1 mass-crossing check can reject an open boundary; the error directs users to `gravity-work-fixer: false` (`meshblock.cpp:850-907`). The implicit closure remains reflecting in the cited operator, so a characteristic ghost fill does not establish a characteristic implicit boundary. The test's restart continuation and missing-reference rejection are explicit in `restart_continuation_and_missing_reference` (`test_radiating_boundary.cpp:445-468`). Saved backgrounds are fixed initialization data; adaptive exterior evolution is not implemented by this callback.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.5. Wrap values through a callback or the layout (`periodic`)

10.5.1. Summary

> **At a glance**
>
> Use an in-block opposite-side copy when a callback remains, or a wrapped neighbour exchange when the layout owns periodicity. Default: off; external faces default reflecting. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.5.2. Derivation

For a periodic field, translation by the domain length leaves its cell integral unchanged. On matched cells the discrete boundary operation is therefore an exact permutation of cell averages, not an interpolation. With owned cells $1, \dots, n_1$ and ghost depth l, the x1 callback gives

$$\begin{aligned} q_{1-l} &= q_{n_1+1-l}, \\ q_{n_1+l} &= q_{l}. \end{aligned} \tag{10.6}$$

`periodic_inner` and `periodic_outer` (`bc_func.cpp:36-50`) implement Equation 10.6. For several blocks, the donor is the wrapped neighbour selected by `neighbor_rank` (`slab_layout.cpp:41-70`), and the same identity applies to the global cell index. This proves exact copying on a matching periodic mesh, not the conservation of independently reconstructed seam fluxes. C1 checks both callback copies and repeated fills.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.5.3. Numerical method

The layout flag `periodic_x` refers to x2, `periodic_y` to x3, and `periodic_z` to x1. YAML sets the first two for periodic x2/x3; x1 sets the layout flag only for layout type `cubed`. A slab x1 periodic callback therefore copies inside the block. Layout-owned wraps have null callbacks and are filled at exchange time. No independent convergence order is associated with this exact permutation.

Figure 10.6 illustrates the operations, with labelled roles independent of colour.

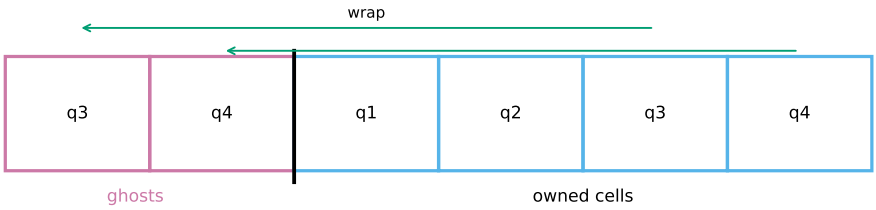


Figure 10.6.: Wrap values through a callback or the layout. Schematic only; no simulation measurements.

10.5.4. Code

Table 10.10.: Implementation sites for periodic.

what	code link	symbol	switch
periodic	`bc_func.cpp:36-50`	`periodic_inner`	See Summary
yaml	`meshblock_options.cpp:88-127`	`periodic_z`	See Summary
yamlxy	`meshblock_options.cpp:139-201`	`periodic_x`	See Summary
null	`meshblock.cpp:138-166`	`bfuncs`	See Summary
wrap	`slab_layout.cpp:41-70`	`periodic_x`	See Summary

1. Parse periodic face names and set the appropriate layout flag. The anchor `periodic_x` is in `meshblock_options.cpp:139-201`.
2. Null layout-owned face callbacks; retain the x1 slab callback. The anchor `bfuncs` is in `meshblock.cpp:138-166`.
3. Copy the opposite owned slab or obtain the wrapped neighbour through the layout. The anchor `periodic_inner` is in `bc_func.cpp:36-50`.

10.5.5. Tests

Table 10.11.: Source-defined assertions for periodic; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_mesh_exchange_python`	Neighbour identity in exchanged scalar ghosts	Exact equality	Test constant / assertion

The assertion anchor `assert` is in `test_mesh_exchange.py:24-49`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.5.6. Limits

In-block copy and neighbour exchange are distinct paths. The outline also inventories the multi-rank exchange and positivity seam suites; their runtime coverage remains pending in Section 12.6. A copied donor factor must stay single-valued across its shared face; this section does not replace the positivity analysis.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.6. Mirror a reflecting wall (`reflecting`)

10.6.1. Summary

> **At a glance**
>
> Mirror cell values across the wall and reverse the normal momentum or velocity row. Scalars retain even parity. Default: reflecting on external faces. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.6.2. Derivation

Use only cell averages here. At a Cartesian lower wall let the first owned cell have index i and let $l = 1, \dots, n_g$ count ghost depth. Integrating an even extension of a scalar over each reflected cell gives its matching owned-cell average. Integrating an odd extension of normal velocity gives the negative average. For an orthogonal wall, density is even and normal momentum is odd, so their ratio has the same odd parity. Thus the callback implements

$$\begin{aligned} q_{i-l} &= q_{i+l-1}, \\ v_{1,i-l} &= -v_{1,i+l-1}. \end{aligned} \tag{10.7}$$

Here q is any even component and v_1 has units $\mathrm{m\,s^{-1}}$. The upper-wall mapping reverses the last owned slab in the same way. `reflecting_inner` (`bc_func.cpp:10-34`) implements Equation 10.7 with a flip and a sign change of row `4-dim`. At a symmetry-preserving Riemann interface, equal densities and opposite normal velocities give zero normal mass flux by reflection symmetry. This argument does not supply a hydrostatic

pressure continuation or a proof for arbitrary nonorthogonal component conventions. C1 verifies the exact discrete parity and idempotence.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.6.3. Numerical method

The input remains in its supplied representation. Conserved and primitive calls negate the designated normal row; `kScalar` only mirrors. Collapsed directions return immediately. The fill precedes reconstruction through the stage ordering in Section 10.9. Odd/even extension is exact for symmetric data; no measured wall convergence order is available here.

Figure 10.7 illustrates the operations, with labelled roles independent of colour.

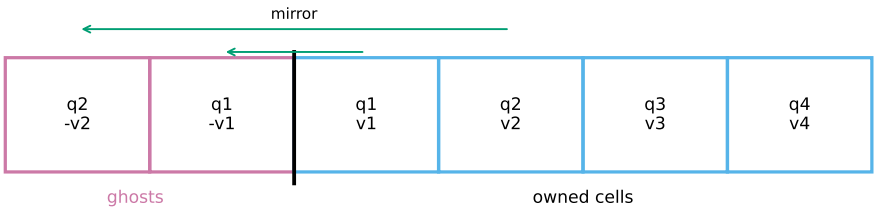


Figure 10.7.: Mirror a reflecting wall. Schematic only; no simulation measurements.

10.6.4. Code

Table 10.12.: Implementation sites for reflecting.

what	code link	symbol	switch
reflect	`bc_func.cpp:10-34`	`reflecting_inner`	See Summary
context	`bc.hpp:19-42`	`BoundaryFuncOptions`	See Summary
corner	`meshblock.cpp:951-978`	`refresh`	See Summary

1. Return for a collapsed direction, then copy and reverse the adjacent owned slab. The anchor `reflecting_inner` is in `bc_func.cpp:10-34`.
2. Negate the normal row for conserved or primitive input; retain even parity for scalar input. The anchor `reflecting_inner` is in `bc_func.cpp:10-34`.
3. Repeat on exchange-owned tangential corner slabs when requested by the block exchange path. The anchor `refresh` is in `meshblock.cpp:951-978`.

10.6.5. Tests

Table 10.13.: Source-defined assertions for reflecting; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wb_wall_corner.release`	Rest at stock and custom walls	Mach at most 1e-12	Test constant / assertion

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wall_saturation.release`	Energy and water conserved during phase change	Relative error below 1e-12	Test constant / assertion

The assertion anchor `EXPECT` is in `test_wb_wall_corner.cpp:166-190`.

The assertion anchor `1.e-12` is in `test_wall_saturation.cpp:59-89`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.6.6. Limits

A mirror of the total state is not a continuation of a stratified equilibrium. The reference-state procedure in Section 10.10 addresses that distinction. Reflection alone does not establish the accuracy of wall diffusion coefficients or nonorthogonal momentum treatment.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.7. Register and classify block faces (`boundary-condition/external/*`)

10.7.1. Summary

> **At a glance**
>
> Face callbacks fill physical ghosts; layout exchange supplies internal and wrapped faces. Classification depends on the predicate used, not only on the boundary label. Default: reflecting. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.7.2. Derivation

This is dispatch infrastructure, so there is no differential equation to discretize. A face is physical exactly when its callback exists. A diffusion wall is a physical face with a recorded reflecting or fixed-temperature name. The hydro predicate `is_x1_wall` instead accepts physical x1 faces whose names do not start with `periodic`. These are different sets: an outflow face passes the hydro predicate but not the diffusion whitelist. This follows directly from `is_wall_boundary` (`meshblock_options.cpp:218-271`) and `is_x1_wall` (`hydro.cpp:204-209`).

A repeated fill must reproduce the same ghost values from the owned slab. This is an idempotence contract, not a guarantee for arbitrary callbacks: the registry comment requires it because a callback also runs on a thin tangential ghost slab (`BC_FUNCTION`, `bc_func.hpp:14-39`). Check C1 exercises this contract for the built-in mirror and copy rules; no spatial order is assigned to dispatch.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.7.3. Numerical method

The face order is x1 inner/outer, x2 inner/outer, x3 inner/outer. In a four-dimensional hydro tensor these map to dimensions 3, 2 and 1. Context carries EOS, coordinates, references, representation and ghost width. Construction nulls exchange-owned callbacks for slab/cubed layouts; the cubed-sphere branch is separate. A non-null custom callback therefore still counts as physical. No accuracy order follows from classification.

Figure 10.8 illustrates the operations, with labelled roles independent of colour.

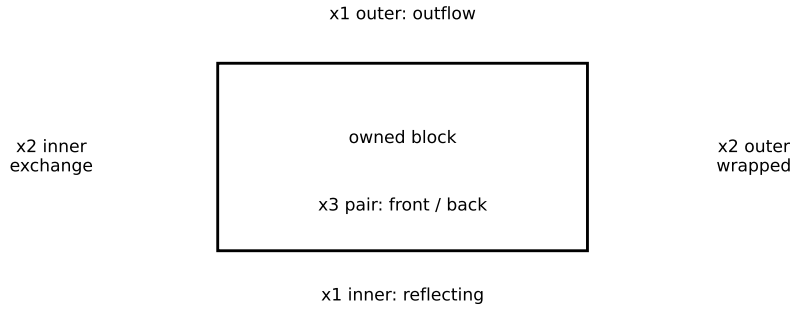


Figure 10.8.: Register and classify block faces. Schematic only; no simulation measurements.

10.7.4. Code

Table 10.14.: Implementation sites for registry.

what	code link	symbol	switch
registry	`bc_func.hpp:14-39`	`BC_FUNCTION`	See Summary
context	`bc.hpp:19-42`	`BoundaryFuncOptions`	See Summary
yaml	`meshblock_options.cpp:88-127`	`periodic_z`	See Summary
yamlxy	`meshblock_options.cpp:139-201`	`periodic_x`	See Summary
classify	`meshblock_options.cpp:218-271`	`is_wall_boundary`	See Summary
null	`meshblock.cpp:138-166`	`bfuncs`	See Summary
x1wall	`hydro.cpp:204-209`	`is_x1_wall`	See Summary

1. Validate YAML keys and append `_inner` or `_outer`; reject names absent from the registry. The anchor `periodic_z` is in `meshblock_options.cpp:88-127`.
2. Set layout periodic flags, then null callbacks owned by exchange during construction. The anchor `bfuncs` is in `meshblock.cpp:138-166`.
3. Classify at the consuming operator: unnamed programmatic callbacks warn once and return false for the diffusion wall test. The anchor `is_wall_boundary` is in `meshblock_options.cpp:218-271`.

10.7.5. Tests

Table 10.15.: Source-defined assertions for registry; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_yaml_keys.release`	Unknown key identifies its path	Exact string containment	Test constant / assertion
`test_wb_wall_corner.release`	Stock and custom-named wall keep rest	Mach at most 1e-12	Test constant / assertion

The assertion anchor `EXPECT` is in `test_yaml_keys.cpp:37-64`.

The assertion anchor `EXPECT` is in `test_wb_wall_corner.cpp:166-190`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.7.6. Limits

The fixed-temperature names occur in the whitelist, but no built-in registration is present in the boundary implementation inspected here. They do not establish an available thermal BC. The comment calling outflow zero-gradient in `is_wall_boundary` is stale relative to the characteristic implementation (`meshblock_options.cpp:218-271`, `bc_func.cpp:108-133`). Cubed-sphere custom panel faces require care wherever a physical-face test is used.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.8. Rectify a mask and close fluid faces against solids (`boundary-condition/internal/*`; `vars["solid"]`)

Local symbols: $D_i(b, l)$ is the minimum flip count after processing i mask entries, ending with bit b and capped run length l; m_b is its required run length (counts). Bits and $\mathrm{cost}_i(b)$, the candidate flip cost at position i, are dimensionless. H is the dimensionless reflection matrix, distinct from the Riemann eigenvector matrix R. These symbols are proposed in the notation issue.

10.8.1. Summary

> **i** At a glance
>
> A static mask selects solid cells. Directional minimum-flip sweeps repair thin runs; mirrored face states and masked implicit rows isolate the fluid segments. Default: max-iter 5; density 10^3 kg m^{-3}; pressure 10^9 Pa. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.8.2. Derivation

At a single fluid-solid interface, copy the fluid reconstructed state to the solid side and negate its normal velocity. This yields the reflecting pair derived in Section 10.6, now at a face of a staircase mask. `InternalBoundaryImpl::forward` (`internal_boundary.cpp:47-95`) applies that rule after reconstructing the fluid side. A face with solid on both sides has no fluid flux interpretation in this argument.

The rectification problem on one line minimizes the count of changed zeros. Set $m_0 = 3$ and $m_1 = 2$. A candidate equal to the original bit costs zero; changing zero to one costs one; changing one to zero has infinite cost. Start with $D_0(0,0) = D_0(1,0) = 0$ and all other states infinite. Extending the current bit increments its run length, capped at m_b. Switching to the other bit starts length one only if the previous length is zero (the start) or at least m_b. For every permitted predecessor, update the candidate state by the minimum of its existing cost and predecessor cost plus the flip cost:

$$D_{i+1}(b,l) \leftarrow \min\bigl(D_{i+1}(b,l), D_i(b_{\mathrm{old}}, l_{\mathrm{old}}) + \mathrm{cost}_i(b)\bigr). \tag{10.8}$$

`compute_min_flips` (`flip_zero_impl.h:24-198`) implements Equation 10.8 and retains the predecessor only on a strict improvement. The terminal run must also reach its minimum. Every admissible prefix ends in one of these states; induction on prefix length shows the recurrence retains the smallest possible cost. Backtracking returns a minimizing line. This proves a line optimum, not a global multidimensional optimum after alternating sweeps. `flip_zero_cpu` (`bc_dispatch.cpp:17-56`) sums line costs atomically. C5 compares an independent dynamic program with exhaustive enumeration for every binary line of lengths two through eight.

For the partial implicit system, $\mathsf{H} = \mathrm{diag}(1,-1,1)$ on density, normal momentum and energy rows. A reflecting ghost increment is $\mathsf{H}\Delta\mathbf{U}_i$. Substitution into the lower neighbour block contributes $\mathsf{B}_i\mathsf{H}\Delta\mathbf{U}_i$ to the diagonal; the upper side contributes $\mathsf{C}_i\mathsf{H}\Delta\mathbf{U}_i$. Thus

$$\begin{aligned} \mathsf{A}_i &\leftarrow \mathsf{A}_i + \mathsf{B}_i\mathsf{H}, \\ \mathsf{A}_i &\leftarrow \mathsf{A}_i + \mathsf{C}_i\mathsf{H}. \end{aligned} \tag{10.9}$$

Apply each row only at its relevant fluid boundary. `Bnd` (`vic_assemble_partial_impl.h:138-144`) implements Equation 10.9; `solid_lower`

(`vic_assemble_partial_impl.h:43-65`) also mirrors the base state used in the face Jacobian. A solid row has diagonal identity divided by Δt and zero neighbours (`setIdentity`, `implicit_dispatch.cpp:56-69`). The masked RHS is zero (`forward_masked`, `hydro.cpp:411-438`), so that row has zero solved increment. C6 verifies the block substitution algebra. The full-system closure has an additional nonperiodic guard (`vic_assemble_full_impl.h:123-126`); this derivation does not assert an untested periodic-solid combination.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.8.3. Numerical method

The mask shape is the three spatial extents. Rectification makes it contiguous (which may alias an already contiguous input), fills all resolved ghost slabs with ones, sweeps x1/x2/x3 and alternates traversal direction until a sweep has no flips. Exhausting `max-iter` throws. Optional callbacks apply afterward.

Initialization saves separate primitive/conserved fill states with velocity or momentum zeroed. Primitive placeholders use the configured density and pressure, zero velocity and zero composition. During a stage, face states mirror at solid interfaces; the time-step sound speed in solid cells is replaced by $10^{-8}\ \mathrm{m\,s^{-1}}$. The implicit solve masks solid increments and requires an unsplit x1 column. After RK combination only hydro conserved cells are refilled; the gravity-work fixer excludes solid mass. Staircase geometry is cell-aligned, not a cut-cell representation, and no geometric convergence order is measured.

Figure 10.9 illustrates the operations, with labelled roles independent of colour.

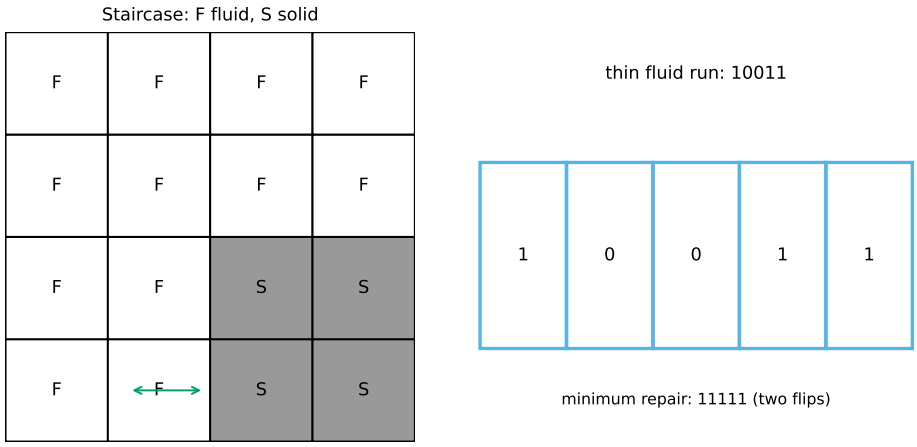


Figure 10.9.: Rectify a mask and close fluid faces against solids. Schematic only; no simulation measurements.

10.8.4. Code

The sweep calls `mark_prim_solid_` in `hydro_forward.cpp:215-219` and the `pib` face mirror in x1 `hydro_forward.cpp:384-387`, x2 `hydro_forward.cpp:592-595` and x3 `hydro_forward.cpp:614-617`.

Table 10.16.: Implementation sites for solids.

what	code link	symbol	switch
internal	`internal_boundary.cpp:25-35`	`max_iter`	See Summary
mark	`internal_boundary.cpp:47-95`	`mark_prim_solid_`	See Summary

what	code link	symbol	switch
rectify	rectify_solid.cpp:109-168	rectify_solid	See Summary
dispatch	bc_dispatch.cpp:17-56	compute_min_flips	See Summary
dp	flip_zero_impl.h:24-198	minRun0	See Summary
backtrack	flip_zero_impl.h:202-267	reconstruct	See Summary
finalinit	meshblock.cpp:454-500	fill_solid_hydro_u	See Summary
dt	hydro.cpp:311-324	solid	See Summary
masked	hydro.cpp:411-438	forward_masked	See Summary
solidrow	implicit_dispatch.cpp:56-69	setIdentity	See Summary
closure	vic_assemble_partial_impl.h:43-65	solid_lower	See Summary
closurediag	vic_assemble_partial_impl.h:138-144	Bnd	See Summary
fullclosure	vic_assemble_full_impl.h:123-126	Bnd	See Summary
refill	meshblock.cpp:782-842	apply_boundaries	See Summary

what	code link	symbol	switch
fixer	`meshblock.cpp:850-907`	`solid`	See Summary

1. Read max iterations and solid placeholders; require the mask shape at initialization. The anchor `max_iter` is in `internal_boundary.cpp:25-35`.
2. Mark ghost slabs solid, perform directional minimum-flip sweeps, reject nonconvergence, then apply optional mask callbacks. The anchor `rectify_solid` is in `rectify_solid.cpp:109-168`.
3. Save fill states, mark primitives, mirror reconstructed interface states and mask the implicit system. The anchor `mark_prim_solid_` is in `internal_boundary.cpp:47-95`.
4. Refill hydro after stage combination and exclude solids from gravity-fixer mass. The anchor `apply_boundaries` is in `meshblock.cpp:782-842`.

10.8.5. Tests

Table 10.17.: Source-defined assertions for solids; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_rectify.release`	Every outer and inner ghost slab is solid	Exact equality	Test constant / assertion
`test_flip_zero_count.release`	Eight-thread counts equal serial over repeats	Exact integer equality	Test constant / assertion

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_implicit_stratified_solid_python`	Segmented and strided solid mass closure	Finite; mass error below 1e-12	Test constant / assertion

The assertion anchor `test2` is in `test_rectify.cpp:91-124`.

The assertion anchor `serial` is in `test_flip_zero_count.cpp:12-48`.

The assertion anchor `1.e-12` is in `test_implicit_stratified_solid.py:277-291`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.8.6. Limits

The per-line optimum does not make the multidimensional repaired mask globally minimal. Exceeding the iteration limit can reject a mask whose last permitted sweep still changes entries. The mask and fill states are static. The cited stage refill only restores hydro, not passive scalars (`meshblock.cpp:782-842`); scalar-solid isolation needs runtime evidence. Diffusion and solid placeholders require separate coefficient analysis; no quantitative placeholder contamination measurement is reused here. Implicit x1 decomposition is explicitly rejected (`hydro.cpp:411-438`).

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.9. Order physical fills, exchange and corner refresh (`apply_boundaries`; `exchange_ghost_zones`)

10.9.1. Summary

> **At a glance**
>
> Physical fills and exchange have different owners. Reapply non-outflow x1 callbacks on exchange-owned tangential slabs so wall corners match the newly received state. Default: called by initialization and stage paths. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.9.2. Derivation

This is an operator-order contract. Copying neighbour values and applying a physical wall need not commute at their overlap: exchange can overwrite a wall corner. Reapplying the wall map to the newly exchanged strip restores the same parity relation as on the owned strip, provided the callback uses interior values and is idempotent. The registry contract (`BC_FUNCTION`, `bc_func.hpp:14-39`) and `refresh` (`meshblock.cpp:951-978`) encode these requirements. C1 includes a two-dimensional corner where exchange overwrites a prior mirror and a restricted second fill restores parity. This is an exact discrete ownership check, not a convergence proof.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.9.3. Numerical method

Initialization captures the boundary background, fills primitives, exchanges them, converts to conserved variables, records solid fill states and fills conserved fields. The standalone block `forward` exchanges before advancing. `Mesh::forward` advances then exchanges; its multi-block fixer path fills physical boundaries again after the shared correction. Each local advance refills solid hydro cells after the RK combine, performs last-stage saturation adjustment when enabled, applies the fixer when not deferred, then fills physical faces.

The block exchange refresh uses only the first two (x1) callbacks, skips outflow, and restricts each call to nonphysical tangential ghost slabs. Scalar primitive ratios are then recomputed. No accuracy order follows solely from this sequence; wall rest and phase-change tests are the available assertions.

Figure 10.10 illustrates the operations, with labelled roles independent of colour.

10.9.4. Code

The primitive initialization `exchange` calls are explicit in `meshblock.cpp:375-396`. Auxiliary scalar `sync_theta` exchanges raw data and then applies face functions with scalar type in `scalar.cpp:140-153`.

Table 10.18.: Implementation sites for timing.

what	code link	symbol	switch
init	`meshblock.cpp:401-431`	`boundary_reference_w`	See Summary
finalinit	`meshblock.cpp:454-500`	`fill_solid_hydro_u`	See Summary

what	code link	symbol	switch
advance	`meshblock.cpp:545-572`	`exchange_ghost_zones`	See Summary
refill	`meshblock.cpp:782-842`	`apply_boundaries`	See Summary
mesh	`mesh.cpp:331-365`	`advance_local`	See Summary
exchange	`meshblock.cpp:913-929`	`exchange_ghost_zones`	See Summary
corner	`meshblock.cpp:951-978`	`refresh`	See Summary
meshex	`mesh.cpp:389-395`	`exchange_ghost_zones`	See Summary
registry	`bc_func.hpp:14-39`	`BC_FUNCTION`	See Summary

1. Initialize and capture independent references before physical fills. The anchor `boundary_reference_w` is in `meshblock.cpp:401-431`.
2. Use the ordering of the selected block or mesh forward API; do not assume their exchange calls occupy the same position. The anchor `advance_local` is in `mesh.cpp:331-365`.
3. After block exchange, refresh x1 wall corners with the conserved/scalar representation and recompute scalar ratios. The anchor `refresh` is in `meshblock.cpp:951-978`.
4. At stage end, fill after enabled saturation adjustment and gravity correction. The anchor `apply_boundaries` is in `meshblock.cpp:782-842`.

10.9.5. Tests

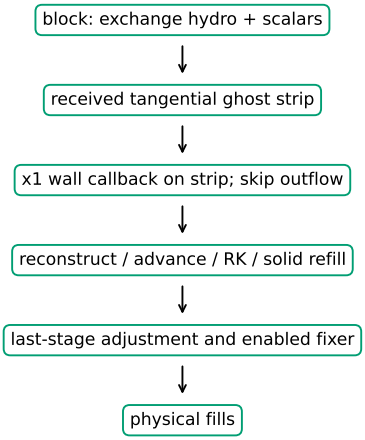


Figure 10.10.: Order physical fills, exchange and corner refresh. Schematic only; no simulation measurements.

Table 10.19.: Source-defined assertions for timing; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wb_wall_corner.release`	Rest survives wall/exchange overlap	Mach at most 1e-12	Test constant / assertion
`test_wall_saturation.release`	Boundary phase-change energy and water	Relative error below 1e-12	Test constant / assertion

The assertion anchor `EXPECT` is in `test_wb_wall_corner.cpp:166-190`.

The assertion anchor `1.e-12` is in `test_wall_saturation.cpp:59-89`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.9.6. Limits

Only x1 callbacks participate in this refresh; outflow is excluded. The public `Mesh::exchange_ghost_zones` method only constructs sync options and calls exchange (`mesh.cpp:389-395`), so callers cannot assume it supplies the block method's refresh. Custom callbacks depending on the full orthogonal shape violate the documented contract.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

10.10. Keep wall ghosts consistent with the hydrostatic reference (`dynamics/wb-wall-clamp`)

10.10.1. Summary

> **At a glance**
>
> For gravity-driven x1 reconstruction, mirror pressure and density perturbations at physical non-outflow faces. The default reference continues density/pressure using owned cells. Default: true. Order: exact discrete operation or local derivation below; coupled solver order not measured.

10.10.2. Derivation

All states here are cell averages; $\chi = \rho/p$ has units $\mathrm{s}^2\,\mathrm{m}^{-2}$. Write the default reference smoothing with the notation-table binomial weights $B = (1, 4, 6, 4, 1)/16$. On a uniform grid its unit sum and zero first moment reproduce linear data. Repeating the first owned χ beyond a wall breaks that cancellation: a Taylor expansion has an uncancelled first-derivative term of order h. Continuing from two owned cells cancels it.

Let owned cells start at index zero, let l count continuation depth, and use positive owned values. The implementation chooses

$$\chi_{-l} = \begin{cases} \chi_0(\chi_0/\chi_1)^l, & \chi_1 > \chi_0, \\ \chi_0 + l(\chi_0 - \chi_1), & \chi_1 \le \chi_0. \end{cases} \tag{10.10}$$

`hydro_ref_x1_wall_rop` (`hydro_ref_x1_impl.h:69-104`) implements Equation 10.10, with fallback to χ_0 if the computed value is not positive.

The first branch linearly continues $\log\chi$, the second χ. Expanding the power branch about equal adjacent values gives the same constant and linear terms as the second branch; the difference is $O(h^2)$ at fixed l. The binomial first moment then cancels the wall's $O(h)$ error for smooth data on a uniform grid. This is a truncation argument, not a fitted solver order. The reference construction is derived in the well-balanced chapter; the underlying note is `docs/derivations/wb-ref-wall.md@e894700ff7aee30b52882e5202b16461413780b0` section 4. No historical measured values from that note are reused. C7 checks the binomial moments, the repeated-cell linear bias and the first-order agreement of the two continuations symbolically.

The sweep subtracts p_{ref} and ρ_{ref}, then mirrors the perturbations:

$$\begin{aligned} p'_{i-l} &= p'_{i+l-1}, \\ \rho'_{i-l} &= \rho'_{i+l-1}. \end{aligned} \tag{10.11}$$

`is_outflow` guards these fills in `hydro_forward.cpp:287-343`, which implements Equation 10.11 only at physical non-outflow faces. If the cell state matches the discrete reference exactly, both perturbations vanish, their mirrors vanish, and a constant-preserving reconstruction returns zero perturbation. Restoring matched reference face values therefore adds no perturbation flux residual. The full pressure/gravity cancellation is the well-balanced scheme's identity, not a consequence of parity alone. C7 checks zero-perturbation preservation.

The report-only checks are `src/snapy_report/ch11/boundary_check.py`; their current output is `src/snapy_report/ch11/boundary_check.out`. They do not call a snapy kernel.

10.10.3. Numerical method

The well-balanced x1 path requires nonzero x1 gravity, a pressure row and an EOS other than shallow water. The old isentropic ghost revision calls

are commented out. The optional mass-covariance path uses one-sided density derivatives and odd corrected-velocity ghosts at the same physical non-outflow ends. Without WB, the gravity path revises reconstructed face pairs instead. Default reference wall clamping passes physical-face flags into the kernel; unlike perturbation mirroring, those flags do not exclude outflow. The continuation requires owned neighbours; thin-block fallbacks belong to the reference kernel tests.

The optional reference variant uses `SNAP_WB_REF4`; `SNAP_X1_CENTROID_EXACT` implies that variant. Both environment switches default off. This section does not rederive their high-order stencils or assign measured wall/seam convergence to them.

Figure 10.11 illustrates the operations, with labelled roles independent of colour.

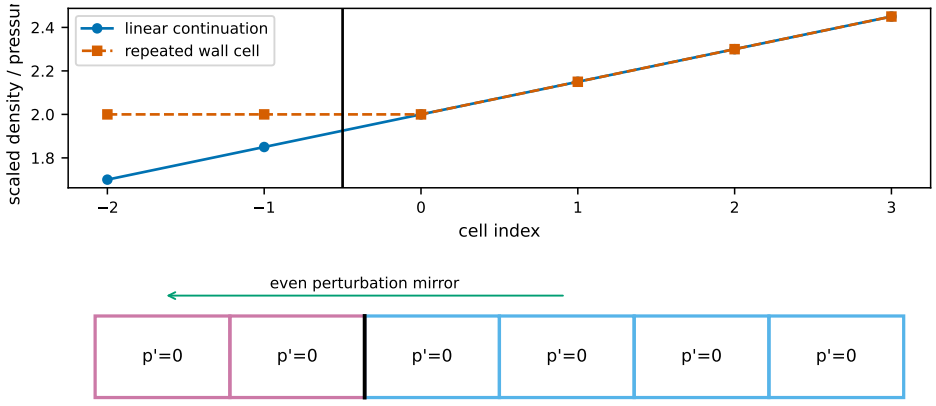


Figure 10.11.: Keep wall ghosts consistent with the hydrostatic reference. Schematic only; no simulation measurements.

10.10.4. Code

`x1_mass_covariance` reads `SNAP_X1_MASS_COVARIANCE` once, default off, in `hydro.cpp:231-236`.

Table 10.20.: Implementation sites for wb.

what	code link	symbol	switch
wboption	`hydro_options.cpp:51-60`	`wb_wall_clamp`	See Summary
wbflags	`hydro_forward.cpp:243-263`	`wb_x1`	See Summary
wb	`hydro_forward.cpp:287-343`	`is_outflow`	See Summary
nonwb	`hydro_forward.cpp:378-383`	`_revise_x1`	See Summary
ref	`hydro.cpp:503-556`	`wall_clamp`	See Summary
wallrop	`hydro_ref_x1_impl.h:69-104`	`hydro_ref_x1_wall_rop`	See Summary
ref4option	`wb_ref4.cpp:85-96`	`SNAP_WB_REF4`	See Summary
centroidoption	`x1_centroid.cpp:94-103`	`SNAP_X1_CENTROID_EXACT`	See Summary

1. Select the gravity/pressure WB branch and construct its reference using physical-face flags. The anchor `wall_clamp` is in `hydro.cpp:503-556`.

2. Subtract the reference and mirror only non-outflow physical-end perturbations. The anchor `is_outflow` is in `hydro_forward.cpp:287-343`.
3. Apply selected mass-covariance wall treatment, reconstruct, and restore the reference; the non-WB path instead revises face pairs. The anchor `_revise_x1` is in `hydro_forward.cpp:378-383`.

10.10.5. Tests

Table 10.21.: Source-defined assertions for wb; no runtime results.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wb_ref_wall.release`	Near-wall reference and reconstructed densities	Two-grid slope at least 1.7	Test constant / assertion
`test_hydro_ref_x1.release`	Wall clamp excludes changed ghost values	Exact equality; off-arm differs	Test constant / assertion

The assertion anchor `EXPECT_GE` is in `test_wb_ref_wall.cpp:229-264`.

The assertion anchor `EXPECT` is in `test_hydro_ref_x1.cpp:217-247`.

Numeric tolerances above are the source test constants; a round-off derivation is not documented for each. Device/rank distinctions and missing runs are recorded in Section 12.6.

10.10.6. Limits

The reference clamp still receives physical-face flags for outflow (`hydro.cpp:503-556`); it is incorrect to summarize all WB wall operations as excluding outflow. An in-block periodic x1 callback can count as physical here while `is_x1_wall` excludes its name (`hydro.cpp:204-209`). The two-resolution test assertion is not the three-resolution fitted order required for a report measurement. Nonuniform-grid wall accuracy and outflow/implicit/WB combinations remain unmeasured in this draft. The derivation is restricted to smooth uniform-grid data.

Untested switch combinations remain subject to Section 11.8. No new CPU, CUDA or multi-rank solver result is claimed.

11. Build-time and run-time switches and configurations

This chapter separates build-time availability, process-global study flags, local YAML scheme choices and test harness controls. Its matrix links a configuration to a registered assertion, with missing runtime evidence stated explicitly. It supplies the configuration reference used by the governing-equation and thermodynamics chapters.

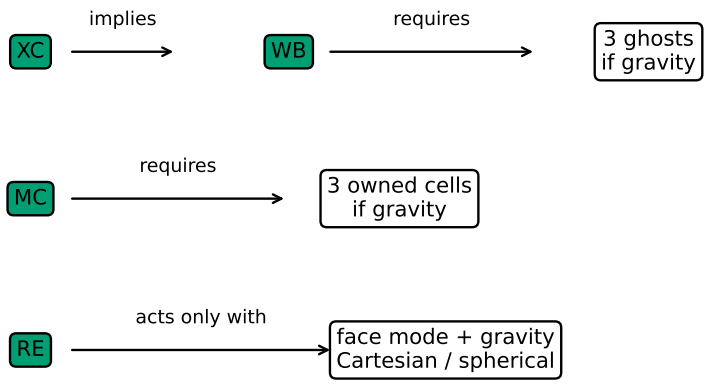


Figure 11.1.: Configuration dependencies: XC implies WB; gravity activates ghost/cell guards; face mode and geometry gate corrected work. Boxes state predicates, not performance results.

Table 11.1.: Chapter inventory.

scheme	switch	default	order	main evidence
Section 11.1	`cmake -DNAME=value`	see table	configuration; physical truncation stated in section	`test_python_import_path_python`
Section 11.6	`configure.h`	generated	configuration; physical truncation stated in section	`test_process_group.<b>`
Section 11.3	`SNAP_*`	four off; radial exact on	configuration; physical truncation stated in section	`test_gravity_work_radial_exact_python`
Section 11.11	`SNAP_WB_REF4`	off	configuration; physical truncation stated in section	`test_wb_ref4_order_python` (CUDA variant registered)

scheme	switch	default	order	main evidence
Section 11.2	SNAP_X1_CENTROID_EXACT	off	configuration; physical truncation stated in section	test_x1_centroid_rest_python (CUDA variant registered)
Section 11.4	SNAP_FLUX_COVARIANCE	off	configuration; physical truncation stated in section	test_horizontal_flux_covariance_python
Section 11.7	SNAP_X1_MASS_COVARIANCE	off	configuration; physical truncation stated in section	test_x1_seam_split_x1_mass_covariance.<b>
Section 11.9	SNAP_GRAVITY_WORK_RADIAL_EXACT	on	configuration; physical truncation stated in section	test_gravity_work_radial_exact_python (CUDA variant registered)

scheme	switch	default	order	main evidence
Section 11.5	`BACKEND / rank variables`	platform-dependent	configuration; physical truncation stated in section	`test_process_group.<b>`
Section 11.10	`test environment`	per registration	configuration; physical truncation stated in section	`test_python_import_path_python`
Section 11.12	`YAML keys`	see table	configuration; physical truncation stated in section	`test_hydro_options.<b>`, `test_yaml_keys.<b>`
Section 11.8	`combined configuration`	as above	configuration; physical truncation stated in section	all entries in the matrix

11.1. Build options and test registration (`cmake -DNAME=value`)

11.1.1. Summary

> **i** At a glance
>
> Default: **see table**. Set build options during CMake configuration. `BUILD_TESTS=ON`, `FULL_TESTS=OFF` and `CUDA=OFF` are the defaults. Registration, enablement and execution are separate states; a registered CUDA test is not evidence of a CUDA run. There is no numerical order for a build selector.

option or cache value	default	effect and coupling
`BUILD_TESTS`	ON	enters the tests subdirectory
`FULL_TESTS`	OFF	adds long reference/decomposition cases
`BUILD_EXAMPLES`	OFF	ineffective: examples subdirectory is unconditional
`CUDA`	OFF	enables CUDA compilation; with UCX needs the commux CUDA sidecar
`NETCDF`	ON	requires NetCDF and emits output macro
`UCX`, `PNETCDF`	ON except Apple OFF	require commux/UCX headers and Python pinc respectively

option or cache value	default	effect and coupling
NMASS	0	legacy mass-index offset; positive values fail the static index assertion
CMAKE_BUILD_TYPE	Release	GNU/Clang Release optimisation or Debug sanitizers
SNAPY_TEST_ PYTHONPATH	empty	prepends the intended package to Python test imports
KINTERA_DATA_DIR	unset	supplies the test nasa9.dat symlink
EIGEN, FMT, GTEST, YAML-CPP	ON in package calls	dependency fetching and cached archives

The CUDA architecture list starts at 60, 61, 70, 75, 80, 86, 89; toolkit 12 adds 90, 12.8 adds 120, and 13 removes 60, 61 and 70. The Torch ABI probe supplies the global C++ ABI definition.

11.1.2. Derivation

No continuum or discrete equation is selected here. The dependency chain is configure, generate, compile, register, then execute. BUILD_TESTS=OFF stops before registration; a non-CUDA build disables matching GPU entries after registration. These are configuration predicates, not a convergence derivation.

11.1.3. Numerical method

Build flags act before any mesh or RK stage exists. Test names use the lower-case build type suffix, represented here by `<b>`. The Python import-path probe must run before numerical Python evidence can be attributed to this checkout.

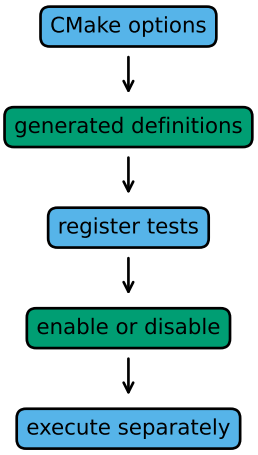


Figure 11.2.: Build options and test registration: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

11.1.4. Code

Table 11.2.: Pinned implementation anchors.

what	code link	symbol	switch
build	CMakeLists.txt: 7-18	BUILD_TESTS	cmake -DNAME=value
release	CMakeLists.txt: 61-68	CMAKE_BUILD_ TYPE	cmake -DNAME=value
arches	CMakeLists.txt: 109-143	CMAKE_CUDA_ ARCHITECTURES	cmake -DNAME=value
abi	CMakeLists.txt: 33-47	_GLIBCXX_USE_ CXX11_ABI	cmake -DNAME=value
examples	CMakeLists.txt: 146-158	BUILD_ EXAMPLES	cmake -DNAME=value
params	parameters.cmake: 1-46	NMASS	cmake -DNAME=value
indices	snap.h:34-55	static_assert	cmake -DNAME=value
packages	macro_add_ package.cmake: 1-34	option	cmake -DNAME=value
ucx	ucx.cmake:1-73	UCX_FOUND	cmake -DNAME=value
sidecar	CMakeLists.txt: 78-106	COMMUX_WITH_ CUDA_RUNTIME	cmake -DNAME=value
importreg	CMakeLists.txt: 316-363	SNAPY_TEST_ PYTHONPATH	cmake -DNAME=value
fulltests	CMakeLists.txt: 368-385	FULL_TESTS	cmake -DNAME=value
cudagate	CMakeLists.txt: 515-536	SNAPY_BUILD_ CUDA	cmake -DNAME=value

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read

at the full pin; no line numbers are carried from the outline's earlier pin.

11.1.5. Tests

Table 11.3.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_python_import_path_python`	package origin; caller path retained	exact path equality	`test_python_import_path.py:7-23`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.1.6. Limits

No configure/build is run for this draft. `BUILD_EXAMPLES`, positive `NMASS` and `NETCDF=OFF` have no dedicated test here. The Torch/package/toolchain combination still needs a CPU configure smoke test before claiming build compatibility.

The complete switch inventory is Section 11.8.

11.2. Spherical cell-to-plain-mean conversion (SNAP_X1_CENTROID_EXACT)

11.2.1. Summary

> **i** At a glance
>
> Default: **off**. Enable with `SNAP_X1_CENTROID_EXACT=1`. The conversion acts on spherical-polar x1 inputs and implies WB even on Cartesian grids. It inherits the nonzero-gravity three-ghost guard. The five-cell moment solve and six-face pressure source remove a measure mismatch; neither is a claim of arbitrary-order time integration.

11.2.2. Derivation

A spherical cell stores $\langle q\rangle_{V_i^{(1)}} = \int r^2 q\,\mathrm{d}r/V_i^{(1)}$, where radius r is in m and $V_i^{(1)} = \int r^2\,\mathrm{d}r$ per unit solid angle is in m^3. The x1 reconstruction instead needs $h^{-1}\int q\,\mathrm{d}r$. Choose a quartic polynomial whose five r^2 cell integrals match five input means, and integrate that polynomial with the plain measure over the target cell. Linearity gives five weights; requiring equality for degrees zero through four fixes them uniquely. `x1_plain_mean_stencils` forms and solves those five equations (`x1_centroid.cpp:104-182`). For pressure p in Pa, integration by parts gives the radial pressure acceleration density

$$-\frac{1}{V_i^{(1)}}\int_{r_{i-1/2}}^{r_{i+1/2}} r^2\partial_1 p\,\mathrm{d}r = -\frac{\Delta_i[r^2p]}{V_i^{(1)}} + \frac{2}{V_i^{(1)}}\int_{r_{i-1/2}}^{r_{i+1/2}} rp\,\mathrm{d}r. \tag{11.1}$$

The surface term is the pressure flux. `x1_pressure_source_stencils` integrates the second term from the quintic through six face pressures (`x1_`

`centroid.cpp:184-229`). Four-point quadrature integrates the degree-six product exactly. The polynomial identities are exact; general smooth fields retain interpolation truncation. `configuration_check.py` verifies the five moments and the integrated pressure identity. Source derivation: `docs/derivations/x1-centroid-spherical.md@e894700ff7aee30b52882e5202b16461413780b0` §2–4.

11.2.3. Numerical method

Five-cell windows move inside the owned region at a clamped wall. Ghost corrections are mirrored, odd for radial velocity at a reflecting wall; seam ghost rows are replaced by the neighbor's own converted values. The pressure operator needs six faces, with two neighbor faces beyond a seam. Unusable stencils return the original means. Only x1 inputs are converted; lateral fluxes retain the original primitives. The spherical pressure operator is also used for the non-hydrostatic split correction. `balance_column` accepts only Cartesian geometry when XC is on.

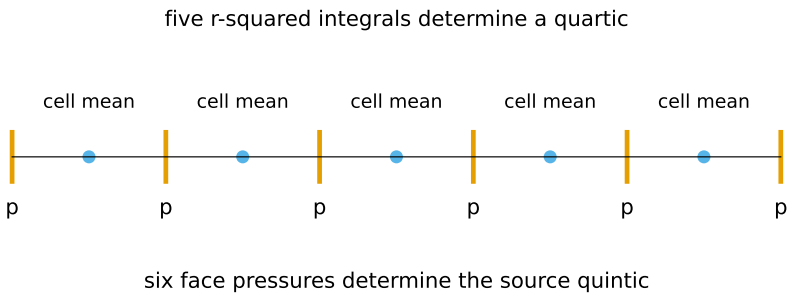


Figure 11.3.: Spherical conversion: sky-blue circles denote five cell means and orange face markers denote the six pressure values. The two polynomial degrees differ.

11.2.4. Code

Table 11.4.: Pinned implementation anchors.

what	code link	symbol	switch
xcflag	x1_centroid.cpp: 92-101	x1_centroid_exact_enabled	SNAP_X1_CENTROID_EXACT
xcmean	x1_centroid.cpp: 104-182	x1_plain_means	SNAP_X1_CENTROID_EXACT
xcpressure	x1_centroid.cpp: 184-229	x1_pressure_source	SNAP_X1_CENTROID_EXACT
xcrun	hydro_forward.cpp: 262-285	wx1	SNAP_X1_CENTROID_EXACT
xcnh	hydro_forward.cpp: 504-538	x1_pressure_source	SNAP_X1_CENTROID_EXACT
wbflag	wb_ref4.cpp: 83-118	wb_ref4_enabled	SNAP_X1_CENTROID_EXACT
guards	hydro.cpp:55-126	TORCH_CHECK	SNAP_X1_CENTROID_EXACT
balance	balance_column.cpp: 29-49	TORCH_CHECK	SNAP_X1_CENTROID_EXACT

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.2.5. Tests

Table 11.5.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_x1_centroid_rest_python` (CUDA variant registered)	force balance with implicit 0/1 and non-hydrostatic 0/1	on below 10^{-10}; off above 10^{-8} near origin	`test_x1_centroid_rest.py:37-44`
`test_x1_seam_split_x1_centroid.<b>`	split/unsplit comparison	relative gap at most 10^{-13}	`test_x1_seam_split.cpp:232-297`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.2.6. Limits

XC does not provide a gnomonic measure conversion. XC with face work, radial exact or FC needs new runtime evidence. Cartesian activation still changes the reference through WB; it is not a universal no-op.

The complete switch inventory is Section 11.8.

11.3. Environment parsing and process lifetime (`SNAP_*`)

11.3.1. Summary

> **At a glance**
>
> Default: **four off; radial exact on**. Set the five scheme variables before process startup. The helper `get_env` itself is not cached; each scheme accessor caches its parsed boolean in a function-local static. Changing the environment after first use does not change that accessor. An A/B comparison therefore needs a fresh child process per arm.

value, compared after lower-casing	WB / XC / FC / MC	radial exact
unset	off	on
empty string	off	on
`0`, `false`, `off`, `no`	off	off
every other string	on	on

There is no trimming or validation of spelling: `false` is on. Every block in one process sees the same cached value; the code does not by itself make different MPI processes agree.

11.3.2. Derivation

No numerical equation is added. Read the environment string, substitute the accessor default only when absent, lower-case it, compare with the false tokens, and cache the result. The empty string is deliberately excluded

from radial exact's false set. `configuration_check.py` enumerates this truth table and the centroid implication.

11.3.3. Numerical method

Launch child A with its environment, evaluate its first accessor call, then run all its blocks with that cached flag. Launch child B separately. The first call can occur during setup before a time step, so changing a flag between RK stages is not a valid experiment.

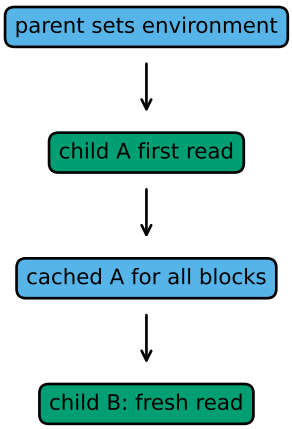


Figure 11.4.: Environment parsing and process lifetime: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

11.3.4. Code

Table 11.6.: Pinned implementation anchors.

what	code link	symbol	switch
env	`layout.hpp:38-50`	`get_env`	`SNAP_*`
flags	`hydro.cpp:217-259`	`gravity_work_radial_exact`	`SNAP_*`
wbflag	`wb_ref4.cpp:83-118`	`wb_ref4_enabled`	`SNAP_*`
xcflag	`x1_centroid.cpp:92-101`	`x1_centroid_exact_enabled`	`SNAP_*`
wbarms	`test_wb_ref4_order.py:130-144`	`SNAP_FLUX_COVARIANCE`	`SNAP_*`
xcarms	`test_x1_centroid_rest.py:109-123`	`SNAP_X1_CENTROID_EXACT`	`SNAP_*`
rearms	`test_gravity_work_radial_exact.py:339-386`	`warned`	`SNAP_*`

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.3.5. Tests

Table 11.7.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_gravity_work_radial_exact_python`	unset equals on; false arm differs	bitwise state comparison	`test_gravity_work_radial_exact.py:339-386`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.3.6. Limits

The independent parser check models the documented contract; it does not execute the C++ static storage. Cross-rank consistency is a launch responsibility. Unknown strings silently enable these switches.

The complete switch inventory is Section 11.8.

11.4. Lateral face covariance and centroid correction (`SNAP_FLUX_COVARIANCE`)

Local symbols (pending notation integration): $\sigma_i^{2,\mathrm{face}}$ is the x1 variance on the lateral face measure in m^2; δ_i^{face} is the cell centroid minus that face's radial centroid in m. These are distinct from the cell variance σ_i^2 and the cell-minus-midpoint offset δ_i.

11.4.1. Summary

> **i** At a glance
>
> Default: **off**. Enable with `SNAP_FLUX_COVARIANCE=1`. The x2/x3 correction has both a covariance term and a cell-to-face centroid term. Each direction's geometric pressure source is corrected only if that direction's flux correction exists. It is a leading smooth-gradient correction, not a fourth-order solver switch.

11.4.2. Derivation

Expand two smooth fields about the cell centroid. The average of their linear product is the product of their averages plus $\sigma_i^2 \partial_1 a \partial_1 b$, where σ_i^2 is the second central moment in m^2. For density-weighted primitives, expansion of the ratios cancels the density-gradient pairs: a transported scalar has leading covariance $\sigma_i^2 \rho \partial_1 v_a \partial_1 q$. Total mass is linear in stored momentum, so its covariance is zero. Species covariance is subtracted from dry mass, preserving their total. A face has its own centroid; shifting from the cell centroid adds $-\delta_i^{\mathrm{face}} \partial_1 \mathbf{F}_a$ to the advective flux being corrected. Thus the implemented enthalpy correction has two pieces:

$$(\Delta \mathbf{F}_a)_E = \sigma_i^{2,\mathrm{face}} \rho \, \partial_1 \left(\frac{\rho e + p}{\rho} \right) \partial_1 v_a - \delta_i^{\mathrm{face}} \partial_1 [(\rho e + p) v_a] , \qquad a \in \{2, 3\}. \tag{11.2}$$

Here e is specific internal energy in $\mathrm{J\,kg^{-1}}$ and v_a is the face-local normal speed in $\mathrm{m\,s^{-1}}$. The subscript E selects the energy row of the flux vector; the correction shown uses enthalpy, excluding kinetic energy. The flux density has units $\mathrm{W\,m^{-2}}$. `_flux_covariance` implements Equation 11.2 (`hydro_forward.cpp:54-191`); the face variance is used in this formula, rather than substituting the cell variance on curved

grids. Adding the term to a shared face flux makes its two neighboring volume-integrated increments cancel. This telescoping is exact; dropping horizontal-gradient products, velocity-squared covariance and the pressure Hessian is a truncation assumption. `configuration_check.py` checks the linear covariance identity and two-cell telescoping. Source derivation: `docs/derivations/289-covariance-x3-curved.md@e894700ff7aee30b52882e5202b16461413780b0` §2A and 4B.

11.4.3. Numerical method

At each hydro call, average the reconstructed left/right primitives, project a copy into the face-local frame, and difference along x1. Fewer than three x1 entries or shallow water returns no correction. Add the correction to each active lateral face flux. The corresponding geometric source uses pressure reduced by the centroid shift times its x1 derivative, while an inactive direction retains ordinary pressure. No separate MPI synchronization is introduced by this switch.

11.4.4. Code

Table 11.8.: Pinned implementation anchors.

what	code link	symbol	switch
flags	`hydro.cpp: 217-259`	`flux_covariance`	`SNAP_FLUX_COVARIANCE`
fc	`hydro_forward.cpp: 54-191`	`_flux_covariance`	`SNAP_FLUX_COVARIANCE`
fcapply	`hydro_forward.cpp: 594-629`	`cov2`	`SNAP_FLUX_COVARIANCE`

what	code link	symbol	switch
fcsource	hydro_ forward.cpp: 715-746	face_ centroid_ shift_x1	SNAP_FLUX_ COVARIANCE

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.4.5. Tests

Table 11.9.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
test_ horizontal_ flux_ covariance_ python	on/off signal, default equality, plain energy budget	scaled onset error below 0.04; drift at most 10^{-12}	test_ horizontal_ flux_ covariance.py: 33-41
test_flux_ covariance_ rows_python	rest, uniform species, energy-offset invariance	rest 10^{-9}; fraction 10^{-13}; offset 10^{-10}	test_flux_ covariance_ rows.py:32-40
test_flux_ covariance_ seams_python	six-panel budgets and direction gating	relative total drift below 10^{-12}	test_flux_ covariance_ seams.py:33-37

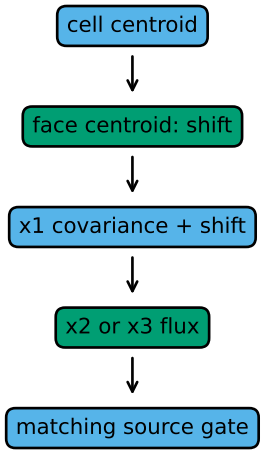


Figure 11.5.: Lateral face covariance and centroid correction: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.4.6. Limits

No new order fit is supplied. Curved-grid centroid terms do not vanish for an isothermal moving state; exact rest requires zero velocity. MPI-rank seams and XC+FC are unmeasured here.

The complete switch inventory is Section 11.8.

11.5. Layout and communication environment (`BACKEND / rank variables`)

11.5.1. Summary

> **At a glance**
>
> Default: **platform-dependent**. `BACKEND` defaults to UCX when compiled with UCX on non-Apple systems, otherwise Gloo. The YAML `distribute/backend` key is accepted but ignored: the environment wins. Unlike the five scheme flags, layout options read their environment during option construction.

environment	default	role
`PROCESS_RANK`	`RANK`, then 0	process rank
`PROCESS_WORLD_SIZE`	`WORLD_SIZE`, then 1	process count
`WORLD_SIZE`	process world size	global world size
`LOCAL_RANK`	0	local rank

environment	default	role
`MASTER_ADDR`	127.0.0.1	rendezvous address
`MASTER_PORT`	random only for single-process	required for either multi-process size indication
`DEVICE`, `DEVICE_ID`	cpu, -1	device choice
`COMMUX_COALESCE`, `COMMUX_GROUP`	1 if unset	UCX batching
`UCX_TLS`	CPU exclusions if unset	excludes CUDA copy/IPC/GDR paths on CPU
`WORKSPACE`, `NC_HOME`	current directory, unset	package-build search paths only

11.5.2. Derivation

No continuum derivation applies. Rank metadata selects the process topology; block layout and neighbor exchange then follow it. A common rendezvous port is necessary for processes to join the same group, so automatic port selection is only admissible for one process.

11.5.3. Numerical method

Construct layout options, validate rendezvous configuration, then instantiate the compiled process-group backend. UCX defaults are installed only if absent. These controls affect seam communication, not stencil coefficients; incorrect rank settings invalidate a multi-rank numerical comparison.

11.5.4. Code

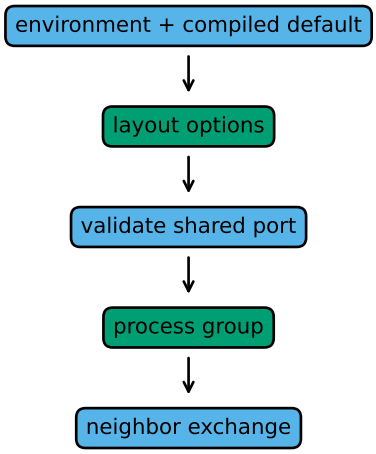


Figure 11.6.: Layout and communication environment: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

Table 11.10.: Pinned implementation anchors.

what	code link	symbol	switch
layoutdefault	`layout.cpp:41-50`	`default_backend`	`BACKEND / rank variables`
layout	`layout.cpp:189-242`	`MASTER_PORT`	`BACKEND / rank variables`
ucxenv	`process_group_ucx.cpp:14-32`	`set_default_env`	`BACKEND / rank variables`
setup	`setup.py:42-63`	`NC_HOME`	`BACKEND / rank variables`

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.5.5. Tests

Table 11.11.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_process_group.<b>`	backend precedence and port requirements	exact strings; required exceptions	`test_process_group.cpp:74-186`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.5.6. Limits

No network or multi-rank run is performed. A YAML backend value cannot override the environment at this pin. Runtime selection of a backend does not prove its library was compiled and linked.

The complete switch inventory is Section 11.8.

11.6. Generated macros and compile definitions (`configure.h`)

11.6.1. Summary

> **At a glance**
>
> Default: **generated**. CMake generates the macro interface used to select compiled readers. No runtime environment assignment can add a backend that was excluded at compile time. This layer has no numerical order.

definition	producer	reader or role
`USE_C10D_GLOO`	always defined	no source reader found; not a backend choice
`USE_UCX` / `NOT_USE_UCX`	UCX option	process-group implementation and platform default

definition	producer	reader or role
`USE_CUDA` / `NOT_USE_CUDA`	CUDA option	mesh/device dispatch and test availability
`NETCDFOUTPUT` / `NO_NETCDFOUTPUT`	NETCDF option	output implementation
`PNETCDFOUTPUT` / `NO_PNETCDFOUTPUT`	PNETCDF option	parallel output implementation
`NMASS`	cache value	legacy indices; use zero
`DISPATCH_MACRO`	nvcc detection	host/device annotation under nvcc, empty otherwise
`KINTERA_ROOT_DIR`, `HARP_ROOT_DIR`	configure source directory	no source reader found
`_GLIBCXX_USE_CXX11_ABI`	Torch probe	all compiled translation units
`HAVE_AVX512_CPU_DEFINITION`, `HAVE_AVX2_CPU_DEFINITION`	non-Apple compiler setup	Torch headers
`COMMUX_WITH_CUDA_RUNTIME`	CUDA with UCX	commux CUDA integration

11.6.2. Derivation

No physical derivation applies. Macro substitution happens before compilation, so runtime selection is restricted to compiled alternatives. The complement macros document an excluded feature rather than implementing a fallback.

11.6.3. Numerical method

Generation uses `configure_file`; readers then compile the available path. ABI and backend selection must agree across linked libraries. These definitions have no cell location, ghost stencil or RK-stage dependence.

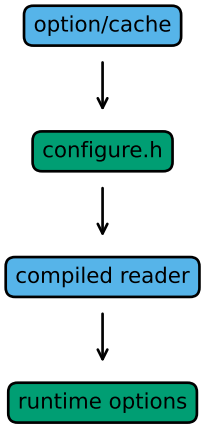


Figure 11.7.: Generated macros and compile definitions: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

11.6.4. Code

Table 11.12.: Pinned implementation anchors.

what	code link	symbol	switch
macro	`configure.h.in: 1-28`	`DISPATCH_MACRO`	`configure.h`

what	code link	symbol	switch
params	parameters.cmake:1-46	NMASS	configure.h
compiler	compilers.cmake:4-35	HAVE_AVX2_CPU_DEFINITION	configure.h
abi	CMakeLists.txt:33-47	_GLIBCXX_USE_CXX11_ABI	configure.h
sidecar	CMakeLists.txt:78-106	COMMUX_WITH_CUDA_RUNTIME	configure.h
layoutdefault	layout.cpp:41-50	default_backend	configure.h

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.6.5. Tests

Table 11.13.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
test_process_group.<b>	platform backend default	exact equality	test_process_group.cpp:74-186

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.6.6. Limits

Indirect default-backend tests do not constitute exhaustive preprocessor coverage. The unused definitions are retained as code facts, not promises of a public runtime API.

The complete switch inventory is Section 11.8.

11.7. Radial velocity mean correction (`SNAP_X1_MASS_COVARIANCE`)

11.7.1. Summary

> **i** At a glance
>
> Default: **off**. Enable with `SNAP_X1_MASS_COVARIANCE=1`. The well-balanced x1 path converts density-weighted radial velocity to a plain mean before reconstruction. Nonzero radial gravity requires at least three owned x1 cells at setup. There is an existing setup-guard test, but no identified numerical accuracy oracle for this switch.

11.7.2. Derivation

On a Cartesian cell of width h in metres, integrate two linear fields about its midpoint: $h^{-1}\int_{-h/2}^{h/2} x_1\,\mathrm{d}x_1 = 0$ and $h^{-1}\int_{-h/2}^{h/2} x_1^2\,\mathrm{d}x_1 = h^2/12$. Consequently the density-weighted velocity exceeds the plain velocity mean by $(h^2/12)\partial_1\rho\partial_1 v_1/\rho$. The code subtracts that leading term:

$$\frac{1}{h}\int_{x_{1,i-1/2}}^{x_{1,i+1/2}} v_1\,\mathrm{d}x_1 \simeq \frac{\langle\rho v_1\rangle_{V_i}}{\langle\rho\rangle_{V_i}} - \frac{h^2}{12\rho_i}\partial_1\rho_i\partial_1 v_{1,i}. \tag{11.3}$$

The left side is integration with the uniform radial measure, not a newly stored field. `rho_1` and `w1` implement Equation 11.3 (`hydro_forward.cpp:314-352`). The integral identity is exact for linear fields; substituting finite differences and nonlinear profiles introduces truncation. `configuration_check.py` verifies the identity with rational polynomial integration. This is re-derived from the implementation; no standalone derivation note was identified.

11.7.3. Numerical method

The radial gravity/non-shallow-water path computes centered density and velocity differences. At reflecting walls, a three-owned-cell one-sided density slope replaces the slope through the reflected ghost. The corrected velocity has odd wall ghosts, is reconstructed with the density/pressure perturbations, and is then restored in the input array. The correction is part of each explicit hydro call; it is not a new implicit matrix term.

11.7.4. Code

Table 11.14.: Pinned implementation anchors.

what	code link	symbol	switch
flags	`hydro.cpp:217-259`	`x1_mass_covariance`	`SNAP_X1_MASS_COVARIANCE`
guards	`hydro.cpp:55-126`	`TORCH_CHECK`	`SNAP_X1_MASS_COVARIANCE`
mc	`hydro_forward.cpp:314-352`	`rho_1`	`SNAP_X1_MASS_COVARIANCE`

what	code link	symbol	switch
mctest	test_x1_seam_split.cpp: 327-360	x1_mass_covariance_needs_3_x1_cells	SNAP_X1_MASS_COVARIANCE

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.7.5. Tests

Table 11.15.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
test_x1_seam_split_x1_mass_covariance.<b>	reject two owned cells; accept three	exception text and no-throw	test_x1_seam_split.cpp: 327-360

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.7.6. Limits

The frozen outline's "no tests" entry is stale at this pin: a guard exists. Interior, wall, seam, CUDA, MPI and combined-switch accuracy remain

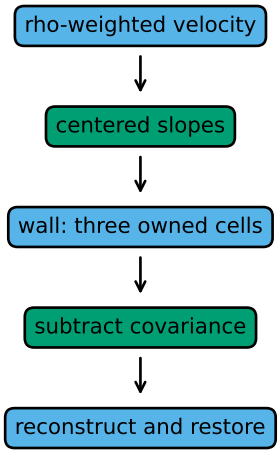


Figure 11.8.: Radial velocity mean correction: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

unmeasured. The Cartesian coefficient does not establish curved-measure exactness.

The complete switch inventory is Section 11.8.

11.8. Switch-combination coverage matrix (`combined configuration`)

11.8.1. Summary

> **At a glance**
>
> Default: **as above**. WB = `SNAP_WB_REF4`, XC = `SNAP_X1_CENTROID_EXACT`, FC = `SNAP_FLUX_COVARIANCE`, MC = `SNAP_X1_MASS_COVARIANCE`, RE = `SNAP_GRAVITY_WORK_RADIAL_EXACT`. XC implies WB. `<b>` is the build suffix, normally `release`. "child" means a script creates fresh processes; "CTest" means the registration sets the variable. Unspecified flags must be sanitized when reproducing a row. The table lists selected evidence, not every test in the project.

Table 11.16.: Registered combinations at the pinned code; none is a new runtime result.

combination	work / grid	registered evidence	meaning
defaults: WB/XC/FC/MC off, RE on	cell+fixer	`test_gravity_work_fixer_python` cell arms (RE forced off, inactive for cell)	default cell contract; not an assertion about every example

combination	work / grid	registered evidence	meaning
RE unset/0/1 (child)	face; Cartesian/spherical	`test_gravity_work_radial_exact_python`; CUDA variant	default equality and corrected/plain budgets
RE 0/1 (child)	face; tall column	`test_implicit_gravity_tall_column_python`	implicit rest comparison
RE=0 (CTest)	face and cell arms	`test_forcing.<b>`, `test_gravity_work_fixer_python`, `test_implicit_face_work_operator_python`, `test_implicit_stratified_solid_python`	ordinary face-work invariant

combination	work / grid	registered evidence	meaning
RE=1 (CTest)	face; Cartesian/spherical	`test_implicit_face_work_operator_radial_exact_python`, `test_implicit_stratified_solid_radial_exact_python`	corrected implicit budget
RE=0/1 (CTest)	face; x1 split	`test_x1_seam_split_radial_exact[_off].<b>`; CUDA variants; `test_x1_seam_split_mp_radial_exact[_off].<b>`	one-sided seam functional; split budget

combination	work / grid	registered evidence	meaning
WB=1 (CTest)	cell; Cartesian	`test_balance_column_wb_ref4.<b>`, `test_face_floor_wb_ref4.<b>`, `test_x1_seam_split_wb_ref4.<b>`; CUDA seam variant; `test_x1_seam_split_mp_wb_ref4.<b>`	fixed point, face floor, seam
WB=1, zero gravity (CTest)	cell; one ghost	`test_x1_seam_split_wb_ref4_gravity_0.<b>`	guard inactive without gravity
WB off/on (child), FC=1, RE default on	face; Cartesian	`test_wb_ref4_order_python`; CUDA variant	combined order oracle

combination	work / grid	registered evidence	meaning
XC=1 (CTest), hence WB	cell; spherical seam / Cartesian balance	`test_balance_column_x1_centroid.<b>`, `test_face_floor_x1_centroid.<b>`, `test_x1_seam_split_x1_centroid.<b>`; CUDA seam variant; `test_x1_seam_split_mp_x1_centroid.<b>`	implied reference and seam conversion
XC off/on (child), implicit 0/1, non-hydrostatic 1/0	cell; spherical	`test_x1_centroid_rest_python`; CUDA variant	pressure/gravity rest
FC unset/0/1 (child), RE=0 (CTest)	face; Cartesian	`test_horizontal_flux_covariance_python`	onset signal and plain budget
FC off/on (child), RE=0 (CTest)	Cartesian/spherical/gnomonic	`test_flux_covariance_rows_python`	rest, species, energy offsets

combination	work / grid	registered evidence	meaning
FC off/on (child)	cell+fixer; six panels and spherical	`test_flux_covariance_seams_python`	panel conservation and direction gating; one process
study flags off, one ghost	cell; spherical	`test_x1_seam_split.<b>`, `test_x1_seam_split_mp.<b>`	baseline thin ghost setup
MC=1 (CTest)	cell; two/three owned cells	`test_x1_seam_split_x1_mass_covariance.<b>`	setup rejection/acceptance only

11.8.2. Derivation

Coverage is a relation between a configuration and a test assertion, not a probability or an accuracy estimate. A row is supported only when both the registration and the test's actual arm agree. A guard assertion supports setup behavior; it does not support an error norm. The anchor audit checks these sources without executing them.

11.8.3. Numerical method

First select a build that contains the required entries, then establish import provenance, sanitize scheme variables, and launch the named arm. Compare the invariant selected by the gravity-work mode. Record CPU/CUDA and rank count with every measured result. The dependency figure shows logical constraints rather than measured performance.

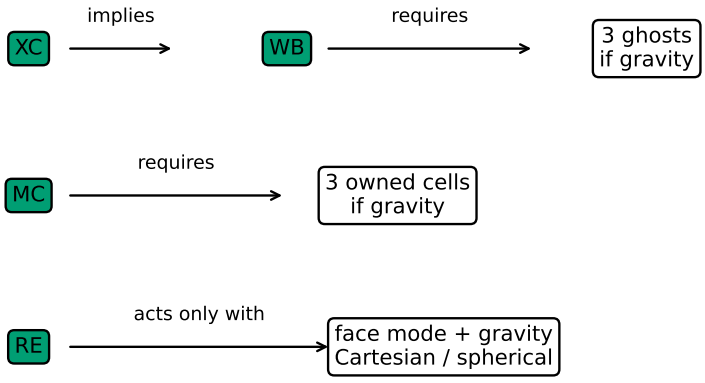


Figure 11.9.: Logical dependencies: labelled arrows mean implies, requires, or acts only with. The diagram is not a sequence of numerical operations.

11.8.4. Code

Table 11.17.: Pinned implementation anchors.

what	code link	symbol	switch
registration	`CMakeLists.txt: 38-104`	`seam_arm`	`combined configuration`
pytests	`CMakeLists.txt: 206-277`	`SNAPY_PYTHON_TESTS`	`combined configuration`
cuda	`CMakeLists.txt: 280-314`	`CUDA`	`combined configuration`
mctest	`test_x1_seam_split.cpp: 327-360`	`x1_mass_covariance_needs_3_x1_cells`	`combined configuration`

what	code link	symbol	switch
wbarms	`test_wb_ref4_order.py:130-144`	`SNAP_FLUX_COVARIANCE`	`combined configuration`
fcrowsarms	`test_flux_covariance_rows.py:260-318`	`SNAP_FLUX_COVARIANCE`	`combined configuration`
fcseamsarms	`test_flux_covariance_seams.py:109-160`	`SNAP_FLUX_COVARIANCE`	`combined configuration`

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.8.5. Tests

Table 11.18.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
all entries in the matrix	registered assertions only	see individual scheme tables	no runtime results in this draft

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.8.6. Limits

New evidence is needed for numerical MC behavior; XC with face work/RE; XC+FC; WB/XC on gnomonic grids; WB with RE=0 under face work; FC across MPI ranks; and broader simultaneous study-switch combinations. WB+FC+RE has a registered order oracle. All-flags combinations are not thereby covered.

The complete switch inventory is Section 11.8.

11.9. Corrected potential-energy face work (`SNAP_GRAVITY_WORK_RADIAL_EXACT`)

11.9.1. Summary

> **i** At a glance
>
> Default: **on**. This switch defaults on, including an empty string. It acts only with nonzero radial gravity, `gravity-work: face` and Cartesian or spherical-polar geometry. Unsupported geometry warns once and retains ordinary face work. Use the corrected potential energy in Equation 11.4 for its budget, not the plain centroid sum. The internal alias is option F (corrected-PE work).

11.9.2. Derivation

For constant g_1 in $\mathrm{m\,s^{-2}}$, $\phi = -g_1 x_1$ in $\mathrm{J\,kg^{-1}}$. Integrating the product of locally linear density and potential gives $\langle \rho\phi \rangle_{V_i} = \rho_i \phi_i - g_1 \sigma_i^2 \partial_1 \rho$. Replace the derivative by the centroid quadratic slope $s_i[\rho]$ to define P. On a fixed mesh this is a linear functional of density, so its time difference uses exactly the same slope:

$$
\begin{aligned}
P &= \sum_i V_i \left(\rho_i \phi_i - g_1 \sigma_i^2 s_i[\rho]\right), \\
\Delta P &= \sum_i V_i \left(\phi_i \Delta\rho_i - g_1 \sigma_i^2 s_i[\Delta\rho]\right).
\end{aligned}
\tag{11.4}
$$

Ordinary face work cancels the first term (with the boundary potential-energy flux). Adding $g_1 \sigma_i^2 s_i[\Delta\rho]$ to the energy increment cancels the second term, cell by cell. `corrected_pe_work` implements that increment (`gravity_work_radial.hpp:13-65`). For implicit work the density increment must be the actual post-clamp increment. The conserved discrete budget is exact up to arithmetic and boundary fluxes; its approximation of smooth continuous potential energy has truncation error. `configuration_check.py` verifies the cancellation for a fixed linear slope matrix. Source derivation: `docs/derivations/curved-gravity-work-weight.md@e894700ff7aee30b52882e5202b16461413780b0` §7.

11.9.3. Numerical method

The slope is quadratic through three centroids, one-sided at each block end, including block seams. Use the cell measure variance, not the lateral face variance. Explicit face work and the implicit operator use the same corrected budget; active solids mask energy deposition. On unsupported grids the setup warning does not abort. The switch is inactive for `face-wallc`, whose wall cells keep cell work. Corrected work replaces the cp3/cp5/weno5 curvature work instead of adding to it, avoiding a double correction.

11.9.4. Code

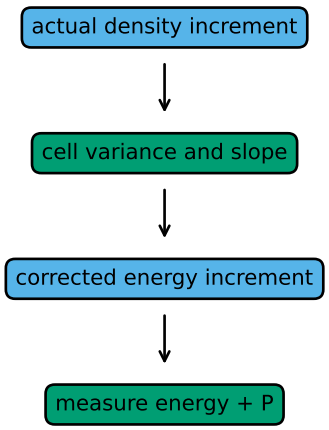


Figure 11.10.: Corrected potential-energy face work: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

Table 11.19.: Pinned implementation anchors.

what	code link	symbol	switch
flags	hydro.cpp: 217-259	radial_exact_ work	SNAP_GRAVITY_ WORK_RADIAL_ EXACT
guards	hydro.cpp:55- 126	TORCH_CHECK	SNAP_GRAVITY_ WORK_RADIAL_ EXACT
radial	gravity_work_ radial.hpp: 13-65	corrected_pe_ work	SNAP_GRAVITY_ WORK_RADIAL_ EXACT
operator	hydro.cpp: 305-309	face_work_in_ operator	SNAP_GRAVITY_ WORK_RADIAL_ EXACT
curvature	hydro_ forward.cpp: 798-849	radial_exact	SNAP_GRAVITY_ WORK_RADIAL_ EXACT

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.9.5. Tests

Table 11.20.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_gravity_work_radial_exact_python` (CUDA variant registered)	explicit/VIC, diagnostic budget, clamp and solids	per-step relative budget 10^{-14}; printed budget 10^{-11}	`test_gravity_work_radial_exact.py:50-57`
`test_x1_seam_split_radial_exact.<b>`	split column corrected budget	relative drift at most 10^{-13}	`test_x1_seam_split.cpp:232-297`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.9.6. Limits

The gnomonic grid does not receive corrected work at this pin. Split/unsplit states need not agree because their one-sided slope functionals differ. A new gnomonic convergence study cannot be replaced by the warning assertion.

The complete switch inventory is Section 11.8.

11.10. Test harness environment (`test environment`)

11.10.1. Summary

> **i** At a glance
>
> Default: **per registration**. Harness variables select test arms and evidence provenance; they are not new hydro schemes. `SNAPY_BUILD_CUDA=0` is installed for non-CUDA builds even if a GPU is visible.

variable	purpose
`SNAPY_BUILD_CUDA`	suppress unavailable CUDA arms
`GTEST_FILTER`	select exactly one seam case per registered arm
`BLOCKS_PER_PROCESS`	exchange topology size
`EXPECT_LOCAL_NEIGHBOR`, `EXPECT_REMOTE_NEIGHBOR`	assert selected exchange topology
`SNAPY_RUN_HYDRO`	locate the cycle-abort test executable
`WB_REF_WALL_DUMP`, `WB_REF_WALL_BETAS`	wall-reference scan/dump aids

`SNAPY_TEST_PYTHONPATH` is a CMake cache path, not one of these solver variables. The generated test environment prepends it while retaining the caller's path.

11.10.2. Derivation

No numerical derivation applies. A selected test must actually execute its assertion: a filter that selects no tests cannot certify a scheme. The seam-arm registration therefore requires a one-test pass marker, and Python CUDA arms use skip code 125 when unavailable.

11.10.3. Numerical method

Register the arm with its environment, select its case, verify its execution marker, then interpret its assertions. Record disabled and skipped tests separately from passes. Supply all scheme flags explicitly or unset them before starting evidence runs: inherited user environment can otherwise contaminate a nominal default arm.

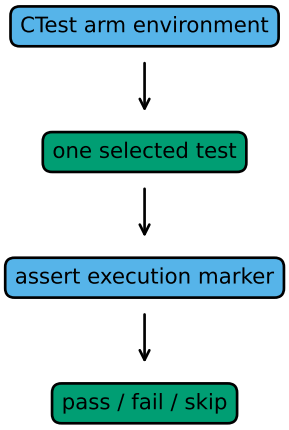


Figure 11.11.: Test harness environment: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

11.10.4. Code

Table 11.21.: Pinned implementation anchors.

what	code link	symbol	switch
registration	CMakeLists.txt: 38-104	seam_arm	test environment
importreg	CMakeLists.txt: 316-363	SNAPY_TEST_ PYTHONPATH	test environment
cudagate	CMakeLists.txt: 515-536	SNAPY_BUILD_ CUDA	test environment
exchangeenv	test_ exchange.cpp: 219-224	BLOCKS_PER_ PROCESS	test environment
exchangeexpect	test_ exchange.cpp: 303-306	EXPECT_ REMOTE_ NEIGHBOR	test environment
runenv	test_uranus_ cycle1_ abort.cpp:45-52	SNAPY_RUN_ HYDRO	test environment
dumpenv	test_wb_ref_ wall.cpp:159-167	WB_REF_WALL_ DUMP	test environment
betaenv	test_wb_ref_ wall.cpp:191-201	WB_REF_WALL_ BETAS	test environment

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.10.5. Tests

Table 11.22.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_python_import_path_python`	correct imported package and preserved caller entry	exact paths; skip 125 if unconfigured	`test_python_import_path.py:7-23`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.10.6. Limits

Harness coverage is not solver coverage. This draft performs no registered CTest run, and a visible GPU is not used to infer CUDA support.

The complete switch inventory is Section 11.8.

11.11. Cell/face-consistent reference (`SNAP_WB_REF4`)

11.11.1. Summary

> **i** At a glance
>
> Default: **off**. Enable the fourth-order reference with `SNAP_WB_REF4=1`; centroid conversion also enables it. With nonzero radial gravity, setup requires at least three ghost cells. The filter preserves cubics on a uniform grid; this algebraic order is not a measured solver convergence rate. The defining filter is Equation 11.5.

11.11.2. Derivation

Let $\chi = \rho/p$ in $\mathrm{s}^2\,\mathrm{m}^{-2}$. Taylor expansion of $\chi(x_{1,i} + lh)$, with integer offset l, gives one term for each weighted moment. For weights $(-1, 4, 10, 4, -1)/16$, the zeroth moment is one and the first three moments vanish. The fourth moment is $-3/2$, so the leading smooth error is $-h^4\partial_1^4\chi/16$. The cell reference is therefore

$$\rho_{\mathrm{ref},i} = p_{\mathrm{ref},i}\frac{-\chi_{i-2} + 4\chi_{i-1} + 10\chi_i + 4\chi_{i+1} - \chi_{i+2}}{16}. \tag{11.5}$$

`wb_ref4_cells` implements Equation 11.5 (`wb_ref4.cpp:233-287`). Cell means have integral $h\rho_{\mathrm{ref},i}$ on a Cartesian cell. Accumulating these integrals at five faces and differentiating their quartic interpolant returns the four-cell face rule $(-1, 7, 7, -1)/12$ in a uniform interior. Thus the face density comes from the same cell field, rather than an independently averaged ratio. At a clamped wall the missing filter values are cubic extrapolations: the first missing value has weights $(4, -6, 4, -1)$ and the second $(10, -20, 15, -4)$. Substitution reproduces all cubics. `configuration_check.py` checks both these moment identities and the face rule by exact

rational integration. Source derivation: `docs/derivations/wb-ref4.md@e894700ff7aee30b52882e5202b16461413780b0` §3–5.

11.11.3. Numerical method

The filter uses five cell ratios and the face map uses four cell means. Clamped walls use owned cells only; seams read exchanged neighbors. The pressure-drop flag is dilated by two cells, which makes its effective reach three cells across a seam. Range or resolution failures retain the base reference. Nonuniform cell pressure uses a cubic face interpolant. With zero gravity no radial reference is built and the ghost-depth guard is inactive. Reference construction precedes x1 perturbation reconstruction on each hydro call.

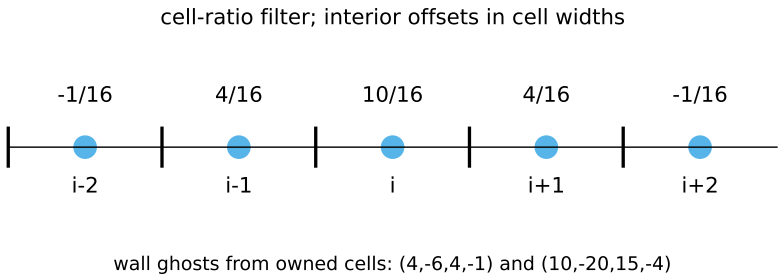


Figure 11.12.: Five-cell reference filter: sky-blue circles are cell ratios, black ticks are faces, and labels give filter and wall-continuation weights.

11.11.4. Code

Table 11.23.: Pinned implementation anchors.

what	code link	symbol	switch
wbflag	wb_ref4.cpp:83-118	wb_ref4_enabled	SNAP_WB_REF4
guards	hydro.cpp:55-126	TORCH_CHECK	SNAP_WB_REF4
wbweights	wb_ref4.cpp:120-163	F	SNAP_WB_REF4
wbface	wb_ref4.cpp:190-221	lagrange_deriv	SNAP_WB_REF4
wbcells	wb_ref4.cpp:233-287	wb_ref4_faces	SNAP_WB_REF4
wbcall	hydro.cpp:564-579	wb_ref4_stencils	SNAP_WB_REF4
wbfacecall	hydro.cpp:642-646	wb_ref4_faces	SNAP_WB_REF4

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.11.5. Tests

Table 11.24.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wb_ref4_order_python` (CUDA variant registered)	three-resolution order; on/off arms with FC	on at least 2.75; off below 2.5	`test_wb_ref4_order.py:34-44`
`test_x1_seam_split_wb_ref4.<b>`	one versus two blocks; bad ghost depth rejected	relative gap at most 10^{-13}	`test_x1_seam_split.cpp:232-297`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.11.6. Limits

No fitted order is measured here. Guards can lower the effective order; wall and gnomonic convergence must not be inferred from the interior moments. The gnomonic and extra switch combinations remain gaps in Section 11.8.

The complete switch inventory is Section 11.8.

11.12. YAML scheme selection and setup guards (`YAML keys`)

11.12.1. Summary

> **At a glance**
>
> Default: **see table**. YAML selects the local numerical options; process-global study switches remain environment settings. Recognized parser blocks reject unknown keys. Direct Python option construction bypasses YAML parsing, so hydro reset rechecks gravity-work constraints.

key	default	coupling
`forcing/const-gravity/gravity-work`	cell	cell, face-wallc or face
`gravity-work-fixer` in that block	true for cell	requires radial-only gravity and nonperiodic x1; non-cell reset clears it
`grav1`, `grav2`, `grav3`	0	disabled flux direction zeros its gravity
`non-hydrostatic`	1	must lie in [0,1]
`dynamics/wb-wall-clamp`	true	required by balance_column
`dynamics/disable-flux-x1/x2/x3`, `verbose`	false	flux/source direction gates
`dynamics/equation-of-state/type`	moist-mixture	shallow-water bypasses radial WB
EOS `limiter`	false	positivity correction

key	default	coupling
EOS density, pressure, temperature floors	10^{-6} kg m^{-3}, 10^{-3} Pa, 20 K	state floors
reconstruction vertical/horizontal `type,scale,shock`	dc,false,false	spatial reconstruction
`dynamics/riemann-solver/type,dir`	roe,omni	interface flux solver
`integration/implicit-scheme`	absent / 0	accepts 0, 1, 9 only
`implicit-advection-cfl`, `shear-cfl`	1, 0	require active implicit scheme; positive / nonnegative finite values
`geometry/cells/nghost`	1	WB/XC with gravity requires at least 3
`geometry/type`	cartesian	controls XC and corrected-work availability
`distribute/layout,nb1,nb2,nb3`	slab,1,1,1	block decomposition
external x1/x2/x3 boundaries	reflecting	periodic x1 incompatible with active fixer
`forcing/fric-heat`	removed	explicit parse error

11.12.2. Derivation

No separate continuum equation is introduced by the parser. The configuration resolves into the schemes described in the preceding sections. In particular, changing face work changes the budget functional; it is not

just an output label. Validation is part of the numerical contract because incompatible options must not silently run.

11.12.3. Numerical method

Parse the recognized sub-blocks, instantiate component options, then reset hydro. YAML rejects a true fixer with non-cell work and nonzero radial gravity; direct option construction is normalized by reset before its remaining guards. Implicit non-cell work without an in-operator path warns. A missing implicit scheme returns explicit options; setting its CFL controls without an active scheme fails.

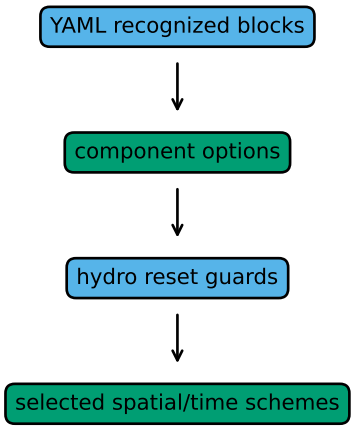


Figure 11.13.: YAML scheme selection and setup guards: labelled boxes show inputs and decisions in evaluation order. This is a configuration or stencil diagram, not measured data.

11.12.4. Code

Table 11.25.: Pinned implementation anchors.

what	code link	symbol	switch
yaml	hydro_options.cpp:24-87	check_keys	YAML keys
gravity	const_gravity.cpp:13-44	gravity_work	YAML keys
implicit	implicit_hydro.cpp:26-98	implicit-scheme	YAML keys
eos	equation_of_state.cpp:50-78	density_floor	YAML keys
recon	reconstruct.cpp:24-38	shock	YAML keys
riemann	riemann_solver.cpp:23-31	roe	YAML keys
coord	coordinate.cpp:156-169	nghost	YAML keys
geometry	coordinate.cpp:63-74	cartesian	YAML keys
boundary	meshblock_options.cpp:100-193	reflecting	YAML keys
guards	hydro.cpp:55-126	TORCH_CHECK	YAML keys
operator	hydro.cpp:305-309	face_work_in_operator	YAML keys

what	code link	symbol	switch
yamltest	`test_yaml_keys.cpp:63-93`	`TEST`	`YAML keys`
hydrotest	`test_hydro_options.cpp:195-258`	`wb_wall_clamp_ships_enabled`	`YAML keys`

Read the table in order: establish the selector, follow its gates and consumers, then inspect the registered assertions. The cited ranges are re-read at the full pin; no line numbers are carried from the outline's earlier pin.

11.12.5. Tests

Table 11.26.: Source assertions, not new solver measurements.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_hydro_options.<b>`, `test_yaml_keys.<b>`	known/default keys and invalid choices	exact booleans and exceptions	source anchors in the verification catalogue
`test_implicit_options_type_python`	implicit type contract	integer/string and exception checks	registration `CMakeLists.txt:206-277`

No snapy runtime test is executed for this chapter. CUDA and multi-rank entries describe registration only. Independent CPU formula/contract checks and their outputs are catalogued in Section 12.7.

11.12.6. Limits

Individual keys have tests, but the Cartesian product of every YAML value and study switch is not covered. Use Section 11.8 to distinguish constrained combinations from missing numerical evidence.

The complete switch inventory is Section 11.8.

12. Verification catalogue

12.1. Overview and code-map verification

12.1.1. Summary

> **i** At a glance
>
> This catalogue distinguishes test assertions from executed results. No snapy binary or installed snapy package was used for this draft. Report-only algebra checks run on CPU and import no snapy.

The tests below support Section 1.5 through Section 1.3. A tolerance copied from a test is a specification, not a measured error. Missing tolerance rationale is retained as missing evidence rather than invented.

12.1.2. Derivation

The derivations of Equation 1.2, Equation 1.4 and Equation 1.5 are checked by the report-only stage check. There is no independent verification-catalogue derivation.

12.1.3. Numerical method

Read the test source, confirm its registration and device selection, then run a package built from the stated pin. The flow in Figure 1.5 separates stage execution from step acceptance. Test outcomes depend on build options; this source inventory is not a generated `ctest -N` list.

12.1.4. Code

Table 12.1.: Test environment contracts.

what	code link	symbol	switch
Python origin	`CMakeLists.txt:350-362`	`SNAPY_TEST_PYTHONPATH`	CMake path
Device parameters	`device_testing.hpp:33-50`	`DeviceTest`	CUDA availability

1. Set `SNAPY_TEST_PYTHONPATH` to the pin's installed build (`CMakeLists.txt:350-362`).
2. Verify the imported origin with `where` before trusting Python numerical results (`test_python_import_path.py:1-23`).
3. Record actual device and skipped cases from the `DeviceTest` selection (`device_testing.hpp:33-50`).

The registration anchor `setup_test` is in `macro_setup_test.cmake:8-34`.

The registration anchor `test_yaml_keys` is in `CMakeLists.txt:7-29`.

The registration anchor `snapy_add_python_test` is in `CMakeLists.txt:204-226`.

The registration anchor `restart_tests` is in `CMakeLists.txt:418-441`.

The `.release` names in this catalogue use the release build suffix; other configurations substitute their own suffix.

12.1.5. Tests

Table 12.2.: Source assertions for ch01.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_restart_cycle_limit`	Resume cycle and logged values	absolute 1e-12; time 1e-9	Runner constants; rationale not documented
`test_check_redo_parallel.release`	Collective redo and rollback	exact equality	Saved tensors are copied back
`test_yaml_keys.release`	Unknown key path appears in error	exact substring	Configuration contract
`test_user_output.release`	Ordered hydro/scalar increments	rtol = atol = 1e-12	Test constants; rationale not documented
`test_user_output.release`	Shared callback; bad keys throw	pointer equality; allclose defaults	API contract; numeric rationale absent
`test_jit_user_forcing_python`	Nested scripted module adds increment	assert_close defaults	Library defaults, not specified by test

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_mesh_multi_block.release`	Local and remote neighbor ghosts	allclose defaults	Library defaults, not exact equality
`test_mesh_exchange_python`	Selected neighbor slab	exact equality	Integer rank field copied
`test_weno5_cuda_line.release`	CUDA and CPU reconstruction agree	absolute < 1e-12	Test constant; rationale not documented
`test_coordinate.release`	Device/dtype parameter coverage	per assertion	Parameter list is not a tolerance
`test_scalar.release`	Initialization, shape, finite increment	allclose defaults; exact shape	API contract; numeric rationale absent
`test_scalar.release`	Positive density, bounded ratio	1e-5 float32; 1e-12 float64	Test constants; rationale not documented
`test_python_import_path_python`	Import origin and PYTHONPATH ordering	exact path comparisons	Package-under-test contract

The assertion anchor `first_restart` is in `run_restart_cycle_limit.py:168-198`.

The assertion anchor `check_redo` is in `test_check_redo_parallel.cpp:43-68`.

The assertion anchor `expect_refused` is in `test_yaml_keys.cpp:22-64`.

The assertion anchor `scripted_stage_forcings_add_tendencies_in_list_order` is in `test_user_output.cpp:576-606`.

The assertion anchor `scripted_module_is_shared_across_parallel_blocks` is in `test_user_output.cpp:611-667`.

The assertion anchor `expected` is in `test_jit_user_forcing.py:44-71`.

The assertion anchor `side_is_uniform` is in `test_mesh_multi_block.cpp:19-95`.

The assertion anchor `ghost` is in `test_mesh_exchange.py:37-50`.

The assertion anchor `expect_matches_cpu` is in `test_weno5_cuda_line.cpp:83-121`.

The assertion anchor `DeviceTest` is in `device_testing.hpp:33-50`.

The assertion anchor `scalar_s` is in `test_scalar.cpp:91-104`.

The assertion anchor `tol` is in `test_scalar.cpp:159-163`.

The assertion anchor `where` is in `test_python_import_path.py:1-23`.

The restart runner uses two processes and checks the first resumed cycle against the uninterrupted run, then checks termination at its configured cycle limit. The parallel redo test likewise requires a process group and restores saved tensors exactly. The C++ multi-block exchange test uses two local blocks and sees both local and remote neighbors, but checks floating values using library-default `allclose`. The Python exchange test uses four local blocks in one process and exact equality for its selected neighbor slab; it does not inspect every face.

The scripted forcing tests are CPU source tests. The scalar and coordinate device fixture enumerates CPU and CUDA in float32 and float64 and skips

unavailable CUDA; it does not instantiate MPS. The long-line reconstruction test compares float64 CPU/CUDA at lengths 32, 1024, 1025, 1030 and 1280 and also tests multiple lines. These are test inputs at the pin, not successful-run counts. The import-path test exits with skip status 125 when the target package path is absent. No CPU, CUDA or multi-rank snapy runtime pass is claimed in this catalogue.

Report-only checks `[C1]`, `[C2]` and `[C3]` have committed output. Symbolic residuals must be exactly zero. The numerical transfer checks permit absolute error of 64 float64 machine epsilons times the larger of one and the sum of absolute face transfers, allowing the short sequence of upwind products, differences and sums. The dry-source branch checks use the same 64-epsilon scale for unit-sized examples. These are algebra checks, not a substitute for an execution of the C++ branches or a convergence ladder.

12.1.6. Limits

Dedicated evidence is missing for split-source ghost freshness, initialization-versus-restart ghost equivalence, all `part` extension combinations, MPS dispatch, and the imported dependency stack at this pin. No runtime switch-combination coverage is added to Section 11.8. Before publication, run the selected CPU tests with verified package origin, record their outcomes and explain the inherited tolerances that lack a rationale. CUDA and multi-rank results need their own provenance and cannot be inferred from a CPU algebra check.

12.2. Thermodynamics evidence

The following is a source inventory at snapy `e894700ff7aee30b52882e5202b16461413780b0`, not a run log. No compiled snapy or kintera tests were run for this draft. The report's independent CPU formula checks

have their own committed outputs; they cannot certify dispatch, CUDA kernels or communication.

12.2.1. EOS maps and reference energies

`moist_mixture` checks round trips and cache refresh in `test_eos.cpp:122-166`. The offset tests are `test_eos.cpp:168-207`.

The ideal-moist temperature sweep uses 50, 100, 200, and 400 K, with relative threshold 10^{-12}, affinity threshold 10^{-12} and offset threshold 10^{-10}; these are regression tolerances, not measured errors: `test_eos_temp2inteng.py:76-155`.

`test_eos_species_registry_python` uses relative 10^{-14} for conversions and exact equality for step output after rewriting the global tables: `test_eos_species_registry.py:93-122`.

The two-card parent split uses 10^{-10} for float64 and 10^{-4} for float32: `test_two_cards_species.cpp:78-90`.

The registry threshold `TOL` is defined in `test_eos_species_registry.py:28-29`.

12.2.2. Flux enthalpy and phase adjustment

The NASA/H2 enthalpy carry assertion includes dry gas and kinetic energy. Its relative tolerance is 10^{-9}, chosen to distinguish the omitted-kinetic-energy error per the source comment: `test_flux_positivity_carry.cpp:424-458`. No historical residual from that comment is treated as a new run.

The wall test requires actual phase change and relative energy and water drift below 10^{-12}: `test_wall_saturation.cpp:63-89`. It is parameterized for CPU and, when compiled, CUDA; neither was executed here.

12.2.3. Registration, gaps and reproducible report checks

The temperature and registry Python tests have default CPU registrations: `CMakeLists.txt:245-256`. Shallow-water examples require `FULL_TESTS`: `CMakeLists.txt:368-379`.

The old 14-variable EOS round trips are commented out: `test_eos.cpp:209-273`. ANEOS has no runtime evidence. A moist-mixture temperature-map sweep, compiled H2 heat-capacity plot and full phase active-set tests remain follow-up evidence. No thermodynamic multi-rank coverage is claimed.

Run `python3 src/snapy_report/ch02/<scheme>_check.py` for each non-interface scheme. The sibling `.out` files record `[C1]` results on the synthetic decks in `formulas.py`. These verify derivation identities with CPU float64 or symbolic algebra, not compiled source behavior.

12.3. Grids and geometry: tests and checks

This catalogue accompanies Chapter 3. The source pin is `snapy@e894700ff7aee30b52882e5202b16461413780b0`. No snapy executable, installed snapy package, CUDA test or multi-rank test was run for this draft; "registered" below describes source evidence, not a measured pass.

12.3.1. Independent report checks

The command `python3 src/snapy_report/ch03/geometry_check.py` checks the closed forms of chapter 3 with numpy and sympy, without importing snapy. Its committed output is `src/snapy_report/ch03/geometry_check.out`; the pytest wrapper `src/tests/test_checks.py` runs it with every other chapter's checks.

Table 12.3.: Independent checks of chapter 3 and their limits.

output label	assertion	tolerance	what it does not establish
`[C1]-[C3]`	radial face centroids, moment and shift; the test rationals	exact symbolic	the compiled helpers
`[C4]-[C5]`	spherical-polar averages, centroid, face-pressure and angular-momentum identities	exact symbolic	the code's area factor (Section 3.3)
`[C6]-[C9]`	gnomonic angle, solid angle, lower/raise, uniform-pressure flux	10^{-8} to 10^{-15}	the velocity source terms
`[C10]`	Morton order of a 4×4 block grid	exact	3-D order, cubed-sphere corners
`[C11]`	cross-panel slide below $n_g - \frac{1}{2}$	exact bound	the exchange itself

12.3.2. Registered tests

Table 12.4.: Registered tests that chapter 3 relies on. C++ entries carry the build suffix (`.release`).

test (ctest name)	assertion	tolerance	pinned source
`test_coordinate.<b>`	decomposed faces bitwise; programmatic grids; spherical-polar and gnomonic metric, transforms and face-pressure sources	bitwise to 10^{-4}	`geometry_matches_athena_reference_formulas test_coordinate.cpp: 146-221`
`test_radial_face_moments.<b>`	moment and shift equal the exact rationals	10^{-14} relative	`closed_forms_match_the_exact_integrals test_radial_face_moments.cpp: 49-91`
`test_cubed_sphere_cell_volume_python`	six panels sum to the shell volume; $\nabla \cdot (r\hat{\mathbf{r}}) = 3$; rest	10^{-12}; 10^{-6}	`ROUNDOFF test_cubed_sphere_cell_volume.py: 126-137`

test (ctest name)	assertion	tolerance	pinned source
`test_cubed_sphere_exchange.<b>`	nb2 = 1, 2, 4 bitwise after one stage; tag collision refused	bitwise	`subdivided_panel_exchange_matches_one_block test_cubed_sphere_exchange.cpp:150-200`
`test_local_horizontal_cells_python`	the horizontal setter resolves every axis	$10^{-12}\Delta x$	`NG test_local_horizontal_cells.py:28-37`
`test_mesh_exchange_python`	slab ghosts hold the neighbour's payload	exact	`torch.all test_mesh_exchange.py:20-50`
`test_cubed_sphere_vertical_velocity_exchange_python`	radial velocity crosses panel edges unchanged	10^{-12}	`ideal_match test_cubed_sphere_vertical_velocity_exchange.py:95-110`
`test_exchange.<b>`, `test_exchange_decomp`	neighbour payloads, local and remote	value bounds	`ghosts_match_expected test_exchange.cpp:135-180`

test (ctest name)	assertion	tolerance	pinned source
`test_flux_covariance_seams_python`	seam conservation with the #289 term	10^{-12} drift	`DRIFT_TOL test_flux_covariance_seams.py:34-93`
`test_flux_positivity_cubedsphere_python`	tracer conservation across a seam with the limiter	10^{-13} drift	`DRIFT_TOL test_flux_positivity_cubedsphere.py:118-135`
`test_pref_local_seam.<b>`	in-process x_1 split equals one block	10^{-6}; 10^{-12}	`local_blocks_restart_the_reference_at_the_seam test_pref_local_seam.cpp:99-143`
`test_x1_seam_split*.<b>`, `test_x1_seam_split_mp*.<b>`	split column equals one block, per switch	10^{-13}	`split_gap test_x1_seam_split.cpp:202-286`
`test_wb_wall_corner.<b>`	wall corners at rest; x_2 split exact	10^{-12}; 0	`user_wall_keeps_u2_zero test_wb_wall_corner.cpp:160-190`

12.3.3. Gaps

No test checks the spherical-polar m_2 and m_3 curvature sources, the synthesised x_2-x_3 ghost corners, the face-state suffix rule on its own, the `cylindrical` type, or `DeviceTest.interpolate_LR` and `GnomonicEquiangle.area_vol`, which print and assert nothing.

12.4. Hydrostatic and well-balanced treatment: test evidence

This catalogue accompanies Chapter 5 (Chapter 5). It lists every ctest entry that carries a Chapter 5 claim at the pin `snapy@e894700ff7aee30b52882e5202b16461413780b0`, with what it asserts, its tolerance and the outcome of one run, and the claims of the chapter's executable check.

12.4.1. The run

All entries below were run once [sha `e894700ff7aee30b52882e5202b16461413780b0`; run: `ctest -V` on a Release build of the pin with CUDA off, 2026-10-10, CPU, double; 1 rank, the `_mp` entries 2 ranks under `torchrun`]. The 26 C++ and CMake entries passed. The two Python entries failed under ctest, because the interpreter imported an older installed snapy instead of the pin's build (`unknown key 'forcing/const-gravity/gravity-work'`); run directly against the pin's build (`python tests/test_wb_ref4_order.py`, `python tests/test_x1_centroid_rest.py`) both pass. Every `_cuda` case was skipped by the build. **Pending: a CUDA run at the pin, and a ctest run of the two Python entries with the import path set to the pin's build.**

12.4.2. ctest entries

Table 12.5.: Chapter 5 ctest entries at the pin and the outcome of one CPU run. The radial-exact seam arms and the mass-covariance arm of `test_x1_seam_split` belong to Chapters 6 and 4.

test (ctest name)	what it asserts	tolerance	outcome
`test_hydro_ref_x1.release`	CPU path equals the tensor path; with the wall clamp ghosts do not reach the interior, without it they do; thin blocks; wall reference independent of the block size	2×10^{-12} double; exact; 10^{-14}	pass (6), CUDA case skipped
`test_hydro_options.release`	`wb-wall-clamp` ships enabled and changes the reference	differ $> 10^{-13}$	pass (11), CUDA case skipped
`test_wb_ref_wall.release`	ρ_{sf}, ρ_L, ρ_R at faces 1-2 next to both walls second order, n_1 32 $\rightarrow$ 64; wall faces within 1.5$\times$ the interior	order ≥ 1.7	pass; orders 1.81-2.12

test (ctest name)	what it asserts	tolerance	outcome
`test_wb_ref4_order_python`	switch on: order ≥ 2.75 at 1 and 3 e-folds; off: < 2.5	`ORDER_ON 2.75`, `ORDER_OFF 2.5`	pass when run against the pin; orders on 3.88-4.07, off 2.04-2.21
`test_x1_centroid_rest_python`	switch on: rest force $< 10^{-10}$; off: $> 10^{-8}$ at $r_0/H = 5$	`TOL_ON 1e-10`, `TOL_OFF 1e-8`	pass when run against the pin; on $\leq 3.3 \times 10^{-14}$, off 2.4×10^{-5}
`test_pref_local_seam.release`	p_{ref} continuous at an in-process seam; four blocks equal one after 200 steps	10^{-6}; 10^{-12}	pass (2)
`test_x1_seam_split.release`	the switches-off arm sets up with $n_g = 1$	runs	pass
`test_x1_seam_split_x1_centroid.release`	split equals one block, n_h 1 and 0	10^{-13}	pass; gap 0

test (ctest name)	what it asserts	tolerance	outcome
`test_x1_seam_split_wb_ref4.release`	split with the resolution flag at the seam equals one block	10^{-13}	pass; gaps 6.8×10^{-15}, 1.7×10^{-15}
`test_x1_seam_split_wb_ref4_gravity_0.release`	gravity off, $n_g = 1$ steps under the switch	runs	pass
`test_x1_seam_split_mp.release`, `_x1_centroid`, `_wb_ref4` (2 ranks)	two ranks equal two blocks and one block	10^{-13}	pass; gaps 0 to 6.8×10^{-15}
`test_face_floor.release`, `_wb_ref4`, `_x1_centroid`	adjacent-density floor; switched fluxes; unresolved column flux 2.83191×10^{-8}	10^{-3} abs; 10^{-3}, 10^{-5} relative	pass (2 each); flux 2.8319×10^{-8} in all arms
`test_balance_column.release`, `_wb_ref4`, `_x1_centroid`	transfer to a block bitwise; marched and moist columns come out at rest; fixed point; refusals; switch predicates	bitwise; 10^{-10}; 10^{-14}	pass (11 each), CUDA cases skipped
`test_hydrostatic.release`	cubed-sphere atmosphere at rest, `non-hydrostatic: 0.`	10^{-8}	pass; 7.84×10^{-15}

test (ctest name)	what it asserts	tolerance	outcome
`test_wb_wall_corner.release`	balanced polytrope keeps $\lvert u_2\rvert/c_s$, $\lvert v\rvert/c_s \le 10^{-12}$; x_2 split equals one block	10^{-12}; exact	pass (4), CUDA cases skipped
`test_bryan_balance_ic`	one pass fails with the cap error; default converges; dry case projects once	exit status, output	pass; default stops after pass 6
`test_straka_redo`	straka at CFL 1.6 to 60 s exits 0 and redoes a step	exit status, output	pass

The CUDA registrations of these files (`test_wb_ref4_order_cuda`, `test_x1_centroid_rest_cuda`, the `_cuda` seam arms and the `_cuda` gtest cases inside the CPU binaries) were not run.

12.4.3. The chapter's executable check

`python3 src/snapy_report/ch05/wbref_check.py` runs on CPU without importing snapy; its committed output is `src/snapy_report/ch05/wbref_check.out`, and `src/tests/test_ch05.py` runs it with the chapter's figure tests. One run [numpy 2.2.4, sympy 1.14.0, double] passes all 21 claims.

Table 12.6.: Claims of the Chapter 5 check and their limits.

output label	assertion	tolerance	what it does not establish
`[C1]`, `[C2]`	six-face interior and wall-row weights	exact rationals	the guards around them
`[C3]`, `[C7]`, `[C8]`	filter moments, cubic wall values, quartic-primitive face weights	exact rationals	the code's stencil assembly
`[C4]`–`[C6]`	repeated wall cell first order; continuation second order; continuation bounds	exact; sampled	non-uniform grids
`[C9]`, `[C10]`, `[C20]`	log-mean, top anchor, uniform test	10^{-14}; closed form	stretched-grid dynamics
`[C11]`	independent rebuild reproduces `test_wb_ref_wall`'s 40 printed numbers	6×10^{-4} relative (printed digits)	the solver's ρ_L, ρ_R

output label	assertion	tolerance	what it does not establish
`[C12]`–`[C14]`	wall orders; six-face order 5.87; face-density offset formula, base order 2.00 and `SNAP_WB_REF4` order 4.68	fitted orders	solver dynamics
`[C15]`	`balance_column`'s iteration converges on a perturbed column	residual $< 10^{-10}$	a convergence proof
`[C16]`–`[C18]`	plain-mean conversion exactness; pressure-source identities; centroid offset	10^{-12}; exact	compiled ghost behaviour
`[C19]`	hydrostatic-mode correction cancels at rest	10^{-12}	behaviour away from rest
`[C21]`	Favre ratio of r^2 means against the plain mean	exact	the size of the $1/R$ remainder

12.5. Moist physics coupling evidence

The following is a source inventory at snapy `e894700ff7aee30b52882e5202b16461413780b0`, not a run log. No compiled snapy test was run for this draft; every tolerance below is read from the test source, and every measured value is missing evidence until a pinned build runs the entry. `<b>` is the build suffix, normally `release`. The report's own CPU checks are listed last. "CUDA case" means the test fixture also runs on CUDA, in float32 and float64, and skips that case at run time without a device; "if built" means the CUDA case exists only in CUDA builds. The checks verify formulas ported from the code and import no snapy.

12.5.1. Saturation adjustment and redo

Table 12.7.: Evidence for Section 9.1, Section 9.2 and Section 9.5.

ctest	asserts	tolerance	device, ranks
`test_wall_saturation.<b>`	energy and water at a wall, condensing and evaporating	10^{-12} rel.	CPU; CUDA if built
`test_balance_column.<b>` (moist case)	moist column at rest; condensate stays 0	10^{-10}; 10^{-12}	CPU
`test_vic_moist_device_python`	CPU and CUDA agree after one step	10^{-9}	CUDA builds only
`test_check_redo_saturation_python`	saturation cause alone; drained; Mesh arm	logical	CPU

ctest	asserts	tolerance	device, ranks
`test_check_redo_saturation_cuda_python`	the same on CUDA	logical	CUDA builds only
`test_check_redo_parallel.<b>`	one decision across ranks	exact	2 ranks
`test_check_redo_floor_python`	floor and NaN causes, Mesh arm	logical	CPU
`test_forcing.<b>` (limiter cases)	species repair redoes; round-off does not	ulp bound	CPU
`test_uranus_cycle1_abort.<b>`	Uranus column through `run_hydro`	exit status	CPU

The wall test is registered once (`CMakeLists.txt:117`, `test_wall_saturation`); the moist balance case runs in `test_balance_column` and in its `_wb_ref4` and `_x1_centroid` arms (`CMakeLists.txt:96-101`, `test_balance_column`). The CUDA arms are registered only in CUDA builds (`CMakeLists.txt:283-285`, `test_check_redo_saturation_cuda`). The moist balance numbers in the moist balance-column team note under `sources/team-notes/` were measured at snapy@fc3534c34d c514805b80ba3ac75bf14b80dd4e12, before the pin, and are not used as evidence here.

12.5.2. Species repairs

Table 12.8.: Evidence for Section 9.3 and Section 9.4.

ctest	asserts	tolerance	device, ranks
`test_condensate_conservation.<b>`	stoichiometric debit of two parents	10^{-12}; 10^{-6}	CPU; CUDA case
`test_cloud_parent_slots.<b>`	parents with unused card species	10^{-12}; 10^{-6}	CPU; CUDA case
`test_diffusion_moist.<b>`	limiter reads nucleation parents	10^{-12}; 10^{-6}	CPU; CUDA case
`test_two_cards_species.<b>`	split and settling after a second card	10^{-10}; 10^{-4} f32	CPU; CUDA case
`test_eos.<b>` (`eos_limiter.*`)	five fix_vapor columns	10^{-14} rel.	CPU
`test_fix_vapor_volume.<b>`	column mass on unequal volumes	10^{-12}; 10^{-6} f32	CPU; CUDA case
`test_parentless_cloud.<b>`	parentless cloud kept, clamped, zero	10^{-12}; 10^{-6} f32	CPU; CUDA case
`test_parentless_cloud_nb1.<b>`	the same with one and two x_1 blocks	10^{-12}; 10^{-6} f32	CPU; CUDA case
`test_parentless_cloud_nb1_mp.<b>`	across two processes	10^{-12} rel.	2 ranks; Gloo arm with UCX

ctest	asserts	tolerance	device, ranks
`test_vapor_column_nb1.<b>`	split vapour column equals one block	bitwise	CPU; CUDA case
`test_fix_vapor_reports_failure_python`	unrepairable column throws	message	CPU; CUDA arm if built
`test_fix_vapor_counts_every_column_python`	one broken column of 16384 found	message	CPU

The single-process repair tests are registered together (`CMakeLists.txt:110-119`, `setup_test`); the two-process test and its Gloo arm (`CMakeLists.txt:144-151`, `test_parentless_cloud_nb1_mp`). Chapter 2's catalogue (Section 12.2) also lists `test_two_cards_species`; its rows are the EOS view of the same test.

12.5.3. Tracers and sedimentation

Table 12.9.: Evidence for Section 9.6 and Section 9.7.

ctest	asserts	tolerance	device, ranks
`test_scalar.<b>`	transport; bound holds both sides	10^{-12}; 10^{-5} f32	CPU; CUDA case

ctest	asserts	tolerance	device, ranks
`test_tracer_dry_convention_python`	init, implicit and relax arms	10^{-12}, 1000-fold signal	CPU
`test_stage_forcing_dry_tracer_python`	user dry sources carry tracers	per arm	CPU
`test_dry_carry_zero_base_python`	0/0 guard of the carry	10^{-12} abs.	CPU
`test_flux_positivity_cubedsphere_python`	dry tracers across panel seams	per arm	CPU; CUDA arm if built
`test_forcing.<b>` (sedimentation cases)	fused equals tensor; sealing; gravity work	10^{-10} rel.; exact	CPU
`test_sedimentation_guards.<b>`	gravity required; flux off; rising from below	10^{-9} rel.	CPU; CUDA case
`test_sedimentation_cubed_seam.<b>`	seam flux, mass conserved	10^{-12} rel.	2 ranks; Gloo arm with UCX

The Gloo arm of the seam test compares against the UCX result through `run_seam_backends.py` (`CMakeLists.txt:132-141`, `test_sedimentation_cubed_seam_gloo`).

12.5.4. Gaps and the report's own checks

No snapy test covers one kinetics step's mass and energy budget, the evaporation rate, or the `jupiter_evap_precip_1d` driver; no test runs the saturation adjustment on more than one rank or across a decomposed reflecting wall (chengcli/snapy#208); the MPS sedimentation path is untested; and the switch coverage matrix (Section 11.8) has no moist rows.

Run `python3 src/snapy_report/ch10/<scheme>_check.py` for `fixvapor`, `borrow`, `kinetics`, `tracer`, `sed` and `redo`; each prints labelled `[Cn]` lines and exits with its number of failed claims, and the sibling `.out` file is its committed output. `src/tests/test_ch10_checks.py` requires every output to be current. These checks run on CPU in float64 (and float32 where stated) and verify ported formulas, not compiled behaviour.

12.6. Boundary-condition verification (`Chapter 11`)

12.6.1. Summary

> **i** At a glance
>
> Separate source-defined solver assertions from executed report checks. No solver tests run for this chapter; seven report formula checks are available. Default: catalogue only, no solver configuration change.

12.6.2. Derivation

The boundary chapter derives the identities exercised by the report checks. No extra numerical scheme is introduced here. A source assertion is evidence of intended coverage, not evidence that a pinned build passed. All solver entries below refer to `e894700ff7aee30b52882e5202b16461413780b0`. Their numerical thresholds are test constants, not measured results.

12.6.3. Numerical method

Run report checks without a compiled solver. For new solver evidence, first configure a CPU-only build at the pin and resolve the pinned dependencies. Record the complete command, deck, dtype, ranks and environment. Run a focused boundary filter before expanding coverage. CUDA and multi-process assertions remain separate claims and are not inferred from a CPU formula check.

Figure 12.1 distinguishes the two evidence paths.

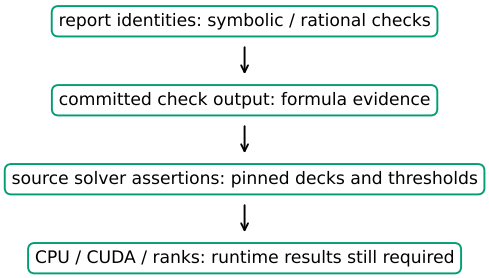


Figure 12.1.: Boundary verification paths: source assertions need a pinned runtime; report identities run independently. No solver result is implied.

12.6.4. Code

Table 12.10.: Boundary test registration anchors.

what	code link	symbol	switch
core test registrations	`CMakeLists.txt: 5-33`	`test_radiating_boundary`	build configuration
wall tests	`CMakeLists.txt: 91-117`	`test_wb_ref_wall`	reference variants
outflow Python API	`CMakeLists.txt: 192-200`	`test_radiating_boundary_python`	Python
implicit solid runner	`CMakeLists.txt: 232-235`	`test_implicit_stratified_solid`	Python
two-process exchange	`CMakeLists.txt: 120-123`	`test_exchange`	parallel
Python exchange	`CMakeLists.txt: 223-225`	`test_mesh_exchange`	Python
positivity inventory	`CMakeLists.txt: 248-253`	`test_flux_positivity`	Python

1. Execute the report derivation script and retain its output.
2. Use `test_radiating_boundary` registration (`CMakeLists.txt:5-33`) for the compiled characteristic tests.
3. Use `test_exchange` registration (`CMakeLists.txt:120-123`) for the distinct two-process exchange path.
4. Retain failures and skips verbatim; no runtime result is attached to this draft.

12.6.5. Tests

Table 12.11.: Source assertions only; all runtime results pending.

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_radiating_boundary.release`	Modes on six faces, three EOS types	0.03 float32; 1e-8 float64	`radiating_modes_all_faces`
same test, mixed faces	Face order and representation	Hydro rtol 1e-12, atol 1e-9	`mixed_faces_preserve_representation_and_order`
same test, dry tracers	Per-dry-mass ratios	Below 1e-12	Dry-tracer assertions
same test, pulses	PLM/WENO5, both signs	Outflow reflection below 0.05	`acoustic_pulses`
same test, restart	Saved reference and continuation	Exact equality; missing reference throws	Restart assertions
same test, joint factor	Composition/tracer convexity	Inequalities and allclose defaults	Joint-limiter assertions
same test, devices	CPU/CUDA agreement	rtol = atol = 1e-10	Device comparison
`test_radiating_boundary_python`	Manual fill preserves reference/interior	Exact equality	Python assertions
`test_yaml_keys.release`	Unknown key path rejected	Exact string containment	Validation helper

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_wb_wall_corner.release`	Rest after wall/corner fill	Mach at most 1e-12	Rest assertions
`test_wall_saturation.release`	Phase-change wall energy/water	Relative error below 1e-12	Conservation assertions
`test_rectify.release`	All ghost slabs solid	Integer equality	Mask fixture
`test_flip_zero_count.release`	Parallel count equals serial	Integer equality, 20 repeats	Thread fixture
`test_implicit_stratified_solid_python`	Fluid segment mass and finite state	Below 1e-12	Runner thresholds
`test_wb_ref_wall.release`	Near-wall reference order	Two-grid slope at least 1.7	Reference assertions
`test_hydro_ref_x1.release`	Clamp excludes ghost perturbations	Exact equality; off-arm inequality	Property tests
`test_cubed_sphere_exchange.release`	Panel subdivision preserves fields	Exact equality	Panel assertions
`test_mesh_exchange_python`	Neighbour identity	Exact equality	Scalar ghost assertions

test (ctest name)	what it asserts	tolerance	where the tolerance comes from
`test_cubed_sphere_vertical_velocity_exchange_python`	Uniform vertical velocity preserved	abs/rel 1e-12	Explicit constants

The `EXPECT` anchor is in `test_radiating_boundary.cpp:46-98`.

The `EXPECT` anchor is in `test_radiating_boundary.cpp:177-259`.

The `EXPECT` anchor is in `test_radiating_boundary.cpp:288-360`.

The `EXPECT_LT` anchor is in `test_radiating_boundary.cpp:395-441`.

The `EXPECT_THROW` anchor is in `test_radiating_boundary.cpp:445-468`.

The `alpha` anchor is in `test_radiating_boundary.cpp:536-561`.

The `allclose` anchor is in `test_radiating_boundary.cpp:563-578`.

The `assert` anchor is in `test_radiating_boundary_python.py:16-36`.

The `EXPECT` anchor is in `test_yaml_keys.cpp:37-64`.

The `EXPECT` anchor is in `test_wb_wall_corner.cpp:166-190`.

The `1.e-12` anchor is in `test_wall_saturation.cpp:59-89`.

The `test2` anchor is in `test_rectify.cpp:91-124`.

The `serial` anchor is in `test_flip_zero_count.cpp:12-48`.

The `solid_run` anchor is in `test_implicit_stratified_solid.py:99-141`.

The `1.e-12` anchor is in `test_implicit_stratified_solid.py:277-291`.

The `EXPECT_GE` anchor is in `test_wb_ref_wall.cpp:229-264`.

The `EXPECT` anchor is in `test_hydro_ref_x1.cpp:217-247`.

The `torch::equal` anchor is in `test_cubed_sphere_exchange.cpp:161-196`.

The `assert` anchor is in `test_mesh_exchange.py:24-49`.

The `ideal_match` anchor is in `test_cubed_sphere_vertical_velocity_exchange.py:81-109`.

The `ABS_TOL` anchor is in `test_cubed_sphere_vertical_velocity_exchange.py:7-15`.

The `saw_remote_neighbor` anchor is in `test_exchange.cpp:292-317`.

The all-face mode suite is device-parameterized; the explicit CPU/CUDA comparison skips when CUDA is unavailable. The panel and wall-reference suites have explicit CPU and conditional CUDA arms. Saturation instantiates CPU and CUDA when built with CUDA. Rectification chooses CUDA when the test helper enables it; the flip-count test is explicitly a CPU one-thread/eight-thread comparison. The Python solid runner exposes a device in its result, so the actual run must record that choice. A local multi-block panel assertion does not imply more than one process; the exchange registration separately requests two processes.

The outline's positivity suites remain an inventory here: their detailed flux budgets belong to the positivity chapter. The source registration links above identify the dry and moist panel variants without claiming they passed. The pulse comparison does not assert the extrapolation arm. The two-grid reference threshold does not satisfy the three-resolution rule for a newly measured report order.

The report checks are `src/snapy_report/ch11/boundary_check.py`, with committed output `src/snapy_report/ch11/boundary_check.out`. They check C1 copy/parity/corner ownership, C2 acoustic inversion, C3 shared admissibility, C4 corner averaging, C5 DP optimality against exhaustive enumeration, C6 block closure, and C7 reference moments/continuation. C5 covers 508 binary inputs; all checks use exact symbolic or rational identities, so their expected residual is exactly zero. These numbers are produced by that script, not snapy.

12.6.6. Limits

No compiled CPU, CUDA or multi-rank solver test has run for this chapter. Runtime sound-speed/EOS behaviour, nonorthogonal momentum conversion, outflow with VIC/WB, thin solid geometries, scalar/diffusion solid coupling, and stretched-grid wall convergence remain missing evidence. A future measured wall-order claim needs at least three resolutions at the pin. The complete configuration inventory is Section 11.8.

12.7. Configuration and switch evidence

This catalogue accompanies Section 11.8. The source pin is `snapy@e894700ff7aee30b52882e5202b16461413780b0`. No snapy executable, installed snapy package, CUDA test or multi-rank test was run for this draft. "Registered" below describes source evidence, not a measured pass.

12.7.1. Independent report checks

The command `python src/snapy_report/ch12/configuration_check.py` runs exact rational/symbolic identities on CPU without

importing snapy. The committed output is `src/snapy_report/ch12/configuration_check.out`. The pytest wrapper is `src/tests/test_ch12_checks.py`.

Table 12.12.: Independent checks and their limits.

output label	assertion	tolerance	what it does not establish
`[C1]`	18 parser cases and 8 implication cases	exact truth table	execution of C++ static caching
`[C2]`	filter, wall continuation and face moments	exact rational zero	measured solver order
`[C3]`	five spherical moments and six pressure identities	exact symbolic zero	compiled geometry/ghost behavior
`[C4]`	covariance, mass-weighted mean and shared-face cancellation	exact symbolic zero	nonlinear MC accuracy or MPI exchange
`[C5]`	energy increment cancels the change of corrected potential energy	exact symbolic zero	limiter, solid and implicit implementation

12.7.2. Registered setup and provenance tests

Table 12.13.: Source-level setup evidence.

test (ctest name)	assertion	tolerance	pinned source
`test_python_import_path_python`	package under supplied root and caller entry retained	exact paths; 125 when unconfigured	`test_python_import_path.py:7-23`
`test_process_group.<b>`	backend precedence and rendezvous port rules	exact strings/exception	`test_process_group.cpp:74-186`
`test_x1_seam_split_x1_mass_covariance.<b>`	two owned cells rejected, three accepted	exception/no-throw	`test_x1_seam_split.cpp:327-360`
`test_hydro_options.<b>`	implicit type rejection and wall-clamp default/override	exception and boolean	`test_hydro_options.cpp:195-258`
`test_yaml_keys.<b>`	unknown gravity, geometry, reconstruction and solver keys rejected	exception	`test_yaml_keys.cpp:63-93`

The ghost-depth rejection and zero-gravity exception are included with the seam tests. A CTest filter must execute its one selected assertion; registration's pass marker guards against an empty filter. Positive mass-

covariance accuracy remains missing even though its minimum-cell guard is present.

12.7.3. Numerical switch oracles

Table 12.14.: Source thresholds are acceptance criteria, not observed values.

family	registered devices / ranks	assertion tolerance (not a new measurement)	source
WB order	CPU and CUDA; one process	on order at least 2.75; off below 2.5, at three resolutions	`test_wb_ref4_order.py:34-44`
XC rest	CPU and CUDA; one process	on below 10^{-10}; near-origin off above 10^{-8}	`test_x1_centroid_rest.py:37-44`
radial exact	CPU and CUDA; one process	relative budget 10^{-14} per step; log agreement 10^{-11}	`test_gravity_work_radial_exact.py:50-57`
FC onset	CPU Python registration	scaled onset error below 0.04; budget 10^{-12}	`test_horizontal_flux_covariance.py:33-41`
FC rows	CPU Python registration	rest 10^{-9}, fraction 10^{-13}, offset 10^{-10}	`test_flux_covariance_rows.py:32-40`

family	registered devices / ranks	assertion tolerance (not a new measurement)	source
FC panel seams	CPU; six blocks in one process	relative total drift below 10^{-12}	`test_flux_covariance_seams.py:33-37`
x1 split	CPU/CUDA single-process and separate CPU two-rank registrations	one-process state/budget gap at most 10^{-13} as selected by arm	`test_x1_seam_split.cpp:232-297`

The tolerance choices separate roundoff budgets, deliberate nonzero controls and discretization-order thresholds. They are the pinned tests' contracts, not new tolerance recommendations. Read Section 11.8 for the exact names and environment of each arm. The two-rank registration does not turn the FC panel test into an MPI test.

12.7.4. Reproduction and missing evidence

Use a CPU-only build for new configuration evidence. Record the full source pin, cache options, package import origin, test name, complete scheme environment, device, precision and rank count with each result. First run the import-path and setup tests, then the desired numerical arms in fresh processes. The model used by a Python assertion must come from the intended checkout rather than an unrelated installed package.

No build is required for the exact report identities. Official citation checking and HTML/PDF rendering remain pending because the frozen tree has

neither the official checker nor the book skeleton, and Quarto is not installed. The chapter provides scheme includes for editor integration rather than modifying editor-owned chapter/configuration files. Missing combinations are numerical MC; XC+face/RE; XC+FC; WB/XC gnomonic; WB+face with RE off; and FC across ranks.

The two-rank assertions are `split_across_two_ranks_matches_one_process` in `test_x1_seam_split_mp.cpp:247-299`.

A. Test index

The 115 tests that snapy's CMake registers at `e894700ff7aee30b52882e5202b16461413780b0`, one row each, sorted by name. The build was configured with `-DBUILD_EXAMPLES=OFF -DBUILD_TESTS=ON -DCMAKE_BUILD_TYPE=Release -DCUDA=OFF -DFULL_TESTS=OFF -DNETCDF=ON -DUCX=ON`; other switches register other tests. `ctest -N` lists 1053; the other 938 are the test suites of dependencies the build fetches (eigen: 938), not snapy's tests.

Table A.1.: The tests registered at chengcli/snapy e894700ff7aee30b52882e5202b16461413780b0.

test	labels	defined at	notes
`test_backward_substitution.release`	—	`setup_test` (`tests/CMakeLists.txt:17`)	
`test_balance_column.release`	—	`setup_test` (`tests/CMakeLists.txt:31`)	
`test_balance_column_wb_ref4.release`	—	`add_test` (`tests/CMakeLists.txt:97`)	
`test_balance_column_x1_centroid.release`	—	`add_test` (`tests/CMakeLists.txt:101`)	

test	labels	defined at	notes
test_bryan_balance_ic	—	add_test (tests/CMakeLists.txt:390)	
test_check_redo_floor_python	mesh, python, redo	snapy_add_python_test (tests/CMakeLists.txt:245)	
test_check_redo_parallel.release	—	setup_parallel_test (tests/CMakeLists.txt:123)	
test_check_redo_saturation_python	eos, mesh, python, redo	snapy_add_python_test (tests/CMakeLists.txt:246)	
test_cloud_parent_slots.release	—	setup_test (tests/CMakeLists.txt:118)	
test_condensate_conservation.release	—	setup_test (tests/CMakeLists.txt:112)	
test_coordinate.release	—	setup_test (tests/CMakeLists.txt:23)	

test	labels	defined at	notes
test_cubed_sphere_cell_volume_python	coordinate, cubed-sphere, python	snapy_add_python_test (tests/CMakeLists.txt:253)	
test_cubed_sphere_exchange.release	—	setup_test (tests/CMakeLists.txt:22)	
test_cubed_sphere_vertical_velocity_exchange_python	cubed-sphere, diagnostic, exchange, python	snapy_add_python_test (tests/CMakeLists.txt:224)	
test_cycle_diagnostics.release	—	setup_test (tests/CMakeLists.txt:27)	
test_cycle_diagnostics_parallel.release	—	setup_parallel_test (tests/CMakeLists.txt:124)	
test_diffusion.release	—	setup_test (tests/CMakeLists.txt:109)	
test_diffusion_moist.release	—	setup_test (tests/CMakeLists.txt:110)	

test	labels	defined at	notes
test_diffusion_x1_scale.release	—	setup_test (tests/CMakeLists.txt:111)	
test_diffusion_x1_scale_python	forcing, python	snapy_add_python_test (tests/CMakeLists.txt:260)	
test_dry_carry_zero_base_python	forcing, python, scalar	snapy_add_python_test (tests/CMakeLists.txt:258)	
test_eos.release	—	setup_test (tests/CMakeLists.txt:7)	
test_eos_species_registry_python	eos, python	snapy_add_python_test (tests/CMakeLists.txt:256)	
test_eos_temp2inteng_python	eos, python	snapy_add_python_test (tests/CMakeLists.txt:247)	

test	labels	defined at	notes
test_exchange.release	—	setup_parallel_test (tests/CMakeLists.txt:121)	
test_exchange_decomp	decomp, gloo	add_test (tests/CMakeLists.txt:444)	6 processes; runs alone
test_exchange_ucx	exchange, ucx	add_test (tests/CMakeLists.txt:155)	
test_face_floor.release	—	setup_test (tests/CMakeLists.txt:92)	
test_face_floor_wb_ref4.release	—	add_test (tests/CMakeLists.txt:97)	
test_face_floor_x1_centroid.release	—	add_test (tests/CMakeLists.txt:101)	
test_fix_vapor_counts_every_column_python	eos, positivity, python	snapy_add_python_test (tests/CMakeLists.txt:228)	
test_fix_vapor_reports_failure_python	eos, positivity, python	snapy_add_python_test (tests/CMakeLists.txt:227)	

test	labels	defined at	notes
test_fix_vapor_volume.release	—	setup_test (tests/CMakeLists.txt:114)	
test_flip_zero_count.release	—	setup_test (tests/CMakeLists.txt:21)	
test_flux_covariance_rows_python	cubed-sphere, hydro, python	snapy_add_python_test (tests/CMakeLists.txt:262)	
test_flux_covariance_seams_python	cubed-sphere, hydro, python	snapy_add_python_test (tests/CMakeLists.txt:264)	
test_flux_positivity_carry.release	—	setup_test (tests/CMakeLists.txt:28)	
test_flux_positivity_cubedsphere_moist_python	cubed-sphere, positivity, python	snapy_add_python_test (tests/CMakeLists.txt:251)	
test_flux_positivity_cubedsphere_python	cubed-sphere, positivity, python	snapy_add_python_test (tests/CMakeLists.txt:249)	

test	labels	defined at	notes
test_flux_positivity_python	positivity, python	snapy_add_python_test (tests/CMakeLists.txt:248)	
test_forcing.release	—	setup_test (tests/CMakeLists.txt:120)	
test_gravity_work_fixer_python	energy, gravity, hydro, python	snapy_add_python_test (tests/CMakeLists.txt:241)	
test_gravity_work_radial_exact_python	gravity, hydro, python	snapy_add_python_test (tests/CMakeLists.txt:242)	
test_horizontal_flux_covariance_python	hydro, python	snapy_add_python_test (tests/CMakeLists.txt:261)	
test_hydro_options.release	—	setup_test (tests/CMakeLists.txt:26)	
test_hydro_ref_x1.release	—	setup_test (tests/CMakeLists.txt:32)	

test	labels	defined at	notes
test_hydrostatic.release	—	setup_test (tests/CMakeLists.txt:25)	
test_implicit_advection_cfl_python	hydro, implicit, python	snapy_add_python_test (tests/CMakeLists.txt:229)	
test_implicit_cfl.release	—	setup_test (tests/CMakeLists.txt:8)	
test_implicit_face_work_jacobian.release	—	setup_test (tests/CMakeLists.txt:9)	
test_implicit_face_work_operator_python	gravity, hydro, implicit, python	snapy_add_python_test (tests/CMakeLists.txt:231)	
test_implicit_face_work_operator_radial_exact_python	gravity, hydro, implicit, python	snapy_add_python_test (tests/CMakeLists.txt:236)	
test_implicit_gravity_tall_column_python	gravity, hydro, implicit, python	snapy_add_python_test (tests/CMakeLists.txt:230)	

test	labels	defined at	notes
test_implicit_options_type_python	implicit, python	snapy_add_python_test (tests/CMakeLists.txt:243)	
test_implicit_stratified_solid_python	gravity, hydro, implicit, python	snapy_add_python_test (tests/CMakeLists.txt:232)	
test_implicit_stratified_solid_radial_exact_python	gravity, hydro, implicit, python	snapy_add_python_test (tests/CMakeLists.txt:236)	
test_jit_user_forcing_python	forcing, python	snapy_add_python_test (tests/CMakeLists.txt:226)	
test_local_horizontal_cells_python	coordinate, mesh, python	snapy_add_python_test (tests/CMakeLists.txt:259)	
test_lu_failure.release	—	setup_test (tests/CMakeLists.txt:18)	

test	labels	defined at	notes
test_mesh_exchange_python	exchange, python	snapy_add_python_test (tests/CMakeLists.txt:223)	
test_mesh_multi_block.release	—	setup_parallel_test (tests/CMakeLists.txt:441)	
test_netcdf_utils.release	—	setup_test (tests/CMakeLists.txt:105)	
test_output_barrier.release	—	setup_parallel_test (tests/CMakeLists.txt:122)	
test_parentless_cloud.release	—	setup_test (tests/CMakeLists.txt:113)	
test_parentless_cloud_nb1.release	—	setup_test (tests/CMakeLists.txt:115)	
test_parentless_cloud_nb1_mp.release	—	setup_parallel_test (tests/CMakeLists.txt:144)	

test	labels	defined at	notes
test_parentless_cloud_nb1_mp_gloo	—	add_test (tests/CMakeLists.txt:146)	
test_plm.release	—	setup_test (tests/CMakeLists.txt:11)	
test_pref_local_seam.release	—	setup_test (tests/CMakeLists.txt:34)	
test_process_group.release	—	setup_test (tests/CMakeLists.txt:107)	
test_python_import_path_python	python	snapy_add_python_test (tests/CMakeLists.txt:322)	skips on exit 125
test_radial_face_moments.release	—	setup_test (tests/CMakeLists.txt:24)	
test_radiating_boundary.release	—	setup_test (tests/CMakeLists.txt:10)	
test_radiating_boundary_python	boundary, python	add_test (tests/CMakeLists.txt:191)	

test	labels	defined at	notes
test_rectify.release	—	setup_test (tests/CMakeLists.txt:20)	
test_refine.release	—	setup_test (tests/CMakeLists.txt:108)	
test_restart_cycle_limit	examples, restart	add_test (tests/CMakeLists.txt:425)	skips on exit 125
test_restart_dt_change	examples, restart	add_test (tests/CMakeLists.txt:425)	skips on exit 125
test_restart_inplace_reorder	examples, restart	add_test (tests/CMakeLists.txt:425)	skips on exit 125
test_restart_key_collision	examples, restart	add_test (tests/CMakeLists.txt:425)	skips on exit 125
test_restart_multiblock	examples, restart	add_test (tests/CMakeLists.txt:404)	
test_restart_output_schedule	examples, restart	add_test (tests/CMakeLists.txt:425)	skips on exit 125
test_riemann.release	—	setup_test (tests/CMakeLists.txt:13)	

test	labels	defined at	notes
test_scalar.release	—	setup_test (tests/CMakeLists.txt:19)	
test_sedimentation_cubed_seam.release	—	setup_parallel_test (tests/CMakeLists.txt:128)	
test_sedimentation_cubed_seam_gloo	exchange	add_test (tests/CMakeLists.txt:133)	skips on exit 125
test_sedimentation_guards.release	—	setup_test (tests/CMakeLists.txt:30)	
test_shear_cfl_python	hydro, implicit, python	snapy_add_python_test (tests/CMakeLists.txt:244)	
test_stage_forcing_dry_tracer_python	forcing, python, scalar	snapy_add_python_test (tests/CMakeLists.txt:257)	
test_straka	reference	add_test (tests/CMakeLists.txt:373)	
test_straka_redo	—	add_test (tests/CMakeLists.txt:383)	

test	labels	defined at	notes
test_tracer_dry_convention_python	forcing, implicit, python, scalar	snapy_add_python_test (tests/CMakeLists.txt:255)	
test_two_cards_species.release	—	setup_test (tests/CMakeLists.txt:119)	
test_uranus_cycle1_abort.release	—	setup_test (tests/CMakeLists.txt:33)	
test_user_output.release	—	setup_test (tests/CMakeLists.txt:106)	
test_vapor_column_nb1.release	—	setup_test (tests/CMakeLists.txt:116)	
test_wall_saturation.release	—	setup_test (tests/CMakeLists.txt:117)	
test_wb_ref4_order_python	hydro, python	snapy_add_python_test (tests/CMakeLists.txt:266)	

test	labels	defined at	notes
test_wb_ref_wall.release	—	setup_test (tests/CMakeLists.txt:93)	
test_wb_wall_corner.release	—	setup_test (tests/CMakeLists.txt:91)	
test_weno.release	—	setup_test (tests/CMakeLists.txt:12)	
test_x1_centroid_rest_python	hydro, python	snapy_add_python_test (tests/CMakeLists.txt:267)	
test_x1_seam_split.release	—	setup_test (tests/CMakeLists.txt:35)	
test_x1_seam_split_mp.release	—	setup_parallel_test (tests/CMakeLists.txt:80)	
test_x1_seam_split_mp_radial_exact.release	—	seam_arm_mp (tests/CMakeLists.txt:88)	

test	labels	defined at	notes
test_x1_seam_split_mp_radial_exact_off.release	—	seam_arm_mp (tests/CMakeLists.txt:89)	
test_x1_seam_split_mp_wb_ref4.release	—	seam_arm_mp (tests/CMakeLists.txt:90)	
test_x1_seam_split_mp_x1_centroid.release	—	seam_arm_mp (tests/CMakeLists.txt:87)	
test_x1_seam_split_radial_exact.release	—	seam_arm (tests/CMakeLists.txt:50)	
test_x1_seam_split_radial_exact_off.release	—	seam_arm (tests/CMakeLists.txt:52)	
test_x1_seam_split_wb_ref4.release	—	seam_arm (tests/CMakeLists.txt:55)	
test_x1_seam_split_wb_ref4_gravity_0.release	—	seam_arm (tests/CMakeLists.txt:58)	
test_x1_seam_split_x1_centroid.release	—	seam_arm (tests/CMakeLists.txt:48)	
test_x1_seam_split_x1_mass_covariance.release	—	seam_arm (tests/CMakeLists.txt:60)	

test	labels	defined at	notes
test_yaml_keys.release	—	setup_test (tests/CMakeLists.txt:29)	

B. Components outside snapy

B.1. External component interfaces

B.1.1. Summary

> **i** At a glance
>
> Kintera supplies thermodynamics and kinetics; pyharp supplies the time integrator. Disort is a required C++ link dependency and Python import, even though the inspected compiled source has no radiative-transfer solver call. Dependency availability does not imply active radiation coupling.

Table B.1.: Requirements, distinct from source pins and tested compatibility.

component	use in snapy	required version at the snapy pin
kintera	moist EOS, saturation adjustment, driver kinetics	at least 2.5.13 in CMake and Python build/runtime metadata

component	use in snapy	required version at the snapy pin
pyharp / CMake Harp	integrator options, stage weights and stage combination	required by CMake; no version minimum or exact version specified
pydisort / CMake Disort	C++ shared-library link and Python package import	required by CMake; no version minimum or exact version specified

`find_package` declares these CMake requirements (`CMakeLists.txt:94-101`). Python `requires` contains `kintera>=2.5.13` (`pyproject.toml:1-13`), as does the runtime `dependencies` list (`pyproject.toml:25-31`). Neither list declares pyharp or pydisort. Their absence is a packaging gap, not evidence that they are optional: `python/__init__.py` imports all three before the extension (`__init__.py:1-8`). The kintera source pin above fixes this report's API evidence; it is not an exact dependency version constraint. No compatible pyharp or pydisort version range is inferred from the unversioned requirement.

B.1.2. Derivation

This appendix inventories interfaces and dependencies; it introduces no numerical scheme or equation. Thermodynamic, integration and moist-coupling derivations belong to the corresponding chapters. A linked library, an imported Python package and a numerical call are separate facts and are recorded separately.

B.1.3. Numerical method

Kintera's `ThermoYImpl` is constructed by the moist-mixture EOS. `compute` converts conserved internal energy to temperature (`VU->T`) and temperature to pressure (`VT->P`); `specific_heat_cv` obtains mixture heat capacity. On the final integrator stage, the mesh block calls `pthermo->forward` with density, internal energy density and species mass fractions, then writes the adjusted fractions back. The pinned kintera `ThermoYImpl::forward` accepts this contract (`kintera thermo_y.cpp:275-284`); it is separate from the example driver's kinetic update. The driver refreshes primitives after dynamics, evaluates rates and their Jacobian, and calls `evolve_implicit`.

Pyharp is used through C++ `harp::Integrator`: snapy parses its options, creates the integrator and invokes `pintg->forward` to combine both hydro and scalar stage increments. This is active solver use of pyharp, independent of any external radiation functionality that package may provide.

Disort is explicitly in the `PUBLIC` libraries of the `snap` shared target, through `DISORT_LIBRARY` (`CMakeLists.txt:55-75`). `FindDisort` locates `disort_release` (`FindDisort.cmake:76-97`) and obtains version information by importing pydisort (`FindDisort.cmake:16-32`). Harp similarly reads `pyharp.__version__` (`FindHarp.cmake:16-31`) and requires its headers/library (`FindHarp.cmake:74-103`). Reading a version does not impose a minimum absent a version argument to `find_package`.

B.1.4. Code

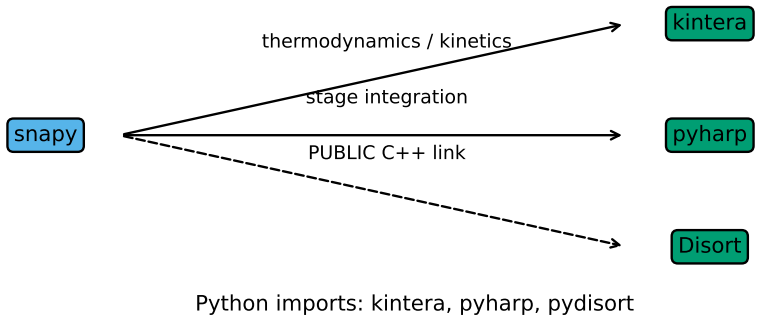


Figure B.1.: Component boundaries: solid labelled arrows show active C++ calls; the dashed Disort edge is a declared C++ link. Text also records Python imports. No performance data are shown.

Table B.2.: Representative active call sites and dependency edges.

what	code link	symbol	switch
construct moist EOS	`moist_mixture.cpp:18-29`	`ThermoYImpl`	component API
temperature and pressure	`moist_mixture.cpp:148-151`	`compute`	component API
mixture heat capacity	`moist_mixture.cpp:42-48`	`specific_heat_cv`	component API
final-stage saturation	`meshblock.cpp:795-816`	`pthermo`	component API

what	code link	symbol	switch
redo failure query	`meshblock.cpp: 1222-1235`	`take_ saturation_ adjustment_ failures`	component API
driver kinetics	`run_ hydro.cpp: 175-191`	`evolve_ implicit`	component API
integrator options	`meshblock_ options.cpp: 40-46`	`IntegratorOptionsImpl`	component API
integrator construction	`meshblock.cpp: 229-235`	`IntegratorImpl`	component API
hydro/scalar stage combination	`meshblock.cpp: 750-769`	`pintg`	component API
three C++ link dependencies	`CMakeLists.txt: 55-75`	`DISORT_ LIBRARY`	component API
Python imports	`__init__.py: 1-8`	`pydisort`	component API

Read the call chain as options, construction, evaluation, then failure handling. The kintera `take_saturation_adjustment_failures` method consumes its accumulated count (`kintera thermo_y.cpp:385-390`); snapy calls it during redo checks. The CMake version-floor comment explicitly names this API as the reason for requiring 2.5.13 (`CMakeLists.txt:94-101`).

B.1.5. Tests

Table B.3.: Documentation validation only.

check	assertion	tolerance	evidence
local anchor audit	pinned file/range exists and contains the recorded symbol	exact text	`reviews/check_appe_anchors.py`
source math-span check	appendix has valid math delimiters	exact syntax	frozen house checker
dependency runtime compatibility	not run	not measured	no build or installed-package execution

No runtime result is claimed for any component version, CPU, CUDA or multi-rank configuration. The appendix's C++ link statement follows the build definition, not a newly inspected linked binary; a platform linker may omit unused dependencies from the resulting dynamic dependency table.

B.1.6. Limits

The frozen outline describes the Disort locator as only supporting Python. The actual `PUBLIC` link list above corrects that description at the source pin. The compiled `src_files` glob (`CMakeLists.txt:35-53`) excludes the legacy diagnostics directory and files with trailing underscores. Inspection of its source directories finds no active Disort radiative-transfer call. Radiation-labelled output metadata is not a solver call. The external radiative time-step limiter mentioned by the outline is not implemented in the pinned snapy tree; this appendix does not assign external runner keys or a radiation scheme to snapy.

The missing pyharp/pydisort Python dependency declarations and unspecified CMake version floors leave reproducible installation and supported compatibility ranges unresolved. The requested kintera pin has been read, not built with this snapy pin. Official citation checking and book rendering await publication tooling and editor integration.